Học ML/AI cơ bản

Lộ trình 40 bài

Ghi chú học tập bằng tiếng Việt

Tháng 11, 2026

Mục lục

ML/AI cơ bản: lộ trình 40 bài 4
Học máy là gì: học một quy tắc từ dữ liệu rồi kiểm tra trên dữ liệu mới 11
Hồi quy tuyến tính và logistic: chọn đường dự đoán hoặc ranh giới xác suất 19
Overfitting, underfitting và bias-variance: học quy luật thay vì học thuộc dữ liệu 28
Regularization: hạn chế học thuộc để cải thiện dự đoán trên dữ liệu mới. 37
Mạng nơ-ron nhiều lớp: phi tuyến giúp ghép các phép biến đổi đơn giản thành hàm linh hoạt. 47
Backprop từ đầu: truyền độ nhạy ngược qua mạng để tính gradient của mọi tham số 56
CNN và RNN: học từ vùng lân cận và từ chuỗi 65
Mô hình sinh: học cách tạo dữ liệu bằng mã, đối kháng và khử nhiễu 75
00 Preface: Toán nền tảng và hướng dẫn đọc về ML/AI cơ bản 85
01 Understanding deep learning requires rethinking generalization (2016) 104
02 Reconciling modern machine learning practice and the bias-variance trade-off (2018) 111
03 Deep Double Descent: Where Bigger Models and More Data Hurt (2019). 117
04 XGBoost: A Scalable Tree Boosting System (2016) 124
05 Why do tree-based models still outperform deep learning on tabular data? (2022) 133
06 Improving neural networks by preventing co-adaptation of feature detectors (2012) 141
07 Deep Residual Learning for Image Recognition (2015). 148
08 Very Deep Convolutional Networks for Large-Scale Image Recognition (2014) 155
09 Efficient Estimation of Word Representations in Vector Space (2013) 162
10 Distributed Representations of Words and Phrases and their Compositionality (2013) 169
11 Neural Machine Translation by Jointly Learning to Align and Translate (2014) 176
12 Sequence to Sequence Learning with Neural Networks (2014) 182
13 Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling (2014) 188
14 Generative Adversarial Networks (2014) 194
15 Auto-Encoding Variational Bayes (2013) 200
16 Denoising Diffusion Probabilistic Models (2020) 206
17 A Simple Framework for Contrastive Learning of Visual Representations (2020) 212
18 Momentum Contrast for Unsupervised Visual Representation Learning (2019). 218
19 On Calibration of Modern Neural Networks (2017) 223
20 Shortcut Learning in Deep Neural Networks (2020) 229
21 Leakage and the Reproducibility Crisis in ML-based Science (2022) 235
22 Underspecification Presents Challenges for Credibility in Modern Machine Learning (2020) 241
23 Explaining and Harnessing Adversarial Examples (2014). 248
24 Intriguing Properties of Neural Networks (2013). 254
25 Deep Learning in Neural Networks: An Overview (2014) 260
26 Representation Learning: A Review and New Perspectives (2012) 269
27 How Transferable Are Features in Deep Neural Networks? (2014) 277
28 Rethinking ImageNet Pre-training (2018) 283
29 Self-training with Noisy Student improves ImageNet classification (2019) 289
30 When Does Label Smoothing Help? (2019) 298
31 mixup: Beyond Empirical Risk Minimization (2017). 307
32 Focal Loss for Dense Object Detection (2017) 317
33 A systematic study of the class imbalance problem in convolutional neural networks (2017) 325

34 A survey of cross-validation procedures for model selection (2009) 333
35 A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification (2021) 342
36 Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles (2016) 350
37 Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning (2015) . . . 357
38 A ConvNet for the 2020s (2022) 365
39 Towards Deep Learning Models Resistant to Adversarial Attacks (2017) 372
40 UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction (2018) 379

Lộ trình

ML/AI cơ bản: lộ trình 40 bài

Thư mục này chứa ghi chú học tập bằng tiếng Việt về **ML/AI cơ bản**: generalization và bias-variance, boosting và dữ liệu bảng, CNN và regularization, embedding và mô hình chuỗi, mô hình sinh (GAN, VAE, diffusion), self-supervised, độ tin cậy (calibration, shortcut, leakage, adversarial), transfer, kỹ thuật huấn luyện, đánh giá và độ không chắc chắn. Phần mở đầu giới thiệu toán nền và các mô hình cổ điển.

Các ghi chú dựa trên bài báo gốc; phạm vi và giới hạn được nêu trong từng chương.

> Đây là các ghi chú diễn giải, không phải bản dịch. Những liên hệ ứng dụng là suy luận; các phép tính đánh dấu "tự tính" là phép tính bổ sung.

Giai đoạn 0: Chuẩn bị

#	File	Nội dung
00	`00_preface_toan_va_huong_dan.md`	Toán nền, mô hình cổ điển, cách đánh giá và hướng dẫn đọc.

Giai đoạn 1: Tổng quát hoá

#	File	Bài	Ý chính
01	`01_2016_rethinking_generalization.md`	Understanding deep learning requires rethinking generalization (2016)	thử nghiệm "randomization test" (huấn luyện với nhãn/pixel ngẫu nhiên) để chỉ ra VC-dimension, Rademacher complexity, uniform stability và regularization tường minh đều…
02	`02_2018_bias_variance_reconciliation.md`	Reconciling modern machine learning practice and the bias-variance trade-off (2018)	đề xuất đường cong "double descent" (giáng kép) thống nhất U-curve bias-variance cổ điển với hiện tượng các mô hình quá tham số hoá (interpolating) vẫn generalize tốt;
03	`03_2019_deep_double_descent.md`	Deep Double Descent: Where Bigger Models and More Data Hurt (2019)	hiện tượng "double descent": khi tăng kích thước mô hình (hoặc số epoch, hoặc số mẫu huấn luyện), lỗi kiểm tra (test error) không đi theo đường cong chữ U cổ điển mà…

Giai đoạn 2: Dữ liệu bảng và boosting

#	File	Bài	Ý chính
04	`04_2016_xgboost.md`	XGBoost: A Scalable Tree Boosting System (2016)	hệ thống gradient tree boosting có thể mở rộng (XGBoost): mục tiêu học có regularization bậc hai, thuật toán tìm điểm chia (split) chính xác và xấp xỉ, weighted quantile…
05	`05_2022_tabular_tree_vs_deep.md`	Why do tree-based models still outperform deep learning on tabular data? (2022)	benchmark thực nghiệm có kiểm soát trên 45 bộ dữ liệu tabular, so sánh tree-based models (Random Forest, GradientBoosting/HistGradientBoosting, XGBoost) với deep…

Giai đoạn 3: CNN và regularization

#	File	Bài	Ý chính
06	`06_2012_dropout_co_adaptation.md`	Improving neural networks by preventing co-adaptation of feature detectors (2012)	"dropout": trong lúc huấn luyện, ngẫu nhiên bỏ (omit) một nửa các đơn vị ẩn ở mỗi lượt trình diễn mẫu huấn luyện, nhằm ngăn các feature detector "đồng thích nghi"…
07	`07_2015_resnet.md`	Deep Residual Learning for Image Recognition (2015)	kiến trúc ResNet, residual connection (skip connection) giải quyết "degradation problem" khi mạng CNN rất sâu, cho phép huấn luyện mạng tới hơn 1000 tầng
08	`08_2014_vgg.md`	Very Deep Convolutional Networks for Large-Scale Image Recognition (2014)	VGGNet, nghiên cứu hệ thống ảnh hưởng của độ sâu ConvNet khi chỉ dùng filter tích chập rất nhỏ (3x3), tạo ra các cấu hình 11-19 tầng trọng số (A đến E, trong đó D/E là…

Giai đoạn 4: Embedding và mô hình chuỗi

#	File	Bài	Ý chính
09	`09_2013_word2vec_cbow_skipgram.md`	Efficient Estimation of Word Representations in Vector Space (2013)	hai kiến trúc log-linear (CBOW và Skip-gram) để học vector từ liên tục ("word vector") từ tập dữ liệu rất lớn, rẻ hơn nhiều so với mô hình ngôn ngữ nơ-ron có tầng ẩn phi…
10	`10_2013_word2vec_negative_sampling.md`	Distributed Representations of Words and Phrases and their Compositionality (2013)	mở rộng Skip-gram (bài 09): negative sampling thay cho hierarchical softmax, subsampling từ tần suất cao để tăng tốc và cải thiện vector từ hiếm, và một phương pháp nhận…

#	File	Bài	Ý chính
11	`11_2014_bahdanau_attention_nmt.md`	Neural Machine Translation by Jointly Learning to Align and Translate (2014)	mô hình dịch máy nơ-ron (NMT) "RNNsearch": encoder hai chiều (bidirectional RNN) tạo một dãy annotation cho câu nguồn, decoder mỗi bước tính một context vector riêng…
12	`12_2014_seq2seq_sutskever.md`	Sequence to Sequence Learning with Neural Networks (2014)	dùng hai LSTM nhiều tầng (một encoder, một decoder) để ánh xạ một dãy có độ dài tùy ý sang một dãy khác có độ dài tùy ý, không giả định gì về cấu trúc bài toán;
13	`13_2014_gru_vs_lstm_chung.md`	Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling (2014)	so sánh thực nghiệm ba loại đơn vị hồi quy (tanh thường, LSTM, GRU) trên mô hình hóa dãy (sequence modeling): nhạc đa âm (polyphonic music) và tín hiệu tiếng nói thô;

Giai đoạn 5: Mô hình sinh

#	File	Bài	Ý chính
14	`14_2014_gan.md`	Generative Adversarial Networks (2014)	khung huấn luyện mô hình sinh (generative model) bằng một trò chơi minimax hai người chơi giữa một mạng sinh G (cố gắng tạo mẫu giống dữ liệu thật) và một mạng phân…
15	`15_2013_vae_aevb.md`	Auto-Encoding Variational Bayes (2013)	suy luận biến phân (variational inference) và học biểu diễn ẩn liên tục bằng một mạng encoder-decoder huấn luyện qua một cận dưới (lower bound) khả vi của…
16	`16_2020_ddpm.md`	Denoising Diffusion Probabilistic Models (2020)	mô hình sinh dựa trên một quá trình khử nhiễu (denoising) Markov nhiều bước: học đảo ngược (reverse) một quá trình thuận (forward) thêm nhiễu Gauss dần vào dữ liệu, tham…

Giai đoạn 6: Self-supervised

#	File	Bài	Ý chính
17	`17_2020_simclr.md`	A Simple Framework for Contrastive Learning of Visual Representations (2020)	SimCLR, một khung contrastive learning tự giám sát (self-supervised) đơn giản để học biểu diễn ảnh, không cần kiến trúc đặc biệt hay memory bank

#	File	Bài	Ý chính
18	`18_2019_moco.md`	Momentum Contrast for Unsupervised Visual Representation Learning (2019)	MoCo, cơ chế contrastive learning tự giám sát dùng “dictionary” các key dạng hàng đợi (queue) cùng encoder key cập nhật theo trung bình động (momentum), tách kích thước…

Giai đoạn 7: Độ tin cậy, shortcut, leakage và adversarial

#	File	Bài	Ý chính
19	`19_2017_calibration.md`	On Calibration of Modern Neural Networks (2017)	Hiện tượng miscalibration (lệch giữa độ tin cậy dự đoán và xác suất đúng thật) ở mạng neural hiện đại;
20	`20_2020_shortcut_learning.md`	Shortcut Learning in Deep Neural Networks (2020)	Shortcut learning: khung khái niệm thống nhất nhiều “triệu chứng” thất bại của deep learning (dataset bias, texture bias, adversarial examples, reward hacking, bias…
21	`21_2022_leakage_reproducibility.md`	Leakage and the Reproducibility Crisis in ML-based Science (2022)	Data leakage trong “ML-based science” (dùng hiệu năng mô hình ML làm bằng chứng cho một claim khoa học);
22	`22_2020_underspecification.md`	Underspecification Presents Challenges for Credibility in Modern Machine Learning (2020)	Underspecification: hiện tượng một ML pipeline có thể trả về nhiều predictor khác nhau cùng đạt hiệu năng iid gần-tối-ưu tương đương, nhưng các predictor này lại hành xử…
23	`23_2014_fgsm.md`	Explaining and Harnessing Adversarial Examples (2014)	Giải thích nguyên nhân tồn tại của adversarial example bằng tính **tuyến tính** (không phải phi tuyến/overfitting như giả thuyết trước đó);
24	`24_2013_intriguing_properties.md`	Intriguing Properties of Neural Networks (2013)	Hai tính chất “kỳ lạ” của deep neural network: (1) thông tin ngữ nghĩa nằm trong toàn bộ không gian activation, không riêng từng unit;

Giai đoạn 8: Biểu diễn và transfer

#	File	Bài	Ý chính
25	`25_2014_deep_learning_overview_schmidhuber.md`	Deep Learning in Neural Networks: An Overview (2014)	tổng quan lịch sử Deep Learning (DL) trong neural network (NN) cho Supervised Learning (SL), Unsupervised Learning (UL) và Reinforcement Learning (RL);

#	File	Bài	Ý chính
26	`26_2012_representation_learning_review.md`	Representation Learning: A Review and New Perspectives (2012)	tổng quan (review) ba hướng tiếp cận representation learning tới năm 2012: mô hình xác suất có biến ẩn (probabilistic models: sparse coding, RBM/Boltzmann machine),…
27	`27_2014_how_transferable_features.md`	How Transferable Are Features in Deep Neural Networks? (2014)	đo định lượng, theo từng tầng, mức độ "general" (dùng được cho task khác) hay "specific" (chỉ hợp với task gốc) của feature trong một mạng CNN tám tầng huấn luyện trên…
28	`28_2018_rethinking_imagenet_pretraining.md`	Rethinking ImageNet Pre-training (2018)	kiểm định lại bằng thực nghiệm có kiểm soát câu hỏi "có cần ImageNet pre-training để đạt độ chính xác cao trên object detection/instance segmentation/keypoint detection…
29	`29_2019_noisy_student.md`	Self-training with Noisy Student improves ImageNet classification (2019)	self-training/semi-supervised learning quy mô lớn cho ImageNet classification: dùng một teacher (EfficientNet) sinh pseudo-label trên 300 triệu ảnh JFT không nhãn, sau…

Giai đoạn 9: Kỹ thuật huấn luyện

#	File	Bài	Ý chính
30	`30_2019_label_smoothing.md`	When Does Label Smoothing Help? (2019)	phân tích thực nghiệm khi nào và vì sao label smoothing (làm "mềm" nhãn one-hot bằng cách trộn với phân phối đều) giúp cải thiện generalization và calibration, nhưng lại…
31	`31_2017_mixup.md`	mixup: Beyond Empirical Risk Minimization (2017)	đề xuất mixup, một nguyên lý data augmentation đơn giản, không phụ thuộc domain, thay thế Empirical Risk Minimization (ERM) bằng Vicinal Risk Minimization (VRM): train…
32	`32_2017_focal_loss.md`	Focal Loss for Dense Object Detection (2017)	phát hiện và xử lý nguyên nhân chính khiến object detector một giai đoạn (one-stage) kém chính xác hơn detector hai giai đoạn (two-stage): mất cân bằng cực độ giữa lớp…

#	File	Bài	Ý chính
33	`33_2017_class_imbalance_cnn.md`	A systematic study of the class imbalance problem in convolutional neural networks (2017)	khảo sát thực nghiệm có hệ thống (trên MNIST, CIFAR-10, ImageNet) về ảnh hưởng của mất cân bằng lớp (class imbalance) lên hiệu năng phân loại của CNN, và so sánh các…

Giai đoạn 10: Đánh giá và độ không chắc chắn

#	File	Bài	Ý chính
34	`34_2009_cross_validation_survey.md`	A survey of cross-validation procedures for model selection (2009)	khảo sát lý thuyết và thực nghiệm (survey, không có thí nghiệm deep learning mới) về các thủ tục cross-validation (CV) dùng để chọn mô hình (model selection), phân biệt…
35	`35_2021_conformal_prediction_intro.md`	A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification (2021)	giới thiệu conformal prediction, một phương pháp tạo prediction set/interval có bảo đảm xác suất bao phủ (coverage) đúng theo nghĩa phân phối-tự do (distribution-free),…
36	`36_2016_deep_ensembles.md`	Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles (2016)	một phương pháp non-Bayesian, đơn giản và dễ scale để ước lượng predictive uncertainty cho neural network, dùng ba thành phần: proper scoring rule làm loss, adversarial…
37	`37_2015_mc_dropout_bayesian.md`	Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning (2015)	chứng minh huấn luyện dropout tương đương một dạng xấp xỉ biến phân (variational inference) cho deep Gaussian process, từ đó đề xuất Monte Carlo dropout (MC dropout: giữ…

Giai đoạn 11: CNN hiện đại, robust và trực quan hoá

#	File	Bài	Ý chính
38	`38_2022_convnext.md`	A ConvNet for the 2020s (2022)	"hiện đại hoá" từng bước một ResNet thông thường theo hướng thiết kế của Swin Transformer (một vision Transformer dạng hierarchical), không thêm bất kỳ cơ chế…
39	`39_2017_pgd_adversarial_training.md`	Towards Deep Learning Models Resistant to Adversarial Attacks (2017)	nhìn adversarial robustness qua lăng kính robust optimization (bài toán min-max/saddle point), từ đó đề xuất Projected Gradient Descent (PGD) như một "universal…

#	File	Bài	Ý chính
40	`40_2018_umap.md`	UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction (2018)	một thuật toán giảm chiều dữ liệu (dimension reduction) tổng quát, xây dựng trên nền lý thuyết hình học Riemann và tô pô đại số (fuzzy simplicial sets), cạnh tranh với…

Cách dùng

Đọc preface (00) trước. Nếu chỉ có thời gian cho ít bài: 01, 02, 07, 11, 19, 20, 21.

Mỗi bài có 11 mục cố định: bối cảnh, ý tưởng chính, phương pháp chi tiết, thí nghiệm, phân tích, giới hạn, liên hệ, gợi ý áp dụng (suy luận), thí nghiệm gợi ý, tóm tắt công thức, kết luận. Mục 6 (giới hạn) là nơi ghi các mâu thuẫn nội tại phát hiện trong paper.

Liên quan

- `math content/` (35 bài): toán nền; `ml basics content/` (40 bài): ML cơ bản; `optimization content/` (51 bài): tối ưu, optimizer, hyperparameter.
- `llm content/`, `rl content/`, `distillation content/`, `pruning content/`, `interp content/`: các lộ trình chuyên đề xây trên ba thư mục nền này.

Nền tảng 01

Học máy là gì: học một quy tắc từ dữ liệu rồi kiểm tra trên dữ liệu mới

Loại: Chương nền tảng, không phải ghi chú paper
Đọc trước: ml basics 00 (lời nói đầu); không cần biết thuật toán học máy trước
Dùng ở: các bài paper ml basics 01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39

> Chương này giải thích những ý cơ bản chung của học máy, không dịch một paper cụ thể. Các phép tính ghi "tự tính" được kiểm lại bằng NumPy.

1. Vì sao cần hiểu học máy trước khi học thuật toán

Học máy là cách dùng ví dụ đã quan sát để chọn một quy tắc dự đoán, rồi kiểm tra xem quy tắc ấy có làm việc trên ví dụ mới hay không. Máy không tự biết thế nào là "tốt": người làm phải chọn dữ liệu, cách biểu diễn, loại mô hình, thước đo lỗi và cách đánh giá.

Hãy tưởng tượng ta muốn dự đoán thời gian giao hàng từ khoảng cách và tình hình giao thông. Một chương trình viết tay có thể chứa quy tắc "mỗi ki-lô-mét cộng thêm vài phút", nhưng thực tế có thêm giờ cao điểm, thời tiết, tuyến đường, cách lấy hàng. Học máy cho phép ước lượng một quy tắc từ các chuyến giao đã có thông tin đầu vào và thời gian thực tế. Nó có thể nắm mẫu hình phức tạp hơn một vài câu lệnh, nhưng chỉ trong giới hạn dữ liệu và loại quy tắc mà ta cho phép.

Một hệ thống học máy cơ bản có năm phần nối nhau: dữ liệu cung cấp ví dụ; mô hình biến đầu vào thành dự đoán; hàm mất mát đo dự đoán sai đến đâu; thuật toán tối ưu điều chỉnh mô hình để giảm lỗi; cách đánh giá đo xem mô hình có hữu ích trên dữ liệu chưa dùng để học không. Hình đầu tiên là bản đồ của năm phần đó.

2. Nhắc nền cần thiết: ví dụ, đặc trưng và nhãn

Một ví dụ thường gồm **đầu vào** x và, nếu có, **nhãn** y. Trong bài toán giao hàng, x có thể chứa khoảng cách, giờ xuất phát và trạng thái giao thông; y là số phút thực tế. Ta thu được nhiều ví dụ như vậy và gọi toàn bộ chúng là một **bộ dữ liệu**.

Một đặc trưng (feature) là một phần thông tin đầu vào mà mô hình được phép sử dụng. Nếu ta chỉ đưa khoảng cách vào x, mô hình không thể tự suy ra hôm đó có mưa hay kẹt xe, trừ khi dấu hiệu ấy đã được mã hóa trong các đặc trưng khác. Mô hình học từ dữ liệu được cung cấp, không đọc được bối cảnh bên ngoài.

Với N ví dụ, ta viết dữ liệu có nhãn thành

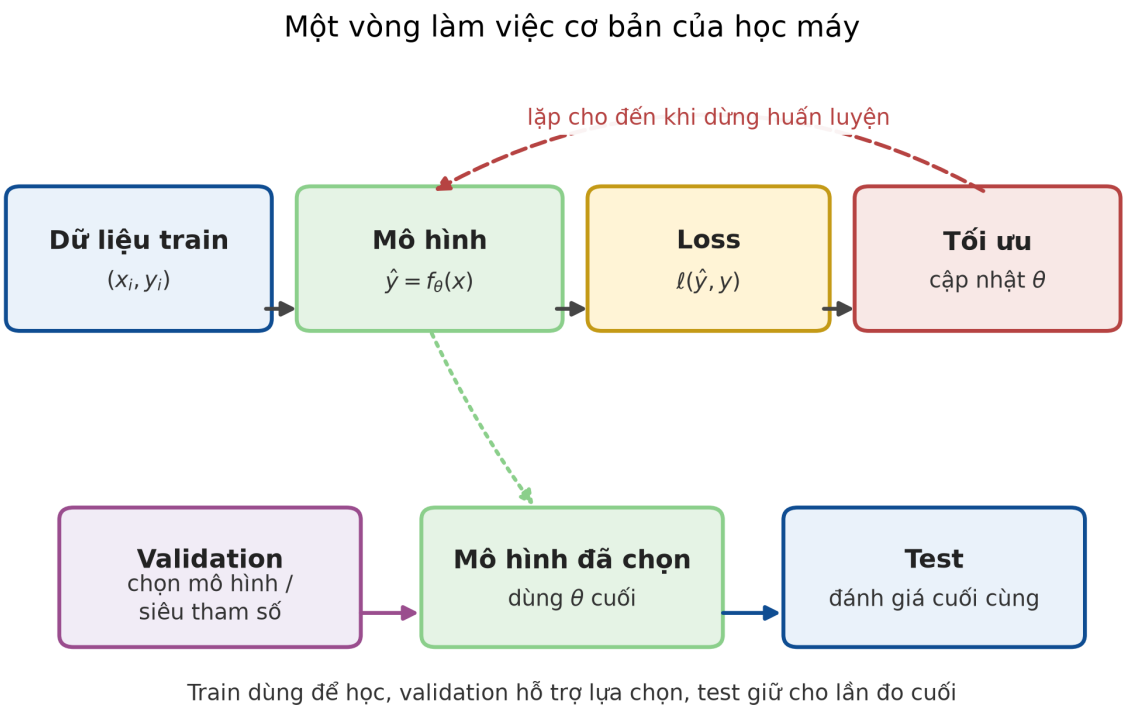
$$\mathcal{D} = \{(x_i, y_i) : i = 1, \dots, N\}.$$

Chỉ số i nói ta đang xét ví dụ thứ i. x_i là đầu vào của ví dụ đó, y_i là nhãn tương ứng. Nếu mỗi ví dụ có d đặc trưng dạng số, có thể xếp dữ liệu đầu vào thành ma trận X có N hàng và d cột: hàng là ví dụ, cột là đặc trưng. Dãy nhãn được xếp thành vector y. Kích thước của tập dữ liệu và kích thước của mỗi ví dụ là hai thứ khác nhau.

Nhãn không nhất thiết là một con số liên tục. Một nhãn **liên tục** như thời gian giao hàng, giá hoặc nhiệt độ dẫn đến bài toán hồi quy. Nhãn **phân loại** như "giao đúng hạn / trễ" hoặc "có lỗi / không lỗi" dẫn đến bài

toán phân loại. Có bài toán không có nhãn rõ ràng; khi ấy ta có thể tìm cụm hoặc cấu trúc trong các đầu vào. Chương 02 sẽ tập trung vào hồi quy tuyến tính và phân loại nhị phân bằng logistic.

3. Trực giác bằng hình: từ ví dụ đến quyết định



Hình 1: Sơ đồ nối dữ liệu, mô hình, loss, cập nhật tham số và đánh giá cuối (minh hoạ, tự tính)

Cách đọc hình: hàng trên là vòng huấn luyện. Dữ liệu train đi qua mô hình, dự đoán được so với nhãn để tính loss, rồi thuật toán tối ưu cập nhật tham số và lặp lại. Hàng dưới là việc chọn và đo mô hình: validation giúp so sánh các lựa chọn; test được giữ lại cho lần đánh giá cuối cùng.

Mũi tên quay lại từ khối tối ưu cho thấy máy không "nghĩ ra" một mô hình hoàn hảo ngay lần đầu. Nó bắt đầu từ một cấu hình tham số, xem lỗi, rồi thay đổi tham số. Mũi tên chấm nối sang mô hình đã chọn nhắc rằng validation không thay thế dữ liệu train: nó giúp con người chọn giữa các cách huấn luyện.

4. Định nghĩa chính thức: mô hình, loss và mục tiêu học

Mô hình là một quy tắc có tham số, ký hiệu f_θ. Đầu vào x đi vào quy tắc, đầu ra $\hat{y} = f_\theta(x)$ là dự đoán. Dấu mũ trên $\hat{y}$ nhắc rằng đây là giá trị mô hình ước lượng, có thể khác nhãn thật y. Tham số θ là những con số bên trong quy tắc mà quá trình học sẽ điều chỉnh, chẳng hạn hệ số góc và hệ số chặn của một đường thẳng.

Trước khi học, ta chọn **họ mô hình** (model class): tập các quy tắc mà mô hình có thể biểu diễn. Nếu chỉ cho phép một đường thẳng, mô hình không thể khớp một đường cong tùy ý. Mạng neural, cây quyết định và hồi quy tuyến tính là những họ mô hình khác nhau. Dữ liệu không thể khiến một họ mô hình biểu diễn được mọi thứ.

Hàm mất mát (loss) gán một số cho mức sai của một dự đoán. Với một ví dụ, ta viết $\ell(f_\theta(x_i), y_i)$. Chọn loss là một quyết định về loại sai lầm nào cần phạt và phạt ra sao. Ví dụ, sai số bình phương phạt lỗi lớn mạnh hơn lỗi nhỏ; cross-entropy phạt mạnh dự đoán xác suất rất cao cho lớp sai.

Vì ta chỉ có một tập hữu hạn ví dụ, ta thường tối thiểu hóa loss trung bình trên tập train:

$$\hat{R}_{\text{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell(f_\theta(x_i), y_i).$$

Đây là **rủi ro thực nghiệm** (empirical risk), tức lỗi đo trên các ví dụ ta đang có. **Tối ưu** (optimization) là tìm tham số làm đại lượng này nhỏ đi. Có thuật toán cho nghiệm chính xác trong một số bài toán đơn giản; với mô hình lớn, thuật toán thường cập nhật gần đúng từng bước. Thư mục `optimization content/` giải thích các phương pháp đó.

Mục tiêu cuối thường rộng hơn train loss. Ta muốn mô hình dự đoán tốt trên dữ liệu mới từ môi trường mà nó sẽ được dùng. Kỳ vọng loss trên phân phối dữ liệu thật được gọi là **rủi ro quần thể** (population risk):

$$R(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{P}}\left[\ell(f_\theta(x), y)\right].$$

Phân phối $\mathcal{P}$ mô tả cách ví dụ mới xuất hiện. Vì không biết toàn bộ phân phối đó, ta không thể tính trực tiếp kỳ vọng; tập train chỉ là một mẫu hữu hạn. Cho nên loss train nhỏ là bằng chứng mô hình khớp các ví dụ train, chưa đủ để kết luận nó dự đoán tốt ngoài tập đó. Khả năng làm việc trên dữ liệu chưa thấy gọi là **tổng quát hóa** (generalization).

5. Ví dụ tay: loss đo lỗi ra sao

Ta dựng một ví dụ nhỏ để tách riêng các ý. Giả sử ba đầu vào là $x_1 = 1$, $x_2 = 2$, $x_3 = 3$, còn mục tiêu tương ứng là $y_1 = 2$, $y_2 = 4$, $y_3 = 6$ (tự tính). Đây là dữ liệu minh họa, không phải phép đo ngoài đời. Ta dùng mô hình đơn giản $\hat{y} = wx$, không có hệ số chặn. w là tham số cần học.

5.1 Dự đoán từ tham số

Nếu $w = 1$, dự đoán lần lượt là $1, 2, 3$ (tự tính). Sai số dự đoán trừ nhãn là $-1, -2, -3$ (tự tính). Ta có thể lấy bình phương từng sai số để không cho lỗi dương triệt tiêu lỗi âm: $1, 4, 9$ (tự tính). Loss bình phương trung bình là

$$L(1) = \frac{1+4+9}{3} = \frac{14}{3} \approx 4.667 \quad \text{(tự tính)}.$$

Cách đọc hình: chấm xanh là nhãn đã quan sát, dấu nhân màu ngọc là dự đoán khi hệ số w bằng 1. Đoạn nét đứt dọc nối hai điểm là phần dư của một ví dụ. Loss bình phương trung bình cộng bình phương độ dài các đoạn rồi chia cho số ví dụ.

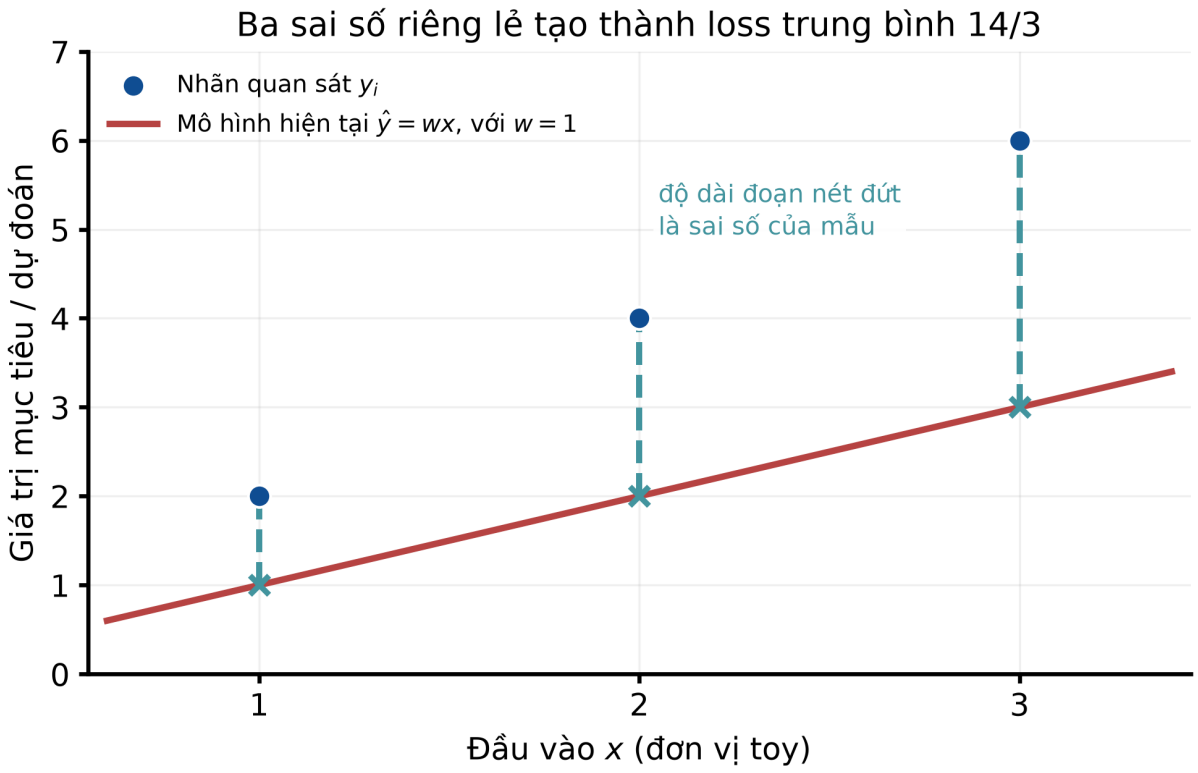
Nếu $w = 0$, cả ba dự đoán bằng 0; loss bình phương trung bình là $(4 + 16 + 36)/3 = 56/3 \approx 18.667$ (tự tính). Nếu $w = 2$, dự đoán trùng cả ba nhãn; loss bằng 0 (tự tính). Trong ví dụ sạch này, ta nhìn thấy tham số tốt nhất ngay. Với dữ liệu thật có nhiễu, thường không có một quy tắc khớp hoàn hảo tất cả ví dụ.

5.2 Tối ưu là thay tham số để hạ loss

Với mọi w, mỗi dự đoán là wx_i, nên loss trên bộ dữ liệu này có thể viết thành

$$L(w) = \frac{(w-2)^2 + (2w-4)^2 + (3w-6)^2}{3} = \frac{14}{3}(w-2)^2 \quad \text{(tự tính)}.$$

Điểm thấp nhất nằm tại $w = 2$. Trong bài này, ta có thể lấy đạo hàm để biết đường cong dốc hướng nào: $L'(w) = \frac{28}{3}(w-2)$ (tự tính). Khi $w < 2$, đạo hàm âm, nghĩa là tăng w một lượng đủ nhỏ sẽ làm loss giảm. Khi $w > 2$, đạo hàm dương, nghĩa là giảm w một lượng đủ nhỏ sẽ làm loss giảm.



Hình 2: Các nhãn, dự đoán của mô hình ban đầu và phần dư theo từng ví dụ (minh hoạ, tự tính)

Một quy tắc cập nhật đơn giản là gradient descent: lấy tham số mới bằng tham số cũ trừ đạo hàm nhân với bước học η. Chọn $\eta = 3/56$ (tự tính) và bắt đầu $w_0 = 0$. Khi đó

$$w_{t+1} = w_t - \eta L'(w_t) = \frac{w_t + 2}{2}.$$

Các tham số đi qua 0, 1, 1.5, 1.75, 1.875 trong năm lần đánh chỉ số từ 0 đến 4 (tự tính). Loss tương ứng giảm từ $56/3$ qua $14/3$, $7/6$, $7/24$ đến $7/96$ (tự tính). Mỗi bước tiến gần $w = 2$, nhưng chưa chạm hẳn sau số bước hữu hạn đó.

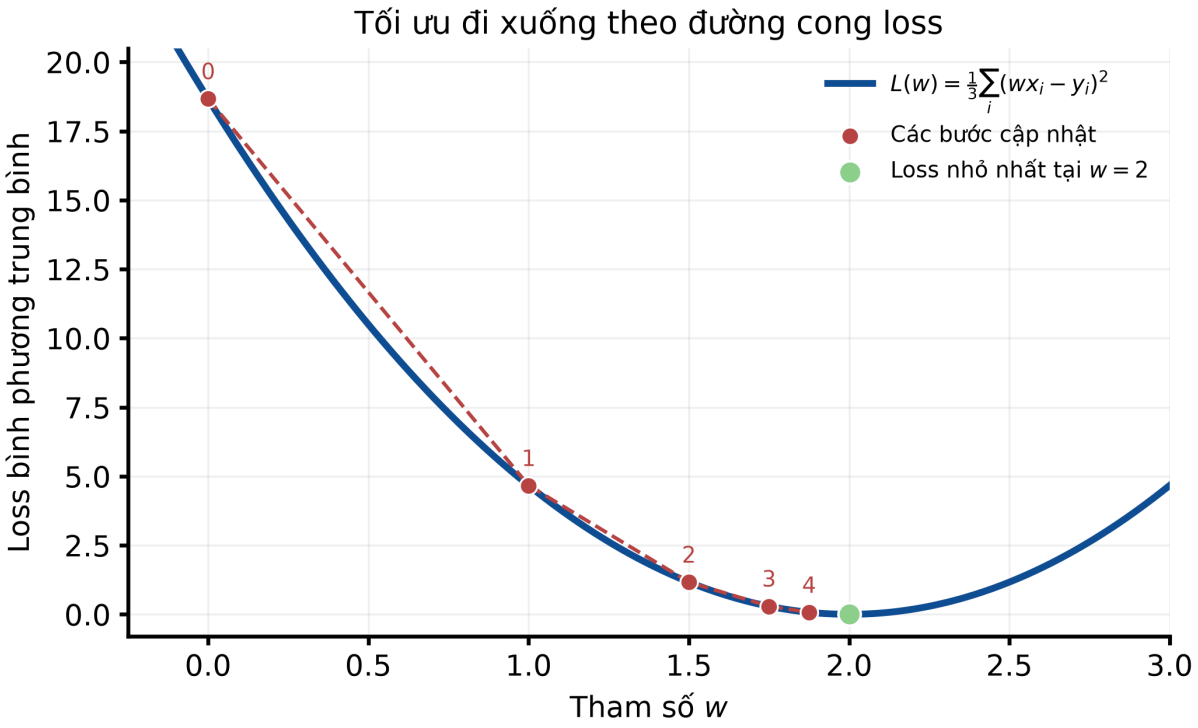
Cách đọc hình: đường cong là loss ứng với từng giá trị của w, chấm đỏ đánh số thứ tự các lần cập nhật. Các chấm tiến xuống và sang phải, gần điểm xanh tại $w = 2$. Đây là một ví dụ một chiều đơn giản; mô hình nhiều tham số có bề mặt loss nhiều chiều và đường đi phức tạp hơn.

Ví dụ này cho thấy bốn khái niệm khác nhau: mô hình tạo dự đoán; loss biến sai số thành một số; tham số quyết định hình dạng dự đoán; tối ưu thay tham số để giảm loss. Đổi cách định nghĩa loss có thể đổi cả mô hình được chọn, ngay cả khi dữ liệu giữ nguyên.

6. Tối ưu không phải là đánh giá

Trong huấn luyện, ta thường tối ưu một hàm trơn có thể tính gradient. Nhưng người dùng có thể quan tâm tới một thước đo khác. Chẳng hạn, bài toán phân loại cuối cùng có thể được chấm bằng tỷ lệ dự đoán đúng (accuracy), trong khi mô hình học bằng cross-entropy. Cross-entropy cung cấp tín hiệu thay đổi liên tục khi xác suất đổi; nhãn dự đoán chỉ đổi khi xác suất hoặc điểm số vượt ngưỡng. Loss là công cụ học, metric là cách mô tả chất lượng mà ta quan tâm.

Một metric phù hợp phải phản ánh hậu quả của lỗi. Nếu bỏ sót bệnh nghiêm trọng hơn báo động nhầm, chỉ



Hình 3: Loss của mô hình toy và các bước cập nhật đi dần về điểm thấp nhất (minh hoạ, tự tính)

báo accuracy tổng thể có thể che mất vấn đề. Nếu dự báo giá trị liên tục, MSE phạt lỗi lớn nặng hơn MAE; MAE diễn giải trực tiếp theo đơn vị của đại lượng dự báo. Không có một metric duy nhất tốt nhất cho mọi tác vụ.

7. Đánh giá: dữ liệu nào được dùng vào việc gì

Ta thường chia dữ liệu thành **train**, **validation** và **test**. Train dùng để điều chỉnh tham số. Validation được dùng trong quá trình phát triển để so sánh kiến trúc, siêu tham số, ngưỡng quyết định hoặc thời điểm dừng. Test được giữ lại để ước lượng lần cuối sau khi các quyết định đã xong. Hình pipeline ở mục 3 tóm tắt vai trò này.

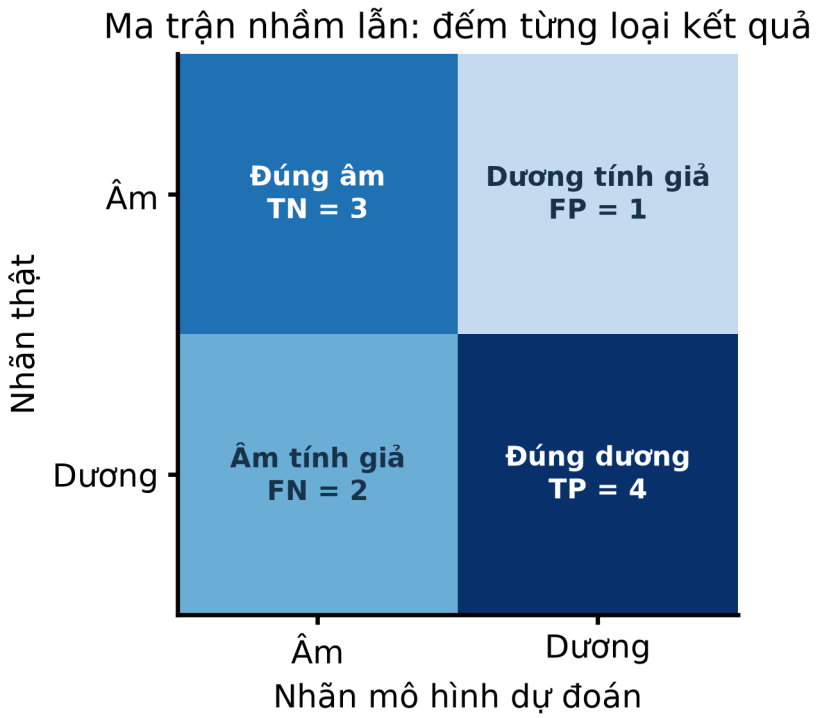
Ví dụ, nếu có 100 chuyến giao có nhãn, ta có thể dành 60 chuyến cho train, 20 cho validation và 20 cho test (tự tính, chỉ là minh họa cách đếm). Những tỷ lệ đó không phải quy tắc chung. Nếu dữ liệu ít, cross-validation có thể luân phiên dùng các phần khác nhau để huấn luyện và kiểm tra lựa chọn; paper ml basics 34 bàn sâu hơn về cross-validation.

Nếu cứ thử hàng trăm cách rồi chọn cách có điểm test tốt nhất, test đã tham gia vào lựa chọn và không còn là phép đo độc lập. Khi đó điểm test thường lạc quan hơn hiệu năng trên dữ liệu mới. Tương tự, nếu cùng một người hoặc cùng một thiết bị xuất hiện ở cả train và test, mô hình có thể nhận ra dấu vết riêng thay vì học quy luật muốn đo. Chọn cách chia cần khớp với tình huống sử dụng: ví dụ dự đoán tương lai nên tách theo thời gian; dự đoán cho người mới nên cân nhắc tách theo người.

Phân phối dữ liệu lúc sử dụng có thể khác lúc thu thập. Thay đổi cách ghi nhận, dân số, mùa vụ hoặc thiết bị có thể làm mô hình kém đi, dù điểm test cũ cao. Đánh giá luôn là một phát biểu có điều kiện: trên tập dữ liệu nào, theo cách chia nào, với metric nào. Paper ml basics 21 thảo luận data leakage và paper ml basics 22 bàn về underspecification, tức nhiều mô hình cùng khớp dữ liệu nhưng hành xử khác nhau ở nơi chưa đo.

7.1 Một phép tính metric bằng tay

Xét một bài toán phát hiện giao hàng trễ, trong đó lớp dương là "trễ". Giả sử bảng đếm có 3 dự đoán âm đúng, 1 dự đoán dương nhầm, 2 trường hợp trễ bị bỏ sót và 4 trường hợp trễ được phát hiện (tự tính). Ta gọi bốn ô đó lần lượt là TN, FP, FN và TP.



Hình 4: Bốn ô của ma trận nhầm lẫn trên ví dụ phân loại nhỏ (minh hoạ, tự tính)

Cách đọc hình: hàng là nhãn thật, cột là nhãn dự đoán. Ô trên trái và dưới phải là dự đoán đúng; hai ô còn lại là hai loại sai khác nhau. Mẫu số khi tính precision không giống mẫu số khi tính recall, vì hai câu hỏi cần trả lời khác nhau.

Accuracy là tỷ lệ dự đoán đúng trên toàn bộ trường hợp: $(3 + 4)/10 = 0.7$ (tự tính). Precision hỏi trong các dự đoán "trễ", có bao nhiêu thực sự trễ: $4/(4 + 1) = 0.8$ (tự tính). Recall hỏi trong các chuyến thực sự trễ, ta bắt được bao nhiêu: $4/(4 + 2) = 2/3 \approx 0.667$ (tự tính). Hai mẫu số đếm hai nhóm khác nhau, và trong ví dụ này chúng còn có độ lớn khác nhau. Với lớp hiếm, accuracy có thể trông cao dù hệ thống gần như không tìm ra ca dương; vì vậy cần đọc metric gắn với mục tiêu.

8. Các biến thể và chi tiết hay bị bỏ qua

Học có giám sát dùng cặp đầu vào và nhãn để học dự đoán. Hồi quy và phân loại là hai dạng thường gặp. **Học không giám sát** chỉ có đầu vào, mục tiêu có thể là gom nhóm hoặc nén thông tin. **Tự giám sát** tạo mục tiêu huấn luyện từ chính dữ liệu đầu vào, như che một phần văn bản rồi yêu cầu mô hình khôi phục. **Học tăng cường** học từ tương tác và phần thưởng. Cách gọi mô tả nguồn tín hiệu học, không phải mức độ "thông minh" của phương pháp.

Tham số và siêu tham số cũng khác nhau. Tham số như w được cập nhật trong quá trình tối ưu. Siêu tham số như tốc độ học, mức phạt regularization hay độ sâu cây thường được chọn bằng validation. Khi đọc một paper, nên kiểm tra tác giả đã chọn mỗi loại bằng dữ liệu nào.

Một mô hình có thể fit train tốt nhưng vẫn thất bại vì nhãn sai, dữ liệu lệch, đặc trưng thiếu, mục tiêu không phù hợp hoặc metric không phản ánh chi phí thật. Ngược lại, một mô hình đơn giản đôi khi là lựa chọn tốt hơn nếu dễ giải thích, ổn định và đủ chính xác cho nhu cầu. Mục tiêu không phải dùng mô hình phức tạp nhất, mà là tạo hệ thống đáng tin cậy trong bối cảnh cụ thể.

9. Nối sang AI

Phân loại ảnh, nhận dạng tiếng nói, dịch máy và mô hình ngôn ngữ khác nhau ở dạng dữ liệu và kiến trúc, nhưng vẫn có các phần chung: biểu diễn đầu vào, mô hình sinh điểm số hoặc dự đoán, mục tiêu huấn luyện, phương pháp tối ưu và giao thức đánh giá.

Trong học tự giám sát, mục tiêu có thể là dự đoán token kế tiếp, phân biệt hai góc nhìn của cùng một ảnh hoặc khôi phục vùng bị che. Dù không có nhãn thủ công thông thường, vẫn có loss và dữ liệu dùng để tối ưu. Paper ml basics 09-10 bàn về word2vec, còn paper ml basics 17-18 nói về contrastive learning.

Trong phân loại ảnh, mô hình thường trả ra điểm cho nhiều lớp. Một bước chuyển điểm thành xác suất và một ngưỡng quyết định ảnh hưởng tới dự đoán cuối, dù tham số của mô hình không đổi. Paper ml basics 19 về calibration giúp phân biệt "thường đoán đúng" với "xác suất dự đoán đáng tin".

Một hệ AI triển khai thực tế còn có tiền xử lý, lưu trữ, thời gian phản hồi, chi phí lỗi và giám sát sau khi đưa vào dùng. Điểm trên một tập benchmark chỉ đo một lát cắt của hệ thống đó. Vì vậy nên đọc đồng thời cách tạo dữ liệu, cách chọn mô hình và cách đánh giá, không chỉ nhìn con số cuối.

10. Sai lầm hay gặp

- Đồng nhất loss train thấp với khả năng dự đoán tốt trên dữ liệu mới. Hai điều này liên quan nhưng không giống nhau.
- Nghĩ rằng mô hình học được thông tin không có trong đầu vào. Nếu đặc trưng thiếu hoặc nhãn sai, tối ưu không tự sửa được nguồn dữ liệu.
- Cho rằng loss và metric là một. Loss thường là mục tiêu để cập nhật tham số, metric là tiêu chuẩn báo cáo hoặc quyết định.
- Chọn mô hình bằng test rồi xem điểm đó như đánh giá độc lập. Mỗi lần dùng test để ra quyết định đều làm test bớt độc lập.
- Chỉ báo accuracy khi lớp dương hiếm hoặc chi phí của hai loại lỗi khác nhau. Cần xem confusion matrix và metric phù hợp.
- Tin rằng một kết quả đánh giá áp dụng cho mọi môi trường. Kết quả chỉ có nghĩa với dữ liệu, cách chia và metric đã nêu.

11. Tóm tắt công thức

Ý	Công thức	Cách đọc
Dữ liệu có nhãn	$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$	Cặp đầu vào và nhãn
Dự đoán	$\hat{y} = f_\theta(x)$	Mô hình dùng tham số θ
Loss trên một mẫu	$\ell(f_\theta(x_i), y_i)$	Sai số theo tiêu chuẩn đã chọn
Loss trung bình train	$\hat{R}_{\text{train}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell(f_\theta(x_i), y_i)$	Đại lượng thường được tối ưu
Risk trên phân phối	$R(\theta) = \mathbb{E}_{(x,y)\sim\mathcal{P}}[\ell(f_\theta(x), y)]$	Mục tiêu trên dữ liệu mới từ $\mathcal{P}$
Cập nhật gradient descent	$\theta_{t+1} = \theta_t - \eta \nabla_\theta \hat{R}_{\text{train}}(\theta_t)$	Đi ngược hướng tăng nhanh nhất của loss
Accuracy nhị phân	$\frac{TP+TN}{TP+TN+FP+FN}$	Tỷ lệ dự đoán đúng

Ý	Công thức	Cách đọc
Precision / recall	$\frac{TP}{TP+FP}$ / $\frac{TP}{TP+FN}$	Độ đúng trong dự đoán dương / độ phủ ca dương thật

12. Bài tập tự tính

Các bài dưới đây dùng số nhỏ để tập đọc công thức. Tính thử trước khi xem lời giải.

Bài 1: loss của dự đoán hằng

Ba nhãn là 2, 4, 6. Một mô hình dự đoán cùng một giá trị $c = 4$ cho mọi đầu vào. Tính MAE và MSE.

Đáp án: sai số tuyệt đối là 2, 0, 2, nên MAE $= (2 + 0 + 2)/3 = 4/3$ (tự tính). Bình phương sai số là 4, 0, 4, nên MSE $= 8/3$ (tự tính). Hai metric có đơn vị khác nhau: MAE cùng đơn vị với nhãn, MSE có đơn vị bình phương.

Bài 2: một bước cập nhật

Với một mẫu $x = 2$, $y = 6$, mô hình là $\hat{y} = wx$. Dùng loss bình phương $L(w) = (wx - y)^2$ và $w = 1$. Tính loss, đạo hàm theo w và tham số mới khi $\eta = 0.1$.

Đáp án: dự đoán là 2, sai số là -4, do đó loss bằng 16 (tự tính). Đạo hàm là $2(wx - y)x = 2(-4)(2) = -16$ (tự tính). Cập nhật cho $w_{\text{mới}} = 1 - 0.1(-16) = 2.6$ (tự tính). Đây là một bước theo đúng gradient của loss chưa chia cho số mẫu.

Bài 3: confusion matrix

Một bộ phân loại có $TP = 8$, $FP = 2$, $FN = 4$, $TN = 6$. Tính accuracy, precision và recall.

Đáp án: tổng là 20 trường hợp (tự tính). Accuracy bằng $(8+6)/20 = 0.7$ (tự tính); precision bằng $8/(8+2) = 0.8$ (tự tính); recall bằng $8/(8+4) = 2/3 \approx 0.667$ (tự tính). Precision và recall trả lời hai câu hỏi khác nhau.

Bài 4: chia dữ liệu

Ta có 50 người, mỗi người có nhiều bản ghi. Mục tiêu là ước lượng chất lượng dự đoán cho một người chưa xuất hiện trong train. Nên chia ngẫu nhiên từng bản ghi hay chia theo người? Giải thích.

Đáp án: chia theo người phù hợp hơn. Nếu cùng một người xuất hiện ở train và test, các đặc điểm ổn định của họ có thể rò sang test. Mục tiêu là người mới, nên toàn bộ bản ghi của mỗi người chỉ nên thuộc một phần dữ liệu. Đây là lựa chọn về thiết kế đánh giá, không phải phép tính số.

Bài 5: loss hay metric

Hai mô hình đạt accuracy bằng nhau. Mô hình A thường gán xác suất 0.51 cho lớp đúng; mô hình B gán 0.95 cho lớp đúng và 0.90 cho một dự đoán sai. Accuracy không tách được hai mô hình. Nên xem thêm loại đo lường nào?

Đáp án: xem loss xác suất như cross-entropy và calibration trên dữ liệu chưa dùng để chọn mô hình. Cross-entropy phân biệt dự đoán đúng nhưng lưỡng lự với dự đoán đúng rất tự tin, đồng thời phạt dự đoán sai tự tin. Cần xem metric bổ sung theo mục tiêu ứng dụng; không có đáp án bằng một con số vì bài không cho toàn bộ xác suất và nhãn.

Tài liệu tham khảo nền: Hastie, Tibshirani và Friedman, *The Elements of Statistical Learning*; Murphy, *Probabilistic Machine Learning*; Goodfellow, Bengio và Courville, *Deep Learning*; ml basics 00 (lời nói đầu).

Nền tảng 02

Hồi quy tuyến tính và logistic: chọn đường dự đoán hoặc ranh giới xác suất

Loại: Chương nền tảng, không phải ghi chú paper
Đọc trước: ml basics 00 (lời nói đầu) và chương nền tảng ml basics 01; đại số phổ thông là đủ để theo các phép tính
Dùng ở: ml basics 04 (XGBoost), 05 (mô hình dữ liệu bảng), 09-10 (word2vec và negative sampling), 19 (calibration), 30 (label smoothing), 32 (focal loss), 33 (mất cân bằng lớp)

> Chương này trình bày hai mô hình nền tảng từ kiến thức chung, không phải bản dịch một paper. Các con số trong ví dụ ghi "tự tính" đã được kiểm lại bằng NumPy.

1. Vì sao học hai mô hình đơn giản này

Hồi quy tuyến tính dự đoán một đại lượng liên tục bằng cách cộng có trọng số các đặc trưng. Logistic regression dùng một tổ hợp tuyến tính của đặc trưng để ước lượng xác suất thuộc một lớp. Cả hai đều dễ tính bằng tay, nên giúp ta thấy rõ mối nối giữa dữ liệu, tham số, loss và quyết định.

Ví dụ, một cửa hàng muốn dự đoán nhu cầu bán ra từ giá, ngày trong tuần và mức khuyến mãi. Một đường tuyến tính cho ta một giá trị dự đoán như 120 sản phẩm. Nếu cửa hàng muốn dự báo "có cần bổ sung hàng không", bài toán chuyển thành phân loại: ta cần xác suất sự kiện xảy ra, rồi chọn ngưỡng hành động. Hai bài toán có đầu vào tương tự nhưng nhãn, đầu ra và loss khác nhau.

"Tuyến tính" không có nghĩa chỉ có một đặc trưng. Mô hình có thể cộng nhiều đặc trưng với các trọng số khác nhau. Với hồi quy, đầu ra là số thực. Với logistic regression, tổ hợp tuyến tính được đưa qua hàm sigmoid để thu được số trong khoảng từ 0 đến 1. Vì vậy logistic regression là một bộ phân loại xác suất, dù phần bên trong nó vẫn tuyến tính theo đặc trưng.

2. Nhắc nền: đầu vào, mục tiêu và sai số

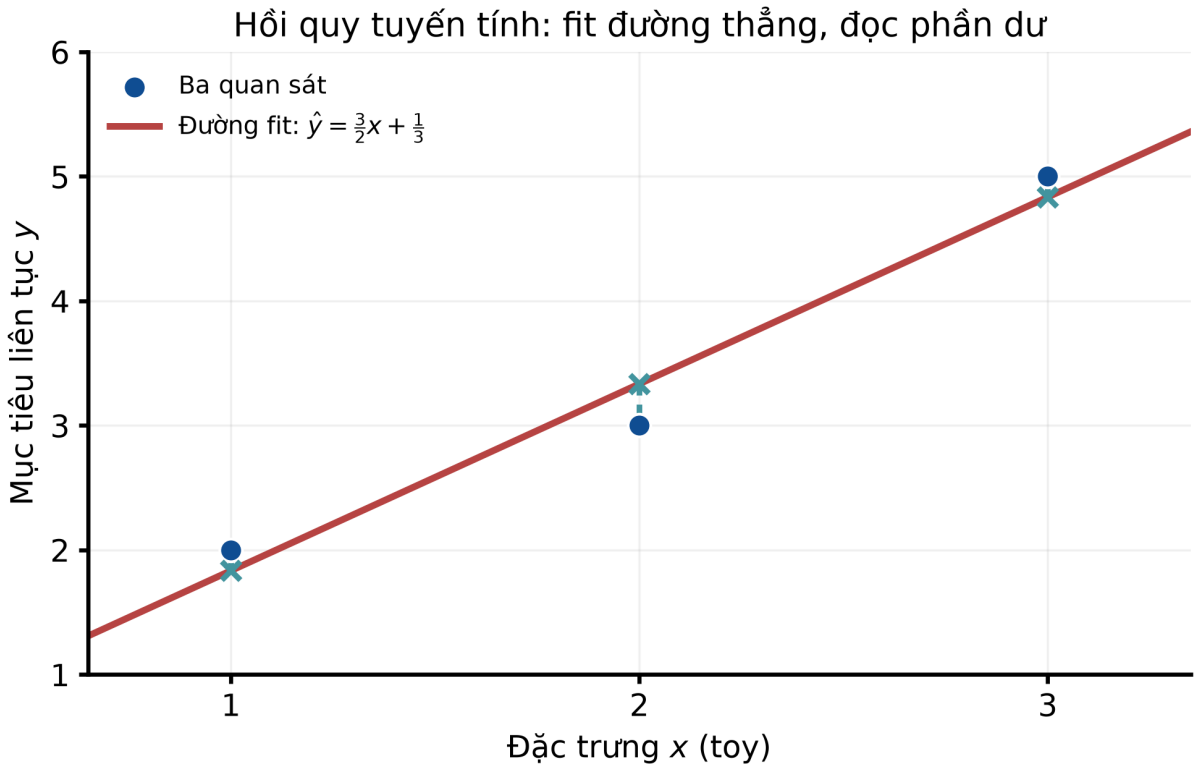
Ta quan sát các cặp (x_i, y_i). x_i là một ví dụ, có thể là một số hoặc vector gồm nhiều đặc trưng; y_i là mục tiêu cần dự đoán. Dấu mũ cho biết dự đoán của mô hình: $\hat{y}_i$. Phần dư của hồi quy thường viết $r_i = y_i - \hat{y}_i$. Nếu phần dư dương, dự đoán thấp hơn nhãn; nếu âm, dự đoán cao hơn.

Khi chỉ có một đặc trưng, ta có thể vẽ mỗi ví dụ thành một điểm trên mặt phẳng. Một đường thẳng gần các điểm là một mô hình hồi quy tuyến tính. Hình dưới cho thấy phần dư theo chiều dọc, vì ta đo sai số của mục tiêu y tại đầu vào x cố định.

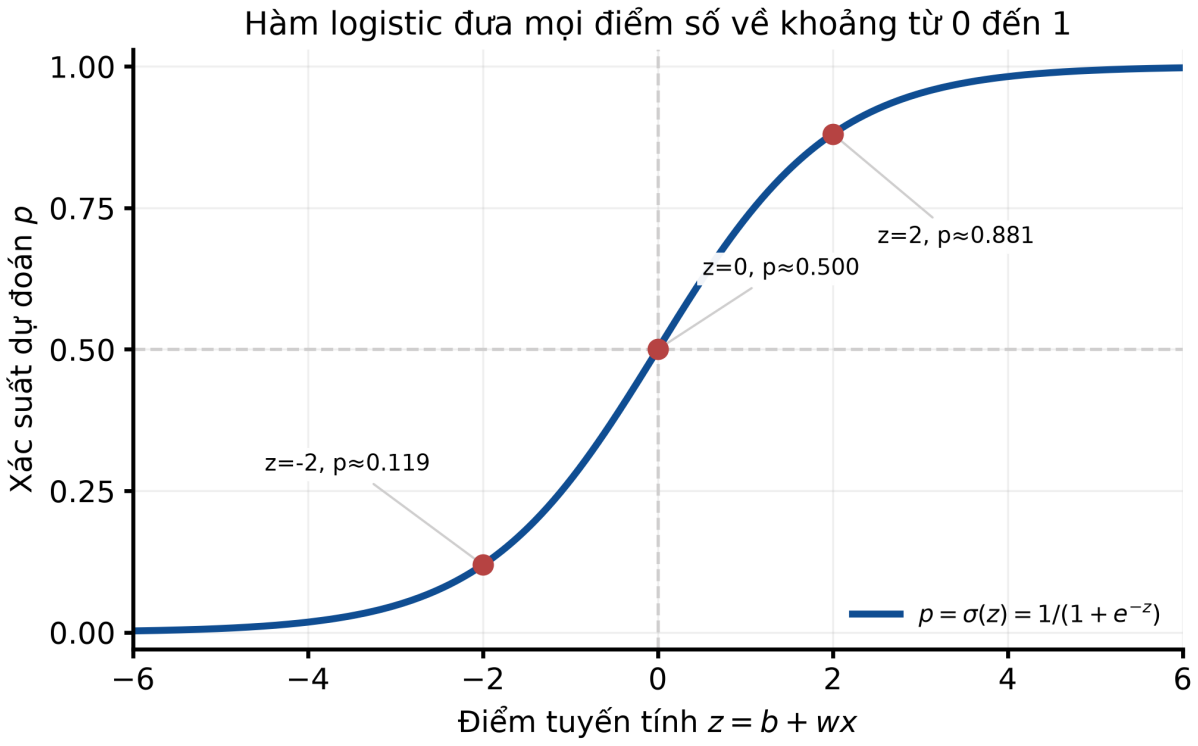
Cách đọc hình: chấm xanh là dữ liệu đã quan sát; đường đỏ là dự đoán của mô hình; dấu nhân và đoạn nét đứt biểu diễn vị trí dự đoán và độ lệch so với nhãn. Mô hình được chọn bằng cách cân bằng các phần dư theo một tiêu chuẩn, ở ví dụ này là tổng bình phương sai số.

Một bộ phân loại nhị phân có nhãn y bằng 0 hoặc 1. Thay vì chỉ nói "lớp nào", logistic regression trước tiên đưa ra xác suất dự đoán p cho lớp 1. Hình sigmoid cho thấy cách một điểm số tuyến tính có thể trở thành xác suất.

Cách đọc hình: trục ngang là điểm số z, trước khi giới hạn giá trị; trục dọc là p sau biến đổi. Điểm số tăng làm xác suất tăng, nhưng xác suất luôn ở trong khoảng từ 0 đến 1. Đường ngang tại 0.5 chỉ là một ngưỡng thường dùng, không phải ranh giới bắt buộc của mọi ứng dụng.



Hình 5: Ba điểm dữ liệu, đường fit tuyến tính và các phần dư (minh hoạ, tự tính)



Hình 6: Đường sigmoid biến điểm số tuyến tính thành xác suất của lớp dương (minh hoạ, tự tính)

3. Hồi quy tuyến tính: từ đường thẳng đến công thức

3.1 Mô hình và ý nghĩa tham số

Với một đặc trưng x, mô hình tuyến tính có dạng

$$\hat{y} = ax + b.$$

Hệ số a là hệ số góc: nếu x tăng một đơn vị thì dự đoán đổi a đơn vị, giữ các yếu tố khác như nhau. Hệ số b là hệ số chặn, tức giá trị dự đoán khi $x = 0$. Ý nghĩa thực tế của b phụ thuộc việc $x = 0$ có hợp lý trong miền dữ liệu hay không.

Với nhiều đặc trưng, mô hình trở thành

$$\hat{y} = b + \sum_{j=1}^{d} w_j x_j.$$

Mỗi w_j cho biết đóng góp tuyến tính của đặc trưng thứ j khi các đặc trưng còn lại giữ nguyên. Nếu đặc trưng có đơn vị khác nhau, độ lớn của các hệ số không thể so trực tiếp một cách đơn giản. Tương quan giữa các đặc trưng cũng làm cách diễn giải từng hệ số khó hơn.

Mô hình vẫn được gọi là tuyến tính theo tham số vì a, b hoặc các trọng số xuất hiện ở bậc một. Ta có thể tạo thêm đặc trưng như x^2 rồi fit $w_1 x + w_2 x^2 + b$; mô hình khi ấy cong theo x nhưng vẫn tuyến tính theo các tham số w_1, w_2, b. Chọn đặc trưng mới đồng nghĩa chọn một họ hàm khác.

3.2 Bình phương tối thiểu

Với dự đoán liên tục, một lựa chọn phổ biến là bình phương sai số. Loss trung bình trên N ví dụ là

$$L(a, b) = \frac{1}{N} \sum_{i=1}^{N} (ax_i + b - y_i)^2.$$

Bình phương làm cho phần dư dương và âm không triệt tiêu nhau, đồng thời phạt lỗi lớn mạnh hơn lỗi nhỏ. Nếu muốn tối thiểu hóa tổng bình phương thay vì trung bình, nghiệm tối ưu của a, b không đổi; chỉ quy mô loss và gradient đổi theo số mẫu.

Với một đặc trưng và có hệ số chặn, nghiệm bình phương tối thiểu có thể viết bằng trung bình của dữ liệu:

$$a^* = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad b^* = \bar{y} - a^* \bar{x}.$$

Thanh ngang trên biến là trung bình mẫu. Công thức cho thấy hệ số góc phụ thuộc cách x và y cùng đổi quanh trung bình. Khi mẫu số bằng 0, mọi x_i như nhau và ta không thể ước lượng hệ số góc từ dữ liệu đó.

Với nhiều đặc trưng, ta xếp mỗi hàng đầu vào thành một hàng của ma trận thiết kế X, thêm một cột toàn số 1 để biểu diễn hệ số chặn, rồi viết $\hat{y} = Xw$. Khi $X^\top X$ khả nghịch, nghiệm thỏa phương trình chuẩn $X^\top X w = X^\top y$, hay $w = (X^\top X)^{-1} X^\top y$. Nếu các cột phụ thuộc tuyến tính hoặc số học không ổn định, dùng phép phân rã QR hoặc SVD an toàn hơn là tính nghịch đảo trực tiếp. Nghiệm tối ưu tồn tại, nhưng không nhất thiết duy nhất nếu thiết kế suy biến.

3.3 Ví dụ tay: fit đường qua ba điểm

Ta dùng ba điểm toy $(x, y) = (1, 2), (2, 3), (3, 5)$ (tự tính). Ta muốn tìm đường $\hat{y} = ax + b$ có tổng bình phương phần dư nhỏ nhất. Đầu tiên, trung bình các đầu vào là $\bar{x} = (1 + 2 + 3)/3 = 2$; trung bình mục tiêu là $\bar{y} = (2 + 3 + 5)/3 = 10/3$ (tự tính).

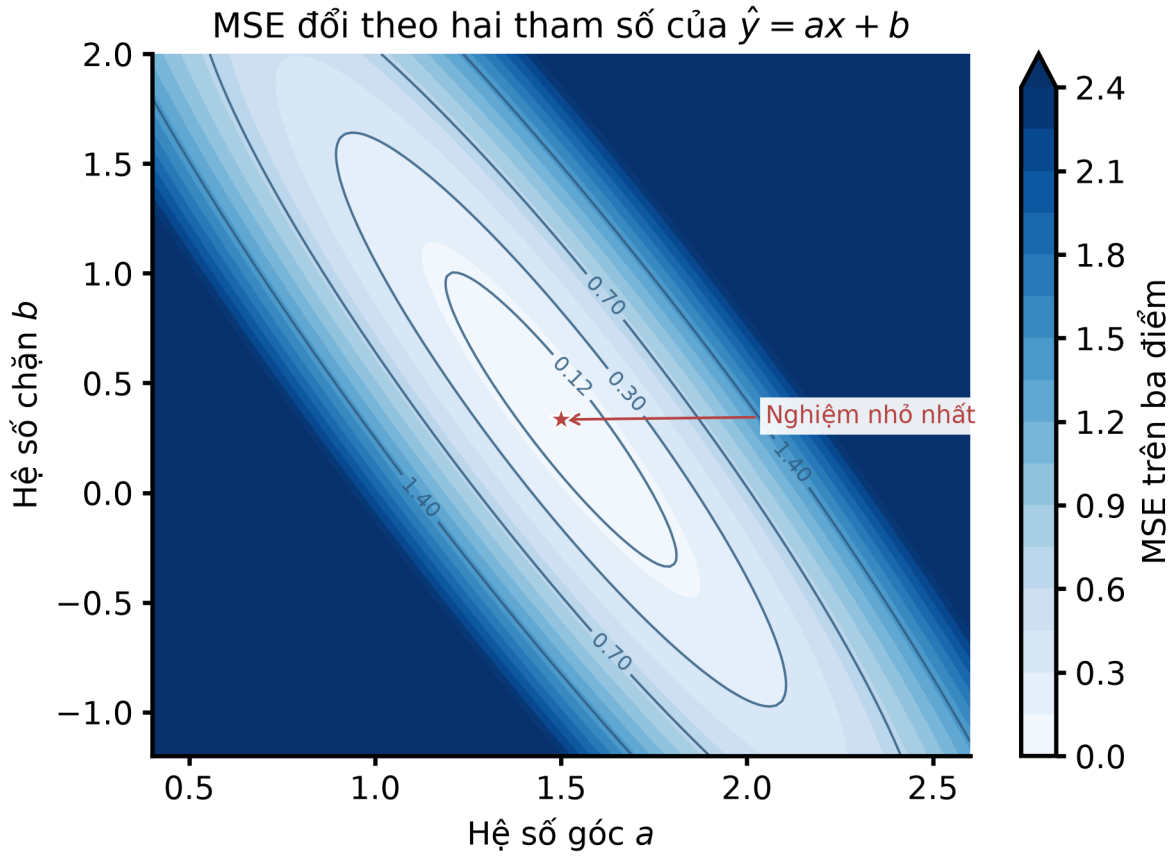
Các độ lệch của x là $-1, 0, 1$; độ lệch của y là $-4/3, -1/3, 5/3$ (tự tính). Tổng tích độ lệch bằng $(-1)(-4/3) + 0(-1/3) + (1)(5/3) = 3$; tổng bình phương độ lệch của x là $1 + 0 + 1 = 2$ (tự tính). Do đó hệ số góc là $a^* = 3/2$ và hệ số chặn là $b^* = 10/3 - (3/2)2 = 1/3$ (tự tính).

Đường fit là $\hat{y} = (3/2)x + 1/3$. Tại ba đầu vào, dự đoán lần lượt là $11/6$, $10/3$, $29/6$ (tự tính). Phần dư theo quy ước nhãn trừ dự đoán là $1/6$, $-1/3$, $1/6$ (tự tính). Tổng bình phương phần dư bằng $(1/6)^2 + (-1/3)^2 + (1/6)^2 = 1/6$, nên MSE bằng $(1/6)/3 = 1/18$ (tự tính). MAE bằng $(1/6 + 1/3 + 1/6)/3 = 2/9$ (tự tính).

Điểm quan trọng không phải nhớ các phân số, mà là quy trình: tính trung bình, đo độ lệch quanh trung bình, lấy hệ số góc, tìm hệ số chặn, rồi thay đầu vào để kiểm sai số. Dữ liệu toy nhỏ nên ta làm tay được; thư viện thống kê thực hiện cùng phép tính trên bộ dữ liệu lớn hơn.

3.4 Hình dạng loss và cách tìm tham số

Với bộ dữ liệu ba điểm trên, MSE theo hai tham số a, b tạo thành một bề mặt cong. Đường đồng mức cho biết những cặp tham số có cùng loss. Điểm đỏ đánh dấu nghiệm nhỏ nhất trên hình. Chỉ một tham số sai cũng làm loss tăng; thay đổi cả hệ số góc và hệ số chặn tạo ra một hướng tìm kiếm trong không gian tham số.



Hình 7: Đường đồng mức của MSE theo hệ số góc và hệ số chặn (minh hoạ, tự tính)

Cách đọc hình: hai trục là hai tham số; màu nhạt hơn chỉ loss thấp hơn trong miền đang vẽ. Các đường đồng mức khép quanh ngôi sao đỏ, nơi nghiệm bình phương tối thiểu được tính từ dữ liệu. Vùng kéo dài cho thấy

nhiều tổ hợp tham số có thể cho loss khá gần nhau.

Đạo hàm của MSE cho ta hướng thay đổi loss khi chỉnh một tham số. Với một đặc trưng, gradient là

$$\frac{\partial L}{\partial a} = \frac{2}{N}\sum_i (ax_i + b - y_i)x_i, \qquad \frac{\partial L}{\partial b} = \frac{2}{N}\sum_i (ax_i + b - y_i).$$

Ta có thể dùng gradient descent để cập nhật a và b. Với hồi quy bình phương tối thiểu, nghiệm đóng thường có thể tìm nhanh; gradient descent vẫn hữu ích như một ví dụ rõ ràng về cách tối ưu và là cách thực tế cho nhiều mô hình lớn không có nghiệm đóng.

3.5 Điều mô hình tuyến tính không tự làm được

Đường thẳng có thể bỏ lỡ quan hệ cong, ngưỡng hoặc tương tác. Ta có thể tạo đặc trưng mới, dùng biến đổi như logarit, hoặc chọn họ mô hình khác. Nhưng tăng độ phức tạp cũng làm mô hình có nhiều cách khớp nhiễu hơn. Mỗi cải tiến phải được so sánh trên dữ liệu dành riêng cho đánh giá, không chỉ dựa vào loss train.

4. Logistic regression: từ điểm số đến xác suất

4.1 Điểm tuyến tính và sigmoid

Trong phân loại nhị phân, mỗi ví dụ có nhãn $y_i \in \{0, 1\}$. Logistic regression trước tiên tính điểm số tuyến tính

$$z_i = b + w^\top x_i,$$

rồi đưa z_i qua sigmoid:

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

Ta diễn giải p_i là xác suất mô hình gán cho lớp dương, còn $1 - p_i$ là xác suất cho lớp âm. Khi $z = 0$, sigmoid bằng $1/2$. Với $z = -2$, xác suất xấp xỉ 0.1192; với $z = 0$, nó bằng 0.5; với $z = 2$, nó xấp xỉ 0.8808 (tự tính). Các điểm này được đánh trên hình sigmoid ở mục 2.

Sigmoid có thể viết ngược dưới dạng log-odds:

$$\log \frac{p_i}{1 - p_i} = b + w^\top x_i.$$

Vậy logistic regression tuyến tính theo log-odds, chứ không tuyến tính theo xác suất. Đặc trưng làm tăng điểm số sẽ tăng xác suất lớp dương, nhưng độ thay đổi xác suất phụ thuộc vùng đang xét: gần 0.5 đường cong dốc; ở hai đầu, xác suất thay đổi chậm hơn.

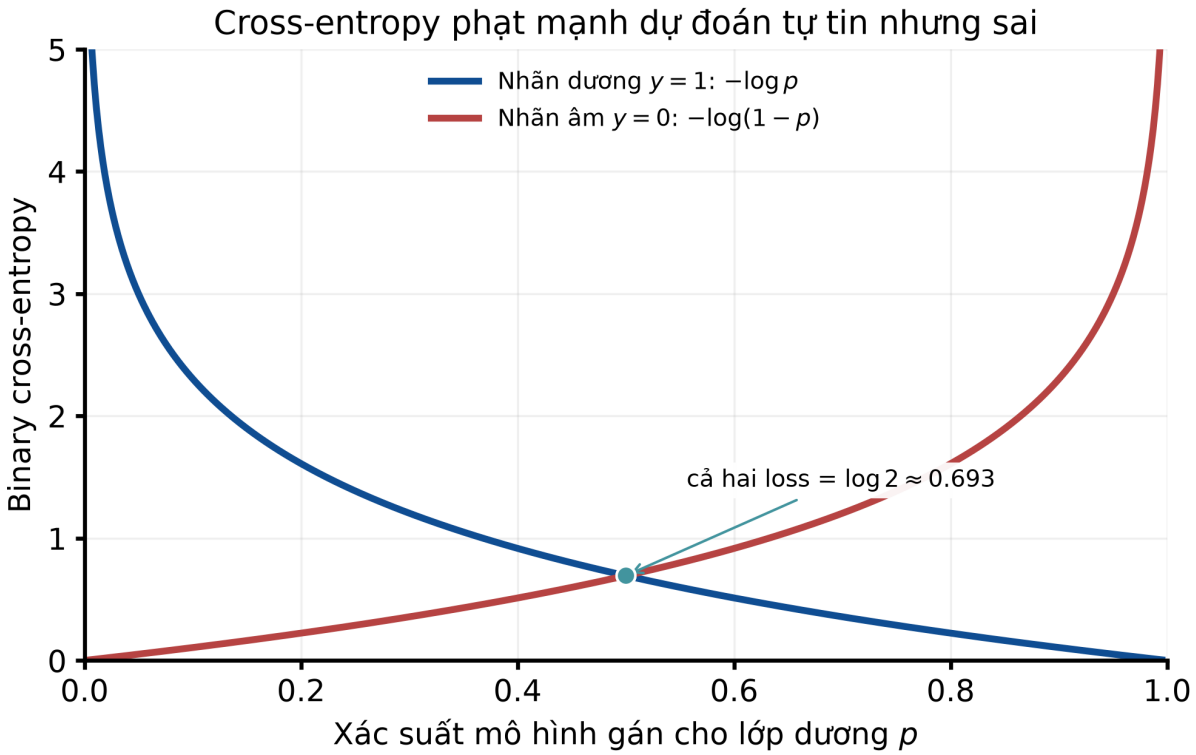
4.2 Binary cross-entropy

Với nhãn nhị phân, một mô hình Bernoulli gán xác suất p_i cho nhãn 1 và $1 - p_i$ cho nhãn 0. Xác suất của nhãn quan sát có thể viết $p_i^{y_i}(1 - p_i)^{1-y_i}$. Lấy âm log xác suất này cho ta binary cross-entropy trên một ví dụ:

$$\ell_i = -y_i \log p_i - (1 - y_i)\log(1 - p_i).$$

Nếu $y_i = 1$, loss rút gọn thành $-\log p_i$: xác suất gần 1 được thưởng bằng loss nhỏ. Nếu $y_i = 0$, loss là $-\log(1 - p_i)$: mô hình cần gán xác suất thấp cho lớp dương. Mục tiêu trên tập train là trung bình các loss

này.



Hình 8: Binary cross-entropy theo xác suất cho nhãn dương và nhãn âm (minh hoạ, tự tính)

Cách đọc hình: đường xanh ứng với nhãn dương, đường đỏ ứng với nhãn âm. Mỗi đường có loss nhỏ khi mô hình gán xác suất phù hợp với nhãn; loss tăng nhanh khi mô hình tự tin vào lớp sai. Tại $p = 0.5$, hai nhãn đều có loss $-\log(0.5) = \log 2 \approx 0.693$ (tự tính).

4.3 Ví dụ tay: một cập nhật logistic

Xét một ví dụ có đặc trưng $x = 1$ và nhãn dương $y = 1$ (tự tính). Mô hình đơn giản là $z = b + wx$. Bắt đầu ở $b = 0, w = 0$, do đó $z = 0$ và $p = 0.5$ (tự tính). Loss là $-\log(0.5) = \log 2 \approx 0.6931$ (tự tính).

Với sigmoid và binary cross-entropy, đạo hàm theo điểm số gọn thành $\partial \ell / \partial z = p - y$. Vì $z = b + wx$, đạo hàm theo hệ số chặn là $p - y$, còn theo trọng số w là $(p - y)x$. Ở ví dụ hiện tại, cả hai đạo hàm bằng -0.5 (tự tính). Đây là dấu hiệu tăng b và w sẽ làm loss giảm cho ví dụ dương này.

Chọn tốc độ học $\eta = 0.5$ (tự tính). Một bước gradient descent làm b và w cùng tăng thêm 0.25, nên tham số mới là $b = 0.25, w = 0.25$ (tự tính). Với $x = 1$, điểm số mới $z = 0.5$ và xác suất mới là $\sigma(0.5) \approx 0.6225$ (tự tính). Tính loss từ giá trị sigmoid chưa làm tròn: $-\log(\sigma(0.5)) \approx 0.4741$ (tự tính). Loss đã giảm, nhưng một mẫu không đủ để kết luận mô hình tốt. Nếu loss của batch được định nghĩa là trung bình, gradient batch cũng là trung bình các đóng góp của từng mẫu; nếu dùng tổng loss thì gradient là tổng và quy mô bước học cần điều chỉnh tương ứng. Khi có nhiều ví dụ, cập nhật phải cân bằng đóng góp của cả nhãn dương lẫn nhãn âm.

4.4 Từ xác suất đến quyết định

Để biến xác suất thành một nhãn cứng, ta chọn **ngưỡng quyết định** τ: dự đoán lớp 1 nếu $p_i \geq \tau$. Ngưỡng 0.5 phổ biến vì tương ứng xác suất hai lớp ngang nhau khi chi phí lỗi bằng nhau và xác suất đã được hiệu chỉnh hợp lý. Trong ví dụ thực tế, chi phí bỏ sót và báo động nhầm có thể khác nhau, vì vậy ngưỡng nên được chọn theo mục tiêu và dữ liệu validation.

Với $\tau = 0.5$, quy tắc sigmoid tương đương dự đoán lớp dương khi $z_i \geq 0$. Nếu chỉ có một đặc trưng và $w > 0$, ranh giới thỏa $x = -b/w$. Đổi ngưỡng sẽ dịch vị trí ranh giới mà không cần huấn luyện lại trọng số. Tuy nhiên, thay ngưỡng không làm xác suất trở nên hiệu chỉnh tốt hơn; nó chỉ đổi cách chuyển xác suất thành hành động.

4.5 Tối ưu logistic regression

Không giống bình phương tối thiểu đơn giản, nghiệm logistic thường được tìm bằng một thuật toán lặp. Với mỗi ví dụ, gradient theo vector trọng số có dạng $(p_i - y_i)x_i$; gradient theo hệ số chặn là $p_i - y_i$. Vì loss ở trên được định nghĩa là trung bình, gradient trên batch cũng là trung bình các đóng góp của từng ví dụ, rồi ta cập nhật theo hướng ngược gradient. Nếu dùng tổng loss thay cho trung bình, gradient là tổng và cần điều chỉnh quy mô bước học. Chi tiết gradient descent và các biến thể nằm trong `optimization content/`.

Khi dữ liệu phân tách hoàn hảo, loss logistic có thể tiếp tục giảm nếu độ lớn trọng số tăng mãi, thay vì đạt một nghiệm hữu hạn có trọng số nhỏ nhất. Regularization như phạt L2 giúp hạn chế độ lớn tham số và có thể cải thiện tổng quát hóa; xem chương nền tảng ml basics 04 về regularization. Điều này cũng nhắc ta rằng "đã tối ưu loss" cần nói rõ có regularization hay không, tiêu chí dừng nào được dùng và nghiệm nào được chọn.

5. Hồi quy tuyến tính và logistic khác nhau ở đâu

Thành phần	Hồi quy tuyến tính	Logistic regression
Loại nhãn thường gặp	Số liên tục y	Nhãn nhị phân $y \in \{0, 1\}$
Giá trị mô hình	$\hat{y} = b + w^\top x$	$p = \sigma(b + w^\top x)$
Loss thường dùng	MSE hoặc MAE	Binary cross-entropy
Đầu ra trực tiếp	Số thực, không bị chặn	Xác suất trong $(0, 1)$
Dùng quyết định cuối	Dự báo giá trị	So xác suất với ngưỡng

Hồi quy tuyến tính không tự giới hạn đầu ra vào một khoảng hợp lệ. Nếu dự đoán tỷ lệ hoặc số lượng không âm, cần cân nhắc biến đổi mục tiêu hoặc mô hình khác. Logistic regression bảo đảm xác suất nằm trong $(0, 1)$, nhưng đường biên quyết định vẫn tuyến tính theo đặc trưng đã đưa vào. Nếu quan hệ thực tế cong, có thể thêm đặc trưng phi tuyến hoặc đổi họ mô hình.

Hai mô hình cũng không nhất thiết dùng cùng thước đo đánh giá. Với hồi quy, MAE và RMSE thường dễ diễn giải theo đơn vị mục tiêu; với phân loại, confusion matrix, precision, recall, ROC AUC và calibration trả lời những câu hỏi khác nhau. Xem chương nền tảng ml basics 01 để biết vì sao metric cần gắn với chi phí ứng dụng.

6. Các biến thể và chi tiết hay bị bỏ qua

Hồi quy với nhiều biến cộng trọng số của nhiều đặc trưng. Nếu hai đặc trưng gần như trùng thông tin, có thể khó tách riêng đóng góp của chúng và nghiệm trở nên nhạy với nhiễu. Ridge thêm mức phạt bình phương trọng số; lasso thêm mức phạt trị tuyệt đối và có thể đẩy một số trọng số về 0. Hai phương pháp thuộc nhóm regularization, sẽ được giải thích trong chương nền tảng ml basics 04.

Đặc trưng phi tuyến như bình phương, tương tác giữa hai biến hoặc biến đổi logarit cho phép đường dự đoán cong theo đầu vào. Ví dụ $\hat{y} = w_0 + w_1 x + w_2 x^2$ có thể uốn cong, dù vẫn tuyến tính theo ba tham số. Việc thêm đặc trưng không tự động làm dự đoán tốt hơn; phải chọn bằng validation.

Logistic đa lớp thường dùng softmax cho một nhãn trong số các lớp loại trừ nhau; softmax tạo một xác suất cho mỗi lớp và các xác suất cộng lại bằng 1. Với bài toán đa nhãn, sigmoid riêng cho từng lớp vẫn là một lựa chọn tự nhiên vì nhiều lớp có thể cùng đúng. Trong trường hợp đó, các xác suất không tự cộng thành 1 và cần quy tắc quyết định hoặc hiệu chỉnh phù hợp.

Mất cân bằng lớp làm cho ngưỡng và metric quan trọng hơn. Nếu lớp dương hiếm, mô hình có thể đạt accuracy cao bằng cách đoán âm hầu hết thời gian. Có thể cân nhắc trọng số lớp, lấy mẫu lại hoặc ngưỡng khác, nhưng lựa chọn cần được kiểm tra trên validation phản ánh môi trường triển khai. Paper ml basics 33 bàn về oversampling cho CNN; paper ml basics 32 về focal loss.

Xác suất không đồng nghĩa độ chắc chắn đã được hiệu chỉnh. Mô hình có thể dự đoán 0.9 cho nhiều trường hợp nhưng chỉ đúng trong 0.7 số đó. Logistic regression có đầu ra dạng xác suất, nhưng dữ liệu lệch, regularization hoặc đổi phân phối vẫn có thể làm xác suất kém hiệu chỉnh. Paper ml basics 19 trình bày reliability diagram và calibration.

7. Nối sang AI

Logistic regression là một bộ phân loại cơ bản, nhưng ý tưởng xác suất nhị phân còn xuất hiện bên trong thuật toán lớn hơn. Trong negative sampling của word2vec, mô hình học phân biệt cặp từ thật với cặp nhiễu bằng một mục tiêu logistic. Xem paper ml basics 10 để biết cách dùng này khác việc phân loại email hay dự đoán sự kiện ở chỗ nào.

Trong mạng neural, tầng cuối có thể dùng sigmoid cho một nhãn nhị phân hoặc softmax cho nhiều lớp. Các tầng trước biến đầu vào thành biểu diễn; tầng cuối vẫn cần một loss và quy tắc tối ưu. Vì thế, hiểu logistic giúp đọc loss phân loại trước khi chuyển sang mạng nhiều lớp ở chương nền tảng ml basics 05-06.

Hồi quy tuyến tính cũng là thành phần của nhiều phương pháp. Gradient boosting thường fit một bộ học yếu vào phần dư hoặc gradient của loss; hồi quy tuyến tính có thể làm mô hình nền để so sánh. Mô hình đơn giản là baseline hữu ích: nếu mô hình phức tạp không cải thiện điểm ngoài train, cần hỏi liệu độ phức tạp có tạo giá trị hay không.

Cuối cùng, công thức không giải quyết vấn đề thiết kế dữ liệu. Nếu nhãn bị rò vào đặc trưng, logistic có thể cho phân loại gần hoàn hảo trong kiểm tra sai cách. Nếu đầu ra cần một quyết định quan trọng, phải xem loại lỗi, ngưỡng, calibration và phân phối triển khai, không chỉ nhìn độ chính xác.

8. Sai lầm hay gặp

- Nghĩ hồi quy tuyến tính chỉ áp dụng khi dữ liệu nằm chính xác trên một đường thẳng. Nó thường là phép xấp xỉ tối ưu theo một loss, nên phần dư vẫn có thể khác 0.
- Nghĩ "tuyến tính" nghĩa chỉ có một đặc trưng hoặc không thể tạo đường cong. Đặc trưng biến đổi cho phép đường cong theo đầu vào, nhưng vẫn tuyến tính theo tham số.
- Dùng hồi quy tuyến tính trực tiếp cho nhãn phân loại rồi coi giá trị ngoài khoảng 0 đến 1 là xác suất. Mô hình không có ràng buộc xác suất.
- Nhầm sigmoid với softmax. Sigmoid hay dùng cho phân loại nhị phân hoặc từng nhãn độc lập; softmax biểu diễn các lớp loại trừ nhau.
- Cho rằng ngưỡng 0.5 luôn tối ưu. Ngưỡng phụ thuộc tỷ lệ lớp, chi phí của từng loại lỗi và mục tiêu vận hành.
- Xem xác suất do logistic đưa ra là đã hiệu chỉnh. Cần so xác suất dự đoán với tỷ lệ đúng thực tế trên dữ liệu đánh giá phù hợp.

9. Tóm tắt công thức

Ý	Công thức	Ghi nhớ
Hồi quy một biến	$\hat{y} = ax + b$	Số liên tục theo đường thẳng
MSE hồi quy	$\frac{1}{N}\sum_i (ax_i + b - y_i)^2$	Bình phương và lấy trung bình phần dư
Hệ số góc bình phương tối thiểu	$a^* = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$	Đồng biến thiên chia độ biến thiên của x

Ý	Công thức	Ghi nhớ
Hệ số chặn	$b^* = \bar{y} - a^*\bar{x}$	Đường đi qua điểm trung bình $(\bar{x}, \bar{y})$
Điểm logistic	$z_i = b + w^\top x_i$	Tổ hợp tuyến tính trước sigmoid
Sigmoid	$p_i = \frac{1}{1+e^{-z_i}}$	Đưa điểm số về xác suất lớp 1
Log-odds	$\log \frac{p_i}{1-p_i} = z_i$	Logistic tuyến tính theo log-odds
Binary cross-entropy	$-y_i \log p_i - (1-y_i)\log(1-p_i)$	Âm log xác suất nhãn quan sát
Gradient theo điểm số	$\frac{\partial \ell_i}{\partial z_i} = p_i - y_i$	Dấu hiệu dự đoán cao hay thấp so với nhãn
Quyết định theo ngưỡng	$\hat{y}_i = \mathbb{1}[p_i \geq \tau]$	Ngưỡng τ phụ thuộc chi phí lỗi

10. Bài tập tự tính

Bài 1: tìm một đường thẳng khớp chính xác

Cho ba điểm $(x, y) = (0, 1), (1, 3), (2, 5)$. Tìm đường $\hat{y} = ax + b$ khớp chính xác cả ba điểm.

Đáp án: khi x tăng 1, y tăng 2, nên $a = 2$; tại $x = 0$, $b = 1$. Đường là $\hat{y} = 2x + 1$, cho dự đoán $1, 3, 5$ và MSE bằng 0 (tự tính).

Bài 2: MSE và MAE của một mô hình

Với điểm $x = 1, 2, 3$ và nhãn $y = 2, 4, 6$, dùng mô hình $\hat{y} = x$. Tính MAE và MSE.

Đáp án: dự đoán là $1, 2, 3$, phần dư theo nhãn trừ dự đoán là $1, 2, 3$ (tự tính). MAE bằng $(1 + 2 + 3)/3 = 2$; MSE bằng $(1 + 4 + 9)/3 = 14/3 \approx 4.667$ (tự tính).

Bài 3: sigmoid và quyết định

Một ví dụ có điểm số $z = 0$. Tính xác suất logistic và quyết định nhãn khi dùng ngưỡng $\tau = 0.5$.

Đáp án: $p = 1/(1 + e^0) = 0.5$ (tự tính). Với quy tắc dự đoán dương khi $p \geq \tau$, nhãn dự đoán là 1 (tự tính). Nếu muốn tránh hòa, ứng dụng phải định nghĩa rõ dùng lớn hơn hay lớn hơn hoặc bằng.

Bài 4: cross-entropy và một bước gradient

Một mẫu có $x = 2, y = 1$. Khởi tạo $b = 0, w = 0$. Tính loss ban đầu; sau đó làm một bước gradient descent với tốc độ học $\eta = 0.25$.

Đáp án: ban đầu $z = 0, p = 0.5$, loss $\log 2 \approx 0.6931$ (tự tính). Gradient theo b là $p - y = -0.5$; gradient theo w là $(p - y)x = -1$ (tự tính). Cập nhật cho $b = 0.125, w = 0.25$ (tự tính). Điểm số mới là $0.125 + 0.25(2) = 0.625$, nên xác suất mới là $\sigma(0.625) \approx 0.6514$ và loss mới là $-\log(\sigma(0.625)) \approx 0.4287$ (tự tính).

Bài 5: chọn ngưỡng theo ưu tiên

Một hệ thống cảnh báo có xác suất dương dự đoán $p = 0.42$. Nếu ngưỡng được chọn là 0.3, hệ thống báo dương hay âm? Nếu tăng ngưỡng lên 0.6, điều gì đổi?

Đáp án: tại ngưỡng 0.3, vì $0.42 \geq 0.3$, dự đoán là dương (tự tính). Tại ngưỡng 0.6, vì $0.42 < 0.6$, dự đoán là âm (tự tính). Tăng ngưỡng thường làm ít trường hợp bị gắn dương hơn; tác động lên precision và recall cần được đo trên dữ liệu validation.

Tài liệu tham khảo nền: Hastie, Tibshirani và Friedman, *The Elements of Statistical Learning*; Murphy, *Probabilistic Machine Learning*; Bishop, *Pattern Recognition and Machine Learning*; ml basics 00 (lời nói đầu).

Nền tảng 03

Overfitting, underfitting và bias-variance: học quy luật thay vì học thuộc dữ liệu

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: dữ liệu, dự đoán, hàm mất mát và trung bình; có thể đọc ml basics nền 01, 02 trước

Dùng ở: ml basics 01, ml basics 02, ml basics 03, ml basics 21, ml basics 34

> Chương này là bài học từ đầu, không phải bản dịch paper. Các phép tính ghi "tự tính" đã được kiểm bằng NumPy; dữ liệu và hình chỉ để minh hoạ.

1. Vì sao cần phân biệt học và học thuộc

Một mô hình **tổng quát hoá** khi nó dùng được điều đã học để dự đoán trên dữ liệu mới thuộc tình huống ta quan tâm. Nếu mô hình chỉ trả lời tốt trên những ví dụ đã gặp, ta chưa có bằng chứng rằng nó đã học được quy luật có ích. **Overfitting**, hay khớp quá mức, là khi cách học đã thích nghi quá nhiều với dữ liệu huấn luyện, kể cả những chi tiết tình cờ, nên kết quả trên dữ liệu mới kém hơn mong muốn. **Underfitting**, hay khớp chưa đủ, là khi mô hình hoặc quá trình học chưa nắm được cả quy luật có thể học từ dữ liệu.

Hãy nghĩ tới người học nhìn điểm trên đồ thị giá nhà. Một đường thẳng có thể bỏ sót quan hệ cong giữa diện tích và giá. Một đường rất ngoằn ngoèo có thể đi chính xác qua mọi căn nhà đã thấy nhưng lại thay đổi dự đoán quá mạnh giữa hai căn gần giống nhau. Đường nào đáng tin còn phụ thuộc dữ liệu mới và mục đích dùng; không thể kết luận chỉ bằng việc nhìn đường nào đi qua các điểm cũ.

Trong AI, ta phải chọn kiến trúc, đặc trưng, thời gian huấn luyện và nhiều cấu hình khác mà không biết hết dữ liệu tương lai. Các tập train, validation và test giúp tổ chức phép thử đó. Phân tích bias-variance giúp hiểu hai nguồn sai số khi lặp lại việc lấy dữ liệu và huấn luyện. Chương này đưa ra những công cụ để hỏi đúng câu hỏi, chứ không hứa một hình dạng đường cong luôn đúng cho mọi mô hình.

2. Nhắc nhanh: dự đoán, mất mát và trung bình

Một mẫu gồm đầu vào x và giá trị cần dự đoán y. Mô hình tạo dự đoán $\hat{y} = h(x)$. Với bài toán dự đoán số, một thước đo đơn giản là **mất mát bình phương** $(\hat{y} - y)^2$: sai khác càng lớn thì điểm phạt càng lớn. Sai số bình phương trung bình, viết tắt MSE, lấy trung bình các điểm phạt trên tập được xét.

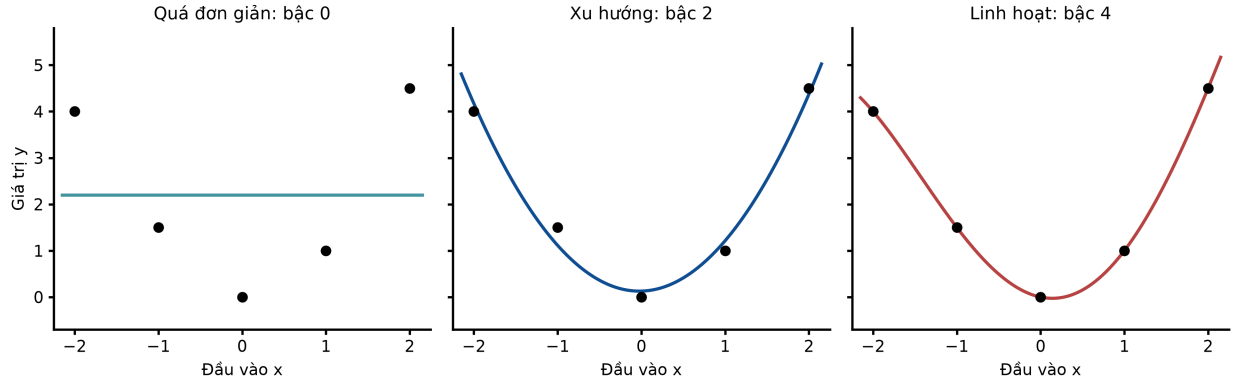
Ví dụ (tự tính): ba giá trị thật là $(1, 2, 3)$, dự đoán là $(1, 1, 5)$. Sai khác lần lượt là $(0, -1, 2)$, bình phương là $(0, 1, 4)$, và MSE bằng $(0 + 1 + 4)/3 = 5/3$. Dự đoán bằng $(2, 2, 2)$ cho các bình phương $(1, 0, 1)$ và MSE bằng $2/3$. Trong ví dụ này, cách thứ hai tốt hơn theo MSE trên ba mẫu đã cho; điều đó chưa tự nó cho biết cách nào tốt hơn trên quần thể.

Tham số là các giá trị được thuật toán học từ tập train, ví dụ hệ số của đường thẳng. **Siêu tham số** là lựa chọn điều khiển cách học, chẳng hạn bậc đa thức, mức phạt hay số bước dừng. Siêu tham số thường được chọn nhờ validation. Ranh giới này phụ thuộc quy trình: nếu một lựa chọn được tự động điều chỉnh từ train thì cách mô tả có thể khác, nhưng phải luôn ghi rõ dữ liệu nào đã ảnh hưởng tới nó.

Một tập dữ liệu chỉ là mẫu hữu hạn. MSE trên tập đó thay đổi nếu lấy mẫu khác. Vì vậy cần phân biệt điểm số ta **tính được trên mẫu** với sai số kỳ vọng trên phân phối dữ liệu tương lai mà ta muốn ước lượng. Nhãn có thể nhiễu, mẫu có thể không đại diện, và ngay cả hai mô hình có sai số thực bằng nhau cũng có thể cho điểm khác nhau trên một tập test nhỏ.

3. Hình dung trước: ba đường đi qua cùng dữ liệu

Giả sử đầu vào là một số và quan hệ nền có dạng gần chữ U. Ta vẽ cùng năm điểm, rồi dùng ba họ hàm khác nhau: hằng số, đa thức bậc hai, và đa thức bậc bốn. Các điểm chỉ là ví dụ tự tạo, không đại diện cho một bộ dữ liệu thực.



Hình 9: Ba mức linh hoạt cùng khớp các điểm dữ liệu tổng hợp (minh hoạ, tự tính)

Cách đọc hình: chấm đen là cùng năm quan sát trong cả ba ô, đường màu là mô hình khớp với chúng. Đường hằng số bên trái không thể uốn theo xu hướng chữ U; đường bậc hai ở giữa nắm xu hướng đó. Đường bậc bốn bên phải đi qua cả năm điểm, kể cả các dao động riêng của chúng. Hình gợi ý rủi ro overfitting, nhưng chưa chứng minh đường bậc bốn kém hơn: muốn biết cần dữ liệu mới hoặc một giả định rõ về quy luật thật.

Độ linh hoạt không đồng nghĩa với số tham số trong mọi tình huống. Nó còn phụ thuộc thuật toán, cách khởi tạo, regularization, dữ liệu và thời gian huấn luyện. Một họ hàm đủ mạnh có thể biểu diễn quy luật tốt, nhưng thuật toán vẫn phải tìm ra nghiệm phù hợp và cần dữ liệu để chọn giữa nhiều nghiệm.

4. Định nghĩa bằng sai số trên train và dữ liệu mới

Gọi tập train là $D_{\text{tr}} = \{(x_i, y_i)\}_{i=1}^{n}$ và mô hình sau khi học từ tập đó là $h_{D_{\text{tr}}}$. Với hàm mất mát ℓ, sai số thực nghiệm trên train là

$$\widehat{R}_{\text{tr}}(h) = \frac{1}{n} \sum_{i=1}^{n} \ell(h(x_i), y_i).$$

Sai số kỳ vọng trên dữ liệu đích được viết

$$R(h) = \mathbb{E}_{(X,Y)\sim P_{\text{target}}} \left[\ell(h(X), Y) \right].$$

Ở đây P_{target} mô tả dữ liệu mà ta thực sự muốn mô hình phục vụ. Trong lý tưởng, train và test độc lập và lấy từ cùng phân phối đích. Độ chênh $R(h) - \widehat{R}_{\text{tr}}(h)$ là một cách đo khoảng cách tổng quát hoá. Ta thường không biết chính xác $R(h)$, nên dùng tập test phù hợp để ước lượng nó sau khi đã hoàn tất chọn mô hình.

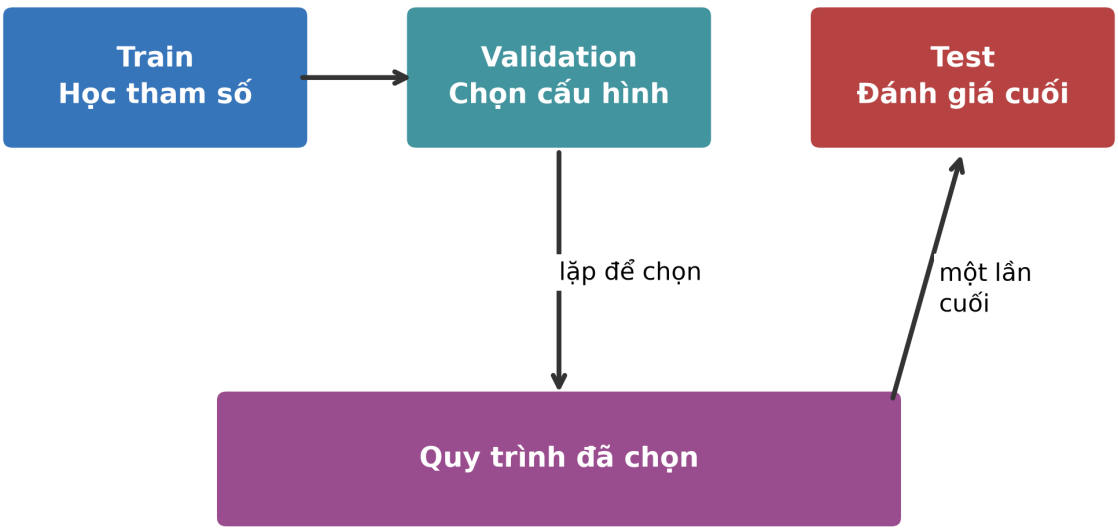
Overfitting thường biểu hiện qua loss train thấp nhưng loss validation cao hơn đáng kể, đặc biệt khi train tiếp làm loss train giảm mà loss validation tăng. Đây là **dấu hiệu cần điều tra**, không phải công thức định nghĩa duy nhất. Loss validation cao còn có thể do hai tập khác phân phối, cỡ mẫu nhỏ, đánh giá sai chế độ, nhãn khác chất lượng hoặc rò rỉ. Ngược lại, khoảng cách nhỏ không bảo đảm kết quả tốt: cả hai loss đều cao có thể là underfitting.

Underfitting thường xuất hiện khi cả train lẫn validation đều chưa đạt mức mong muốn. Nguyên nhân có thể là họ mô hình quá hạn chế, thiếu thông tin ở đầu vào, huấn luyện chưa đủ, tối ưu thất bại, hoặc ràng buộc quá mạnh. Trước khi tăng kích thước mô hình, hãy kiểm cách tính loss, nhãn và xem thuật toán có thực sự giảm được loss train hay không.

Ví dụ so sánh (tự tính), với MSE càng thấp càng tốt: mô hình A có train bằng 4 và validation bằng 5; mô hình B có train bằng $0{,}1$ và validation bằng 8. B khớp train hơn nhưng A dự đoán validation tốt hơn. Nếu mức loss chấp nhận được là khoảng 1, cả A lẫn B đều chưa đáp ứng yêu cầu; khoảng cách nhỏ của A không biến nó thành mô hình tốt. Ngưỡng chấp nhận phải đến từ bài toán và đơn vị của loss, không tự suy ra từ các số này.

5. Train, validation và test: ai được nhìn dữ liệu nào?

Ba tập đóng các vai trò khác nhau trong một quy trình ra quyết định. Điều cần giữ là ranh giới thông tin, không phải một tỉ lệ chia cố định cho mọi bài toán.



Hình 10: Vai trò của tập train, validation và test trong quy trình học (minh hoạ, tự tính)

Cách đọc hình: train cung cấp dữ liệu để học tham số, validation giúp so sánh cấu hình và chọn cách dừng. Sau các vòng thử và chọn, ta cố định quy trình rồi dùng test để đánh giá cuối. Mũi tên quay về tập test chỉ đi sau khi đã chọn: nếu xem test rồi đổi mô hình theo kết quả đó, test đã tham gia lựa chọn và không còn là phép đánh giá độc lập như dự định.

- **Train:** dùng để khớp trọng số, cũng như ước lượng các bước xử lý được học từ dữ liệu, như trung bình và độ lệch chuẩn cho chuẩn hoá.
- **Validation:** dùng để chọn đặc trưng, kiến trúc, mức regularization, ngưỡng quyết định hoặc checkpoint. Nó có thể được xem nhiều lần, nên quyết định cuối đã phụ thuộc vào validation.
- **Test:** dùng để báo cáo đánh giá cuối của quy trình đã chốt. Không dùng nhãn test để chọn cấu hình. Nếu sau khi xem kết quả ta tiếp tục sửa, cần một tập đánh giá mới độc lập cho nhận định cuối sau sửa.

Không có tỉ lệ train/validation/test nào tối ưu trong mọi bài. Test phải đủ lớn và đủ đại diện để ước lượng thước

đo quan trọng; validation cũng phải đủ để so sánh; phần còn lại dành cho học. Với dữ liệu ít, cross-validation có thể tận dụng tốt hơn dữ liệu cho lựa chọn, nhưng vẫn cần thiết kế đánh giá cuối tách biệt. Với dữ liệu rất lớn hoặc hiếm biến cố, cỡ từng tập và số ví dụ thuộc nhóm khó thường có ý nghĩa hơn một phần trăm mặc định.

6. Ví dụ tay: chọn mô hình mà không dùng test để chọn

Giả sử hai mô hình A và B đã được huấn luyện trên cùng hai mẫu, rồi dự đoán trên hai mẫu validation. Ta dùng MSE và ghi đầy đủ các giá trị trong ví dụ (tự tính):

Tập	Giá trị thật	Dự đoán A	Dự đoán B
Train	$(1, 3)$	$(1, 2)$	$(1, 3)$
Validation	$(2, 2)$	$(1, 1)$	$(0, 3)$

Bước 1. Trên train, A có bình phương sai số $(0, 1)$, nên MSE bằng $(0 + 1)/2 = 0{,}5$. B dự đoán đúng cả hai mẫu, nên MSE train bằng 0.

Bước 2. Trên validation, A có bình phương sai số $(1, 1)$, nên MSE bằng $(1 + 1)/2 = 1$. B có bình phương sai số $(4, 1)$, nên MSE bằng $(4 + 1)/2 = 2{,}5$. Nếu hai mô hình là các ứng viên duy nhất và ta chỉ chọn theo MSE validation này, ta chọn A, dù B khớp train hoàn hảo.

Bước 3. Khoá quyết định chọn A. Giả sử hai nhãn test là $(2, 4)$ và A dự đoán $(2, 3)$. Bình phương sai số là $(0, 1)$, nên MSE test của A bằng $0{,}5$. Không so A với dự đoán test của B rồi đổi lựa chọn; nếu làm vậy, nhãn test đã trở thành một phần của phép chọn (tự tính).

Cỡ mỗi tập trong ví dụ chỉ có hai mẫu để dễ tính tay. Do đó không nên rút ra rằng sai số thực của A chính xác bằng $0{,}5$. Trên dữ liệu thật ta cần tập đủ lớn, thước đo phù hợp, kiểm mức bất định và kiểm tính đại diện. Chênh lệch nhỏ trên một tập validation hữu hạn cũng có thể do may rủi.

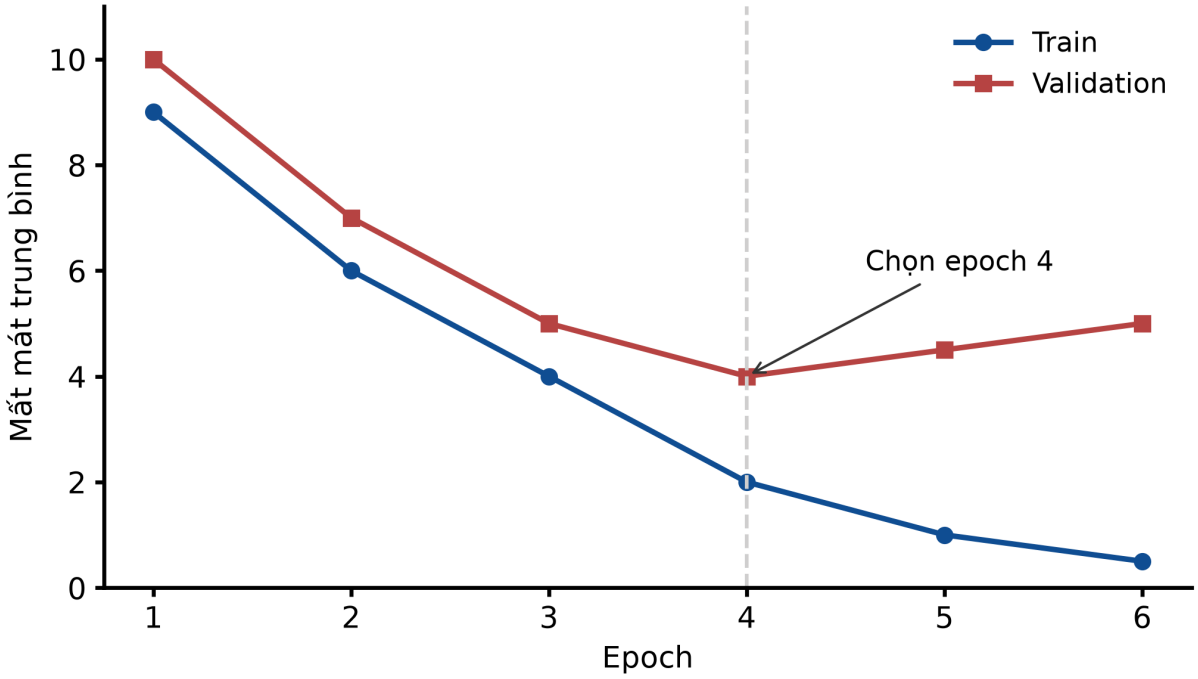
7. Theo dõi quá trình học thay vì chỉ nhìn điểm cuối

Một **epoch** là một lượt đi qua tập train theo cách được định nghĩa trong quy trình huấn luyện. Sau mỗi epoch, ta có thể lưu mô hình và tính loss trên train, validation. Với loss càng thấp càng tốt, chọn checkpoint có validation loss thấp nhất là một cách ra quyết định thường dùng.

Cách đọc hình: đường xanh tiếp tục giảm từ epoch 1 đến 6, còn đường đỏ đạt giá trị thấp nhất ở epoch 4 rồi tăng. Đường chấm đánh dấu checkpoint được chọn theo validation, không phải điểm train thấp nhất. Các giá trị được dựng để minh hoạ và không dự báo rằng mọi đường học đều trơn hoặc luôn đi theo hình này.

Ở đây chuỗi loss validation là $(10, 7, 5, 4, 4{,}5, 5)$ và chuỗi train là $(9, 6, 4, 2, 1, 0{,}5)$. Giá trị nhỏ nhất của validation là 4 ở epoch 4, còn train ở đó bằng 2 (tự tính). Sau epoch 4, loss train vẫn giảm nhưng validation tăng, là dấu hiệu phù hợp với khớp quá mức trong ví dụ. Cơ chế dừng sớm và cách lưu checkpoint được trình bày kỹ ở ml basics nền 04.

Cần tính hai đường theo cách so sánh hợp lý. Ví dụ loss train được ghi trong khi dropout và augmentation đang bật, còn validation được tính với dropout tắt và trên ảnh gốc, thì độ chênh có thể một phần do điều kiện đánh giá khác nhau. Nếu muốn so trực tiếp, có thể tính thêm loss trên train sau mỗi epoch bằng cùng chế độ đánh giá như validation, không cập nhật trọng số. Điều đó không thay thế được việc có một tập validation độc lập.



Hình 11: Loss train và validation theo epoch, với epoch được chọn theo validation (minh hoạ, tự tính)

8. Bias và variance: tưởng tượng huấn luyện lại nhiều lần

Cố định một đầu vào x và giả sử ta có thể nhiều lần lấy tập train mới từ cùng một quy trình lấy mẫu, rồi huấn luyện lại cùng một thuật toán. Mỗi tập có thể sinh ra một dự đoán khác nhau ở x. **Bias** tại điểm đó là độ lệch giữa dự đoán trung bình qua các tập train và giá trị trung bình thật cần dự đoán. **Variance** là mức các dự đoán dao động quanh dự đoán trung bình của chính chúng.

Bias không phải sai số của một mô hình cụ thể, còn variance ở đây không phải phương sai của nhãn trong một tập cụ thể. Chúng nói về hành vi của **quy trình học** khi dữ liệu train thay đổi. Một thuật toán có thể ổn định nhưng lệch có hệ thống; thuật toán khác có thể trung bình đúng mà từng lần dự đoán phân tán mạnh. Dùng một tập train duy nhất không đủ để đo trực tiếp cả hai đại lượng này.

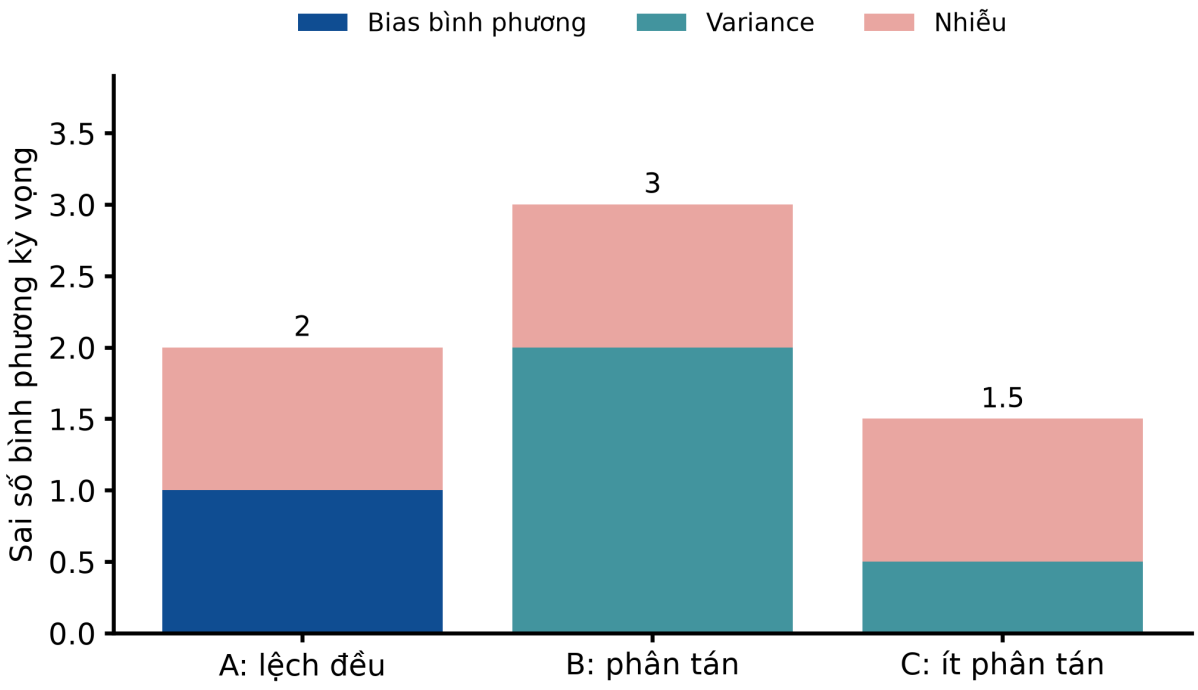
Hãy xét quy luật trung bình tại một điểm là $f(x) = 2$. Ta giả định mỗi trong bốn tập train có xác suất bằng nhau và tạo ra một trong bốn dự đoán ở bảng dưới. Các số là mô hình đồ chơi để tính tay, không phải số liệu thực nghiệm (tự tính):

Thuật toán	Bốn dự đoán	Trung bình	Bias bình phương	Variance
A	$(1, 1, 1, 1)$	1	1	0
B	$(0, 2, 2, 4)$	2	0	2
C	$(1, 2, 2, 3)$	2	0	$0{,}5$

Với A, trung bình bằng 1, nên bias bằng $1 - 2 = -1$, bình phương bằng 1; mọi dự đoán trùng nhau nên variance bằng 0. Với B, trung bình bằng $(0 + 2 + 2 + 4)/4 = 2$; bình phương độ lệch so với trung bình là $(4, 0, 0, 4)$ nên variance bằng $8/4 = 2$. Với C, trung bình cũng bằng 2, còn các bình phương độ lệch là $(1, 0, 0, 1)$ nên variance bằng $2/4 = 0{,}5$ (tự tính).

Giả sử thêm nhiễu nhãn độc lập có trung bình bằng 0 và variance bằng 1 ở điểm đang xét. Khi đó sai số bình phương kỳ vọng của A, B, C lần lượt bằng 2, 3, $1{,}5$ (tự tính). C tốt nhất trong ba thuật toán giả định này. B

có bias bằng không nhưng tổng sai số lớn hơn A: tối thiểu hoá một thành phần riêng lẻ chưa đủ.



Hình 12: Phân rã sai số kỳ vọng của ba thuật toán giả định tại một điểm (minh hoạ, tự tính)

Cách đọc hình: mỗi cột là tổng của bias bình phương, variance dự đoán và variance nhiễu. Cả ba cột có cùng phần nhiễu vì đang xét cùng một điểm và cùng cơ chế sinh nhãn. Các số trên cột là tổng 2, 3 và 1,5; so sánh chiều cao cả cột thay vì chỉ nhìn một màu.

9. Công thức phân rã và điều kiện áp dụng

Giả sử $Y = f(x) + \varepsilon$, với $f(x) = \mathbb{E}[Y \mid X = x]$, $\mathbb{E}[\varepsilon \mid X = x] = 0$ và $\mathrm{Var}(\varepsilon \mid X = x) = \sigma^2(x)$. Gọi $\hat{f}_D(x)$ là dự đoán từ tập huấn luyện ngẫu nhiên D, và $\bar{f}(x) = \mathbb{E}_D[\hat{f}_D(x)]$. Với mẫu đánh giá mới độc lập với tập train tại điểm đang xét, ta có

$$\mathbb{E}_{D,\varepsilon}\left[(Y - \hat{f}_D(x))^2\right] = \underbrace{(\bar{f}(x) - f(x))^2}_{\text{bias bình phương}} + \underbrace{\mathbb{E}_D[(\hat{f}_D(x) - \bar{f}(x))^2]}_{\text{variance}} + \underbrace{\sigma^2(x)}_{\text{nhiễu}}.$$

Vì sao ba phần cộng được? Viết sai số thành $f(x) - \bar{f}(x) + \bar{f}(x) - \hat{f}_D(x) + \varepsilon$ rồi bình phương. Hạng đầu là hằng số khi lấy kỳ vọng theo tập train và nhiễu; hai phần sau có trung bình bằng không. Với giả định độc lập và trung bình nhiễu bằng không, các tích chéo có kỳ vọng bằng không, để lại đúng ba bình phương ở trên.

Công thức chính xác này dành cho **hồi quy với loss bình phương** và kỳ vọng theo các nguồn ngẫu nhiên nêu rõ. Với phân loại, cross-entropy hay các loss khác, không được tự mang nguyên công thức này sang mà không định nghĩa phép phân rã tương ứng. Khi xét trung bình trên nhiều đầu vào, lấy tiếp kỳ vọng theo phân phối của X. Nếu phân phối đích thay đổi, các đại lượng và kết luận cũng có thể thay đổi.

Từ “nhiễu không thể giảm” cần hiểu có điều kiện: nếu chỉ được dùng đầu vào hiện tại và quy luật sinh dữ liệu đã cho, một bộ dự đoán điểm không loại bỏ được phần ngẫu nhiên còn lại của Y. Thêm thông tin đầu vào phù hợp hoặc cải thiện phép đo có thể làm phần bất định có điều kiện nhỏ đi. Nhiễu không phải lý do để bỏ qua nhãn sai có thể sửa.

10. Bias-variance là một cách nhìn, không phải luật chữ U

Trong một số họ mô hình cổ điển, tăng độ linh hoạt làm bias giảm và variance tăng, nên đường sai số test có dạng chữ U: lúc đầu mô hình học tốt hơn, sau đó sự nhạy với mẫu lấn át lợi ích. Đó là trực giác có ích để bắt đầu và giải thích hình ba đường khớp dữ liệu, nhưng không bảo đảm xảy ra ở mọi bài toán.

Mạng lớn có thể khớp dữ liệu train rất tốt mà vẫn dự đoán dữ liệu mới tốt trong một số điều kiện. Khi thay đổi kích thước mô hình, số bước huấn luyện hoặc dữ liệu, có thể quan sát các dạng phức tạp hơn, trong đó có **double descent**: sai số giảm, tăng gần vùng nội suy, rồi lại giảm trong một số thí nghiệm. Hãy đọc ml basics 02 và ml basics 03 để xem bối cảnh cụ thể. Không suy ra rằng mô hình càng lớn luôn tốt, hay double descent xuất hiện trên mọi bộ dữ liệu.

Thêm dữ liệu thường giúp giảm bất định do mẫu, nhưng chỉ khi dữ liệu bổ sung có chất lượng và phù hợp với mục tiêu. Tăng dữ liệu train không sửa được định nghĩa nhãn sai, leakage hoặc việc test thuộc một môi trường rất khác. Tương tự, regularization có thể thay đổi cả bias lẫn variance; không có quy tắc rằng một kỹ thuật luôn cải thiện mọi bài toán.

11. Chia dữ liệu thực tế và những chi tiết dễ bỏ qua

11.1 Tách theo đơn vị độc lập và thời gian

Nếu một bệnh nhân có nhiều ảnh, đưa ảnh của cùng bệnh nhân vào cả train và test có thể làm điểm test quá lạc quan khi mục tiêu là dự đoán cho bệnh nhân mới. Hãy chia theo bệnh nhân trước, rồi tạo các mẫu hoặc biến thể bên trong từng tập. Cùng ý tưởng áp dụng cho người dùng, tài liệu, sản phẩm hoặc nguồn dữ liệu khi đơn vị triển khai mới là các nhóm này.

Nếu dự đoán tương lai từ quá khứ, chia ngẫu nhiên các thời điểm có thể cho mô hình biết thông tin đến sau thời điểm dự đoán. Khi đó hãy dùng train ở quá khứ, validation ở giai đoạn sau và test ở giai đoạn sau nữa, bảo đảm mọi đặc trưng đều thực sự có sẵn tại thời điểm dự đoán. Chia theo thời gian còn giúp nhìn thấy thay đổi phân phối, dù không thể bảo đảm tương lai giống đúng giai đoạn test.

11.2 Leakage trong xử lý và tạo đặc trưng

Data leakage là thông tin không hợp lệ tại thời điểm dự đoán hoặc thông tin từ tập đánh giá lọt vào quá trình xây dựng mô hình, khiến đánh giá không phản ánh cách dùng dự kiến. Chuẩn hoá, chọn đặc trưng, điền giá trị thiếu hoặc học embedding trên toàn bộ tập trước khi chia có thể truyền thông tin từ validation/test sang train. Hãy học các bước phụ thuộc dữ liệu chỉ trên phần train của mỗi lần chia, sau đó áp cùng phép biến đổi lên validation và test.

Ví dụ (tự tính): train có hai giá trị 1 và 3, trung bình train là 2, độ lệch chuẩn theo trung bình bình phương là 1. Một điểm validation có giá trị 10 được biến đổi bằng hai thống kê train thành $(10-2)/1=8$. Không tính lại trung bình bằng cách gộp điểm validation với train. Quy tắc tương tự áp dụng trong từng fold cross-validation; một số phép biến đổi không cần học thống kê từ dữ liệu thì có thể định nghĩa trước, miễn không dùng thông tin không sẵn có khi triển khai.

11.3 Cross-validation và vòng chọn lồng nhau

Trong k-fold cross-validation, ta chia phần dữ liệu phát triển thành k phần. Ở mỗi lượt, giữ một phần làm validation, huấn luyện trên các phần còn lại, rồi tính điểm ở phần được giữ. Mỗi phần lần lượt đóng vai trò validation. Trung bình các điểm giúp so sánh cấu hình bớt phụ thuộc vào một cách chia, dù các điểm giữa các fold không độc lập hoàn toàn vì các tập train chồng lên nhau.

Sau khi chọn cấu hình bằng cross-validation, có thể huấn luyện lại trên toàn bộ dữ liệu phát triển và đánh giá một lần trên test độc lập. Nếu cần ước lượng cả quy trình chọn khi không có test riêng, dùng cross-validation lồng nhau: vòng trong chọn cấu hình, vòng ngoài giữ dữ liệu chưa dùng ở vòng trong để đánh giá. Với dữ

liệu nhóm hoặc thời gian, cách chia fold cũng phải tôn trọng nhóm hoặc thời gian; không áp dụng chia ngẫu nhiên mặc định một cách máy móc.

11.4 Validation cũng có thể bị học thuộc

Thử quá nhiều cấu hình và chọn cấu hình có điểm validation tốt nhất khiến lựa chọn hưởng lợi từ dao động ngẫu nhiên của tập đó. Người làm thí nghiệm cũng có thể vô tình thích nghi với validation bằng cách liên tục thay thiết kế sau khi nhìn kết quả. Vì vậy test phải được giữ tách biệt, nhật ký thử nghiệm phải rõ ràng, và khi thực sự thay mục tiêu sau test, nên thu thập đánh giá độc lập mới.

Test độc lập không cứu được một tập test sai mục tiêu. Nếu hệ thống sẽ gặp dữ liệu từ bệnh viện mới hoặc thời gian mới, test cùng nguồn cũ có thể không đo được rủi ro đó. Nên báo cáo thêm đánh giá theo nhóm, thời gian hay điều kiện triển khai có ý nghĩa, kèm cỡ mẫu và độ bất định nếu có.

12. Nối sang AI

Khả năng ghi nhớ của mạng sâu. Ml basics 01 đặt câu hỏi vì sao mạng đủ sức khớp cả các nhãn không có quy luật vẫn có thể tổng quát hoá trên các bài toán phù hợp. Bài học cho người mới là loss train thấp chỉ cho biết khớp dữ liệu huấn luyện; nó không tự chứng minh quy luật hữu ích đã được học.

Độ phức tạp và double descent. Ml basics 02 và ml basics 03 mở rộng bức tranh chữ U. Chúng giúp đặt câu hỏi độ phức tạp đang thay đổi theo chiều nào, và đánh giá được thực hiện trên phân phối nào, thay vì xếp mọi mô hình lớn vào cùng một kết luận.

Dữ liệu lặp và leakage. Với bộ dữ liệu văn bản, ảnh hay bệnh án, các bản sao gần giống và cùng nguồn có thể làm ranh giới train/test tưởng tách mà thực ra không tách được trường hợp cần dự đoán mới. Ml basics 21 bàn sâu về các dạng rò rỉ và tái lập đánh giá.

Chọn mô hình có phương pháp. Ml basics 34 mô tả nhiều cách cross-validation. Cần chọn cách chia phản ánh mục tiêu, chứ không chỉ chạy một công cụ chia fold rồi coi mọi điểm số đều có cùng ý nghĩa.

Can thiệp khi có dấu hiệu khớp quá mức. Có thể xem xét thêm dữ liệu phù hợp, regularization, giảm khả năng khớp hoặc dừng sớm; nhưng trước đó hãy kiểm khả năng leakage, phân phối và điều kiện đánh giá. Ml basics nền 04 giải thích L1, L2, dropout, augmentation và early stopping bằng các ví dụ nhỏ.

13. Sai lầm hay gặp

- **Chọn theo train loss duy nhất.** Train loss đo trên dữ liệu đã dùng để học, nên thường lạc quan hơn đối với dữ liệu chưa thấy. Hãy dùng validation để lựa chọn và test độc lập để báo cáo cuối.
- **Coi train thấp, validation cao là bằng chứng duy nhất của overfitting.** Cần kiểm cả lỗi nhãn, lệch phân phối, kích thước tập và sự khác biệt giữa chế độ train với chế độ đánh giá.
- **Dùng test như bảng xếp hạng khi chỉnh liên tục.** Một khi dựa vào test để sửa cấu hình, kết quả test đó không còn đánh giá độc lập quy trình sau sửa.
- **Làm sạch và chuẩn hoá trên toàn bộ dữ liệu trước khi chia.** Các thống kê được học từ tập đánh giá có thể rò rỉ; hãy fit phép xử lý trong train hoặc trong từng fold train.
- **Nghĩ bias và variance là thuộc tính cố định của một mô hình đã fit.** Phân rã ở mục 9 nói về cả quy trình khi lấy lại tập train, tại một phân phối và một loss xác định.
- **Tin rằng đồ thị chữ U, thêm dữ liệu, hoặc tăng mô hình luôn cho cùng kết quả.** Đây là giả thuyết cần đo trong bài toán cụ thể; còn phân phối triển khai và cách tính thước đo đều phải được nói rõ.

14. Tóm tắt công thức và vai trò

Khái niệm	Công thức hoặc vai trò	Cách hiểu
MSE trên mẫu	$\frac{1}{n}\sum_i(\hat{y}_i - y_i)^2$	trung bình sai số bình phương của các mẫu đã xét
Rủi ro đích	$R(h) = \mathbb{E}[\ell(h(X), Y)]$	loss kỳ vọng trên phân phối đích
Khoảng cách tổng quát hoá	$R(h) - \widehat{R}_{\text{tr}}(h)$	chênh rủi ro đích và loss train
Bias ở x	$\bar{f}(x) - f(x)$	độ lệch của dự đoán trung bình
Variance ở x	$\mathbb{E}_D[(\hat{f}_D(x) - \bar{f}(x))^2]$	dao động qua các tập train
Phân rã MSE	MSE $=$ bias^2 $+$ variance $+ \sigma^2$	đúng dưới giả định ở mục 9
Train, validation, test	học, chọn, đánh giá cuối	ba vai trò thông tin khác nhau

15. Bài tập tự tính

Năm bài đi từ tính loss đến hiểu quy trình. Đáp án ngay dưới mỗi bài, các số đều đã kiểm bằng NumPy.

Bài 1 (dễ). Giá trị thật là $(2, 4, 6)$ và dự đoán là $(1, 4, 8)$. Tính từng sai số bình phương và MSE.

Đáp án. Sai số dự đoán trừ giá trị thật là $(-1, 0, 2)$, bình phương thành $(1, 0, 4)$. MSE bằng $(1+0+4)/3 = 5/3$ (tự tính). Đơn vị của MSE là bình phương đơn vị của nhãn.

Bài 2 (dễ vừa). A có train MSE bằng 1 và validation MSE bằng 2. B có train MSE bằng 0 và validation MSE bằng 5. Chọn theo validation thì chọn gì? Vì sao chưa thể biết sai số thực chính xác?

Đáp án. Chọn A vì $2 < 5$, nếu hai ứng viên được so bằng cùng validation và MSE là tiêu chí. A có khoảng cách validation trừ train bằng 1, B bằng 5 (tự tính). Validation chỉ là một mẫu hữu hạn; điểm của nó có thể dao động và không bảo đảm đúng bằng rủi ro trên phân phối đích.

Bài 3 (vừa). Tại điểm có $f(x) = 3$, bốn tập train đồng xác suất cho dự đoán $(1, 3, 3, 5)$. Tính trung bình, bias bình phương và variance của dự đoán. Nếu variance nhiễu bằng 2 và các giả định mục 9 đúng, tính MSE kỳ vọng.

Đáp án. Trung bình là $(1 + 3 + 3 + 5)/4 = 3$, bias bình phương là $(3 - 3)^2 = 0$. Bình phương lệch khỏi trung bình là $(4, 0, 0, 4)$ nên variance bằng $8/4 = 2$. MSE kỳ vọng bằng $0 + 2 + 2 = 4$ (tự tính).

Bài 4 (vừa). Loss validation ở năm checkpoint lần lượt là $(4, 3, 2, 3, 4)$. Chọn checkpoint nào theo quy tắc loss nhỏ nhất? Nếu dự đoán test của checkpoint được chọn là $(2, 5)$ và nhãn là $(1, 5)$, MSE test bằng bao nhiêu?

Đáp án. Chọn checkpoint thứ ba vì loss nhỏ nhất bằng 2. Bình phương sai số test là $(1, 0)$, nên MSE test bằng $1/2 = 0{,}5$ (tự tính). Không dùng điểm test này để quay lại chọn một checkpoint khác.

Bài 5 (khó hơn). Tập train có giá trị đặc trưng $(2, 4, 6)$, tập validation có giá trị 10. Dùng trung bình và độ lệch chuẩn quần thể của train để chuẩn hoá validation. Nếu trước khi chia ta lại dùng cả bốn giá trị để fit chuẩn hoá, thông tin nào đã đi sai ranh giới?

Đáp án. Trung bình train là 4, variance train là $((2-4)^2 + (4-4)^2 + (6-4)^2)/3 = 8/3$, nên độ lệch chuẩn là $\sqrt{8/3}$. Giá trị validation sau chuẩn hoá là $(10 - 4)/\sqrt{8/3} = 3\sqrt{6}/2 \approx 3{,}674$ (tự tính). Fit bằng cả bốn điểm làm thống kê chuẩn hoá phụ thuộc vào validation: thông tin từ phần dùng để chọn đã chảy vào bước học trên train. Có thể áp phép biến đổi train đã fit cho validation, nhưng không fit lại trên validation.

Tài liệu tham khảo nền: Hastie, Tibshirani & Friedman, The Elements of Statistical Learning; Bishop, Pattern Recognition and Machine Learning; Zhang et al. 2016 (ml basics 01); Belkin et al. 2018 (ml basics 02); Arlot & Celisse 2009 (ml basics 34).

Nền tảng 04

Regularization: hạn chế học thuộc để cải thiện dự đoán trên dữ liệu mới

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: loss, gradient, train/validation/test và overfitting; xem ml basics nền 01, 02, 03

Dùng ở: ml basics 01, ml basics 06, ml basics 29, ml basics 31, ml basics 37

> Chương này giải thích nền tảng từ đầu, không phải bản dịch paper. Các phép tính ghi "tự tính" đã được kiểm bằng NumPy; ví dụ và hình là dữ liệu minh hoạ.

1. Vì sao cần regularization

Regularization là cách đưa thêm giả định hoặc ràng buộc vào quá trình học nhằm giúp mô hình tổng quát hoá tốt hơn. Nó có thể phạt trọng số lớn, thay đổi thời điểm dừng, ngẫu nhiên bỏ bớt đơn vị ẩn, hoặc tạo thêm biến thể có ý nghĩa của dữ liệu. Mục đích thực dụng là làm mô hình bớt phụ thuộc vào những chi tiết tình cờ của tập train, nhưng không có kỹ thuật nào bảo đảm cải thiện trên mọi bài toán.

Ở chương trước, một đường quá ngoằn ngoèo có thể khớp chính xác các điểm train mà dự đoán kém ở điểm mới. Giới hạn độ ngoằn ngoèo là một giả định: ta tin rằng quy luật đích không thay đổi quá mạnh giữa các ví dụ tương tự. Nếu giả định đó phù hợp, sự hạn chế có thể giúp. Nếu quy luật thực sự có biến động sắc nét mà dữ liệu phản ánh đúng, ràng buộc quá mạnh lại có thể làm mô hình bỏ sót quy luật.

Chương này đi qua năm cách thường gặp. L1 và L2 thêm số hạng phạt vào hàm mục tiêu. Early stopping chọn thời điểm kết thúc quá trình tối ưu. Dropout làm thay đổi mạng trong lúc huấn luyện bằng mặt nạ ngẫu nhiên. Data augmentation tạo đầu vào mới từ dữ liệu train theo những phép biến đổi được chọn. Cách chọn cường độ của từng kỹ thuật phải dựa vào validation và mục tiêu triển khai.

2. Nhắc nhanh nền tảng

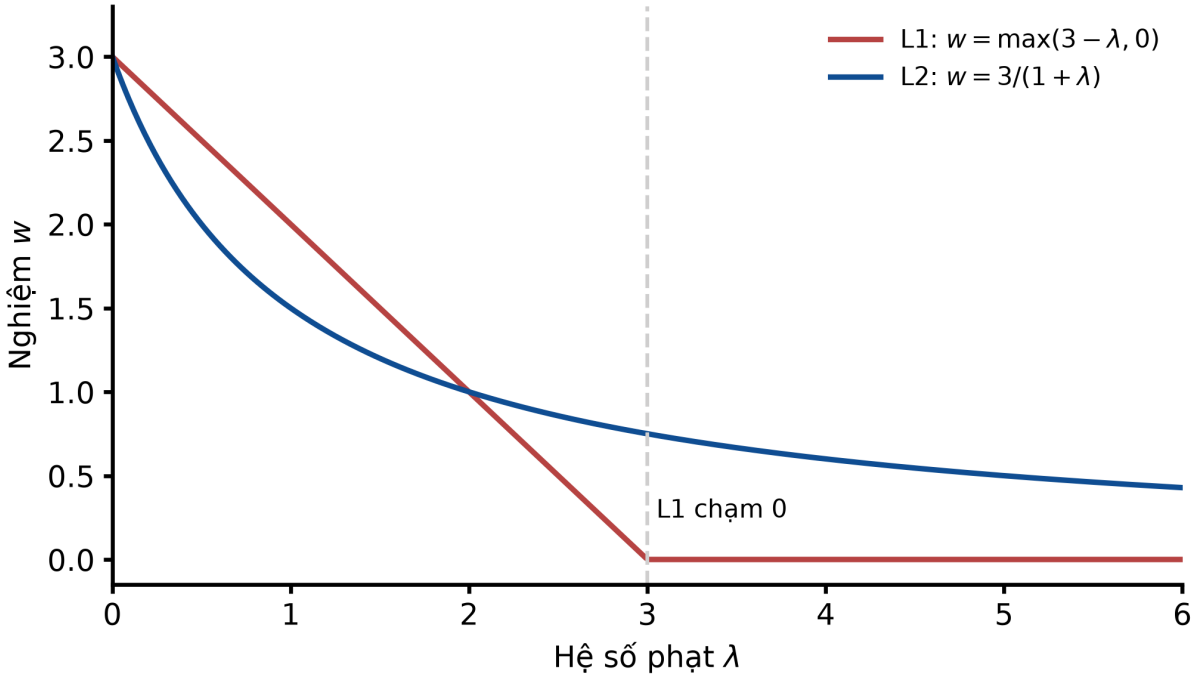
Một mô hình dự đoán $\hat{y} = h_w(x)$ bằng tham số w. Thuật toán huấn luyện thường tìm tham số có **loss train** nhỏ. Với ba nhãn $(1, 2, 3)$ và dự đoán $(1, 1, 5)$, các sai số bình phương là $(0, 1, 4)$, nên MSE train bằng $(0 + 1 + 4)/3 = 5/3$ (tự tính). MSE chỉ đo trung bình trên các ví dụ đó; validation và test giúp kiểm tra trên các ví dụ không dùng để học tham số.

Đạo hàm cho biết một hàm thay đổi như thế nào khi ta thay đổi đầu vào. Với $L(w) = \frac{1}{2}(w - 3)^2$, ta có $L'(w) = w - 3$. Gradient descent với learning rate η cập nhật $w_{t+1} = w_t - \eta L'(w_t)$. Nếu $w_0 = 0$ và $\eta = 0{,}5$ thì gradient đầu là -3 và $w_1 = 0 - 0{,}5(-3) = 1{,}5$ (tự tính). Đi ngược gradient ở ví dụ này đưa w về gần nghiệm 3.

Mức phạt, xác suất dropout, số epoch hoặc độ mạnh của augmentation là các lựa chọn của quy trình, thường được gọi là **siêu tham số**. Giá trị tốt phụ thuộc cách định nghĩa loss, kích thước dữ liệu, kiến trúc và phân phối đích. So sánh các giá trị đó trên cùng validation, sau đó giữ test độc lập để đánh giá cuối như ở ml basics nền 03.

3. Trực giác: phạt để cân bằng hai mục tiêu

Hãy tưởng tượng ta chọn một tham số w sao cho dữ liệu train thích w gần 3. Đồng thời ta có một lý do để ưu tiên w gần 0. Mức phạt quyết định ta đánh đổi hai mong muốn này mạnh đến đâu. L1 đo độ lớn bằng trị tuyệt đối, L2 đo bằng bình phương.



Hình 13: Nghiệm L1 và L2 thay đổi khi tăng hệ số phạt cho ví dụ một tham số (minh hoạ, tự tính)

Cách đọc hình: trục ngang là hệ số phạt, trục dọc là giá trị w tối ưu của bài toán được định nghĩa ở mục 4. Cả hai đường bắt đầu ở 3 khi không phạt; trong ví dụ này đường L1 chạm đúng 0 tại hệ số phạt 3, còn L2 tiến gần 0 mà không chạm trong miền hệ số hữu hạn được vẽ. Đây là tính chất của ví dụ và mục tiêu có quy ước nêu bên dưới, không phải công thức cho mọi bài toán.

Đường đi của nghiệm hữu ích vì nó phân biệt hai câu hỏi. Một là loss train thay đổi thế nào khi áp phạt; hai là loss validation thay đổi thế nào. Tăng phạt thường làm nghiệm khác đi và có thể làm loss train gốc tăng, nhưng lợi ích tổng quát hoá cần được đo trên validation, không thể đọc trực tiếp chỉ từ hình trọng số.

4. L1 và L2: định nghĩa và ví dụ tính tay

4.1 Mục tiêu có số hạng phạt

Gọi $L_{\text{data}}(w)$ là loss trung bình trên train và $w = (w_1, \ldots, w_d)$ là các trọng số cần phạt. Một quy ước thông dụng là

$$J_{\text{L1}}(w) = L_{\text{data}}(w) + \lambda \sum_{j=1}^{d} |w_j|, \qquad J_{\text{L2}}(w) = L_{\text{data}}(w) + \frac{\lambda}{2} \sum_{j=1}^{d} w_j^2,$$

với $\lambda \geq 0$. Số λ điều khiển cường độ phạt: bằng 0 là bỏ số hạng phạt, lớn hơn sẽ coi trọng việc thu nhỏ trọng số hơn, theo thang loss đã chọn. Trong tài liệu khác, hệ số $\frac{1}{2}$ hoặc cách chia theo số mẫu có thể đặt ở chỗ khác; phải đọc đúng công thức trước khi so sánh giá trị λ.

Ở đây $\sum_j |w_j|$ là chuẩn L1, còn $\sum_j w_j^2$ là **bình phương** chuẩn L2. Nói tắt “phạt L2” thường chỉ số hạng bình phương này. Tham số chặn bias thường được bỏ khỏi phạt trong hồi quy, nhưng phải kiểm quy ước của mô hình và thư viện cụ thể; không phải mọi triển khai đều làm giống nhau.

4.2 L2 trên một trọng số

Xét $L_{\text{data}}(w) = \frac{1}{2}(w-3)^2$ và đặt $\lambda = 1$. Đây là một hàm mất mát đơn giản được dựng ra để nhìn rõ tác động của penalty. Khi đó

$$J_{\text{L2}}(w) = \frac{1}{2}(w-3)^2 + \frac{1}{2}w^2.$$

Bước 1. Lấy đạo hàm: $J'_{\text{L2}}(w) = (w-3) + w = 2w - 3$.

Bước 2. Cho đạo hàm bằng không: $2w - 3 = 0$, nên $w = 1{,}5$. Đạo hàm bậc hai bằng $2 > 0$, vì thế đây là cực tiểu duy nhất. Không phạt thì cực tiểu của loss gốc ở $w = 3$.

Bước 3. Tại $w = 1{,}5$, loss dữ liệu bằng $\frac{1}{2}(1{,}5-3)^2 = 1{,}125$ và penalty bằng $\frac{1}{2}(1{,}5)^2 = 1{,}125$. Tổng mục tiêu bằng $2{,}25$. Nếu giữ $w = 3$, loss dữ liệu bằng 0 nhưng penalty bằng $4{,}5$, nên tổng lớn hơn (tự tính). Nghiệm có phạt chấp nhận loss train gốc cao hơn để được trọng số nhỏ hơn.

Với λ bất kỳ không âm và cùng loss gốc, giải $w - 3 + \lambda w = 0$ cho $w_{\text{L2}} = 3/(1+\lambda)$. Với hệ số phạt hữu hạn, nghiệm dương và tiến dần về 0 khi hệ số phạt tăng. Đây là lời giải cho hàm bậc hai riêng ở đây; nghiệm của mạng hay hồi quy nhiều đặc trưng cần được tính theo bài toán tương ứng.

4.3 L1 trên cùng loss

Với $\lambda = 1$, mục tiêu là

$$J_{\text{L1}}(w) = \frac{1}{2}(w-3)^2 + |w|.$$

Bước 1. Xét miền $w > 0$. Ở đây $|w| = w$, nên đạo hàm bằng $w - 3 + 1 = w - 2$. Điểm dừng là $w = 2$, nằm đúng trong miền đang xét.

Bước 2. Trên miền $w < 0$, $|w| = -w$, đạo hàm bằng $w - 4$, luôn âm. Hàm giảm khi đi từ trái sang phải trong miền này, nên không có cực tiểu ở một điểm âm. Ở điểm nối 0, so sánh trực tiếp cũng cho $J(0) = 4{,}5 > J(2) = 2{,}5$.

Bước 3. Tại $w = 2$, loss dữ liệu bằng $\frac{1}{2}(2-3)^2 = 0{,}5$, penalty bằng 2, tổng bằng $2{,}5$ (tự tính). Vậy nghiệm là 2. Không so trực tiếp $2{,}25$ của L2 với $2{,}5$ của L1 để tuyên bố phương pháp nào tổng quát hoá tốt hơn: chúng là hai mục tiêu khác nhau, và chưa có điểm validation nào ở đây.

Nếu thay hệ số phạt bằng $\lambda \geq 0$, nghiệm của ví dụ này là $w_{\text{L1}} = \max(3 - \lambda, 0)$. Các giá trị được vẽ ở hình đường phạt có thể kiểm bằng bảng sau (tự tính):

λ	Nghiệm L1	Nghiệm L2
0	3	3
1	2	$1{,}5$
2	1	1
3	0	$0{,}75$
4	0	$0{,}6$

Với một điểm tối ưu không phạt bất kỳ z, bài toán $\frac{1}{2}(w-z)^2 + \lambda|w|$ có nghiệm **soft threshold**

$$w^* = \text{sign}(z)\max(|z| - \lambda, 0).$$

Ví dụ (tự tính): nếu $z = -3$ và $\lambda = 1$ thì nghiệm là -2; nếu $z = -1$ và $\lambda = 2$ thì nghiệm bằng 0. Ở $w = 0$, hàm trị tuyệt đối có góc nhọn và đạo hàm thông thường không tồn tại. Điều kiện tối ưu dùng **dưới gradient**: đạo hàm của loss trơn tại 0 có thể được cân bằng bởi một số bất kỳ trong đoạn $[-\lambda, \lambda]$. Điều này giúp nghiệm L1 đạt đúng 0, tức tạo nghiệm thưa.

5. Những điều L1, L2 không tự bảo đảm

5.1 Thưa, nhỏ, và tương quan

L1 có xu hướng tạo trọng số đúng bằng không trong nhiều bài toán lồi phù hợp, vì vậy đôi khi được dùng để chọn đặc trưng. Không phải mọi triển khai gradient thông thường đều trả số không chính xác do cách tối ưu và độ chính xác số; thuật toán chuyên dụng như proximal methods có thể tạo số không tường minh. Đặc trưng có trọng số bằng không theo một mô hình đã fit cũng chưa tự chứng minh nó vô ích trong mọi mô hình hoặc trong quan hệ nhân quả.

L2 thường thu nhỏ nhiều trọng số một cách liên tục thay vì đặt chúng bằng không chính xác. Trong bài toán nhiều đặc trưng tương quan, nó thường phân chia ảnh hưởng giữa các đặc trưng đó; L1 có thể chọn một trong các đặc trưng tương tự. Cả hai hành vi phụ thuộc dữ liệu và mô hình. Nếu hai đặc trưng gần trùng nhau, việc một trong chúng bị L1 bỏ không có nghĩa chỉ đặc trưng còn lại có thông tin.

Có thể phối hợp hai hình phạt, thường gọi là elastic net, với hai hệ số không âm. Chọn L1, L2 hay phối hợp dựa vào mục tiêu, sự ổn định và validation. Một mô hình thưa có thể dễ lưu trữ hoặc dễ kiểm tra hơn, nhưng độ thưa không thay thế phép kiểm chất lượng và không tự bảo đảm giải thích đúng cơ chế của dữ liệu.

5.2 Thang đo đặc trưng và bias

Nếu một đặc trưng đo bằng mét rồi được đổi thành centimet, hệ số cần thiết để giữ dự đoán không đổi sẽ thay đổi theo tỉ lệ. Phạt trực tiếp các hệ số trên thang đo không đồng nhất có thể ưu tiên hoặc trừng phạt đặc trưng chỉ vì đơn vị đo. Trong hồi quy tuyến tính, chuẩn hoá các đặc trưng bằng thống kê fit trên train thường giúp penalty có ý nghĩa dễ so sánh hơn.

Ví dụ (tự tính): dự đoán $2x$ không đổi nếu viết $x' = 100x$ và dùng trọng số $0{,}02$ trước x'. Tuy nhiên, bình phương trọng số thay từ 4 thành $0{,}0004$, dù cùng hàm dự đoán. Vì vậy, so sánh mức phạt cần xét cả thang đặc trưng. Thống kê chuẩn hoá phải học từ train rồi áp sang validation và test, tránh leakage.

Bias của lớp là tham số chặn trong phương trình, không phải khái niệm bias trong phân rã bias-variance. Nhiều quy trình không phạt tham số chặn vì nó chỉ dịch mức dự đoán chung, nhưng không nên mặc định mọi thư viện đều loại nó. Kiểm rõ nhóm tham số nào được phạt, đặc biệt khi mạng còn có tham số của các lớp normalization.

5.3 L2 và weight decay

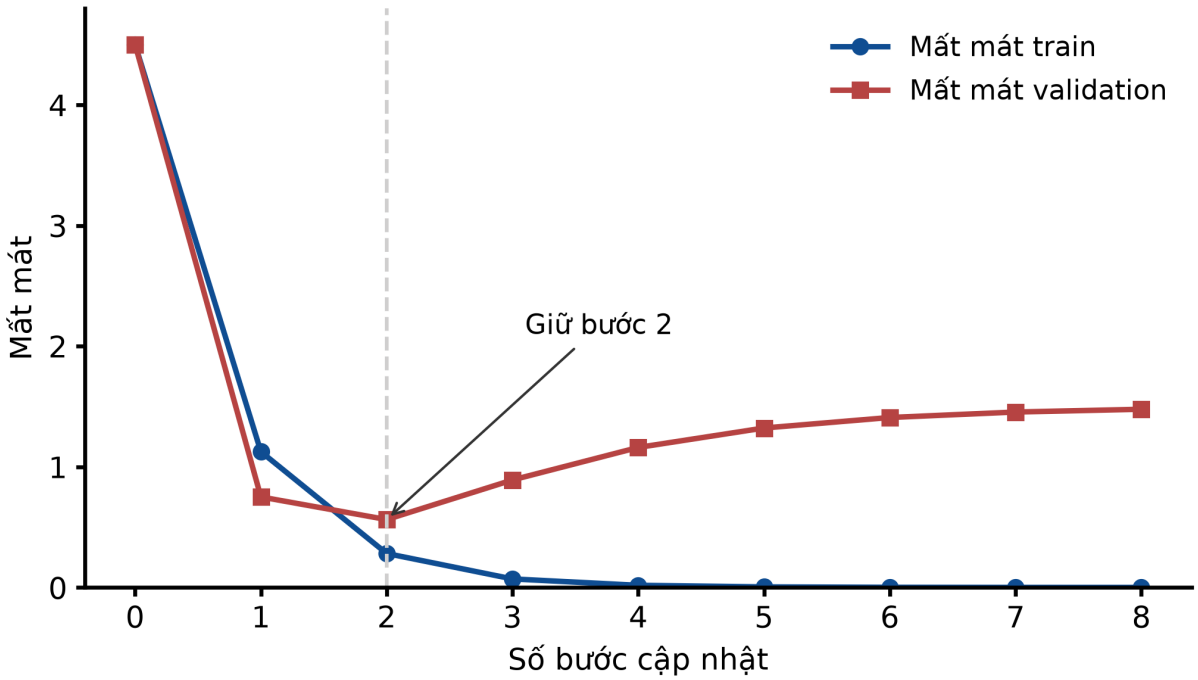
Với gradient descent thông thường, thêm $\frac{\lambda}{2}\|w\|_2^2$ vào loss cho cập nhật

$$w_{t+1} = w_t - \eta\left(\nabla L_{\text{data}}(w_t) + \lambda w_t\right) = (1 - \eta\lambda)w_t - \eta\nabla L_{\text{data}}(w_t).$$

Số hạng $(1 - \eta\lambda)w_t$ trông như co trọng số về không, và với quy ước này nó đúng bằng weight decay kết hợp trong bước gradient descent đơn giản. Với optimizer thích nghi, thêm L2 vào gradient khiến phần phạt đi qua phép chia thích nghi theo từng tọa độ; weight decay tách rời áp phần co riêng. Hai cách không còn nói chung tương đương. Ml basics hoặc optimization về optimizer giúp theo dõi công thức cụ thể; không nên đồng nhất tên tham số trong mọi thư viện.

6. Early stopping: dừng ở checkpoint có validation tốt

Huấn luyện lâu hơn thường tiếp tục giảm loss train, nhưng một checkpoint muộn hơn không nhất thiết dự đoán validation tốt hơn. **Early stopping** theo dõi một chỉ số validation và dừng khi không còn cải thiện trong số lần kiểm đã chọn. Sau đó ta khôi phục checkpoint tốt nhất đã lưu, thay vì mặc định dùng checkpoint cuối cùng.



Hình 14: Early stopping trên ví dụ gradient descent một chiều (minh hoạ, tự tính)

Cách đọc hình: đường xanh là loss train và đường đỏ là một loss validation giả định, cùng tính từ giá trị tham số sau mỗi bước. Loss train tiếp tục giảm, nhưng validation nhỏ nhất tại bước 2 và tăng về sau. Vạch dọc chỉ checkpoint cần lưu theo quy tắc chọn validation nhỏ nhất; hình chỉ minh hoạ một khả năng, không phải mọi quá trình học.

Ta có thể tính chính xác các điểm đầu trong hình. Đặt loss train $L_{\text{tr}}(w) = \frac{1}{2}(w-3)^2$, learning rate $\eta = 0{,}5$, và $w_0 = 0$. Cập nhật là $w_{t+1} = w_t - 0{,}5(w_t - 3)$. Để minh hoạ mục tiêu đánh giá khác, đặt loss validation giả định là $L_{\text{val}}(w) = (w-2)^2 + 0{,}5$. Hai loss ở ví dụ này có hệ số tỉ lệ khác nhau, nên hãy so sự thay đổi của từng đường theo thời gian thay vì diễn giải khoảng cách giữa chúng. Bốn bước cho (tự tính):

Bước t	w_t	Loss train	Loss validation
0	0	4,5	4,5
1	1,5	1,125	0,75
2	2,25	0,28125	0,5625
3	2,625	0,0703125	0,890625
4	2,8125	0,017578125	1,16015625

Bước 1. Từ $w_0 = 0$, gradient train là -3, nên $w_1 = 1{,}5$.

Bước 2. Gradient tại $1{,}5$ là $-1{,}5$, nên $w_2 = 1{,}5 - 0{,}5(-1{,}5) = 2{,}25$.

Bước 3. Validation tại đó bằng $(2{,}25-2)^2+0{,}5=0{,}5625$, nhỏ hơn tại bước 1. Ở bước 3 nó đã tăng lên $0{,}890625$, dù loss train giảm (tự tính). Nếu chỉ các bước trong bảng là ứng viên, quy tắc chọn theo validation giữ w_2.

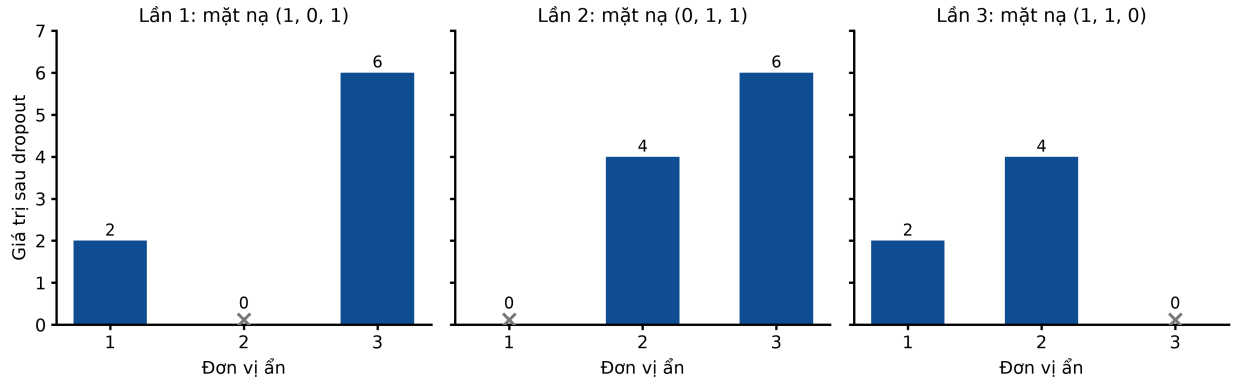
Trong thực tế, validation dao động. **Patience** là số lần kiểm liên tiếp không cải thiện đủ để quy trình chấp nhận tiếp tục chờ trước khi dừng. Chẳng hạn, nếu quy định patience bằng hai lần kiểm và sau điểm tốt nhất có hai lần liên tiếp không tốt hơn theo tiêu chí đặt trước, ta dừng sau lần thứ hai đó rồi khôi phục điểm tốt nhất. Tần suất kiểm, hướng của chỉ số, ngưỡng cải thiện và cách xử lý bằng điểm đều cần ghi rõ.

Early stopping không tự động là một penalty L2 tường minh trong mọi mô hình. Với một số bài toán tuyến tính và thiết lập nhất định, đường đi của nghiệm cho thấy liên hệ định lượng giữa dừng sớm và thu nhỏ hệ số; trong mạng phi tuyến, cách tối ưu và khởi tạo cũng quan trọng. Theo dõi validation quá nhiều lần vẫn có thể khiến lựa chọn thích nghi với validation, nên test cuối cần được giữ riêng.

7. Dropout: thay đổi những đơn vị hoạt động khi huấn luyện

7.1 Mặt nạ ngẫu nhiên

Trong một lớp mạng, gọi $h=(h_1,\dots,h_d)$ là vector đầu ra của các đơn vị ẩn trước dropout. Mỗi lần áp dropout trong lúc train, ta lấy một mặt nạ nhị phân: một số đơn vị được giữ, số còn lại bị đặt bằng 0 cho lượt tính đó. Trọng số không bị xoá vĩnh viễn; ở lượt sau mặt nạ mới có thể giữ một tập khác.



Hình 15: Ba mặt nạ dropout ngược khác nhau áp lên cùng vector ẩn (minh hoạ, tự tính)

Cách đọc hình: vector trước dropout là $(1,2,3)$ và xác suất giữ mỗi đơn vị là $0{,}5$. Dấu nhân màu xám trên vạch không chỉ đơn vị bị bỏ trong lượt đó; cột xanh là giá trị được giữ và nhân đôi. Ba ô là ba mặt nạ có thể xảy ra, không phải ba bước cập nhật bắt buộc và không phải toàn bộ các mặt nạ có thể.

Một trực giác là các đơn vị không thể dựa chắc vào cùng một nhóm đồng hành ở mọi lượt, nên mạng được khuyến khích học dự đoán có ích qua nhiều mặt nạ. Đây là cách hiểu hữu ích, không phải chứng minh rằng dropout luôn ngăn mọi dạng phụ thuộc hoặc luôn cải thiện tổng quát hoá. Xem ml basics 06 để biết bối cảnh của phương pháp.

7.2 Dropout ngược và ví dụ số

Gọi p là xác suất bỏ và $q=1-p$ là xác suất giữ, với $0<q\le 1$. Với **inverted dropout**, các biến mặt nạ m_j nhận 1 với xác suất q và 0 với xác suất p, và trong train ta dùng

$$\tilde{h}_j=\frac{m_j h_j}{q}.$$

Khi đánh giá thông thường, tắt dropout và dùng trực tiếp h_j. Cách chia cho q trong train giữ kỳ vọng của từng

đầu ra lớp bằng giá trị gốc, vì $\mathbb{E}[m_j] = q$ và do đó $\mathbb{E}[\tilde{h}_j] = h_j$. Tính đúng ở từng đơn vị không có nghĩa đầu ra của cả mạng phi tuyến ở chế độ đánh giá luôn bằng kỳ vọng đầu ra của mọi mạng con.

Ví dụ (tự tính): $h = (1, 2, 3)$, $p = 0{,}5$, $q = 0{,}5$, mặt nạ $m = (1, 0, 1)$. Nhân từng phần tử cho $(1, 0, 3)$, rồi chia cho $0{,}5$ được $\tilde{h} = (2, 0, 6)$. Với mặt nạ $(0, 1, 1)$, kết quả là $(0, 4, 6)$; với $(1, 1, 0)$, kết quả là $(2, 4, 0)$. Đây là đúng ba ô của hình dropout.

Nhìn riêng đơn vị có $h_j = 2$: với xác suất $0{,}5$ đầu ra bằng 0, với xác suất $0{,}5$ đầu ra bằng 4. Trung bình là $0{,}5 \cdot 0 + 0{,}5 \cdot 4 = 2$, bằng giá trị trước dropout. Phương sai là $0{,}5(0-2)^2 + 0{,}5(4-2)^2 = 4$ (tự tính). Tổng quát với h_j cố định, variance của dropout ngược là $h_j^2(1-q)/q$.

Kỳ vọng trên là theo mặt nạ tại một đầu ra cố định, không phải trung bình ba ô trong hình dropout; ba ô chỉ là ba mặt nạ được chọn để giải thích. Nếu p quá cao, quá ít tín hiệu được giữ ở từng lượt và việc học có thể khó. Chọn vị trí và xác suất dropout theo validation, không áp một giá trị mặc định cho mọi kiến trúc.

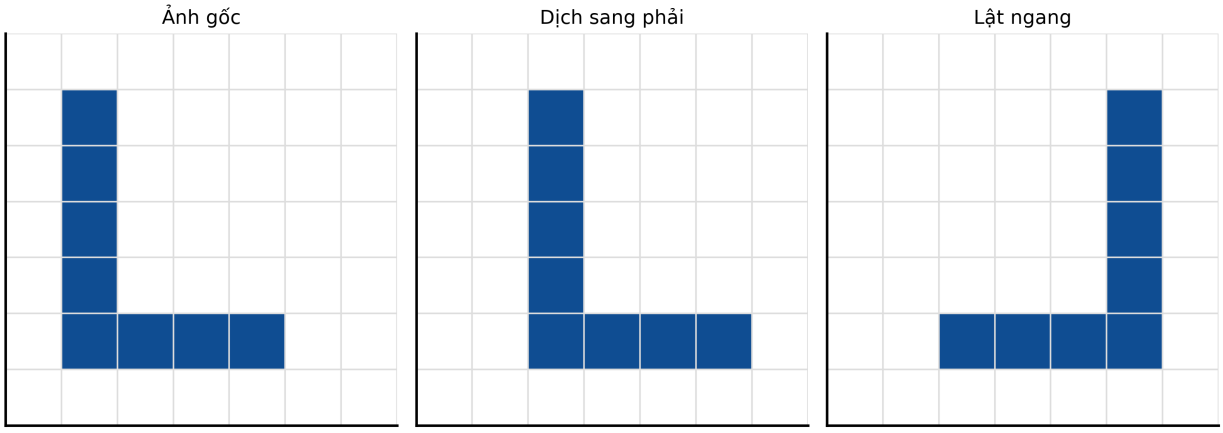
7.3 Train, đánh giá và ngoại lệ có chủ đích

Trong đánh giá thông thường, dropout tắt để kết quả ổn định và khớp quy ước huấn luyện kể trên. Nhớ chuyển mạng sang chế độ đánh giá trước khi tính validation và test; khi tiếp tục train thì bật lại chế độ train. Một số kỹ thuật như MC dropout cố ý giữ dropout bật và tính nhiều lượt để nghiên cứu bất định, nhưng đó là một quy trình dự đoán riêng, cần mô tả rõ và đánh giá riêng; xem ml basics 37.

Dropout không làm mô hình vĩnh viễn nhỏ đi và không thay thế việc kiểm rò rỉ dữ liệu. Các biến thể có thể dùng chung mặt nạ theo nhóm hoặc theo chiều không gian, nên quy tắc độc lập từng đơn vị là một thiết lập cụ thể, không phải định nghĩa của mọi biến thể. Loss train có dropout bật cũng không nhất thiết so sánh trực tiếp được với loss validation có dropout tắt.

8. Data augmentation: thêm biến thể phù hợp nhãn

Data augmentation tạo phiên bản biến đổi của mẫu train để mô hình học những thay đổi mà ta tin là hợp lệ trong tác vụ. Ý tưởng quan trọng không phải tăng số bản sao bất kỳ, mà là đưa kiến thức về thế giới và nhãn vào dữ liệu: thay đổi gì có thể xảy ra mà mục tiêu vẫn giữ nguyên, hoặc nhãn cần được biến đổi như thế nào để còn đúng.



Hình 16: Ảnh tổng hợp và hai phép biến đổi dịch, lật ngang (minh hoạ, tự tính)

Cách đọc hình: bên trái là ảnh gốc dạng chữ L trên lưới; giữa là ảnh dịch một ô sang phải; phải là ảnh lật ngang. Hai phép biến đổi tạo ảnh khác từ cùng một nguồn, nhưng chúng không tự bảo đảm nhãn còn đúng. Nếu nhãn phân biệt chữ L với hình phản chiếu của nó, phép lật ngang làm thay đổi nhãn hoặc làm mẫu không còn thuộc lớp cũ.

Với nhận dạng một vật thể không phụ thuộc vị trí nhỏ trong ảnh, dịch nhẹ có thể là giả định hợp lý. Trong nhận dạng ký tự, ảnh y khoa hoặc bài toán phân biệt trái phải, lật có thể phá nghĩa. Với dữ liệu văn bản, thay một từ có thể đổi sắc thái phủ định; với dữ liệu thời gian, xáo trộn trật tự có thể phá quan hệ nhân quả. Kiểm phép biến đổi trên ví dụ thật cùng người hiểu miền dữ liệu là một phần thiết kế, không phải bước trang trí.

8.1 Tạo trước hay trong lúc train

Một cách là tạo sẵn nhiều bản biến đổi từ mỗi mẫu train; cách khác là lấy phép biến đổi ngẫu nhiên mỗi khi mẫu được đưa vào mô hình. Cả hai làm mô hình thấy nhiều góc nhìn, nhưng các biến thể từ cùng một nguồn **không độc lập như mẫu mới thu thập**. Tạo thêm bản sao không làm xuất hiện thông tin về những nhóm dân cư hay điều kiện mà tập gốc không đại diện.

Phải chia theo nguồn độc lập trước, rồi chỉ tạo các biến thể train từ nguồn train. Nếu tạo ảnh lật và ảnh gốc rồi chia từng ảnh ngẫu nhiên, hai phiên bản gần giống của cùng nguồn có thể lọt vào train và test, làm điểm test lạc quan cho mục tiêu gặp nguồn mới. Validation thường được giữ theo phân phối đánh giá mong muốn; nếu cố ý dùng nhiều góc nhìn khi suy luận, đó là **test-time augmentation**, một phần của quy trình dự đoán cần được chốt trước và đánh giá nhất quán.

8.2 Khi nhãn cần biến đổi theo đầu vào

Với phân loại mà phép biến đổi giữ nhãn, ảnh sau biến đổi dùng cùng nhãn như trước. Với bài toán định vị hoặc phân đoạn, phải biến đổi cả hộp giới hạn hoặc mặt nạ theo hình học tương ứng. Với một số bài toán, phép biến đổi đòi hỏi đổi nhãn: lật trái phải có thể đổi câu trả lời từ "trái" sang "phải". Không được áp quy tắc giữ nhãn cho mọi tác vụ.

Mixup là một ví dụ khác: tạo tổ hợp lồi của hai đầu vào và trộn nhãn theo cùng hệ số, xem ml basics 31. Ví dụ (tự tính), cho trước hệ số trộn cố định $\lambda = 0{,}25$, cùng $x_a = (0, 4)$ và $x_b = (4, 0)$, ta được $x' = \lambda x_a + (1 - \lambda)x_b = 0{,}25x_a + 0{,}75x_b = (3, 1)$. Ký hiệu λ ở đây là hệ số trộn, không phải hệ số phạt L1 hay L2. Nếu nhãn one-hot lần lượt là $(1, 0)$ và $(0, 1)$, nhãn trộn là $(0{,}25, 0{,}75)$. Đây là nhãn mềm được định nghĩa bởi quy trình mixup, không phải một khẳng định rằng ảnh trộn có duy nhất một lớp cứng.

9. Chọn cường độ và đọc kết quả

Một quy trình hợp lý bắt đầu bằng baseline rõ ràng, cùng cách chia dữ liệu và cùng thước đo. Sau đó chọn một tập giá trị ứng viên cho từng kỹ thuật, huấn luyện trên train và so sánh theo validation. Mô hình, lịch học, số lần thử, hạt giống ngẫu nhiên và cách đánh giá đều có thể ảnh hưởng kết quả. Nếu có nguồn lực, lặp một số cấu hình với các hạt giống khác để phân biệt xu hướng với biến động ngẫu nhiên.

Đừng chỉ đo loss train có cộng penalty rồi so nó với loss validation không cộng penalty. Hai số đó không cùng đại lượng. Nên báo cáo riêng loss dữ liệu train, loss validation và, nếu cần để hiểu tối ưu, tổng mục tiêu train có penalty. Một mô hình có penalty lớn có thể có tổng mục tiêu cao hơn trong khi loss dự đoán validation tốt hơn; điều đó không mâu thuẫn.

Nếu cả train và validation đều kém sau khi tăng regularization, có thể ràng buộc đã quá mạnh hoặc tối ưu chưa đủ. Nếu train tốt hơn rõ nhưng validation vẫn kém, có thể thử tăng regularization, có thêm dữ liệu phù hợp, hoặc thay đổi biểu diễn. Các dấu hiệu này không tự phân biệt được mọi nguyên nhân: trước tiên hãy kiểm chia tập, chất lượng nhãn và sự phù hợp của phân phối đích. Khi các thành phần đã được chọn, đánh giá cuối trên test độc lập.

10. Nối sang AI

Mạng đủ sức ghi nhớ. Ml basics 01 cho thấy cần suy nghĩ cẩn thận về mối quan hệ giữa khả năng khớp train, regularization và tổng quát hoá. Một kỹ thuật regularization hữu ích không có nghĩa chỉ riêng nó giải thích toàn bộ hiệu năng của một mạng sâu.

Dropout và đồng thích nghi. Ml basics 06 trình bày động cơ ngẫu nhiên bỏ đơn vị trong huấn luyện. Quy ước inverted dropout ở chương này giúp tính kỳ vọng rõ ràng; khi đọc một tài liệu khác, hãy kiểm xem phép co được áp lúc train hay lúc đánh giá.

Mẫu biến đổi và nhãn mềm. Ml basics 31 dùng phép trộn đầu vào cùng nhãn để tạo ví dụ huấn luyện. Giá trị của cách này vẫn phải đo theo tác vụ, vì giả định về các điểm nằm giữa hai ví dụ không phù hợp như nhau trong mọi miền.

Nhiễu và dữ liệu chưa gắn nhãn. Ml basics 29 là một ví dụ mở rộng hơn về huấn luyện có nhiễu và dữ liệu chưa gắn nhãn. Augmentation và dropout ở đây là thành phần của một quy trình lớn, không nên tách chúng khỏi cách tạo nhãn và cách đánh giá khi đọc kết quả.

Bất định khi dự đoán. Ml basics 37 xét việc dùng dropout trong nhiều lượt suy luận. Trường hợp đó khác đánh giá thông thường tắt dropout ở chỗ mục tiêu và quy tắc tổng hợp đầu ra đều được định nghĩa lại.

11. Sai lầm hay gặp

- **Nghĩ regularization càng mạnh càng tốt.** Hình phạt hoặc nhiễu quá lớn có thể ngăn mô hình học quy luật hữu ích. Hãy chọn cường độ trên validation.
- **So tổng loss có penalty với validation loss không penalty.** Hai đại lượng chứa số hạng khác nhau. Hãy tách loss dữ liệu và số hạng phạt khi theo dõi.
- **Cho rằng L1 luôn loại đúng các đặc trưng vô ích và L2 luôn tạo số không.** Cách tối ưu, tương quan đặc trưng và thang đo đều ảnh hưởng kết quả; trọng số bằng không không phải bằng chứng nhân quả.
- **Nhầm L2 với mọi dạng weight decay.** Với gradient descent đơn giản chúng có thể tương đương theo quy ước ở mục 5, nhưng với optimizer thích nghi thì thêm penalty vào gradient khác weight decay tách rời.
- **Quên chế độ dropout hoặc lưu sai checkpoint.** Validation thông thường cần chế độ đánh giá; early stopping cần lưu và khôi phục checkpoint tốt nhất, không chỉ dừng ở checkpoint cuối.
- **Tăng cường dữ liệu mà không kiểm nhãn và ranh giới tập.** Phép lật, cắt hay trộn có thể đổi nhãn; các biến thể từ cùng nguồn phải được chia đúng theo đơn vị độc lập.

12. Tóm tắt công thức và thao tác

Khái niệm	Công thức hoặc thao tác	Vai trò
L1	$L_{\text{data}} + \lambda \sum_j \lvert w_j \rvert$	phạt độ lớn tuyệt đối, có thể tạo nghiệm thưa
L2	$L_{\text{data}} + \frac{\lambda}{2} \sum_j w_j^2$	phạt bình phương độ lớn, thu nhỏ trọng số
Soft threshold	$\text{sign}(z)\max(\lvert z \rvert - \lambda, 0)$	nghiệm L1 của bài toán một chiều ở mục 4
Early stopping	lưu checkpoint tốt nhất theo validation	chọn thời gian học mà không dùng test
Inverted dropout	$\tilde{h}_j = m_j h_j / q$, với $q = 1 - p > 0$	giữ kỳ vọng từng đơn vị khi train
Phương sai dropout	$\text{Var}(\tilde{h}_j) = h_j^2 (1 - q)/q$	lượng nhiễu tại đầu ra đơn vị cố định
Augmentation	biến đổi mẫu và nhãn tương ứng	đưa giả định về biến thể hợp lệ vào train

13. Bài tập tự tính

Năm bài từ dễ đến khó. Đáp án ngay dưới từng bài, các số đã kiểm bằng NumPy.

Bài 1 (dễ). Với $w = (3, -4)$ và $\lambda = 0{,}5$, tính penalty L1 $\lambda \sum_j \lvert w_j \rvert$ và penalty L2 $\frac{\lambda}{2} \sum_j w_j^2$.

Đáp án. Tổng trị tuyệt đối bằng $3 + 4 = 7$, nên penalty L1 bằng $0{,}5 \cdot 7 = 3{,}5$. Tổng bình phương bằng $9 + 16 = 25$, nên penalty L2 bằng $(0{,}5/2) \cdot 25 = 6{,}25$ (tự tính). Hai giá trị không nói phương pháp nào tốt hơn trên validation.

Bài 2 (vừa). Tìm nghiệm của $J(w) = \frac{1}{2}(w - 4)^2 + \frac{1}{2}w^2$. Nếu thay số hạng cuối bằng $2|w|$, nghiệm là gì?

Đáp án. Với L2, đạo hàm là $(w - 4) + w = 2w - 4$, bằng không tại $w = 2$; đạo hàm bậc hai dương. Với L1, soft threshold của $z = 4$ ở ngưỡng 2 cho $w = \max(4 - 2, 0) = 2$ (tự tính). Hai nghiệm trùng ở ví dụ này dù mục tiêu khác nhau; thay hệ số có thể cho nghiệm khác.

Bài 3 (vừa). Validation loss ở sáu lần kiểm là $(3, 2, 1{,}5, 1{,}6, 1{,}7, 1{,}4)$. Dùng patience bằng hai lần không cải thiện liên tiếp, không có ngưỡng tối thiểu, thì dừng sau lần nào và khôi phục checkpoint nào? Quy trình này có nhìn thấy lần kiểm thứ sáu không?

Đáp án. Điểm tốt nhất đến lần ba là $1{,}5$. Hai lần bốn và năm đều không cải thiện, nên dừng sau lần năm và khôi phục checkpoint lần ba (tự tính). Lần sáu không được thực hiện theo quy tắc dừng này; vì vậy phương pháp có thể bỏ lỡ một cải thiện muộn hơn.

Bài 4 (vừa). Dùng inverted dropout với $h = (2, -1)$, xác suất bỏ $p = 0{,}25$ và mặt nạ $m = (1, 0)$. Tính đầu ra, rồi tính kỳ vọng đầu ra của đơn vị thứ nhất qua mọi lần lấy mặt nạ.

Đáp án. Xác suất giữ là $q = 0{,}75$. Đầu ra là $(2/0{,}75, 0) = (8/3, 0)$. Đơn vị thứ nhất bằng $8/3$ với xác suất $0{,}75$ và bằng 0 với xác suất $0{,}25$, nên kỳ vọng bằng $0{,}75 \cdot 8/3 + 0{,}25 \cdot 0 = 2$ (tự tính).

Bài 5 (khó hơn). Một bài phân loại có hai mẫu với đầu vào $x_a = (2, 0)$, $x_b = (0, 6)$ và nhãn one-hot $y_a = (1, 0)$, $y_b = (0, 1)$. Dùng mixup với hệ số trộn cố định cho trước $\lambda = 0{,}75$ cho mẫu thứ nhất. Tính đầu vào và nhãn mới; giải thích vì sao không nên tạo hai biến thể của cùng nguồn rồi chia chúng ngẫu nhiên vào train và test khi mục tiêu là đo trên nguồn mới.

Đáp án. $x' = 0{,}75(2, 0) + 0{,}25(0, 6) = (1{,}5, 1{,}5)$ và $y' = 0{,}75(1, 0) + 0{,}25(0, 1) = (0{,}75, 0{,}25)$ (tự tính). Hai biến thể của cùng nguồn gần phụ thuộc nhau; chia chúng sang hai tập có thể cho mô hình thấy thông tin rất giống nguồn trong test. Hãy chia theo nguồn trước, sau đó tạo biến thể trong phần train.

Tài liệu tham khảo nền: Hastie, Tibshirani & Friedman, The Elements of Statistical Learning; Goodfellow, Bengio & Courville, Deep Learning; Hinton et al. 2012 (ml basics 06); Zhang et al. 2017 (ml basics 31); Gal & Ghahramani 2015 (ml basics 37).

Nền tảng 05

Mạng nơ-ron nhiều lớp: phi tuyến giúp ghép các phép biến đổi đơn giản thành hàm linh hoạt

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: ml basics 01 về dữ liệu, mô hình và hàm mất mát; có thể đọc ngay cả khi chưa học đạo hàm

Dùng ở: ml basics 06 (dropout), ml basics 07 (ResNet), ml basics 08 (VGG), ml basics 14 (GAN), ml basics 23 (đối kháng)

> Đây là chương học khái niệm từ đầu, không phải bản dịch paper. Các phép tính ghi "tự tính" đã được kiểm bằng NumPy.

1. Vì sao cần mạng nhiều lớp?

Một mạng nơ-ron nhiều lớp là một chuỗi phép biến đổi có tham số, xen kẽ với các hàm kích hoạt phi tuyến. Nó nhận dữ liệu, biến đổi dữ liệu qua các lớp, rồi tạo ra dự đoán. AI dùng cấu trúc này để học các quy tắc phức tạp từ ví dụ, chẳng hạn từ pixel đến nhãn ảnh, từ từ ngữ đến dự đoán token tiếp theo, hoặc từ trạng thái đến hành động.

Một phép biến đổi tuyến tính đơn lẻ chỉ có thể tạo ranh giới phẳng và quan hệ có dạng đơn giản. Dữ liệu thực tế thường cần nhiều bước: phát hiện tín hiệu cục bộ, ghép chúng thành mẫu lớn hơn, rồi ánh xạ mẫu đó sang câu trả lời. Mạng nhiều lớp lặp lại hai thao tác dễ hiểu: trộn thông tin bằng phép affine và biến dạng nó bằng hàm kích hoạt. Từng lớp không cần tự giải toàn bộ bài toán; nhiều lớp nối nhau có thể xây dần một phép tính hữu ích.

Từ "nơ-ron" gợi ý sinh học, nhưng ở đây nó chỉ là một phép tính nhỏ. Một neuron nhận vài con số, lấy tổng có trọng số, cộng độ lệch, rồi thường áp một hàm kích hoạt. Một lớp gom nhiều neuron cùng tính trên đầu vào đó. Mạng là cách nối những lớp như vậy.

2. Nhắc nhanh nền cần thiết

2.1 Vector là một bó số

Vector $x = (x_1, x_2)^\top$ là cách ghi gọn hai đặc trưng của một ví dụ. Chẳng hạn một điểm trên mặt phẳng có tọa độ $(2, 3)^\top$. Nếu ta nhân vector đó với 2, cả hai tọa độ đều được nhân với 2, thành $(4, 6)^\top$ (tự tính). Vector chỉ là dữ liệu có thứ tự, chưa tự nói nó là ảnh, văn bản hay bảng số.

2.2 Tích vô hướng trộn thông tin

Với hai vector cùng số thành phần, tích vô hướng là tổng từng cặp nhân với nhau: $w^\top x = w_1 x_1 + w_2 x_2$. Ví dụ $w = (3, -1)^\top$ và $x = (2, 4)^\top$ cho $w^\top x = 3 \cdot 2 + (-1) \cdot 4 = 2$ (tự tính). Dấu và độ lớn của từng trọng số quyết định mỗi đặc trưng đóng góp theo hướng nào.

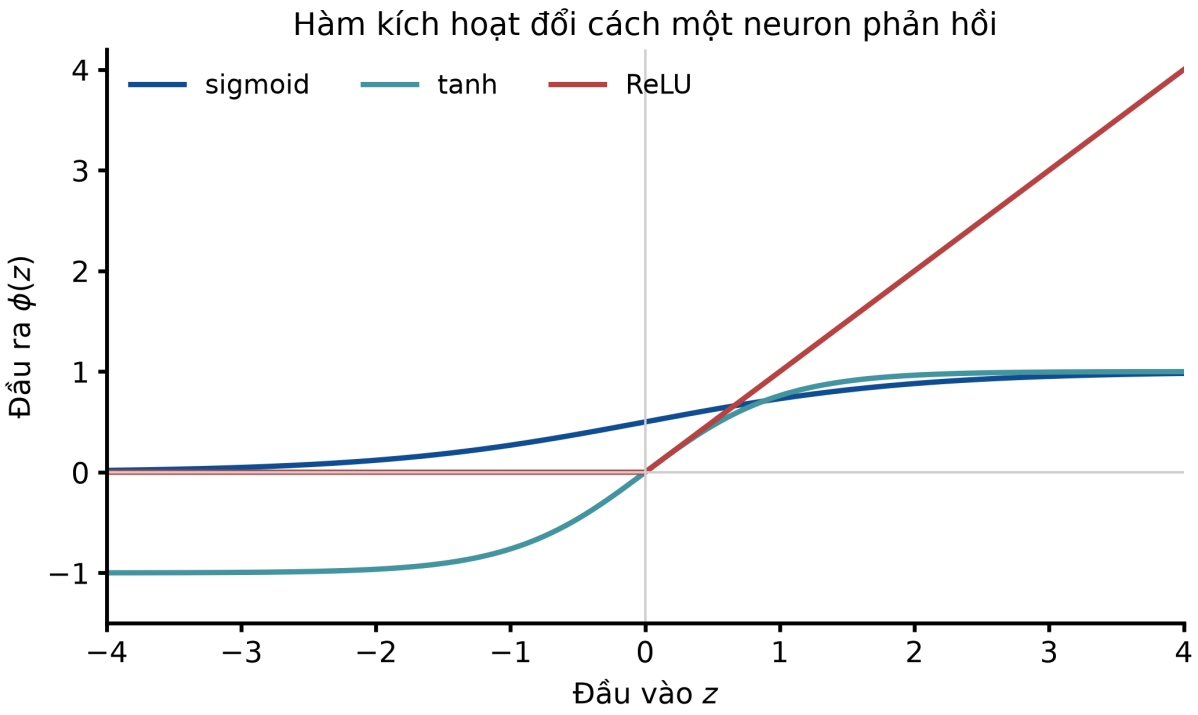
2.3 Phép affine gồm trộn và dịch

Một phép affine có dạng $z = Wx + b$. Ma trận W trộn và co giãn các thành phần đầu vào; vector b dịch kết quả. Trong bài toán một đầu ra, công thức trở thành $z = w^\top x + b$. Với ví dụ vừa rồi, nếu $b = 1$ thì $z = 3$ (tự tính).

Phép tuyến tính thuần tuý thường được viết Wx. Phép affine cho phép dịch vị trí gốc, nên trong mạng ta thường dùng từ "lớp tuyến tính" theo lối gọi quen thuộc dù lớp đó có thể có bias và chính xác là affine. Chương này nói rõ khi điều đó quan trọng.

3. Trực giác bằng hình: một neuron uốn cách phản hồi

Trước khi viết định nghĩa đầy đủ, hãy nhìn ba hàm kích hoạt thường gặp. Cùng một giá trị tổng có trọng số z, mỗi hàm trả ra một kiểu tín hiệu khác nhau.



Hình 17: Đường cong sigmoid, tanh và ReLU minh họa các phản hồi phi tuyến khác nhau (minh hoạ, tự tính)

Cách đọc hình: trục ngang là giá trị trước kích hoạt, trục dọc là đầu ra sau kích hoạt. Sigmoid nén đầu ra vào khoảng từ 0 đến 1; tanh nén quanh 0; ReLU giữ phần dương và chặn phần âm. Các đường cong được vẽ từ định nghĩa hàm, không phải từ dữ liệu huấn luyện.

Một hàm kích hoạt không tự biết lớp nào đúng. Nó chỉ cho mạng thêm một phép biến đổi phi tuyến. Trọng số học cách chọn tổ hợp đầu vào, còn kích hoạt cung cấp hình dạng mà một phép nhân ma trận đơn lẻ không thể tạo ra.

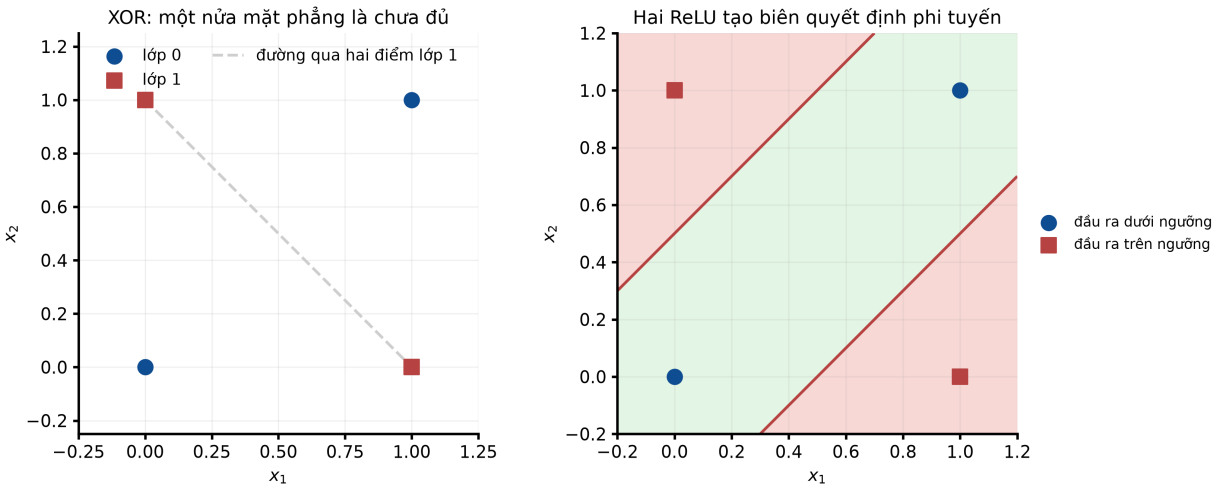
4. Vì sao cần phi tuyến: ví dụ XOR

XOR là phép toán trên hai bit: đầu ra là 1 khi hai đầu vào khác nhau, và là 0 khi chúng bằng nhau. Bảng của nó có bốn hàng. Ta xem đầu vào là điểm (x_1, x_2) trên mặt phẳng, với lớp 1 ở hai góc chéo và lớp 0 ở hai góc còn lại.

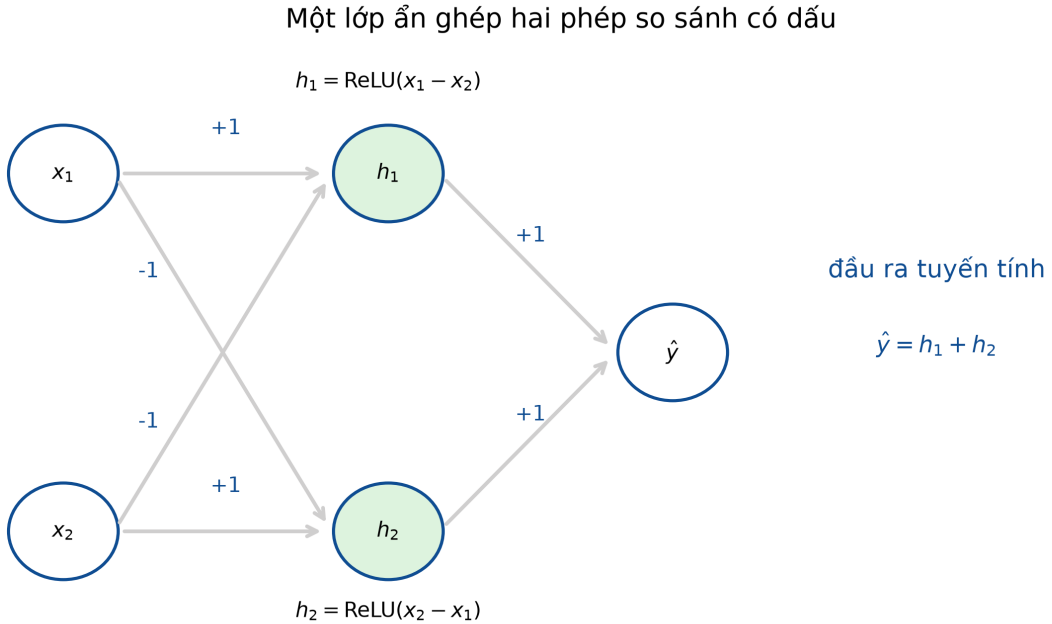
Cách đọc hình: bên trái, hai điểm lớp 1 nằm ở hai góc đối diện, nên một đường thẳng không thể đặt tất cả điểm lớp 1 về một phía và tất cả điểm lớp 0 về phía còn lại. Bên phải, màu nền cho biết lớp dự đoán sau ngưỡng 0,5, đường viền là nơi đầu ra chạm ngưỡng; vùng lớp 1 dự đoán hai điểm chéo là dương. Các vị trí và đường biên là minh hoạ từ công thức ở phần ví dụ tay.

Một lớp affine với đầu ra sigmoid vẫn có biên quyết định tuyến tính: sigmoid biến đổi giá trị, nhưng ngưỡng của nó tương ứng với một ngưỡng trên $w^\top x + b$. Thêm lớp affine mà không có kích hoạt giữa chúng cũng chưa giải quyết được XOR. Ta cần một phép biến dạng ở giữa để tạo đặc trưng mới.

Mạng dưới đây tính hai phép so sánh có dấu rồi cộng chúng. Hai neuron ẩn lần lượt phát hiện khi đầu vào thứ nhất lớn hơn đầu vào thứ hai và khi điều ngược lại xảy ra.



Hình 18: Bốn điểm XOR và biên quyết định tạo bởi hai neuron ReLU (minh hoạ, tự tính)



Hình 19: Mạng ReLU hai neuron ẩn tạo đầu ra bằng trị tuyệt đối hiệu hai đầu vào (minh hoạ, tự tính)

Cách đọc hình: mỗi mũi tên đi từ một giá trị đầu vào tới một phép tính; nhãn trên mũi tên là trọng số. Neuron ẩn áp ReLU sau khi cộng các đầu vào có trọng số. Đầu ra cộng hai tín hiệu ẩn, vì thế chỉ có một trong hai nhánh dương khi hai đầu vào khác nhau.

5. Định nghĩa mạng nhiều lớp

5.1 Một neuron

Neuron thứ j lấy đầu vào x, trọng số hàng $w_j^\top$, bias b_j, rồi tính

$$z_j = w_j^\top x + b_j, \qquad a_j = \phi(z_j).$$

z_j được gọi là tổng trước kích hoạt (pre-activation), còn a_j là đầu ra sau kích hoạt. Hàm ϕ là sigmoid, tanh, ReLU hoặc một lựa chọn khác. Trọng số đặt mức ảnh hưởng của từng đặc trưng; bias đặt ngưỡng dịch chuyển.

5.2 Một lớp

Gom m neuron thành một lớp. Mỗi neuron có một hàng trong W, do đó

$$z = Wx + b, \qquad a = \phi(z),$$

trong đó ϕ được áp riêng lên từng thành phần. Nếu đầu vào có n số và lớp có m neuron, thì W có kích thước $m \times n$, b có m thành phần, và z, a có m thành phần.

Phép affine trộn thông tin giữa các thành phần. Kích hoạt xử lý từng thành phần sau phép trộn. Thứ tự này có ý nghĩa: nói chung, áp kích hoạt trước rồi trộn sẽ cho phép tính khác với trộn trước rồi áp kích hoạt.

5.3 Nhiều lớp nối tiếp

Với L lớp, đặt $a^{(0)} = x$. Mỗi lớp ẩn và lớp đầu ra có thể được viết

$$z^{(\ell)} = W^{(\ell)} a^{(\ell-1)} + b^{(\ell)}, \qquad a^{(\ell)} = \phi^{(\ell)}(z^{(\ell)}),$$

cho các lớp $\ell = 1, \dots, L$. Kích thước đầu ra lớp này phải khớp kích thước đầu vào lớp sau. Thường các lớp ẩn dùng một kích hoạt phi tuyến, trong khi lớp cuối chọn cách xuất điểm số phù hợp với bài toán. Hồi quy có thể dùng đầu ra tuyến tính; phân loại nhiều lớp thường biến điểm số thành phân phối xác suất bằng softmax.

Ở đây "độ sâu" là số lớp biến đổi được xếp tuần tự. Có tài liệu đếm cả đầu vào, có tài liệu không; để tránh nhầm, ta gọi rõ số lớp ẩn và có tính lớp đầu ra hay không. "Độ rộng" thường nói số neuron ở một lớp. Tăng độ sâu và tăng độ rộng là hai cách mở rộng khác nhau, không đảm bảo kết quả tốt hơn nếu dữ liệu, cách học và tính toán không phù hợp.

6. Ví dụ tay: mạng tính XOR từng đầu vào

Ta định nghĩa $h_1 = \mathrm{ReLU}(x_1 - x_2)$, $h_2 = \mathrm{ReLU}(x_2 - x_1)$, và $\hat{y} = h_1 + h_2$. ReLU được định nghĩa là $\mathrm{ReLU}(t) = \max(0, t)$. Mỗi phép tính dưới đây là tự tính và đã kiểm bằng NumPy.

6.1 Đầu vào bằng nhau

Với $(x_1, x_2) = (0, 0)$, hiệu ở cả hai nhánh bằng 0. ReLU trả về 0 cho mỗi nhánh, do đó $\hat{y} = 0$ và ngưỡng 0,5 phân loại điểm này vào lớp 0 (tự tính).

Với $(x_1, x_2) = (1, 1)$, hai hiệu vẫn bằng 0. Ta lại được $h_1 = 0$, $h_2 = 0$, $\hat{y} = 0$, tức lớp 0 (tự tính). Nhánh ẩn không chỉ so sánh từng đầu vào với một hằng số; nó so sánh quan hệ giữa hai đầu vào.

6.2 Đầu vào khác nhau

Với $(1, 0)$, hiệu thứ nhất bằng 1 nên $h_1 = 1$; hiệu thứ hai bằng âm 1 nên $h_2 = 0$. Tổng đầu ra là 1, vượt ngưỡng 0,5 (tự tính).

Với $(0, 1)$, nhánh thứ nhất bằng 0, nhánh thứ hai bằng 1, tổng đầu ra cũng là 1 (tự tính). Kết quả là bảng XOR: hai đầu vào giống nhau tạo 0, hai đầu vào khác nhau tạo 1. Ngưỡng 0,5 chỉ là cách đổi điểm liên tục thành nhãn nhị phân trong ví dụ, không phải một phần bắt buộc của mạng.

Một biểu thức tương đương là $\hat{y} = |x_1 - x_2|$. Hai ReLU tạo hai nửa dấu của hiệu: $\max(0, x_1 - x_2)$ và $\max(0, x_2 - x_1)$. Với đầu vào bit, tổng bằng 1 nếu hai bit khác nhau và bằng 0 nếu chúng giống nhau. Với đầu vào thực bất kỳ, đầu ra là độ lớn hiệu, nên mạng không chỉ tra lại bốn dòng bảng.

6.3 Lớp ẩn tạo tọa độ hữu ích

Đầu vào ban đầu (x_1, x_2) không thể phân loại XOR bằng một đường thẳng. Sau lớp ẩn, mạng tạo hai đặc trưng (h_1, h_2). Các điểm của hai lớp giờ có thể được tách bởi một ngưỡng trên $h_1 + h_2$. Nói cách khác, lớp ẩn xây một cách biểu diễn khác; lớp cuối làm phép phân loại trở nên đơn giản trong biểu diễn đó.

Đây là một ví dụ nhỏ về học biểu diễn. Trong ứng dụng, không ai cần thiết kế thủ công mọi đặc trưng ẩn như ví dụ này. Huấn luyện điều chỉnh trọng số sao cho các lớp tự tìm được biểu diễn hữu ích cho mục tiêu đã chọn.

7. Vì sao xếp các lớp tuyến tính thôi vẫn chưa đủ

Giả sử hai lớp liên tiếp không có kích hoạt: lớp đầu $z = W_1 x + b_1$, lớp sau $y = W_2 z + b_2$. Thay biểu thức thứ nhất vào biểu thức thứ hai được

$$y = W_2 W_1 x + W_2 b_1 + b_2.$$

Đây vẫn là một phép affine duy nhất. Dù ta chèn thêm nhiều lớp affine, cả chuỗi vẫn rút gọn thành một phép affine tương đương. Vì vậy số lớp trên sơ đồ tự nó chưa tạo ra sức biểu đạt phi tuyến.

Khi đặt kích hoạt giữa các lớp, phép rút gọn đó không còn đúng. ReLU tạo các vùng mà trong mỗi vùng mạng hành xử tuyến tính, còn ở ranh giới nó có thể đổi hệ số. Ghép nhiều vùng như vậy cho phép tạo hàm gấp khúc phức tạp hơn một đường thẳng.

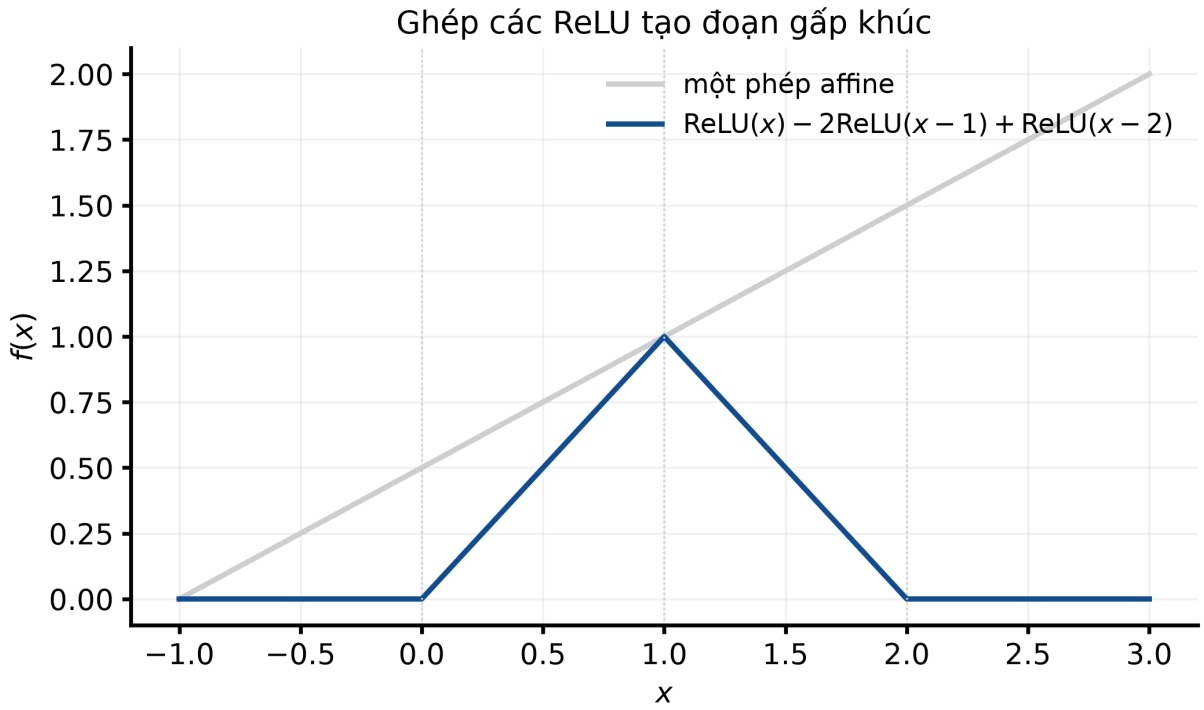
Cách đọc hình: đường xám là một phép affine nên độ dốc giữ nguyên trên toàn trục. Đường xanh là tổ hợp của ba ReLU, có các điểm gấp tại những ngưỡng được đánh dấu. Các hàm tuyến tính từng đoạn là trực giác hữu ích cho mạng ReLU, nhưng không nên suy rộng nguyên xi sang mọi kích hoạt.

8. Tham số, kích thước và cách đọc kiến trúc

Một lớp có n đầu vào và m neuron thường có mn trọng số cùng m bias, nếu lớp dùng bias. Mỗi hàng của ma trận trọng số kết nối mọi đầu vào với một neuron. Vì thế lớp dày đặc (fully connected) có số tham số tăng theo cả chiều rộng đầu vào lẫn số neuron đầu ra.

Trong mạng XOR ở trên, lớp đầu nhận hai số và có hai neuron, nên có bốn trọng số cùng hai bias; lớp đầu ra nhận hai số và có một neuron, nên có hai trọng số cùng một bias. Tổng cộng chín tham số (tự tính). Kiểm tra bằng NumPy: kích thước các khối tham số lần lượt là $(2, 2)$, $(2,)$, $(1, 2)$ và $(1,)$; cộng số phần tử của bốn khối được 9.

Số tham số là một phần chi phí, không phải thước đo đầy đủ cho chất lượng hay tốc độ. Ma trận lớn có thể được tính hiệu quả trên phần cứng song song; một mạng có ít tham số vẫn có thể tốn nhiều phép toán nếu tái



Hình 20: Tổ hợp ReLU tạo hàm tuyến tính từng đoạn thay vì một đường affine duy nhất (minh hoạ, tự tính)

sử dụng chúng nhiều lần. Dữ liệu ảnh, văn bản và chuỗi thời gian thường dùng kết nối có cấu trúc thay vì nối mọi đầu vào với mọi neuron.

Ký hiệu kiến trúc như $2 \to 4 \to 1$ thường có nghĩa là hai đầu vào, một lớp ẩn rộng bốn và một đầu ra. Ký hiệu có thể bỏ qua bias, loại kích hoạt và cách lớp cuối diễn giải đầu ra. Khi đọc một sơ đồ, hãy hỏi: mỗi mũi tên là kết nối nào, hàm nào nằm giữa các lớp, và đầu ra là điểm số hay xác suất?

9. Các hàm kích hoạt và chi tiết thường bị bỏ qua

9.1 Sigmoid

Sigmoid được định nghĩa là $\sigma(z) = 1/(1 + e^{-z})$. Đầu ra nằm giữa 0 và 1, nên nó hữu ích khi một đầu ra được diễn giải như xác suất của biến nhị phân. Khi z rất dương hoặc rất âm, đầu ra tiến gần một đầu mút và đạo hàm nhỏ; tín hiệu tối ưu qua nhiều tầng có thể yếu đi. Điều này thường được gọi là bão hoà.

9.2 Tanh

Tanh có đầu ra trong khoảng từ âm 1 đến 1 và đối xứng quanh gốc. Một số giá trị âm và dương giúp biểu diễn tín hiệu có dấu. Tanh cũng bão hoà ở hai phía xa gốc, do đó đạo hàm cũng nhỏ khi đầu vào có độ lớn lớn.

9.3 ReLU và họ hàng

ReLU giữ nguyên đầu vào dương, trả 0 cho đầu vào âm. Nó đơn giản và không bão hoà ở phía dương, nhưng đạo hàm bằng 0 ở phía âm. Nếu một neuron luôn nhận đầu vào âm trong dữ liệu huấn luyện, cập nhật gradient có thể không mở lại nhánh đó; hiện tượng này thường được gọi là neuron chết.

Leaky ReLU cho một độ dốc nhỏ ở phía âm. GELU và SiLU dùng các dạng cong trơn hơn, thường gặp trong kiến trúc hiện đại. Tên hàm chưa đủ để kết luận mô hình sẽ học tốt: kết quả còn phụ thuộc vào khởi tạo, phân phối dữ liệu, độ sâu, cách chuẩn hoá và bộ tối ưu.

9.4 Hàm kích hoạt ở đầu ra

Lớp đầu ra không nhất thiết dùng cùng kích hoạt với lớp ẩn. Hồi quy một đại lượng không bị chặn thường dùng đầu ra tuyến tính. Phân loại nhị phân có thể dùng sigmoid; phân loại nhiều lớp loại trừ nhau thường biến các điểm số thành xác suất bằng softmax. Với dự đoán nhiều nhãn độc lập, dùng một sigmoid cho từng nhãn có thể hợp lý hơn một softmax chung.

Softmax nhận vector điểm số z và trả $p_i = e^{z_i} / \sum_j e^{z_j}$. Tổng các xác suất bằng 1, nhưng điều đó phản ánh giả định rằng các lớp loại trừ nhau. Nếu nhiều nhãn cùng đúng, xác suất độc lập thường phù hợp hơn. Chi tiết về cross-entropy nằm trong chương nền tảng tương ứng của Toán và bài paper liên quan.

9.5 Kích hoạt ảnh hưởng đến việc học

Backprop cần đạo hàm của kích hoạt để truyền tín hiệu ngược. ReLU có điểm gấp tại 0, nơi đạo hàm cổ điển không tồn tại; tập dưới đạo hàm là $[0, 1]$, và phần mềm chọn một quy ước để tính gradient tại đúng điểm đó. Với một mẫu ngẫu nhiên liên tục, việc đúng chính xác tại ngưỡng thường không phải trường hợp chủ đạo, nhưng khi tự viết đạo hàm thì vẫn cần nêu quy ước.

Kích hoạt cũng thay đổi thang đo tín hiệu giữa các lớp. Nếu tín hiệu co nhỏ liên tục, hoặc bị khuếch đại liên tục, huấn luyện mạng sâu trở nên khó. Khởi tạo, chuẩn hoá và kết nối tắt là một số cách giải quyết vấn đề truyền tín hiệu; xem ResNet ở ml basics nen07 và các chương nền tối ưu hoá khi cần tìm hiểu tiếp.

10. Mạng nhiều lớp xuất hiện trong AI ra sao?

- **Ảnh:** lớp đầu có thể phản hồi với cạnh, vùng sáng tối hay hoa văn đơn giản; các lớp tiếp theo kết hợp chúng thành bộ phận và đối tượng. CNN áp đặt cấu trúc không gian để không phải nối mọi pixel với mọi neuron; phần đó được giới thiệu ở ml basics nen07.
- **Văn bản:** một lớp có thể trộn thông tin từ các vector token; nhiều lớp nối tiếp tạo biểu diễn có ngữ cảnh. Mạng hồi quy và mô hình attention khác nhau ở cách truyền thông tin, nhưng vẫn gồm những phép biến đổi tham số và phi tuyến.
- **Sinh dữ liệu:** mạng có thể biến một vector nhiễu hoặc mã tiềm ẩn thành ảnh hay chuỗi. Trong GAN, hai mạng được tối ưu với mục tiêu đối kháng; trong VAE, mạng mã hoá và giải mã một biến tiềm ẩn. Các họ mô hình này có chương riêng trong lộ trình.
- **Phân loại và hồi quy:** lớp cuối biến biểu diễn ẩn thành điểm số, xác suất hoặc giá trị dự đoán. Mạng không tự biết tiêu chí đúng; hàm mất mát và dữ liệu huấn luyện quy định tín hiệu điều chỉnh tham số.
- **Tính toán gradient:** gradient $\nabla_\theta L$ cho biết mất mát nhạy thế nào với từng tham số, và hướng $-\nabla_\theta L$ là hướng giảm cục bộ khi xét bước đủ nhỏ. Chương kế tiếp dùng mạng nhỏ để tính gradient này bằng tay; math nen07 trình bày quy tắc chuỗi tổng quát.

Các mô tả về lớp đầu và lớp sâu là trực giác, không phải cam kết rằng từng neuron có ý nghĩa đơn giản hay riêng biệt. Một mạng có thể lưu trữ khái niệm qua nhiều đơn vị cùng lúc; cách diễn giải nội bộ cần phương pháp riêng.

11. Sai lầm hay gặp

- Nghĩ rằng càng nhiều lớp thì luôn càng tốt. Độ sâu tăng khả năng biểu diễn, nhưng cũng làm tối ưu, tính toán và kiểm soát tổng quát hoá khó hơn.
- Xem mỗi neuron như một đơn vị sinh học hoàn chỉnh. Trong mô hình, neuron chỉ là một phép tính số học; ý nghĩa của nó phụ thuộc vào mạng và dữ liệu.
- Bỏ qua kích hoạt rồi cho rằng mạng nhiều lớp vẫn phi tuyến. Chuỗi lớp affine không có kích hoạt vẫn tương đương một lớp affine.
- Nghĩ ReLU tự phân loại. ReLU chỉ biến đổi số; lớp cuối và quy tắc dự đoán mới ánh xạ biểu diễn thành câu trả lời.

- Đồng nhất điểm số với xác suất. Một đầu ra tuyến tính có thể là điểm số âm hoặc dương không giới hạn; cần phép biến đổi phù hợp và kiểm tra hiệu chỉnh nếu muốn diễn giải xác suất.
- Nghĩ một hàm kích hoạt tốt nhất cho mọi nhiệm vụ. Lựa chọn phụ thuộc vào kiến trúc, mục tiêu, phân phối tín hiệu và cách huấn luyện.

12. Tóm tắt công thức và ký hiệu

Khái niệm	Công thức hoặc cách hiểu
Neuron affine	$z = w^\top x + b$
Đầu ra neuron	$a = \phi(z)$
Một lớp	$z = Wx + b$, kích hoạt áp theo từng thành phần
Nhiều lớp	$a^{(\ell)} = \phi^{(\ell)}(W^{(\ell)} a^{(\ell-1)} + b^{(\ell)})$
Số tham số lớp dày đặc có bias	$mn + m$ với n đầu vào, m đầu ra
ReLU	$\max(0, z)$
Sigmoid	$1/(1 + e^{-z})$
Tác dụng của phi tuyến	Giúp chuỗi lớp không rút gọn thành một phép affine

13. Bài tập tự tính

Bài 1

Một neuron có $w = (2, -1)^\top$, $x = (3, 1)^\top$ và $b = 2$. Tính z và đầu ra ReLU.

Đáp án: $z = 2 \cdot 3 + (-1) \cdot 1 + 2 = 7$, nên ReLU cho 7 (tự tính). NumPy xác nhận phép nhân vector, cộng bias và áp $\max(0, z)$ cho kết quả đó.

Bài 2

Với mạng XOR trong ví dụ chính, tính $(h_1, h_2, \hat{y})$ tại $(x_1, x_2) = (1, 0)$ và $(1, 1)$.

Đáp án: tại $(1, 0)$, $(h_1, h_2, \hat{y}) = (1, 0, 1)$; tại $(1, 1)$, $(0, 0, 0)$ (tự tính). Thay trực tiếp các hiệu vào ReLU và cộng hai đầu ra cho đúng các giá trị này trong NumPy.

Bài 3

Một lớp dày đặc có 3 đầu vào và 4 neuron đầu ra, có bias. Có bao nhiêu trọng số và tổng cộng bao nhiêu tham số?

Đáp án: ma trận có $4 \cdot 3 = 12$ trọng số; bias có 4 số; tổng là 16 tham số (tự tính). NumPy tạo mảng kích thước $(4, 3)$ và vector kích thước $(4,)$, tổng số phần tử bằng 16.

Bài 4

Cho hai lớp affine $z = W_1 x + b_1$ và $y = W_2 z + b_2$. Viết một lớp affine tương đương khi không có kích hoạt ở giữa.

Đáp án: thay z vào biểu thức y, ta được $y = (W_2 W_1)x + (W_2 b_1 + b_2)$. Có thể kiểm bằng NumPy trên mọi lựa chọn ma trận tương thích kích thước; đây là đồng nhất thức đại số, không phụ thuộc một bộ số cụ thể.

Bài 5

Giải thích vì sao thêm một sigmoid sau một lớp tuyến tính chưa giải quyết XOR bằng một biên phân loại phi tuyến.

Đáp án: sigmoid đơn điệu nên điều kiện $\sigma(w^\top x + b) > t$ tương đương một ngưỡng tuyến tính trên $w^\top x + b$ với mọi ngưỡng t nằm giữa 0 và 1. Hình dạng điểm số bị nén, nhưng đường phân chia trên mặt phẳng đầu vào vẫn là một đường thẳng. Cần một lớp ẩn với phi tuyến để tạo biểu diễn mới.

Tài liệu tham khảo nền: Goodfellow, Bengio và Courville, *Deep Learning*; Bishop, *Pattern Recognition and Machine Learning*; bài ml basics 07 (ResNet), bài ml basics 08 (VGG), math nen07 (chain rule và backprop); ml basics nen07 (CNN và RNN).

Nền tảng 06

Backprop từ đầu: truyền độ nhạy ngược qua mạng để tính gradient của mọi tham số

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: ml basics nen05 về mạng nhiều lớp và hàm kích hoạt; math nen06 về đạo hàm và gradient, math nen07 về quy tắc chuỗi

Dùng ở: ml basics 06 (dropout), ml basics 07 (ResNet), ml basics 14 (GAN), math 02 (tự động tính đạo hàm), math 18 (gradient trong mạng sâu)

> Đây là chương học cách tính từ đầu, không phải bản dịch paper. Các phép tính ghi "tự tính" đã được kiểm bằng NumPy.

1. Vì sao cần backprop?

Backpropagation, hay lan truyền ngược, là cách áp dụng quy tắc chuỗi để tính đạo hàm của mất mát theo từng tham số của mạng. Lượt xuôi cho dự đoán và mất mát; lượt ngược cho biết nếu thay đổi nhỏ từng trọng số thì mất mát thay đổi theo hướng nào. Bộ tối ưu dùng các gradient đó để đề xuất cập nhật.

Mạng có thể chứa hàng triệu hay nhiều hơn tham số. Lấy đạo hàm bằng cách viết toàn bộ công thức khai triển sẽ lặp lại nhiều phép tính. Backprop tổ chức các phép tính thành một đồ thị: giữ lại một số giá trị của lượt xuôi, rồi truyền tín hiệu độ nhạy theo chiều ngược. Mỗi phép toán chỉ cần quy tắc đạo hàm cục bộ; tại nút có nhiều nhánh, ta cộng các đóng góp.

Backprop không phải là tên khác của gradient descent. Backprop tính gradient; gradient descent hoặc một bộ tối ưu khác dùng gradient để cập nhật tham số. Tách hai công việc này giúp hiểu vì sao có thể đổi optimizer mà vẫn dùng cùng một phép lan truyền ngược.

2. Nhắc nhanh nền cần thiết

2.1 Đạo hàm là độ nhạy cục bộ

Đạo hàm $f'(x)$ mô tả đầu ra đổi xấp xỉ bao nhiêu khi đầu vào đổi một lượng nhỏ. Nếu $f(x) = x^2$, thì $f'(x) = 2x$; tại $x = 3$, độ dốc bằng 6 (tự tính). Khi một biến đổi có độ dốc lớn, một thay đổi nhỏ ở đầu vào có thể làm đầu ra đổi nhiều hơn.

Gradient của một hàm mất mát scalar theo vector tham số là vector các đạo hàm riêng. Nếu $\theta = (\theta_1, \theta_2)$, thì $\nabla_\theta L = (\partial L/\partial\theta_1, \partial L/\partial\theta_2)^\top$. Gradient chỉ ra độ tăng nhanh nhất cục bộ; muốn giảm hàm, một bước cơ bản đi ngược hướng này.

2.2 Quy tắc chuỗi trên một đường

Nếu $u = g(x)$ rồi $L = f(u)$, thì

$$\frac{dL}{dx} = \frac{dL}{du}\frac{du}{dx}.$$

Độ nhạy ở cuối được nhân với độ nhạy của phép tính phía trước. Ví dụ $u = 2x$ và $L = u^2$; tại $x = 3$, $u = 6$, $dL/du = 12$ và $du/dx = 2$, nên $dL/dx = 24$ (tự tính). Đạo hàm trực tiếp của $4x^2$ cũng cho $8x = 24$ tại điểm đó.

2.3 Nhiều nhánh phải cộng ảnh hưởng

Nếu x đi vào hai nhánh rồi cả hai góp vào L, ta phải cộng hai đạo hàm qua từng nhánh. Ví dụ $u = x^2$, $v = 3x$, $L = u + v$, tại $x = 2$ nhánh thứ nhất đóng góp 4, nhánh thứ hai đóng góp 3, tổng độ nhạy là 7 (tự tính). Việc một giá trị được dùng lại nhiều chỗ cũng tạo nhiều đóng góp về cùng biến.

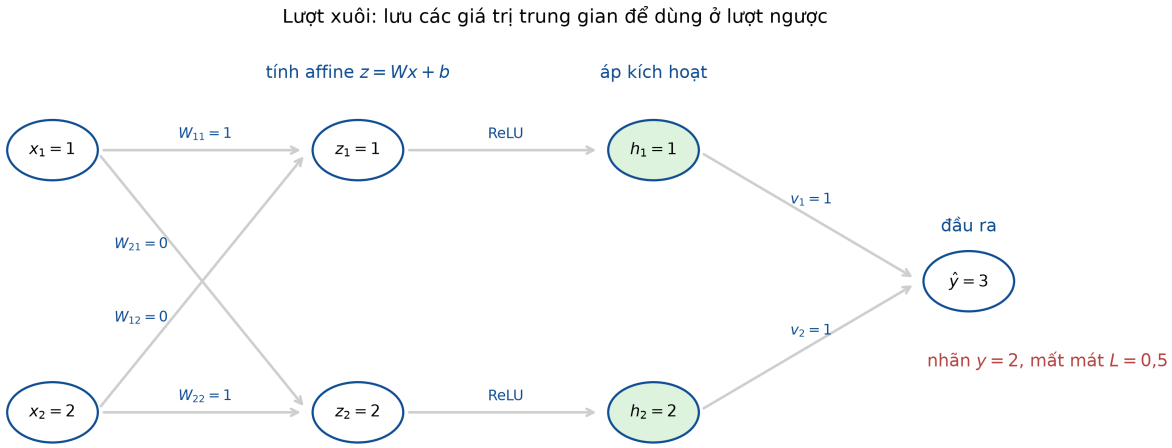
2.4 Kích thước vector và ma trận

Ta xem vector gradient là cột. Nếu lớp affine $z = Wx + b$ có m đầu ra và n đầu vào, thì W có kích thước $m \times n$. Với vector độ nhạy $\delta = \nabla_z L$ có m thành phần, gradient theo trọng số có dạng tích ngoài $\delta x^\top$, kích thước $m \times n$, đúng bằng kích thước W. Kiểm tra kích thước là cách tốt để phát hiện chuyển vị sai.

Math nen07 trình bày quy tắc chuỗi ở dạng tổng quát với Jacobian. Ở đây ta tập trung vào trường hợp hữu ích trong học sâu: đầu ra cuối thường là một số mất mát, nên không cần lập toàn bộ ma trận đạo hàm từ mọi tham số đến mọi giá trị trung gian.

3. Nhìn hình trước: lượt xuôi và lượt ngược

Mạng nhỏ có hai đầu vào, hai neuron ẩn và một đầu ra. Trước tiên ta tính các giá trị đi từ trái sang phải. Sau đó, bắt đầu ở mất mát và truyền các đạo hàm từ phải sang trái.



Hình 21: Đồ thị tính toán và các giá trị lượt xuôi của mạng mẫu (minh hoạ, tự tính)

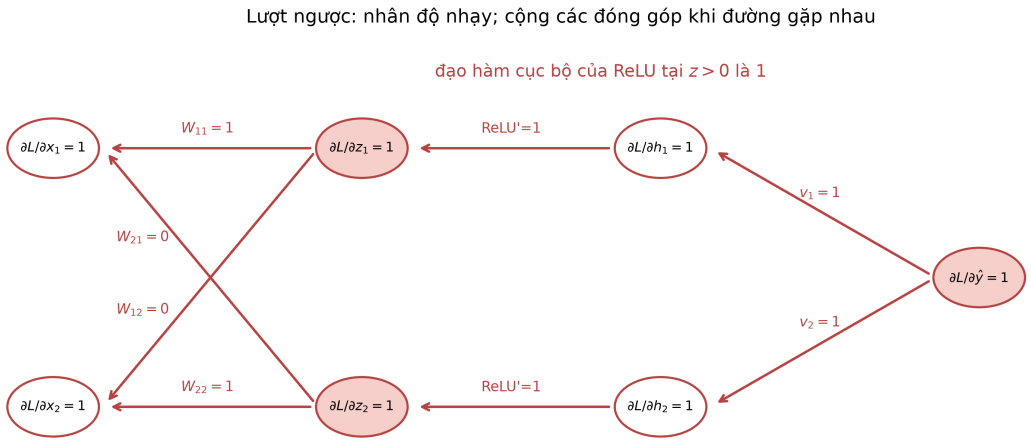
Cách đọc hình: giá trị đầu vào đi vào một lớp affine, sau đó qua ReLU và phép cộng đầu ra. Các con số trong vòng tròn là kết quả lượt xuôi; nhãn bên dưới cho biết nhãn đúng và mất mát của ví dụ. Những giá trị trung gian này được giữ lại vì các phép đạo hàm ở lượt ngược sẽ cần chúng.

Cách đọc hình: mũi tên đỏ chạy ngược chiều tính toán. Tại mỗi nút, độ nhạy nhận từ phía sau được nhân với đạo hàm cục bộ; nếu một nút có nhiều đường đi tới mất mát thì các khoản đóng góp được cộng. Hình dùng ví dụ mà các giá trị trước ReLU đều dương, nên đạo hàm ReLU tại đó bằng 1.

Hai lượt trên cùng một mạng có mục đích khác nhau. Lượt xuôi tính đầu ra và lưu giá trị; lượt ngược tính độ nhạy của mất mát theo các giá trị và tham số. Backprop là một thuật toán tổng quát trên đồ thị tính toán, không chỉ dành cho mạng nơ-ron, nhưng mạng nơ-ron là nơi cấu trúc này đặc biệt hữu ích.

4. Định nghĩa mạng và hàm mất mát mẫu

Xét một ví dụ huấn luyện với đầu vào $x \in \mathbb{R}^2$ và nhãn scalar y. Mạng có hai neuron ẩn ReLU, sau đó là một đầu ra tuyến tính:



Hình 22: Các mũi tên đỏ biểu diễn chiều truyền đạo hàm về đầu vào qua mạng (minh hoạ, tự tính)

$$z = W_1 x + b_1, \qquad h = \mathrm{ReLU}(z), \qquad \hat{y} = v^\top h + c.$$

Ta dùng mất mát bình phương nửa: $L = \frac{1}{2}(\hat{y} - y)^2$. Hệ số $\frac{1}{2}$ giúp đạo hàm của bình phương gọn; nó không làm thay đổi vị trí cực tiểu. Đây là lựa chọn để tập tính, không phải tuyên bố rằng mọi bài toán nên dùng mất mát này.

Trong mạng mẫu ta đặt

$$x = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad W_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad b_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \quad v = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad c = 0, \quad y = 2.$$

W_1 có hàng ứng với neuron ẩn và cột ứng với đầu vào. Đầu ra $\hat{y}$ là một số; nhãn y là mục tiêu đã cho. Các phép tính của ví dụ dưới đây là tự tính và đã được kiểm bằng NumPy.

5. Tính xuôi từng bước

5.1 Lớp affine đầu tiên

Ta nhân ma trận trọng số với đầu vào và cộng bias:

$$z = W_1 x + b_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

Neuron thứ nhất giữ thành phần đầu, neuron thứ hai giữ thành phần thứ hai. Trong ví dụ này, các giá trị trước kích hoạt đều dương.

5.2 Hàm kích hoạt

ReLU áp lên từng thành phần, cho $h = (1, 2)^\top$ (tự tính). Vì cả hai đầu vào ReLU đều dương, ta sẽ dùng đạo hàm cục bộ bằng 1 ở cả hai nhánh.

5.3 Đầu ra và mất mát

Đầu ra là $\hat{y} = 1 \cdot 1 + 1 \cdot 2 + 0 = 3$. Sai số là $\hat{y} - y = 3 - 2 = 1$, do đó $L = \frac{1}{2} \cdot 1^2 = 0{,}5$ (tự tính). Ta lưu ít nhất $x, z, h, \hat{y}$ cùng tham số cần dùng lại; nhãn y cũng được giữ để tính đạo hàm mất mát.

Mục tiêu của lượt ngược là tìm, chẳng hạn, $\partial L / \partial v_1$: nếu chỉ thay đổi trọng số thứ nhất ở lớp cuối một lượng nhỏ, mất mát sẽ đổi thế nào? Ta cũng cần cùng câu trả lời cho mọi bias và trọng số trong W_1.

6. Tính ngược từ mất mát về đầu ra

Ta ký hiệu $\bar{q} = \partial L / \partial q$ cho độ nhạy của mất mát theo đại lượng q. Ký hiệu gạch trên là một cách viết tiện, không phải phép biến đổi mới. Bắt đầu ở L và lần lượt áp quy tắc chuỗi.

Vì $L = \frac{1}{2}(\hat{y} - y)^2$, giữ nhãn cố định và lấy đạo hàm theo dự đoán:

$$\bar{\hat{y}} = \frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 1.$$

Đây là hạt giống độ nhạy được đưa vào nút đầu ra. Không nên kết luận mọi gradient bằng 1; giá trị 1 chỉ đúng ở nút dự đoán trong ví dụ này. Từ đây, mỗi phép toán ở phía trước sẽ biến đổi nó theo đạo hàm cục bộ.

7. Tính gradient của lớp đầu ra

Ta có $\hat{y} = v_1 h_1 + v_2 h_2 + c$. Đạo hàm theo một trọng số đầu ra là giá trị ẩn tương ứng, còn đạo hàm theo h_j là trọng số kết nối tương ứng. Do đó

$$\nabla_v L = \bar{\hat{y}} h = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \qquad \frac{\partial L}{\partial c} = \bar{\hat{y}} = 1, \qquad \bar{h} = \bar{\hat{y}} v = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Các giá trị được tính từ $h = (1, 2)^\top$, $v = (1, 1)^\top$ và $\bar{\hat{y}} = 1$ (tự tính). Gradient theo v cùng kích thước với v; gradient theo bias là scalar. Mỗi thành phần $\bar{h}_j$ là độ nhạy của mất mát khi ta tạm thay đổi đầu ra neuron ẩn đó.

Ở dạng tổng quát, nếu $\hat{y} = v^\top h + c$, thì $\nabla_v L = \bar{\hat{y}} h$, $\bar{h} = \bar{\hat{y}} v$ và $\partial L / \partial c = \bar{\hat{y}}$. Các công thức này nói đúng điều trực giác: trọng số nhân với tín hiệu ẩn ở lượt xuôi, còn độ nhạy truyền ngược về tín hiệu ẩn qua trọng số đó.

8. Đi qua ReLU và lớp đầu

8.1 Qua ReLU

ReLU có đạo hàm 1 khi đối số dương và 0 khi đối số âm. Tại đúng 0, đạo hàm cổ điển không tồn tại; tập dưới đạo hàm của ReLU là $[0, 1]$. Trong ví dụ này không cần chọn quy ước vì các đầu vào đều dương. Vì $z = (1, 2)^\top$ dương từng thành phần, ta có

$$\bar{z} = \bar{h} \odot \mathrm{ReLU}'(z) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

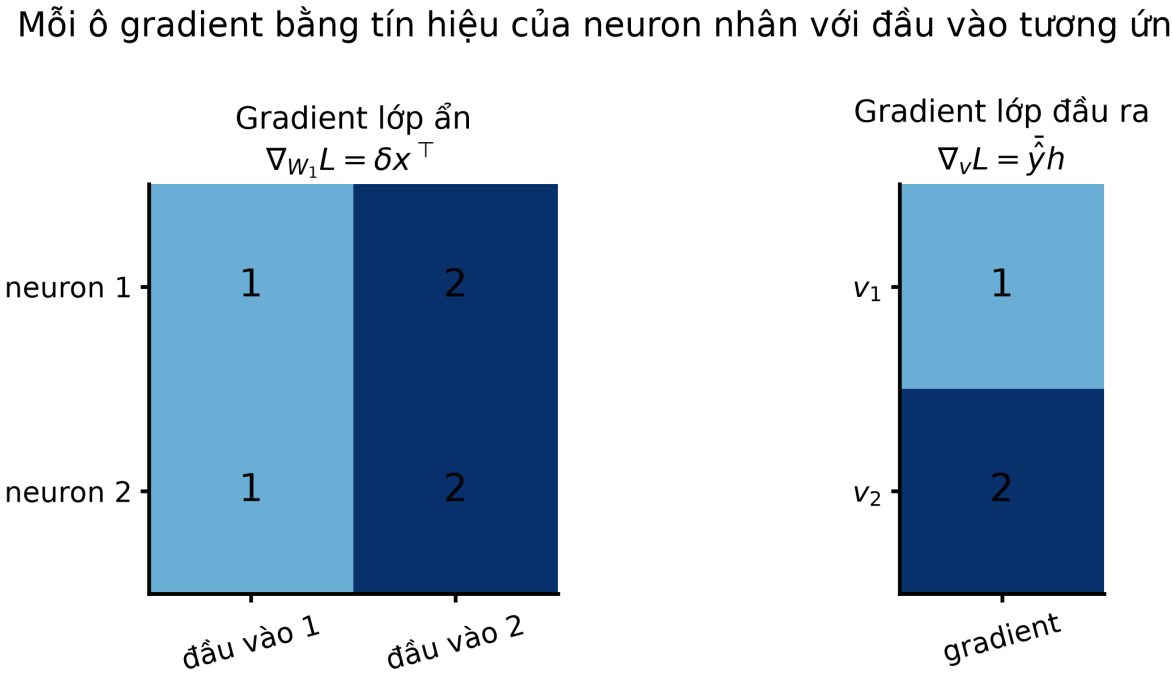
Ký hiệu $\odot$ là nhân từng thành phần. Tín hiệu không đổi vì cả hai hệ số đạo hàm cục bộ đều bằng 1 (tự tính). Nếu một đầu vào ReLU âm, tín hiệu của nhánh đó bị nhân với 0 và không truyền qua nút ấy cho mẫu hiện tại.

8.2 Gradient theo trọng số và bias ẩn

Neuron thứ i có $z_i = \sum_j (W_1)_{ij} x_j + (b_1)_i$. Đạo hàm theo một trọng số $(W_1)_{ij}$ bằng x_j nhân với độ nhạy $\bar{z}_i$. Gộp các phần tử thành ma trận:

$$\nabla_{W_1} L = \bar{z} x^\top = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 1 & 2 \end{pmatrix}, \qquad \nabla_{b_1} L = \bar{z} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Mỗi phần tử ma trận và vector bias là tự tính. Chẳng hạn trọng số ở hàng thứ hai, cột thứ nhất tác động lên z_2 qua $x_1 = 1$; độ nhạy $\bar{z}_2 = 1$, vậy đạo hàm của mất mát theo trọng số đó là 1. Trọng số cùng cột thứ hai có gradient 2 vì đầu vào tương ứng bằng 2.



Hình 23: Ma trận gradient là tích ngoài giữa tín hiệu sai số và đầu vào của lớp (minh hoạ, tự tính)

Cách đọc hình: mỗi hàng ứng với một neuron nhận tín hiệu độ nhạy δ; mỗi cột ứng với một thành phần đầu vào. Ô tại hàng i, cột j là tích độ nhạy của neuron với thành phần đầu vào đó. Lớp đầu ra có cùng quy tắc, với đầu vào của lớp là vector ẩn h.

8.3 Gradient theo đầu vào

Cũng có thể hỏi mỗi đặc trưng đầu vào ảnh hưởng đến mất mát bao nhiêu. Từ $z = W_1 x + b_1$, ta có

$$\nabla_x L = W_1^\top \bar{z} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Kết quả bằng $(1, 1)^\top$ (tự tính). Chuyển vị xuất hiện vì ta truyền độ nhạy từ đầu ra của lớp về các tọa độ đầu vào. Nếu W_1 không đối xứng hoặc không phải ma trận vuông, phép chuyển vị càng dễ nhận ra; công thức tổng quát vẫn là $W_1^\top \bar{z}$.

9. Gộp lượt xuôi và ngược thành thuật toán

Với một lớp tổng quát $z = Wa + b$, $h = \phi(z)$ và độ nhạy nhận được $\bar{h}$, lượt ngược có thể thực hiện theo thứ tự:

1. Nhân từng thành phần với đạo hàm kích hoạt: $\bar{z} = \bar{h} \odot \phi'(z)$.

2. Tính gradient trọng số bằng tích ngoài: $\nabla_W L = \bar{z} a^\top$.
3. Gradient bias bằng độ nhạy trước kích hoạt: $\nabla_b L = \bar{z}$.
4. Gửi độ nhạy về lớp trước: $\bar{a} = W^\top \bar{z}$.

Nếu có nhiều lớp, lặp các bước theo chiều từ lớp cuối về lớp đầu. Kết quả gradient của mỗi tham số có cùng kích thước với tham số đó. Trình tự này xuất phát trực tiếp từ quy tắc chuỗi và quy tắc đạo hàm của phép nhân ma trận với vector.

Đồ thị tính toán có thể có phép cộng nhánh, kết nối tắt hay một biến dùng nhiều lần. Khi đó, các độ nhạy gửi về cùng một nút được cộng. Nếu chỉ một đường đi về thì chỉ nhân các đạo hàm cục bộ trên đường. Đây là điểm quan trọng trong mạng residual: đường tắt tạo thêm một đường truyền tín hiệu, và gradient cộng đóng góp của các nhánh theo quy tắc chuỗi.

Backprop không tính mọi đạo hàm riêng của mọi nút trung gian với mọi đầu vào. Nó tính tác động của một đầu ra scalar theo nhiều tham số bằng cách truyền một vector độ nhạy ngược qua đồ thị. Đây là lý do rất nhiều gradient có thể được lấy với chi phí cùng bậc với một lượt tính xuôi, thay vì tính từng tham số bằng cách chạy lại mạng riêng lẻ.

10. Kiểm tra gradient và cập nhật tham số

10.1 Kiểm tra một gradient bằng sai phân

Đạo hàm có thể được kiểm tra gần đúng bằng sai phân trung tâm:

$$\frac{\partial L}{\partial \theta_i} \approx \frac{L(\theta_i + \epsilon) - L(\theta_i - \epsilon)}{2\epsilon}.$$

Ta giữ các tham số còn lại không đổi. Với ví dụ này, gradient theo phần tử hàng 1, cột 2 của W_1 bằng 2 (tự tính). Vì mọi pre-activation đang dương và đầu ra tuyến tính trong vùng lân cận, mất mát theo phần tử đó là hàm bậc hai; sai phân trung tâm khớp đạo hàm trong số học chính xác. NumPy với $\epsilon = 10^{-6}$ cho giá trị gần 2, sai khác nhỏ do làm tròn số dấu phẩy động (tự tính).

Sai phân hữu hạn hữu ích để phát hiện lỗi triển khai, nhưng không phải cách huấn luyện chính vì phải đánh giá mất mát nhiều lần cho mỗi tham số và sai số phụ thuộc vào ϵ. Khi kiểm một mạng, nên dùng một mô hình nhỏ và tham số tránh điểm gãy của ReLU.

10.2 Một bước gradient descent

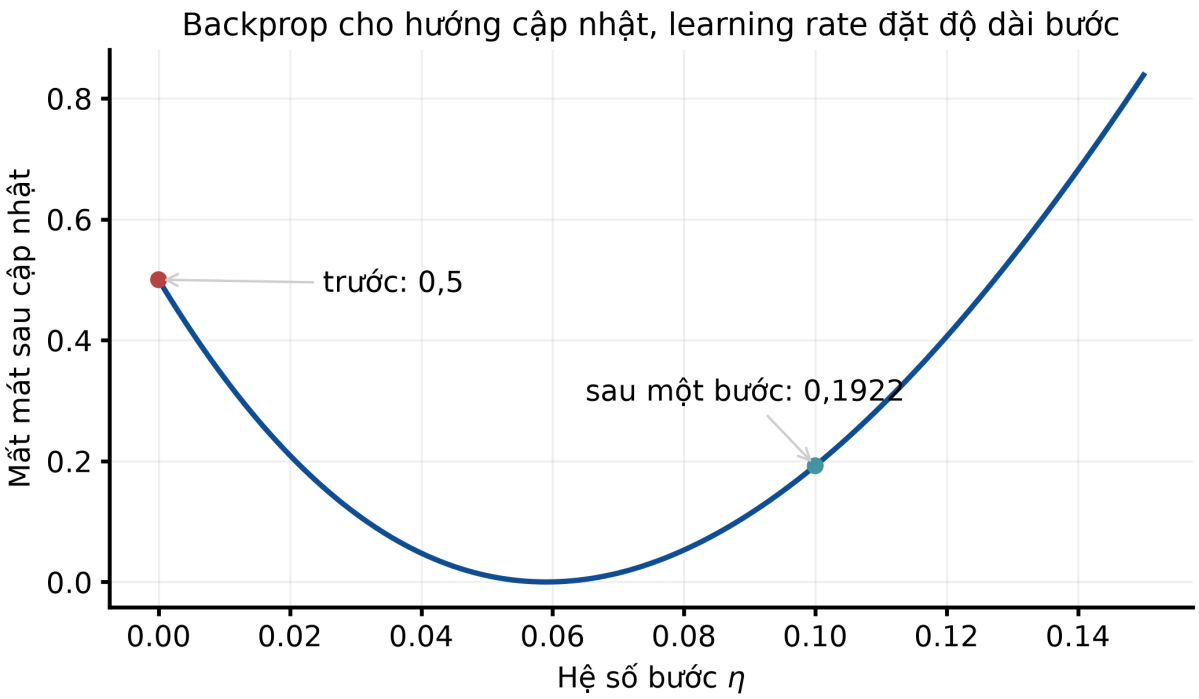
Gradient cho biết độ dốc; để cập nhật cần chọn hệ số bước η. Quy tắc gradient descent đơn giản là $\theta_{\text{mới}} = \theta - \eta \nabla_\theta L$. Với $\eta = 0{,}1$ và gradient vừa tính, ta được

$$W_{1,\text{mới}} = \begin{pmatrix} 0{,}9 & -0{,}2 \\ -0{,}1 & 0{,}8 \end{pmatrix}, \quad b_{1,\text{mới}} = \begin{pmatrix} -0{,}1 \\ -0{,}1 \end{pmatrix}, \quad v_{\text{mới}} = \begin{pmatrix} 0{,}9 \\ 0{,}8 \end{pmatrix}, \quad c_{\text{mới}} = -0{,}1.$$

Các tham số mới là tự tính. Chẳng hạn: phần tử $W_{1,12}$ chuyển từ 0 thành $0 - 0{,}1 \cdot 2 = -0{,}2$; phần tử v_2 chuyển từ 1 thành $1 - 0{,}1 \cdot 2 = 0{,}8$.

Tính lại lượt xuôi sau cập nhật: pre-activation là $(0{,}4, 1{,}4)^\top$, đều còn dương; đầu ra mới là $0{,}9 \cdot 0{,}4 + 0{,}8 \cdot 1{,}4 - 0{,}1 = 1{,}38$. Mất mát mới là $\frac{1}{2}(1{,}38 - 2)^2 = 0{,}1922$ (tự tính). Với ví dụ và bước đã chọn, mất mát giảm từ 0,5 xuống 0,1922. Một bước lớn hơn không được đảm bảo sẽ tiếp tục giảm.

Cách đọc hình: trục ngang là hệ số bước dùng đồng thời cho toàn bộ tham số, trục dọc là mất mát sau khi cập nhật rồi tính lại dự đoán. Điểm ở 0 là mất mát trước cập nhật; điểm ở 0,1 là bước đã tính tay. Đường cong chỉ đúng cho mạng và mẫu dữ liệu được chọn, không phải đường học phổ quát.



Hình 24: Mất mát sau cập nhật thay đổi theo hệ số bước trong ví dụ mạng nhỏ (minh hoạ, tự tính)

Backprop dừng ở gradient. Chọn hệ số bước, dùng momentum hay Adam, chia dữ liệu thành batch và quyết định dừng huấn luyện là các lựa chọn của tối ưu hoá. Chương nền tối ưu hoá sẽ nói riêng về những lựa chọn đó.

11. Batch dữ liệu và cách lấy trung bình

Với nhiều mẫu, ta thường cộng hoặc lấy trung bình mất mát theo batch. Đạo hàm của tổng bằng tổng đạo hàm từng mẫu; đạo hàm của trung bình là trung bình các gradient. Nếu dùng tổng thay vì trung bình, độ lớn gradient tăng theo số mẫu trong batch, nên phải tính đến quy ước đó khi đặt hệ số bước.

Với ma trận đầu vào batch X có một mẫu mỗi hàng, cách viết affine thường là $Z = XW^\top + b^\top$ nếu mỗi hàng là một vector đầu vào. Trong ví dụ trên ta dùng vector cột và công thức $z = Wx + b$. Hai quy ước tương đương sau khi chuyển vị, nhưng không được trộn quy ước giữa các dòng tính.

Tất cả mẫu trong batch cùng đi qua một bộ tham số. Gradient theo trọng số cộng đóng góp từ các mẫu, còn gradient bias cũng cộng hoặc lấy trung bình tương ứng. Nếu sử dụng mask, trọng số chia sẻ hay chuẩn hoá theo batch, đồ thị tính toán có thêm các nhánh và thao tác; quy tắc cơ bản vẫn là áp dụng đạo hàm cục bộ rồi tích lũy đóng góp.

12. Backprop, tự động vi phân và đạo hàm giải tích

Có ba cách thường được nhắc đến khi lấy đạo hàm. Đạo hàm giải tích biến đổi biểu thức bằng tay; sai phân hữu hạn ước lượng đạo hàm từ giá trị hàm ở các điểm lân cận; tự động vi phân áp quy tắc chuỗi trên các phép toán mà chương trình thực hiện. Backprop là chế độ tự động vi phân ngược cho một đầu ra scalar và nhiều đầu vào như tham số.

Vì áp dụng quy tắc chuỗi chính xác cho các phép toán đã cài, tự động vi phân khác với việc xấp xỉ đạo hàm bằng cách lấy hai giá trị gần nhau. Nó cũng khác với ký hiệu tính toán biểu tượng, vốn cố tạo ra biểu thức đạo hàm tổng quát. Hệ thống tự động vi phân lưu đồ thị hoặc giá trị trung gian, gọi các quy tắc đạo hàm cục

bộ rồi truyền độ nhạy.

Một hàm kích hoạt không khả vi tại một vài điểm vẫn có thể dùng trong tối ưu thực tế. Với ReLU tại 0, đạo hàm cổ điển không tồn tại; ta có thể chọn một phần tử của tập dưới đạo hàm $[0, 1]$, hoặc quy ước riêng của thư viện, để tính gradient. Điều này không biến hàm thành khả vi cổ điển ở mọi nơi; nó chỉ xác định quy tắc tính gradient cho thuật toán.

Khi đọc mã huấn luyện, hãy phân biệt lượt tạo dự đoán, phép tính mất mát, gọi hàm gradient tự động, và bước optimizer cập nhật tham số. Các khuôn khổ phần mềm thường gộp chúng vào vài dòng, nhưng về khái niệm đó là các phần khác nhau.

13. Điều gì có thể làm gradient khó truyền?

Qua nhiều tầng, gradient là tích của nhiều độ nhạy cục bộ. Nếu nhiều hệ số có độ lớn nhỏ hơn 1, tích có thể co lại; nếu nhiều hệ số lớn hơn 1, tích có thể tăng mạnh. Hiện tượng này được gọi tương ứng là biến mất và bùng nổ gradient. Nó liên quan tới độ sâu, khởi tạo, lựa chọn kích hoạt và cấu trúc mạng.

Sigmoid và tanh có đạo hàm nhỏ ở vùng bão hoà; ReLU có đạo hàm 0 trên nhánh âm; một chuỗi trọng số cũng có thể làm đổi thang đo tín hiệu. Kết nối residual tạo thêm đường truyền, chuẩn hoá có thể ổn định thang đo, còn khởi tạo được thiết kế để giữ phương sai ở mức hợp lý. Đây là những chủ đề riêng, không thể giải quyết chỉ bằng đổi một công thức backprop.

Gradient là thông tin cục bộ tại bộ tham số hiện tại. Nó không cho biết trực tiếp nghiệm toàn cục, không đảm bảo mỗi bước optimizer sẽ giảm mất mát và cũng không nói rằng mọi tham số cần thay đổi cùng mức. Nó là thành phần cần thiết của học dựa trên gradient, nhưng chất lượng học còn phụ thuộc dữ liệu, mục tiêu và quy tắc cập nhật.

14. Sai lầm hay gặp

- Nhầm backprop với optimizer. Backprop tính gradient; optimizer quyết định cách dùng gradient để cập nhật.
- Truyền tín hiệu xuôi khi tính gradient. Lượt ngược bắt đầu từ đầu ra mất mát và đi ngược thứ tự các phép tính.
- Quên cộng các đường nhánh. Nếu một giá trị được dùng ở nhiều nơi, gradient tại nó là tổng các đóng góp gửi về.
- Nhầm gradient theo đầu vào với gradient theo trọng số. Với lớp affine, gradient trọng số là tích ngoài giữa độ nhạy và đầu vào; gradient đầu vào dùng ma trận chuyển vị nhân độ nhạy.
- Tính gradient theo ReLU mà quên giá trị trước kích hoạt. Đạo hàm cục bộ phụ thuộc dấu của pre-activation.
- Nghĩ gradient giảm mất mát trong mọi bước. Hướng âm gradient là hướng giảm tức thời ở quy mô nhỏ; hệ số bước quá lớn hoặc tính phi tuyến có thể làm mất mát tăng.

15. Tóm tắt công thức

Nút hoặc phép toán	Lượt xuôi	Quy tắc lượt ngược
Mất mát bình phương nửa	$L = \frac{1}{2}(\hat{y} - y)^2$	$\bar{\hat{y}} = \hat{y} - y$
Đầu ra affine	$\hat{y} = v^\top h + c$	$\nabla_v L = \bar{\hat{y}} h$, $\bar{h} = \bar{\hat{y}} v$, $\bar{c} = \bar{\hat{y}}$
Kích hoạt phần tử	$h = \phi(z)$	$\bar{z} = \bar{h} \odot \phi'(z)$
Lớp affine	$z = Wa + b$	$\nabla_W L = \bar{z} a^\top$, $\nabla_b L = \bar{z}$, $\bar{a} = W^\top \bar{z}$
Gradient descent	không thuộc lượt xuôi của mạng	$\theta_{\text{mới}} = \theta - \eta \nabla_\theta L$

Nút hoặc phép toán	Lượt xuôi	Quy tắc lượt ngược
Nút cộng hoặc biến được dùng nhiều lần	tính giá trị xuôi theo đúng phép toán	cộng các gradient gửi về cùng biến

16. Bài tập tự tính

Bài 1

Với $L = \frac{1}{2}(\hat{y} - y)^2$, $\hat{y} = 5$ và $y = 3$, tính mất mát và $\partial L / \partial \hat{y}$.

Đáp án: sai số là 2, nên $L = \frac{1}{2} \cdot 2^2 = 2$ và $\partial L / \partial \hat{y} = 2$ (tự tính). NumPy xác nhận hai giá trị khi áp trực tiếp công thức.

Bài 2

Một neuron tuyến tính có $\hat{y} = w_1 x_1 + w_2 x_2 + b$, $x = (2, 3)^\top$, $w = (1, -1)^\top$, $b = 0$. Với nhãn $y = 0$ và mất mát bình phương nửa, hãy tìm dự đoán, mất mát và gradient theo w.

Đáp án: dự đoán $2 - 3 = -1$, mất mát $\frac{1}{2}(-1)^2 = 0{,}5$, độ nhạy đầu ra là -1, nên $\nabla_w L = (-1)(2, 3)^\top = (-2, -3)^\top$ (tự tính). NumPy tính trực tiếp cùng dự đoán, mất mát và vector gradient.

Bài 3

Với mạng của chương, tại $x = (1, 2)^\top$, $W_1 = I$, $b_1 = 0$, $v = (1, 1)^\top$, $c = 0$, nhãn 2, hãy nêu h, $\hat{y}$, L và $\nabla_{W_1} L$.

Đáp án: $h = (1, 2)^\top$, $\hat{y} = 3$, $L = 0{,}5$, $\nabla_{W_1} L = \begin{pmatrix} 1 & 2 \\ 1 & 2 \end{pmatrix}$ (tự tính). Tính xuôi và lượt ngược bằng các phép ma trận NumPy cho đúng các kết quả này.

Bài 4

Nếu $z = Wa + b$, độ nhạy nhận về là $\bar{z} = (2, 0)^\top$ và đầu vào $a = (3, 4, 5)^\top$, hãy tìm kích thước và giá trị $\nabla_W L$.

Đáp án: W có kích thước 2×3, gradient là tích ngoài $\bar{z} a^\top = \begin{pmatrix} 6 & 8 & 10 \\ 0 & 0 & 0 \end{pmatrix}$, cũng có kích thước 2×3 (tự tính). NumPy xác nhận từng hàng là độ nhạy neuron nhân với vector đầu vào.

Bài 5

Vì sao gradient theo đầu vào của lớp $z = Wa + b$ là $W^\top \bar{z}$, trong khi gradient theo trọng số là $\bar{z} a^\top$?

Đáp án: mỗi đầu vào ảnh hưởng đến mọi đầu ra qua các hàng của W, nên các đóng góp phải được cộng lại và phép gộp đó là $W^\top \bar{z}$. Mỗi trọng số W_{ij} chỉ tác động lên đầu ra z_i qua đầu vào a_j, nên đạo hàm là $\bar{z}_i a_j$; gom mọi cặp thành ma trận cho tích ngoài $\bar{z} a^\top$. Kích thước hai kết quả lần lượt bằng kích thước a và W.

Tài liệu tham khảo nền: Goodfellow, Bengio và Courville, *Deep Learning*; math nen06 (đạo hàm, gradient, Jacobian); math nen07 (chain rule và backprop); math 02 (automatic differentiation); ml basics nen05 (mạng nhiều lớp).

Nền tảng 07

CNN và RNN: học từ vùng lân cận và từ chuỗi

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector, nhân và cộng số, hàm kích hoạt, mạng nhiều lớp; nên xem thêm backprop ở chương nền 06

Dùng ở: bài ml basics 07 và 08 về mạng ảnh; 11, 12, 13 về chuỗi; 33 và 38 về CNN

> Đây là chương học từ đầu, không phải bản dịch hay tóm tắt một paper. Các phép tính ghi "tự tính" là ví dụ minh hoạ và đã được kiểm bằng numpy.

1. Vì sao cần hai cách nhìn này

Một câu ngắn: **CNN học cùng một phép nhận diện trên nhiều vùng lân cận; RNN xử lý từng phần của chuỗi và mang một trạng thái sang bước tiếp theo.** Cả hai vẫn là mạng có tham số được học bằng cách giảm hàm mất mát, nhưng mỗi loại tận dụng một cấu trúc khác nhau của dữ liệu.

Ảnh không chỉ là một danh sách số. Các điểm ảnh nằm cạnh nhau thường tạo thành nét, cạnh và hình dạng; cùng một cạnh có thể xuất hiện ở góc trái hoặc góc phải. Một bộ lọc nhỏ được áp lại trên ảnh cho phép mô hình tìm cùng kiểu mẫu ở nhiều vị trí. Đó là trực giác của mạng tích chập, thường gọi tắt là CNN. CNN không tự biết thế nào là con mèo: nó học các bộ lọc hữu ích từ dữ liệu và tín hiệu huấn luyện.

Câu nói hay bản ghi âm lại có thứ tự: một từ đến sau một từ khác, âm thanh hiện tại nối với âm thanh trước đó. Mạng hồi quy theo thời gian, RNN, nhận từng phần theo thứ tự; trạng thái tóm tắt thông tin đã đi qua và được cập nhật khi phần mới tới. Nó giúp mô hình đưa ngữ cảnh trước đó vào quyết định hiện tại. Chữ "hồi quy" trong tên này nói đến việc lặp lại trạng thái, không phải hồi quy tuyến tính dự đoán một con số.

CNN cũng có thể dùng cho chuỗi và RNN cũng có thể áp cho dữ liệu được duyệt theo thứ tự từ ảnh. Phân biệt theo **kiểu kết nối**, không theo tên bộ dữ liệu. Chương này làm một ảnh nhỏ bằng tay, một chuỗi nhỏ bằng tay, rồi chỉ ra giới hạn của hai trực giác ấy.

2. Nhắc nền với các số nhỏ

Một **vector** là danh sách số có thứ tự; một **ma trận** là bảng số theo hàng và cột. Ta có thể xem ảnh xám là một ma trận: mỗi ô chứa độ sáng của một điểm ảnh. Ảnh màu thông dụng còn có nhiều kênh, chẳng hạn đỏ, xanh lá và xanh dương; tại mỗi vị trí có một số cho từng kênh. Giá trị trong ví dụ là số đơn giản để tính tay, không phải quy ước duy nhất để lưu ảnh.

Tích vô hướng nhân các phần tử tương ứng rồi cộng lại. Với $(1, 2)$ và $(3, 4)$, tích là $1 \cdot 3 + 2 \cdot 4 = 11$ (tự tính). Tích này lớn khi hai vector cùng có giá trị đáng kể ở những vị trí mà trọng số coi trọng. Một bộ lọc ảnh làm phép tính tương tự với một vùng nhỏ trong ma trận.

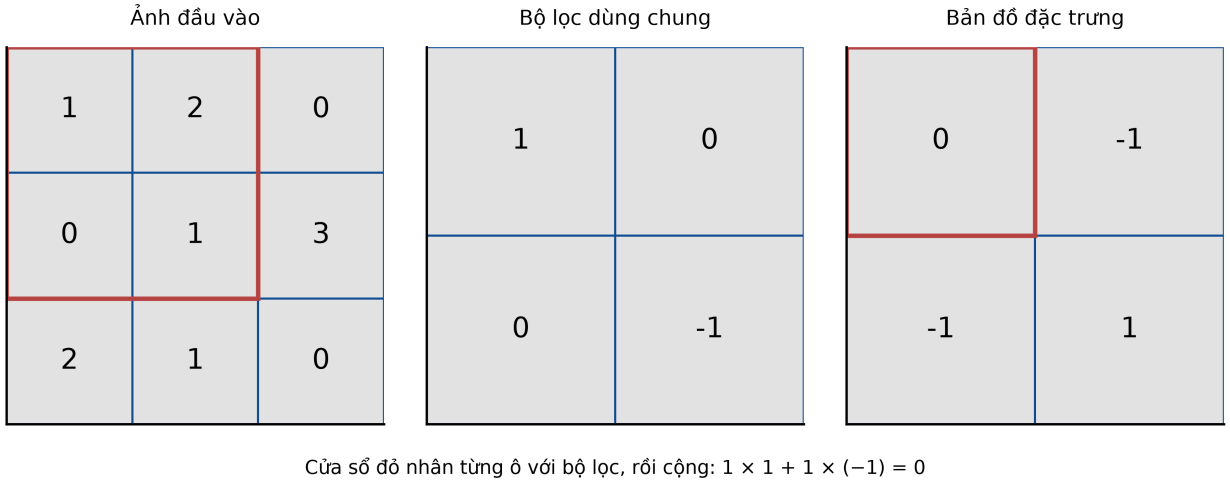
Một **hàm kích hoạt** đưa kết quả tính được qua một quy tắc, thường là phi tuyến. Ví dụ $\mathrm{ReLU}(a) = \max(0, a)$, nên $\mathrm{ReLU}(-2) = 0$ và $\mathrm{ReLU}(3) = 3$ (tự tính). Phi tuyến cho phép mạng xếp nhiều lớp biểu diễn những quan hệ phong phú hơn một phép biến đổi tuyến tính đơn.

Tham số là số được tối ưu từ dữ liệu, ví dụ các trọng số trong một bộ lọc; **đầu vào** là dữ liệu đang được xử lý, còn **trạng thái** là giá trị trung gian thay đổi theo ví dụ và, với RNN, theo từng bước. Ba loại này có thể đều là số nhưng không đóng vai trò giống nhau.

Với chuỗi x_1, x_2, x_3, chỉ số cho biết thứ tự. Nếu gán $(x_1, x_2, x_3) = (1, 2, 0)$ thì bước đầu nhìn 1, sau đó nhìn 2, cuối cùng nhìn 0. Đổi thứ tự tạo một chuỗi khác, dù tập các số vẫn như cũ (tự tính). Điều này quan trọng với câu như “chó cắn người” so với “người cắn chó”.

3. CNN trước công thức: một cửa sổ đi qua ảnh

Hãy tưởng tượng một tấm kính nhỏ đặt lên góc ảnh. Ta nhân từng ô dưới kính với một số tương ứng trên kính, cộng lại và ghi kết quả vào một ô của bảng mới. Sau đó dịch kính sang vị trí kế tiếp và **dùng lại chính các số trên kính**. Tấm kính là bộ lọc, hay kernel; bảng kết quả là bản đồ đặc trưng. Việc dùng lại trọng số là **chia sẻ tham số**.



Hình 25: Cửa sổ và bộ lọc tạo bản đồ đặc trưng cho ảnh nhỏ (minh hoạ, tự tính)

Cách đọc hình: bảng trái là ảnh, giữa là một bộ lọc, phải là các kết quả. Viền đỏ trên ảnh chọn góc trên trái, còn ô đỏ của bảng phải chứa kết quả của riêng vùng đó. Dịch cửa sổ lần lượt sang phải và xuống dưới vẫn dùng cùng bộ lọc; toàn bộ phép tính có tại mục 5.

Một bộ lọc chỉ tạo ra một loại đáp ứng. Mạng thường học nhiều bộ lọc và nhận nhiều bản đồ đặc trưng: một bộ có thể phản ứng với kiểu chuyển sáng tối này, bộ khác phản ứng với kiểu khác. Diễn giải từng bộ như một khái niệm đặt tên sẵn có thể hữu ích, nhưng không có đảm bảo rằng mọi bộ lọc đều tương ứng một vật thể dễ gọi tên.

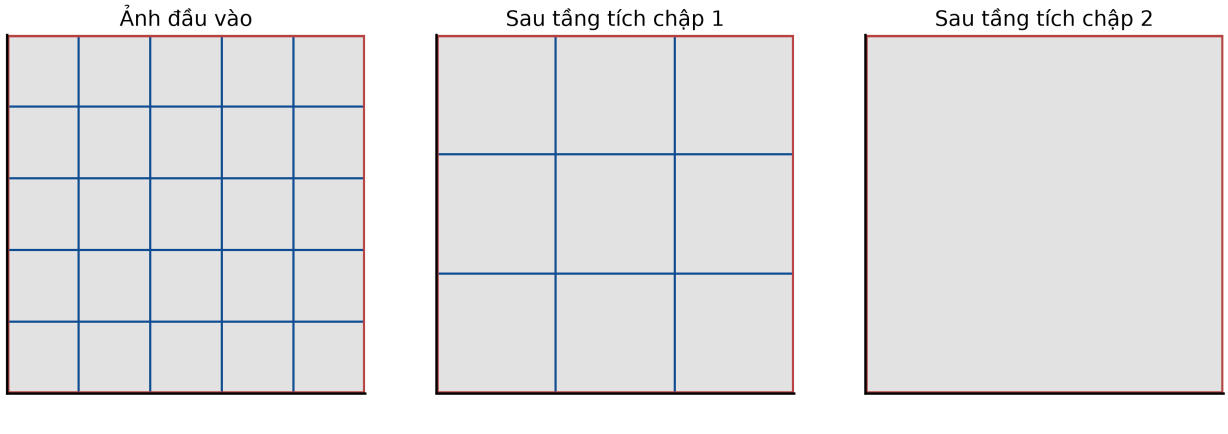
Hình tiếp theo cho thấy vì sao xếp các tầng cục bộ vẫn có thể tạo thông tin về một vùng rộng. Đầu ra một tầng phụ thuộc vào vùng nhỏ của tầng trước; đầu ra tầng sau phụ thuộc vào nhiều vùng nhỏ đã chồng lấn.

Cách đọc hình: ảnh bên trái có năm hàng và năm cột, tầng giữa có ba hàng và ba cột, còn đầu ra bên phải có một ô. Với bộ lọc ba nhân ba, bước một và không đệm, mỗi ô tầng giữa nhìn một vùng ba nhân ba của ảnh. Ô cuối nhìn tất cả chín ô tầng giữa, nên vùng đầu vào có thể ảnh hưởng nó là toàn bộ bảng năm nhân năm. Đó là **trường nhìn lý thuyết**, không có nghĩa mọi ô trong vùng tác động mạnh bằng nhau.

4. CNN: định nghĩa và các lựa chọn kích thước

Với ảnh một kênh X, bộ lọc K có chiều cao k_h và chiều rộng k_w, bước dịch s, chưa thêm đệm, ta định nghĩa tại vị trí đầu ra (i, j):

$$Y_{i,j} = b + \sum_{u=0}^{k_h-1} \sum_{v=0}^{k_w-1} K_{u,v} X_{is+u,\, js+v}.$$



Hình 26: Trường nhìn mở rộng khi xếp hai tầng tích chập (minh hoạ, tự tính)

Ở đây b là bias chung cho bản đồ do bộ lọc này tạo ra, và các chỉ số bắt đầu từ 0. Sau đó có thể áp hàm kích hoạt từng ô: $H_{i,j} = \mathrm{ReLU}(Y_{i,j})$. Ở đây từ tích chập theo thói quen deep learning chỉ phép **tương quan chéo**: không lật bộ lọc trước khi nhân. Trong định nghĩa tích chập toán học, kernel được lật; khi kernel được học, cả hai quy ước vẫn có thể biểu diễn cùng họ phép biến đổi nếu dùng nhất quán.

Bước dịch (stride) là số ô dịch cửa sổ giữa hai vị trí liên tiếp. Bước lớn hơn thường cho ít vị trí đầu ra hơn. **Đệm** (padding) thêm các ô ở biên, thường là số 0, trước khi quét; điều này ảnh hưởng cách xử lý mép ảnh và kích thước. Đệm không có nghĩa là ta quan sát thêm nội dung ảnh thật ở ngoài biên.

Với một chiều đầu vào dài n, bộ lọc dài k, đệm p ô ở mỗi đầu, bước s và không giãn kernel, kích thước chiều đầu ra là

$$n_{\text{ra}} = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1.$$

Dấu sàn nghĩa là lấy số nguyên không vượt quá giá trị bên trong. Công thức áp riêng cho chiều cao và chiều rộng, giả sử cửa sổ hợp lệ và phần tử lọc cách nhau một ô. Ví dụ ảnh dài 5, kernel dài 3, $p = 0$, $s = 1$ cho $(5 - 3)/1 + 1 = 3$ vị trí; cùng ảnh với $p = 1$ cho 5 vị trí (tự tính). Có nhiều quy ước padding tự động, nhất là khi stride lớn hơn một; hãy kiểm định nghĩa của thư viện thay vì đoán.

Với ảnh nhiều kênh, mỗi bộ lọc có trọng số ở **mọi kênh đầu vào**, và kết quả cộng qua cả kênh. Ví dụ một ảnh RGB có ba kênh thì một kernel không phải chỉ là một bảng hai chiều độc lập: nó có một bảng trọng số cho mỗi kênh rồi cộng ba kết quả. Số **bộ lọc đầu ra** quyết định số kênh đầu ra. Mỗi bộ lọc có một bias nếu lớp dùng bias. Nếu không dùng nhóm hay chia kênh đặc biệt, số tham số là

$$k_h k_w C_{\text{vào}} C_{\text{ra}} + C_{\text{ra}}.$$

Ví dụ kernel hai nhân hai, ba kênh đầu vào, hai kênh đầu ra có $2 \cdot 2 \cdot 3 \cdot 2 + 2 = 26$ tham số kể cả bias (tự tính). Số tham số không nhân với số vị trí đầu ra vì cùng trọng số được chia sẻ theo vị trí. Tính toán thực tế vẫn tăng khi phải áp bộ lọc ở nhiều vị trí.

5. Tính tay CNN: một bộ lọc, bốn vị trí

Các số ở đây khớp hình cửa sổ tích chập và đều là ví dụ (tự tính), đã kiểm bằng numpy. Cho ảnh một kênh và bộ lọc

$$X = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 0 \end{pmatrix}, \qquad K = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

Chọn $s = 1$, $p = 0$, $b = 0$, dùng quy ước không lật bộ lọc. Đầu vào có ba hàng, bộ lọc có hai hàng, nên mỗi chiều đầu ra có $(3 - 2)/1 + 1 = 2$ vị trí. Ta nhận một bảng hai nhân hai.

Bước 1, góc trên trái. Cửa sổ là $\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$. Nhân theo vị trí và cộng: $1 \cdot 1 + 2 \cdot 0 + 0 \cdot 0 + 1 \cdot (-1) = 0$. Giá trị ghi vào $Y_{0,0}$ là 0.

Bước 2, góc trên phải. Cửa sổ là $\begin{pmatrix} 2 & 0 \\ 1 & 3 \end{pmatrix}$. Ta được $2 \cdot 1 + 0 \cdot 0 + 1 \cdot 0 + 3 \cdot (-1) = -1$, nên $Y_{0,1} = -1$.

Bước 3, góc dưới trái. Cửa sổ là $\begin{pmatrix} 0 & 1 \\ 2 & 1 \end{pmatrix}$. Ta được $0 \cdot 1 + 1 \cdot 0 + 2 \cdot 0 + 1 \cdot (-1) = -1$, nên $Y_{1,0} = -1$.

Bước 4, góc dưới phải. Cửa sổ là $\begin{pmatrix} 1 & 3 \\ 1 & 0 \end{pmatrix}$. Ta được $1 \cdot 1 + 3 \cdot 0 + 1 \cdot 0 + 0 \cdot (-1) = 1$, nên $Y_{1,1} = 1$.

Ghép kết quả và, nếu muốn, áp ReLU:

$$Y = \begin{pmatrix} 0 & -1 \\ -1 & 1 \end{pmatrix}, \qquad \mathrm{ReLU}(Y) = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

Bộ lọc cụ thể này lấy ô trên trái trừ ô dưới phải của từng cửa sổ. Số âm không phải lỗi: nó cho biết thứ tự chênh lệch ngược với hướng được bộ lọc coi là dương. ReLU trong ví dụ làm mất các giá trị âm, do đó bảng sau kích hoạt không giữ được toàn bộ thông tin của bảng trước. Thông thường trọng số trong K không được chọn bằng tay như ở đây mà được học từ dữ liệu.

6. Các biến thể và chi tiết CNN dễ bỏ sót

6.1 Pooling và độ phân giải

Một lớp **pooling** gộp các giá trị trong một vùng. Max pooling lấy giá trị lớn nhất, average pooling lấy trung bình. Ví dụ trên bảng $\begin{pmatrix} 0 & -1 \\ -1 & 1 \end{pmatrix}$, max pooling trên cả bảng cho 1, còn average pooling cho $-0{,}25$ (tự tính). Gộp làm giảm số vị trí và bỏ một phần thông tin vị trí; đó là một lựa chọn kiến trúc chứ không phải bước bắt buộc của mọi CNN.

CNN có thể giảm kích thước bằng stride, pooling hoặc cách khác. Giảm quá sớm có thể bỏ chi tiết cần cho dự đoán từng điểm ảnh. Ngược lại, giữ mọi vị trí ở mọi tầng có thể tốn bộ nhớ và tính toán. Nhiệm vụ phân loại cả ảnh và nhiệm vụ tìm biên chính xác có nhu cầu khác nhau.

6.2 Dịch chuyển, tính tương đương và bất biến

Nếu dịch một mẫu và áp cùng kernel, đáp ứng cũng dịch theo trong vùng mà mọi cửa sổ hợp lệ. Ta gọi đó là **tương đương theo dịch chuyển**: đầu ra thay đổi có cấu trúc cùng đầu vào. **Bất biến** là đầu ra không đổi khi đầu vào thay đổi. Một bản đồ tích chập không tự bất biến với dịch chuyển vì vị trí đáp ứng thường cũng dịch. Pooling hoặc tổng hợp toàn cục có thể tạo bớt nhạy vị trí; mép ảnh, padding và stride có thể phá sự tương đương chính xác.

Hai tính chất này giúp hiểu vì sao chia sẻ kernel phù hợp khi cùng đặc trưng có thể xuất hiện tại nhiều nơi. Chúng không bảo đảm nhận diện tốt mọi phép xoay, thay đổi độ sáng hoặc vật bị che. Việc đó còn phụ thuộc dữ liệu, thiết kế và cách huấn luyện.

6.3 Trường nhìn, kênh và độ sâu

Trường nhìn của một ô là vùng đầu vào **có đường kết nối** đến ô đó. Hai tầng kernel ba nhân ba, stride một, không giãn có trường nhìn năm nhân năm ở đầu vào, như hình trường nhìn ở trên: tầng sau nhìn ba ô liên tiếp ở tầng trước, mỗi ô trước lại nhìn ba ô, tổng độ rộng là $3 + (3 - 1) = 5$ (tự tính). Khi có stride hoặc giãn kernel, cần tính khoảng cách giữa các ô liên quan, không chỉ cộng kích thước kernel.

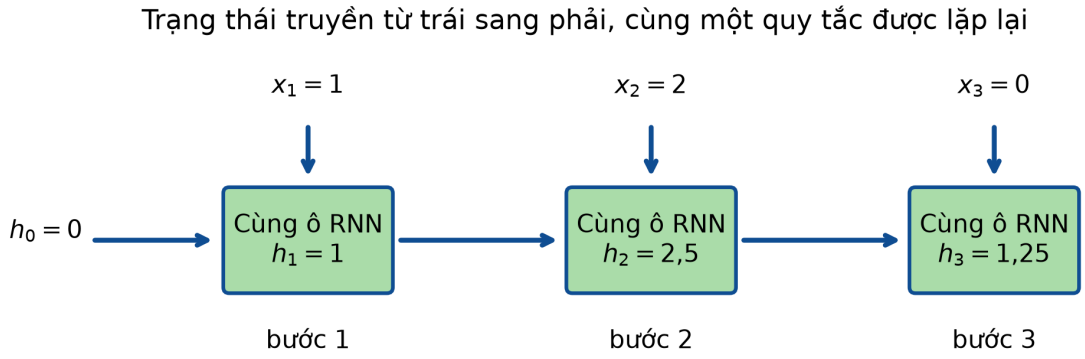
Nhiều kênh không đồng nghĩa nhiều điểm ảnh trong không gian. Sau một tầng, cùng một vị trí có thể có nhiều số mô tả các đáp ứng khác nhau. Việc xếp tầng cho phép tổ hợp các đáp ứng cục bộ thành biểu diễn có ngữ cảnh rộng hơn, nhưng không bắt buộc tầng đầu là cạnh và tầng sau là vật hoàn chỉnh trong mọi mạng.

6.4 Skip connection

Mạng sâu có thể dùng **đường tắt** cộng đầu vào của một khối với biến đổi của nó, dạng $y = x + F(x)$ khi kích thước tương thích. Điều này tạo thêm đường truyền tín hiệu và gradient, là trực giác cần có trước khi đọc ResNet. Nó không bảo đảm mọi mạng sâu sẽ dễ huấn luyện hoặc bỏ qua mọi vấn đề tối ưu. Xem bài ml basics 07 để hiểu thiết kế và thực nghiệm cụ thể.

7. RNN trước công thức: một ô được mở theo thời gian

Hãy đọc một câu từng từ. Sau mỗi từ, ta cập nhật ghi chú ngắn về phần đã đọc; khi từ kế tiếp đến, ta đọc nó cùng ghi chú cũ để viết ghi chú mới. RNN làm một việc có cấu trúc như vậy: đầu vào tại bước t và trạng thái từ bước trước cùng xác định trạng thái hiện tại. Trạng thái là một vector do mạng học cách sử dụng, không phải một bản tóm tắt bằng tiếng Việt có thể luôn giải thích được.



Hình 27: Một ô RNN được mở ra theo ba bước của chuỗi (minh hoạ, tự tính)

Cách đọc hình: các ô xanh là ba lần áp dụng cùng quy tắc, không phải ba bộ tham số độc lập. Đầu vào đi từ trên xuống, còn trạng thái đi từ trái sang phải. Với quy tắc số ở mục 9, trạng thái lần lượt là 1, $2{,}5$ và $1{,}25$; trạng thái đầu bằng 0 là lựa chọn của ví dụ.

Cách vẽ này gọi là **mở cuộn theo thời gian**. Nó không tạo ra ba mạng được huấn luyện riêng: cùng trọng số được dùng lặp tại từng bước. Số lần áp quy tắc có thể thay đổi theo độ dài chuỗi, mặc dù cách tổ chức batch và phần đệm trong chương trình cần được xử lý cẩn thận.

8. RNN: định nghĩa trạng thái và đầu ra

Một RNN cơ bản có thể viết

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b_h), \qquad o_t = W_o h_t + b_o.$$

Ở đây x_t là đầu vào ở bước t; h_t là trạng thái ẩn; W_x, W_h, W_o là các ma trận được học; b_h, b_o là bias; ϕ thường là một hàm kích hoạt như tanh. Tuỳ nhiệm vụ, o_t có thể đi qua softmax để biểu diễn xác suất theo từ vựng hoặc qua lớp khác để dự đoán. Ở dạng vector cột, nếu x_t có d phần tử và h_t có m phần tử thì W_x có cỡ $m \times d$, W_h có cỡ $m \times m$.

Ở mỗi bước, $W_x x_t$ đưa dữ liệu mới vào, $W_h h_{t-1}$ đưa ảnh hưởng trạng thái cũ vào, và bias cho phép dịch kết quả trước kích hoạt. Cùng một W_x, W_h, b_h dùng tại tất cả các bước của **cùng lớp**. Chia sẻ này tương tự ý tưởng áp cùng bộ lọc ở các vị trí ảnh, nhưng thứ tự phụ thuộc của RNN là tuần tự.

Trạng thái ban đầu h_0 có thể đặt bằng 0, được học, hoặc nhận từ một thành phần khác. Việc đặt bằng 0 không nói dữ liệu đầu tiên là 0: đó chỉ là giá trị khởi đầu của bộ nhớ mô hình. Nếu hai chuỗi độc lập nằm cạnh nhau trong batch thì cần quyết định có đặt lại trạng thái giữa chúng hay không; vô tình giữ trạng thái của một chuỗi có thể làm rò rỉ thông tin sang chuỗi sau.

Đầu ra có thể dùng ở **mỗi bước**, ví dụ gán nhãn cho từng từ, hoặc chỉ tại **bước cuối**, ví dụ phân loại một chuỗi dựa trên h_T. Cũng có thể có bộ mã hoá và bộ giải mã cho chuỗi đầu vào, đầu ra khác độ dài. Không có một quy tắc duy nhất rằng số bước vào phải bằng số bước ra.

9. Tính tay RNN: trạng thái tích luỹ và quên dần

Để tính bằng tay, xét một phiên bản tuyến tính với trạng thái và đầu vào là số đơn:

$$h_t = \frac{1}{2} h_{t-1} + x_t, \qquad h_0 = 0, \qquad (x_1, x_2, x_3) = (1, 2, 0).$$

Đây là trường hợp đặc biệt với ϕ là hàm đồng nhất. RNN thực tế thường dùng vector và phi tuyến; chọn quy tắc đơn giản này giúp thấy rõ phép truyền trạng thái. Mọi số sau đây đều là ví dụ (tự tính), đã kiểm bằng numpy.

Bước 1. Thay trạng thái ban đầu và đầu vào thứ nhất: $h_1 = \frac{1}{2} \cdot 0 + 1 = 1$. Trạng thái sau bước này giữ thông tin từ x_1.

Bước 2. Thay h_1 và x_2: $h_2 = \frac{1}{2} \cdot 1 + 2 = 2{,}5$. Một nửa trạng thái cũ được cộng với đầu vào mới. Trạng thái $2{,}5$ không chỉ là đầu vào thứ hai: nó chứa ảnh hưởng của cả hai bước.

Bước 3. Với $x_3 = 0$, $h_3 = \frac{1}{2} \cdot 2{,}5 + 0 = 1{,}25$. Không có đầu vào mới khác 0 nhưng trạng thái vẫn còn khác 0. Nếu bỏ trạng thái cũ trước mỗi bước, kết quả sẽ khác.

Thay lùi từng bước cho thấy rõ nguồn đóng góp:

$$h_3 = \tfrac{1}{4} x_1 + \tfrac{1}{2} x_2 + x_3 + \tfrac{1}{8} h_0 = \tfrac{1}{4} + 1 + 0 + 0 = 1{,}25.$$

Đầu vào cũ hơn trong quy tắc này bị nhân hệ số nhỏ hơn: ảnh hưởng của x_1 lên h_3 là $\frac{1}{4}$, còn ảnh hưởng của x_2 là $\frac{1}{2}$. Đây là kết luận cho ví dụ đã chọn, không phải mô tả đầy đủ mọi RNN. Một mô hình có cổng có thể học những cách giữ hoặc thay đổi trạng thái phức tạp hơn.

Nếu đổi thứ tự thành $(2, 1, 0)$, cùng quy tắc cho $h_1 = 2$, $h_2 = 2$, $h_3 = 1$ (tự tính). Cùng ba số nhưng kết quả khác, vì ảnh hưởng được nhân khác nhau theo vị trí. Đây là một cách nhìn cụ thể về ý nghĩa của thứ tự.

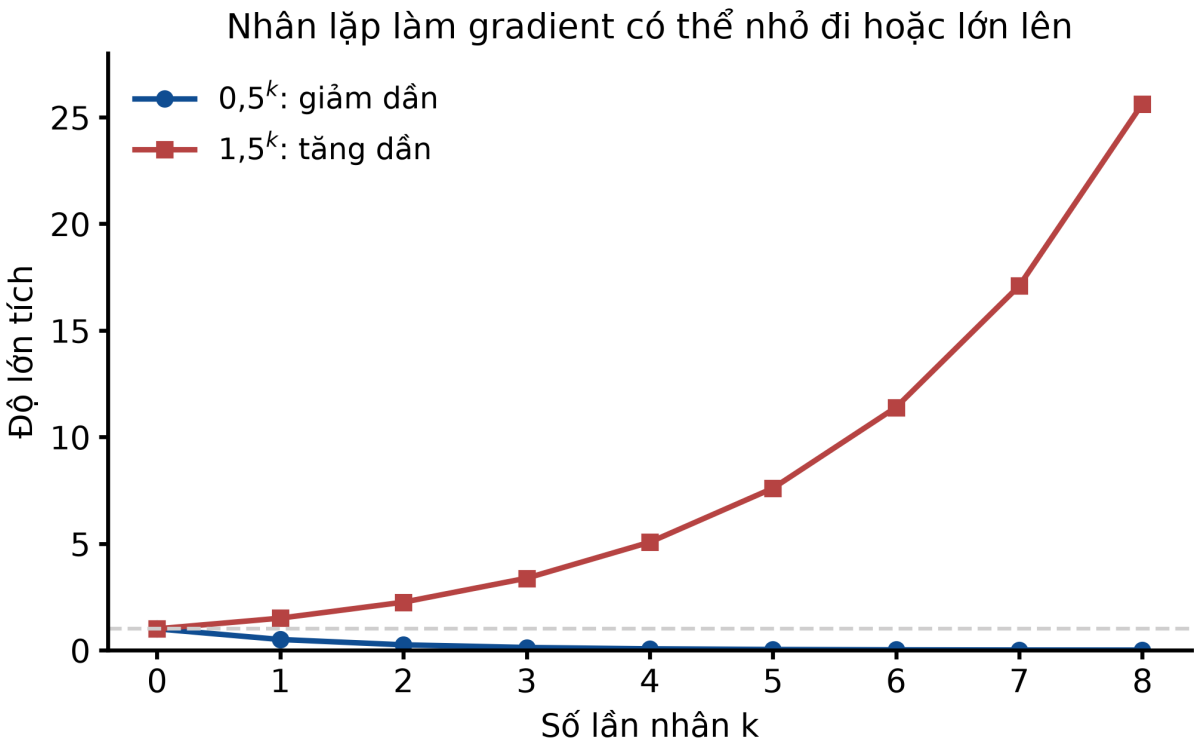
10. Huấn luyện qua thời gian và các biến thể RNN

10.1 Backpropagation through time

Khi đã mở RNN thành chuỗi các phép tính, ta lan truyền gradient ngược qua từng bước, gọi là **backpropagation through time**, viết tắt BPTT. Vì cùng tham số xuất hiện nhiều lần, đóng góp gradient của các lần sử dụng phải được cộng lại trước khi cập nhật. Quy tắc chain rule giống backprop của mạng nhiều lớp; điều mới là cùng một tham số được dùng ở nhiều thời điểm.

Trong ví dụ tuyến tính trên, giữ các đầu vào cố định thì $\frac{\partial h_t}{\partial h_{t-1}} = \frac{1}{2}$. Vì thế $\frac{\partial h_3}{\partial h_0} = (\frac{1}{2})^3 = \frac{1}{8} = 0{,}125$ (tự tính). Đạo hàm nhỏ nói rằng thay đổi rất nhỏ của trạng thái ban đầu chỉ ảnh hưởng ít tới trạng thái cuối theo quy tắc này. Nó không có nghĩa trạng thái ban đầu hay một từ cũ luôn không liên quan trong mọi chuỗi.

10.2 Gradient mất dần và bùng nổ



Hình 28: Tích các hệ số nhỏ hơn hoặc lớn hơn một qua nhiều bước (minh hoạ, tự tính)

Cách đọc hình: trục ngang là số lần nhân, trục đứng là độ lớn tích. Đường xanh giảm vì mỗi lần nhân với $0{,}5$, đường đỏ tăng vì mỗi lần nhân với $1{,}5$. Đây là mô hình số đơn giản cho hiện tượng gradient mất dần hoặc bùng nổ, chứ không phải đồ thị huấn luyện của một mạng cụ thể.

Trong RNN vector, gradient liên quan đến tích các ma trận Jacobian qua thời gian và còn chịu ảnh hưởng của hàm kích hoạt, đầu vào và hướng vector. Không thể kết luận chính xác chỉ từ một trọng số riêng lẻ. Tuy vậy, hình dung nhiều lần nhân giúp hiểu vì sao tín hiệu học từ xa có thể khó giữ. Gradient clipping giới hạn độ lớn gradient khi nó quá lớn; nó không tự khôi phục tín hiệu đã mất vì quá nhỏ.

10.3 LSTM, GRU và hai chiều

LSTM và **GRU** thêm các cổng học được để điều khiển thông tin được giữ, cập nhật hoặc đưa ra. Một trực giác là thay vì luôn nhân trạng thái cũ với cùng một hệ số cố định như ví dụ, mô hình có những đường và hệ số phụ thuộc vào dữ liệu. Cổng giúp tạo đường truyền thuận lợi hơn cho thông tin xa, nhưng không bảo đảm

nhớ được mọi khoảng cách hay loại bỏ mọi khó khăn tối ưu. Bài ml basics 13 trình bày chi tiết phép tính và so sánh.

Một RNN **hai chiều** có thể xử lý chuỗi theo cả chiều trái sang phải và phải sang trái rồi ghép biểu diễn. Điều đó hữu ích khi toàn bộ chuỗi đã có sẵn, nhưng trạng thái tại một vị trí có thể phụ thuộc phần sau vị trí ấy. Vì vậy không được coi đầu ra hai chiều là dự đoán chỉ dựa trên quá khứ trong bài toán thời gian thực.

Một **seq2seq** dùng bộ mã hoá xử lý đầu vào và bộ giải mã tạo đầu ra. Attention cho bộ giải mã cách tham chiếu tới nhiều trạng thái thay vì chỉ dựa vào một vector tóm tắt cố định; bài ml basics 11 và 12 phát triển hai ý này. RNN không phải lựa chọn duy nhất cho chuỗi: cách xây dựng mô hình phải hợp với dữ liệu và yêu cầu suy luận.

10.4 Chuỗi dài, batch và mặt nạ

Các chuỗi trong cùng batch có thể khác độ dài, nên thường được đệm đến cùng kích thước. Mặt nạ cho biết vị trí nào là dữ liệu thật, vị trí nào là phần đệm; mất mát và phép tổng hợp cần tôn trọng mặt nạ. Nếu gộp cả phần đệm như quan sát thật, mô hình và phép đánh giá có thể bị lệch.

BPTT cắt cụt chỉ lan truyền gradient qua một cửa sổ hữu hạn để giảm chi phí bộ nhớ. Trạng thái thuận có thể tiếp tục mang thông tin từ trước cửa sổ, nhưng gradient của cửa sổ hiện tại không được lan xuyên qua điểm cắt về các bước trước đó. Đó là đánh đổi giữa chi phí và phạm vi tín hiệu học, không phải bằng chứng rằng thông tin cũ đã biến mất khỏi trạng thái.

11. Nối sang các bài AI

Nhận dạng ảnh. Bài VGG xếp nhiều tầng tích chập nhỏ; dùng trực giác cửa sổ và trường nhìn để đọc cách một đơn vị tầng sau có thể dựa trên vùng ảnh rộng hơn. Số tầng, bộ lọc, dữ liệu và kết quả cụ thể nằm trong bài ml basics 08, không suy ra từ ví dụ hai nhân hai ở đây.

Mạng rất sâu. Bài ResNet đưa đường tắt vào các khối CNN. Hãy tách câu hỏi “kiến trúc biểu diễn được gì” khỏi “tối ưu có tìm được tham số đó không”; cả hai đều ảnh hưởng kết quả. Mô hình một bộ lọc trong mục 5 chỉ minh hoạ phép cục bộ, không mô phỏng toàn bộ ResNet.

Dịch máy và căn chỉnh. Trong seq2seq, một bộ đọc chuỗi nguồn và bộ khác tạo chuỗi đích. Attention cho phép mỗi bước giải mã dùng thông tin từ các vị trí nguồn khác nhau. Trước khi đọc bài ml basics 11 và 12, hãy chỉ được đâu là x_t, đâu là trạng thái, đầu ra nào có sẵn khi dự đoán.

Ngôn ngữ và âm thanh. Đầu vào có thể là vector từ hoặc vector đặc trưng âm thanh thay vì một số. Cùng công thức RNN, kích thước vector, cách tạo biểu diễn và hàm mất mát quyết định bài toán cụ thể. Không nên coi chỉ số thời gian trong công thức là một khoảng thời gian vật lý bằng nhau trong mọi ứng dụng.

Các CNN hiện đại. Những thiết kế như ConvNeXt vẫn dùng phép tích chập nhưng có nhiều lựa chọn khác về khối mạng, chuẩn hoá và huấn luyện. Đừng đồng nhất CNN với đúng sơ đồ tích chập rồi pooling đơn giản ở đây; xem bài ml basics 38 để đối chiếu kiến trúc đầy đủ.

12. Sai lầm hay gặp

- **Nghĩ mỗi vị trí ảnh học một bộ trọng số riêng.** Một kernel thông thường được chia sẻ trên các vị trí; các kernel khác nhau vẫn có các trọng số khác nhau.
- **Nhầm tích chập với tích vô hướng của toàn ảnh.** Mỗi ô đầu ra nhìn một vùng và tất cả kênh liên quan, không nhất thiết nhìn cả ảnh ngay tại tầng đầu.
- **Coi tương đương dịch chuyển là bất biến tuyệt đối.** Bản đồ thường dịch theo đầu vào, và biên, đệm, stride có thể làm tính chất không còn chính xác.
- **Bỏ qua kích thước và bias.** Kernel, số kênh, padding và stride cùng quyết định dạng đầu ra và số tham số. Lớp không dùng bias sẽ không có số hạng bias trong công thức đếm.

- **Coi mỗi ô mở cuộn là một bộ tham số mới.** RNN chia sẻ tham số theo bước; trạng thái và đầu vào thay đổi còn trọng số của lớp được dùng lại.
- **Nghĩ trạng thái là bản ghi đầy đủ hoặc LSTM nhớ vô hạn.** Trạng thái có kích thước hữu hạn và được học cho nhiệm vụ; cổng giúp nhưng không bảo đảm giữ mọi chi tiết xa.

13. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Tương quan chéo	$Y_{i,j} = b + \sum_{u,v} K_{u,v} X_{is+u,js+v}$	cùng bộ lọc quét từng cửa sổ, một kênh
Kích thước đầu ra	$\lfloor (n + 2p - k)/s \rfloor + 1$	một chiều, kernel không giãn
Số tham số CNN	$k_h k_w C_{\text{vào}} C_{\text{ra}} + C_{\text{ra}}$	lớp thường có bias, không nhóm kênh
ReLU	$\max(0, a)$	giữ số dương, đưa số âm về 0
RNN	$h_t = \phi(W_x x_t + W_h h_{t-1} + b_h)$	cập nhật trạng thái bằng dữ liệu và trạng thái cũ
Ví dụ RNN	$h_t = \frac{1}{2} h_{t-1} + x_t$	quy tắc một chiều dùng ở mục 9
Gradient ví dụ	$\partial h_T / \partial h_0 = (\frac{1}{2})^T$	tín hiệu giảm theo số bước trong ví dụ

14. Bài tập tự tính

Tự làm trước khi mở đáp án. Các phép tính và đáp án đã kiểm bằng numpy; mỗi bài dùng giả định ghi ngay trong đề.

Bài 1 (dễ). Ảnh một kênh kích thước năm nhân năm, kernel ba nhân ba, stride một, không đệm. Bản đồ đầu ra có bao nhiêu hàng và cột?

Đáp án. Mỗi chiều có $(5 - 3)/1 + 1 = 3$ vị trí; đầu ra ba nhân ba (tự tính). Có chín vị trí, nhưng chỉ một bộ trọng số của kernel được dùng lặp lại.

Bài 2 (dễ vừa). Với $X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $K = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$, bias 0 và không lật kernel, tính một ô đầu ra, rồi áp ReLU.

Đáp án. Nhân rồi cộng được $1 \cdot 1 + 2 \cdot 0 + 3 \cdot 0 + 4 \cdot (-1) = -3$; ReLU cho 0 (tự tính). Có đúng một cửa sổ vì ảnh và kernel cùng kích thước.

Bài 3 (vừa). Một lớp tích chập thường có kernel ba nhân ba, hai kênh vào, bốn kênh ra và bias cho mỗi kênh ra. Có bao nhiêu tham số?

Đáp án. Mỗi bộ lọc có $3 \cdot 3 \cdot 2 = 18$ trọng số. Bốn bộ có $18 \cdot 4 = 72$ trọng số; cộng bốn bias được 76 tham số (tự tính). Kích thước không gian của ảnh không xuất hiện trong phép đếm tham số chia sẻ này.

Bài 4 (vừa). Cho $h_t = \frac{1}{2} h_{t-1} + x_t$, $h_0 = 0$ và đầu vào $(2, 0, 1)$. Tính ba trạng thái.

Đáp án. $h_1 = 2$, $h_2 = \frac{1}{2} \cdot 2 + 0 = 1$, $h_3 = \frac{1}{2} \cdot 1 + 1 = 1{,}5$ (tự tính). Dù bước hai nhận 0, trạng thái vẫn chứa ảnh hưởng của bước một.

Bài 5 (khó hơn). Với $h_t = a h_{t-1} + x_t$, coi các x_t không đổi, $a = 0{,}5$ và $h_0 = 0$. Tính $\partial h_4 / \partial h_1$ và giải thích nếu tăng khoảng cách thì điều gì xảy ra trong quy tắc này.

Đáp án. Có ba cạnh từ h_1 qua h_2, h_3 tới h_4. Mỗi cạnh có đạo hàm a, nên chain rule cho $a^3 = 0{,}125$ (tự tính). Nếu khoảng cách tăng thì tích các hệ số $0{,}5$ giảm dần về 0; điều này chỉ chứng minh hiện tượng cho quy tắc tuyến tính cụ thể, không cho mọi RNN.

Tài liệu tham khảo nền: Goodfellow, Bengio & Courville, Deep Learning; Dumoulin & Visin, A Guide to Convolution Arithmetic for Deep Learning; ghi chú ml basics 07, 08, 11, 12 và 13.

Nền tảng 08

Mô hình sinh: học cách tạo dữ liệu bằng mã, đối kháng và khử nhiễu

Loại: chương nền tảng (không phải ghi chú paper)

Đọc trước: vector và hàm mất mát ở mức cơ bản; xác suất, phân phối Gaussian và mạng nhiều lớp sẽ được nhắc lại nhanh

Dùng ở: bài ml basics 14 (GAN), 15 (VAE), 16 (diffusion), 26 (học biểu diễn) và 40 (không gian biểu diễn)

> Chương này dạy trực giác từ đầu, không phải bản dịch hay tóm tắt paper. Các phép tính ghi "tự tính" là ví dụ minh hoạ và đã được kiểm bằng numpy.

1. Vì sao cần mô hình sinh

Một câu ngắn: **mô hình sinh học một quy tắc để tạo mẫu có cấu trúc giống loại dữ liệu đã học**. Nếu dữ liệu là ảnh khuôn mặt, mẫu sinh cũng có dạng một ảnh; nếu dữ liệu là âm thanh, mẫu sinh có thể là một đoạn sóng. Mục tiêu cụ thể phụ thuộc bài toán: tạo mẫu mới, hoàn thiện phần thiếu, tìm biểu diễn hoặc ước lượng xác suất.

Hãy so với bài toán nhận dạng: đưa ảnh vào, nhận một nhãn. Một mô hình sinh có thể đi theo chiều khác, lấy một nguồn ngẫu nhiên hoặc điều kiện làm đầu vào và tạo ra dữ liệu. Tuy nhiên phân biệt này không tuyệt đối: một mô hình có thể vừa có bộ phận sinh vừa có bộ phận nhận dạng, còn chất lượng phân loại và chất lượng mẫu sinh là hai điều khác nhau.

Chương này đi qua bốn ý tưởng bằng bốn hình: **autoencoder** nén rồi tái tạo; **GAN** có bộ sinh và bộ phân biệt; **VAE** mô tả mã bằng một phân phối để vừa tái tạo vừa tổ chức không gian mã; **diffusion** học đảo một quá trình thêm nhiễu. Autoencoder thông thường trước hết là mô hình học biểu diễn và tái tạo: bản thân nó chưa đảm bảo có thể lấy một mã ngẫu nhiên rồi tạo được mẫu hợp lý. Ta học nó trước vì nó là chiếc cầu trực giác sang VAE.

Một mẫu trông đẹp không đủ chứng minh mô hình đã học đúng phân phối. Nó có thể tạo ra ít loại mẫu, bỏ qua phần hiếm, hoặc lặp lại dữ liệu huấn luyện. Đánh giá mô hình sinh cần nói rõ mục tiêu và kiểm cả chất lượng lẫn độ đa dạng, thay vì dựa vào một bức ảnh được chọn đẹp nhất.

2. Nhắc nền: vector, sai số, xác suất và lấy mẫu

Một ảnh số có thể được xem là một vector dài x: mỗi thành phần lưu một giá trị điểm ảnh hoặc kênh. Để tính tay, ta dùng vector nhỏ hơn nhiều, chẳng hạn $x = (2, 1)$. Mạng nhận vector, biến đổi nó qua các lớp và trả ra vector khác. Các số trong ví dụ chỉ nhằm làm rõ phép tính, không phải số liệu của một tập ảnh.

Nếu bản tái tạo là $\hat{x} = (1{,}5;\ 1{,}5)$, một cách đo sai khác là **trung bình bình phương sai số** theo từng thành phần:

$$\mathrm{MSE}(x, \hat{x}) = \frac{1}{2}\left[(2 - 1{,}5)^2 + (1 - 1{,}5)^2\right] = 0{,}25.$$

Đây là phép tự tính đã kiểm bằng numpy. Bình phương phạt chênh lệch âm và dương như nhau; chia cho số thành phần cho ra trung bình. Người khác có thể dùng tổng thay cho trung bình, hoặc dùng một hàm mất mát khác phù hợp dữ liệu, nên khi so các con số phải kiểm quy ước.

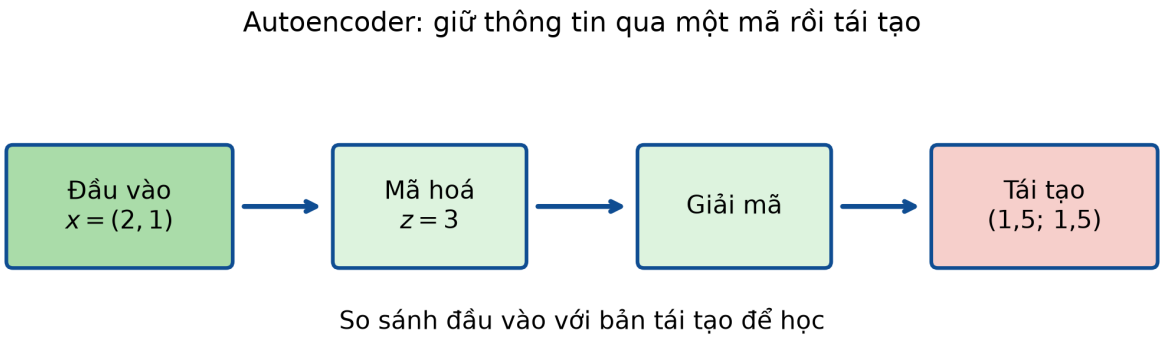
Một **phân phối xác suất** mô tả những giá trị nào có thể xuất hiện và mức độ có thể xảy ra của chúng. Một **mẫu** là kết quả cụ thể rút theo quy tắc đó. Ví dụ xúc xắc công bằng có sáu kết quả có thể xảy ra, mỗi mặt có xác suất $1/6$ (tự tính); một lần tung chỉ cho một trong các mặt. Với biến liên tục như Gaussian, ta nói mật độ xác suất và xác suất trên khoảng, không coi giá trị mật độ tại một điểm là xác suất của đúng điểm ấy.

Gaussian một chiều $\mathcal{N}(\mu, \sigma^2)$ được mô tả bởi trung bình μ và phương sai σ^2; độ lệch chuẩn σ không âm là căn bậc hai của phương sai. Gaussian chuẩn $\mathcal{N}(0, 1)$ có trung bình 0 và phương sai 1. Với $\mu = 1$ và $\sigma = 0{,}5$, phương sai bằng $0{,}25$ (tự tính). Một giá trị rút từ Gaussian có thể nằm rất xa trung bình, dù giá trị gần trung bình thường có mật độ cao hơn.

Một **không gian ẩn** hay không gian mã chứa các vector z dùng làm biểu diễn trung gian. Tên "ẩn" chỉ rằng nó không phải nhãn được cung cấp trực tiếp cho mỗi chiều. Một chiều của z không mặc nhiên mang ý nghĩa dễ đặt tên, và khoảng cách giữa hai mã chỉ hữu ích theo cách do mô hình cùng cách huấn luyện tạo ra.

3. Autoencoder: nhìn đường đi trước

Hãy đưa một bức ảnh qua một cửa hẹp, rồi cố vẽ lại ảnh từ thứ đã đi qua cửa đó. Cửa hẹp buộc hệ thống lựa chọn thông tin nào hữu ích cho việc tái tạo, nếu kiến trúc và cách học được thiết kế sao cho nó không thể đơn giản sao chép mọi thứ. Bộ **mã hoá** biến đầu vào thành mã, còn bộ **giải mã** biến mã thành bản tái tạo.



Hình 29: Autoencoder đưa dữ liệu qua mã và tái tạo lại (minh hoạ, tự tính)

Cách đọc hình: đọc từ trái qua phải, đầu vào hai số được biến thành một mã $z = 3$, sau đó giải mã thành hai số. Bản tái tạo khác đầu vào, nên đường đi này có sai số. Mục 5 giải thích rõ các trọng số giả định và cách tính sai số; hình không ngụ ý đây là trọng số đã học tốt nhất.

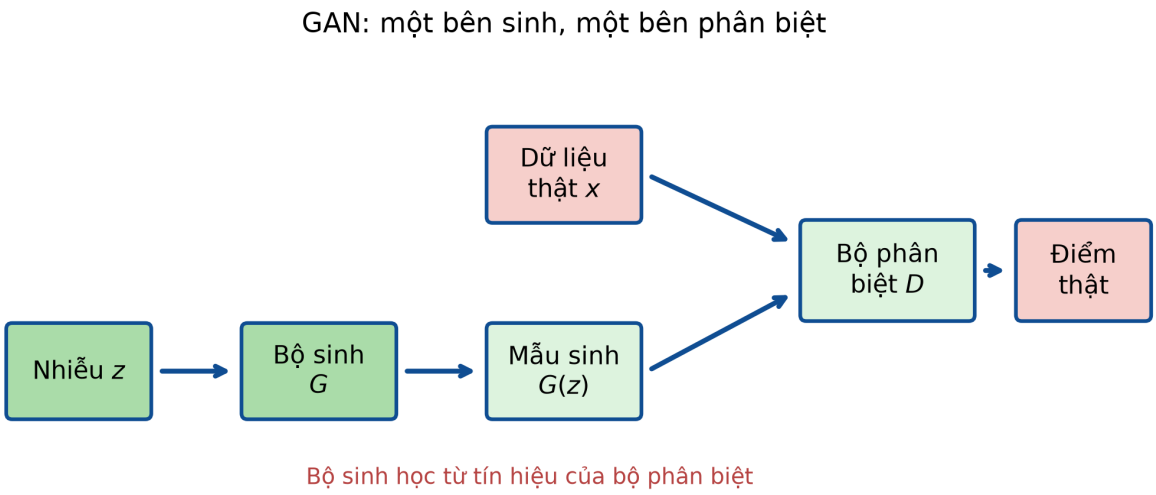
Ta có thể viết $z = f_\phi(x)$ và $\hat{x} = g_\theta(z)$, trong đó f_ϕ là bộ mã hoá, g_θ là bộ giải mã, còn ϕ, θ là tham số. Một mục tiêu thường gặp là giảm trung bình mất mát tái tạo $\ell(x, \hat{x})$ trên dữ liệu huấn luyện. Mô hình học cả hai phép biến đổi sao cho mã giữ lại thông tin có ích theo hàm mất mát đã chọn.

Nếu mã có ít chiều hơn đầu vào, ta gọi đó là một dạng **nút thắt**. Nhưng ít chiều không tự đảm bảo mã là "ý nghĩa" theo điều con người muốn: mô hình có thể giữ những khác biệt giúp giảm sai số mà bỏ một đặc tính quan trọng cho nhiệm vụ khác. Autoencoder cũng có thể có mã không hẹp mà dùng các ràng buộc khác, ví dụ nhiễu hoặc sparsity, để tránh lời giải sao chép tầm thường.

4. GAN: nhìn cuộc chơi trước

Giả sử một bên tạo mẫu, một bên cố phân biệt mẫu thật và mẫu sinh. Bộ sinh nhận một vector ngẫu nhiên z rồi tạo $G(z)$; bộ phân biệt nhận một mẫu và trả ra điểm $D(\cdot)$ diễn giải như xác suất mẫu đến từ dữ liệu thật

trong cách thiết lập chuẩn. Khi huấn luyện, tín hiệu từ bộ phân biệt giúp bộ sinh thay đổi để mẫu sinh khó bị phân biệt hơn.



Hình 30: Bộ sinh và bộ phân biệt trong GAN (minh hoạ, tự tính)

Cách đọc hình: nhánh dưới lấy nhiễu đưa vào bộ sinh và tạo mẫu, nhánh trên đưa dữ liệu thật vào. Cả hai loại mẫu đều được đưa qua cùng bộ phân biệt; đầu ra là điểm phân biệt, không phải nhãn được bảo đảm đúng. Mũi tên mô tả đường tính tiến, còn khi huấn luyện gradient đi ngược để cập nhật tham số.

Cần phân biệt hai việc. Khi cập nhật bộ phân biệt, ta muốn tăng khả năng phân biệt dữ liệu thật với dữ liệu do bộ sinh hiện tại tạo ra. Khi cập nhật bộ sinh, ta giữ vai trò của tín hiệu từ bộ phân biệt để thay đổi tham số bộ sinh. Trong nhiều cách cài đặt, hai bước này được xen kẽ; trạng thái cân bằng lý thuyết và hành vi của một lần huấn luyện hữu hạn không phải cùng một điều.

Một mục tiêu GAN nguyên thuỷ ở dạng kỳ vọng là

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1 - D(G(z)))].$$

p_{data} là phân phối dữ liệu đích, p_z là phân phối nhiễu đầu vào, và G đẩy phân phối nhiễu đó thành phân phối mẫu sinh. Kỳ vọng là trung bình theo phân phối tương ứng; trong huấn luyện thực tế nó thường được ước lượng bằng batch hữu hạn. Một mục tiêu khác cho bộ sinh thường được dùng để cải thiện tín hiệu gradient khi bắt đầu; không nên đồng nhất mọi biến thể GAN với đúng một biểu thức duy nhất.

Tại một điểm mà mô hình sinh tạo ra nhiều loại kết quả nhưng kém đa dạng hơn dữ liệu thật, chất lượng từng mẫu vẫn có thể cao. Hiện tượng bộ sinh chỉ tạo một số dạng và bỏ những dạng khác được gọi là **mode collapse**. Bộ phân biệt mạnh hay yếu, lịch cập nhật và lựa chọn mục tiêu đều ảnh hưởng quá trình học; không thể bảo đảm hết hiện tượng này chỉ bằng sơ đồ hai hộp.

5. Tính tay autoencoder từ đầu đến sai số

Tất cả số trong mục này đều là ví dụ (tự tính), đã kiểm bằng numpy. Hãy chọn bộ mã hoá và giải mã tuyến tính đơn giản:

$$x = \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \qquad z = \begin{pmatrix} 1 & 1 \end{pmatrix} x, \qquad \hat{x} = \begin{pmatrix} \frac{1}{2} \\ \frac{1}{2} \end{pmatrix} z.$$

Bước 1, mã hoá. Tích một hàng với vector cột cho $z = 1 \cdot 2 + 1 \cdot 1 = 3$. Từ hai số đầu vào, bộ mã hoá giữ một số là tổng của chúng.

Bước 2, giải mã. Nhân z với hai trọng số $\frac{1}{2}$ được $\hat{x} = (\frac{1}{2} \cdot 3, \frac{1}{2} \cdot 3)^\top = (1{,}5;\ 1{,}5)^\top$. Bộ giải mã chia tổng đều cho hai thành phần.

Bước 3, so với đầu vào. Sai khác $x - \hat{x} = (0{,}5;\ -0{,}5)^\top$. Bình phương từng thành phần được $(0{,}25;\ 0{,}25)$, tổng bằng $0{,}5$ và trung bình bằng $0{,}25$. Đó là giá trị MSE đã nêu ở mục 2.

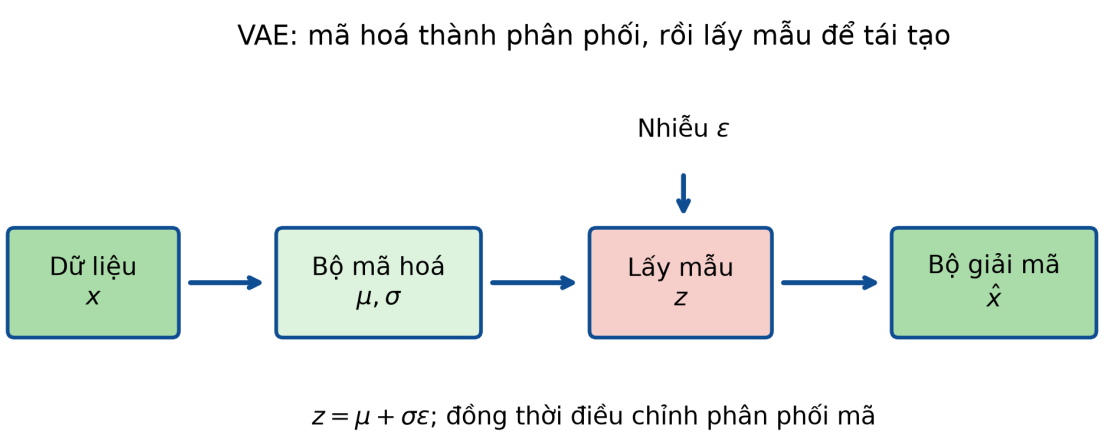
Bước 4, hiểu thông tin bị mất. Đầu vào $(1, 2)^\top$ cũng được mã hoá thành $z = 3$ và giải mã thành $(1{,}5;\ 1{,}5)^\top$ (tự tính). Chỉ nhìn mã không thể biết đầu vào ban đầu là vector nào trong hai vector ấy. Đây là mất thông tin do phép mã hoá cụ thể, không phải định lý rằng mọi mã một chiều luôn không thể phân biệt bất kỳ tập dữ liệu hữu hạn nào.

Trong huấn luyện, ta thường chưa biết các trọng số tốt và sẽ cập nhật chúng qua gradient để giảm mất mát trung bình trên nhiều ví dụ. Phép tính trên chỉ thực hiện **một lượt suy luận** với trọng số giả định; nó không chứng minh những trọng số này tối ưu cho một tập dữ liệu.

Tại sao không lấy bất kỳ z nào rồi mong có ảnh hợp lý? Một autoencoder thông thường được huấn luyện để giải mã các mã do bộ mã hoá tạo ra từ dữ liệu, nhưng không nhất thiết được huấn luyện để giải mã tốt toàn bộ không gian mã. Các khoảng giữa hoặc vùng xa những mã đó có thể dẫn đến đầu ra vô nghĩa. Ta cần quy tắc bổ sung nếu muốn lấy mẫu một cách có hệ thống.

6. VAE: nhìn phân phối của mã trước

Thay vì gán cho mỗi đầu vào một mã cố định, VAE dùng bộ mã hoá cho ra một **phân phối trên mã**. Ta lấy mẫu z từ phân phối ấy rồi giải mã. Đồng thời mục tiêu huấn luyện khuyến khích các phân phối mã gần một phân phối tham chiếu chung, thường chọn Gaussian chuẩn. Điều này tạo một cầu nối từ mã của dữ liệu sang quy trình lấy mẫu từ prior khi sinh dữ liệu.



Hình 31: VAE mã hoá thành phân phối và lấy mẫu trước khi giải mã (minh hoạ, tự tính)

Cách đọc hình: bộ mã hoá không trả trực tiếp một số z duy nhất mà trả các tham số μ, σ. Nhiễu ϵ cùng các tham số ấy tạo ra một mã cụ thể, rồi bộ giải mã dùng mã đó để dự đoán dữ liệu. Trong huấn luyện còn có một

thành phần điều chỉnh phân phối mã; mũi tên trong hình không thay thế công thức mục tiêu.

Gọi $q_\phi(z \mid x)$ là phân phối do bộ mã hoá xấp xỉ cho mã khi biết x; $p(z)$ là prior, tức phân phối giả định cho mã trước khi biết một quan sát; $p_\theta(x \mid z)$ là mô hình phân phối dữ liệu khi biết mã. Trong trường hợp Gaussian một chiều đơn giản:

$$q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi(x)^2), \qquad p(z) = \mathcal{N}(0, 1).$$

Đây là một lựa chọn mô hình thường dùng chứ không phải định nghĩa duy nhất của VAE. Trong nhiều chiều, phiên bản đơn giản dùng Gaussian có hiệp phương sai đường chéo; một mạng có thể dự đoán log phương sai rồi biến đổi để bảo đảm phương sai dương.

Lấy mẫu từ q_ϕ có thể viết lại thành

$$\epsilon \sim \mathcal{N}(0, 1), \qquad z = \mu_\phi(x) + \sigma_\phi(x)\epsilon.$$

Đây là **tái tham số hoá**: nguồn ngẫu nhiên ϵ được tách khỏi các tham số do mạng dự đoán. Với một giá trị ϵ đã rút, biểu thức z là hàm khả vi theo μ và σ, cho phép gradient của ước lượng mất mát đi ngược qua bước này. Một lần lấy mẫu không phải toàn bộ phân phối; nhiều lần rút khác nhau có thể cho nhiều mã khác nhau cho cùng x.

7. Mục tiêu VAE và một ví dụ số

Một VAE tối đa hoá cận dưới bằng chứng, thường gọi là **ELBO**:

$$\mathcal{L}(x) = \mathbb{E}_{z \sim q_\phi(z|x)}[\log p_\theta(x \mid z)] - D_{\mathrm{KL}}(q_\phi(z \mid x) \| p(z)) \,.$$

Hạng đầu khuyến khích mô hình gán xác suất hoặc mật độ cao cho quan sát khi giải mã mã lấy từ bộ mã hoá. Hạng sau là phân kỳ KL đo khác biệt có hướng giữa phân phối mã cho x và prior. KL không âm và bằng 0 khi hai phân phối trùng nhau gần như khắp nơi. Dấu trừ cho thấy tối đa ELBO phải cân bằng khả năng tái tạo với việc đưa phân phối mã gần prior. Mất mát dùng để **giảm** có thể viết là âm của ELBO; hệ số và dạng thực tế có thể thay đổi theo biến thể.

Với likelihood Gaussian có phương sai cố định, hạng tái tạo có liên hệ với sai số bình phương cộng hằng số. Nhưng VAE nói chung không được định nghĩa bằng duy nhất MSE; dữ liệu nhị phân hay mô hình likelihood khác dẫn đến dạng mất mát khác. ELBO là cận dưới của log likelihood, không phải lúc nào cũng bằng log likelihood.

Xét Gaussian một chiều $q = \mathcal{N}(\mu, \sigma^2)$ và prior chuẩn $p = \mathcal{N}(0, 1)$. Khi $\sigma > 0$, KL có công thức

$$D_{\mathrm{KL}}(q \| p) = \frac{1}{2}\left(\mu^2 + \sigma^2 - 1 - \log \sigma^2\right),$$

trong đó log là log tự nhiên. Mọi phép tính sau đây là ví dụ (tự tính), đã kiểm bằng numpy.

Bước 1, tham số phân phối. Chọn $\mu = 1$, $\sigma = 0{,}5$. Khi đó phương sai là $\sigma^2 = 0{,}25$. Giá trị $0{,}5$ là độ lệch chuẩn, không phải phương sai.

Bước 2, lấy một mẫu minh hoạ. Giả sử lần rút cho $\epsilon = 2$. Khi ấy $z = 1 + 0{,}5 \cdot 2 = 2$. Nếu lần khác cho $\epsilon = -2$ thì $z = 0$. Hai giá trị này chỉ minh hoạ hai kết quả có thể, không phải các kết quả duy nhất của Gaussian liên tục.

Bước 3, tính KL. Thay vào công thức:

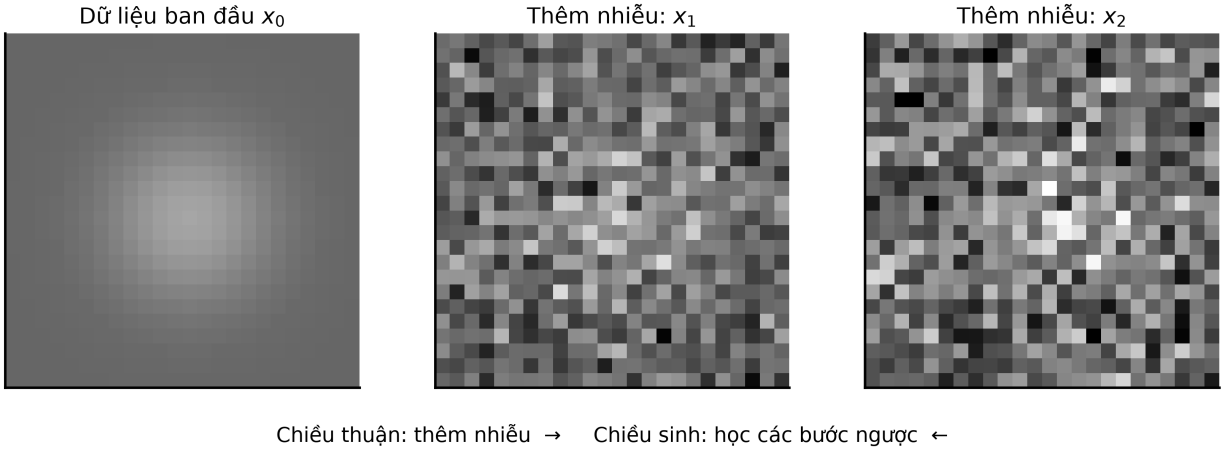
$$\begin{aligned} D_{\mathrm{KL}}(q\|p) &= \frac{1}{2}(1^2 + 0{,}25 - 1 - \log 0{,}25) \\ &\approx \frac{1}{2}(0{,}25 + 1{,}386294) \\ &\approx 0{,}818147. \end{aligned}$$

Bước 4, hiểu kết quả. Số dương phản ánh $\mathcal{N}(1, 0{,}25)$ khác $\mathcal{N}(0, 1)$. Nếu $\mu = 0$ và $\sigma = 1$, biểu thức KL bằng $\frac{1}{2}(0 + 1 - 1 - \log 1) = 0$ (tự tính). KL gần 0 không tự chứng minh mẫu giải mã đẹp, vì chất lượng còn phụ thuộc bộ giải mã và hạng tái tạo.

Khi **sinh mẫu mới**, ta thường rút $z \sim p(z)$ rồi dùng bộ giải mã để lấy mẫu hoặc dự đoán từ $p_\theta(x \mid z)$. Ta không cần cung cấp ảnh gốc cho bộ mã hoá ở bước này. Khi **tái tạo một ảnh đã có**, đường đi lại là ảnh x, bộ mã hoá, mẫu z, rồi bộ giải mã. Hai tác vụ dùng chung một số thành phần nhưng không giống nhau.

8. Diffusion: nhìn đường thêm nhiễu và đường sinh

Bắt đầu bằng dữ liệu sạch, thêm một lượng nhiễu nhỏ ở mỗi bước. Khi lặp đủ nhiều với lịch thích hợp, tín hiệu ban đầu ngày càng khó nhận ra và phân phối cuối gần một phân phối đơn giản. Sau đó ta học một quá trình ngược: bắt đầu từ nhiễu và đi từng bước về phía dữ liệu. Trực giác không phải "xóa nhiễu tùy ý một lần", mà là dùng mô hình dự đoán có điều kiện theo mức nhiễu để xây dựng chuỗi bước sinh.



Hình 32: Quá trình thêm nhiễu dần và chiều sinh ngược trong diffusion (minh hoạ, tự tính)

Cách đọc hình: các ô từ trái qua phải là dữ liệu nhân tạo ban đầu, sau một và sau hai bước thêm Gaussian với cùng mức nhiễu mỗi bước. Cả ba dùng cùng thang sáng để so sánh; tín hiệu nhìn thấy kém rõ hơn khi nhiễu được đưa vào. Mũi tên ngược biểu diễn ý tưởng sinh bằng các bước đã học, không phải kết quả đã chạy bộ khử nhiễu trong hình.

Với một dạng Gaussian thường gặp, bước thuận là

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t I\Big), \qquad 0 < \beta_t < 1.$$

Nói bằng lời: nhân tín hiệu bước trước với $\sqrt{1-\beta_t}$ rồi cộng nhiễu Gaussian độc lập có độ lệch chuẩn $\sqrt{\beta_t}$. I cho biết hiệp phương sai đẳng hướng trong dạng viết này. Lịch β_t định mức nhiễu từng bước; nó là lựa chọn thiết kế chứ không có một giá trị đúng cho mọi mô hình.

Quá trình thuận thường không có tham số cần học. Mô hình học một họ chuyển tiếp ngược $p_\theta(x_{t-1} \mid x_t)$ hoặc một đại lượng tương đương hữu ích cho việc xây dựng nó. Trong DDPM, một cách huấn luyện phổ biến cho

mạng dự đoán nhiễu đã đưa vào ở mức t. Bài ml basics 16 trình bày rõ thêm mục tiêu và các giả định. Mô hình dự đoán nhiễu không được mặc định biết chính xác ảnh gốc từ một quan sát nhiễu bất kỳ.

Ở chiều sinh, bắt đầu từ một mẫu nhiễu có phân phối đơn giản, sau đó lấy các bước ngược theo mô hình đã học để có mẫu cuối. Một số bộ lấy mẫu có yếu tố ngẫu nhiên, một số lựa chọn thiết kế có thể tạo đường đi xác định khi đã cố định đầu vào; không phải mọi diffusion đều phải chạy cùng số bước hay cùng thuật toán lấy mẫu.

9. Tính tay hai bước thêm nhiễu

Để dễ tính, xét dữ liệu chỉ có một chiều thay vì ảnh. Chọn $x_0 = 2$, $\beta_1 = \beta_2 = 0{,}36$. Khi đó $1 - \beta_t = 0{,}64$, căn là $0{,}8$, còn $\sqrt{\beta_t} = 0{,}6$ (tự tính). Các nhiễu chuẩn được rút độc lập; giả sử trong lần minh hoạ chúng nhận các giá trị $\epsilon_1 = -1$ và $\epsilon_2 = 1$.

Bước 1. Dùng $x_1 = 0{,}8x_0 + 0{,}6\epsilon_1$: $x_1 = 0{,}8 \cdot 2 + 0{,}6 \cdot (-1) = 1$ (tự tính).

Bước 2. Dùng $x_2 = 0{,}8x_1 + 0{,}6\epsilon_2$: $x_2 = 0{,}8 \cdot 1 + 0{,}6 \cdot 1 = 1{,}4$ (tự tính). Giá trị tăng so với x_1 không mâu thuẫn với việc quá trình thêm nhiễu: nhiễu có thể dương hoặc âm, và một đường mẫu không cần tiến gần 0 đều đặn.

Nếu biết $x_1 = 1$ và quy tắc nhưng không biết ϵ_1 hay x_0, không thể suy duy nhất cả hai đại lượng từ một phương trình. Chiều ngược của mô hình sinh dựa vào phân phối dữ liệu đã học và trạng thái hiện tại để ước lượng chuyển tiếp hợp lý, không phải phép đảo đại số chắc chắn khôi phục đúng nhiễu ngẫu nhiên đã rút.

Có thể viết trực tiếp phân phối của x_t khi đã biết x_0. Đặt $\alpha_t = 1 - \beta_t$ và $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. Khi các bước thuận độc lập theo cách trên,

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

Công thức này là cách lấy **một mẫu tại thời điểm t theo phân phối biên có điều kiện trên** x_0; không có nghĩa ta dùng cùng một ϵ độc lập đó để tái tạo toàn bộ quỹ đạo Markov. Với lịch ở ví dụ, $\bar{\alpha}_2 = 0{,}64^2 = 0{,}4096$, nên hệ số tín hiệu là $\sqrt{0{,}4096} = 0{,}64$ (tự tính). Khi $\bar{\alpha}_t$ tiến gần 0, thành phần tín hiệu trực tiếp nhỏ đi và phân phối thuận tiến gần Gaussian chuẩn dưới những điều kiện thích hợp.

10. So sánh bốn đường đi

Cách	Đường đi chính	Tín hiệu học hoặc ràng buộc	Sinh mẫu mới
AE	$x \to z \to \hat{x}$	giảm sai số tái tạo, có thể thêm ràng buộc	decoder tạo đầu ra từ mã, nhưng lấy mã ngẫu nhiên chưa được bảo đảm hợp lý
GAN	$z \to G(z) \to D$	bộ phân biệt và bộ sinh có mục tiêu đối kháng	lấy z từ prior rồi chạy bộ sinh
VAE	$x \to q_\phi(z \mid x) \to z \to p_\theta(x \mid z)$	hạng tái tạo và KL trong ELBO	lấy z từ prior rồi dùng bộ giải mã
DM	dữ liệu được thêm nhiễu; học chiều ngược	dự đoán hoặc tham số hoá chuyển tiếp ngược theo mức nhiễu	bắt đầu từ nhiễu rồi thực hiện chuỗi bước ngược

Trong bảng, AE là autoencoder và DM là mô hình diffusion. Bảng là bản đồ để không lẫn các vai trò, không phải bảng xếp hạng. Một mô hình có thể kết hợp nhiều ý tưởng, chẳng hạn dùng không gian ẩn cùng diffusion; chi tiết cụ thể quyết định việc huấn luyện và sinh. Thời gian chạy, chất lượng và độ đa dạng phải được đo trong cùng điều kiện trước khi so sánh.

Đầu vào của bộ giải mã autoencoder trong giai đoạn tái tạo thường là mã do encoder tạo; đầu vào bộ sinh GAN thường là một mẫu từ phân phối được chọn; đầu vào quá trình diffusion ở thời điểm sinh thường là nhiễu. Các từ "mã", "nhiễu" và "mẫu" có thể cùng biểu diễn bằng vector nhưng tham gia các phân phối và bước tính khác nhau.

11. Những chi tiết dễ bỏ qua khi áp dụng

11.1 Sinh có điều kiện và không điều kiện

Một mô hình **không điều kiện** học tạo mẫu từ phân phối dữ liệu tổng thể. Mô hình **có điều kiện** còn nhận thông tin c, chẳng hạn một nhãn lớp hay mô tả, và nhắm tới mẫu phù hợp với điều kiện đó. Có thể đưa điều kiện vào bộ sinh GAN, bộ giải mã VAE hoặc mô hình dự đoán nhiễu. Ký hiệu chung $p_\theta(x \mid c)$ không xác định cách kiến trúc mã hoá c.

Đánh giá mô hình có điều kiện cần xem đồng thời mẫu có hợp lý và có khớp điều kiện. Một ảnh rõ nét nhưng sai nội dung yêu cầu không phải mẫu có điều kiện tốt. Ngược lại một phương pháp không điều kiện không có nghĩa là nó không dùng dữ liệu huấn luyện: "không điều kiện" chỉ nói về thông tin cấp thêm tại lúc sinh.

11.2 Mẫu, phân phối và xác suất

Sinh được một mẫu là thực hiện một lần rút từ cơ chế của mô hình. Sinh nhiều mẫu có thể cho cái nhìn về độ đa dạng, nhưng số lượng mẫu hữu hạn không chứng minh hai phân phối trùng nhau. Một bộ sinh ánh xạ z thành x cũng không tự cung cấp công thức mật độ $p_\theta(x)$ dễ tính cho mọi điểm x.

VAE có mô hình xác suất rõ ràng nhưng tích phân theo biến ẩn có thể khó tính; ELBO là một đại lượng có thể tối ưu để chặn dưới log likelihood. GAN nguyên thuỷ mô tả một bộ sinh mẫu và một trò chơi phân biệt, không bắt buộc biểu diễn mật độ tường minh. So sánh likelihood giữa các họ mô hình cần biết chính xác đại lượng nào đã được tính hoặc ước lượng.

11.3 Tái tạo và sáng tạo mẫu

Bản tái tạo gần một ảnh đầu vào đo mức mô hình giữ thông tin của ảnh đó. Một mẫu mới lại đánh giá khả năng tạo ra kết quả hợp lý khi không cung cấp chính ảnh đó. Thành công ở tác vụ này không tự động bảo đảm thành công ở tác vụ kia. Với VAE, so ảnh gốc và ảnh tái tạo là phép kiểm hữu ích, nhưng phải lấy mẫu từ prior để kiểm đường sinh.

Trong GAN, bộ phân biệt được học cùng bộ sinh nên điểm của nó không phải thước đo khách quan đủ để so sánh các mô hình khác nhau. Khi hai phân phối trùng nhau trong điều kiện lý tưởng của GAN gốc, một bộ phân biệt tối ưu không thể nhận biết nguồn tốt hơn cơ hội tại những nơi hai phân phối cùng có hỗ trợ. Kết quả lý thuyết này không cho phép kết luận một lần huấn luyện thực tế có điểm gần một nửa đã học đúng phân phối.

11.4 Độ đa dạng, ghi nhớ và dữ liệu

Một tập mẫu đẹp có thể che việc mô hình thường xuyên sinh cùng một kiểu. Hãy xem nhiều mẫu được chọn có quy tắc, so độ đa dạng và kiểm các vùng dữ liệu hiếm khi bài toán yêu cầu. Nếu lo mô hình ghi nhớ dữ liệu huấn luyện, so gần trùng là một phép kiểm cần thiết, nhưng chỉ nhìn vài cặp ảnh không đủ kết luận về toàn bộ mức độ ghi nhớ.

Chất lượng và thiên lệch của dữ liệu huấn luyện ảnh hưởng những gì mô hình học. Một mô hình sinh hình ảnh y khoa giả định chẳng hạn không tự trở thành công cụ chẩn đoán chỉ vì tạo ảnh trông thật; xác nhận mục đích sử dụng cần quy trình đánh giá riêng. Không có một con số đẹp nào thay thế được mô tả dữ liệu, cách chọn mẫu và điều kiện đánh giá.

12. Nối sang các bài AI

Học biểu diễn. Autoencoder cho một ví dụ cụ thể về việc biến dữ liệu thành mã trung gian có thể dùng cho tác vụ khác. Khi đọc bài ml basics 26, hãy hỏi hàm mất mát khuyến khích giữ thông tin nào và việc đánh giá biểu diễn có dùng nhãn của tác vụ đích hay không.

GAN. Bài ml basics 14 trình bày trò chơi đối kháng, bộ phân biệt tối ưu trong giả thiết lý tưởng và khó khăn huấn luyện. Sơ đồ ở đây chỉ giúp xác định đường đi của mẫu và tín hiệu, không thay thế chứng minh của bài.

VAE. Bài ml basics 15 giải thích suy luận biến phân, ELBO và tái tham số hoá. Hãy theo dõi riêng $q_\phi(z \mid x)$ ở chiều mã hoá, prior $p(z)$ ở bước sinh và $p_\theta(x \mid z)$ ở chiều giải mã. Tránh gọi cả ba là "phân phối của ảnh".

Diffusion. Bài ml basics 16 phát triển lịch nhiễu, phân phối thuận và cách tham số hoá chiều ngược. Ví dụ hai bước trong chương chỉ làm rõ từng phép nhân và cộng; nó không tái hiện một thí nghiệm hay chỉ số từ paper.

Quan sát không gian mã. Các phương pháp giảm chiều như trong bài ml basics 40 có thể giúp nhìn dữ liệu hoặc biểu diễn trong hai chiều. Một đồ thị đẹp chỉ là phép chiếu và có thể làm thay đổi quan hệ lân cận; nó không chứng minh một chiều mã đã học một khái niệm nhân quả hay bảo đảm chất lượng sinh.

13. Sai lầm hay gặp

- **Cho rằng mọi autoencoder đều là bộ sinh tốt.** Tái tạo mã của dữ liệu không bảo đảm mọi mã rút ngẫu nhiên có bộ giải mã cho mẫu hợp lý.
- **Nhầm độ lệch chuẩn với phương sai.** Nếu $\sigma = 0{,}5$ thì phương sai là $0{,}25$; công thức lấy mẫu dùng σ còn ký hiệu Gaussian thường ghi σ^2.
- **Nhầm encoder VAE với prior.** $q_\phi(z \mid x)$ phụ thuộc dữ liệu cụ thể; $p(z)$ thường được lấy mẫu không cần x khi sinh mới.
- **Coi điểm của bộ phân biệt GAN là xác suất tuyệt đối đáng tin cho mọi dữ liệu.** Nó phụ thuộc bài toán, dữ liệu và trạng thái huấn luyện của chính bộ phân biệt.
- **Nghĩ đường diffusion thuận bị đảo chính xác bằng đại số.** Bước thêm nhiễu là ngẫu nhiên; mô hình ngược ước lượng phân phối chuyển tiếp chứ không tự biết mẫu nhiễu đã dùng.
- **Đánh giá bằng một mẫu đẹp.** Cần quy trình lấy mẫu rõ ràng và phép kiểm phù hợp cho chất lượng, độ đa dạng, điều kiện đầu vào và dữ liệu liên quan.

14. Tóm tắt công thức

Khái niệm	Công thức	Ý nghĩa
Autoencoder	$z = f_\phi(x)$, $\hat{x} = g_\theta(z)$	nén hoặc biến đổi rồi tái tạo
MSE	$\frac{1}{d}\sum_{i=1}^{d}(x_i - \hat{x}_i)^2$	trung bình sai số bình phương
Bộ sinh GAN	$\hat{x} = G(z)$, $z \sim p_z$	biến nhiễu thành một mẫu
Tái tham số hoá VAE	$z = \mu + \sigma\epsilon$, $\epsilon \sim \mathcal{N}(0,1)$	tách nhiễu khỏi tham số Gaussian
ELBO	$\mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)] - D_{\mathrm{KL}}(q_\phi \Vert p)$	cân bằng hạng dữ liệu và KL
KL Gaussian	$\frac{1}{2}(\mu^2 + \sigma^2 - 1 - \log\sigma^2)$	với $q = \mathcal{N}(\mu, \sigma^2)$ và prior chuẩn
Diffusion thuận	$x_t = \sqrt{1-\beta_t}\,x_{t-1} + \sqrt{\beta_t}\,\epsilon_t$	một bước thêm nhiễu chuẩn độc lập
Biên có điều kiện	$x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$	lấy trực tiếp tại bước t

15. Bài tập tự tính

Năm bài dưới đây đi từ đường tái tạo đến nhiễu. Hãy tự tính trước khi xem đáp án; tất cả kết quả số đã kiểm bằng numpy.

Bài 1 (dễ). Một bản gốc là $(1, 3)$ và bản tái tạo là $(2, 2)$. Tính MSE theo trung bình hai thành phần.

Đáp án. Hai sai số là -1 và 1, bình phương lần lượt bằng 1 và 1. Trung bình là $(1 + 1)/2 = 1$ (tự tính). Nếu dùng tổng bình phương thay vì MSE thì được 2, nên phải nêu rõ quy ước.

Bài 2 (dễ vừa). Bộ mã hoá cho $z = x_1 + x_2$, bộ giải mã trả $(z/2, z/2)$. Với $x = (4, 2)$, hãy tìm z, bản tái tạo và MSE.

Đáp án. $z = 4 + 2 = 6$; bản tái tạo là $(3, 3)$. Sai số bình phương là 1 ở mỗi chiều nên MSE bằng 1 (tự tính). Cùng mã này cũng có thể xuất phát từ các đầu vào khác có tổng bằng 6.

Bài 3 (vừa). Trong một VAE một chiều, $\mu = 2$, $\sigma = 0{,}5$ và mẫu nhiễu là $\epsilon = -2$. Tính z bằng tái tham số hoá, rồi cho biết phương sai của phân phối mã.

Đáp án. $z = 2 + 0{,}5 \cdot (-2) = 1$ và phương sai bằng $\sigma^2 = 0{,}25$ (tự tính). Kết quả z thay đổi khi lần rút ϵ thay đổi.

Bài 4 (vừa). Với $q = \mathcal{N}(1, 1)$ và $p = \mathcal{N}(0, 1)$, tính $D_{\mathrm{KL}}(q \| p)$ theo công thức mục 7.

Đáp án. $\mu = 1$, $\sigma = 1$, nên KL bằng $\frac{1}{2}(1^2 + 1^2 - 1 - \log 1) = 0{,}5$ (tự tính). Hai phương sai bằng nhau nhưng hai trung bình khác nhau, nên KL dương.

Bài 5 (khó hơn). Cho một bước diffusion $x_1 = \sqrt{1 - \beta} x_0 + \sqrt{\beta}\epsilon$, với $x_0 = 1$, $\beta = 0{,}36$, $\epsilon = -1$. Tính x_1. Nếu chỉ biết x_1 và β, có thể tìm duy nhất x_0 không?

Đáp án. $x_1 = 0{,}8 \cdot 1 + 0{,}6 \cdot (-1) = 0{,}2$ (tự tính). Không thể tìm duy nhất: chẳng hạn $x_0 = 0{,}25$ và $\epsilon = 0$ cũng cho $x_1 = 0{,}8 \cdot 0{,}25 + 0 = 0{,}2$ (tự tính). Cả hai là các giá trị có thể thuộc hỗ trợ Gaussian, nhưng không có nghĩa xác suất của từng điểm riêng lẻ trong phân phối liên tục là dương.

Tài liệu tham khảo nền: Goodfellow, Bengio & Courville, Deep Learning; Goodfellow et al., Generative Adversarial Nets; Kingma & Welling, Auto-Encoding Variational Bayes; Ho et al., Denoising Diffusion Probabilistic Models; ghi chú ml basics 14, 15 và 16.

Bài 00

Preface: Toán nền tảng và hướng dẫn đọc về ML/AI cơ bản

Tác giả: ghi chú tổng hợp từ kiến thức nền của người biên soạn (không phải ghi chú một paper cụ thể)
Venue / arXiv: không áp dụng
Nguồn: các bài 01-40 trong thư mục này; phần kiến thức cổ điển không có trên arXiv (hồi quy tuyến tính, logistic regression, k-NN, SVM, cây quyết định, random forest, boosting, Naive Bayes, k-means, PCA, EM/GMM, perceptron, backprop) được trình bày lại ở đây từ kiến thức nền chung của lĩnh vực, có thể đối chiếu với Hastie, Tibshirani và Friedman (sách "The Elements of Statistical Learning", gọi tắt ESL), Murphy ("Probabilistic Machine Learning"), Goodfellow, Bengio và Courville ("Deep Learning", gọi tắt GBC), Bishop ("Pattern Recognition and Machine Learning" và bản mới "Deep Learning: Foundations and Concepts"); không trích số trang cụ thể vì đây là kiến thức nền chung, không phải trích dẫn một ấn bản nào.
Chủ đề: ML/AI từ con số 0: các paradigm học máy, lý thuyết ERM và bias-variance, các mô hình cổ điển có công thức và ví dụ tính tay, mạng neural từ perceptron đến attention và mô hình sinh, cách đánh giá mô hình, các cạm bẫy thường gặp, thuật ngữ, cách đọc paper, lịch đọc 40 bài.
Vị trí trong lộ trình: Bài số 00, đọc trước tất cả. Sau đó đi theo 01-40 theo lịch đọc ở mục 10; người mới hoàn toàn nên đọc kỹ mục 1-6 của bài này trước khi mở bài 01.

> Phần mở đầu này tổng hợp kiến thức nền, không diễn giải một bài báo riêng. Các ví dụ số được đánh dấu "tự tính"; khi đối chiếu với một bài nghiên cứu, cần chú ý thiết lập và giả định cụ thể của bài đó.

1. ML là gì và bản đồ các paradigm

1.1 Định nghĩa

Machine learning (ML, học máy): một chương trình học quy tắc từ dữ liệu thay vì được lập trình sẵn từng quy tắc. Định nghĩa thường trích (Tom Mitchell, 1997, không phải arXiv): một chương trình học từ kinh nghiệm E đối với tác vụ T và thước đo hiệu năng P, nếu hiệu năng trên P tăng theo E.

Ba thành phần luôn phải xác định rõ khi đọc bất kỳ bài toán ML nào:

- **Tác vụ T:** phân loại ảnh, dịch văn bản, đọc chữ trong ảnh scan (OCR), sinh văn bản tiếp theo.
- **Dữ liệu/kinh nghiệm E:** có nhãn hay không, có tương tác hay không, cố định hay sinh ra liên tục.
- **Thước đo P:** accuracy, perplexity, CER/WER (mục 6.5), hay một reward.

1.2 Bốn paradigm chính

Paradigm	Dữ liệu	Mục tiêu	Ví dụ
Supervised (có giám sát)	cặp (x, y), y là nhãn đúng	học hàm $f_\theta(x) \approx y$	phân loại ảnh, hồi quy giá nhà, OCR (ảnh vào, text ra)
Unsupervised (không giám sát)	chỉ có x, không nhãn	tìm cấu trúc ẩn (cụm, chiều ẩn, mật độ)	k-means, PCA, học biểu diễn
Self-supervised (tự giám sát)	chỉ có x, nhãn được tạo tự động từ x	học biểu diễn tốt bằng một tác vụ giả (pretext task)	dự đoán từ tiếp theo (ngôn ngữ), contrastive learning (ml basics 17, 18), masked modeling
Reinforcement learning (RL, học tăng cường)	agent tương tác với environment, nhận reward	tối đa tổng reward dài hạn	game, robot, RLHF cho LLM

Ranh giới không tuyệt đối: self-supervised thường là bước tiền xử lý để rồi fine-tune supervised; RLHF cho LLM (thư mục `rl content/`) cần một reward model được huấn luyện có giám sát từ các cặp so sánh. Pretraining ngôn ngữ hiện đại (dự đoán token tiếp theo) về hình thức là supervised trên nhãn tự sinh, nên thường được gọi là self-supervised.

1.3 Bản đồ bốn khối của một hệ ML

Mọi hệ thống ML, từ hồi quy tuyến tính đến LLM, đều có 4 khối:

1. **Biểu diễn (representation):** cách dữ liệu được đưa vào mô hình (đặc trưng tay, embedding, pixel thô, token). Biểu diễn ảnh hưởng trực tiếp tới thông tin mà mô hình có thể khai thác.
2. **Hàm giả thuyết (hypothesis class):** tập các hàm f_θ mà mô hình có thể biểu diễn (đường thẳng, cây, mạng neural L lớp). Mục 2 và 3 nói về việc chọn hàm giả thuyết.
3. **Hàm mất (loss) và cách tối ưu:** đo "sai bao nhiêu" và cách cập nhật θ để giảm sai số (gradient descent và các biến thể, xem thư mục `optimization content/`).
4. **Cách đánh giá:** đo hiệu năng trên dữ liệu chưa thấy (mục 6), vì mục tiêu cuối không phải là khớp dữ liệu huấn luyện mà là **tổng quát hoá (generalization)**.

1.4 Vì sao cần một preface riêng cho ML cơ bản

Nhiều bài trong sách khảo sát các hiện tượng và phương pháp thực hành của ML hiện đại, như double descent, calibration, shortcut learning và data leakage. Mục 2–5 giới thiệu những kiến thức cổ điển cần thiết, kèm ví dụ tính tay để làm rõ công thức.

---

2. ERM, bias-variance, overfitting, regularization và double descent

2.1 Empirical Risk Minimization (ERM)

Giả sử dữ liệu $(x_i, y_i)_{i=1}^N$ lấy i.i.d. (độc lập, cùng phân phối) từ một phân phối $\mathcal{D}$ chưa biết. **Risk thật (true risk)** của một hàm f là kỳ vọng mất mát trên toàn bộ phân phối:

$$R(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\big[\ell(f(x), y)\big].$$

Ta không có $\mathcal{D}$, chỉ có N mẫu, nên thay bằng **empirical risk (rủi ro thực nghiệm)**:

$$\boxed{\hat{R}(f) = \frac{1}{N}\sum_{i=1}^{N} \ell(f(x_i), y_i)}$$

ERM là chọn f trong một hàm giả thuyết $\mathcal{F}$ (ví dụ mọi mạng neural với một kiến trúc cố định) để tối thiểu $\hat{R}(f)$. Vấn đề cốt lõi của ML: $\hat{R}(f)$ nhỏ không đảm bảo $R(f)$ nhỏ, vì f được chọn dựa trên chính N mẫu đó (nó có thể "học vẹt" N mẫu mà không học được quy luật chung). **Generalization gap** là $R(f) - \hat{R}(f)$, và toàn bộ lý thuyết học thống kê xoay quanh việc chặn đại lượng này.

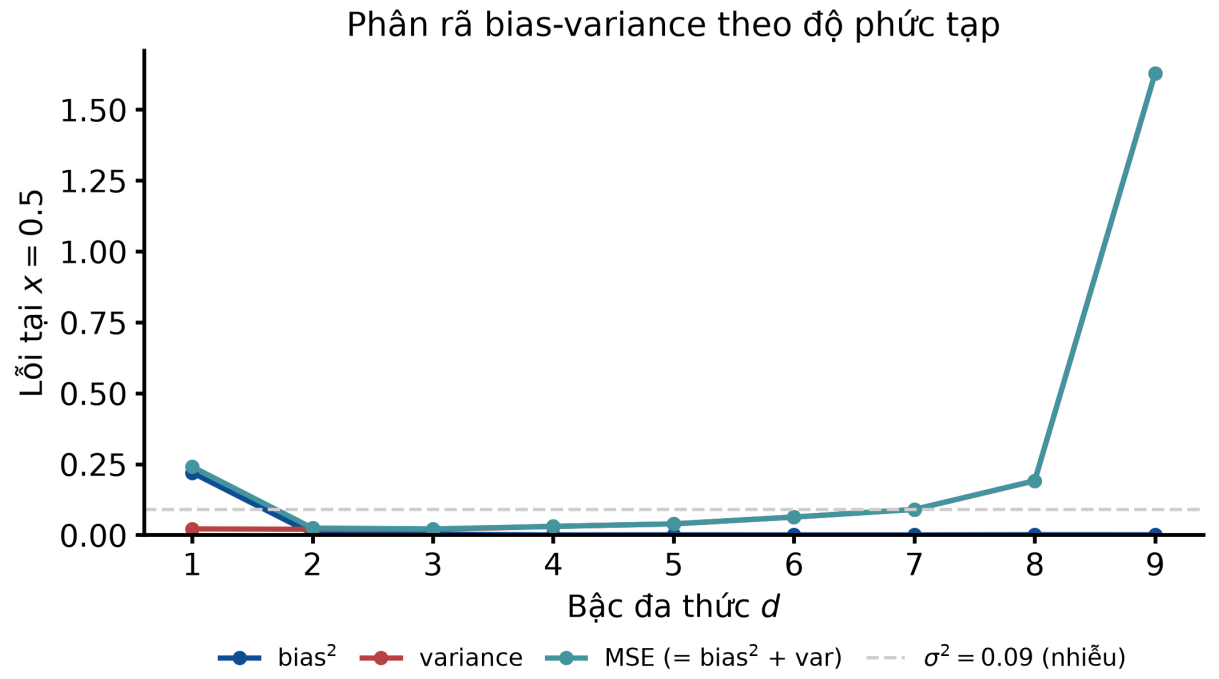
Các hàm loss ℓ thường gặp:

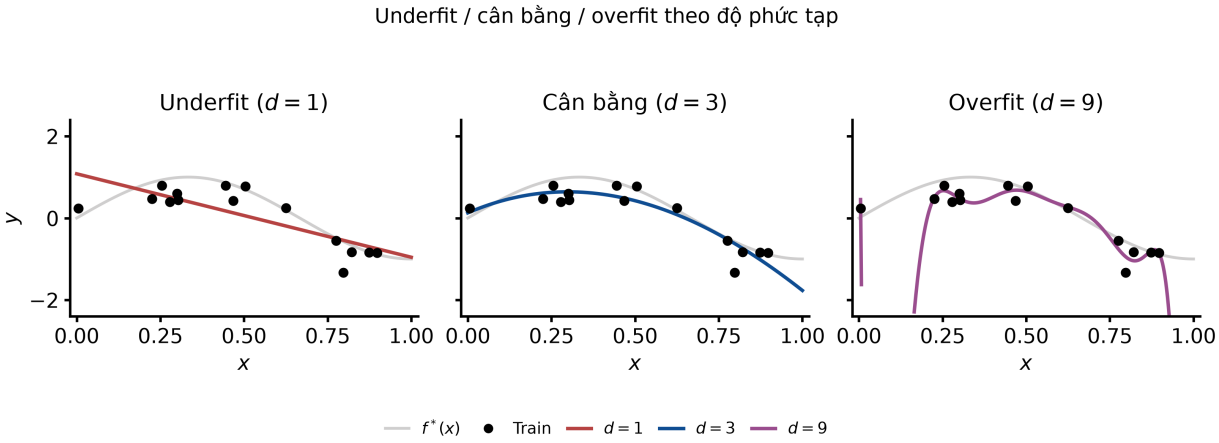
Tác vụ	Loss	Công thức
Hồi quy	Squared error (MSE)	$\ell(f(x), y) = (f(x) - y)^2$
Hồi quy bền với outlier	Huber loss	bậc hai gần 0, tuyến tính khi sai số lớn
Phân loại nhị phân	Binary cross-entropy	$-y \log p - (1-y)\log(1-p)$, $p = f(x) \in (0,1)$

Tác vụ	Loss	Công thức
Phân loại đa lớp	Cross-entropy (softmax)	$-\log p_y$, $p = \text{softmax}(f(x))$ (mục 3.3)
Margin-based	Hinge loss (SVM)	$\max(0, 1 - y \cdot f(x))$, $y \in \{-1, +1\}$

2.2 Phân rã bias-variance

Với hồi quy và squared error, giả sử $y = f^*(x) + \varepsilon$ với nhiễu ε có kỳ vọng 0, phương sai σ^2, độc lập với x. Gọi $\hat{f}_D$ là mô hình học được từ một tập dữ liệu D cụ thể (lấy mẫu ngẫu nhiên). Tại một điểm x cố định, kỳ vọng sai số bình phương trên các cách lấy mẫu D khác nhau phân rã thành ba phần không âm:





Cùng một tập train nhiễu, đa thức bậc 1 underfit, bậc 3 cân bằng, bậc 9 overfit so với hàm thật f^* (minh hoạ, tự tính)

Phân rã bias², variance và MSE theo bậc đa thức tại $x = 0.5$ với $f^*(x) = \sin(1.5\pi x)$, $\sigma = 0.3$, $n = 15$, 300 lần lặp: bậc thấp bị underfit (bias cao), bậc cao bị overfit (variance cao) (minh hoạ, tự tính)

$$\mathbb{E}_{D,\varepsilon}\left[(\hat{f}_D(x) - y)^2\right] = \underbrace{\left(\mathbb{E}_D[\hat{f}_D(x)] - f^*(x)\right)^2}_{\text{bias}^2} + \underbrace{\text{Var}_D[\hat{f}_D(x)]}_{\text{variance}} + \underbrace{\sigma^2}_{\text{nhiễu không giảm được}}$$

- **Bias (độ chệch):** mô hình trung bình (lấy trung bình trên nhiều tập train khác nhau) lệch bao xa so với hàm thật f^*. Mô hình quá đơn giản (ít tham số, nhiều giả định) có bias cao: **underfitting**.
- **Variance (phương sai):** dự đoán thay đổi bao nhiêu nếu đổi tập train. Mô hình quá phức tạp (nhiều tham số, khớp cả nhiễu) có variance cao: **overfitting**.
- **Nhiễu σ^2:** không thể giảm bằng bất kỳ mô hình nào, vì nó là tính ngẫu nhiên thật của dữ liệu (ví dụ ảnh scan có nhiễu máy quét).

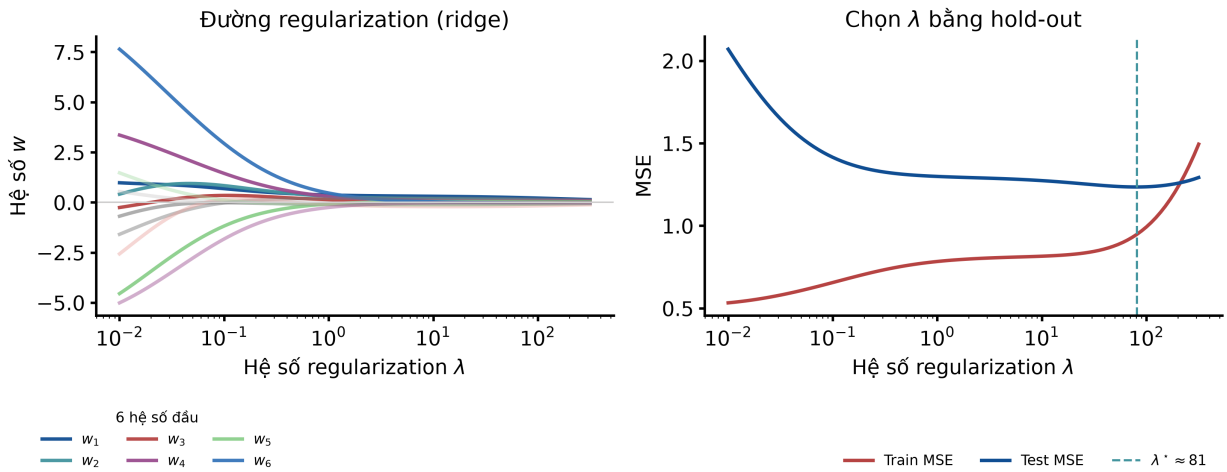
Ví dụ tự tính (chạy bằng numpy, hàm thật $f^*(x) = \sin(1.5\pi x)$, nhiễu chuẩn $\sigma = 0.3$, 15 điểm train lấy ngẫu nhiên trong $[0, 1]$, lặp lại 300 lần với các tập train khác nhau, đo tại $x = 0.5$, mô hình là đa thức bậc d khớp bằng bình phương tối thiểu thường, không regularization):

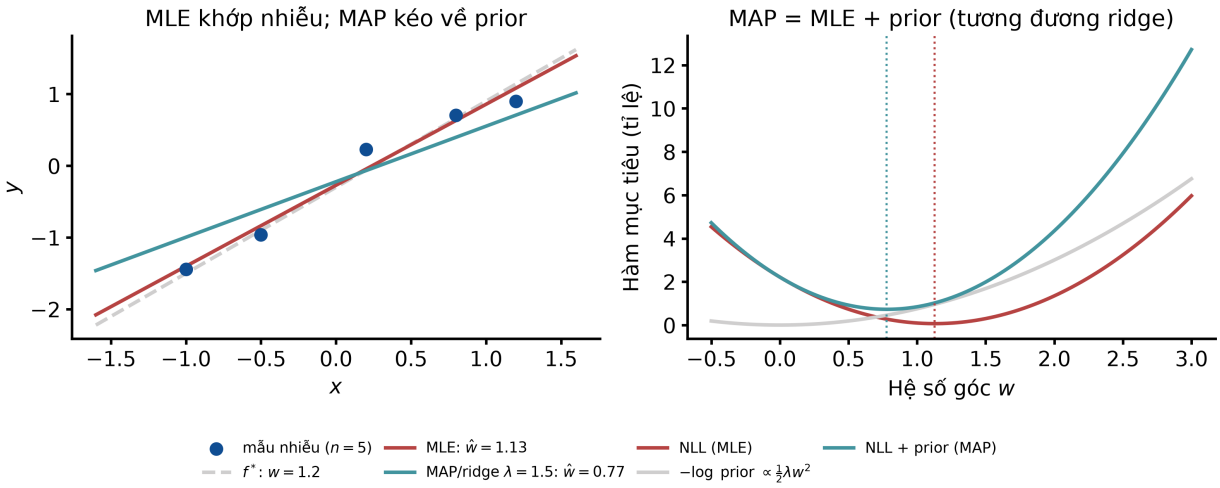
Bậc đa thức d	bias^2	variance	MSE đo được	Diễn giải
1 (đường thẳng)	0.2196	0.0212	0.2408	underfit: bias chiếm gần hết lỗi
3	0.0025	0.0162	0.0187	cân bằng tốt, lỗi thấp nhất trong ba mức
9	0.0007	0.7518	0.7525	overfit: bias gần 0 nhưng variance chiếm 99.9% lỗi, MSE còn tệ hơn cả bậc 1

Lưu ý $\sigma^2 = 0.09$ là tham số sinh dữ liệu, còn MSE ở bảng trên đo so với f^* (không phải so với y nhiễu) tại một điểm cụ thể, nên không trực tiếp so sánh hai số này. Trong ví dụ này, tăng độ phức tạp làm giảm bias nhưng tăng variance; tổng lỗi đạt mức thấp nhất ở bậc trung gian. Xu hướng này không phải quy luật đơn điệu cho mọi họ mô hình.

2.3 Regularization: đổi một ít bias để giảm nhiều variance

Regularization (điều chuẩn) là mọi kỹ thuật hạn chế hàm giả thuyết hoặc phạt độ phức tạp, nhằm giảm variance (đổi lại thường tăng bias một chút). Ba dạng chính:





Hồi quy 1D $n = 5$: MLE khớp nhiễu còn MAP/ridge ($\lambda = 1.5$) kéo hệ số về prior; MAP = NLL + prior (minh hoạ, tự tính)

Đường hệ số ridge theo λ và đường Train/Test MSE; $\lambda^\star$ chọn tại điểm Test MSE thấp nhất trên hold-out (minh hoạ, tự tính)

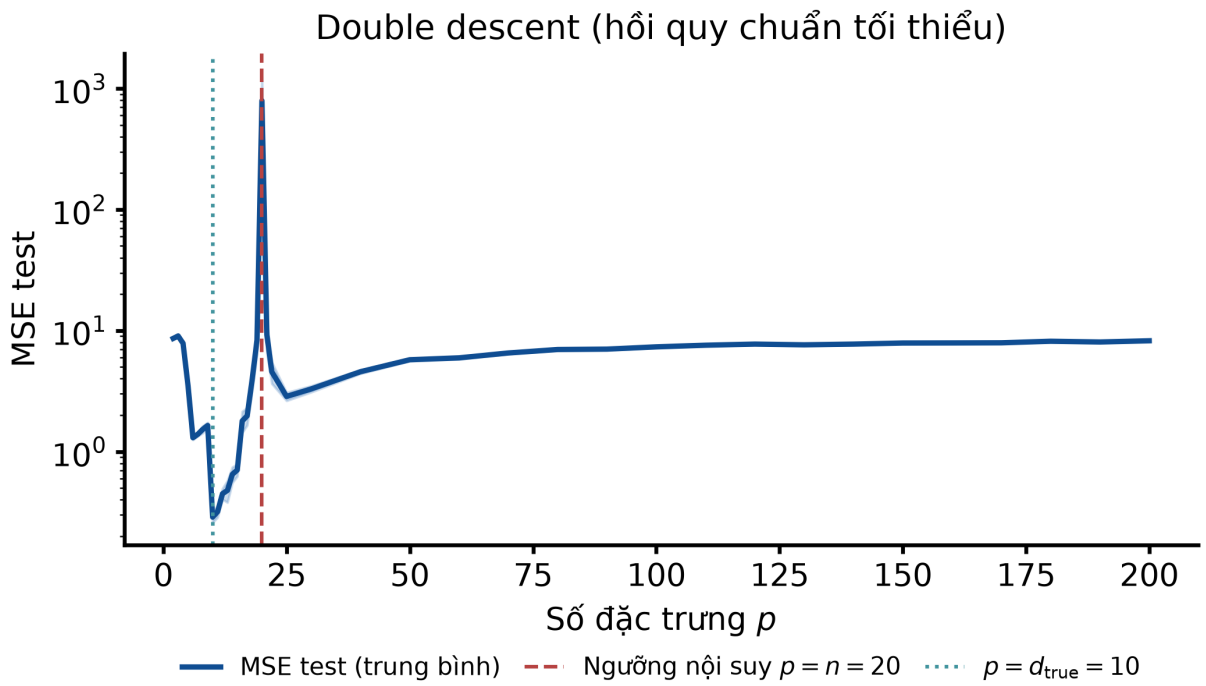
- **Ràng buộc tham số:** ℓ_2 (ridge, ứng với Gaussian prior lên θ) hoặc ℓ_1 (lasso, ứng với Laplace prior, tạo ra nghiệm thưa). Tối thiểu $\hat{R}(f) + \lambda\|\theta\|_2^2$ hoặc $+\lambda\|\theta\|_1$.
- **Giới hạn hàm giả thuyết:** giảm độ sâu cây, giảm số neuron, early stopping (ngừng train trước khi khớp hết nhiễu).
- **Thêm nhiễu vào quá trình học:** dropout (ml basics 06), data augmentation, label smoothing (ml basics 30).

λ (hệ số regularization) là một **siêu tham số (hyperparameter)**: chọn bằng validation set hoặc cross-validation (mục 6.2), không bao giờ bằng train loss (vì $\lambda = 0$ luôn cho train loss thấp nhất nhưng test loss có thể rất tệ).

2.4 Double descent: khi trực giác bias-variance cổ điển không đủ

Trực giác cổ điển (hình chữ U) giả định: tăng độ phức tạp mô hình mãi thì variance tăng mãi, lỗi test tăng mãi sau điểm tối ưu. Quan sát thực nghiệm trong deep learning (và một số mô hình tuyến tính, ml basics 03) cho thấy một đường cong khác: lỗi test giảm, **tăng vọt** tại một điểm gọi là **interpolation threshold** (ngưỡng mô hình vừa đủ khả năng khớp hoàn hảo tập huấn luyện; trong một số mô hình đơn giản, ngưỡng này gần nơi số tham số bằng số mẫu), rồi **giảm trở lại** khi tiếp tục tăng tham số qua ngưỡng đó. Đây gọi là **double descent (giảm kép)**.

Ví dụ tự tính minh hoạ cơ chế (không phải mạng neural sâu, mà hồi quy tuyến tính với nghiệm chuẩn tối thiểu (minimum-norm least squares), cách dựng chuẩn trong các bài về double descent tuyến tính): sinh $n = 20$ mẫu train từ $y = z^\top \beta^* + \varepsilon$ với $\beta^* \in \mathbb{R}^{10}$ (chỉ 10 đặc trưng thật mang tín hiệu), rồi thêm $p - 10$ đặc trưng nhiễu thuần (không liên quan y) để tổng số đặc trưng là p; với $p \le n$ giải bình phương tối thiểu thường, với $p > n$ giải nghiệm chuẩn tối thiểu bằng pseudo-inverse $w = X^+ y$ (chọn trong vô số nghiệm khớp hoàn hảo train, nghiệm có $\|w\|_2$ nhỏ nhất); đo MSE trên 200 điểm test, lấy trung bình 30 lần lặp:



MSE test của hồi quy chuẩn tối thiểu theo số đặc trưng p: chữ U cổ điển rồi đỉnh tại ngưỡng nội suy $p = n$, sau đó giảm lại (double descent) (minh hoạ, tự tính)

p (số đặc trưng)	MSE test	Vị trí
10	0.539	$p = d_{\text{true}}$, tốt nhất trong vùng $p \leq n$
19	6.927	rất gần ngưỡng nội suy
20	26445	$p = n$: đỉnh, ma trận X gần suy biến, nghiệm bùng nổ
21	7.766	vừa qua ngưỡng, giảm rất nhanh
200	4.199	không hội tụ về mức tốt nhất (0.539 ở $p = 10$) mà dừng ở mức trung bình

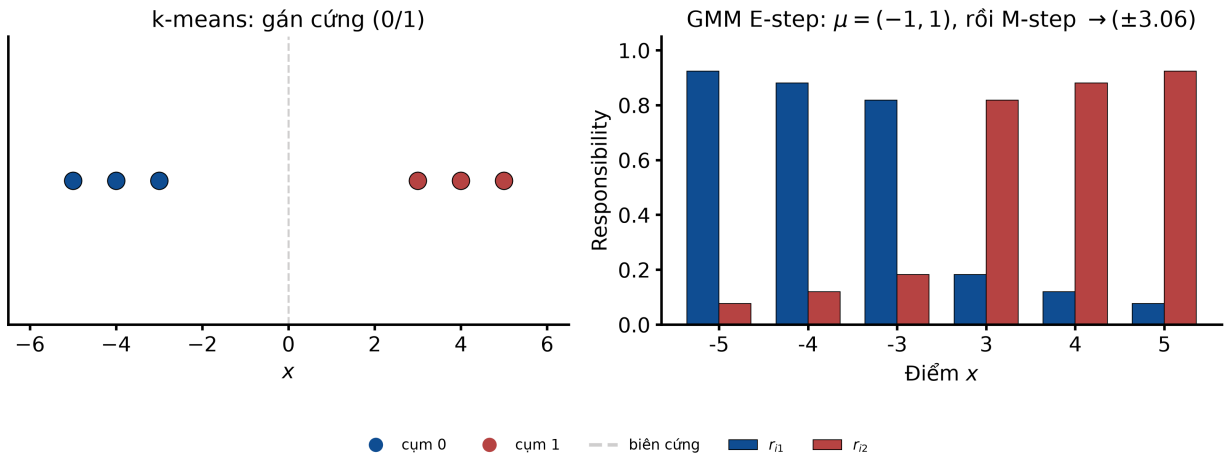
Đỉnh tại $p = n = 20$ là thật và rất mạnh, đúng dự đoán lý thuyết; sau đỉnh lỗi **không** giảm đơn điệu về 0 mà tăng nhẹ trở lại ở p rất lớn, vì nghiệm chuẩn tối thiểu phải "chia sẻ" biên độ cho các hướng nhiễu thuần; mức tốt nhất toàn cục vẫn là vùng p nhỏ, đúng cấu trúc ($p = 10 = d_{\text{true}}$). Bài 03 (deep double descent) đo trên mạng neural thật theo ba trục (độ rộng, epoch, lượng dữ liệu), bức tranh đầy đủ hơn nhiều.

Thông điệp thực hành: số tham số so với số mẫu không dự đoán trực tiếp overfitting như trực giác cổ điển; mô hình rất lớn có thể tổng quát hoá tốt, miễn cách tối ưu có xu hướng tìm nghiệm "đơn giản" trong số các nghiệm khớp hoàn hảo train (với mạng neural, SGD có xu hướng tương tự nhưng chưa hiểu rõ hoàn toàn, xem `math content/` các bài về NTK và benign overfitting).

4. Học không giám sát cổ điển: k-means, PCA, EM/GMM

4.1 k-means

Chia N điểm thành K cụm, mỗi cụm có một **centroid** (tâm), tối thiểu tổng bình phương khoảng cách từ mỗi điểm tới centroid cụm của nó:



k-means gán cứng vs GMM responsibility mềm (số preface); M-step đưa $\mu \to \approx \pm 3.06$ (minh hoạ, tự tính)

$$\min_{\{c_k\},\{z_i\}} \sum_{i=1}^{N} \|x_i - c_{z_i}\|_2^2$$

với $z_i \in \{1, \ldots, K\}$ là cụm của điểm i. Bài toán này NP-khó nếu tối ưu toàn cục; thuật toán **Lloyd** (thường gọi tắt là "k-means") lặp hai bước cho tới khi không đổi:

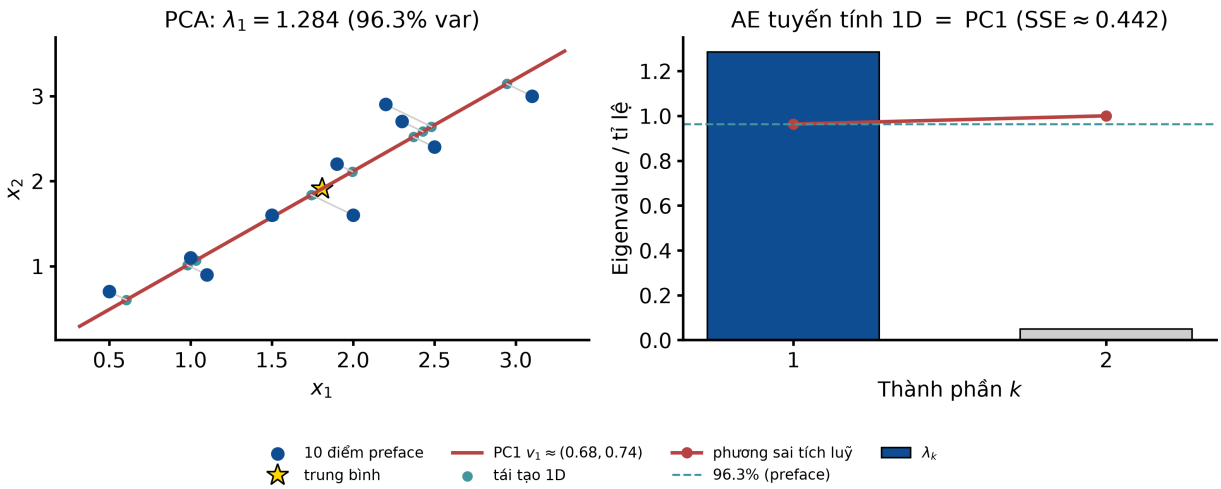
1. **Gán (assignment):** $z_i \leftarrow \arg\min_k \|x_i - c_k\|_2^2$ (gán mỗi điểm vào centroid gần nhất).
2. **Cập nhật (update):** $c_k \leftarrow$ trung bình của các điểm có $z_i = k$.

Đây chính là một trường hợp đặc biệt của **EM** (mục 4.3): bước gán giống E-step với gán cứng (hard assignment) thay vì xác suất mềm, bước cập nhật giống M-step.

Ví dụ tự tính. 7 điểm 2D: $(1,1), (1.5,2), (3,4), (5,7), (3.5,5), (4.5,5), (3.5,4.5)$. Khởi tạo 2 centroid tại $(1,1)$ và $(5,7)$. Vòng 1: $(1,1), (1.5,2)$ gần centroid 1, 5 điểm còn lại gần centroid 2; centroid mới $c_1 = (1.25, 1.5)$, $c_2 = (3.9, 5.1)$. Vòng 2: gán lại không đổi: **hội tụ** sau 1 vòng cập nhật thật. k-means luôn hội tụ hữu hạn bước nhưng có thể về **cực tiểu địa phương** khác nhau tùy khởi tạo (thực hành: chạy nhiều lần hoặc k-means++). K là siêu tham số, chọn bằng elbow method hoặc silhouette score.

4.2 PCA (Principal Component Analysis)

Bài toán: tìm K hướng trực giao (gọi là **principal component**) trong không gian đặc trưng, sao cho chiếu dữ liệu đã trừ trung bình lên các hướng đó giữ được **phương sai lớn nhất** (tương đương: sai số tái tạo nhỏ nhất khi nén về K chiều rồi dựng lại).



PCA 10 điểm preface: PC1 $\approx 96.3\%$ var; AE tuyến tính 1D = chiếu PC1 (SSE≈ 0.442) (minh hoạ, tự tính)

Với dữ liệu đã trừ trung bình $\tilde{X} \in \mathbb{R}^{N\times d}$ (mỗi hàng một điểm), ma trận hiệp phương sai $\Sigma = \frac{1}{N-1}\tilde{X}^\top \tilde{X} \in \mathbb{R}^{d\times d}$. Nghiệm: các principal component là các **eigenvector** của Σ, sắp theo **eigenvalue** giảm dần; eigenvalue chính là phương sai dữ liệu giải thích được theo hướng eigenvector tương ứng.

$$\boxed{\Sigma v_k = \lambda_k v_k, \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0}$$

Tỉ lệ phương sai giải thích bởi K component đầu: $\sum_{k\leq K} \lambda_k \big/ \sum_k \lambda_k$.

Ví dụ tự tính. 10 điểm 2D có tương quan dương rõ (dữ liệu: $(2.5, 2.4)$, $(0.5, 0.7)$, $(2.2, 2.9)$, $(1.9, 2.2)$, $(3.1, 3.0)$, $(2.3, 2.7)$, $(2.0, 1.6)$, $(1.0, 1.1)$, $(1.5, 1.6)$, $(1.1, 0.9)$). Trung bình $= (1.81,\ 1.91)$. Ma trận hiệp phương sai:

$$\Sigma = \begin{pmatrix} 0.6166 & 0.6154 \\ 0.6154 & 0.7166 \end{pmatrix}$$

Eigenvalue: $\lambda_1 = 1.284$, $\lambda_2 = 0.0491$. Eigenvector PC1: $v_1 \approx (0.678,\ 0.735)$. PC1 chiếm $1.284/(1.284 + 0.0491) \approx 96.3\%$ phương sai, PC2 chỉ 3.7% (hợp lý vì hai tọa độ tương quan rất mạnh). Chiếu 3 điểm đầu lên PC1: $[0.828,\ -1.778,\ 0.992]$. Khớp hoàn toàn với `sklearn.decomposition.PCA`.

Liên hệ autoencoder (mục 5.8): autoencoder tuyến tính với nút thắt 1 chiều có nghiệm tối thiểu sai số tái tạo đúng bằng phép chiếu lên PC1 ở trên (kiểm tự tính bằng SVD, sai số tái tạo ≈ 0.442). PCA là "autoencoder tuyến tính" theo đúng nghĩa đó.

4.3 EM và Gaussian Mixture Model (GMM)

Gaussian Mixture Model: mô hình sinh dữ liệu từ hỗn hợp K phân phối Gauss, mỗi điểm sinh ra bằng cách chọn thành phần k với xác suất π_k (prior của cụm) rồi lấy mẫu $\mathcal{N}(\mu_k, \Sigma_k)$. Mật độ tổng: $p(x) = \sum_k \pi_k \mathcal{N}(x; \mu_k, \Sigma_k)$. Khác k-means, GMM cho **gán mềm (soft assignment)**: mỗi điểm có một xác suất thuộc mỗi cụm, không gán cứng.

EM (Expectation-Maximization) là thuật toán lặp hai bước để khớp tham số (π_k, μ_k, Σ_k) bằng maximum likelihood khi có biến ẩn (cụm z_i chưa biết):

- **E-step:** với tham số hiện tại, tính **responsibility** $r_{ik} = P(z_i = k \mid x_i)$ bằng định lý Bayes: $r_{ik} = \dfrac{\pi_k \mathcal{N}(x_i; \mu_k, \Sigma_k)}{\sum_j \pi_j \mathcal{N}(x_i; \mu_j, \Sigma_j)}$.

- **M-step:** cập nhật tham số bằng trung bình có trọng số r_{ik}: $\mu_k \leftarrow \frac{\sum_i r_{ik} x_i}{\sum_i r_{ik}}$, $\pi_k \leftarrow \frac{1}{N}\sum_i r_{ik}$, tương tự cho Σ_k (phương sai có trọng số quanh μ_k mới).

Mỗi vòng EM không giảm log-likelihood (chứng minh bằng bất đẳng thức Jensen áp lên ELBO, cùng họ ý tưởng với VAE ở mục 5.9), nhưng chỉ hội tụ về cực trị địa phương.

Ví dụ tự tính (1 chiều, minh hoạ một vòng E-step và M-step). Dữ liệu 6 điểm thật ra có hai cụm rõ: $\{-5, -4, -3\}$ và $\{3, 4, 5\}$. Khởi tạo (cố ý lệch, gần nhau): $\mu_1 = -1, \mu_2 = 1$, phương sai $= 4$ cho cả hai, $\pi_1 = \pi_2 = 0.5$.

E-step: responsibility thuộc cụm 1 cho $[-5, -4, -3, 3, 4, 5]$ là $[0.924,\ 0.881,\ 0.818,\ 0.182,\ 0.119,\ 0.076]$ (ba điểm âm gần $\mu_1 = -1$ hơn $\mu_2 = 1$, dù cả hai mean chưa "đúng chỗ").

M-step: $\mu_1^{\text{mới}} = -3.064$, $\mu_2^{\text{mới}} = 3.064$ (nhảy gần -4 và 4 chỉ sau một vòng), phương sai cả hai cụm **tăng** lên 7.276 (so với 4 ban đầu) vì gán còn "mềm" (điểm -3 vẫn 18% xác suất thuộc cụm 2). Chạy EM tới hội tụ (sklearn, tự tính): $\mu \approx (4, -4)$ (nhãn cụm đổi chỗ, **label switching**, vô hại), $\pi = (0.5, 0.5)$, phương sai ≈ 0.667 cho cả hai, đúng phương sai thật của cụm 3 điểm quanh tâm nó.

---

5. Mạng neural: từ perceptron đến attention và mô hình sinh

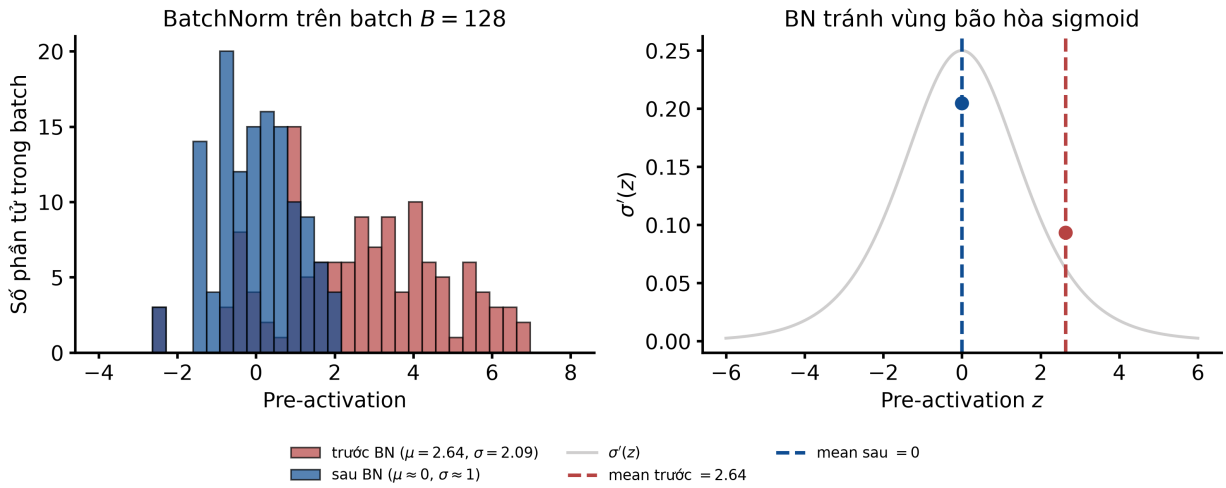
5.1 Perceptron

Mô hình neural đầu tiên (Rosenblatt, 1958, không phải arXiv): phân loại nhị phân $\hat{y} = \text{sign}(w^\top x + b)$. **Luật học perceptron:** với mỗi điểm bị phân loại sai, cập nhật $w \leftarrow w + \eta\, y\, x$, $b \leftarrow b + \eta\, y$ (không cập nhật nếu đã đúng).

Ví dụ tự tính. 4 điểm: $(1,1) \to +1$, $(2,-1) \to -1$, $(-1,-1) \to -1$, $(-2,1) \to +1$. Khởi tạo $w = [0,0], b = 0$, $\eta = 1$. Lần lượt: điểm 2 sai, $w \leftarrow [-2,1], b \leftarrow -1$; điểm 3 sai, $w \leftarrow [-1,2], b \leftarrow -2$; sang epoch 2, điểm 1 sai với w, b hiện tại, $w \leftarrow [0,3], b \leftarrow -1$. Từ đây cả 4 điểm đều đúng với $w = [0,3], b = -1$ (kiểm: $3 - 1 = 2 > 0$ cho điểm 1,4; $-3 - 1 = -4 < 0$ cho điểm 2,3): **hội tụ** sau 3 lần cập nhật. **Định lý hội tụ perceptron** (Novikoff, 1962): nếu dữ liệu tách tuyến tính được với margin $\gamma > 0$, perceptron hội tụ sau số bước hữu hạn, chặn trên tỉ lệ nghịch với γ^2. Nếu dữ liệu **không** tách tuyến tính (như XOR ở mục 3.5), perceptron không bao giờ hội tụ: đây là lý do lịch sử khiến nghiên cứu mạng neural chững lại trong thập niên 1970 (Minsky và Papert, 1969) cho tới khi có MLP và backprop.

5.2 MLP và backpropagation

Multi-Layer Perceptron (MLP): xếp nhiều tầng perceptron có activation phi tuyến, mỗi tầng $a^{(l)} = \phi(W^{(l)} a^{(l-1)} + b^{(l)})$. Không có activation phi tuyến ϕ, xếp nhiều tầng tuyến tính vẫn chỉ cho một hàm tuyến tính (tích các ma trận vẫn là một ma trận), nên ϕ là thứ cho MLP khả năng biểu diễn hàm phi tuyến phức tạp.



BatchNorm chuẩn hoá batch lệch về $(0, 1)$; tránh bão hòa sigmoid (minh hoạ, tự tính)

Backpropagation là cách tính gradient của loss theo mọi tham số bằng quy tắc chuỗi (chain rule), đi từ lớp cuối (gần loss) ngược về lớp đầu, tái sử dụng các đạo hàm trung gian (tránh tính lại từ đầu cho mỗi tham số, đây là lý do backprop có chi phí chỉ gấp khoảng 2-3 lần một lượt forward, không phải gấp số tham số lần).

Ví dụ tự tính: một lượt forward và backward đầy đủ trên mạng 2 input, 1 tầng ẩn 2 neuron (sigmoid), 1 output (sigmoid). Trọng số: $W^{(1)} = \begin{pmatrix} 0.3 & -0.2 \\ 0.4 & 0.1 \end{pmatrix}$, $b^{(1)} = [0.1, -0.1]$, $W^{(2)} = [0.5, -0.3]$, $b^{(2)} = 0.2$. Input $x = [0.5, 0.1]$, nhãn đích $y = 0.9$, loss $= \frac{1}{2}(a^{(2)} - y)^2$.

Forward: $z^{(1)} = W^{(1)}x + b^{(1)} = [0.23,\ 0.11]$; $a^{(1)} = \sigma(z^{(1)}) = [0.5572,\ 0.5275]$; $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} = 0.3204$; $a^{(2)} = \sigma(z^{(2)}) = 0.5794$; loss $= \frac{1}{2}(0.5794 - 0.9)^2 \approx 0.0514$.

Backward (dùng $\sigma'(z) = \sigma(z)(1 - \sigma(z))$): $\delta^{(2)} = (a^{(2)} - y)\,\sigma'(z^{(2)}) \approx (-0.3206) \times 0.2437 \approx -0.0781$. Gradient tầng 2: $\partial L/\partial W^{(2)} = \delta^{(2)}\,a^{(1)\top} \approx [-0.0435,\ -0.0412]$, $\partial L/\partial b^{(2)} = \delta^{(2)} \approx -0.0781$. Lan ngược: $\delta^{(1)} = (W^{(2)\top}\delta^{(2)}) \odot \sigma'(z^{(1)}) \approx [-0.00964,\ 0.00584]$. Gradient tầng 1: $\partial L/\partial W^{(1)} = \delta^{(1)}x^{\top} \approx \begin{pmatrix} -0.00482 & -0.000964 \\ 0.00292 & 0.000584 \end{pmatrix}$, $\partial L/\partial b^{(1)} = \delta^{(1)}$. Các giá trị được làm tròn để minh họa quy tắc chuỗi.

Trực giác cần nhớ: $\delta^{(l)}$ ("lỗi" lan truyền tới tầng l) luôn bằng (lỗi từ tầng sau, lan qua trọng số kết nối) nhân (đạo hàm activation tại điểm hoạt động hiện tại). Nếu đạo hàm activation rất nhỏ ở nhiều tầng liên tiếp (sigmoid bão hòa gần 0 hoặc 1 có đạo hàm gần 0), tích nhiều số nhỏ làm δ về gần 0 ở các tầng đầu: đây là **vanishing gradient**, lý do mạng rất sâu dùng sigmoid/tanh khó train, và lý do ReLU, residual connection (ml basics 07), và các kỹ thuật chuẩn hóa (`optimization content/38, 39`: batch norm, layer norm) ra đời.

5.3 Activation function

Activation	Công thức	Đạo hàm lớn nhất	Nhược điểm chính
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$	0.25 (tại $z = 0$)	bão hòa hai đầu, vanishing gradient, output không đối xứng quanh 0
Tanh	$\tanh(z)$	1.0 (tại $z = 0$)	vẫn bão hòa hai đầu
ReLU	$\max(0, z)$	1 (với $z > 0$), không khả vi tại 0	"neuron chết" nếu $z < 0$ luôn (gradient $= 0$ mãi)
Leaky ReLU	$\max(\alpha z, z)$, α nhỏ (ví dụ 0.01)	1	không còn hoàn toàn chết neuron nhưng vẫn gấp khúc

Activation	Công thức	Đạo hàm lớn nhất	Nhược điểm chính
GELU/SiLU (Swish)	$z\,\sigma(1.702z)$ (xấp xỉ GELU) hoặc $z\,\sigma(z)$	gần 1.1	tốn tính toán hơn ReLU, dùng phổ biến trong transformer hiện đại

5.4 Khởi tạo trọng số (initialization)

Khởi tạo tất cả 0 làm mọi neuron cùng tầng học giống nhau (gradient giống nhau do đối xứng), không khác gì một neuron: phải khởi tạo ngẫu nhiên để "phá vỡ đối xứng". Hai sơ đồ phổ biến, chọn phương sai khởi tạo để giữ phương sai activation ổn định qua các tầng (không nổ, không biến mất):

- **Xavier/Glorot** (cho sigmoid/tanh): $\text{Var}(W) = \dfrac{2}{n_{in} + n_{out}}$ (hoặc biến thể $1/n_{in}$).
- **Kaiming/He** (cho ReLU, vì ReLU "giết" một nửa activation về 0 nên cần phương sai gấp đôi để bù): $\text{Var}(W) = \dfrac{2}{n_{in}}$ (`optimization content/37`, chi tiết đạo hàm đầy đủ).

5.5 Convolutional Neural Network (CNN)

Convolution (tích chập, thực ra trong deep learning hầu hết là cross-correlation, không lật kernel): trượt một bộ lọc (kernel) nhỏ qua đầu vào, tại mỗi vị trí tính tích vô hướng giữa kernel và vùng đầu vào tương ứng.

Ví dụ tự tính (1D, để nhìn rõ cơ chế). Input $x = [1, 2, 3, 5, 8, 13, 21]$ (dãy Fibonacci), kernel $= [1, 0, -1]$ (bộ phát hiện biên/sai phân). Output (valid, trượt 1 bước): $[-2, -3, -5, -8, -13]$, đúng bằng $x_i - x_{i+2}$ tại mỗi vị trí: kernel này tính "độ dốc 2 bước", một bộ phát hiện thay đổi cục bộ.

Hai tính chất làm CNN hiệu quả cho ảnh (liên hệ bias-variance ở mục 2): **parameter sharing** (cùng kernel dùng ở mọi vị trí, giảm số tham số so với MLP nối đầy đủ, giảm variance) và **translation equivariance** (vật thể dịch chuyển thì feature map dịch chuyển tương ứng, một prior hợp lý cho ảnh, giảm bias). **Pooling** (max/average trên một vùng nhỏ) giảm kích thước không gian và tăng bất biến dịch cục bộ. ResNet (ml basics 07) thêm **residual connection** $a^{(l)} = \phi(W^{(l)}a^{(l-1)}) + a^{(l-1)}$ để gradient có đường tắt không qua activation, giải quyết vanishing gradient khi xếp hàng trăm tầng.

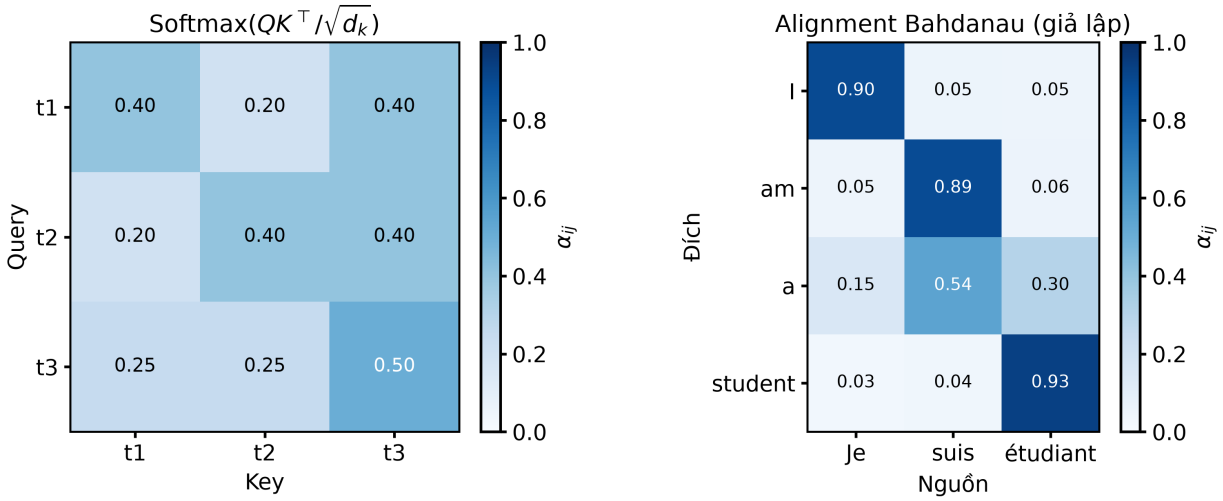
5.6 RNN, LSTM, GRU

Recurrent Neural Network (RNN): xử lý dữ liệu tuần tự bằng một trạng thái ẩn $h_t = \phi(W_{hh}h_{t-1} + W_{xh}x_t + b)$ cập nhật qua từng bước thời gian, dùng chung trọng số ở mọi bước (parameter sharing theo thời gian, giống CNN chia sẻ theo không gian). Vấn đề: backprop qua T bước thời gian (backprop through time, BPTT) nhân $\sigma'(\cdot)$ và W_{hh} liên tiếp T lần; nếu $\|W_{hh}\|$ hơi lớn hơn 1, gradient **nổ (exploding)**; nếu hơi nhỏ hơn 1, gradient **biến mất (vanishing)** theo cấp lũy thừa với T, khiến RNN thuần khó học quan hệ xa trong chuỗi dài.

LSTM (Hochreiter và Schmidhuber, 1997) và **GRU** (Cho et al., 2014) thêm **cổng (gate)** điều khiển luồng thông tin: LSTM có cổng quên/vào/ra cùng một "cell state" $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ cộng tuyến tính qua thời gian (không qua activation phi tuyến lặp lại), giúp gradient truyền xa hơn RNN thuần (giống residual connection nhưng theo chiều thời gian). GRU gộp cổng quên và vào thành một cổng cập nhật, ít tham số hơn, thường tương đương LSTM (ml basics 13 so sánh trực tiếp).

5.7 Attention

Scaled dot-product attention (công thức chuẩn của transformer, llm content/01): với truy vấn (query) Q, khóa (key) K, giá trị (value) V (mỗi hàng một token):



Ma trận trọng số scaled dot-product attention (ví dụ 3 token trong preface) và alignment mềm kiểu Bahdanau giả lập Anh–Pháp (minh hoạ, tự tính)

$$\boxed{\text{Attention}(Q,K,V) = \text{softmax}\Big(\frac{QK^\top}{\sqrt{d_k}}\Big)V}$$

$QK^\top$ đo "độ giống nhau" giữa mỗi query và key; chia $\sqrt{d_k}$ giữ phương sai điểm số ổn định khi d_k lớn (không chia, softmax bão hòa về one-hot, gradient gần 0, giống vanishing gradient mục 5.3); softmax biến điểm số thành trọng số dương tổng 1; nhân V là trung bình có trọng số các "giá trị" theo độ liên quan.

Ví dụ tự tính. 3 token, $d_k = 2$, $Q = K = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}$, $V = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$. Điểm số thô $QK^\top/\sqrt{2}$ cho token 1: $[0.707,\ 0,\ 0.707]$. Softmax: trọng số $\approx [0.401,\ 0.198,\ 0.401]$. Output cho token 1: $\approx [3.0,\ 4.0]$. Đây là **self-attention một đầu**; transformer thật dùng nhiều **head** song song, cộng **positional encoding** vì phép tính trên không phân biệt thứ tự token (**permutation equivariant**). Cơ chế gần giống ml basics 11 (Bahdanau attention, 2014, ra đời trước transformer, dùng cho RNN encoder-decoder, mục 5.6).

5.8 Autoencoder

Autoencoder: mạng gồm encoder $z = f_\theta(x)$ (nén x về biểu diễn ẩn chiều thấp z) và decoder $\hat{x} = g_\phi(z)$ (dựng lại), train bằng loss tái tạo $\|x - \hat{x}\|_2^2$, không cần nhãn. Nút thắt cổ chai ($\dim z < \dim x$) buộc mô hình học biểu diễn **nén** giữ thông tin quan trọng nhất. Như mục 4.2 trình bày, autoencoder tuyến tính với nút thắt K chiều học đúng không gian K principal component đầu của PCA; autoencoder phi tuyến có thể học cấu trúc phi tuyến mà PCA không nắm được, nên được xem như "PCA phi tuyến".

5.9 Mô hình sinh: GAN, VAE, diffusion (mức khung)

Ba họ mô hình sinh (generative model) chính, học phân phối $p(x)$ (hoặc một cách để lấy mẫu từ nó) từ dữ liệu không nhãn:

GAN (ml basics 14): hai mạng chơi minimax. Generator G biến nhiễu $z \sim p(z)$ thành dữ liệu giả $G(z)$; discriminator D cố phân biệt thật/giả; G cố lừa D:

$$\min_G \max_D \ \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))]$$

Nghiệm lý thuyết: G sinh đúng phân phối dữ liệu thật, $D \equiv 0.5$. Thực hành: minimax khó ổn định, hiện tượng **mode collapse** (G chỉ sinh vài kiểu output lặp lại) phổ biến.

VAE (ml basics 15): phiên bản xác suất của autoencoder. Encoder học $q_\phi(z \mid x)$ (thường Gauss) xấp xỉ posterior $p(z \mid x)$; decoder $p_\theta(x \mid z)$ sinh dữ liệu từ z; train bằng tối đa **ELBO**:

$$\text{ELBO} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - \text{KL}\big(q_\phi(z \mid x) \,\|\, p(z)\big)$$

Số hạng đầu là loss tái tạo; số hạng hai phạt q_ϕ lệch khỏi prior $p(z) = \mathcal{N}(0, I)$, giữ không gian ẩn "trơn" để lấy mẫu sinh dữ liệu mới được. **Reparameterization trick** ($z = \mu + \sigma \odot \epsilon$) cho phép lan gradient qua bước lấy mẫu ngẫu nhiên.

Diffusion (ml basics 16, DDPM): forward thêm nhiễu Gauss dần qua T bước tới khi thành nhiễu thuần; mô hình học reverse (dự đoán nhiễu $\epsilon_\theta(x_t, t)$ để khử dần) từ nhiễu thuần dựng lại dữ liệu. Nhiều bước khử nhiễu **nhỏ và dễ** thay vì sinh trực tiếp một bước **khó**, đổi lại cần hàng chục-hàng nghìn bước lấy mẫu (chậm hơn GAN/VAE lúc suy luận).

Mô hình	Huấn luyện ổn định?	Chất lượng mẫu điển hình	Tốc độ sinh	Có log-likelihood tường minh?
GAN	khó (minimax)	rất sắc nét nhưng dễ thiếu đa dạng	nhanh (1 lượt forward)	không trực tiếp
VAE	dễ (một loss, gradient thường)	thường mờ hơn GAN (do tối thiểu một chặn dưới, không phải likelihood thật)	nhanh (1 lượt forward)	chỉ có chặn dưới (ELBO)
Diffusion	dễ nhất (dự đoán nhiễu, loss MSE đơn giản)	tốt nhất trong ba họ ở nhiều benchmark hiện nay	chậm (nhiều bước)	có thể suy ra nhưng phức tạp

7. Cạm bẫy thường gặp

7.1 Data leakage (rò rỉ dữ liệu)

Thông tin từ validation/test "lọt" vào quá trình train một cách gián tiếp, làm số liệu báo cáo lạc quan giả. Các dạng hay gặp:

- **Leakage qua tiền xử lý:** chuẩn hóa (mean, std), chọn đặc trưng, hay PCA tính trên **toàn bộ** dữ liệu trước khi chia train/test; đúng ra phải fit trên train rồi áp dụng (transform) lên val/test.
- **Leakage qua nhóm:** nhiều mẫu của cùng một thực thể (nhiều ảnh cùng một trang tài liệu, nhiều dòng cùng một bệnh nhân) rơi vào cả train và test; mô hình "nhớ" thực thể đó chứ không học quy luật chung. Phải chia theo nhóm (group split), không chia theo từng mẫu.
- **Leakage thời gian:** dùng dữ liệu tương lai để dự đoán quá khứ (ví dụ đặc trưng tính trên toàn bộ lịch sử bao gồm cả sau thời điểm dự đoán). Với dữ liệu theo thời gian, phải chia theo mốc thời gian, không trộn ngẫu nhiên.
- **Leakage qua đặc trưng "ăn gian":** một đặc trưng vô tình chứa thông tin về nhãn mà sẽ không có lúc suy luận thật (ví dụ ID bệnh nhân tương quan với bệnh viện, bệnh viện tương quan với chẩn đoán).

Bài 21 trong thư mục này khảo sát leakage và các vấn đề reproducibility liên quan một cách hệ thống.

7.2 Shortcut learning (học đường tắt)

Mô hình đạt độ chính xác cao bằng cách khai thác một **tương quan giả (spurious correlation)** trong train set, không phải đặc trưng thật của tác vụ: ví dụ phân loại "chó sói" dựa vào có tuyết trong ảnh nền (vì ảnh train chó sói thường chụp ở vùng có tuyết) thay vì hình dạng con vật; mô hình OCR học "đoán" dựa vào vị trí/bố

cục trang thường gặp trong train thay vì đọc chữ thật. Mô hình vẫn đạt accuracy cao trên test set **nếu** test set có cùng tương quan giả, nhưng sụp hoàn toàn khi gặp dữ liệu không còn tương quan đó (shortcut không tổng quát hoá). Bài 20 trong thư mục này hệ thống hoá hiện tượng này qua nhiều lĩnh vực (ảnh, NLP, y tế).

Cách phát hiện: kiểm tra mô hình trên tập test **có chủ đích phá vỡ tương quan giả** (ví dụ chó sói trên cỏ, không tuyết); dùng kỹ thuật giải thích (saliency map, attention visualization) xem mô hình "nhìn" vào đâu; so sánh hiệu năng trên nhiều nguồn dữ liệu khác nhau.

7.3 Distribution shift và class imbalance

Distribution shift: phân phối dữ liệu lúc suy luận (production) khác phân phối lúc train. Hai dạng chính: **covariate shift** ($P(x)$ đổi nhưng $P(y \mid x)$ giữ nguyên, ví dụ ảnh test chụp bằng máy quét khác) và **label shift** (tỉ lệ lớp đổi nhưng đặc trưng mỗi lớp giữ nguyên). **Underspecification** (ml basics 22): khi nhiều mô hình có cùng hiệu năng train/test nhưng hành xử khác nhau trên dữ liệu lệch phân phối, vì quy trình train không đủ ràng buộc để chọn đúng một "cách giải" duy nhất trong số nhiều cách đạt cùng train loss.

Class imbalance: khi một lớp hiếm hơn nhiều lớp khác (ví dụ 1% dữ liệu là lớp dương), mô hình tối thiểu loss trung bình có thể đạt accuracy cao bằng cách gần như luôn đoán lớp đa số. Cách xử lý (ml basics 33 khảo sát trên CNN): **resampling** (oversample lớp hiếm hoặc undersample lớp đa số), **class weight** (tăng trọng số loss cho lớp hiếm), **focal loss** (ml basics 32, giảm trọng số các mẫu đã dễ phân loại đúng, tập trung vào mẫu khó, công thức $\text{FL} = -(1-p_t)^{\gamma} \log p_t$ với p_t là xác suất dự đoán cho lớp đúng, γ điều khiển mức giảm trọng số mẫu dễ).

7.4 P-hacking benchmark và đa so sánh

Khi một benchmark công khai được dùng lại hàng trăm, hàng nghìn lần bởi cộng đồng (hoặc nội bộ một nhóm thử rất nhiều biến thể rồi chỉ báo cáo biến thể tốt nhất trên test set), số tốt nhất trên benchmark đó dần vượt xa hiệu năng thật trên phân phối rộng hơn, hiện tượng tương tự **p-hacking** trong thống kê (thử nhiều cách tới khi có kết quả "có ý nghĩa" rồi chỉ báo cáo cách đó): nếu thử k mô hình ngẫu nhiên không khác nhau thật và chỉ chọn mô hình tốt nhất trên test set để báo cáo, điểm số tốt nhất đó kỳ vọng cao hơn hiệu năng thật của mô hình trung bình một lượng tăng theo k (và theo phương sai đánh giá, liên hệ trực tiếp "độ lệch chuẩn giữa các fold" ở mục 6.2: benchmark nhỏ có phương sai đo lớn, dễ bị "hên" một lần).

Thực hành giảm rủi ro: giữ một **held-out test set thật sự chỉ dùng một lần** (không nhìn khi phát triển); báo cáo nhiều seed; dùng **benchmark riêng** độc lập với benchmark dùng để chọn mô hình (ví dụ với OCR: tài liệu từ nguồn khác với bộ test công khai); hoài nghi khi một cải tiến nhỏ (dưới độ lệch chuẩn đo được) được quảng bá là "vượt trội".

9. Cách đọc paper ML một cách phản biện

1. **Baseline có công bằng không?** Một "phương pháp mới" có thắng vì bản chất tốt hơn, hay vì baseline bị tune kém, thiếu tiền xử lý, hay dùng ít dữ liệu hơn?
2. **Metric có đúng bài toán không?** Accuracy trên dữ liệu mất cân bằng, BLEU cho câu quá ngắn, PPL cho tác vụ sinh dài: mỗi metric có điểm mù riêng (mục 6).
3. **Train/val/test có tách đúng không?** Có nguy cơ leakage qua nhóm, qua thời gian, qua tiền xử lý không (mục 7.1)?
4. **Kết quả là một điểm hay một đường cong?** Một con số "tốt nhất" ở một mức siêu tham số không nói gì về độ ổn định; tìm độ lệch chuẩn giữa seed/fold (mục 6.2).
5. **Có đang shortcut không?** Độ chính xác cao có thể đến từ tương quan giả trong chính benchmark đó (mục 7.2); kiểm tra trên phân phối khác nếu có thể.
6. **Ablation có tách đúng nguyên nhân không?** Cải tiến đến từ ý tưởng chính, hay từ một chi tiết kỹ thuật phụ (tăng dữ liệu, tăng compute, tinh chỉnh kỹ hơn)?

7. **Benchmark có bị p-hack không?** Benchmark đó đã bị cộng đồng (hay chính nhóm tác giả) thử đi thử lại bao nhiêu lần (mục 7.4)?
8. **Paper có tự mâu thuẫn không?** Số trong bảng và số trong text có khớp? Công thức có đúng với định nghĩa đã nêu trước đó?
9. **Giả định nào ẩn trong công thức?** (i.i.d., tuyến tính, Gauss, đủ dữ liệu hiệu chuẩn...) Giả định đó có đúng với bài toán đang xét không?

---

10. Lịch đọc gợi ý cho 40 bài trong thư mục này

Nhóm theo 12 giai đoạn, giữ đúng thứ tự số bài (không đảo); có thể điều chỉnh thứ tự theo chủ đề quan tâm. Ba bài ở giai đoạn 8 và 12 (23, 24, 39) cùng chủ đề robustness/adversarial, nên có thể đọc nối với nhau trước nếu quan tâm riêng chủ đề đó, nhưng vẫn giữ nguyên số thứ tự ở đây.

Giai đoạn 1: lý thuyết tổng quát hoá (01-03).

Bài	File	Lý do đọc
01	`01_2016_rethinking_generalization.md`	gán nhãn ngẫu nhiên vẫn khớp hoàn hảo, đặt lại câu hỏi vì sao mạng tổng quát hoá tốt
02	`02_2018_bias_variance_reconciliation.md`	nối bias-variance cổ điển với double descent, minh chứng trên nhiều họ mô hình
03	`03_2019_deep_double_descent.md`	đo double descent trên mạng neural thật theo ba trục, đề xuất effective model complexity

Giai đoạn 2: mô hình cổ điển trên dữ liệu bảng (04-05).

Bài	File	Lý do đọc
04	`04_2016_xgboost.md`	biến gradient boosting (mục 3.8) thành hệ thống chạy nhanh trên dữ liệu lớn
05	`05_2022_tabular_tree_vs_deep.md`	benchmark có kiểm soát lý giải vì sao cây/boosting vẫn thắng mạng neural trên dữ liệu bảng

Giai đoạn 3: kiến trúc và regularization nền tảng (06-08).

Bài	File	Lý do đọc
06	`06_2012_dropout_co_adaptation.md`	bài gốc dropout, chống đồng thích nghi giữa các neuron
07	`07_2015_resnet.md`	residual connection, train được CNN hơn 1000 tầng
08	`08_2014_vgg.md`	khảo sát hệ thống độ sâu CNN với filter 3x3, đối chiếu ResNet

Giai đoạn 4: biểu diễn văn bản và chuỗi (09-13).

Bài	File	Lý do đọc
09	`09_2013_word2vec_cbow_skipgram.md`	học embedding từ rẻ hơn mô hình ngôn ngữ có tầng ẩn, self-supervised sớm

Bài	File	Lý do đọc
10	`10_2013_word2vec_negative_sampling.md`	negative sampling làm softmax trên vốn từ khổng lồ tính được
11	`11_2014_bahdanau_attention_nmt.md`	attention đầu tiên cho NMT, tiền thân của attention transformer
12	`12_2014_seq2seq_sutskever.md`	encoder-decoder bằng hai LSTM, nền cho dịch máy và sinh text
13	`13_2014_gru_vs_lstm_chung.md`	so sánh thực nghiệm tanh/LSTM/GRU, đặt tên chính thức cho GRU

Giai đoạn 5: mô hình sinh (14-16).

Bài	File	Lý do đọc
14	`14_2014_gan.md`	khung minimax generator/discriminator
15	`15_2013_vae_aevb.md`	suy luận biến phân, reparameterization trick, nền VAE/ELBO
16	`16_2020_ddpm.md`	mô hình khử nhiễu nhiều bước, họ sinh thứ ba

Giai đoạn 6: học tự giám sát contrastive (17-18).

Bài	File	Lý do đọc
17	`17_2020_simclr.md`	contrastive learning đơn giản, không cần memory bank
18	`18_2019_moco.md`	dictionary dạng hàng đợi và momentum encoder, đối chiếu SimCLR

Giai đoạn 7: tin cậy được của mô hình (19-22).

Bài	File	Lý do đọc
19	`19_2017_calibration.md`	đo miscalibration của mạng hiện đại, ECE/MCE, temperature scaling
20	`20_2020_shortcut_learning.md`	khung thống nhất nhiều kiểu thất bại tổng quát hoá
21	`21_2022_leakage_reproducibility.md`	taxonomy 8 loại data leakage trong ML dùng làm bằng chứng khoa học
22	`22_2020_underspecification.md`	nhiều predictor cùng hiệu năng iid nhưng khác nhau khi lệch phân phối

Giai đoạn 8: tính bền vững đối nghịch, phần 1 (23-24).

Bài	File	Lý do đọc
23	`23_2014_fgsm.md`	giải thích adversarial example bằng tính tuyến tính, đề xuất FGSM
24	`24_2013_intriguing_properties.md`	phát hiện sớm nhất về adversarial example

Giai đoạn 9: tổng quan và học biểu diễn/transfer (25-28).

Bài	File	Lý do đọc
25	`25_2014_deep_learning_overview_schmidhuber.md`	tổng quan lịch sử deep learning cho cả SL/UL/RL
26	`26_2012_representation_learning_review.md`	tổng quan ba hướng học biểu diễn đến 2012
27	`27_2014_how_transferable_features.md`	đo mức độ đặc trưng CNN chuyển giao được sang tác vụ khác
28	`28_2018_rethinking_imagenet_pretraining.md`	kiểm định lại: có thật sự cần ImageNet pretraining, tranh luận với bài 27

Giai đoạn 10: kỹ thuật huấn luyện thực dụng (29-33).

Bài	File	Lý do đọc
29	`29_2019_noisy_student.md`	self-training quy mô lớn bằng pseudo-label và nhiễu
30	`30_2019_label_smoothing.md`	làm mềm nhãn giúp generalization/calibration nhưng hại distillation
31	`31_2017_mixup.md`	tổ hợp lồi của cặp mẫu, thay ERM bằng Vicinal Risk Minimization
32	`32_2017_focal_loss.md`	hạ trọng số động mẫu dễ, xử lý mất cân bằng cực độ
33	`33_2017_class_imbalance_cnn.md`	khảo sát hệ thống các cách xử lý class imbalance trên CNN

Giai đoạn 11: đánh giá, uncertainty, ensemble (34-37).

Bài	File	Lý do đọc
34	`34_2009_cross_validation_survey.md`	khảo sát các thủ tục cross-validation, phân biệt estimation và identification
35	`35_2021_conformal_prediction_intro.md`	prediction set có bảo đảm coverage phân phối-tự do
36	`36_2016_deep_ensembles.md`	uncertainty bằng proper scoring rule, adversarial training, ensemble
37	`37_2015_mc_dropout_bayesian.md`	dropout lúc test tương đương một dạng biến phân, suy ra MC dropout

Giai đoạn 12: kiến trúc hiện đại, robustness nâng cao, giảm chiều (38-40).

Bài	File	Lý do đọc
38	`38_2022_convnext.md`	hiện đại hoá ResNet theo hướng Swin Transformer mà không dùng self-attention
39	`39_2017_pgd_adversarial_training.md`	nhìn robustness qua lăng kính min-max, PGD adversarial training
40	`40_2018_umap.md`	giảm chiều phi tuyến dựa trên hình học Riemann, đối chiếu PCA

Bài quan trọng nhất cho người mới hoàn toàn (nhận định của người biên soạn): bài **01** (`01_2016_rethinking_generalization.md`). Lý do: nó buộc người đọc đối mặt ngay với câu hỏi trung tâm của mục

2 (ERM và bias-variance có đủ giải thích ML hiện đại không?) bằng một thí nghiệm rất dễ hiểu (gán nhãn ngẫu nhiên), trước khi các bài sau (02, 03, 19-22) mở rộng theo nhiều hướng. Lựa chọn thứ hai đáng chú ý nếu ưu tiên tính thực dụng hàng ngày hơn là lý thuyết: bài **34** (cross-validation survey), vì gần như mọi quy trình ML thực tế đều cần cross-validation đúng cách (mục 6.2) trước khi bàn tới lý thuyết sâu hơn.

---

11. Gợi ý đọc theo chủ đề

Các chương sau mở rộng những khái niệm trong phần nền tảng:

- **Tổng quát hóa và độ phức tạp:** bài 01, bài 02 và bài 03.
- **Kiến trúc và biểu diễn:** ResNet, attention và tổng quan representation learning.
- **Đánh giá và độ tin cậy:** calibration, leakage, cross-validation và conformal prediction.
- **Mô hình sinh:** GAN, VAE và DDPM.

---

12. Năm câu tự kiểm tra

Câu 1. Trong ví dụ bias-variance tự tính ở mục 2.2, mô hình bậc 9 có bias$^2 \approx 0.0007$ (rất nhỏ) nhưng MSE ≈ 0.7525 (lớn hơn cả mô hình bậc 1 có bias$^2 \approx 0.2196$). Giải thích bằng công thức phân rã bias-variance.

Đáp án.

Với lỗi đo so với f^* như trong bảng, MSE $=$ bias$^2 +$ variance; nếu đo so với y nhiễu độc lập thì cộng thêm σ^2. Ở bậc 9, variance ≈ 0.7518 chiếm gần hết MSE: mô hình đủ phức tạp để khớp sát nhiễu riêng của mỗi tập train, nên dự đoán tại $x = 0.5$ dao động mạnh giữa các tập train. Ở bậc 1, bias cao (quá đơn giản để theo đường sin) nhưng variance rất thấp (đường thẳng ổn định). MSE bậc 9 lớn hơn vì variance tăng nhanh hơn bias giảm: đây chính là overfitting.

Câu 2. Vì sao bài toán XOR không thể giải bằng SVM kernel linear nhưng giải được bằng kernel polynomial bậc 2 hoặc bằng ánh xạ tay $(x_1, x_2) \mapsto (x_1, x_2, x_1 x_2)$?

Đáp án.

XOR: $(0,0) \to 0$, $(1,1) \to 0$, $(0,1) \to 1$, $(1,0) \to 1$, đối xứng hoàn toàn, không đường thẳng nào trong (x_1, x_2) tách được hai lớp. Kernel linear đạt độ chính xác train chỉ 0.5 (tự tính). Thêm đặc trưng $x_1 x_2$ tạo chiều mới mang thông tin "hai tọa độ cùng dấu hay không"; trong không gian 3 chiều này, siêu phẳng tuyến tính ($w \approx [2, 2, -4]$, $b \approx -1$) tách hoàn hảo cả 4 điểm. Kernel polynomial bậc 2 làm điều tương đương một cách ngầm, vì nó tính được tích vô hướng ứng với số hạng bậc 2 như $x_1 x_2$ mà không cần viết tay φ.

Câu 3. Một nút cha có 4 mẫu lớp 0 và 4 mẫu lớp 1 (Gini $= 0.5$). Một luật chia tách hoàn hảo: nhánh trái 4 mẫu lớp 0, nhánh phải 4 mẫu lớp 1. Tính Gini sau khi chia và information gain.

Đáp án.

Cả hai nhánh thuần khiết: Gini trái $= 1 - 1^2 - 0^2 = 0$, Gini phải $= 0$. Gini trung bình có trọng số $= \frac{4}{8} \times 0 + \frac{4}{8} \times 0 = 0$. Information gain $= 0.5 - 0 = 0.5$: mức gain tối đa có thể, vì cả hai nhánh sau chia đều thuần khiết tuyệt đối.

Câu 4. Trong ví dụ backprop tay ở mục 5.2, $\delta^{(2)} \approx -0.0781$ và $\sigma'(z^{(1)})$ có giá trị khoảng 0.247 và 0.249 (gần giá trị lớn nhất 0.25 của đạo hàm sigmoid). Nếu xếp thêm 3 tầng ẩn sigmoid nữa, mỗi tầng cũng có $\sigma' \approx 0.25$ và trọng số liên kết cỡ 1, hãy ước lượng δ ở tầng xa nhất giảm theo hệ số bao nhiêu so với $\delta^{(2)}$, và nêu ý nghĩa.

Đáp án.

Mỗi tầng lan ngược nhân thêm hệ số cỡ $\sigma'(z) \approx 0.25$. Qua 3 tầng thêm, hệ số nhân dồn $\approx 0.25^3 = 0.015625$: δ ở tầng xa nhất chỉ còn khoảng 1.6% so với $\delta^{(2)}$ ban đầu. Đây là cơ chế **vanishing gradient**: mỗi tầng sigmoid cộng thêm làm gradient về các tầng đầu nhỏ đi theo cấp lũy thừa, khiến mạng rất sâu học chậm nếu không có residual connection (mục 5.5), ReLU (mục 5.3, đạo hàm bằng 1 khi active), hay batch/layer normalization (`optimization content/38, 39`).

Câu 5. Một nhóm phát triển OCR báo cáo: "model đạt CER 2% trên test set nội bộ". Dựa vào mục 6.1, 6.2, 7.1, 7.4, hãy nêu ít nhất ba câu hỏi cần đặt ra trước khi tin số này phản ánh hiệu năng thật trên dữ liệu người dùng.

Đáp án.

Ví dụ: (1) Test set nội bộ tách khỏi train/validation theo nhóm (cùng tài liệu/nguồn scan không rơi vào cả hai) hay có leakage (mục 7.1)? (2) Số 2% có kèm độ lệch chuẩn/khoảng tin cậy, hay chỉ một lần chạy trên tập nhỏ (mục 6.2)? (3) Test set đó đã bị dùng chọn đi chọn lại giữa nhiều phiên bản model trước khi báo số cuối, có nguy cơ p-hacking benchmark (mục 7.4) không? (4) Phân phối tài liệu test có giống dữ liệu người dùng thật, hay có nguy cơ distribution shift/shortcut learning (mục 7.2, 7.3)?

Tài liệu này không có đường link gốc vì không ghi chú một paper cụ thể. Sách nền nên đọc kèm: Hastie, Tibshirani, Friedman, "The Elements of Statistical Learning" (bản 2, 2009) cho mục 2-4; Goodfellow, Bengio, Courville, "Deep Learning" (2016) cho mục 5; Murphy, "Probabilistic Machine Learning: An Introduction" (2022) và Bishop, "Pattern Recognition and Machine Learning" (2006) hoặc Bishop và Bishop, "Deep Learning: Foundations and Concepts" (2023) cho cả hai. Không trích số trang vì đây là kiến thức nền chung, không phải trích dẫn một ấn bản cụ thể.

Bài 01

Understanding deep learning requires rethinking generalization (2016)

Tác giả: Chiyuan Zhang (MIT), Samy Bengio, Moritz Hardt (Google Brain), Benjamin Recht (UC Berkeley), Oriol Vinyals (Google DeepMind).
Venue / arXiv: arXiv:1611.03530 (v2, 26/02/2017); văn bản trích xuất không ghi dòng venue hội nghị; theo trích dẫn của bài khác (Belkin et al. 2018, xem bài 02) đây là ICLR 2017 (ngoài paper, kiến thức chung).
Nguồn: https://arxiv.org/abs/1611.03530
Chủ đề: thử nghiệm "randomization test" (huấn luyện với nhãn/pixel ngẫu nhiên) để chỉ ra VC-dimension, Rademacher complexity, uniform stability và regularization tường minh đều không giải thích được vì sao mạng nơ-ron sâu generalize tốt; kèm chứng minh mạng 2 tầng có thể biểu diễn mọi hàm trên mẫu hữu hạn.
Vị trí trong lộ trình: Bài mở đầu cho nhóm về tổng quát hóa; đọc tiếp bài 02 về double descent và bài 03 về hiện tượng này trong mạng sâu.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Mạng nơ-ron sâu thường có số tham số lớn hơn nhiều số mẫu huấn luyện (ví dụ Inception trên CIFAR10 có 1,6 triệu tham số cho 50.000 ảnh). Theo lý thuyết học thống kê cổ điển, một mô hình "đủ lớn để ghi nhớ bất cứ gì" (overparameterized) phải cần regularization tường minh (weight decay, dropout, ...) hoặc bị giới hạn bởi một thước đo độ phức tạp (VC-dimension, Rademacher complexity, uniform stability) để không overfit. Nhưng thực tế, các mạng rất lớn vẫn generalize tốt (lỗi huấn luyện nhỏ, lỗi kiểm tra cũng nhỏ) mà không cần lý do rõ ràng từ các thước đo đó.

Khái niệm cần nắm trước khi đọc:

- **Generalization error / gap**: hiệu giữa lỗi huấn luyện và lỗi kiểm tra.
- **VC-dimension, Rademacher complexity $\hat{R}_n(H)$ (công thức 1 trong bài báo)**: các thước đo "khả năng của một họ hàm H khớp được bao nhiêu loại nhãn ngẫu nhiên ± 1". $\hat{R}_n(H) = \mathbb{E}_\sigma\left[\sup_{h\in H} \frac{1}{n}\sum_{i=1}^{n} \sigma_i h(x_i)\right]$ với $\sigma_i \in \{\pm 1\}$ ngẫu nhiên đều.
- **Uniform stability**: đo độ nhạy của thuật toán học khi đổi một mẫu huấn luyện; không phụ thuộc vào nhãn thật.
- **Randomization test**: kỹ thuật thống kê phi tham số (non-parametric), ở đây là so sánh hành vi mô hình khi huấn luyện trên nhãn/ảnh thật và trên nhãn/ảnh ngẫu nhiên.

Câu hỏi của bài báo: cái gì phân biệt một mạng nơ-ron generalize tốt với một mạng generalize kém, nếu không phải là các thước đo lý thuyết cổ điển?

2. Ý tưởng chính

Nhóm tác giả đưa ra 3 phát hiện thực nghiệm và 1 kết quả lý thuyết:

1. **"Deep neural networks easily fit random labels"**: khi thay toàn bộ nhãn huấn luyện bằng nhãn ngẫu nhiên, các mạng chuẩn (Inception, AlexNet, MLP) vẫn đạt lỗi huấn luyện = 0, dù lỗi kiểm tra chỉ bằng đoán ngẫu nhiên. Việc này chỉ chậm hơn một hằng số nhỏ so với huấn luyện trên nhãn thật, và không cần đổi siêu tham số.

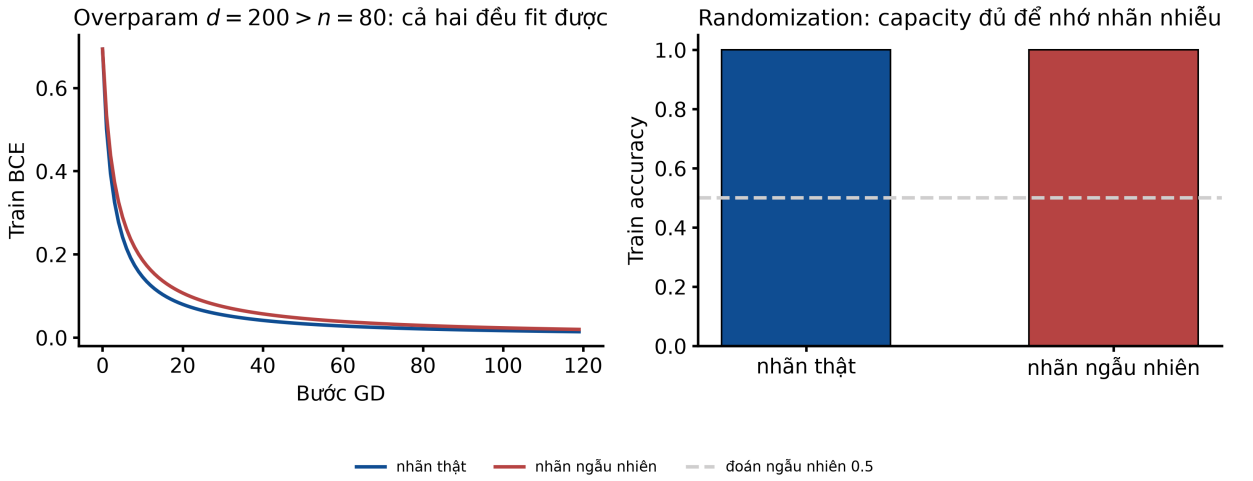
2. **Regularization tường minh (weight decay, dropout, data augmentation) không phải điều kiện cần cũng không phải điều kiện đủ** để kiểm soát generalization error; nó cải thiện lỗi kiểm tra như một "tham số tinh chỉnh" hơn là một cơ chế nền tảng.
3. **Biểu diễn hữu hạn mẫu (finite-sample expressivity)**: một mạng ReLU 2 tầng với $p = 2n + d$ tham số có thể biểu diễn bất kỳ gán nhãn nào cho bất kỳ mẫu kích thước n trong không gian d chiều. Vì số tham số thực tế của các mạng sâu thường vượt số mẫu, về lý thuyết chúng "đủ sức" ghi nhớ toàn bộ dữ liệu huấn luyện, nên thước đo độ phức tạp theo kiểu VC/Rademacher mất tác dụng (chúng cho $\hat{R}_n(H) \approx 1$, một chặn trên vô dụng).
4. **Implicit regularization qua SGD**: với mô hình tuyến tính, SGD khởi tạo ở $w_0 = 0$ luôn hội tụ về nghiệm có chuẩn ℓ_2 nhỏ nhất trong số các nghiệm khớp hoàn toàn dữ liệu (kernel trick). Đây là một lời giải thích khả dĩ (nhưng chưa đầy đủ) cho vì sao SGD "tự" tìm ra nghiệm generalize được mà không cần regularizer tường minh.

3. Phương pháp chi tiết

3.1 Randomization test (Mục 2 của paper)

Trên cùng kiến trúc, cùng hyperparameter, paper so sánh 6 biến thể dữ liệu:

- **True labels**: nhãn gốc.
- **Partially corrupted labels**: với xác suất p mỗi nhãn bị đổi thành một lớp ngẫu nhiên đều.
- **Random labels**: toàn bộ nhãn bị đổi ngẫu nhiên.
- **Shuffled pixels**: chọn một phép hoán vị pixel cố định, áp dụng cho mọi ảnh (train và test).
- **Random pixels**: mỗi ảnh có phép hoán vị pixel riêng.
- **Gaussian**: sinh pixel từ phân phối Gauss khớp trung bình/phương sai ảnh gốc (không còn cấu trúc ảnh).



Overparam $d > n$: fit được cả nhãn thật và nhãn ngẫu nhiên (randomization test) (minh hoạ, tự tính)

Thí nghiệm trên hai bộ dữ liệu: **CIFAR10** (Inception nhỏ, AlexNet nhỏ, MLP) và **ImageNet ILSVRC 2012** (Inception V3, huấn luyện phân tán async SGD với 50 worker). Trên CIFAR10 dùng SGD với momentum 0,9, learning rate khởi điểm 0,1 (Inception nhỏ) hoặc 0,01 (AlexNet nhỏ/MLP), decay 0,95 mỗi epoch; khi thử nhãn/pixel ngẫu nhiên, tắt toàn bộ regularizer tường minh trừ khi nói khác.

3.2 Khảo sát vai trò của regularization (Mục 3)

So sánh có/không: **data augmentation** (crop ngẫu nhiên, lật, xoay, ...), **weight decay** (ℓ_2 trên trọng số), **dropout** (chỉ dùng ở Inception V3 trên ImageNet trong thí nghiệm này). Đo train/test accuracy khi tắt từng regularizer, và so sánh thêm hiệu ứng ngầm (implicit) của **early stopping** và **batch normalization** (bật/tắt BN trong một biến thể "Inception w/o BatchNorm" giống Inception nhưng bỏ hết lớp BN).

3.3 Biểu diễn hữu hạn mẫu (Mục 4, chứng minh ở Phụ lục C)

Định lý 1 (paper). Tồn tại mạng ReLU 2 tầng với $2n + d$ trọng số có thể biểu diễn bất kỳ hàm nào trên một mẫu kích thước n trong d chiều.

Ý tưởng chứng minh: xây hàm

$$c(x) = \sum_{j=1}^{n} w_j \max\{\langle a, x\rangle - b_j, 0\}$$

Chọn a, b sao cho $x_i = \langle a, z_i\rangle$ thoả tính chất lồng nhau $b_1 < x_1 < b_2 < \cdots < b_n < x_n$ (khả thi vì các z_i phân biệt). **Bổ đề 1**: với dãy lồng nhau này, ma trận $A = [\max\{x_i - b_j, 0\}]_{ij}$ là tam giác dưới và có hạng đầy (giá trị riêng nhỏ nhất $= \min_i x_i - b_i > 0$), nên hệ $y = Aw$ luôn giải được. Số tham số: $a \in \mathbb{R}^d$ (d tham số), $b, w \in \mathbb{R}^n$ ($2n$ tham số) $\Rightarrow p = 2n + d$ (tự tính: cộng $d + n + n$).

Hệ quả 1: đánh đổi chiều rộng lấy chiều sâu, với độ sâu k tuỳ ý, tồn tại mạng rộng $O(n/k)$, tổng $O(n + d)$ trọng số, vẫn biểu diễn được mọi hàm trên mẫu n điểm (chia miền giá trị x_i thành b khoảng, mỗi khoảng n/b điểm xử lý ở một tầng, cộng thêm một "multiplexer" chọn đầu ra theo khoảng).

3.4 Implicit regularization qua mô hình tuyến tính (Mục 5)

Với ERM tuyến tính $\min_w \frac{1}{n}\sum_i \text{loss}(w^T x_i, y_i)$ (phương trình 2), nếu $d \geq n$ và ma trận dữ liệu X ($n \times d$) có hạng n, hệ $Xw = y$ có vô số nghiệm. Hessian của hàm mục tiêu không phụ thuộc w và suy biến tại mọi điểm tối ưu, nên "độ cong" không phân biệt được nghiệm nào generalize tốt.

Chạy SGD từ $w_0 = 0$: cập nhật $w_{t+1} = w_t - \eta_t e_t x_{i_t}$ luôn cho $w = \sum_i \alpha_i x_i$ (nằm trong span dữ liệu, $w = X^T\alpha$). Khớp hoàn toàn nhãn ($Xw = y$) dẫn tới phương trình kernel:

$$XX^T\alpha = y \quad \text{(phương trình 3)}$$

chỉ phụ thuộc tích vô hướng giữa các điểm dữ liệu (đây chính là kernel trick, suy ra "một cách vòng vo" theo lời tác giả). Nghiệm này tương đương nghiệm có chuẩn ℓ_2 nhỏ nhất của $Xw = y$: trong số mọi w khớp dữ liệu, SGD hội tụ về nghiệm chuẩn nhỏ nhất.

4. Thí nghiệm và kết quả chính

Fitting random labels (Hình 1, Bảng 1 và 2). Trên CIFAR10 và ImageNet, mọi kiến trúc thử nghiệm đều đạt train accuracy 100% (hoặc rất gần) khi nhãn bị xáo trộn hoàn toàn, không cần đổi hyperparameter:

Model (CIFAR10)	#params	Cấu hình	Train acc	Test acc
Inception	1.649.402	crop có, wd có	100,0	89,05
Inception	1.649.402	crop không, wd không	100,0	85,75
Inception (nhãn ngẫu nhiên)	1.649.402	không reg	100,0	9,78
Inception w/o BatchNorm (nhãn ngẫu nhiên)	1.649.402	không reg	100,0	10,12
AlexNet (nhãn ngẫu nhiên)	1.387.786	không reg	99,82	9,86
MLP 3x512 (nhãn ngẫu nhiên)	1.735.178	không reg	100,0	10,48
MLP 1x512 (nhãn ngẫu nhiên)	1.209.866	không reg	99,34	10,61

(nguồn: Bảng 1 của paper; test accuracy ~10% khi nhãn ngẫu nhiên đúng bằng đoán ngẫu nhiên trên 10 lớp).

Trên **ImageNet** (1000 lớp, Inception V3, Bảng 2 phụ lục): với nhãn ngẫu nhiên, không augmentation/weight decay/dropout, train top-1 đạt **95,20%** (top-5 99,14%), test top-1 chỉ 0,11% (ngẫu nhiên lý thuyết là 0,1%).

Paper nhấn mạnh: dù chưa đạt 100% tuyệt đối như CIFAR10, 95,2% trên 1 triệu nhãn ngẫu nhiên trong 1000 lớp "vẫn rất đáng ngạc nhiên", và không có tinh chỉnh hyperparameter nào được thực hiện khi đổi từ nhãn thật sang nhãn ngẫu nhiên (paper suy đoán có thể đạt 100% nếu tinh chỉnh thêm).

Partially corrupted labels (Hình 1b, 1c). Mọi mức nhiễu nhãn từ 0 đến 1 đều được khớp hoàn hảo (train error luôn 0); thời gian hội tụ tăng dần theo mức nhiễu; test error (= generalization error vì train error = 0) tăng dần và tiến tới ~90% khi nhiễu tiến tới 1 (bằng tỉ lệ đoán ngẫu nhiên trên CIFAR10 10 lớp).

Vai trò regularization trên ImageNet (Bảng 2 phụ lục, nhãn thật):

aug	weight decay	dropout	train top-1/top-5	test top-1/top-5
có	có	có	92,18 / 99,21	77,84 / 93,92
có	không	không	92,33 / 99,17	72,95 / 90,43
không	có	không	90,60 / 100,0	67,18 (72,57) / 86,44 (91,31)
không	không	không	99,53 / 100,0	59,80 (63,16) / 80,38 (84,49)

(số trong ngoặc là best test accuracy đạt được trong quá trình huấn luyện, dùng để minh hoạ tiềm năng của early stopping). Khi tắt hết regularizer, top-1 test giảm còn 59,80% so với 77,84% khi bật hết, tức giảm khoảng **18 điểm %** (paper nêu rõ số "18%"; tự tính chính xác: $77{,}84 - 59{,}80 = 18{,}04$ điểm %). Riêng augmentation (không kèm wd/dropout) đạt 72,95% top-1, cho thấy augmentation đóng góp nhiều hơn weight decay/dropout. Inception không regularizer đạt top-5 80,38%, so với AlexNet vô địch ILSVRC 2012 là 83,6% (paper dẫn lại số của Krizhevsky et al. 2012) – tác giả kết luận thay đổi kiến trúc mang lại lợi ích lớn hơn là thêm regularizer.

Batch normalization và early stopping (Hình 2). Trên CIFAR10, so Inception có/không BN (tắt hết regularizer khác): BN giúp ổn định quá trình học nhưng chỉ cải thiện test accuracy khoảng **3-4%**. Trên CIFAR10, early stopping không cho thấy lợi ích rõ; trên ImageNet, early stopping "có thể" (paper dùng "potentially" và chú thích rằng cần một test set tách biệt để khẳng định chặt) cải thiện generalization khi các regularizer khác bị tắt.

Kernel / mô hình tuyến tính (Mục 5, Bảng 3 phụ lục, dữ liệu MNIST/CIFAR10):

Dữ liệu	Tiền xử lý	Test error
MNIST	không	1,2%
MNIST	Gabor wavelet	0,6%
CIFAR10	không (Gaussian kernel trên pixel)	46%
CIFAR10	random conv-net (32.000 filter ngẫu nhiên)	17% (giảm tiếp còn 15% khi thêm ℓ_2 regularization, không dùng augmentation)

Ma trận kernel MNIST cần 30GB RAM, giải trong dưới 3 phút trên máy 24 core/256GB RAM bằng LAPACK thông thường. Thêm regularization **không** cải thiện performance ở cả hai trường hợp MNIST (điều ngược với kỳ vọng "cần regularizer để tránh overfit"). Chuẩn ℓ_2 của nghiệm minimum-norm: MNIST không tiền xử lý ≈ 220; với Gabor wavelet, chuẩn tăng lên ≈ 390 nhưng test error lại giảm một nửa (từ 1,2% xuống 0,6%) – paper dùng đây làm minh chứng rằng "chuẩn nhỏ nhất" chỉ là một phần nhỏ của câu chuyện generalization, không phải chỉ báo đầy đủ.

Vì đây là paper thực nghiệm + 1 định lý biểu diễn (không phải bài toán tối ưu benchmark theo nghĩa thông thường), không có "SOTA number" để so sánh; mục đích là chứng minh/bác bỏ giả thuyết lý thuyết bằng thực nghiệm có kiểm soát.

5. Phân tích / ablation

- **Random pixels/Gaussian hội tụ nhanh hơn random labels** (Hình 1a): tác giả giải thích vì ảnh đã bị làm "tách biệt" hơn (less similar to each other) so với ảnh tự nhiên cùng lớp, nên mạng dễ gán nhãn tuỳ ý hơn.
- **AlexNet với weight decay thất bại khi khớp nhãn ngẫu nhiên** (Bảng 4 phụ lục: "Failed to converge"), trong khi Inception, MLP 3x512, MLP 1x512 vẫn khớp được (100%, 100%, 99,21%). Đây là trường hợp ngoại lệ duy nhất ghi nhận được của regularization "chặn" được việc fit nhãn ngẫu nhiên ở hệ số mặc định.
- **Augmentation + nhãn ngẫu nhiên (Phụ lục E)**: ở hệ số weight decay mặc định (0,95 → ... paper ghi "0,95 to 0,999" khi đổi hệ số), random cropping cho train accuracy 99,93%, augmentation đầy đủ cho 99,28% – vẫn gần như khớp hoàn toàn; chỉ khi **tăng hệ số weight decay lên 0,999 và huấn luyện lâu hơn** mới quan sát được hiện tượng không khớp hoàn toàn (early sign of overfitting prevention) với cả hai augmentation.
- **Ablation BN** (Inception vs Inception w/o BatchNorm, Bảng 1): cả fitting nhãn ngẫu nhiên lẫn nhãn thật đều cho chênh lệch nhỏ (ví dụ nhãn thật test 85,75% vs 82,00% không BN tức khoảng 3-4 điểm %, không BN cũng vẫn fit random label tới 100% train / chỉ 10,12% test) – BN không phải "chìa khoá" của hiện tượng ghi nhớ.
- **So sánh population-level vs finite-sample expressivity**: các kết quả cũ (Cybenko 1989, Telgarsky 2016, ...) chứng minh khả năng biểu diễn trên toàn miền hàm, cần cỡ mẫu đa thức theo chiều và hàm mũ theo độ sâu để suy ra kết quả hữu hạn mẫu – không thực tế. Paper chỉ ra cách tiếp cận trực tiếp ở mức hữu hạn mẫu đơn giản hơn nhiều bậc.

6. Giới hạn

Do tác giả nêu:

- Thí nghiệm ImageNet không đạt train accuracy 100% tuyệt đối với nhãn ngẫu nhiên (chỉ 95,20%) và tác giả thừa nhận có thể cần tinh chỉnh hyperparameter thêm để đạt 100%, nhưng không thử.
- Chuẩn ℓ_2 nhỏ nhất "không dự đoán được generalization" (ví dụ MNIST wavelet) – tác giả tự nhận đây chỉ là "a very small piece of the generalization story", chưa giải thích đầy đủ vì sao SGD hay kiến trúc cụ thể generalize tốt hơn kiến trúc khác.
- Lý thuyết implicit regularization chỉ phân tích được cho **mô hình tuyến tính**; với mạng nơ-ron sâu phi tuyến thật, paper chỉ nêu "cần nghiên cứu thêm" (Mục 1.1), không chứng minh gì tương tự cho mạng sâu.
- Không giải thích được tại sao một số kiến trúc generalize tốt hơn kiến trúc khác dù cùng khả năng ghi nhớ nhãn ngẫu nhiên.

Nhận xét:

- Thí nghiệm chỉ trên ảnh phân loại (CIFAR10, ImageNet); không có ngôn ngữ, không có OCR/VLM, nên kết luận về "overparameterization luôn memorize" cần kiểm lại cho miền khác.
- Bảng 2 có vài số trong ngoặc khác số ngoài (best test trong quá trình huấn luyện) – đây là hai đại lượng khác nhau (test cuối vs best test), người đọc cần phân biệt rõ khi trích dẫn.
- Không có thí nghiệm đo trực tiếp "khoảng cách" giữa random-label-fit và real-label-fit theo kích thước mô hình (đây chính là việc bài 02/03 sẽ làm bằng double descent) – 01 chỉ dừng ở "overparam đủ để ghi nhớ", chưa dựng đường cong risk theo capacity.

7. Liên hệ với các bài khác

- **ml basics 02** (Belkin et al., Reconciling modern ML practice and bias-variance trade-off): trực tiếp lấy cảm hứng từ hiện tượng "mô hình khớp hoàn toàn dữ liệu vẫn generalize tốt" mà bài 01 chứng minh

bằng thực nghiệm, và đặt nó vào một đường cong "double descent" thống nhất với U-curve cổ điển; bài 02 trích dẫn thẳng bài 01 ([42] trong Belkin et al.).

- **ml basics 03** (Nakkiran et al., Deep Double Descent): mở rộng ý tưởng của 01 và 02 sang cả kích thước mô hình, số epoch, kích thước dữ liệu; vẫn dùng chung quan sát "mô hình quá tham số hoá không overfit theo nghĩa cổ điển".
- **ml basics 06** (Hinton et al., dropout gốc 2012): là một trong các regularizer tường minh được bài 01 đánh giá lại ("không cần thiết, không đủ") – nên đọc 06 trước để biết dropout "được kỳ vọng" làm gì, rồi đọc 01 để thấy giới hạn của kỳ vọng đó.
- **pruning content/08_2018_lottery_ticket_hypothesis.md**: cũng khai thác tính overparameterization của mạng (tìm "winning ticket" nhỏ bên trong mạng lớn); câu hỏi liên quan: nếu mạng lớn chỉ cần một phần nhỏ tham số để đạt hiệu năng, thì phần tham số "dư" trong bài 01 thực chất dùng để ghi nhớ gì?
- **pruning content/12_2020_state_of_neural_network_pruning.md**: tổng kết nhiều nghiên cứu về overparameterization và pruning, có thể dùng để đối chiếu quan điểm "số tham số lớn không đồng nghĩa với overfit" của bài 01 với thực hành pruning hiện tại.
- **llm content/03_2020_scaling_laws_kaplan.md**: scaling laws cho thấy mô hình lớn hơn (nhiều tham số hơn) có loss thấp hơn một cách có hệ thống, phần nào cùng tinh thần "mô hình lớn không tự động overfit" mà bài 01 chỉ ra ở quy mô nhỏ hơn (CIFAR10/ImageNet, kiến trúc 2016).

8. Gợi ý áp dụng

Khả năng khớp dữ liệu huấn luyện không đủ để chứng minh khả năng tổng quát hóa. Hãy theo dõi kết quả trên tập kiểm tra độc lập và xem regularization như một yếu tố cần đánh giá thực nghiệm, thay vì lời giải duy nhất cho khoảng cách train–test.

9. Thí nghiệm gợi ý

1. **Tái lập randomization test ở quy mô nhỏ**: dùng CNN nhỏ (vài trăm nghìn tham số) trên subset CIFAR10 (ví dụ 5.000 ảnh) chạy; huấn luyện với nhãn thật và nhãn ngẫu nhiên hoàn toàn; đo train/test accuracy và số epoch tới khi train accuracy = 100%. Kỳ vọng: train accuracy tiến về 100% ở cả hai trường hợp, test accuracy nhãn ngẫu nhiên gần 10% (ngẫu nhiên); diễn giải: xác nhận hiện tượng "dễ ghi nhớ" vẫn đúng ở quy mô nhỏ hơn paper gốc.
2. **Quét mức corruption nhãn** $p \in \{0, 0.2, 0.4, 0.6, 0.8, 1.0\}$ trên cùng mô hình nhỏ: đo test error sau khi train error = 0. Kỳ vọng: test error tăng gần tuyến tính theo p, tiến tới tỉ lệ đoán ngẫu nhiên khi $p \to 1$ (giống Hình 1c); diễn giải: xác nhận mô hình "học phần signal còn lại, ghi nhớ phần noise".
3. **Kiểm chứng Định lý 1 bằng tay**: với $n = 50$ điểm ngẫu nhiên 1 chiều ($d = 1$), dựng mạng ReLU 2 tầng theo công thức $c(x) = \sum w_j \max\{\langle a, x\rangle - b_j, 0\}$ với $2n + d = 101$ tham số, giải hệ $Aw = y$ bằng NumPy cho một vector nhãn y tuỳ ý; đo lỗi khớp (kỳ vọng xấp xỉ 0 vì A khả nghịch theo Bổ đề 1). Đây chạy được trên CPU, không cần GPU.
4. **Ablation regularizer trên mô hình nhỏ**: huấn luyện CNN nhỏ trên subset CIFAR10 với 4 cấu hình (có/không augmentation) x (có/không weight decay); đo test accuracy. Kỳ vọng: chênh lệch vài điểm %, không có cấu hình nào thất bại hoàn toàn (trừ khi học bị chặn như AlexNet+weight decay trên nhãn ngẫu nhiên); diễn giải: xác nhận hoặc bác bỏ "regularizer không phải yếu tố nền tảng" ở quy mô nhỏ.
5. **Tái lập kernel-trick nhỏ (Mục 5)**: với subset MNIST $n = 2.000$, tính kernel Gram $K = XX^T$ (2.000×2.000, đủ nhỏ để chạy trên CPU/RAM thường), giải $K\alpha = y$ (one-hot), đo test error trên 1.000 ảnh test. Kỳ vọng: test error thấp hơn hẳn so với 46% kiểu CIFAR10 vì MNIST dễ hơn; diễn giải: minh hoạ trực tiếp "minimum-norm interpolation vẫn generalize tốt" mà không cần regularizer.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Rademacher complexity	$\hat{R}_n(H) = \mathbb{E}_\sigma\left[\sup_{h\in H} \frac{1}{n}\sum_i \sigma_i h(x_i)\right]$ (công thức 1)
ERM tuyến tính	$\min_w \frac{1}{n}\sum_i \text{loss}(w^T x_i, y_i)$ (công thức 2)
Phương trình kernel (SGD, $w_0 = 0$, khớp hoàn toàn)	$XX^T\alpha = y$ (công thức 3)
Số tham số đủ để biểu diễn mọi hàm trên n mẫu, d chiều, mạng 2 tầng	$p = 2n + d$ (Định lý 1)
Mạng sâu k tầng tương đương	rộng $O(n/k)$, tổng $O(n + d)$ trọng số (Hệ quả 1)
Test error khi nhiễu nhãn $\to 1$	tiến tới tỉ lệ đoán ngẫu nhiên (90% trên CIFAR10 10 lớp)
Tác động tắt hết regularizer (ImageNet, Inception V3)	top-1 giảm từ 77,84% xuống 59,80% (giảm khoảng 18 điểm %)
Tác động Batch Norm lên generalization (CIFAR10)	khoảng 3-4 điểm %
Test error kernel trên MNIST	1,2% (không tiền xử lý), 0,6% (Gabor wavelet)
Test error kernel trên CIFAR10	46% (pixel thô) → 17% (random conv-net 32.000 filter) → 15% (thêm ℓ_2)

11. Kết luận ngắn

Bằng cách huấn luyện các mạng chuẩn (Inception, AlexNet, MLP) trên nhãn/pixel bị xáo trộn hoàn toàn, paper chứng minh thực nghiệm rằng mạng nơ-ron sâu dễ dàng ghi nhớ (memorize) dữ liệu huấn luyện bất kể cấu trúc có ý nghĩa hay không, khiến các thước đo độ phức tạp cổ điển (VC-dimension, Rademacher complexity, uniform stability) không còn khả năng giải thích generalization. Regularization tường minh chỉ cải thiện vài điểm %, không phải điều kiện cần. Một định lý biểu diễn đơn giản ($p = 2n + d$ cho mạng 2 tầng) xác nhận "đủ tham số là đủ để ghi nhớ bất cứ gì". Hướng gợi mở quan trọng nhất cho các bài sau (02, 03) là: nếu ghi nhớ hoàn toàn dữ liệu không ngăn cản generalization, thì cơ chế thật phải nằm ở loại nghiệm mà thuật toán tối ưu (SGD) chọn ra (implicit regularization/inductive bias), không phải ở dung lượng mô hình.

Nguồn gốc: https://arxiv.org/abs/1611.03530

Bài 02

Reconciling modern machine learning practice and the bias-variance trade-off (2018)

Tác giả: Mikhail Belkin (Ohio State University), Daniel Hsu (Columbia University), Siyuan Ma (Ohio State University), Soumik Mandal (Ohio State University).
Venue / arXiv: arXiv:1812.11118 (v2, 10/09/2019); venue: PNAS (paper tự ghi trong lời cảm ơn "the PNAS submission").
Nguồn: https://arxiv.org/abs/1812.11118
Chủ đề: đề xuất đường cong "double descent" (giáng kép) thống nhất U-curve bias-variance cổ điển với hiện tượng các mô hình quá tham số hoá (interpolating) vẫn generalize tốt; minh chứng bằng Random Fourier Features, Random ReLU Features, mạng nơ-ron 2 tầng, random forest và L2-boosting trên nhiều bộ dữ liệu.
Vị trí trong lộ trình: Đọc sau bài 01 về khả năng ghi nhớ và trước bài 03 về double descent trong mạng sâu.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Bias-variance trade-off là một trong những nguyên lý nền tảng của học máy cổ điển (Geman, Bienenstock, Doursat 1992; sách "Elements of Statistical Learning" - ESL, dẫn ở ref [21]): để chọn độ phức tạp của họ hàm H (ví dụ kiến trúc mạng nơ-ron), cần cân bằng:

1. H quá nhỏ: mọi hàm trong H đều underfit (empirical risk lớn), dự đoán kém trên dữ liệu mới.
2. H quá lớn: bộ tối ưu empirical risk (ERM) có thể overfit các mẫu nhiễu trong dữ liệu huấn luyện, dẫn tới empirical risk nhỏ nhưng true risk (rủi ro kiểm tra) lớn.

Việc này tạo ra đường **U-shaped risk curve cổ điển** (Hình 1a): risk test giảm rồi tăng theo độ phức tạp (capacity) của H, điểm thấp nhất là "sweet spot". Hệ quả giáo trình được trích dẫn thẳng từ ESL (trang 221): "a model with zero training error is overfit to the training data and will typically generalize poorly".

Nhưng thực hành hiện đại (mạng nơ-ron lớn, kernel machine) thường được huấn luyện để **khớp hoàn toàn (interpolate)** dữ liệu huấn luyện (train risk = 0), và vẫn cho rủi ro kiểm tra rất thấp, kể cả khi dữ liệu có nhiễu nhãn cao. Đây là mâu thuẫn trực tiếp với trực giác bias-variance cổ điển.

Khái niệm cần nắm:

- **Capacity của** H: trong bài này, được đồng nhất với **số tham số** cần để xác định một hàm trong H.
- **Interpolation threshold**: ngưỡng capacity mà tại đó mô hình vừa đủ khả năng khớp hoàn toàn (train risk = 0) dữ liệu huấn luyện.
- **"Classical" regime** (dưới ngưỡng): vùng mà đường U-shape cổ điển áp dụng. **"Modern" / interpolating regime** (trên ngưỡng): vùng các mô hình khớp hoàn toàn dữ liệu.

2. Ý tưởng chính

Phát hiện chính của bài báo: khi tăng capacity của H vượt qua interpolation threshold, **rủi ro kiểm tra tiếp tục giảm thêm một lần nữa**, thường xuống thấp hơn cả điểm sweet-spot của đường U cổ điển. Hình dạng tổng thể gọi là **"double descent" risk curve** (Hình 1b): U-curve cổ điển ở phần dưới ngưỡng, nối với một đoạn giảm thứ hai ở phần trên ngưỡng, nối nhau tại một đỉnh (peak) đúng tại ngưỡng interpolation.

Cơ chế được đề xuất (posited mechanism): ở bên phải ngưỡng, mọi hàm trong H đều khớp hoàn toàn dữ liệu (train risk = 0), vậy tại sao một số hàm lại generalize tốt hơn hàm khác? Câu trả lời: capacity của H không quyết định việc hàm học được có khớp với **inductive bias** phù hợp hay không. Với các bài toán khảo sát, inductive bias phù hợp là **độ trơn/đều (smoothness)** đo theo một chuẩn không gian hàm nào đó. Chọn hàm trơn nhất trong số các hàm khớp hoàn toàn dữ liệu là một dạng dao cạo Occam. Khi H càng lớn, càng có nhiều hàm ứng viên khớp dữ liệu để chọn, nên có thể tìm được hàm nội suy có chuẩn nhỏ hơn (tức "đơn giản hơn") -> giải thích vì sao tăng capacity vượt ngưỡng lại cải thiện rủi ro kiểm tra.

Paper minh chứng cơ chế này nhất quán trên 3 lớp mô hình khác hẳn nhau về cách "chọn" nghiệm nội suy: Random Fourier/ReLU Features (minimum-norm regression tường minh), mạng nơ-ron (inductive bias từ SGD), và ensemble cây quyết định (inductive bias từ việc lấy trung bình).

3. Phương pháp chi tiết

3.1 Random Fourier Features (RFF) (Mục 2)

Họ hàm $\mathcal{H}_N$ với N tham số phức:

$$h(x) = \sum_{k=1}^{N} a_k \, \phi(x; v_k), \quad \phi(x; v) := e^{\sqrt{-1}\langle v, x\rangle}$$

với $v_1, \ldots, v_N$ lấy độc lập từ phân phối chuẩn tắc trong $\mathbb{R}^d$ (được xem là mạng 2 tầng với tầng đầu cố định trọng số). Khi $N \to \infty$, $\mathcal{H}_N$ tiến gần Reproducing Kernel Hilbert Space (RKHS) của kernel Gauss, gọi là $\mathcal{H}_\infty$. Học bằng ERM với squared loss; khi $N > n$ (nghiệm không duy nhất), chọn nghiệm có chuẩn ℓ_2 nhỏ nhất của hệ số $(a_1, \ldots, a_N)$ (xấp xỉ chuẩn RKHS $\|h\|_{\mathcal{H}_\infty}$, vốn khó tính trực tiếp cho hàm tổng quát trong $\mathcal{H}_N$).

3.2 Random ReLU Features (Phụ lục C.2)

Tương tự RFF nhưng $\phi(x; v) := \max(\langle v, x\rangle, 0)$, v_k lấy từ phân phối đều trên mặt cầu đơn vị; hệ số a_k học bằng hồi quy tuyến tính (có thể thêm ridge regularization nhỏ để ổn định số học gần ngưỡng nội suy, ví dụ $\lambda = 4 \times 10^{-6}$ cho SVHN).

3.3 Mạng nơ-ron fully-connected và backpropagation (Mục 2, Phụ lục C.3)

Mạng 1 tầng ẩn với H đơn vị ẩn; số tham số $= (d + 1) \cdot H + (H + 1) \cdot K$ (d = số chiều input, K = số lớp output). Huấn luyện bằng SGD (momentum 0,95, cài bằng Keras). Để giảm độ nhạy khởi tạo của ERM không lồi ở vùng dưới ngưỡng ($N < n$), paper dùng **"weight reuse"**: huấn luyện mạng nhỏ H_1 đơn vị trước, rồi dùng trọng số đó khởi tạo H_1 đơn vị đầu của mạng lớn hơn $H_2 > H_1$ đơn vị (phần còn lại khởi tạo ngẫu nhiên N(0; 0,01)); mạng nhỏ nhất khởi tạo theo Glorot-uniform. Dưới ngưỡng: learning rate giảm 10% mỗi 500 epoch, dừng khi train error = 0 hoặc sau 6000 epoch. Trên ngưỡng: step size cố định, dừng sau 6000 epoch. Paper cũng chạy lại **không** dùng weight reuse (khởi tạo ngẫu nhiên toàn bộ) để kiểm tra độ nhạy của quan sát với cách khởi tạo (Phụ lục C.3, Hình 9c).

3.4 Cây quyết định và ensemble (Mục 3)

Capacity của một cây quyết định đơn được đo bằng **số lá**: cây 2 lá tương ứng hàm hằng từng khúc biên trục toạ độ; cây n lá có thể nội suy n mẫu huấn luyện. Dưới ngưỡng, đây tái lập U-curve cổ điển đã biết trong thống kê (ESL [21]). Trên ngưỡng, paper dùng **số cây trong ensemble** làm trục capacity thứ hai: lấy trung bình (average, không bootstrap resampling) của nhiều cây nội suy độc lập, giống phương pháp "Perfect Random Tree Ensembles - PERT" (Cutler & Zhao 2001). Ngoài random forest, paper còn thử **L2-boosting** (Friedman 2001): mỗi cây giới hạn tối đa 10 lá, dùng shrinkage thấp (0,85) để nhanh đạt ngưỡng nội suy, sau đó lấy trung bình nhiều "forest" (mỗi forest gồm N_{tree} cây) để đi tiếp qua ngưỡng, với capacity tổng = (N_{tree}, N_{forest}).

3.5 Bộ dữ liệu (Bảng 1, Phụ lục B)

Dataset	Kích thước tập train đầy đủ	Số chiều d	Số lớp
CIFAR-10	5×10^4	1024	10
MNIST	6×10^4	784	10
SVHN	$7{,}3 \times 10^4$	1024	10
TIMIT	$1{,}1 \times 10^6$	440	48
20-Newsgroups	$1{,}6 \times 10^4$	100	20

Ảnh chuyển về grayscale, scale $[0, 1]$; TIMIT chuẩn hoá z-score theo đặc trưng; 20-Newsgroups dùng vector nhúng từ (GloVe, ref [29]) cộng gộp. Hầu hết thí nghiệm lấy subsample $n = 10^4$ để giảm chi phí tính.

4. Thí nghiệm và kết quả chính

RFF trên MNIST (Hình 2, $n = 10^4$, 10 lớp). Interpolation threshold đúng tại $N = 10^4 = n$. Với $N \ll n$: test risk theo đúng dạng U cổ điển, đỉnh tại $N = n$ (gần tỉ lệ đoán ngẫu nhiên, test zero-one loss quan sát tới khoảng 88% ở vùng gần ngưỡng theo đồ thị). Mô hình đạt interpolation với ít tham số nhất ($N = n$) cho độ chính xác **kém nhất** (paper: với phân loại, mô hình này "không có khả năng dự đoán"). Khi N tăng vượt n, độ chính xác cải thiện mạnh, **vượt qua** điểm tốt nhất của đường U phía dưới ngưỡng; $h_{n,\infty}$ (kernel machine đầy đủ, giới hạn $N \to \infty$) vượt trội mọi $h_{n,N}$ hữu hạn.

Random ReLU 1-D minh hoạ cơ chế (Hình 3). Khớp 10 điểm dữ liệu bằng Random ReLU Features: với $N = 40$, chuẩn hệ số (đã nhân $\sqrt{N}$) ≈ 695; với $N = 4000$ (100 lần nhiều hơn), chuẩn ≈ 159 (tự tính: giảm khoảng $695/159 \approx 4{,}4$ lần dù số feature tăng 100 lần). Cả hai đều nội suy đúng 10 điểm, nhưng hàm $N = 4000$ "trông" trơn hơn hàm $N = 40$ dù về mặt từng đoạn cả hai đều là hàm tuyến tính-từng-khúc (piecewise linear) không trơn tuyệt đối.

Mạng nơ-ron fully-connected trên MNIST (Hình 4, $n = 4 \times 10^3$, $d = 784$, $K = 10$). Số tham số $= (d + 1)H + (H + 1)K$; interpolation threshold $= n \cdot K$ (tự tính: $4.000 \times 10 = 40.000$ tham số, khớp với mốc đường chấm đứng trong Hình 4 ở vùng ≈ 40 (x10^3) tham số trên trục hoành). Cả zero-one loss và squared loss đều thể hiện double descent: risk tăng khi tiến tới 40.000 tham số, sau đó giảm liên tục khi số tham số tiếp tục tăng tới 800×10^3.

Random forest trên MNIST (Hình 5, $n = 10^4$). Trục hoành là tỉ lệ N_{leaf}^{max}/N_{tree} với các mốc $10/1,\ 1000/1,\ 2000/1,\ 2000/10,\ 2000/20$: double descent thể hiện rõ cả ở squared loss và zero-one loss (U-shape ít rõ hơn ở zero-one loss).

L2-boosting (Hình 12, Phụ lục E). Cây giới hạn ≤ 10 lá, shrinkage 0,85; capacity $= (N_{tree}, N_{forest})$: quan sát double descent ở cả squared loss và zero-one loss, với overfitting cổ điển rõ trước ngưỡng (squared loss). Với shrinkage cao hơn (0,1), đường cong "ít rõ hơn do hiệu ứng regularization của shrinkage cao" (paper tự nhận, không có số liệu cụ thể kèm theo).

Vì đây là paper lý thuyết + thực nghiệm minh chứng cơ chế (không phải benchmark SOTA), không có bảng so sánh với baseline bên ngoài; "kết quả chính" là việc đường cong double descent **tái lập nhất quán** trên 5 bộ dữ liệu (MNIST, CIFAR-10, SVHN, TIMIT, 20-Newsgroups) và 4 họ mô hình (RFF, Random ReLU, mạng nơ-ron, cây/ensemble).

5. Phân tích / ablation

- **Cơ chế toán học cho RFF (Mục 2, lập luận định tính):** khi $N \ll n$, train risk gần test risk (lý thuyết thống kê cổ điển), nên tăng N cải thiện cả hai. Khi N tiến gần n (ngưỡng nội suy), các đặc trưng ít/không có tín hiệu thật buộc phải khớp dữ liệu gần hoàn hảo, gây overfit cổ điển, đỉnh tại chỗ khớp

chính xác tuyệt đối. Qua ngưỡng, mọi $\mathcal{H}_N$ đủ giàu để đạt train risk 0; nhưng không chắc chứa nghiệm chuẩn nhỏ nhất $h_{n,\infty} \in \mathcal{H}_\infty$ với N hữu hạn -- tăng N giúp xấp xỉ tốt hơn nghiệm đó, nên chuẩn của $h_{n,N}$ giảm dần đơn điệu khi N tăng, kéo theo test risk giảm (đoạn giáng thứ hai).

- **Định lý xấp xỉ (Phụ lục A).** Với dữ liệu không nhiễu, $y_i = h^*(x_i)$, $h^* \in \mathcal{H}_\infty$, với mọi hàm nội suy $h \in \mathcal{H}_\infty$: $\sup_{x\in\Omega} |h(x) - h^*(x)| < Ae^{-B(n/\log n)^{1/d}}(\|h^*\|_{\mathcal{H}_\infty} + \|h\|_{\mathcal{H}_\infty})$. Nghiệm chuẩn nhỏ nhất $h_{n,\infty}$ có chuẩn $\leq \|h^*\|$, nên đạt chặn nhỏ nhất trong định lý này -- biện minh toán học (chỉ cho trường hợp không nhiễu) cho việc ưu tiên nghiệm chuẩn nhỏ. Chặn này suy giảm theo mũ $n^{1/d}$ (curse of dimensionality hiện rõ qua d trong số mũ).
- **Inductive bias khác nhau nhưng cùng hội tụ.** RFF/ReLU random features: minimum-norm regression tường minh trong không gian đặc trưng. Mạng nơ-ron tổng quát: inductive bias từ bản thân quy trình SGD; khi chỉ huấn luyện tầng cuối (như RFF), SGD khởi tạo 0 cũng hội tụ về nghiệm chuẩn nhỏ nhất; với mạng nhiều tầng tổng quát, paper chỉ nói có "bằng chứng thực nghiệm đáng kể và một số bằng chứng lý thuyết" ủng hộ một inductive bias tương tự, **không chứng minh chặt**. Random forest: lấy trung bình nhiều cây nội suy không trơn cho ra nghiệm trơn hơn, tốt hơn từng cây riêng lẻ. Với kernel machine, cả ba cách (minimum norm tường minh, SGD, trung bình quỹ đạo Gaussian process - tính posterior mean) đều dẫn về **đúng một nghiệm** $h_{n,\infty}$.
- **Ablation khởi tạo cho mạng nơ-ron (Phụ lục C.3, Hình 9).** Không dùng "weight reuse" (khởi tạo ngẫu nhiên toàn bộ, Hình 9c): phương sai tăng mạnh, train loss không giảm đơn điệu, nhưng **double descent vẫn quan sát rõ**, cho thấy hiện tượng không phải "nhân tạo" do kỹ thuật khởi tạo đặc biệt.
- **Lợi ích thực hành của over-parameterization cho optimization (Mục "Optimization and practical considerations").** Paper dẫn (Soltanolkotabi et al. 2018) rằng các phương pháp cục bộ như SGD hội tụ về global minimum của training risk trong chế độ quá tham số hoá; nghĩa là mô hình lớn vừa dễ tối ưu vừa có test risk thấp đồng thời -- một lợi ích kép không có ở chế độ cổ điển.

6. Giới hạn

Do tác giả nêu:

- Định lý xấp xỉ (Phụ lục A) chỉ áp dụng cho dữ liệu **không nhiễu**; với nhiễu, chỉ có "bằng chứng thực nghiệm" (dẫn bài khác [4]), không có định lý tương ứng trong bài này.
- Với mạng nơ-ron tổng quát, "hành vi của SGD... không được hiểu đầy đủ" ("not fully understood"); inductive bias chuẩn nhỏ nhất chỉ là giả thuyết được hỗ trợ bởi bằng chứng, chưa chứng minh.
- Độ phức tạp tính toán của ERM với mạng nơ-ron khiến đường cong double descent "khó quan sát" trong thực tế: ở chế độ dưới ngưỡng, bài toán không lồi làm SGD rất nhạy khởi tạo; đỉnh tại ngưỡng nội suy chỉ xảy ra trong một **dải tham số hẹp**, nếu lấy mẫu kiến trúc thưa có thể bỏ lỡ hoàn toàn và tạo ấn tượng sai rằng "tăng kích thước luôn luôn chỉ cải thiện performance". Early stopping trong thực hành cũng có hiệu ứng regularization mạnh, "khiến khó quan sát đỉnh nội suy".
- Paper tự tính: dữ liệu lớn như ImageNet ($\sim 10^6$ mẫu, $\sim 10^3$ lớp) "có thể cần mạng $\sim 10^9$ tham số để đạt nội suy", lớn hơn nhiều mô hình ImageNet thực tế dùng trong thực hành -- với các mô hình đó, **chế độ U-curve cổ điển vẫn phù hợp hơn** để hiểu generalization, double descent không phải "luôn luôn đúng ở mọi kích thước thực hành".

Nhận xét:

- Hầu hết thí nghiệm dùng **mạng 1 tầng ẩn** (không sâu), không có batch normalization, không có data augmentation hay optimizer hiện đại (Adam, ...); do đó phần lớn gánh nặng "double descent có đúng với mạng sâu hiện đại không" được chuyển sang bài 03 (Nakkiran et al., dùng ResNet/CNN sâu thật).
- Dữ liệu train luôn được subsample xuống khá nhỏ ($n \leq 10^4$ cho hầu hết thí nghiệm hình ảnh) để giảm chi phí; chưa rõ đường cong có giữ dạng tương tự ở n lớn hơn nhiều bậc độ lớn (bài 03 xét vấn đề này một phần qua trục "epoch-wise").
- Tuyên bố "cả ba loại mô hình đều hội tụ về nghiệm chuẩn nhỏ nhất" chỉ **chứng minh được cho kernel**

machine; với mạng nơ-ron và random forest đây là suy luận bằng tương tự + thực nghiệm, không phải định lý.

7. Liên hệ với các bài khác

- **ml basics 01** (Zhang et al., Understanding deep learning requires rethinking generalization): là nguồn trực tiếp của câu hỏi nghiên cứu ("mô hình khớp hoàn toàn dữ liệu vẫn generalize tốt"); bài 02 trích dẫn thẳng bài 01 ở ref [42] và dùng lại tinh thần "regularization không phải điều kiện cần".
- **ml basics 03** (Nakkiran et al., Deep Double Descent): kế thừa trực tiếp khung "double descent" của bài 02, mở rộng sang mạng sâu hiện đại thật (ResNet, CNN), thêm trục số epoch huấn luyện (epoch-wise double descent) và trục kích thước dữ liệu, với label noise kiểu bài 01.
- **pruning content/08_2018_lottery_ticket_hypothesis.md**: cùng chủ đề overparameterization; nếu một mạng lớn (ở vùng "modern/interpolating regime" theo bài 02) chứa một "winning ticket" nhỏ hơn nhiều vẫn đạt hiệu năng tương đương, điều này gợi ý double descent không chỉ về *số tham số thô* mà còn về *cấu trúc nghiệm* mà thuật toán tìm ra.
- **pruning content/12_2020_state_of_neural_network_pruning.md**: tổng kết các quan điểm về quá tham số hoá/pruning, có thể dùng để đối chiếu với khung "interpolation threshold" của bài 02 khi đánh giá một mô hình đã bị cắt tỉa còn ở chế độ nào (dưới/trên ngưỡng).
- **llm content/03_2020_scaling_laws_kaplan.md**: scaling laws cho LLM mô tả loss giảm trơn theo quy mô ở miền rất lớn tham số/dữ liệu; đáng chú ý đây là vùng "modern regime" rất xa về bên phải so với ngưỡng nội suy mà bài 02 khảo sát, cho thấy double descent (ở gần ngưỡng) và scaling laws (ở rất xa ngưỡng) mô tả hai hiện tượng khác pha của cùng một "siêu đường cong" risk-vs-capacity.

8. Gợi ý áp dụng

Đường cong rủi ro theo độ phức tạp có thể không đơn điệu, nhất là gần ngưỡng nội suy. Khi so sánh mô hình, nên khảo sát nhiều mức độ phức tạp và tách kết luận về phép lấy trung bình dự đoán khỏi giả định về lấy trung bình trọng số.

9. Thí nghiệm gợi ý

1. **Tái lập double descent với MLP 1 tầng ẩn trên MNIST subset** ($n = 4.000$, $d = 784$, $K = 10$, giống Hình 4): quét H từ rất nhỏ tới rất lớn quanh ngưỡng H sao cho tổng tham số $\approx n \cdot K = 40.000$; đo train/test zero-one loss và squared loss. Kỳ vọng: đỉnh risk quanh 40.000 tham số, giảm tiếp khi tăng H xa hơn; diễn giải: xác nhận vị trí ngưỡng nội suy đúng bằng công thức $n \cdot K$.
2. **RFF toy 1-D hoặc MNIST nhỏ:** cài công thức RFF ở Mục 3.1, quét N quanh n (ví dụ $n = 1.000$), đo test error và chuẩn hệ số ℓ_2. Kỳ vọng: cả hai đạt đỉnh tại $N = n$ rồi giảm; diễn giải: quan sát trực tiếp cơ chế "chuẩn giảm dần sau ngưỡng" mà paper mô tả.
3. **Ablation weight-reuse vs random-init quanh ngưỡng:** chạy 5 seed cho mạng nhỏ ở đúng vùng ngưỡng nội suy, so sánh phương sai test error giữa chiến lược khởi tạo "weight reuse" (Mục 3.3) và khởi tạo ngẫu nhiên hoàn toàn. Kỳ vọng: random-init có phương sai cao hơn hẳn gần ngưỡng; diễn giải: minh hoạ vì sao double descent "dễ bị nhiễu optimization che mất" như paper cảnh báo ở Mục 6.
4. **Random forest double descent trên tập nhỏ (ví dụ sklearn digits hoặc 2.000 ảnh MNIST):** quét N_{leaf}^{max} và N_{tree} như Hình 5, không bootstrap. Kỳ vọng: U-curve rồi giảm lại khi tăng N_{tree} với cây full-depth; diễn giải: double descent không chỉ của riêng mạng nơ-ron.
5. **Mô phỏng "vùng nguy hiểm" cho một bài toán phân loại ký tự nhỏ (proxy OCR):** dùng classifier nhỏ (ví dụ MLP hoặc CNN nhỏ) phân loại ký tự cắt từ ảnh (character crop), quét số tham số quanh số mẫu train có sẵn; đo test accuracy. Kỳ vọng: nếu gặp đúng vùng ngưỡng, test accuracy có thể tụt tạm thời; diễn giải: bài học thực hành cho việc chọn kích thước mô hình OCR sau pruning khớp với lượng dữ liệu phục hồi hiện có.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
RFF	$h(x) = \sum_{k=1}^{N} a_k e^{\sqrt{-1}\langle v_k, x\rangle}$, $v_k \sim \mathcal{N}(0, I)$
Random ReLU feature	$\phi(x; v) = \max(\langle v, x\rangle, 0)$, v đều trên mặt cầu đơn vị
Số tham số MLP 1 tầng ẩn	$(d+1)H + (H+1)K$
Interpolation threshold (phân loại K lớp, n mẫu)	$N = n$ (RFF phức); $n \cdot K$ tham số (MLP)
Chặn xấp xỉ RKHS (Định lý 1, không nhiễu)	$\sup_x \lvert h(x) - h^*(x)\rvert < Ae^{-B(n/\log n)^{1/d}}(\lVert h^*\rVert + \lVert h\rVert)$
Ví dụ tự tính: ngưỡng Hình 4	$n \cdot K = 4.000 \times 10 = 40.000$ tham số
Ví dụ tự tính: chuẩn ReLU feature $N = 40$ vs $N = 4000$	$695 \to 159$, giảm $\approx 4{,}4\times$
Capacity cây quyết định	số lá; n lá nội suy n mẫu
Capacity ensemble sau ngưỡng	số cây nội suy trung bình N_{tree} (random forest) hoặc (N_{tree}, N_{forest}) (L2-boosting)

11. Kết luận ngắn

Bài báo thống nhất U-curve bias-variance cổ điển với hiện tượng mô hình quá tham số hoá hiện đại bằng đường cong **double descent**: ngay tại ngưỡng nội suy risk đạt đỉnh (giống overfit cổ điển), nhưng vượt ngưỡng risk lại giảm tiếp vì các họ hàm lớn hơn chứa nhiều nghiệm nội suy hơn, cho phép thuật toán học (minimum-norm regression, SGD, hay trung bình ensemble) chọn ra nghiệm "trơn"/chuẩn nhỏ hơn. Cơ chế này được minh chứng nhất quán trên RFF, Random ReLU feature, mạng nơ-ron 1 tầng ẩn, random forest và L2-boosting, trên 5 bộ dữ liệu khác nhau, nhưng chỉ chứng minh chặt (Định lý xấp xỉ) cho trường hợp kernel machine không nhiễu. Bài 03 sẽ kiểm định ý tưởng này với mạng sâu hiện đại và thêm trục epoch/kích thước dữ liệu.

Nguồn gốc: https://arxiv.org/abs/1812.11118

Bài 03

Deep Double Descent: Where Bigger Models and More Data Hurt (2019)

Tác giả: Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak (Harvard University), Ilya Sutskever (OpenAI).
Venue / arXiv: arXiv:1912.02292 (v1, 04/12/2019); paper không nêu venue hội nghị/tạp chí nào trong văn bản.
Nguồn: https://arxiv.org/abs/1912.02292
Chủ đề: hiện tượng "double descent": khi tăng kích thước mô hình (hoặc số epoch, hoặc số mẫu huấn luyện), lỗi kiểm tra (test error) không đi theo đường cong chữ U cổ điển mà giảm rồi tăng rồi giảm lại; đề xuất khái niệm "effective model complexity" (EMC) để gộp ba dạng double descent (theo mô hình, theo epoch, theo số mẫu) vào một giả thuyết chung.
Vị trí trong lộ trình: Đọc sau bài 01 về khả năng ghi nhớ và bài 02 về double descent; bài này khảo sát thêm ảnh hưởng của số epoch và số mẫu.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Lý thuyết thống kê cổ điển (bias-variance trade-off, xem Hastie et al. dẫn trong bài báo) nói rằng: mô hình "phức tạp" hơn (nhiều tham số, nhiều bậc tự do) có **bias** thấp hơn nhưng **variance** cao hơn. Khi độ phức tạp vượt một ngưỡng, variance sẽ áp đảo, nên lỗi kiểm tra (lỗi kiểm tra) đi theo đường **chữ U**: giảm rồi tăng khi mô hình quá lớn so với dữ liệu. Đây là lý do sách giáo khoa cổ điển khuyên "đừng dùng mô hình quá lớn".

Nhưng mạng nơ-ron hiện đại có hàng triệu tham số, nhiều hơn cả số điểm dữ liệu, và vẫn hoạt động rất tốt. Paper dẫn lại kết quả của Zhang et al. (2016) (ml basics 01): mạng đủ lớn có thể khớp (fit) gần hoàn hảo cả nhãn ngẫu nhiên, nghĩa là dung lượng ghi nhớ của nó vượt xa số mẫu huấn luyện. Thực hành của ngành lại cho trực giác ngược: "mô hình lớn hơn luôn tốt hơn" (paper dẫn chứng AlexNet của Krizhevsky et al. 2012, GPipe của Huang et al. 2018, GoogLeNet "Going deeper" của Szegedy et al. 2015, GPT-2 của Radford et al. 2019).

Hai trực giác này ("lớn hơn thì tệ hơn" theo lý thuyết cổ điển, và "lớn hơn thì tốt hơn" theo thực hành) tưởng như mâu thuẫn. Belkin et al. (2018) (ml basics 02) là người đầu tiên đề xuất một cách giải thích tổng quát: có **hai pha**. Khi mô hình còn "thiếu tham số" (under-parameterized, độ phức tạp nhỏ hơn số mẫu), lỗi kiểm tra theo đúng chữ U cổ điển. Khi mô hình đủ lớn để **nội suy** (interpolate, tức đạt lỗi huấn luyện ≈ 0), tăng thêm độ phức tạp lại làm giảm lỗi kiểm tra, đúng với trực giác "mô hình lớn hơn tốt hơn". Hai pha này nối với nhau tại một điểm gọi là **ngưỡng nội suy** (interpolation threshold), nơi lỗi kiểm tra đạt đỉnh. Họ đặt tên hiện tượng này là "double descent" và minh chứng trên decision tree, random features, mạng 2 lớp với hàm mất ℓ_2, trên MNIST và CIFAR-10.

Bài báo này (Nakkiran et al. 2019) đặt câu hỏi: hiện tượng này có xảy ra trên các mạng sâu hiện đại (ResNet, CNN, Transformer), với các thiết lập huấn luyện thực tế (SGD, Adam, data augmentation, label noise) không? Và nếu "độ phức tạp" không chỉ là số tham số mà còn phụ thuộc cả quy trình huấn luyện (số epoch, lượng dữ liệu), thì double descent có xảy ra theo các trục đó không?

2. Ý tưởng chính

Bài báo đề xuất một đại lượng mới gọi là **Effective Model Complexity (EMC)**: độ phức tạp hiệu dụng của một *quy trình huấn luyện* T (không chỉ của kiến trúc), định nghĩa là số mẫu lớn nhất mà T có thể khớp gần như hoàn hảo (lỗi huấn luyện ≈ 0). Vì EMC phụ thuộc vào quy trình huấn luyện, tăng số epoch huấn luyện cũng làm tăng EMC, không chỉ tăng số tham số mới làm tăng EMC.

Giả thuyết Double Descent tổng quát (Hypothesis 1, phi hình thức): với một phân phối dữ liệu D, quy trình huấn luyện T dựa trên mạng nơ-ron, và $\varepsilon > 0$ nhỏ, xét bài toán dự đoán nhãn từ n mẫu của D:

- **Vùng thiếu tham số (under-parameterized):** nếu $\text{EMC}(T)$ nhỏ hơn đáng kể so với n, thì mọi thay đổi làm tăng EMC của T sẽ **giảm** lỗi kiểm tra.
- **Vùng dư tham số (over-parameterized):** nếu $\text{EMC}(T)$ lớn hơn đáng kể so với n, thì mọi thay đổi làm tăng EMC của T cũng sẽ **giảm** lỗi kiểm tra.
- **Vùng tham số tới hạn (critically parameterized):** nếu $\text{EMC}(T) \approx n$, thì một thay đổi làm tăng EMC có thể làm **tăng hoặc giảm** lỗi kiểm tra (không chắc chiều nào).

Từ giả thuyết này, paper suy ra và kiểm chứng thực nghiệm ba dạng double descent:

1. **Model-wise double descent:** cố định số bước huấn luyện lớn, tăng dần kích thước mô hình (số tham số); lỗi kiểm tra giảm, tăng (quanh ngưỡng nội suy), rồi giảm lại.
2. **Epoch-wise double descent:** cố định một mô hình đủ lớn, tăng dần số epoch huấn luyện; vì tăng epoch cũng tăng EMC, lỗi kiểm tra cũng giảm, tăng, rồi giảm lại theo thời gian huấn luyện. Hệ quả: dừng sớm (early stopping) không phải lúc nào cũng là chiến lược tốt nhất.
3. **Sample-wise non-monotonicity:** cố định mô hình và quy trình huấn luyện, thay đổi số mẫu huấn luyện n; vì tăng n cũng dịch điểm tới hạn (nơi EMC $\approx n$) sang kích thước mô hình lớn hơn, có những vùng kích thước mô hình mà thêm dữ liệu không giúp gì, và trong một số trường hợp thêm dữ liệu còn làm **tệ hơn**.

Tham số ε được bài báo chọn theo kinh nghiệm là $\varepsilon = 0.1$ (không có cách chọn "có nguyên tắc"), và paper nói rõ các khái niệm "nhỏ hơn đáng kể" / "lớn hơn đáng kể" chỉ là phi hình thức.

3. Phương pháp chi tiết

3.1 Định nghĩa EMC (chính xác, Định nghĩa 1)

$$\text{EMC}_{D,\varepsilon}(T) := \max\{\, n \mid \mathbb{E}_{S\sim D^n}[\text{Error}_S(T(S))] \le \varepsilon \,\}$$

trong đó T là một quy trình huấn luyện (nhận n mẫu gán nhãn S, trả về một bộ phân loại $T(S)$), và $\text{Error}_S(M)$ là lỗi trung bình của M trên chính tập train S (train error, không phải test error).

Ví dụ minh hoạ nhỏ (tự tính, không lấy từ paper, chỉ để hiểu định nghĩa): giả sử "mô hình" là một bảng tra cứu (lookup table) có đúng 5 ô nhớ: nó có thể ghi nhớ hoàn hảo bất kỳ 5 mẫu bất kỳ nhưng không suy luận gì thêm. Nếu huấn luyện trên $n = 5$ mẫu, mỗi mẫu được gán một ô nhớ, train error $= 0 \le \varepsilon = 0.1$. Nếu $n = 6$, bảng chỉ có 5 ô nên ít nhất 1 mẫu trong 6 bị bỏ sót, train error trung bình $\ge 1/6 \approx 0.167 > 0.1$ (tự tính: $1/6 = 0.1667$). Vậy $\text{EMC}_{\varepsilon=0.1}(T) = 5$ cho mô hình bảng tra cứu 5 ô này. Đây chính là cách đọc trực quan của Định nghĩa 1: EMC là "sức chứa" tối đa mà quy trình huấn luyện còn ghi nhớ gần hoàn hảo.

3.2 Ba họ kiến trúc dùng trong thí nghiệm (Mục 4, Phụ lục B.1)

- **ResNet18:** biến thể Preactivation ResNet18 (He et al. 2016), 4 block, mỗi block 2 lớp BatchNorm-ReLU-Convolution. Độ rộng 4 block là $[k, 2k, 4k, 8k]$, stride $[1, 2, 2, 2]$. ResNet18 chuẩn ứng với $k = 64$.

- **CNN 5 lớp (Standard CNN):** 4 lớp Conv-BatchNorm-ReLU-MaxPool với độ rộng $[k, 2k, 4k, 8k]$ (kernel 3, stride 1, padding 1; MaxPool $[1, 2, 2, 8]$) cộng 1 lớp fully-connected. Với $k = 64$: 1.558.026 tham số, đạt > 90% độ chính xác test trên CIFAR-10 khi có data augmentation.
- **Transformer:** kiến trúc encoder-decoder 6 lớp, 8 head mỗi lớp (Vaswani et al. 2017), triển khai bằng fairseq (Ott et al. 2019). Độ rộng được chỉnh qua chiều embedding d_{model}, với $d_{ff} = 4d_{model}$.

Nhiễu nhãn (label noise): với xác suất p, một mẫu train giữ nhãn đúng với xác suất $1 - p$ và nhận một nhãn sai ngẫu nhiên đều với xác suất p; nhiễu được lấy mẫu **một lần** (không đổi theo epoch).

3.3 Thiết lập huấn luyện (Mục 4, Phụ lục B.2-B.3)

- ResNet/CNN: hàm mất cross-entropy; hai cách tối ưu: (1) Adam, learning rate 10^{-4}, 4K epoch; (2) SGD với lịch learning rate nghịch-căn $\gamma(t) = \gamma_0/\sqrt{1 + \lfloor t/512 \rfloor}$, $\gamma_0 = 0.1$, 500K bước gradient, không momentum. Batch size 128 cho mọi thí nghiệm ảnh.
- Transformer: huấn luyện 80K bước gradient (mục 4 của paper), label smoothing 10%, không dropout. (Phụ lục B.1 lại ghi "huấn luyện 80 bước gradient" khi mô tả kiến trúc Transformer, có khả năng là lỗi đánh máy thiếu chữ "K" so với mục 4; chúng ta coi 80K là số đúng vì khớp với mô tả chi tiết ở phần thí nghiệm, nhưng đây là một mâu thuẫn nhỏ trong chính văn bản paper, xem thêm mục 6.)
- Không áp dụng regularization nào (không weight decay, không dropout) trừ khi nói rõ; khởi tạo theo mặc định của PyTorch.

3.4 Model-wise double descent (Mục 5)

Cố định số bước tối ưu lớn (huấn luyện "tới khi xong"), tăng dần k (hoặc d_{model}), đo test error. Quan sát chính: mọi thay đổi làm tăng ngưỡng nội suy (thêm label noise, thêm data augmentation, thêm số mẫu train) đều **dịch đỉnh lỗi test sang kích thước mô hình lớn hơn**.

3.5 Epoch-wise double descent (Mục 6)

Cố định một kiến trúc đủ lớn, theo dõi test error theo số epoch. Vì tăng thời gian huấn luyện làm tăng EMC, mô hình đủ lớn sẽ trải qua: EMC nhỏ hơn n ở đầu huấn luyện (test error giảm), EMC $\approx n$ ở giữa (test error tăng, giống overfitting cổ điển), rồi EMC vượt xa n khi huấn luyện đủ lâu (test error giảm lại). Mô hình "cỡ trung" (chỉ vừa đủ đạt train error ≈ 0 khi huấn luyện xong) sẽ chỉ thấy nửa đầu của đường cong (U cổ điển, early stopping có lợi). Mô hình quá nhỏ (không bao giờ đạt train error ≈ 0) chỉ thấy test error giảm đơn điệu.

3.6 Sample-wise non-monotonicity (Mục 7)

Cố định mô hình và quy trình huấn luyện, thay đổi số mẫu train n. Tăng n có hai hiệu ứng ngược nhau: (a) nói chung làm giảm lỗi (trực giác "nhiều dữ liệu hơn thì tốt hơn"); (b) dịch đỉnh double-descent sang kích thước mô hình lớn hơn (vì ngưỡng nội suy EMC $\approx n$ dịch theo n). Ở một dải kích thước mô hình, hai hiệu ứng này **triệt tiêu nhau**: thêm dữ liệu không giúp gì, thậm chí thêm dữ liệu còn làm **tệ hơn** nếu hiệu ứng (b) thắng hiệu ứng (a).

Tự tính: Hình 3 của paper so sánh $n = 4000$ và $n = 18000$ mẫu IWSLT'14 de-en cho Transformer; paper nói đây là "tăng số mẫu train lên hệ số 4.5". Kiểm lại: $18000/4000 = 4.5$ (tự tính, khớp đúng số paper nêu).

4. Thí nghiệm và kết quả chính

Phạm vi thí nghiệm: hơn một bậc độ lớn về kích thước mô hình trên nhiều kiến trúc, nhiều tập dữ liệu (CIFAR-10, CIFAR-100, IWSLT'14 de-en, WMT'14 en-fr), nhiều optimizer (SGD, SGD+weight decay, Adam), có/không data augmentation, nhiều mức label noise. Phụ lục A tổng hợp trong "Bảng A"; dưới đây là một số dòng tiêu biểu (trích, không phải toàn bộ bảng):

Dataset	Kiến trúc	Optimizer	Augment	% Noise	Model-wise DD	Epoch-wise DD
CIFAR-10	CNN	SGD	có	0	không	không
CIFAR-10	CNN	SGD	có	10	có	có
CIFAR-10	ResNet	Adam	có	15	có	(không đo riêng)
CIFAR-10	ResNet	Adam	có	20	có	có
CIFAR-100	ResNet	Adam	có	0	có	không
CIFAR-100	ResNet	Adam	có	10	có	có
IWSLT'14 de-en	Transformer	Adam	-	0	có	không
WMT'14 en-fr	Transformer	Adam	-	0	có	không

Các kết quả chính (số liệu/mô tả lấy trực tiếp từ paper):

- **Hình 1:** ResNet18 độ rộng thay đổi, CIFAR-10, 15% label noise, Adam 4K epoch: test error giảm, tăng (quanh k nhỏ), rồi giảm lại khi k tiếp tục tăng; ResNet18 chuẩn ($k = 64$) nằm ở vùng dư tham số.
- **Hình 4:** trên CIFAR-100 vẫn có đỉnh lỗi test rõ **ngay cả khi không có label noise**; trên CIFAR-10 chỉ có "cao nguyên" (plateau) khi không nhiễu, và cao nguyên này biến thành đỉnh rõ khi thêm nhiễu.
- **Double descent cho robust error (huấn luyện đối kháng, Phụ lục E.2.4, Hình 26):** ResNet18 trên CIFAR-10 (subsample 25.000 mẫu, không label noise), PGD 10 bước, bán kính ℓ_2 $\epsilon = 0.5$ và 1.0, SGD batch 128, learning rate 0.1 trong 400 epoch rồi 0.01 trong 400 epoch tiếp theo: vẫn thấy đỉnh double descent rõ rệt trên **robust test error**, dù không có label noise.
- **Epoch-wise (Hình 9, 10):** ResNet18 trên CIFAR-10 với 20% label noise cho thấy 3 chế độ: mô hình quá nhỏ (test error giảm đơn điệu theo epoch), mô hình "cỡ trung" (chữ U, early stopping tốt), mô hình lớn (giảm, tăng, giảm lại qua toàn bộ quá trình huấn luyện). Hiện tượng này bền với nhiều optimizer (Adam, SGD, SGD+Momentum=0.9) và nhiều lịch learning rate (hằng số, nghịch-căn, "dynamic drop").
- **Sample-wise (Hình 11, 12):** CNN 5 lớp trên CIFAR-10+20% noise: có dải kích thước mô hình (tô xanh) mà tăng gấp 2 lần số mẫu không cải thiện test error, và dải khác (tô đỏ) mà tăng gấp 4 lần số mẫu cũng không cải thiện. Với Transformer trên IWSLT'14 (Hình 3, 11b): tăng số mẫu từ 4000 lên 18000 (4.5 lần, tự tính như mục 3.6) làm test loss **tệ hơn** ở một số kích thước mô hình.
- **Phụ lục D (Random Features):** double descent không phải hiện tượng riêng của mạng sâu. Với Random Fourier Features (tương đương mạng 2 lớp, lớp 1 ngẫu nhiên cố định, lớp 2 huấn luyện bằng MSE) trên Fashion-MNIST, với số đặc trưng ngẫu nhiên (= kích thước mô hình) $d = 1000$ (Hình 15), EMC của mô hình này đúng bằng d, nên đỉnh lỗi rơi chính xác trên đường $n = d$ trong lưới (test error) theo cả n và d (Hình 14).

Paper không có một "bộ benchmark" cố định để so sánh với baseline khác; đây là một tập thí nghiệm mô tả hiện tượng (phenomenon paper), không phải một phương pháp đề xuất để đạt SOTA.

5. Phân tích / ablation

- **Trực giác về vùng tới hạn (Mục 5, Discussion):** ở đúng ngưỡng nội suy, paper cho rằng gần như chỉ có **một** mô hình khớp được train set, và mô hình này rất nhạy với nhiễu/model-misspecification: buộc nó khớp cả phần nhãn nhiễu sẽ phá vỡ cấu trúc toàn cục, gây test error cao. Khi dư tham số nhiều, có **rất nhiều** mô hình khớp train set, và SGD có xu hướng tìm một mô hình "hấp thụ" nhiễu mà vẫn tổng quát hoá tốt trên phần còn lại.
- **Ensembling (Phụ lục E.3, Hình 28, 29):** ensemble 5 mô hình (bầu đa số) giúp cải thiện **nhiều nhất** đúng ở vùng tới hạn, phù hợp với trực giác "mô hình ở vùng tới hạn không ổn định/nhạy nhiễu".

- **Nhiễu nhãn không phải cốt lõi (Mục 8):** paper cho rằng double descent không phải *về* label noise, mà về **model mis-specification** nói chung; label noise chỉ là một cách tiện lợi để tăng độ khó/mis-specification của phân phối. Lập luận: nếu nhiễu là "giả ngẫu nhiên" (pseudorandom, tất định và khả nghịch) thì hiệu suất của bộ phân loại Bayes tối ưu không đổi, nhưng vẫn quan sát double descent giống như nhiễu thật.
- **EMC khác Rademacher complexity / VC dimension (Mục 8):** hai đại lượng cổ điển này chỉ phụ thuộc kiến trúc + phân phối đầu vào, không phụ thuộc **nhãn** hay **quy trình huấn luyện**, nên không thể giải thích vì sao thêm label noise dịch đỉnh double descent, và không thể nắm bắt hiệu ứng theo thời gian huấn luyện (epoch-wise double descent) hay theo data augmentation.
- **Weight decay như một trục độ phức tạp khác (Phụ lục E.2.2, Hình 22):** cố định mô hình (ResNet18, CIFAR-10, 20% noise, SGD + lịch nghịch-căn), thay đổi hệ số weight decay λ từ 0 đến 0.1: quan sát đường cong double-descent tương tự, với "độ phức tạp mô hình" lúc này được điều khiển bởi λ (giảm λ tương ứng tăng EMC). Đây là kết quả "sơ bộ" (preliminary) theo paper.
- **Early stopping thường loại bỏ double descent (Mục 8, Phụ lục E.2.3):** nếu dừng sớm tối ưu, mô hình không kịp đạt train error ≈ 0 nên EMC chưa chạm n, do đó hầu hết thí nghiệm không thấy double descent khi early-stopping tối ưu. Nhưng có **một** trường hợp vẫn thấy double descent dù early-stopping tối ưu: ResNet18 trên CIFAR-100 không label noise (Hình 19). Với các thí nghiệm ngôn ngữ (Hình 23, 24), early-stopping tối ưu luôn cho "nhiều mẫu hơn thì tốt hơn", không có trường hợp dữ liệu nhiều hơn làm hại khi early-stopping tối ưu được dùng.

6. Giới hạn

Do tác giả nêu:

- Hiểu đầy đủ *cơ chế* gây ra model-wise double descent trong mạng sâu "vẫn là câu hỏi mở quan trọng" (paper chỉ có trực giác và phân tích chặt cho trường hợp tuyến tính, không có lý thuyết đầy đủ cho mạng sâu).
- Không có cách chọn ε "có nguyên tắc" (dùng heuristic $\varepsilon = 0.1$); các khái niệm "nhỏ hơn đáng kể"/"lớn hơn đáng kể" chỉ phi hình thức; độ rộng của "vùng tới hạn" phụ thuộc phân phối và quy trình huấn luyện theo cách chưa hiểu đầy đủ.
- Chưa quan sát được trường hợp "nhiều dữ liệu hơn làm hại" khi dùng early-stopping tối ưu, nhưng cũng không có lý do loại trừ khả năng này; để ngỏ cho nghiên cứu tương lai.

Nhận xét:

- Mâu thuẫn nhỏ trong văn bản: Mục 4 ghi huấn luyện Transformer "80K bước gradient", nhưng Phụ lục B.1 lại ghi "huấn luyện ... trong 80 bước gradient" khi liệt kê lại thiết lập Transformer. Rất có khả năng là lỗi đánh máy (thiếu "K"), nhưng đây là một chỗ paper tự mâu thuẫn mà chúng ta không tự sửa, chỉ ghi nhận.
- Toàn bộ thực nghiệm là phân loại ảnh (CIFAR-10/100) và một tác vụ dịch máy (IWSLT, WMT); không có thí nghiệm nào trên mô hình ngôn ngữ quy mô lớn, OCR, hay mô hình đa phương thức.
- Đây là một "phenomenon paper": mô tả và thống nhất quan sát thực nghiệm bằng một giả thuyết, không đề xuất phương pháp huấn luyện mới hay thuật toán chọn kích thước mô hình tối ưu.
- Lý thuyết chặt (Mục 3, Phụ lục C) chỉ có cho hồi quy tuyến tính/random features; với mạng sâu, các giải thích chỉ là trực giác định tính, không có bằng chứng toán học.
- Thuật ngữ "Error" trong công thức dịch máy (Phụ lục B.3) được paper định nghĩa là cross-entropy per-token, hơi khác "mean error" tổng quát trong Định nghĩa 1 (dùng cho phân loại); hai nơi dùng ký hiệu Error_S với ý nghĩa cụ thể khác nhau theo tác vụ, cần đọc kỹ ngữ cảnh khi áp dụng công thức EMC cho bài toán sinh văn bản.

7. Liên hệ với các bài khác

- **ml basics 01** (Zhang et al., "Understanding deep learning requires rethinking generalization"): bài double descent dẫn trực tiếp kết quả "mạng khớp được nhãn ngẫu nhiên" của 01 làm tiền đề cho câu hỏi "vậy sao mô hình dư tham số vẫn tổng quát hoá tốt".
- **ml basics 02** (Belkin et al., "Reconciling modern machine learning practice and the bias-variance trade-off"): đây là bài double descent **gốc** mà bài này mở rộng trực tiếp (cùng ý tưởng hai pha, cùng tên "double descent"); 02 chứng minh trên decision tree/random features/mạng 2 lớp, bài này chứng minh trên ResNet/CNN/Transformer hiện đại và thêm 2 trục mới (epoch, số mẫu).
- **interp content/08_2023_superposition_double_descent.md:** liên hệ trực tiếp về chủ đề (tên file nêu rõ double descent), khảo sát double descent qua góc nhìn superposition/biểu diễn nội tại của mạng, một hướng giải thích *cơ chế* mà bài Nakkiran et al. còn để ngỏ (mục 6).
- **llm content/03_2020_scaling_laws_kaplan.md** (Kaplan et al., Scaling Laws): liên hệ **chủ đề** (cả hai đều nói về việc tăng kích thước mô hình/dữ liệu ảnh hưởng đến loss), nhưng cần lưu ý rõ: tác giả của bài scaling laws là **Jared Kaplan**, khác với tác giả thứ hai của bài double descent là **Gal Kaplun**, hai người khác nhau dù tên gần giống nhau; đây chỉ là liên hệ về nội dung (scaling laws cho thấy xu hướng *trung bình, đơn điệu* khi tăng quy mô rất lớn, còn double descent chỉ ra các *dao động cục bộ không đơn điệu ở* vùng tới hạn) không phải liên hệ tác giả.
- **pruning content/08_2018_lottery_ticket_hypothesis.md** (Lottery Ticket Hypothesis): cùng quan tâm đến hiện tượng dư tham số (over-parameterization) của mạng sâu hiện đại; lottery ticket tìm mạng con nhỏ bên trong mạng lớn vẫn huấn luyện tốt, double descent giải thích một phần vì sao "mạng lớn, dư tham số" lại dễ huấn luyện và tổng quát hoá hơn mạng vừa đủ tham số.

8. Gợi ý áp dụng

Theo dõi cả lỗi huấn luyện lẫn lỗi kiểm định theo kích thước mô hình, số bước và lượng dữ liệu có thể giúp nhận diện hiện tượng double descent. Những đường cong trong bài không bảo đảm xu hướng tương tự sẽ xuất hiện ở mọi tác vụ hay quy trình huấn luyện.

9. Thí nghiệm gợi ý

1. **Tái hiện model-wise double descent thu nhỏ:** huấn luyện CNN 5 lớp (độ rộng $k \in \{4, 8, 16, 32, 64\}$) trên một tập con CIFAR-10 (ví dụ 4.000 mẫu) với 15-20% label noise, Adam learning rate 10^{-4}, vài nghìn epoch. Đo: test error theo k. Kỳ vọng: thấy đỉnh lỗi quanh k mà mô hình vừa đạt train error ≈ 0; nếu không thấy đỉnh rõ, thử tăng % label noise (paper nói nhiễu làm đỉnh rõ hơn).
2. **Epoch-wise double descent cho một mô hình cố định:** chọn mô hình ở mục 1 với k lớn (dư tham số), huấn luyện 2.000-4.000 epoch, log test error mỗi 20 epoch. Kỳ vọng: test error giảm, tăng (quanh epoch mà train error chạm 0), rồi giảm lại nếu huấn luyện đủ lâu; nếu chỉ thấy chữ U không giảm lại, có thể mô hình chưa đủ "dư tham số" hoặc chưa huấn luyện đủ lâu.
3. **Kiểm chứng rẻ bằng Random Features (không cần GPU mạnh):** cài lại thí nghiệm Phụ lục D ở quy mô nhỏ: một tập con Fashion-MNIST hoặc MNIST (ví dụ 500 mẫu), random Fourier features với số đặc trưng d chạy từ 50 đến 1.000, lớp 2 huấn luyện bằng least-squares (có thể giải closed-form, không cần GPU). Đo test error theo (n, d). Kỳ vọng: đỉnh lỗi nằm gần đường $n = d$ (vì EMC = d chính xác trong thiết lập này), là cách rẻ nhất để kiểm tra trực giác EMC trước khi làm thí nghiệm mạng sâu tốn kém hơn.
4. **Double descent theo trục weight decay:** với mô hình nhỏ ở mục 1 (một k cố định, dư tham số, có label noise), quét hệ số weight decay λ từ 0 đến 0.1 (ví dụ 8-10 giá trị), huấn luyện tới hội tụ, đo test error theo λ. Kỳ vọng: đường cong tương tự Hình 22 của paper (test error cao nhất ở một λ trung gian, không phải ở hai đầu).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa
Effective Model Complexity	$\text{EMC}_{D,\varepsilon}(T) = \max\{n \mid \mathbb{E}_{S\sim D^n}[\text{Error}_S(T(S))] \leq \varepsilon\}$
Ngưỡng ε dùng trong thí nghiệm	$\varepsilon = 0.1$ (heuristic, không có cơ sở hình thức)
Vùng thiếu tham số	$\text{EMC}(T) \ll n$: tăng EMC $\Rightarrow$ giảm test error (chữ U cổ điển)
Vùng tới hạn	$\text{EMC}(T) \approx n$: tăng EMC có thể tăng hoặc giảm test error
Vùng dư tham số	$\text{EMC}(T) \gg n$: tăng EMC $\Rightarrow$ giảm test error
Model-wise double descent	cố định số bước huấn luyện lớn, quét kích thước mô hình
Epoch-wise double descent	cố định kiến trúc (đủ lớn), quét số epoch (epoch cũng làm tăng EMC)
Sample-wise non-monotonicity	cố định mô hình/quy trình, quét số mẫu n; tăng n dịch đỉnh sang phải
Random Features (Phụ lục D)	$\text{EMC} = d$ (số đặc trưng ngẫu nhiên) chính xác; đỉnh lỗi nằm trên đường $n = d$
Nhiễu nhãn (label noise)	xác suất p: giữ nhãn đúng với $1 - p$, nhãn sai ngẫu nhiên đều với p, lấy mẫu một lần

11. Kết luận ngắn

Paper chứng minh bằng thực nghiệm rộng (ResNet, CNN, Transformer; CIFAR-10/100, IWSLT, WMT; SGD/Adam; có/không label noise, data augmentation) rằng hiện tượng double descent của Belkin et al. (ml basics 02) không phải ngoại lệ mà xuất hiện trong nhiều thiết lập deep learning được khảo sát, và mở rộng nó vượt khỏi "số tham số" sang một khái niệm thống nhất là **effective model complexity** của cả quy trình huấn luyện. Từ đó suy ra hai hệ quả thực hành quan trọng: (1) huấn luyện lâu hơn đôi khi tự sửa được overfitting (epoch-wise double descent), nên early stopping không phải luôn luôn tối ưu; (2) nhiều dữ liệu hơn không phải luôn luôn tốt hơn ở một mô hình và quy trình huấn luyện cố định, vì ngưỡng tới hạn dịch theo số mẫu. Cơ chế sâu bên trong mạng nơ-ron hiện đại vẫn chưa được bài báo giải thích đầy đủ bằng lý thuyết, chỉ có trực giác và các kết quả chặt cho trường hợp tuyến tính.

Nguồn gốc: https://arxiv.org/abs/1912.02292

Bài 04

XGBoost: A Scalable Tree Boosting System (2016)

Tác giả: Tianqi Chen, Carlos Guestrin (University of Washington).
Venue / arXiv: arXiv:1603.02754 (v3, 10/06/2016); KDD '16 (22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining), 13-17/08/2016, San Francisco, CA.
Nguồn: https://arxiv.org/abs/1603.02754
Chủ đề: hệ thống gradient tree boosting có thể mở rộng (XGBoost): mục tiêu học có regularization bậc hai, thuật toán tìm điểm chia (split) chính xác và xấp xỉ, weighted quantile sketch, xử lý dữ liệu sparse, và các tối ưu hệ thống (cache-aware, block storage, out-of-core, nén, chia ổ đĩa).
Vị trí trong lộ trình: Đọc trước bài 05, nơi XGBoost là một baseline cây trong so sánh trên dữ liệu bảng.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Trước khi vào paper, vài khái niệm cơ bản cho người chưa biết gì về cây quyết định và boosting:

- **Cây quyết định (decision tree) hồi quy (CART)**: một hàm $f(x)$ chia không gian đặc trưng (feature space) thành các vùng (lá, leaf) bằng các câu hỏi dạng "feature k > ngưỡng?", mỗi lá gán một giá trị số (score) cố định cho mọi điểm rơi vào đó.
- **Ensemble (tổ hợp mô hình)**: dự đoán cuối = tổng/trung bình dự đoán của nhiều mô hình con. **Boosting** là một kiểu ensemble trong đó các mô hình con được thêm **tuần tự**, mỗi mô hình mới cố sửa lỗi còn lại của tổ hợp trước đó (khác với **bagging** như Random Forest, nơi các cây được huấn luyện độc lập, song song, trên các mẫu con ngẫu nhiên).
- **Gradient boosting**: coi việc thêm từng cây là một bước giảm gradient (gradient descent) trong "không gian hàm" (function space): cây mới được huấn luyện để khớp với gradient (phần dư có hướng) của hàm mất mát (loss) tại vị trí dự đoán hiện tại, thay vì khớp trực tiếp với nhãn y.

Đến năm 2016, gradient tree boosting (còn gọi là GBM hay GBRT) đã được bài báo ghi nhận là "kỹ thuật học máy nổi bật trong nhiều ứng dụng" (Mục 1), cho kết quả tốt trên nhiều benchmark phân loại chuẩn, và LambdaMART (một biến thể boosting cho ranking) đạt kết quả tốt nhất cho bài toán ranking. Nhưng paper chỉ ra: các hướng "out-of-core computation, cache-aware và sparsity-aware learning" chưa được khai thác trong các hệ thống cây boosting hiện có, và chưa có hệ thống **end-to-end** nào kết hợp đủ các khía cạnh này.

Bằng chứng về tầm ảnh hưởng thực tế mà bài báo trích dẫn: trong 29 giải pháp vô địch các cuộc thi được đăng trên blog của Kaggle năm 2015, có 17 giải pháp dùng XGBoost (8 giải pháp chỉ dùng XGBoost, phần còn lại kết hợp XGBoost với neural net trong ensemble); phương pháp phổ biến thứ hai là deep neural net, xuất hiện trong 11 giải pháp. Tại KDDCup 2015, XGBoost được mọi đội trong top-10 sử dụng.

Câu hỏi đặt ra: làm sao xây một hệ thống tree boosting **có thể mở rộng** (scalable) tới hàng tỷ ví dụ, dùng ít tài nguyên hơn các hệ thống hiện có, mà vẫn giữ (hoặc cải thiện) độ chính xác?

2. Ý tưởng chính

XGBoost = một mục tiêu học (objective) có regularization cho tree ensemble, cộng với một tập các thuật toán và tối ưu hệ thống để huấn luyện mục tiêu đó ở quy mô lớn. Các ý tưởng cốt lõi:

1. **Mục tiêu học có regularization** (Mục 2.1): ngoài loss thông thường, paper cộng thêm một số hạng $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ phạt số lá T và độ lớn trọng số lá w, giúp làm mượt các trọng số học được để

tránh overfitting.

2. **Xấp xỉ bậc hai (second-order Taylor) của loss** (Mục 2.2): với một hàm loss khả vi lồi bất kỳ, xấp xỉ bậc hai quanh dự đoán hiện tại cho một hàm mục tiêu đơn giản chỉ phụ thuộc vào gradient g_i và Hessian h_i của từng điểm dữ liệu. Nhờ đó suy ra công thức đóng (closed-form) cho trọng số lá tối ưu và một **điểm số cấu trúc cây** (structure score) dùng để đánh giá chất lượng một cách chia cây, tương tự độ đo impurity (như Gini) trong cây quyết định thường, nhưng tổng quát cho mọi loss khả vi bậc hai.
3. **Weighted quantile sketch**: một cấu trúc dữ liệu mới (có bảo đảm lý thuyết, chứng minh ở phụ lục) để tìm các điểm phân vị (quantile) xấp xỉ trên dữ liệu có trọng số, cần cho thuật toán tìm split xấp xỉ khi dữ liệu không có trọng số bằng nhau (mà h_i đóng vai trò trọng số).
4. **Sparsity-aware split finding**: học một "hướng mặc định" (default direction) tại mỗi nút cây để xử lý giá trị thiếu/sparse một cách thống nhất, thay vì cần tiền xử lý riêng.
5. **Tối ưu hệ thống**: lưu dữ liệu theo cấu trúc "block" dạng compressed column (CSC), truy cập cache-aware, nén block, và chia (shard) dữ liệu trên nhiều ổ đĩa để hỗ trợ tính toán ngoài bộ nhớ (out-of-core) và phân tán (distributed).

3. Phương pháp chi tiết

3.1 Mục tiêu học có regularization (Mục 2.1, Eq. 1-2)

Với tập dữ liệu n ví dụ, m đặc trưng, tree ensemble dùng K hàm cộng để dự đoán:

$$\hat{y}_i = \phi(x_i) = \sum_{k=1}^{K} f_k(x_i), \quad f_k \in \mathcal{F} \tag{1}$$

trong đó $\mathcal{F} = \{f(x) = w_{q(x)}\}$ là không gian các cây hồi quy (CART): q ánh xạ một ví dụ vào chỉ số lá, T là số lá, $w \in \mathbb{R}^T$ là vector điểm số các lá. Mục tiêu học:

$$L(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k), \qquad \Omega(f) = \gamma T + \tfrac{1}{2}\lambda \|w\|^2 \tag{2}$$

l là loss khả vi lồi bất kỳ (ví dụ squared loss cho hồi quy, logistic loss cho phân loại). Ω phạt cây có nhiều lá (γ) và lá có trọng số lớn (λ), đóng vai trò tương tự L2-regularization trong mô hình tuyến tính. Paper ghi: khi $\gamma = \lambda = 0$, mục tiêu quay lại gradient tree boosting truyền thống. Kỹ thuật regularization tương tự cũng dùng trong Regularized Greedy Forest (RGF), nhưng XGBoost "đơn giản hơn và dễ song song hoá hơn".

3.2 Gradient tree boosting: xấp xỉ bậc hai (Mục 2.2, Eq. 3-6)

Vì tham số là các hàm (cây) nên không tối ưu trực tiếp bằng gradient descent kiểu Euclid được; model được huấn luyện theo kiểu cộng dần (additive): ở bước t, thêm f_t để tối thiểu

$$L^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Xấp xỉ bậc hai (Taylor) quanh $\hat{y}^{(t-1)}$:

$$L^{(t)} \simeq \sum_{i=1}^{n} \left[l(y_i, \hat{y}^{(t-1)}) + g_i f_t(x_i) + \tfrac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

với $g_i = \partial_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$, $h_i = \partial^2_{\hat{y}^{(t-1)}} l(y_i, \hat{y}^{(t-1)})$ là đạo hàm bậc một và bậc hai của loss tại dự đoán hiện tại. Bỏ hằng số, mục tiêu đơn giản ở bước t:

$$\tilde{L}^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(x_i) + \tfrac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t) \tag{3}$$

Gọi $I_j = \{i \mid q(x_i) = j\}$ là tập ví dụ rơi vào lá j. Khai triển Ω và viết lại theo từng lá:

$$\tilde{L}^{(t)} = \sum_{j=1}^{T} \left[(\sum_{i \in I_j} g_i) w_j + \tfrac{1}{2} (\sum_{i \in I_j} h_i + \lambda) w_j^2 \right] + \gamma T \tag{4}$$

Đây là T hàm bậc hai độc lập theo w_j (một cho mỗi lá). Với cấu trúc cây $q(x)$ cố định, trọng số lá tối ưu có công thức đóng:

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda} \tag{5}$$

và giá trị mục tiêu tối ưu tương ứng (dùng làm **điểm số cấu trúc cây**, structure score, để so sánh các cấu trúc cây khác nhau, tương tự độ đo impurity trong cây quyết định bình thường):

$$\tilde{L}^{(t)}(q) = -\tfrac{1}{2} \sum_{j=1}^{T} \frac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T \tag{6}$$

Ví dụ nhỏ (tự tính). Giả sử một lá chứa 2 điểm với $g_1 = -1.0,\ h_1 = 1$ và $g_2 = 0.4,\ h_2 = 1$, $\lambda = 1$. Trọng số lá tối ưu theo (5): $w^* = -\dfrac{-1.0 + 0.4}{1 + 1 + 1} = -\dfrac{-0.6}{3} = 0.2$ (tự tính). Đóng góp của lá này vào (6): $-\tfrac{1}{2} \cdot \dfrac{(-0.6)^2}{3} = -\tfrac{1}{2} \cdot \dfrac{0.36}{3} = -0.06$ (tự tính).

3.3 Công thức lợi ích khi chia một nút (split gain, Eq. 7)

Vì không thể duyệt hết mọi cấu trúc cây, thuật toán tham lam (greedy) bắt đầu từ một lá và thêm nhánh dần. Nếu chia một nút thành I_L, I_R ($I = I_L \cup I_R$), độ giảm loss sau khi chia:

$$L_{split} = \tfrac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma \tag{7}$$

Công thức này được dùng trong thực hành để đánh giá các split ứng viên (candidate split).

Ví dụ nhỏ (tự tính). 3 điểm trong một nút với $g = [-1,\ -0.5,\ 0.8]$, $h = [1, 1, 1]$, $\lambda = 1$, $\gamma = 0.1$. Giả sử chia điểm 1, 2 sang trái và điểm 3 sang phải: $G_L = -1.5,\ H_L = 2$; $G_R = 0.8,\ H_R = 1$; $G = -0.7,\ H = 3$. Áp dụng (7):

$$L_{split} = \tfrac{1}{2} \left[\frac{(-1.5)^2}{2 + 1} + \frac{0.8^2}{1 + 1} - \frac{(-0.7)^2}{3 + 1} \right] - 0.1 = \tfrac{1}{2} [0.75 + 0.32 - 0.1225] - 0.1 \approx 0.379$$

(tự tính: $0.5 \times 0.9575 - 0.1 = 0.37875$). Vì $L_{split} > 0$, chia này có lợi (giảm loss ròng sau khi trừ phí độ phức tạp γ); nếu gain âm, thuật toán không chia (tương đương điều kiện dừng/pruning).

3.4 Shrinkage và column subsampling (Mục 2.3)

Hai kỹ thuật bổ sung chống overfitting:

- **Shrinkage** (Friedman): nhân trọng số của cây mới thêm với hệ số η (tương tự learning rate), giảm ảnh hưởng từng cây riêng lẻ, để dành "chỗ" cho các cây sau cải thiện tiếp.
- **Column (feature) subsampling**: mỗi cây/mỗi nhánh chỉ dùng một tập con cột ngẫu nhiên, kỹ thuật vay từ Random Forest. Theo phản hồi người dùng trích trong paper, column subsampling đôi khi chống overfit tốt hơn row subsampling (vốn cũng được hỗ trợ).

3.5 Thuật toán tìm split: exact greedy và xấp xỉ (Mục 3)

Exact greedy (Algorithm 1): với mỗi đặc trưng, sắp xếp dữ liệu theo giá trị đặc trưng, quét tuyến tính để tích luỹ G_L, H_L và tính gain (7) cho mọi điểm chia khả dĩ, lấy gain lớn nhất trên tất cả đặc trưng. Đây là cách scikit-learn, R gbm, và bản một máy của XGBoost dùng. Nhược điểm: không hiệu quả khi dữ liệu không nằm hết trong bộ nhớ.

Thuật toán xấp xỉ (Algorithm 2): đề xuất các điểm chia ứng viên theo phân vị (percentile) của đặc trưng, rồi gộp thống kê gradient vào các "bucket" giữa các điểm ứng viên, tìm gain tốt nhất trong số các bucket. Có hai biến thể:

- **Global**: đề xuất toàn bộ ứng viên một lần ở đầu, dùng lại cho mọi mức của cây; cần ít bước đề xuất hơn nhưng cần nhiều ứng viên hơn (vì không tinh chỉnh lại sau mỗi lần chia).
- **Local**: đề xuất lại sau mỗi lần chia; cần ít ứng viên hơn cho cùng độ chính xác, phù hợp hơn cho cây sâu.

Thực nghiệm trên Higgs 10M dataset (Hình 3, Mục 3.2) cho thấy: local cần ít bucket hơn global để đạt cùng Test AUC; global có thể đạt độ chính xác tương đương local nếu cho đủ số ứng viên; với mức xấp xỉ hợp lý (tham số ϵ), chiến lược quantile đạt độ chính xác gần bằng exact greedy.

3.6 Weighted quantile sketch (Mục 3.3, chứng minh ở Phụ lục A)

Vấn đề: với dữ liệu có trọng số (ở đây, trọng số là Hessian h_i), cần tìm các điểm phân vị xấp xỉ sao cho hàm rank

$$r_k(z) = \frac{1}{\sum_{(x,h)\in D_k} h} \sum_{(x,h)\in D_k,\ x<z} h \tag{8}$$

(tỉ lệ khối lượng h của các điểm có giá trị đặc trưng nhỏ hơn z) có các điểm ứng viên $\{s_{k,1}, \dots, s_{k,l}\}$ thoả

$$|r_k(s_{k,j}) - r_k(s_{k,j+1})| < \epsilon, \quad s_{k,1} = \min_i x_{ik},\ s_{k,l} = \max_i x_{ik} \tag{9}$$

Paper chỉ ra: mục tiêu (3) có thể viết lại thành weighted squared loss với nhãn g_i/h_i và trọng số h_i, đó là lý do h_i đóng vai trò trọng số trong bài toán tìm quantile. Trước paper này, chưa có thuật toán quantile sketch nào hỗ trợ dữ liệu có trọng số với bảo đảm lý thuyết; các phương pháp cũ phải lấy mẫu ngẫu nhiên (có rủi ro thất bại) hoặc dùng heuristic không bảo đảm. XGBoost đề xuất một cấu trúc dữ liệu mới có phép **merge** (ghép hai summary, lỗi xấp xỉ là max của hai lỗi thành phần, Định lý A.1) và **prune** (giảm số điểm về $b+1$, lỗi tăng thêm $1/b$, Định lý A.2), cho phép dùng trong các khung phân tán/streaming giống GK summary cổ điển nhưng tổng quát hoá cho dữ liệu có trọng số. Phụ lục A trình bày đầy đủ định nghĩa, 2 định lý và chứng minh.

3.7 Sparsity-aware split finding (Mục 3.4, Algorithm 3)

Dữ liệu sparse phát sinh từ 3 nguồn paper liệt kê: (1) giá trị thiếu, (2) nhiều mục bằng 0 trong thống kê, (3) sản phẩm phụ của feature engineering như one-hot encoding. Ý tưởng: học một **hướng mặc định** tại mỗi nút; khi một giá trị bị thiếu, ví dụ đó tự động đi theo hướng mặc định. Thuật toán 3 chỉ quét các mục không thiếu $I_k = \{i \in I \mid x_{ik} \neq \text{missing}\}$ hai lần (giả định thiếu đi phải, rồi giả định thiếu đi trái), chọn hướng cho gain

lớn hơn. Nhờ đó độ phức tạp tỉ lệ tuyến tính với số mục không-thiếu, không phải tổng số mục. Thực nghiệm trên Allstate-10K (Hình 5): thuật toán sparsity-aware nhanh hơn **50 lần** so với bản naive không xét sparsity.

3.8 Thiết kế hệ thống (Mục 4)

- **Column block** (Mục 4.1): lưu dữ liệu trong các "block" theo định dạng compressed column (CSC), mỗi cột đã sắp xếp theo giá trị đặc trưng, tính một lần và dùng lại cho mọi vòng lặp sau. Nhờ đó tìm split ở mọi lá trong một lần quét block (không cần sắp xếp lại), hỗ trợ song song hoá việc thu thập thống kê theo cột, và dễ áp dụng column subsampling (chỉ chọn một tập con cột trong block). Phân tích độ phức tạp thời gian (Mục 4.1): exact greedy gốc là $O(Kd\|x\|_0 \log n)$ (d = độ sâu tối đa, K = số cây, $\|x\|_0$ = số mục không-thiếu); với cấu trúc block giảm xuống $O(Kd\|x\|_0 + \|x\|_0 \log n)$ (phần $\log n$ chỉ tính một lần, khấu trừ). Với thuật toán xấp xỉ: từ $O(Kd\|x\|_0 \log q)$ xuống $O(Kd\|x\|_0 + \|x\|_0 \log B)$ (q = số ứng viên đề xuất, thường 32-100; B = số dòng tối đa mỗi block).
- **Cache-aware access** (Mục 4.2): vì thống kê gradient được truy cập theo thứ tự đặc trưng (không liên tục theo chỉ số dòng), gây cache-miss khi dữ liệu lớn. Giải pháp: mỗi thread có buffer nội bộ, prefetch thống kê vào buffer rồi mới tích luỹ theo mini-batch, biến phụ thuộc đọc/ghi trực tiếp thành phụ thuộc dài hơn. Thực nghiệm (Hình 7, Mục 4.2): cache-aware nhanh gấp đôi so với bản naive trên Allstate 10M và Higgs 10M (ít khác biệt trên bộ 1M vì đã vừa cache).
- **Block cho out-of-core** (Mục 4.3): chia dữ liệu ra nhiều block lưu trên đĩa, dùng thread riêng để prefetch block vào bộ nhớ song song với tính toán. Hai kỹ thuật: (a) **block compression** theo cột, giải nén khi nạp vào bộ nhớ (đạt tỉ lệ nén khoảng 26% đến 29% trên các bộ dữ liệu đã thử; với chỉ số dòng, dùng offset 16-bit tương ứng 2^{16} ví dụ mỗi block); (b) **block sharding**: chia dữ liệu luân phiên ra nhiều đĩa, mỗi đĩa có một thread prefetch riêng, thread huấn luyện đọc luân phiên từ các buffer.

4. Thí nghiệm và kết quả chính

Bảng 1 (so sánh tính năng các hệ thống boosting cây, Mục 5):

Hệ thống	Exact greedy	Approx global	Approx local	Out-of-core	Sparsity-aware	Parallel
XGBoost	có	có	có	có	có	có
pGBRT	không	không	có	không	không	có
Spark MLLib	không	có	không	không	một phần	có
H2O	không	có	không	không	một phần	có
scikit-learn	có	không	không	không	không	không
R GBM	có	không	không	không	một phần	không

Bảng 2 (dataset dùng trong thực nghiệm, Mục 6.2):

Dataset	n	m	Bài toán
Allstate	10M	4227	phân loại claim bảo hiểm
Higgs Boson	10M	28	phân loại event vật lý
Yahoo LTRC	473K	700	learning-to-rank
Criteo	1.7 tỷ	67	dự đoán click-through rate

Phân loại (Higgs-1M, exact greedy, 500 cây, Bảng 3): XGBoost 0.6841s/cây, Test AUC 0.8304; XGBoost (colsample=0.5) 0.6401s, AUC 0.8245; scikit-learn 28.51s, AUC 0.8302; R.gbm 1.032s, AUC 0.6224. XGBoost nhanh hơn scikit-learn hơn 10 lần với AUC tương đương; R.gbm nhanh hơn XGBoost về tốc độ mỗi cây nhưng AUC thấp hơn nhiều vì chỉ mở rộng một nhánh của cây mỗi lần (nhanh nhưng kém chính xác hơn).

Learning-to-rank (Yahoo LTRC, 500 cây, Bảng 4, Hình 10): XGBoost 0.826s/cây, NDCG@10 = 0.7892; XGBoost (colsample=0.5) 0.506s, NDCG = 0.7913; pGBRT 2.576s, NDCG = 0.7915 (pGBRT là hệ thống công bố tốt nhất trước đó cho bài toán này, chỉ hỗ trợ xấp xỉ, còn XGBoost ở đây dùng exact greedy). XGBoost chạy nhanh hơn, NDCG gần tương đương.

Sparsity-aware (Allstate-10K, Hình 5): thuật toán có xét sparsity nhanh hơn **50 lần** so với bản naive.

Cache-aware (Hình 7, các bộ Allstate/Higgs 1M và 10M): nhanh gấp đôi so với bản naive khi dữ liệu lớn (10M dòng); ít khác biệt ở 1M dòng.

Chọn block size (Hình 9): 2^{16} ví dụ/block cân bằng giữa cache và song song hoá; quá nhỏ làm song song kém hiệu quả, quá lớn gây cache-miss.

Out-of-core (Criteo, 1 máy AWS c3.8xlarge: 32 vcore, 2 ổ SSD 320GB, 60GB RAM, Hình 11): thuật toán cơ bản chỉ xử lý được tới khoảng 200 triệu ví dụ (hết dung lượng đĩa/khung thực nghiệm không đủ ở các điểm lớn hơn); thêm nén cho tốc độ gấp 3; thêm chia 2 ổ đĩa cho gấp đôi nữa; phương pháp cuối cùng xử lý được **toàn bộ 1.7 tỷ ví dụ** trên một máy.

Phân tán (YARN trên EC2, máy m3.2xlarge: 8 vcore, 30GB RAM, 2 SSD 80GB, so với Spark MLLib và H2O trên 32 máy, Hình 12): XGBoost chạy nhanh hơn 10 lần mỗi vòng lặp so với Spark và 2.2 lần so với bản tối ưu của H2O (nhưng H2O nạp dữ liệu chậm nên tổng thời gian end-to-end còn kém hơn thế); XGBoost xử lý trơn tru toàn bộ 1.7 tỷ ví dụ chỉ với 4 máy. Khi tăng số máy từ 4 lên 32 (Hình 13), hiệu năng tăng gần tuyến tính, có phần hơi siêu tuyến tính vì nhiều máy hơn đồng nghĩa nhiều cache file hơn.

Ghi chú: đây là một **systems paper**, phần "thí nghiệm" tập trung vào tốc độ/khả năng mở rộng trên 4 bộ dữ liệu cụ thể, không phải một khảo sát độ chính xác rộng trên nhiều benchmark công khai.

5. Phân tích / ablation

Vì đây là systems paper, các "ablation" chủ yếu là so sánh từng thành phần hệ thống có/không:

- **Column subsampling: hiệu ứng trái chiều giữa hai bài toán.** Trên Higgs-1M (phân loại), colsample=0.5 làm AUC **giảm** nhẹ (0.8245 so với 0.8304) vì theo giải thích của tác giả, Higgs có ít đặc trưng quan trọng nên chọn tham lam trên toàn bộ đặc trưng có lợi hơn. Trên Yahoo LTRC (ranking), colsample=0.5 lại làm NDCG **tăng** nhẹ (0.7913 so với 0.7892), tác giả cho rằng subsampling giúp chống overfit ở đây. Đây là một điểm paper tự nêu rõ: hiệu ứng của column subsampling **phụ thuộc dataset**, không nhất quán một chiều.
- **Exact vs. approximate (global/local):** trên Higgs 10M, local cần ít candidate bucket hơn global cho cùng độ chính xác; global cần nhiều candidate hơn nhưng có thể đạt độ chính xác tương đương; với ϵ hợp lý, xấp xỉ đạt AUC gần bằng exact greedy.
- **Cache-aware prefetching:** chỉ có lợi rõ (gấp đôi tốc độ) khi dữ liệu đủ lớn (10M dòng) để thống kê gradient vượt quá cache CPU; ở 1M dòng lợi ích nhỏ hơn nhiều, cho thấy đây là một tối ưu "có điều kiện quy mô", không phải luôn luôn cần.
- **Kích thước block:** đường cong chữ U ngược theo số luồng (Hình 9): quá nhỏ thì song song kém hiệu quả do workload mỗi thread quá nhỏ; quá lớn thì cache-miss; 2^{16} là điểm cân bằng thực nghiệm trên 2 bộ dữ liệu thử.
- **Nén + chia ổ đĩa (out-of-core):** hai cải tiến này **cộng gộp** (nén cho 3x, chia ổ đĩa cho thêm 2x), cho thấy chi phí I/O đĩa và chi phí giải nén/CPU là hai nút thắt tách biệt có thể tối ưu độc lập.
- **Số máy trong cụm phân tán:** scaling gần tuyến tính nhưng hơi siêu tuyến tính, một hiệu ứng phụ (nhiều máy hơn = nhiều cache tổng hơn) chứ không phải do thuật toán song song hoá hoàn hảo hơn ở quy mô lớn; đây là điểm paper tự giải thích rõ, tránh ngộ nhận rằng thuật toán "siêu tuyến tính" theo nghĩa thuật toán học.

6. Giới hạn

Do tác giả/hoàn cảnh bài báo thể hiện (không phải diễn giải quá mức, mà quan sát trực tiếp từ nội dung):

- Thí nghiệm tập trung vào **tốc độ/khả năng mở rộng**, không có một khảo sát độ chính xác có kiểm soát trên nhiều bộ dữ liệu công khai đa dạng; "tầm ảnh hưởng" của XGBoost chủ yếu được minh chứng bằng số liệu thống kê các giải pháp thắng cuộc trên Kaggle/KDDCup, không phải một bảng so sánh accuracy trực tiếp với các phương pháp khác trên benchmark chuẩn.
- Không có so sánh trực tiếp XGBoost với Random Forest trong phần thực nghiệm (Random Forest chỉ được nhắc tới như nguồn gốc của ý tưởng column subsampling).
- Siêu tham số thực nghiệm (max depth = 8, shrinkage = 0.1, không column subsampling trừ khi nói rõ) được chọn cố định cho "mọi thí nghiệm" với chú thích ngắn "có thể tìm kết quả tương tự với độ sâu khác", không có khảo sát độ nhạy siêu tham số định lượng.
- Phần chứng minh lý thuyết của weighted quantile sketch (Phụ lục A) không đi kèm đo thực nghiệm riêng về sai số xấp xỉ của chính cấu trúc dữ liệu này ở quy mô lớn; đánh giá thực tế chỉ gián tiếp qua so sánh AUC/tốc độ giữa các mức ϵ trong Hình 3.
- Phần "out-of-core" nói các điểm dữ liệu bị thiếu trong Hình 11 "do hết dung lượng đĩa", nghĩa là chính thí nghiệm gặp giới hạn hạ tầng khi đo bản naive; đây là giới hạn của phép đo, không hẳn của phương pháp.

Nhận xét:

- Bài 05 (Grinsztajn và cộng sự) sau này cho thấy, khi so sánh có kiểm soát trên nhiều bộ dữ liệu tabular với ngân sách tìm siêu tham số công bằng, cây boosting (bao gồm XGBoost) vẫn thắng deep learning trên dữ liệu cỡ trung bình; nhưng đó là một câu hỏi khác (inductive bias trên dữ liệu tabular đa dạng, dataset cỡ 10K), khác với trọng tâm của bài này (tốc độ/khả năng mở rộng tới hàng tỷ ví dụ). Hai bài không mâu thuẫn, chỉ trả lời hai câu hỏi khác nhau.
- Bảng 1 tự giới thiệu XGBoost là hệ thống duy nhất có đủ mọi tính năng ("có" ở mọi cột); đây là bảng do chính tác giả lập, nên cần đọc với con mắt phê bình (self-reported comparison), dù các số liệu tốc độ cụ thể (Bảng 3, 4) được đo thực nghiệm.

7. Liên hệ với các bài khác

- **ml basics 05** (Grinsztajn và cộng sự, 2022, "Why do tree-based models still outperform deep learning on tabular data?"): dùng trực tiếp XGBoost làm một trong ba mô hình cây baseline (cùng Random Forest và GradientBoostingTrees), và bảng hyperparameter search space của XGBoost trong bài 05 là minh chứng thực nghiệm mở rộng câu hỏi "cây tốt tới mức nào trên tabular" mà XGBoost (bài này) đã đặt nền.
- **ml basics 01, 02** (Zhang et al. 2016, arXiv 1611.03530; Belkin et al. 2018, arXiv 1812.11118; về generalization và bias-variance trade-off): khung lý thuyết chung về đánh đổi độ phức tạp mô hình và khả năng tổng quát hoá; γ, λ trong mục tiêu (2) của XGBoost là một cài đặt cụ thể của tư tưởng "phạt độ phức tạp để tránh overfit" mà các bài đó bàn ở mức tổng quát hơn (và cho neural net).
- **pruning content/12_2020_state_of_neural_network_pruning.md**: liên hệ gián tiếp, không phải về thuật toán cụ thể mà về tinh thần: cả pruning neural net và split-finding trong cây đều là bài toán "chọn phần nào của mô hình đáng giữ lại / chia tiếp dựa trên một điểm số importance", dù công cụ (structure score dựa gradient/Hessian cho cây, Taylor importance hoặc magnitude cho neural net) khác nhau hoàn toàn về kiến trúc áp dụng.

8. Gợi ý áp dụng

XGBoost phù hợp làm baseline mạnh cho dữ liệu dạng bảng. Bên cạnh lựa chọn siêu tham số, cách tổ chức dữ liệu, truy cập bộ nhớ và xử lý giá trị thiếu là những yếu tố cần xem xét khi đánh giá tốc độ thực tế.

9. Thí nghiệm gợi ý

1. **Boosting bằng tay trên dữ liệu đồ chơi.** Cài một gradient boosting bậc hai đơn giản (dùng công thức (5)-(7)) trên một tập hồi quy 1D tổng hợp (~200 điểm, ví dụ $y = \sin(x) +$ nhiễu), so với `DecisionTreeRegressor` của scikit-learn ở mỗi bước; đo MSE train/test theo số cây $T \in \{1, 5, 20, 100\}$ với vài giá trị shrinkage η. Kỳ vọng: η nhỏ hơn làm train loss giảm chậm hơn nhưng test loss ổn định hơn ở T lớn (ít overfit hơn); đây là ví dụ cụ thể, nhỏ, minh hoạ công thức chính của bài.
2. **Column subsampling có nhất quán không?** Trên một bộ tabular nhỏ có sẵn (ví dụ UCI wine-quality, chạy được trên CPU), dùng `xgboost` (thư viện gốc), sweep `colsample_bytree` $\in \{1.0, 0.8, 0.5, 0.3\}$, đo test AUC/accuracy. Kỳ vọng: không có hướng thắng tuyệt đối, có thể tái lập quan sát của paper rằng hiệu ứng tuỳ dataset (Mục 5 ở trên).
3. **Đo tốc độ sparsity-aware ở quy mô nhỏ.** Tạo dữ liệu sparse tổng hợp (ví dụ 80% giá trị 0 sau one-hot encoding của vài cột categorical), so sánh thời gian huấn luyện `xgboost` (có cơ chế default-direction built-in) với một cài đặt gradient boosting tự viết không xử lý sparsity (duyệt hết mọi mục kể cả 0); đo hệ số tăng tốc, so với con số 50 lần trong paper (không kỳ vọng đạt đúng 50 lần ở quy mô nhỏ, chỉ kỳ vọng thấy xu hướng cùng chiều).
4. **So sánh `tree_method` xấp xỉ vs chính xác.** Trên một bộ ~100K dòng (phù hợp CPU hoặc GPU nhỏ), chạy `xgboost` với `tree_method='exact'` so với `'approx'`/`'hist'` (số bin khác nhau, đóng vai trò giống $1/\epsilon$ trong weighted quantile sketch), đo cả AUC và thời gian/cây. Kỳ vọng: AUC gần nhau khi số bin đủ lớn, thời gian giảm rõ khi dùng histogram xấp xỉ, minh hoạ trực tiếp Hình 3 của paper.
5. **Rank function thủ công.** Với 8-10 điểm 1D có trọng số h_i tuỳ ý (tự chọn), tính hàm rank $r_k(z)$ theo công thức (8) bằng tay/bằng script ngắn, tìm các điểm chia thoả (9) với $\epsilon = 0.25$ (tức khoảng 4 bucket), so với các điểm chia phân vị không trọng số; quan sát khi h_i rất lệch (một điểm có h lớn hơn nhiều các điểm khác), quantile có trọng số đặt nhiều điểm chia gần điểm đó hơn.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / mô tả
Mục tiêu học	$L(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$, $\Omega(f) = \gamma T + \frac{1}{2}\lambda \lVert w \rVert^2$
Gradient, Hessian	$g_i = \partial_{\hat{y}^{(t-1)}} l$, $h_i = \partial^2_{\hat{y}^{(t-1)}} l$ (đạo hàm bậc 1, 2 của loss)
Trọng số lá tối ưu	$w_j^* = -\dfrac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$
Điểm số cấu trúc cây	$\tilde{L}^{(t)}(q) = -\frac{1}{2} \sum_j \dfrac{(\sum_{i \in I_j} g_i)^2}{\sum_{i \in I_j} h_i + \lambda} + \gamma T$
Split gain	$L_{split} = \frac{1}{2}\left[\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{G^2}{H+\lambda}\right] - \gamma$
Hàm rank có trọng số	$r_k(z) = \frac{1}{\sum h} \sum_{x<z} h$; điểm ứng viên thoả $\lvert r_k(s_{k,j}) - r_k(s_{k,j+1}) \rvert < \epsilon$
Shrinkage	nhân trọng số cây mới với η (giống learning rate)
Column subsampling	chọn tập con cột ngẫu nhiên mỗi cây/nhánh, vay từ Random Forest
Default direction	hướng mặc định học được tại mỗi nút để xử lý giá trị thiếu/sparse
Độ phức tạp exact greedy	gốc $O(Kd\lVert x \rVert_0 \log n)$, với block $O(Kd\lVert x \rVert_0 + \lVert x \rVert_0 \log n)$
Độ phức tạp xấp xỉ	gốc $O(Kd\lVert x \rVert_0 \log q)$, với block $O(Kd\lVert x \rVert_0 + \lVert x \rVert_0 \log B)$
Block size thực nghiệm tốt	2^{16} ví dụ/block

11. Kết luận ngắn

XGBoost kết hợp một mục tiêu học có regularization bậc hai (cho công thức đóng cho trọng số lá và điểm số cấu trúc cây), một thuật toán weighted quantile sketch có bảo đảm lý thuyết cho tìm split xấp xỉ trên dữ liệu có trọng số, một cơ chế sparsity-aware học hướng mặc định cho giá trị thiếu, và một loạt tối ưu hệ thống (block CSC, cache-aware prefetching, nén, sharding đĩa) giúp mở rộng tree boosting tới hàng tỷ ví dụ trên một máy hoặc một cụm nhỏ, nhanh hơn 10 lần so với scikit-learn và các hệ phân tán khác trong các so sánh của bài báo. Đây là một systems paper: đóng góp chính là về tốc độ/khả năng mở rộng, không phải một khảo sát độ chính xác rộng; vai trò của XGBoost trong lộ trình các bài tiếp theo (bài 05) là làm baseline cây đại diện khi so sánh với deep learning trên dữ liệu tabular.

Nguồn gốc: https://arxiv.org/abs/1603.02754

Bài 05

Why do tree-based models still outperform deep learning on tabular data? (2022)

Tác giả: Léo Grinsztajn (Soda, Inria Saclay), Edouard Oyallon (ISIR, CNRS, Sorbonne University), Gaël Varoquaux (Soda, Inria Saclay).
Venue / arXiv: arXiv:2207.08815 (v1, 18/07/2022); paper tự ghi "Preprint. Under review." ở cuối trang 1, không nêu venue chính thức trong văn bản đã tải.
Nguồn: https://arxiv.org/abs/2207.08815
Chủ đề: benchmark thực nghiệm có kiểm soát trên 45 bộ dữ liệu tabular, so sánh tree-based models (Random Forest, GradientBoosting/HistGradientBoosting, XGBoost) với deep learning (MLP, Resnet, FT_Transformer, SAINT), và một khảo sát thực nghiệm lý giải vì sao cây vẫn thắng, qua 3 phát hiện về sự khác biệt inductive bias.
Vị trí trong lộ trình: Đọc sau bài 04 về XGBoost để có cơ sở so sánh mô hình cây và mạng neural trên dữ liệu bảng.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Dữ liệu tabular (dạng bảng) là dữ liệu có các cột không đồng nhất về bản chất (tuổi, thu nhập, loại sản phẩm...), khác với ảnh (lưới pixel đồng nhất, có cấu trúc không gian) hay văn bản (chuỗi token). Đây là dạng dữ liệu rất phổ biến trong thực tế (bảo hiểm, y tế, quảng cáo...).

Deep learning đã tạo tiến bộ vượt trội trên ảnh, văn bản, audio, nhưng trên dữ liệu tabular, theo bài báo, "bức tranh mơ hồ hơn": các mô hình ensemble dựa trên cây quyết định như XGBoost vẫn là công cụ được ưa chuộng nhất bởi người thực hành và trong các cuộc thi khoa học dữ liệu. Lý do được bài báo đưa ra: các kiến trúc deep learning thường được thiết kế để nắm bắt các bất biến (invariance) và phụ thuộc không gian của dữ liệu (ví dụ CNN khai thác tính bất biến dịch chuyển của ảnh); trên dữ liệu tabular (đặc trưng không đồng nhất, mẫu nhỏ, giá trị cực đoan), rất khó tìm ra các bất biến tương ứng.

Có một dòng nghiên cứu tích cực tạo kiến trúc deep học riêng cho tabular, nhưng paper chỉ ra các tuyên bố "thắng/ngang cây" của các công trình này đã bị nghi ngờ: một Resnet đơn giản được cho là cạnh tranh được với một số mô hình mới (Gorishniy et al. 2021), và hầu hết các phương pháp mới "có vẻ fail trên tập dữ liệu mới" (Shwartz-Ziv và Armon, 2021). Nguyên nhân sâu hơn: không có benchmark chuẩn cho tabular (khác với ImageNet cho ảnh, hay (Super)GLUE cho NLP), tập dữ liệu tabular công khai thường nhỏ hơn nhiều so với các subdomain khác, cộng với các vấn đề tái lập chung của machine learning (công sức tune siêu tham số không đồng đều giữa các phương pháp so sánh, không tính đến phương sai thống kê trong benchmark).

Câu hỏi của bài báo: (1) sau khi kiểm soát công bằng các vấn đề trên, cây vẫn thắng deep learning trên tabular không? (2) nếu có, **vì sao**?

2. Ý tưởng chính

Paper có hai phần gắn chặt với nhau:

1. **Một benchmark tabular chuẩn với 45 tập dữ liệu**, được chọn theo tiêu chí rõ ràng (đồng nhất hoá các vấn đề phụ như categorical/missing data), so sánh 3 mô hình cây (Random Forest, GBT/HistGBT, XGBoost) với 4 mô hình deep (MLP, Resnet, FT_Transformer, SAINT) qua một quy trình random

search ~400 lần thử mỗi tập dữ liệu, lặp lại 15 lần để có ước lượng kiểu bootstrap. Kết quả: **cây vẫn là state-of-the-art trên dữ liệu cỡ trung bình (~10K mẫu)**, ngay cả khi không tính đến việc NN chậm hơn.

2. **Một khảo sát thực nghiệm về inductive bias**: bằng cách biến đổi dữ liệu (làm mượt target, thêm/bớt đặc trưng vô nghĩa, xoay ngẫu nhiên hệ trục đặc trưng) để thu hẹp hoặc nới rộng khoảng cách hiệu năng giữa cây và NN, paper rút ra 3 phát hiện giải thích tại sao cây phù hợp hơn với tabular: (1) NN có thiên hướng học hàm **quá mượt** so với target thực tế thường không mượt; (2) các kiến trúc kiểu MLP **không bền** với đặc trưng vô nghĩa (uninformative features), rất phổ biến trong tabular; (3) dữ liệu tabular **không bất biến khi xoay hệ trục** (mỗi cột mang ý nghĩa riêng), nhưng MLP-like là bất biến xoay, nên "lạc" thông tin về hướng gốc của dữ liệu.

Từ đó, paper đề ra 3 "thách thức" cho ai muốn xây kiến trúc NN chuyên cho tabular: (1) bền với đặc trưng vô nghĩa, (2) giữ được hướng (orientation) gốc của dữ liệu, (3) dễ học hàm bất quy tắc (irregular).

3. Phương pháp chi tiết

3.1 45 bộ dữ liệu tham chiếu (Mục 3.1)

Dataset chủ yếu lấy từ OpenML, chọn theo các tiêu chí: **cột không đồng nhất** (loại dataset ảnh/signal mà mọi cột là cùng loại tín hiệu); **không high-dimensional** (tỉ lệ số đặc trưng/số mẫu $d/n < 1/10$); loại dataset thiếu tài liệu (giữ lại nếu cột ẩn tên nhưng rõ là đặc trưng không đồng nhất); **i.i.d.** (loại time series/stream); **dữ liệu thực** (loại dataset nhân tạo, trừ một số mô phỏng có ý nghĩa thực tế như Higgs); **không quá nhỏ** (loại dataset có dưới 4 đặc trưng hoặc dưới 3.000 mẫu); **không quá dễ** (loại dataset nếu Logistic/Linear Regression default đạt điểm chênh dưới 5% so với cả Resnet default và HistGradientBoosting default, tức gần như đã chạm giới hạn Bayes); **không tất định** (loại dataset mà target là hàm tất định của input, chủ yếu là game như poker/chess, vì các tác giả cho rằng loại dữ liệu này khác bản chất với tabular thực tế).

3.2 Loại bớt các vấn đề phụ (Mục 3.2)

Để giữ bài toán tập trung vào đặc thù tabular, paper chuẩn hoá: **train set cỡ trung bình** (cắt về tối đa 10.000 mẫu; có khảo sát riêng cỡ lớn 50.000 mẫu ở Phụ lục A.2); **không có missing data** (loại cột có nhiều thiếu, rồi loại dòng còn thiếu); **cân bằng lớp** (với phân loại, nếu nhiều lớp thì lấy 2 lớp đông nhất và giữ nửa mỗi lớp); **categorical cardinality thấp** (loại categorical feature có trên 20 giá trị khác nhau); **numerical cardinality cao** (loại numerical feature có dưới 10 giá trị duy nhất; numerical có đúng 2 giá trị được chuyển thành categorical).

3.3 Quy trình benchmark có tính đến chi phí chọn hyperparameter (Mục 3.3)

Vì việc tune hyperparameter tạo phương sai không kiểm soát được trên benchmark (đặc biệt với ngân sách nhỏ), paper thiết kế quy trình lấy mẫu đồng thời cả phương sai của việc tune và việc tăng dần ngân sách đánh giá: random search (≈ 400 lần/dataset, theo Bergstra et al. 2013), không gian siêu tham số lấy từ Hyperopt-Sklearn khi có, từ paper gốc khi có thể, và từ Gorishniy et al. (2021) cho MLP/Resnet/XGBoost. Với mỗi số lần thử n, tính tổ hợp siêu tham số tốt nhất trên validation set trong n lần đó, rồi đánh giá trên test set; lặp lại 15 lần xáo thứ tự random search để có ước lượng kiểu bootstrap về điểm test kỳ vọng. Mọi random search đều bắt đầu bằng hyperparameter mặc định của mô hình. Code và dữ liệu thô của toàn bộ 20.000 giờ tính toán (compute-hours) random search được chia sẻ công khai (github.com/LeoGrin/tabular-benchmark) để các nhà nghiên cứu khác kiểm thử rẻ các phương pháp mới.

3.4 Gộp kết quả qua các dataset: ADTM (Mục 3.4)

Để so sánh công bằng giữa các dataset có độ khó khác nhau, paper dùng độ đo giống **distance to the minimum (ADTM khi lấy trung bình)**, theo Wistuba et al. (2015) và Feurer et al. (2021): chuẩn hoá tuyến tính (affine) điểm test accuracy/R^2 về $[0, 1]$ giữa mô hình tốt nhất và mô hình ở phân vị lỗi 10% (phân loại) hoặc 50%

(hồi quy), **không dùng mô hình tệ nhất tuyệt đối** (vì điểm tệ nhất thường do mô hình/siêu tham số ngoại lệ, không đại diện độ khó dataset). Với hồi quy, điểm âm (dưới phân vị 50%) bị cắt về 0.

Ví dụ nhỏ (tự tính). Giả sử trên một dataset, mô hình tốt nhất đạt accuracy 0.90, mô hình ở quantile lỗi 10% đạt 0.70 (đóng vai trò "điểm sàn"), và một mô hình B đang xét đạt 0.85. Điểm ADTM chuẩn hoá của B:

$$\text{ADTM}_B = \frac{\text{acc}_B - \text{acc}_{\text{sàn}}}{\text{acc}_{\text{best}} - \text{acc}_{\text{sàn}}} = \frac{0.85 - 0.70}{0.90 - 0.70} = \frac{0.15}{0.20} = 0.75$$

(tự tính). Điểm 1.0 nghĩa là ngang mô hình tốt nhất trên dataset đó, 0.0 nghĩa là ngang hoặc kém hơn mô hình ở quantile lỗi 10%.

3.5 Chuẩn bị dữ liệu (Mục 3.5)

Hạn chế xử lý tay: **Gaussianize** đặc trưng cho huấn luyện NN (dùng `QuantileTransformer` của scikit-learn); với hồi quy, log-transform target nếu phân phối lệch nặng (ví dụ giá nhà), và thêm tuỳ chọn Gaussianize target khi fit rồi biến đổi ngược khi đánh giá (`TransformedTargetRegressor` + `QuantileTransformer`); **OneHotEncoder** cho các mô hình không xử lý categorical một cách tự nhiên.

3.6 Các mô hình benchmark (Mục 4.1)

- **Cây**: Random Forest, GradientBoostingTrees (GBT), hoặc HistGradientBoostingTrees khi có categorical feature, và **XGBoost** (Chen và Guestrin, 2016, chính là "ml basics 04").
- **MLP**: MLP cổ điển theo Gorishniy et al. (2021), cải tiến duy nhất là scheduler `ReduceOnPlateau` của PyTorch.
- **Resnet**: giống MLP nhưng có dropout, batch/layer normalization, và skip connection.
- **FT_Transformer**: một Transformer đơn giản kết hợp module embedding cho cả đặc trưng categorical và numerical (Gorishniy et al., 2021); được chọn vì đã được benchmark "một cách thuyết phục" so với cây và các mô hình tabular khác, nên có thể coi là "trường hợp tốt nhất" (best case) cho deep learning trên tabular.
- **SAINT**: Transformer có module embedding và cơ chế **inter-sample attention** (attention giữa các mẫu trong batch, không chỉ giữa các đặc trưng của một mẫu) (Somepalli et al., 2021); được chọn vì là mô hình deep tốt nhất trong review của Borisov et al. (2021), và để khảo sát tác dụng của inter-sample attention.

3.7 Phần cứng và không gian siêu tham số (Phụ lục A.3)

NN chạy trên GPU (nhiều loại: Quadro RTX 6000, TITAN Xp, A100, V100, Tesla T4, A40, TITAN RTX, TITAN V), cây chạy trên CPU (AMD EPYC 7742/7702, Intel Xeon E5-2660 v2 / Gold 6226R). Mặc định cho mô hình cây là mặc định của scikit-learn; mọi NN chạy tối đa 300 epoch với early stopping (patience 40 cho MLP/Resnet/FT_Transformer, 10 cho SAINT) và checkpoint mô hình tốt nhất trên validation.

Không gian siêu tham số cho XGBoost (Bảng 5, liên hệ trực tiếp "ml basics 04"):

Siêu tham số	Phân phối lấy mẫu
max_depth	UniformInt$[1, 11]$
n_estimators	UniformInt$[100, 6000,$ bước $200]$
min_child_weight	LogUniformInt$[1, 10^2]$
subsample	Uniform$[0.5, 1]$
learning_rate	LogUniform$[10^{-5}, 0.7]$
colsample_bylevel, colsample_bytree	Uniform$[0.5, 1]$
gamma	LogUniform$[10^{-8}, 7]$

Siêu tham số	Phân phối lấy mẫu
lambda	LogUniform$[1, 4]$
alpha	LogUniform$[10^{-8}, 10^{2}]$

So với bài 04 (bản gốc XGBoost dùng `max_depth=8` cố định, `shrinkage=0.1`, không column subsampling trừ khi nói rõ, cho mọi thí nghiệm hệ thống), ở đây các siêu tham số được tìm kiếm ngẫu nhiên trên một dải rộng cho **mỗi** trong 45 dataset, phản ánh đúng tinh thần "random search công bằng" mà bài 05 nhấn mạnh. Với FT_Transformer, dải siêu tham số gồm `num_layers` UniformInt$[1, 6]$, `learning_rate` LogUniform$[10^{-5}, 10^{-3}]$, `attention_dropout` Uniform$[0, 0.5]$,... (Bảng 1); với MLP, `num_layers` UniformInt$[1, 8]$, `learning_rate` LogUniform$[10^{-5}, 10^{-2}]$ (Bảng 3).

4. Thí nghiệm và kết quả chính

Tune hyperparameter không giúp NN đạt state-of-the-art (Mục 4.2, Hình 1, 2). Trên cả dataset chỉ-numerical và dataset có-categorical, cây thắng ở **mọi** ngân sách random search, và khoảng cách hiệu năng "vẫn rộng ngay cả sau một số lớn lần thử". Điều này **chưa tính** đến việc mỗi lần thử của NN nói chung chậm hơn của cây (so sánh theo thời gian ở Phụ lục A.2.1, Hình 7, 8: nhìn theo thời gian random search thay vì số lần thử, "sự vượt trội của cây trở nên rõ rệt hơn nữa", vì với cùng thời gian, cây luôn đạt điểm cao hơn hẳn NN).

Categorical variable không phải điểm yếu chính của NN (Mục 4.2). Categorical feature thường được coi là vấn đề lớn cho NN trên tabular, nhưng kết quả chỉ-numerical cho khoảng cách hẹp hơn so với khi có cả categorical; tuy vậy, **phần lớn khoảng cách vẫn còn** khi chỉ dùng đặc trưng numerical, nghĩa là categorical không phải là lý do chính khiến cây thắng.

Quy mô lớn (Phụ lục A.2.2): mở rộng sang dataset lớn hơn (train set cắt ở 50.000 mẫu so với 10.000 mẫu), với tập con dataset có đủ 50.000+ mẫu: "trong hầu hết trường hợp, tăng cỡ train set làm hẹp khoảng cách giữa NN và cây", nhưng paper để ngỏ việc nghiên cứu nghiêm ngặt xu hướng này cho công trình sau (ít dataset đủ lớn thoả điều kiện lọc của benchmark).

Hyperparameter nào quan trọng (Phụ lục A.6). Dùng một Random Forest để đo feature importance của từng siêu tham số đối với điểm số chuẩn hoá: **learning rate là siêu tham số quan trọng nhất** cho cả NN và gradient-boosted trees. Với mô hình cây, **độ sâu cây** là siêu tham số quan trọng tiếp theo, và "cây sâu hơn giúp ích, ngay cả với gradient-boosted trees" (liên hệ trực tiếp tới Finding 1 ở Mục 5 dưới: cây sâu học được pattern bất quy tắc hơn). Với XGBoost cụ thể: `learning_rate` có `rf_importance` = 0.26 (cao nhất), `min_child_weight` = 0.07, các mức `max_depth` lớn (7 đến 11) có `lin_coef` dương và khá cao (0.30 đến 0.35), cho thấy xu hướng cây sâu hơn tương quan với điểm số cao hơn.

5. Phân tích / ablation

Ba "finding" chính của paper chính là ba thí nghiệm ablation bằng cách **biến đổi dữ liệu**, giữ mô hình cố định, để cách ly từng khác biệt inductive bias (Mục 5, chỉ trên đặc trưng numerical, bài toán phân loại, dataset cỡ trung bình):

Finding 1: NN lệch về nghiệm quá mượt (Mục 5.2). Làm mượt target trên train set bằng Gaussian kernel smoother với ma trận hiệp phương sai Σ (hiệp phương sai thực nghiệm của dữ liệu, nhân với bình phương "lengthscale"):

$$K(x^*, x) = \exp\Big(-\tfrac{1}{2}(x^* - x)^T \Sigma^{-1} (x^* - x) \Big), \qquad \tilde{Y}(X_i) = \frac{\sum_j K(X_i, X_j) Y(X_j)}{\sum_j K(X_i, X_j)}$$

Lengthscale 0 tương ứng dữ liệu gốc (không làm mượt). Kết quả (Hình 3): với lengthscale nhỏ, làm mượt target **giảm rõ** accuracy của cây, nhưng **hầu như không ảnh hưởng** NN. Diễn giải: target thực tế trong các dataset này **không mượt**; cây (học hàm hằng từng khúc) không có thiên hướng mượt, còn NN thiên về hàm tần số thấp (phù hợp với "spectral bias" của Rahaman et al. 2019), nên khi hàm target vốn đã không mượt, NN "mất" ít hơn khi nó bị làm mượt thêm (vì NN đằng nào cũng không học được phần bất quy tắc). Minh hoạ cụ thể trên dataset "electricity" (Hình 20, Phụ lục A.4.1): Random Forest đạt train accuracy hoàn hảo và test accuracy 85%, cao hơn MLP mặc định (80%); Random Forest học được pattern bất quy tắc theo trục "ngày" (date feature) mà MLP không học được.

Finding 2: đặc trưng vô nghĩa ảnh hưởng MLP-like nhiều hơn (Mục 5.3). Bỏ dần đặc trưng theo thứ tự importance (đo bằng Random Forest): accuracy của GBT "không bị ảnh hưởng nhiều khi bỏ tới một nửa đặc trưng"; accuracy của GBT **chỉ huấn luyện trên các đặc trưng bị bỏ** (các đặc trưng ít quan trọng nhất) rất thấp tới khi bỏ 20% và còn thấp tới khi bỏ 50%, cho thấy phần lớn các đặc trưng này **vô nghĩa** (uninformative), không chỉ dư thừa (redundant). Thí nghiệm bỏ/thêm đặc trưng vô nghĩa (Hình 5): bỏ đặc trưng vô nghĩa làm **hẹp** khoảng cách giữa Resnet và FT_Transformer/cây; thêm đặc trưng vô nghĩa (lấy mẫu Gaussian chuẩn, không tương quan với target hay đặc trưng khác) làm **nới rộng** khoảng cách. Kết luận: MLP-like (Resnet) **kém bền** với đặc trưng vô nghĩa, và vì dataset tabular có nhiều đặc trưng loại này, đây giải thích một phần kết quả Mục 4.2.

Finding 3: dữ liệu không bất biến khi xoay, nên quy trình học cũng không nên bất biến (Mục 5.4). Lý do MLP nhạy với đặc trưng vô nghĩa: MLP **bất biến xoay** theo nghĩa Ng (2004) (học trên train rồi test không đổi khi áp một phép xoay/ma trận đơn vị lên đặc trưng ở cả train và test). Ng (2004) chứng minh: bất kỳ quy trình học bất biến xoay nào có **độ phức tạp mẫu (sample complexity) xấu nhất tăng tối thiểu tuyến tính theo số đặc trưng không liên quan**; trực giác: để bỏ đặc trưng vô nghĩa, một thuật toán bất biến xoay phải trước tiên tìm lại hướng gốc của đặc trưng rồi mới chọn đặc trưng ít thông tin nhất, nhưng thông tin nằm trong hướng gốc đã mất khi xoay. Thực nghiệm (Hình 6a): chỉ Resnet thực sự bất biến xoay (khớp lý thuyết); xoay ngẫu nhiên **đảo ngược thứ hạng hiệu năng**: NN vượt cây, Resnet vượt FT_Transformer. Paper cho rằng điều này gợi ý tính bất biến xoay **không mong muốn** trên tabular, vì (khác tầm nhìn máy, nơi có đối xứng tự nhiên) ở đây tồn tại một "hệ trục tự nhiên" (hệ trục gốc, nơi mỗi cột mang ý nghĩa riêng, như tuổi, cân nặng) mà không mô hình bất biến xoay nào phục hồi lại được. Liên hệ với đặc trưng vô nghĩa (Hình 6b): bỏ nửa đặc trưng ít quan trọng trước khi xoay làm giảm hiệu năng mọi mô hình trừ Resnet, nhưng mức giảm nhẹ hơn so với dùng toàn bộ đặc trưng. Paper liên hệ phát hiện này với lý do các embedding layer (cả cho numerical feature, như trong SAINT và trong Gorishniy et al. 2022) cải thiện hiệu năng: embedding **phá vỡ** tính bất biến xoay; vì nhiều loại embedding khác nhau đều cải thiện hiệu năng, tác giả suy đoán chính việc **có** một embedding phá vỡ bất biến (không nhất thiết là loại embedding cụ thể nào) mới là phần quan trọng, và đề xuất hướng nghiên cứu tìm cách phá vỡ bất biến xoay rẻ hơn embedding.

Thảo luận thêm với Kadra et al. (Phụ lục A.5). Paper ghi nhận: dù trong benchmark của mình cây thắng ở mọi ngân sách tune, điều này "có thể không còn đúng" khi thêm các kỹ thuật regularization bổ sung vào random search, như data augmentation; cụ thể Kadra et al. (2021a) cho thấy tìm một "cocktail" kết hợp 13 kỹ thuật regularization cho MLP có thể cạnh tranh với XGBoost sau nửa giờ tune (cho cả hai mô hình), nhưng dataset trong công trình đó khá khác, đặc biệt có cả dataset "tất định" kiểu game (bị benchmark 05 loại bỏ có chủ ý), nơi phương pháp của Kadra hoạt động rất tốt và góp phần đáng kể vào kết quả tổng.

6. Giới hạn

Do tác giả nêu (Mục 6, "Limitation"): nghiên cứu để mở các câu hỏi: còn inductive bias nào khác của cây giải thích hiệu năng của chúng trên tabular; đánh giá sẽ thay đổi ra sao trên dataset rất nhỏ, hoặc rất lớn; cách xử lý tốt nhất cho các vấn đề đặc thù như missing data hay categorical cardinality cao, cho cả NN và cây; và đánh giá sẽ thay đổi thế nào nếu tính luôn missing data.

Nhận xét:

- Phần so sánh theo số lần thử (Hình 1, 2) không tính chi phí thời gian khác nhau giữa NN (GPU) và cây (CPU); con số thời gian chỉ xuất hiện ở phụ lục (Hình 7, 8), và bản thân paper cũng nói phần cứng dùng để đo thời gian là hỗn hợp nhiều loại GPU/CPU khác nhau nên "không nên coi là so sánh tốc độ chặt chẽ".
- Benchmark phân loại chỉ dùng **nhị phân hoá** (lấy 2 lớp đông nhất, cân bằng 50/50) khi dataset gốc có nhiều lớp, nên không phản ánh đầy đủ bài toán multi-class thực tế.
- Loại bỏ **hoàn toàn** dòng/cột có missing data; đây là cách ly biến số có chủ ý (để tập trung vào câu hỏi nghiên cứu chính) nhưng nghĩa là kết luận "cây thắng" của paper chưa kiểm chứng trong điều kiện có missing data, vốn rất phổ biến trong thực tế tabular.
- 3 "finding" là các ablation gây biến đổi dữ liệu (smoothing, thêm/bớt feature, xoay), cho bằng chứng **tương quan mạnh** (quan sát cùng chiều, lặp lại trên nhiều dataset) nhưng không phải tất cả đều tách bạch hoàn toàn nguyên nhân: ví dụ với embedding "phá vỡ bất biến xoay", paper tự nhận đây là suy đoán dựa trên quan sát "nhiều loại embedding khác nhau đều giúp", chưa có thí nghiệm cách ly biến "có embedding" khỏi các thay đổi khác mà embedding mang lại (như tăng số chiều biểu diễn, thêm tham số).
- Phần Phụ lục A.5 tự thừa nhận: với một không gian tìm kiếm regularization đủ phong phú (cocktail 13 kỹ thuật của Kadra et al.), khoảng cách cây/NN có thể không còn đúng như kết luận chính của paper; nghĩa là kết luận "cây vẫn SOTA" có điều kiện vào không gian hyperparameter/regularization cụ thể mà paper chọn khảo sát, chưa phải một quy luật tuyệt đối.
- Văn bản paper tự ghi "Preprint. Under review." (chưa qua bình duyệt tại thời điểm văn bản đã tải), nên các kết luận nên được đọc như kết quả benchmark công khai, có thể còn thay đổi qua review.

7. Liên hệ với các bài khác

- **ml basics 04** (XGBoost, Chen và Guestrin 2016): là một trong ba mô hình cây baseline trực tiếp trong benchmark 05; không gian siêu tham số XGBoost (Bảng 5 ở Mục 3.7 trên) mở rộng việc tune so với cấu hình cố định trong bài 04 gốc (`max_depth=8`, `shrinkage=0.1`).
- **ml basics 01, 02** (Zhang et al. 2016, arXiv 1611.03530 "Understanding deep learning requires rethinking generalization" và Belkin et al. 2018, arXiv 1812.11118 "Reconciling modern machine learning practice and the bias-variance trade-off"): khung lý thuyết tổng quát về khả năng tổng quát hoá của mô hình overparameterized. Bài 05 bổ sung một góc nhìn thực nghiệm quan trọng: trên **cùng** một loại dữ liệu (tabular), sự khác biệt về **inductive bias cụ thể** (rotation invariance, spectral bias) giữa hai họ mô hình (cây và NN) quan trọng hơn việc chỉ xét "mô hình nào có capacity/overparameterization lớn hơn".
- **llm content/01_2017_attention_is_all_you_need.md** (Transformer gốc): FT_Transformer và SAINT, hai baseline deep trong bài 05, đều là Transformer áp dụng cho đặc trưng tabular (qua một lớp embedding categorical/numerical); hiểu kiến trúc attention gốc giúp hiểu vì sao các mô hình này vẫn bất biến xoay/nhạy với đặc trưng vô nghĩa theo cách tương tự MLP (bản chất self-attention trên các token-đặc-trưng không tự động phá vỡ bất biến xoay nếu không có embedding riêng biệt theo cột).
- **pruning content/12_2020_state_of_neural_network_pruning.md**: liên hệ gián tiếp về chủ đề "inductive bias quyết định khả năng nén/tổng quát hoá của một kiến trúc", dù pruning áp dụng cho neural net còn bài 05 so sánh hai họ mô hình khác nhau hoàn toàn.

8. Gợi ý áp dụng

Khi làm việc với dữ liệu bảng, nên so sánh mô hình cây tăng cường với mạng neural trên cùng cách chia dữ liệu và ngân sách điều chỉnh. Các đặc tính của dữ liệu, như đặc trưng không hữu ích và hàm mục tiêu không trơn, có thể ảnh hưởng tới lựa chọn kiến trúc.

9. Thí nghiệm gợi ý

1. **Tái lập Finding 1 ở quy mô nhỏ.** Lấy một dataset tabular nhỏ (ví dụ OpenML "wine_quality" hay "california housing", chạy trên CPU), huấn luyện một MLP nhỏ và một GradientBoostingTrees; làm mượt target train bằng Gaussian kernel ở vài lengthscale (có thể dùng công thức Mục 3.4 bên trên với Σ là hiệp phương sai thực nghiệm của vài đặc trưng quan trọng nhất); đo test accuracy/R^2 theo lengthscale. Kỳ vọng: GBT giảm hiệu năng rõ hơn MLP khi lengthscale tăng, lặp lại xu hướng của Hình 3 ở quy mô nhỏ.
2. **Tái lập Finding 2.** Trên cùng dataset nhỏ, thêm dần $k \in \{0, 5, 20, 50\}$ cột nhiễu Gaussian chuẩn (không tương quan với target), đo test accuracy của MLP so với XGBoost. Kỳ vọng: khoảng cách giữa hai mô hình nới rộng khi k tăng, MLP giảm hiệu năng nhanh hơn XGBoost.
3. **Tái lập Finding 3.** Trên cùng dataset, sinh một ma trận xoay ngẫu nhiên (orthogonal) bằng `scipy.stats.special_ortho_group`, áp lên cả train và test; so sánh test accuracy của MLP và XGBoost trước/sau xoay. Kỳ vọng: XGBoost giảm mạnh sau xoay (vì cây dựa vào ngưỡng theo từng cột gốc), MLP gần như không đổi (nếu đủ gần bất biến xoay trên lý thuyết).
4. **Kiểm tra tầm quan trọng của learning rate.** Chạy random search nhỏ (~30-50 lần thử, trên CPU hoặc GPU) cho một MLP nhỏ trên 1 dataset tabular, ghi lại learning rate và test score mỗi lần thử; fit một Random Forest nhỏ dự đoán score từ các siêu tham số, lấy feature importance. Kỳ vọng: learning rate có importance cao nhất, lặp lại phát hiện ở Phụ lục A.6.
5. **Tính ADTM thủ công.** Với kết quả thí nghiệm 4 (hoặc một random search nhỏ khác), tính đường cong ADTM theo công thức Mục 3.4 ở số lần thử tăng dần (ví dụ mỗi 5 lần thử), vẽ đường cong "best-so-far" theo ADTM; so sánh với đường "best-so-far" theo accuracy thô để thấy ADTM giúp so sánh công bằng hơn khi có nhiều dataset độ khó khác nhau (nếu chạy thêm trên dataset thứ hai).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / mô tả
ADTM (chuẩn hoá điểm)	$\frac{\text{score} - \text{score}_{\text{quantile sàn}}}{\text{score}_{\text{best}} - \text{score}_{\text{quantile sàn}}}$; sàn = quantile lỗi 10% (phân loại) hoặc 50% (hồi quy)
Gaussian kernel smoother	$K(x^*, x) = \exp(-\frac{1}{2}(x^* - x)^T \Sigma^{-1}(x^* - x))$; $\tilde{Y}(X_i) = \frac{\sum_j K(X_i, X_j) Y(X_j)}{\sum_j K(X_i, X_j)}$
Rotation invariance (Ng 2004)	quy trình học không đổi khi áp phép xoay (ma trận đơn vị/orthogonal) lên đặc trưng ở cả train và test
Hệ quả lý thuyết (Ng 2004)	độ phức tạp mẫu xấu nhất của thuật toán bất biến xoay tăng tối thiểu tuyến tính theo số đặc trưng không liên quan
Finding 1	NN thiên về hàm mượt (spectral bias); cây (hàm hằng từng khúc) học hàm bất quy tắc tốt hơn
Finding 2	MLP-like kém bền với đặc trưng vô nghĩa (không chỉ dư thừa); cây bền hơn
Finding 3	MLP-like gần bất biến xoay, mất thông tin hướng gốc có ý nghĩa; embedding phá vỡ bất biến này
3 thách thức cho tabular NN	(1) bền với đặc trưng vô nghĩa, (2) giữ hướng gốc dữ liệu, (3) dễ học hàm bất quy tắc
Mô hình benchmark	cây: Random Forest, GBT/HistGBT, XGBoost; deep: MLP, Resnet, FT_Transformer, SAINT

11. Kết luận ngắn

Trên một benchmark 45 tập dữ liệu tabular cỡ trung bình, được xây để kiểm soát các vấn đề tái lập phổ biến (siêu tham số tuning không công bằng, thiếu benchmark chuẩn), tree-based models (Random Forest, GBT/HistGBT, XGBoost) vẫn vượt các kiến trúc deep learning (MLP, Resnet, FT_Transformer, SAINT) ở mọi ngân sách tune, kể cả sau rất nhiều lần random search, và khoảng cách còn rõ hơn nếu tính thời gian tune. Ba thí nghiệm biến đổi dữ liệu cho thấy nguyên nhân nằm ở sự khác biệt inductive bias: NN thiên về nghiệm mượt trong khi target tabular thường bất quy tắc, NN kém bền với đặc trưng vô nghĩa (rất phổ biến trong tabular), và NN gần bất biến xoay nên mất thông tin về hướng gốc có ý nghĩa của từng cột dữ liệu, một bất biến mà bản thân dữ liệu tabular không có. Paper đề ra 3 thách thức thiết kế cho NN chuyên tabular, nhưng tự nhận còn nhiều câu hỏi mở (tập dữ liệu rất nhỏ/rất lớn, missing data, và khả năng kết luận thay đổi nếu mở rộng không gian regularization như trong công trình của Kadra et al.).

Nguồn gốc: https://arxiv.org/abs/2207.08815

Bài 06

Improving neural networks by preventing co-adaptation of feature detectors (2012)

Tác giả: G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, R. R. Salakhutdinov (Department of Computer Science, University of Toronto).
Venue / arXiv: arXiv:1207.0580 (v1, 03/07/2012); paper không nêu venue hội nghị/tạp chí trong văn bản (đây là bản báo cáo kỹ thuật ngắn, định dạng giống bài gửi tạp chí tổng hợp, không ghi rõ nơi xuất bản).
Nguồn: https://arxiv.org/abs/1207.0580
Chủ đề: "dropout": trong lúc huấn luyện, ngẫu nhiên bỏ (omit) một nửa các đơn vị ẩn ở mỗi lượt trình diễn mẫu huấn luyện, nhằm ngăn các feature detector "đồng thích nghi" (co-adapt) phụ thuộc lẫn nhau quá mức, qua đó giảm overfitting; đây là bài báo arXiv gốc giới thiệu kỹ thuật này (trước bài JMLR 2014 "Dropout: A Simple Way to Prevent Neural Networks from Overfitting" đầy đủ hơn của cùng nhóm tác giả).
Vị trí trong lộ trình: Đọc cùng bài 01 về khả năng ghi nhớ, trước các bài 07 và 08 về kiến trúc mạng ảnh.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Một mạng nơ-ron truyền thẳng (feedforward) học các **feature detector** (đơn vị ẩn) bằng cách điều chỉnh trọng số các kết nối đến. Nếu quan hệ giữa đầu vào và đầu ra đúng là phức tạp, và mạng có đủ đơn vị ẩn để mô hình hoá nó chính xác, thì (đặc biệt khi dữ liệu huấn luyện có hạn) sẽ có **rất nhiều** cách đặt trọng số khớp gần hoàn hảo tập huấn luyện. Mỗi cách đặt trọng số như vậy sẽ dự đoán khác nhau trên dữ liệu kiểm tra (test), và hầu hết sẽ tệ hơn trên test so với trên train, vì các feature detector đã được "điều chỉnh để làm việc tốt cùng nhau" (co-adapt) trên đúng tập train, không phải trên phân phối thật.

Đây chính là hiện tượng **overfitting**. Bài báo đề xuất một cách giảm overfitting bằng cách phá vỡ chủ động các "sự đồng thích nghi" này ngay trong lúc huấn luyện: ngẫu nhiên bỏ đi một nửa đơn vị ẩn ở mỗi lần trình diễn mỗi mẫu huấn luyện. Khi đó một đơn vị ẩn không thể "cậy nhờ" vào sự có mặt chắc chắn của các đơn vị khác, nên nó buộc phải học một đặc trưng **có ích một cách độc lập**, hữu dụng trong nhiều ngữ cảnh nội tại khác nhau của mạng, thay vì chỉ hữu dụng khi kết hợp đúng với một vài đơn vị cụ thể khác.

2. Ý tưởng chính

Dropout trong huấn luyện: *ở mỗi lần đưa một mẫu huấn luyện qua mạng, mỗi đơn vị ẩn bị loại khỏi mạng với xác suất 0.5 (độc lập với các đơn vị khác), nên mỗi mẫu thực chất được đưa qua một "mạng con" khác nhau (lấy mẫu từ một họ lớn các mạng), nhưng tất cả các mạng con này **chia sẻ chung một bộ trọng số** cho các đơn vị còn hiện diện.

Góc nhìn thứ hai, tương đương: model averaging hiệu quả. Cách chuẩn để giảm lỗi test là lấy trung bình dự đoán của rất nhiều mạng khác nhau, nhưng huấn luyện và chạy nhiều mạng riêng biệt rất tốn kém. Dropout cho phép huấn luyện (ngầm) một số rất lớn các mạng chỉ trong thời gian huấn luyện của **một** mạng.

Dropout ở lúc kiểm tra (test-time): "mean network". Khi dự đoán, paper dùng một mạng duy nhất chứa **tất cả** đơn vị ẩn, nhưng **giảm một nửa** các trọng số đi ra (outgoing weights) của mỗi đơn vị, để bù lại việc lúc này có gấp đôi số đơn vị hoạt động so với lúc huấn luyện (trung bình). Paper chứng minh: với mạng 1 lớp ẩn N đơn vị và lớp đầu ra softmax, dùng "mean network" này **tương đương chính xác** với việc lấy trung bình hình học (geometric mean) của 2^N phân phối xác suất dự đoán bởi toàn bộ 2^N mạng con khả dĩ. Nếu các mạng con không dự đoán hoàn toàn giống nhau, mean network được đảm bảo gán log-xác suất cho nhãn

đúng **cao hơn hoặc bằng** trung bình log-xác suất của từng mạng con riêng lẻ (dẫn kết quả từ Hinton 2002). Với hồi quy (đầu ra tuyến tính, lỗi bình phương), lỗi bình phương của mean network luôn tốt hơn hoặc bằng trung bình lỗi bình phương của các mạng con.

3. Phương pháp chi tiết

3.1 Ràng buộc trọng số (max-norm) thay cho phạt trọng số (weight penalty)

Thay vì phạt (penalize) độ dài bình phương (L_2 norm) của **toàn bộ** vector trọng số, paper đặt một **chặn trên** (upper bound) cho L_2 norm của vector trọng số **đến** (incoming) mỗi đơn vị ẩn riêng lẻ. Nếu một bước cập nhật trọng số làm vi phạm ràng buộc này, trọng số của đơn vị đó được chuẩn hoá lại (chia cho một hệ số) để thoả ràng buộc. Vì dùng ràng buộc cứng thay vì hình phạt mềm, trọng số không thể "nổ" (grow very large) dù bước cập nhật đề xuất lớn đến đâu, nên có thể bắt đầu với **learning rate rất lớn** và giảm dần theo thời gian, cho phép tìm kiếm không gian trọng số kỹ hơn so với cách bắt đầu từ trọng số nhỏ và learning rate nhỏ.

3.2 Cập nhật trọng số bằng SGD với momentum (chi tiết MNIST, Phụ lục A.1)

Công thức cập nhật (ký hiệu theo paper):

$$\Delta w_t = p_t\,\Delta w_{t-1} - (1-p_t)\,\epsilon_t\,\langle \nabla_w L \rangle, \qquad w_t = w_{t-1} + \Delta w_t$$

với lịch learning rate và momentum:

$$\epsilon_t = \epsilon_0 f^{\,t}, \qquad p_t = \begin{cases} p_i + \dfrac{t}{T}(p_f - p_i), & t < T \\ p_f, & t \geq T \end{cases}$$

Giá trị dùng cho MNIST: $\epsilon_0 = 10.0$, $f = 0.998$ (learning rate giảm theo cấp số nhân mỗi epoch), $p_i = 0.5$, $p_f = 0.99$, $T = 500$ (momentum tăng dần tuyến tính từ 0.5 lên 0.99 trong 500 epoch đầu rồi giữ nguyên). Trọng số khởi tạo từ phân phối chuẩn trung bình 0, độ lệch chuẩn 0.01. Ràng buộc L_2 bình phương tối đa cho vector trọng số đến mỗi đơn vị ẩn: $l = 15$ (chọn bằng cross-validation). Không dùng weight decay. Minibatch 100, hàm mất cross-entropy, huấn luyện 3.000 epoch.

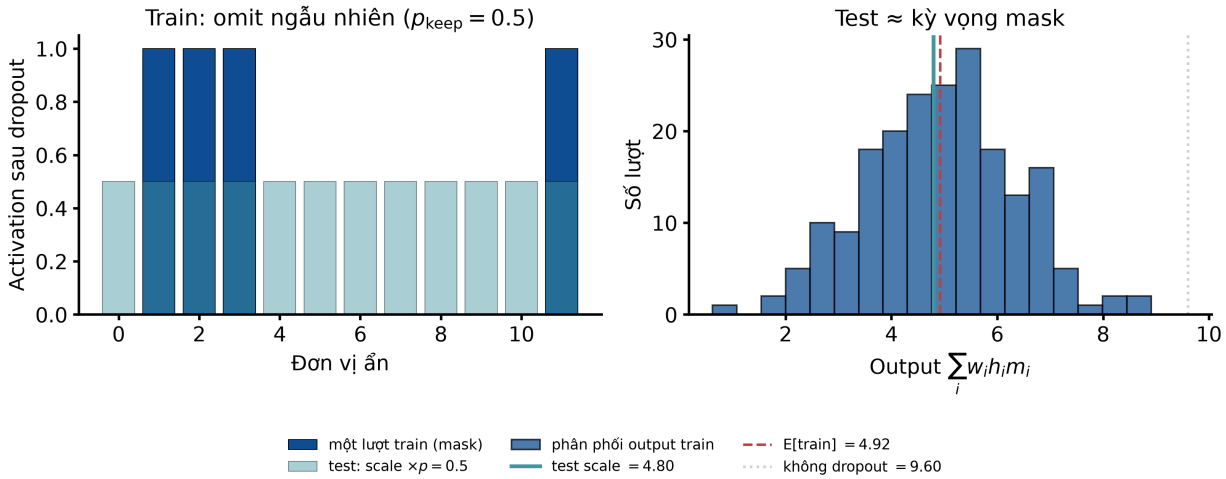
Tự tính (kiểm lại công thức learning rate): với $\epsilon_0 = 10.0$, $f = 0.998$, sau $t = 100$ epoch: $\epsilon_{100} = 10.0 \times 0.998^{100}$. Vì $0.998^{100} = e^{100 \ln 0.998} \approx e^{100 \times (-0.002002)} = e^{-0.2002} \approx 0.8185$, nên $\epsilon_{100} \approx 8.19$ (tự tính), tức sau 100 epoch learning rate mới giảm khoảng 18%, cho thấy lịch giảm khá chậm lúc đầu so với giá trị khởi điểm rất lớn $\epsilon_0 = 10.0$.

3.3 Mức dropout dùng trong thực nghiệm

- Lớp ẩn: dropout 50% (dùng xuyên suốt paper; paper nói các xác suất "cực đoan" hơn (quá cao hoặc quá thấp) cho kết quả kém hơn, và dropout ở **mọi** lớp ẩn tốt hơn dropout chỉ ở một lớp).
- Lớp đầu vào (visible units): paper thử dropout đầu vào cũng có ích, nhưng thường nên giữ lại **nhiều hơn 50%** đầu vào (ví dụ 20% dropout đầu vào, giữ 80%).
- Có thể học riêng xác suất dropout cho từng đơn vị (so sánh hiệu năng trung bình trên validation set khi có/không đơn vị đó) để cải thiện thêm một chút.
- Trong các lớp tích chập (convolutional), dropout **ít lợi ích hơn** (xem mục 5), vì chia sẻ trọng số đã giảm sẵn số tham số và giảm khả năng overfitting của lớp đó.

3.4 Dropout kết hợp pretraining sinh (generative pretraining)

Khi finetune một mô hình đã pretrain sinh (Deep Belief Network hoặc Deep Boltzmann Machine, huấn luyện bằng Contrastive Divergence), paper dùng learning rate **nhỏ** và **không** ràng buộc trọng số, để không phá vỡ các feature detector mà pretraining đã học được.



Dropout train ($p = 0.5$) vs test scale; E[train] khớp gần test scale (minh hoạ, tự tính)

4. Thí nghiệm và kết quả chính

Paper trình bày kết quả trên 5 bộ dữ liệu khác nhau, tất cả đo bằng số lỗi hoặc % lỗi trên test set, so sánh **có** và **không** dropout trên cùng kiến trúc:

Bộ dữ liệu	Không dropout	Có dropout	Ghi chú
MNIST, FFN chuẩn (không aug/conv/pretrain)	160 lỗi (SOTA công bố trước đó)	~130 lỗi (50% dropout ẩn + ràng buộc L_2); ~110 lỗi (thêm 20% dropout đầu vào)	Hình 1, nhiều kiến trúc (784-800-800-10, ...)
MNIST, finetune Deep Belief Net (784-500-500-2000)	118 lỗi	92 lỗi	backprop chuẩn vs. dropout 50% khi finetune
MNIST, finetune Deep Boltzmann Machine (5 lần pretrain độc lập)	103, 97, 94, 93, 88 lỗi (trung bình đề cập trong Phụ lục A.2: khoảng 94)	83, 79, 78, 78, 77 lỗi (trung bình 79 lỗi)	"mean of 79 errors is a record" cho phương pháp không dùng tri thức tiên nghiệm/tăng cường dữ liệu
TIMIT (nhận diện khung âm vị, core test set)	22.7% lỗi	19.7% lỗi	4 lớp ẩn 4000 đơn vị, dropout 50% ẩn + 20% input; "record" cho phương pháp không dùng thông tin người nói
CIFAR-10 (nhận diện vật thể, không data augmentation)	16.6% lỗi (so với SOTA công bố trước đó 18.5%)	15.6% lỗi	3 lớp conv + pooling + 1 lớp locally-connected; dropout 50% chỉ ở lớp locally-connected cuối
ImageNet 2010 (1000 lớp)	48.6% lỗi (so với SOTA công bố trước đó 45.7%, đạt được bằng 1 mạng đơn, so với 47.2% của đội thắng cuộc thi dùng trung bình 6 mô hình)	42.4% lỗi	dropout 50% ở 2 lớp globally-connected cuối (lớp 6 trong 7 lớp trọng số)

Bộ dữ liệu	Không dropout	Có dropout	Ghi chú
Reuters RCV1-v2 (phân loại văn bản, 50 lớp, 402.738 văn bản, chia đều train/test ≈ 201.369/201.369)	31.05% lỗi	29.62% lỗi	kiến trúc 2000-2000-1000-50 (Phụ lục C)

Kiểm tra số liệu (tự tính): trung bình 5 số lỗi backprop chuẩn trên MNIST+DBM mà văn bản chính liệt kê (103, 97, 94, 93, 88) là $(103+97+94+93+88)/5 = 475/5 = 95.0$ (tự tính), còn Phụ lục A.2 ghi "khoảng 94 lỗi" cho cùng con số trung bình này; đây là một chênh lệch nhỏ (95 so với 94) giữa phần kết quả chính và phụ lục, có thể do làm tròn, nhưng là điểm mâu thuẫn nhỏ trong chính văn bản paper mà chúng ta không tự sửa. Ngược lại, trung bình 5 số lỗi dropout (83, 79, 78, 78, 77) là $(83+79+78+78+77)/5 = 395/5 = 79.0$ (tự tính), khớp đúng với "mean of 79 errors" mà cả hai nơi trong paper đều nêu.

ImageNet 2010: ảnh gốc 256×256, huấn luyện trên patch 224×224 (+ lật ngang) để tăng cường dữ liệu; test bằng cách lấy trung bình dự đoán trên 10 patch của ảnh test (patch giữa, 4 patch góc, và ảnh lật ngang của cả 5 patch này). Mạng có 7 lớp trọng số (5 conv + 2 globally-connected); mô hình **không** dropout bỏ hẳn 2 lớp globally-connected vì "chúng gây overfitting nghiêm trọng khi không có dropout". Huấn luyện trên 1 GPU NVIDIA GTX 580 bằng cuda-convnet: khoảng 4 ngày với dropout, khoảng 2 ngày không dropout cho ImageNet; khoảng 90 phút cho CIFAR-10.

5. Phân tích / ablation

- **Hình ảnh trực quan về đặc trưng học được (Phụ lục A.3, Hình 5):** so sánh 100 đặc trưng ngẫu nhiên ở lớp ẩn thứ nhất của mạng 784-500-500 huấn luyện có/không dropout (không pretraining sinh). Đặc trưng học bằng dropout "đơn giản hơn, giống các nét vẽ (strokes)"; đặc trưng học bằng backprop chuẩn "khó diễn giải hơn". Đây là minh chứng trực tiếp cho tuyên bố chính: dropout buộc mỗi đơn vị học một đặc trưng có ích độc lập, ít đồng thích nghi hơn.
- **Dropout kém lợi ích hơn trong lớp tích chập (Phụ lục F):** vì các neuron trong một bank tích chập **chia sẻ** cùng bộ filter, số tham số đã giảm mạnh so với mạng kết nối đầy đủ; điều này tự nó đã giảm khả năng overfitting của lớp, nên dropout "far less advantageous" ở các lớp conv. Đây giải thích vì sao trong các thí nghiệm CIFAR-10/ImageNet, dropout chỉ áp dụng ở các lớp fully-connected hoặc locally-connected cuối, không áp dụng ở các lớp conv.
- **So sánh với Bayesian model averaging:** dropout đơn giản hơn nhiều so với lấy trung bình Bayes (vốn cần MCMC để lấy mẫu mô hình từ posterior với các lớp mô hình phức tạp như mạng nơ-ron); dropout giả định (ngầm, qua chia sẻ trọng số trong huấn luyện) rằng cuối cùng mọi mạng con sẽ được cho trọng số gần như bằng nhau khi tổng hợp.
- **So sánh với bagging:** dropout có thể xem là một dạng **bagging cực đoan**, nơi mỗi "mô hình" (mạng con) chỉ được huấn luyện trên **một** mẫu, và mỗi tham số bị regularize rất mạnh vì được **chia sẻ** với tham số tương ứng ở mọi mạng con khác. Paper cho rằng đây là một regularizer tốt hơn cách co trọng số về 0 (weight shrinkage) thông thường.
- **Trường hợp cực đoan: Naive Bayes.** Khi mỗi đặc trưng đầu vào được huấn luyện độc lập để dự đoán nhãn rồi nhân các phân phối dự đoán lại ở lúc kiểm tra, ta có naive Bayes; paper lưu ý cách này thường tốt hơn hồi quy logistic khi dữ liệu huấn luyện rất ít, như một dạng tương tự dropout "cực đoan".
- **Liên hệ sinh học (suy đoán, cuối bài):** paper nêu một sự tương tự thú vị với lý thuyết về vai trò của sinh sản hữu tính (sex) trong tiến hoá (dẫn Livnat, Papadimitriou, Feldman, PNAS 2008): sinh sản hữu tính "phá vỡ" các tập gen đồng thích nghi lớn, buộc một chức năng được thực hiện bằng nhiều cách nhỏ hơn, độc lập hơn, giúp tiến hoá tránh "ngõ cụt" cần thay đổi đồng thời nhiều gen, và giảm khả năng một thay đổi nhỏ của môi trường gây sụt giảm lớn về độ thích nghi (chính là "overfitting" trong học máy).

Đây là một liên hệ mang tính suy đoán của tác giả, không phải một kết quả thực nghiệm.

6. Giới hạn

Do tác giả nêu (gián tiếp, qua cách trình bày):

- Paper không đưa ra lý thuyết chặt giải thích vì sao dropout hoạt động tốt; lập luận chủ yếu dựa trên trực giác "model averaging" và "chống đồng thích nghi", kèm một kết quả toán học hẹp (so sánh mean network với trung bình log-xác suất, chỉ chứng minh chặt cho mạng 1 lớp ẩn + softmax hoặc hồi quy tuyến tính).
- Nhiều lựa chọn thiết kế (bao nhiêu lớp dropout, tỉ lệ dropout đầu vào, ràng buộc trọng số $l = 15$) được chọn bằng dò trên validation set/cross-validation, không có công thức tổng quát.
- Thời gian huấn luyện tăng đáng kể với dropout (ImageNet: khoảng 4 ngày so với 2 ngày không dropout, cùng một GPU GTX 580), do cần nhiều epoch hơn để "mạng con" khác nhau ở mỗi mẫu đều học được.

Nhận xét:

- Mâu thuẫn số liệu nhỏ: trung bình 5 lỗi backprop chuẩn trên thí nghiệm DBM-MNIST là 95 theo tính toán trực tiếp từ 5 số liệu được nêu (103, 97, 94, 93, 88), nhưng Phụ lục A.2 nói "khoảng 94 lỗi" (xem mục 4).
- Mâu thuẫn kiến trúc: văn bản chính nói mạng Reuters là "2 lớp fully-connected 2000 đơn vị ẩn" (ngụ ý cả hai lớp ẩn đều 2000 đơn vị), nhưng Phụ lục C lại ghi rõ kiến trúc dùng để báo cáo kết quả là **2000-2000-1000-50** (lớp ẩn thứ hai chỉ có 1000 đơn vị, không phải 2000) và còn thử nghiệm thêm một kiến trúc khác **2000-1000-1000-50**. Đây là một mô tả không khớp giữa văn bản chính (đơn giản hoá) và phụ lục (chi tiết hơn); chúng ta giữ số liệu phụ lục làm chính xác hơn và ghi nhận sự khác biệt này, không tự "sửa" văn bản chính.
- Đây là bài báo rất ngắn (7 trang chính + phụ lục), không có so sánh với các kỹ thuật regularization khác (weight decay đơn thuần, data augmentation đơn thuần) trên cùng điều kiện một cách hệ thống; phần lớn so sánh là "có dropout" với "không dropout" trên kiến trúc gần giống nhau, không phải ablation đầy đủ.
- Không có thí nghiệm nào trên dữ liệu dạng ảnh văn bản/OCR hay dữ liệu tuần tự dài; tất cả đều là phân loại ảnh/âm thanh/văn bản dạng túi từ (bag-of-words), không có tác vụ sinh chuỗi.

7. Liên hệ với các bài khác

- **ml basics 01** (Zhang et al., "rethinking generalization"): cùng chủ đề overfitting/khả năng ghi nhớ của mạng nơ-ron; dropout là một trong những công cụ thực hành sớm nhất để kiểm soát khoảng cách giữa train và test error mà 01 đặt câu hỏi về mặt lý thuyết.
- **ml basics 03** (Nakkiran et al., "Deep Double Descent"): bài double descent có một thí nghiệm phụ lục (Hình 22 trong 03) cho thấy double descent cũng xảy ra theo trục weight decay; dropout là một trục regularization khác có thể đặt câu hỏi tương tự (không được paper dropout này kiểm tra, nhưng là hướng liên hệ tự nhiên).
- **distillation content/02_2015_distilling_knowledge_hinton.md** (Hinton, Vinyals, Dean, "Distilling the Knowledge in a Neural Network"): cùng tác giả chính Geoffrey Hinton; hai bài đều xoay quanh ý tưởng "nhiều mô hình/nhiều cấu hình ẩn chứa thông tin hữu ích có thể gộp lại": dropout gộp ngầm rất nhiều mạng con qua chia sẻ trọng số và trung bình hoá tại test-time (mean network), trong khi distillation gộp tri thức của một (hoặc nhiều) mô hình giáo viên vào một mô hình học sinh qua soft target; có thể xem dropout và distillation là hai cách khác nhau để "nén" lợi ích của ensemble/model-averaging vào một mạng duy nhất.
- **pruning content/12_2020_state_of_neural_network_pruning.md** ("What is the State of Neural Network Pruning?"): dropout và pruning là hai hướng tiếp cận khác nhau để kiểm soát dung lượng hiệu dụng của mạng (dropout giảm dung lượng hiệu dụng *động*, ngẫu nhiên trong lúc huấn luyện; pruning

giảm dung lượng *tĩnh*, cố định sau khi huấn luyện xong); so sánh hai hướng giúp hiểu rõ hơn vì sao một mạng dư tham số có thể "chịu được" cả việc bỏ ngẫu nhiên đơn vị (dropout) và bỏ vĩnh viễn một phần tham số (pruning) mà vẫn hoạt động tốt.

8. Gợi ý áp dụng

Dropout là một lựa chọn regularization có thể thử khi mô hình overfit, nhưng tác dụng phụ thuộc dữ liệu và kiến trúc. Cần phân biệt cách chuẩn hóa trong triển khai cụ thể và chế độ huấn luyện với chế độ suy luận.

9. Thí nghiệm gợi ý

1. **Tái hiện kết quả MNIST gốc ở quy mô nhỏ:** huấn luyện mạng 784-800-800-10 trên MNIST đầy đủ (hoặc tập con), so sánh test error giữa (a) không dropout, (b) dropout 50% ẩn + ràng buộc L_2 như Phụ lục A.1, (c) thêm 20% dropout đầu vào. Đo số lỗi trên 10.000 ảnh test. Kỳ vọng: xu hướng giảm lỗi dần từ (a) đến (c) giống paper (không cần khớp đúng số 160/130/110, chỉ cần đúng **chiều** và độ lớn tương đối).
2. **Trực quan hoá đặc trưng lớp 1 (tái hiện Hình 5):** với cùng kiến trúc 784-500-500, vẽ 100 bộ lọc (trọng số đến) của lớp ẩn thứ nhất khi huấn luyện có/không dropout. Đo (định tính): đặc trưng dropout có "giống nét vẽ" hơn không; có thể định lượng bằng độ thưa (sparsity) hoặc entropy không gian của từng bộ lọc.
3. **Kiểm tra tuyên bố "dropout ít lợi ích ở lớp conv":** trên một CNN nhỏ cho CIFAR-10 (ví dụ 3-4 lớp conv + 1-2 lớp FC), thử 3 cấu hình: (a) không dropout, (b) dropout 50% chỉ ở lớp conv, (c) dropout 50% chỉ ở lớp FC cuối. Đo test accuracy. Kỳ vọng theo paper: (c) tốt hơn rõ rệt so với (a); (b) cải thiện ít hoặc không đáng kể so với (a), thậm chí có thể kém hơn (c).
4. **So sánh "mean network" (1 lần truyền) với Monte Carlo averaging (N lần truyền):** trên một mạng nhỏ đã huấn luyện với dropout, so sánh độ chính xác/log-loss khi dự đoán bằng (a) mean network (tắt dropout, dùng trọng số đã scale, 1 lần truyền) so với (b) trung bình xác suất của $N = 50$ lần truyền ngẫu nhiên có dropout bật (Monte Carlo dropout). Đo chênh lệch log-loss giữa (a) và (b), và thời gian suy luận tương ứng. Kỳ vọng: (a) gần (b) về độ chính xác nhưng nhanh hơn nhiều lần, minh chứng trực tiếp tuyên bố "mean network xấp xỉ tốt averaging".
5. **Dropout trong fine-tune dữ liệu ít (mô phỏng recovery sau pruning):** lấy một mạng nhỏ (hoặc một transformer nhỏ huấn luyện trước) và fine-tune trên một tập dữ liệu rất nhỏ (ví dụ vài trăm mẫu, mô phỏng tập hồi phục hạn chế), so sánh validation loss với dropout $\in \{0, 0.1, 0.3, 0.5\}$ ở các lớp FC/projection. Đo: tại mức dropout nào validation loss thấp nhất, và liệu dropout có giúp khi dữ liệu fine-tune rất ít (kỳ vọng có ích) so với khi dữ liệu fine-tune khá nhiều (kỳ vọng ít ích hoặc không đổi).

10. Tóm tắt công thức / khái niệm

Khái niệm	Giá trị / công thức
Dropout rate lớp ẩn	0.5 (dùng xuyên suốt paper)
Dropout rate đầu vào	thường < 50% (ví dụ 20% trong thí nghiệm MNIST/TIMIT)
Ràng buộc trọng số	chặn L_2-norm bình phương vector trọng số đến mỗi đơn vị ẩn $\leq l$; $l = 15$ cho MNIST
Mean network (test-time)	dùng toàn bộ đơn vị ẩn, giảm một nửa trọng số **ra** của mỗi đơn vị
Tương đương (1 lớp ẩn + softmax)	mean network $\equiv$ trung bình hình học của 2^N phân phối dự đoán
Cập nhật trọng số	$\Delta w_t = p_t \Delta w_{t-1} - (1 - p_t)\epsilon_t \langle \nabla_w L \rangle$, $\epsilon_t = \epsilon_0 f^t$
Tham số lịch (MNIST)	$\epsilon_0 = 10.0$, $f = 0.998$, $p_i = 0.5 \to p_f = 0.99$ trong $T = 500$ epoch
MNIST (FFN chuẩn, không aug)	160 lỗi $\to$ 130 lỗi (dropout ẩn) $\to$ 110 lỗi (+ dropout input)
TIMIT (core test, frame error)	22.7% $\to$ 19.7%

Khái niệm	Giá trị / công thức
CIFAR-10 (không augmentation)	16.6% → 15.6%
ImageNet 2010 (1 mạng đơn)	48.6% → 42.4%
Reuters RCV1-v2 (50 lớp)	31.05% → 29.62%

11. Kết luận ngắn

Paper giới thiệu dropout: ngẫu nhiên loại bỏ 50% đơn vị ẩn ở mỗi mẫu huấn luyện để ngăn các feature detector "đồng thích nghi" quá mức, kết hợp với ràng buộc (không phạt) độ dài trọng số để cho phép learning rate lớn ban đầu. Ở lúc kiểm tra, dùng "mean network" (trọng số ra giảm một nửa) để xấp xỉ rẻ việc lấy trung bình một số rất lớn mạng con, có cơ sở toán học chặt cho trường hợp 1 lớp ẩn + softmax hoặc hồi quy tuyến tính. Thực nghiệm trên 5 bộ dữ liệu khác nhau (MNIST, TIMIT, CIFAR-10, ImageNet, Reuters) đều cho thấy dropout cải thiện rõ rệt so với không dropout, kể cả khi kết hợp với pretraining sinh, và đạt một số kết quả tốt nhất công bố tại thời điểm đó cho các phương pháp không dùng tri thức tiên nghiệm. Dropout ít lợi ích hơn ở các lớp tích chập (đã có ít tham số sẵn do chia sẻ trọng số). Đây là một trong những kỹ thuật regularization có ảnh hưởng lâu dài nhất của deep learning, tuy vậy với các mô hình Transformer pretrain quy mô lớn hiện đại, mức độ cần thiết của dropout đã giảm đi nhiều so với bối cảnh gốc năm 2012.

Nguồn gốc: https://arxiv.org/abs/1207.0580

Bài 07

Deep Residual Learning for Image Recognition (2015)

Tác giả: Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun (Microsoft Research).
Venue / arXiv: arXiv:1512.03385 (v1, 10/12/2015); paper không nêu venue hội nghị trong bản PDF này (được biết đến rộng rãi là CVPR 2016, nhưng đây là thông tin ngoài paper, kiến thức chung).
Nguồn: https://arxiv.org/abs/1512.03385
Chủ đề: kiến trúc ResNet, residual connection (skip connection) giải quyết "degradation problem" khi mạng CNN rất sâu, cho phép huấn luyện mạng tới hơn 1000 tầng.
Vị trí trong lộ trình: Đọc cùng bài 08 về VGG: ResNet dùng mạng plain kiểu VGG để khảo sát khó khăn khi tăng độ sâu.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Từ 2012, các mạng nơ-ron tích chập (CNN, convolutional neural network) sâu đã tạo ra loạt bứt phá trong nhận dạng ảnh (AlexNet 2012, sau đó là các mạng "rất sâu" như VGG với 16-19 tầng, GoogLeNet với 22 tầng). Bằng chứng thực nghiệm cho thấy **độ sâu** (depth, số tầng có trọng số xếp chồng lên nhau) là yếu tố quan trọng: mạng sâu hơn tích hợp được đặc trưng (feature) ở nhiều mức độ trừu tượng (thấp, trung, cao) trong một kiến trúc end-to-end.

Câu hỏi tự nhiên: học một mạng tốt hơn có đơn giản là xếp thêm tầng không? Trở ngại đầu tiên là **vanishing/exploding gradient** (đạo hàm lan truyền ngược về các tầng đầu bị triệt tiêu hoặc bùng nổ theo cấp số nhân do nhân liên tiếp qua nhiều tầng), nhưng vấn đề này phần lớn đã được giải quyết bằng cách khởi tạo trọng số chuẩn hoá (normalized initialization) và các tầng chuẩn hoá trung gian như **Batch Normalization (BN)**, giúp mạng vài chục tầng có thể bắt đầu hội tụ với SGD + backpropagation.

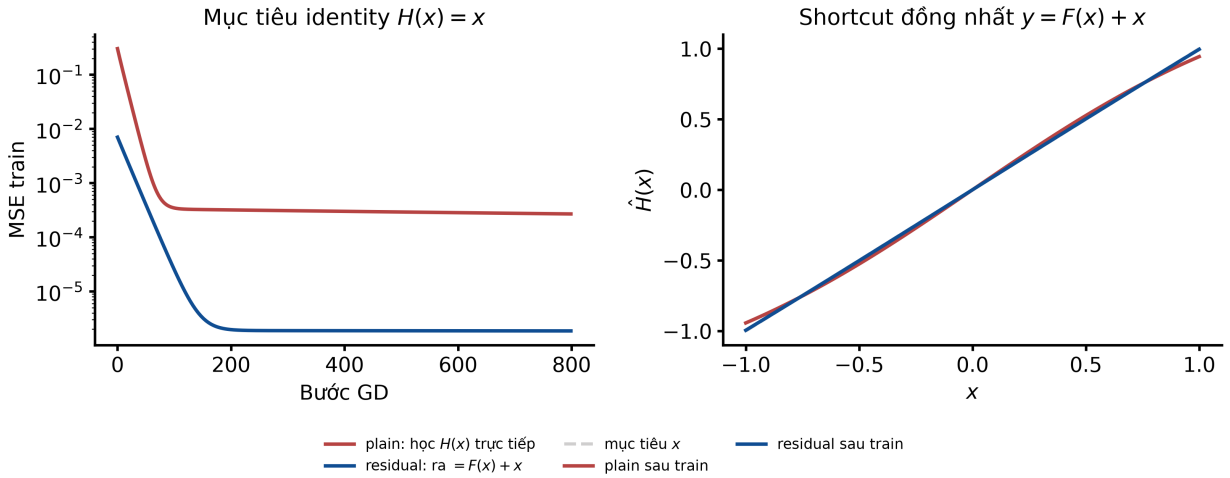
Nhưng khi mạng sâu hơn **đã** hội tụ được, một hiện tượng khác xuất hiện: **degradation problem** (bài toán suy giảm). Khi tăng độ sâu, độ chính xác tăng tới một điểm rồi **giảm nhanh**, và quan trọng là điều này **không phải do overfitting**: mạng sâu hơn lại có **training error cao hơn** mạng nông hơn (lỗi ngay trên tập huấn luyện, chưa nói đến tập test). Hình 1 của bài báo minh họa: mạng "plain" (không có residual) 56 tầng có training error và lỗi kiểm tra cao hơn mạng plain 20 tầng trên CIFAR-10.

Đây là nghịch lý: về lý thuyết, một mạng sâu hơn luôn có thể đạt lỗi huấn luyện **không cao hơn** mạng nông hơn, vì ta có thể xây dựng nghiệm bằng tay: các tầng thêm vào chỉ cần học **ánh xạ đồng nhất** (identity mapping, $f(x) = x$), còn các tầng còn lại sao chép nguyên mạng nông đã học. Nghiệm này tồn tại về lý thuyết, nhưng thực nghiệm cho thấy các bộ giải (solver) hiện tại (SGD) **không tìm được** nghiệm tốt bằng hoặc tốt hơn nghiệm này trong thời gian khả thi. Bài báo đặt câu hỏi: làm sao giúp bộ giải dễ học ánh xạ đồng nhất (hoặc gần đồng nhất) hơn khi cần?

Khái niệm cần nắm trước khi đọc tiếp: **feature map** là output (dạng lưới không gian x số kênh) của một tầng tích chập; **stride** là bước nhảy khi trượt filter (stride 2 làm giảm một nửa kích thước không gian); **FLOPs** (floating point operations) đo lượng tính toán, ở đây tính dạng "multiply-adds"; **top-1/top-5 error** trên ImageNet là tỉ lệ ảnh mà nhãn đúng không phải là dự đoán hạng 1 (top-1) hoặc không nằm trong 5 dự đoán có điểm cao nhất (top-5); **ensemble** là gộp dự đoán của nhiều mô hình (ví dụ trung bình điểm softmax) để tăng độ chính xác; **mAP** (mean Average Precision) là chỉ số cho bài toán phát hiện vật thể (object detection), trung bình độ chính xác trên nhiều lớp vật thể.

2. Ý tưởng chính

Thay vì để các tầng xếp chồng học trực tiếp một hàm mục tiêu $H(x)$ (ánh xạ từ đầu vào sang đầu ra mong muốn của khối đó), tác giả để các tầng học hàm **residual** (phần dư) $F(x) := H(x) - x$, rồi tái tạo lại hàm gốc bằng $F(x) + x$. Về mặt biểu diễn, hai dạng này "tương đương" (nếu nhiều tầng phi tuyến có thể xấp xỉ $H(x)$ thì cũng có thể xấp xỉ $H(x) - x$), nhưng tác giả giả thuyết rằng **độ dễ học** (ease of optimization) khác nhau: nếu ánh xạ đồng nhất là tối ưu hoặc gần tối ưu cho một khối, thì với công thức residual, bộ giải chỉ cần đẩy trọng số của F về 0 (dễ) thay vì phải học cả một ánh xạ đồng nhất bằng nhiều tầng phi tuyến (khó hơn, theo quan sát thực nghiệm).



Học identity: residual $y = F(x) + x$ hội tụ MSE thấp hơn plain MLP (minh hoạ, tự tính)

Về mặt kiến trúc, điều này được thực hiện bằng **shortcut connection** (kết nối tắt, còn gọi là skip connection): đầu ra của khối là $y = F(x) + x$, cộng trực tiếp đầu vào x vào đầu ra của vài tầng tích chập liên tiếp (Hình 2 của bài báo). Khi dùng **ánh xạ đồng nhất** cho shortcut (không nhân ma trận, không thêm tham số, không thêm FLOPs đáng kể), ta có một "**mạng residual**" (ResNet) có **cùng số tham số, cùng độ sâu, cùng độ rộng** với mạng "plain" tương ứng (mạng không có shortcut), nên có thể so sánh công bằng.

Kết quả chính mà ý tưởng này mang lại: mạng residual **dễ tối ưu hơn** mạng plain cùng cấu hình (ResNet sâu hơn luôn tốt hơn hoặc bằng ResNet nông hơn trong thực nghiệm của bài báo, còn mạng plain sâu hơn có thể **tệ hơn** mạng plain nông hơn), và mạng residual **tận dụng được độ sâu** để tăng độ chính xác liên tục, kể cả ở độ sâu 152 tầng (vẫn có độ phức tạp tính toán thấp hơn VGG) hay thậm chí hơn 1000 tầng (thử nghiệm trên CIFAR-10).

3. Phương pháp chi tiết

3.1 Khối residual (building block)

Với x là đầu vào của một vài tầng xếp chồng (không nhất thiết là toàn mạng), khối residual được định nghĩa (phương trình 1 của paper):

$$y = F(x, \{W_i\}) + x$$

Khi hai vế không cùng chiều (ví dụ do đổi số kênh), dùng phép chiếu tuyến tính W_s qua shortcut (phương trình 2):

$$y = F(x, \{W_i\}) + W_s x$$

Với khối 2 tầng ở Hình 2, $F = W_2\,\sigma(W_1 x)$ với σ là ReLU; phép cộng $F(x) + x$ được thực hiện trước phi tuyến thứ hai (tức là $\sigma(y)$ áp dụng sau khi cộng). Shortcut đồng nhất **không thêm tham số, không thêm FLOPs đáng kể** (ngoài phép cộng element-wise), nên khối residual và khối plain tương ứng có đúng cùng số tham số, độ sâu, độ rộng, chi phí tính toán.

3.2 Hai kiến trúc minh họa cho ImageNet (Hình 3, Mục 3.3)

- **Plain network** (mạng nền, không shortcut): lấy cảm hứng từ triết lý VGG (toàn bộ filter 3x3), theo 2 quy tắc thiết kế: (i) cùng kích thước feature map thì cùng số filter; (ii) khi kích thước feature map giảm một nửa (qua tầng conv có stride 2) thì số filter tăng gấp đôi, để giữ "độ phức tạp tính toán mỗi tầng" tương đối ổn định. Mạng kết thúc bằng global average pooling + 1 tầng fully-connected (FC) 1000 chiều + softmax. Ví dụ plain-34 có 34 tầng có trọng số.
- **Residual network**: chính là plain network nhưng chèn thêm shortcut sau mỗi cặp 2 tầng 3x3. Khi số kênh tăng (ở ranh giới giữa các "stage"), có hai lựa chọn: (A) shortcut vẫn là ánh xạ đồng nhất, đệm thêm 0 vào các chiều tăng thêm (không thêm tham số); (B) dùng shortcut chiếu (projection, qua conv 1x1) để khớp chiều, chỉ dùng khi cần tăng chiều; nếu dùng projection cho **mọi** shortcut (cả khi không cần tăng chiều) thì gọi là tuỳ chọn (C).

Plain-34 có **3.6 tỷ FLOPs**, so với VGG-19 là **19.6 tỷ FLOPs**: tỉ lệ 3.6/19.6 ≈ 0.1837, khớp với con số paper nêu là "chỉ 18%" (tự tính: $3.6/19.6 = 0.1837 \approx 18\%$). ResNet-152 có 11.3 tỷ FLOPs, vẫn thấp hơn VGG-16/19 (15.3/19.6 tỷ FLOPs), dù **sâu gấp khoảng 8 lần** VGG (tự tính: $152/19 = 8.0$, đúng với câu paper viết "8× deeper than VGG nets").

3.3 Kiến trúc bottleneck cho mạng rất sâu (Hình 5, Mục 3.3)

Với các mạng 50/101/152 tầng, để giảm thời gian huấn luyện, mỗi khối residual đổi từ 2 tầng 3x3 thành 3 tầng dạng **bottleneck**: $1 \times 1 \to 3 \times 3 \to 1 \times 1$, trong đó hai tầng 1×1 có vai trò **giảm rồi khôi phục** số kênh, còn tầng 3×3 hoạt động ở số kênh đã bị "ép nhỏ" (bottleneck). Theo Bảng 1 của paper, tại stage `conv3_x` (kích thước feature map 28×28) của ResNet-50, mỗi khối "ổn định" (không phải khối đầu stage phải đổi chiều) có cấu trúc: đầu vào 512 kênh $\to$ $1 \times 1, 128 \to 3 \times 3, 128 \to 1 \times 1, 512$.

Ví dụ số (tự tính), minh họa vì sao bottleneck "kinh tế" hơn: dùng đúng cấu hình bottleneck ở trên (bỏ bias):

$$\#W_{\text{bottleneck}} = \underbrace{512 \times 128}_{1\times1\text{ giảm}} + \underbrace{3 \times 3 \times 128 \times 128}_{3\times3} + \underbrace{128 \times 512}_{1\times1\text{ khôi phục}} = 65536 + 147456 + 65536 = 278528$$

So sánh với một khối giả định "2 tầng 3×3" (kiểu khối cơ bản dùng ở ResNet-18/34) nhưng hoạt động ở **cùng độ rộng 512** (giả định minh họa của người ghi chú để so chi phí, **không phải** cấu hình thật mà paper dùng cho ResNet-34, vì ResNet-34 dùng độ rộng nhỏ hơn 64/128/256/512 tuỳ stage, không phải toàn bộ 512):

$$\#W_{\text{2 tầng 3×3, 512}} = 2 \times (3 \times 3 \times 512 \times 512) = 2 \times 2.359.296 = 4.718.592$$

Tỉ lệ: $278528/4718592 \approx 0.059$, tức khối bottleneck chỉ tốn khoảng **5.9%** số tham số (tự tính; tức rẻ hơn khoảng 17 lần) so với khối 2-tầng-3x3 giả định ở cùng độ rộng. Đây là lý do thực tế (chứ không phải vì hiệu quả về độ chính xác) mà paper chọn bottleneck cho các mạng sâu: ghi chú ở chân trang 4 của paper nói rõ "**deeper non-bottleneck ResNets... also gain accuracy from increased depth... but are not as economical**", và paper cũng nhấn mạnh shortcut đồng nhất (không tham số) đặc biệt quan trọng cho bottleneck: nếu đổi shortcut đồng nhất thành shortcut chiếu ở khối bottleneck, độ phức tạp thời gian và số tham số của shortcut đó sẽ **gấp đôi** (vì shortcut phải nối hai đầu có số kênh cao).

3.4 Thiết lập huấn luyện

ImageNet: ảnh resize với cạnh ngắn hơn lấy ngẫu nhiên trong $[256, 480]$ (scale augmentation), crop 224×224 ngẫu nhiên (hoặc lật ngang), trừ pixel-mean, color augmentation theo AlexNet. BN áp dụng ngay sau mỗi conv, trước phi tuyến. Khởi tạo trọng số theo He et al. (2015, PReLU paper). SGD, mini-batch 256, learning rate khởi đầu 0.1, chia 10 khi lỗi chững lại (plateau), huấn luyện tới 60×10^4 iteration, weight decay 0.0001, momentum 0.9, **không dùng dropout**. Test: 10-crop (so sánh) hoặc dạng fully-convolutional + đa tỉ lệ $\{224, 256, 384, 480, 640\}$ (để lấy kết quả tốt nhất).

CIFAR-10: input 32×32, tầng đầu 3×3 conv, rồi $6n$ tầng 3×3 trên 3 kích thước feature map $\{32, 16, 8\}$ (mỗi kích thước $2n$ tầng), số filter $\{16, 32, 64\}$ tương ứng, tổng $6n + 2$ tầng có trọng số. Với $n = \{3, 5, 7, 9\}$ cho mạng 20/32/44/56 tầng; $n = 18$ cho 110 tầng; $n = 200$ cho 1202 tầng. Batch 128 trên 2 GPU, lr 0.1 (riêng 110 tầng phải "warm-up" bằng lr 0.01 khoảng 400 iteration đầu vì 0.1 quá lớn để bắt đầu hội tụ, sau đó quay lại 0.1), chia 10 tại 32k và 48k iteration, dừng ở 64k iteration; weight decay 0.0001, momentum 0.9, không dropout; data augmentation: đệm 4 pixel mỗi cạnh rồi crop ngẫu nhiên 32×32 (hoặc lật ngang).

4. Thí nghiệm và kết quả chính

Degradation problem được xác nhận (Bảng 2, ImageNet): mạng plain 34 tầng có top-1 error **28.54%**, **cao hơn** mạng plain 18 tầng (**27.94%**), dù không gian nghiệm của plain-18 là tập con của plain-34 (nên về lý thuyết plain-34 phải không tệ hơn). Khi thêm shortcut (ResNet): ResNet-34 đạt **25.03%**, **tốt hơn** ResNet-18 (**27.88%**) 2.8 điểm phần trăm, và tốt hơn plain-34 tới 3.51 điểm (tự tính: $28.54 - 25.03 = 3.51$, khớp với câu paper viết "giảm top-1 error 3.5%"). Ngoài ra ResNet-18 **hội tụ nhanh hơn** plain-18 ở giai đoạn đầu, dù độ chính xác cuối cùng tương đương (mạng "không quá sâu" thì SGD vẫn tìm được nghiệm plain tốt).

Identity vs. projection shortcut (Bảng 3): 3 tuỳ chọn A (đệm 0, không tham số), B (chiếu khi cần tăng chiều), C (chiếu toàn bộ) đều tốt hơn plain rõ rệt; B nhích hơn A (vì các chiều đệm 0 ở A không học residual thật); C nhích hơn B nhưng paper quy cho phần tham số thêm từ 13 shortcut chiếu, kết luận **khác biệt A/B/C nhỏ**, chiếu không phải yếu tố cốt lõi giải quyết degradation (nên các mạng sâu hơn trong phần sau dùng B để đỡ tốn bộ nhớ/tính toán).

Mạng bottleneck sâu hơn (Bảng 3, 4): ResNet-50 (3.8 tỷ FLOPs): top-1/top-5 = 22.85%/6.71%; ResNet-101 (7.6 tỷ FLOPs): 21.75%/6.05%; ResNet-152 (11.3 tỷ FLOPs): 21.43%/5.71% trên validation; trên **test set đơn mô hình** ResNet-152 đạt top-5 **4.49%** (Bảng 4), và **ensemble 6 mô hình** (trong đó 2 là 152 tầng) đạt **3.57%** top-5 trên test set (Bảng 5), đoạt **giải nhất ILSVRC 2015** hạng mục classification.

CIFAR-10 (Bảng 6): ResNet-110 đạt lỗi test tốt nhất **6.43%** (trung bình 5 lần chạy: 6.61% ± 0.16%), với chỉ **1.7M tham số**, ít hơn FitNet (2.5M, 8.39%) và Highway 19 tầng (2.3M, 7.54%/7.72±0.16%). ResNet-1202 (19.4M tham số) đạt training error **dưới 0.1%** nhưng test error **7.93%**, **tệ hơn** ResNet-110, dù cả hai có training error tương đương, tác giả quy cho **overfitting** (mạng 1202 tầng "có thể quá lớn một cách không cần thiết" cho tập CIFAR-10 nhỏ, và paper chủ đích không dùng maxout/dropout để giữ trọng tâm vào bài toán tối ưu hoá).

Object detection/localization (dùng backbone ResNet thay VGG trong Faster R-CNN): trên PASCAL VOC2007 (huấn luyện "07+12"), thay VGG-16 (mAP 73.2%) bằng ResNet-101 cho mAP **76.4%** (Bảng 7); trên COCO, VGG-16 đạt mAP@[.5,.95] = 21.2% còn ResNet-101 đạt **27.2%**, tăng tuyệt đối 6.0 điểm, tương đương **28% cải thiện tương đối** (tự tính: $6.0/21.2 = 0.283 \approx 28\%$, khớp với câu paper viết "28% relative improvement", cũng là số được nêu lại ở phần tóm tắt/abstract). Sau khi thêm box refinement + global context + multi-scale testing, mô hình đơn đạt mAP@.5 = 55.7%, @[.5,.95] = 34.9% (test-dev), ensemble đạt mAP@.5 = 59.0%, @[.5,.95] = 37.4%, giúp đoạt giải nhất COCO detection 2015. Trên VOC2007/2012 test (sau khi fine-tune từ mô hình COCO), đạt mAP 85.6%/83.8% (Bảng 10, 11), hơn 10 điểm so với kết quả SOTA trước đó trên VOC2012. ImageNet detection: mô hình đơn đạt mAP 58.8% (test), ensemble 62.1%, đoạt giải nhất, vượt vị trí thứ 2 tới 8.5 điểm tuyệt đối. ImageNet localization: mô hình đơn (RPN+RCNN)

đạt top-5 localization error 10.6% (val), ensemble 9.0% (test), **giảm 64% tương đối** so với kết quả ILSVRC 2014 tốt nhất (tự tính theo paper: so với VGG 25.3% ở Bảng 14, $(25.3 - 9.0)/25.3 \approx 0.644 \approx 64\%$, khớp con số paper nêu), đoạt giải nhất.

5. Phân tích / ablation

- **Độ lớn phản hồi của tầng (Hình 7):** đo độ lệch chuẩn (std) của output mỗi tầng 3×3 (sau BN, trước phi tuyến). ResNet có std **nhỏ hơn** mạng plain tương ứng, ủng hộ giả thuyết rằng hàm residual $F(x)$ nói chung gần 0 hơn là một hàm không-residual bất kỳ. Mạng ResNet **sâu hơn** (110 so với 56, 20 tầng) có độ lớn phản hồi càng nhỏ hơn mỗi tầng riêng lẻ, tức khi có nhiều tầng hơn, mỗi tầng chỉ "chỉnh" tín hiệu một lượng nhỏ hơn.
- **Thử huấn luyện lâu hơn (chú thích chân trang):** huấn luyện mạng plain với 3 lần số iteration vẫn **không** hết degradation, cho thấy đây không đơn thuần là vấn đề "chưa hội tụ đủ lâu".
- **Loại trừ nguyên nhân vanishing gradient:** mạng plain dùng BN nên tín hiệu forward có variance khác 0 và gradient backward có norm "khỏe mạnh", vậy degradation không do gradient biến mất hoàn toàn theo nghĩa cổ điển. Tác giả **suy đoán** (chưa chứng minh) nguyên nhân có thể là tốc độ hội tụ giảm theo cấp số nhân của mạng plain sâu, và nói rõ "lý do khó khăn tối ưu hoá này sẽ được nghiên cứu trong tương lai".
- **>1000 tầng (ResNet-1202):** không gặp khó khăn tối ưu hoá (train error <0.1%), nhưng test error tệ hơn ResNet-110 dù train error tương đương, tác giả gán cho overfitting trên tập nhỏ CIFAR-10, không phải lại một dạng degradation khác.

6. Giới hạn

Do tác giả nêu:

- Không có lý thuyết đầy đủ giải thích tại sao mạng plain sâu lại khó tối ưu hoá hơn ("lý do sẽ được nghiên cứu trong tương lai"); mới loại trừ được giả thuyết vanishing gradient bằng quan sát gián tiếp (norm gradient, BN), chưa chứng minh nguyên nhân thật.
- Giả thuyết nền tảng rằng nhiều tầng phi tuyến có thể xấp xỉ asymptotic hàm mong muốn "vẫn là câu hỏi mở" (chú thích 2 của paper, dẫn Montúfar et al. 2014).
- ResNet-1202 trên CIFAR-10 cho test error tệ hơn 110 tầng; paper không áp dụng regularization mạnh (maxout/dropout) để giữ trọng tâm vào vấn đề tối ưu hoá, và nói "kết hợp với regularization mạnh hơn có thể cải thiện kết quả, chúng tôi sẽ nghiên cứu trong tương lai" (nghĩa là bản thân paper **chưa** giải quyết triệt để trường hợp mạng cực sâu trên tập nhỏ).
- Phần multi-scale testing cho detection (Phụ lục B) chỉ làm ở bước Fast R-CNN, "chưa làm multi-scale training vì thời gian hạn chế" (tác giả tự nhận).

Nhận xét:

- Toàn bộ thí nghiệm là về **CNN cho ảnh** (classification/detection/localization); paper không thử nghiệm trên kiến trúc tuần tự (RNN) hay Transformer, nên các khẳng định (ví dụ về độ dễ tối ưu hoá của residual) chỉ được paper này kiểm chứng trong phạm vi CNN; việc residual connection trở thành chuẩn mực cho Transformer sau này là một phát triển về sau (không phải điều paper này chứng minh).
- Với các mạng bottleneck (50/101/152 tầng), số tham số/FLOPs **không hoàn toàn bằng nhau** giữa các mô hình so sánh (khác plain-34 vs ResNet-34 ở Bảng 2 là so sánh cùng tham số); nên một phần cải thiện của ResNet-50/101/152 so với ResNet-34 có thể đến từ tăng dung lượng mô hình, không chỉ từ residual connection tự thân. Paper không tách biệt rõ hai yếu tố này ở các mạng bottleneck.
- Các siêu tham số (lr 0.1, chia 10 theo plateau, batch 256, weight decay 0.0001) là thực hành phổ biến năm 2015, không nhất thiết còn là best practice hiện nay (ví dụ lịch lr cosine, warmup dài hơn, AdamW phổ biến hơn SGD cho nhiều bài toán khác).

7. Liên hệ với các bài khác

- **ml basics 08 (VGG)** (file `08_2014_vgg.md`, đọc ngay sau bài này): là mạng plain-net làm mốc so sánh trực tiếp trong paper (Hình 3, Bảng 1, Bảng 3-5 đều có cột VGG); công thức đếm tham số/FLOPs của VGG dùng lại ở đây để tính tỉ lệ 18% và "8× deeper".
- **pruning content/04_2016_pruning_filters_convnets.md**: các phương pháp pruning filter trong bài đó thử nghiệm trực tiếp trên VGG và cả các mạng kiểu ResNet; hiểu cấu trúc khối residual ở đây giúp hiểu tại sao cắt filter trong một khối residual phải tính tới ràng buộc chiều giữa $F(x)$ và đường tắt x.
- **llm content/29_2020_vit.md**: Vision Transformer thay tích chập bằng attention trên patch ảnh, nhưng **vẫn giữ residual connection** quanh mỗi khối attention/MLP; đây là minh chứng cho việc ý tưởng cốt lõi của ResNet (cộng tắt x vào output khối) tồn tại qua sự thay đổi hoàn toàn kiểu tầng bên trong (từ conv sang attention).
- **llm content/09_2020_pre_ln_transformer.md**: bài đó bàn về vị trí LayerNorm quanh residual connection trong Transformer (Pre-LN và tính ổn định khi huấn luyện mạng rất sâu); đây là phần tiếp nối hiện đại của chính câu hỏi "làm sao huấn luyện mạng rất sâu ổn định" mà bài ResNet này đặt ra lần đầu cho CNN, dù cơ chế chuẩn hoá cụ thể khác nhau (BN theo batch vs LayerNorm theo feature).
- **distillation content/01_2014_do_deep_nets_need_to_be_deep.md**: bài đó hỏi liệu mạng nông có thể đạt dung lượng tương đương mạng sâu nếu được "dạy" đúng cách (distillation); bài ResNet này cho góc nhìn ngược: với đúng kiến trúc (residual), mạng sâu học **dễ hơn** mạng nông cùng dung lượng biểu diễn, một đối chiếu thú vị về vai trò của độ sâu.

8. Gợi ý áp dụng

Residual connection cung cấp một cách thiết kế mạng sâu thuận lợi hơn cho tối ưu hóa; bottleneck giúp thay đổi chi phí của từng khối. Khi thiết kế kiến trúc, cần đo cả chất lượng lẫn độ trễ thực tế, thay vì chỉ so số tầng hoặc FLOPs.

9. Thí nghiệm gợi ý

1. **Tái tạo degradation problem ở quy mô nhỏ:** trên CIFAR-10 (hoặc một tập ảnh nhỏ tương tự, chạy được), huấn luyện 2 mạng "plain" (không shortcut) 20 tầng và 56 tầng theo đúng công thức $6n+2$ của paper (Mục 3.4), đo training/test error theo từng epoch. Kỳ vọng: mạng 56 tầng có training error **cao hơn** mạng 20 tầng (tái hiện Hình 1); nếu không thấy hiện tượng này, kiểm tra lại BN/khởi tạo trước khi kết luận.
2. **So sánh plain vs ResNet cùng tham số:** thêm shortcut đồng nhất vào đúng 2 mạng ở thí nghiệm 1 (giữ nguyên số tầng, số kênh, tức đúng tinh thần Bảng 2), đo lại training/test error. Kỳ vọng: ResNet-56 tốt hơn hoặc bằng ResNet-20, khác hẳn trường hợp plain; đo thêm thời gian train mỗi epoch để thấy chi phí thêm của phép cộng residual là không đáng kể.
3. **Đo độ lớn phản hồi theo tầng (tái hiện Hình 7 ở quy mô nhỏ):** với 2 mạng đã train ở thí nghiệm 2, tính độ lệch chuẩn output mỗi tầng 3×3 (trước ReLU) trên một batch validation, vẽ theo thứ tự tầng. Kỳ vọng: các tầng của ResNet có std nhỏ hơn plain tương ứng. Đây là công cụ chẩn đoán có thể thử áp dụng (suy luận) để tìm khối "gần như không làm gì" trong mạng trước khi thử loại bỏ khối.
4. **"Xoá" residual sau khi train (mô phỏng cắt tầng):** với ResNet đã train ở thí nghiệm 2, với từng khối residual, thử đặt $F(x)=0$ (chỉ giữ đường tắt x) và đo tụt accuracy trên tập validation cho từng khối riêng lẻ. Khối nào tụt ít nhất là khối "ít quan trọng nhất" theo tiêu chí này. Đây là một phiên bản đồ chơi, quy mô nhỏ của các phương pháp cắt tầng kiểu ShortGPT cho LLM, **không có trong paper này**, chỉ là một gợi ý thực nghiệm.
5. **So sánh tuỳ chọn A/B/C cho shortcut ở quy mô nhỏ:** trên cùng một ResNet nhỏ, thử 3 biến thể xử lý tăng chiều (đệm 0, chiếu một phần, chiếu toàn bộ) như Bảng 3, đo accuracy và số tham số. Kỳ vọng: khác biệt nhỏ giữa 3 phương án, khớp với kết luận paper rằng phép chiếu không phải yếu tố cốt lõi.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Khối residual	$y = F(x, \{W_i\}) + x$ (Eq.1)
Khối residual khi đổi chiều	$y = F(x, \{W_i\}) + W_s x$ (Eq.2)
Quy tắc thiết kế plain net	cùng kích thước feature map → cùng số filter; giảm một nửa kích thước → tăng gấp đôi filter
FLOPs plain-34 vs VGG-19	3.6×10^9 vs 19.6×10^9, tỉ lệ ≈ 18% (tự tính)
Độ sâu ResNet-152 vs VGG-19	$152/19 = 8.0$ lần (tự tính, khớp "8× deeper")
Khối bottleneck	$1 \times 1 \to 3 \times 3 \to 1 \times 1$ (giảm rồi khôi phục số kênh)
Tham số bottleneck (ví dụ 512→128→128→512)	$65536 + 147456 + 65536 = 278528$ (tự tính)
Top-1 err plain-34 vs ResNet-34	28.54% vs 25.03%, giảm 3.51 điểm (tự tính)
Top-5 test tốt nhất (ensemble, ILSVRC15)	3.57%
ResNet-110 CIFAR-10	1.7M tham số, lỗi test tốt nhất 6.43%
COCO mAP@[.5,.95]: VGG-16 → ResNet-101	21.2% → 27.2%, cải thiện tương đối ≈ 28% (tự tính)
ImageNet localization: VGG → ResNet (ensemble)	25.3% → 9.0%, giảm tương đối ≈ 64% (tự tính)

11. Kết luận ngắn

ResNet giải quyết "degradation problem" của mạng CNN rất sâu bằng một thay đổi kiến trúc đơn giản: để các khối học hàm residual $F(x)$ và cộng thẳng đầu vào x qua shortcut connection, giúp bộ giải SGD dễ xấp xỉ ánh xạ đồng nhất khi cần. Kết quả là mạng có thể sâu tới 152 tầng (vẫn rẻ hơn VGG) hoặc hơn 1000 tầng (CIFAR-10) mà vẫn tối ưu hoá tốt, mang lại cải thiện lớn và đồng bộ trên classification, detection, localization (nhiều giải nhất ILSVRC & COCO 2015). Hạn chế lớn nhất do chính tác giả thừa nhận là **chưa có lý thuyết đầy đủ** giải thích cơ chế degradation, chỉ có bằng chứng thực nghiệm và suy đoán. Ý tưởng residual connection sau đó trở thành thành phần gần như bắt buộc trong mọi kiến trúc deep learning hiện đại, kể cả ngoài phạm vi CNN mà bài báo này khảo sát.

Nguồn gốc: https://arxiv.org/abs/1512.03385

Bài 08

Very Deep Convolutional Networks for Large-Scale Image Recognition (2014)

Tác giả: Karen Simonyan, Andrew Zisserman (Visual Geometry Group, Dept. Engineering Science, University of Oxford).
Venue / arXiv: arXiv:1409.1556 (v1, 04/09/2014; bản đọc là v6, 10/04/2015); paper tự ghi "Published as a conference paper at ICLR 2015".
Nguồn: https://arxiv.org/abs/1409.1556
Chủ đề: VGGNet, nghiên cứu hệ thống ảnh hưởng của độ sâu ConvNet khi chỉ dùng filter tích chập rất nhỏ (3x3), tạo ra các cấu hình 11-19 tầng trọng số (A đến E, trong đó D/E là VGG-16/VGG-19 nổi tiếng).
Vị trí trong lộ trình: Đọc cùng bài 07 về ResNet để đối chiếu mạng tích chập plain với mạng dùng residual connection.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Sau thành công của AlexNet (Krizhevsky et al., 2012) trên ImageNet, nhiều nhóm tìm cách cải thiện kiến trúc ConvNet cho ILSVRC (ImageNet Large-Scale Visual Recognition Challenge). Các cải tiến ở ILSVRC-2013 (Zeiler & Fergus; Sermanet et al., tức OverFeat) tập trung vào việc dùng **cửa sổ tiếp nhận (receptive field) nhỏ hơn và stride nhỏ hơn ở tầng tích chập đầu tiên**, hoặc huấn luyện và kiểm tra "dense" trên toàn ảnh ở nhiều tỉ lệ (scale).

Bài báo này chọn khảo sát một khía cạnh khác của thiết kế ConvNet: **độ sâu**. Ý tưởng là **cố định** mọi yếu tố khác của kiến trúc (loại filter, tầng pooling, tầng fully-connected...) và chỉ tăng dần số tầng tích chập, điều này khả thi vì dùng filter rất nhỏ (3×3) ở mọi tầng. Khái niệm cần nắm: **receptive field** (vùng ảnh gốc mà một neuron ở tầng sau "nhìn thấy", càng nhiều tầng chồng lên thì vùng này càng lớn dù filter mỗi tầng nhỏ); **max-pooling** (lấy giá trị lớn nhất trong một vùng 2×2 để giảm kích thước không gian, ở đây stride 2); **fully-connected (FC) layer** (tầng kết nối đầy đủ, mỗi neuron nhận input từ toàn bộ neuron tầng trước, khác với tầng tích chập chỉ nhìn một vùng cục bộ); **top-1/top-5 error** như đã định nghĩa ở các bài khác trong lộ trình (ml basics 07).

2. Ý tưởng chính

Dùng duy nhất filter 3×3 (filter nhỏ nhất còn giữ được khái niệm "trên/dưới/trái/phải/giữa") xếp chồng nhiều lớp, thay cho các filter lớn hơn (11×11 ở AlexNet, 7×7 ở OverFeat/Zeiler-Fergus) ở các tầng đầu. Lý do: **nhiều tầng filter nhỏ** cho **cùng receptive field** với **ít tham số hơn** và **nhiều phi tuyến (ReLU) hơn** so với **một tầng filter lớn**, nên vừa tăng khả năng biểu diễn (nhiều phi tuyến) vừa có tác dụng "chính quy hoá" (regularization) tự nhiên nhờ giảm tham số.

Từ nguyên tắc đó, tác giả xây dựng 6 cấu hình (A, A-LRN, B, C, D, E) tăng dần độ sâu từ 11 đến 19 tầng trọng số, giữ nguyên mọi thứ khác (loại filter, số tầng pooling, cấu trúc 3 tầng FC cuối). Kết quả: độ chính xác tăng đơn điệu theo độ sâu (saturate ở khoảng 19 tầng cho tập dữ liệu này), hai cấu hình sâu nhất D (16 tầng, "VGG-16") và E (19 tầng, "VGG-19") đạt độ chính xác cạnh tranh với/vượt kỷ lục đương thời (GoogLeNet), và đặc biệt, các đặc trưng (feature) học được **tổng quát hoá (transfer) rất tốt** sang nhiều tập dữ liệu/bài toán khác mà không cần fine-tune, chỉ cần huấn luyện một SVM tuyến tính trên đặc trưng cố định.

3. Phương pháp chi tiết

3.1 Kiến trúc chung (Mục 2.1)

Input cố định 224×224 RGB, chỉ tiền xử lý bằng trừ giá trị RGB trung bình (tính trên tập train). Chuỗi tầng tích chập dùng receptive field 3×3 (filter nhỏ nhất còn giữ được khái niệm trên/dưới/trái/phải/giữa trong paper), stride 1, padding 1 (giữ nguyên kích thước không gian sau conv). Một cấu hình (C) có thêm filter 1×1 (một phép biến đổi tuyến tính theo kênh, cộng thêm phi tuyến ReLU). Có 5 tầng max-pooling 2×2 stride 2 nằm rải giữa các tầng conv (không phải sau mọi tầng conv). Sau chuỗi conv là 3 tầng FC: 2 tầng 4096 kênh, tầng cuối 1000 kênh (số lớp ILSVRC) + softmax. Mọi tầng ẩn dùng ReLU. Chỉ một cấu hình (A-LRN) có thêm Local Response Normalisation (LRN); các cấu hình khác không dùng vì (sẽ thấy ở Mục 4) LRN không cải thiện mà còn tốn thêm bộ nhớ/thời gian tính toán.

3.2 Các cấu hình A-E (Bảng 1, 2)

Cấu hình	Số tầng trọng số	Đặc điểm riêng	Số tham số (Bảng 2)
A	11 (8 conv + 3 FC)	nông nhất	133M
A-LRN	11	như A + LRN sau tầng conv đầu	133M
B	13	thêm 2 conv3-64 so với A	133M
C	16	thêm 3 tầng conv1-256/512/512 so với B	134M
D	16	như C nhưng conv1 → conv3 (chính là "VGG-16")	138M
E	19	thêm 3 conv3 nữa so với D (chính là "VGG-19")	144M

Độ rộng (số kênh) bắt đầu từ 64 ở tầng đầu, **tăng gấp đôi sau mỗi lần max-pool**, tới tối đa 512. Dù độ sâu lớn, tổng số tham số (133-144M) **không lớn hơn** một mạng nông hơn nhưng dùng filter/receptive field lớn hơn (paper dẫn chứng OverFeat có 144M tham số dù nông hơn).

3.3 Vì sao nhiều filter 3x3 tốt hơn ít filter lớn (Mục 2.3)

Một chồng **2 tầng** 3×3 (không pooling ở giữa) có receptive field hiệu dụng 5×5; chồng **3 tầng** 3×3 có receptive field hiệu dụng 7×7 (vì mỗi tầng conv thêm "nở" biên 1 pixel mỗi phía, 3 tầng liên tiếp nở tổng 3 pixel mỗi phía quanh tâm, cho $1 + 2 \times 3 = 7$). Với cùng số kênh vào/ra C, số tham số của chồng 3 tầng 3×3 là:

$$3 \times (3^2 C^2) = 27C^2$$

còn một tầng 7×7 duy nhất có:

$$7^2 C^2 = 49C^2$$

Tỉ lệ: $49/27 \approx 1.8148$, nghĩa là tầng 7×7 đơn tốn khoảng **81% nhiều tham số hơn** (tự tính: $(49-27)/27 = 22/27 \approx 0.8148 \approx 81\%$, khớp đúng con số "81% more" mà paper nêu). Ngoài việc tiết kiệm tham số, chồng 3 tầng còn có **3 lớp phi tuyến ReLU** thay vì 1, làm hàm quyết định "phân biệt được nhiều hơn" (more discriminative) theo cách nói của paper. Tác giả xem đây như một dạng **chính quy hoá ép buộc** (regularisation) lên bộ filter lớn, buộc nó phải phân rã qua các filter 3×3 (có phi tuyến chen giữa).

Riêng filter 1×1 (cấu hình C) không làm tăng receptive field (vẫn nhìn đúng 1 pixel vị trí đó theo mọi kênh), về bản chất là phép chiếu tuyến tính theo kênh, nhưng vẫn **thêm được phi tuyến** (ReLU) mà không ảnh hưởng

receptive field, ý tưởng này paper ghi nhận lấy từ "Network in Network" (Lin et al., 2014).

3.4 Huấn luyện (Mục 3.1)

Tối ưu hoá hàm mất mát multinomial logistic regression (tức cross-entropy nhiều lớp) bằng mini-batch gradient descent + momentum. Batch size 256, momentum 0.9, weight decay (L2 penalty) 5×10^{-4}, **dropout 0.5** ở 2 tầng FC đầu (4096 kênh). Learning rate khởi đầu 10^{-2}, giảm 10 lần khi độ chính xác validation chững lại, giảm tổng 3 lần, dừng sau 370K iteration (74 epoch). Tác giả "suy đoán" (conjecture, chưa chứng minh) rằng dù có nhiều tham số và sâu hơn AlexNet, mạng này cần **ít epoch hơn** để hội tụ nhờ (a) chính quy hoá ngầm từ độ sâu lớn và filter nhỏ, (b) kỹ thuật khởi tạo trước (pre-initialisation) nêu dưới đây.

Khởi tạo: huấn luyện cấu hình A trước (đủ nông để khởi tạo ngẫu nhiên trực tiếp, trọng số lấy từ phân phối chuẩn, mean 0, variance 10^{-2}, bias 0); các mạng sâu hơn khởi tạo 4 tầng conv đầu và 3 tầng FC cuối từ mạng A đã học, các tầng giữa khởi tạo ngẫu nhiên. Tác giả ghi chú rằng **sau khi nộp bài**, họ nhận ra có thể khởi tạo ngẫu nhiên trực tiếp theo phương pháp Glorot & Bengio (2010) mà không cần bước "huấn luyện trước A rồi truyền lại" này, nhưng **không** re-run lại toàn bộ thí nghiệm chính với cách khởi tạo mới này (chỉ là một câu ghi chú thêm trong văn bản).

Tăng cường dữ liệu bằng tỉ lệ (scale jittering): gọi S là cạnh ngắn hơn của ảnh sau khi resize đẳng hướng (isotropic rescaling), từ đó crop 224×224. Hai cách đặt S: (i) cố định ($S = 256$ hoặc $S = 384$, mạng $S = 384$ khởi động từ trọng số mạng $S = 256$ với lr nhỏ hơn 10^{-3}); (ii) **multi-scale training**: mỗi ảnh lấy ngẫu nhiên $S \in [S_{min}, S_{max}] = [256, 512]$ (vì lý do tốc độ, mạng multi-scale được fine-tune toàn bộ tầng từ một mạng $S = 384$ đã huấn luyện, không train multi-scale từ đầu).

3.5 Kiểm tra (Mục 3.2) và cài đặt (Mục 3.3)

Ở test, ảnh được resize đẳng hướng về cạnh ngắn Q (**không nhất thiết bằng** S lúc train). Các tầng FC được **chuyển thành tầng tích chập** (tầng FC đầu → conv 7×7, 2 tầng FC sau → conv 1×1), biến toàn mạng thành "fully-convolutional", áp dụng "dense" lên toàn ảnh (không crop) ở kích thước gốc đã resize, cho ra một bản đồ điểm số theo lớp (class score map) với độ phân giải không gian thay đổi theo kích thước ảnh input, sau đó **sum-pool (trung bình không gian)** để ra vector điểm số cố định chiều. Có thêm trung bình với ảnh lật ngang (horizontal flip). Cách này tránh phải tính lại mạng cho từng crop riêng lẻ như kiểu "multi-crop" của AlexNet/OverFeat, nhưng paper cũng đánh giá riêng multi-crop (150 crop: lưới 5×5 x 2 lật x 3 tỉ lệ) để so sánh.

Cài đặt dựa trên Caffe, có sửa để chạy đa GPU (song song dữ liệu, chia batch cho từng GPU, đồng bộ gradient), đạt speedup 3.75 lần trên hệ 4-GPU so với 1 GPU (NVIDIA Titan Black); một mạng cần **2-3 tuần** huấn luyện.

4. Thí nghiệm và kết quả chính

Dữ liệu: ILSVRC-2012 (1000 lớp; 1.3M ảnh train, 50K ảnh validation, 100K ảnh test không công khai nhãn).

Đánh giá đơn tỉ lệ (Bảng 3): lỗi giảm đơn điệu theo độ sâu: A (29.6%/10.4% top-1/top-5) → E (tốt nhất trong nhóm single-scale, 25.5%/8.0% với S jitter $[256, 512]$, $Q = 384$). A-LRN (29.7%/10.5%) **không tốt hơn** A, xác nhận LRN không giúp ích cho các cấu hình sâu hơn (B-E bỏ LRN). Cấu hình C (16 tầng, có conv1x1) **kém hơn** D (cùng 16 tầng, toàn conv3x3): ở $S = 256$, C đạt 27.3%/9.0% còn D đạt 27.0%/8.8% (số liệu paper, Bảng 3), cho thấy việc có thêm receptive field không gian (D) quan trọng hơn chỉ thêm phi tuyến mà không mở rộng receptive field (C). Riêng việc so B với một "mạng nông" thay mỗi cặp conv3x3 bằng một conv5x5 (cùng receptive field): mạng nông này có top-1 error **cao hơn B khoảng 7%** (đo trên center crop), xác nhận "mạng sâu filter nhỏ thắng mạng nông filter lớn" ở cùng receptive field. Scale jittering lúc train ($S \in [256, 512]$) cho kết quả tốt hơn rõ rệt so với S cố định, dù test chỉ dùng 1 tỉ lệ.

Đánh giá đa tỉ lệ (Bảng 4): test ở 3 giá trị Q quanh S (hoặc $\{S_{min}, (S_{min} + S_{max})/2, S_{max}\}$ với mạng jitter) cho kết quả tốt hơn test đơn tỉ lệ. Kết quả **single-network tốt nhất**: 24.8%/7.5% top-1/top-5 validation (D

hoặc E, train S jitter, test $Q = \{256, 384, 512\}$); trên test set, cấu hình E đạt top-5 **7.3%**.

Multi-crop và fusion (Bảng 5, 6): multi-crop nhích hơn dense một chút (D: 24.6%/7.5% vs 24.8%/7.5%), kết hợp cả hai tốt hơn cả hai riêng lẻ (D: 24.4%/7.2%), lý giải do cách xử lý biên (boundary condition) khác nhau của phép tích chập (crop đệm 0, dense đệm bằng pixel lân cận thật). Kết hợp (ensemble) nhiều mạng bằng trung bình softmax: nộp bài ILSVRC (7 mạng, dense) đạt 7.3% top-5 test; sau khi nộp bài, chỉ dùng 2 mạng tốt nhất (D+E, multi-scale, multi-crop+dense) đạt **6.8%** top-5 test, tốt hơn cả 7 mạng gộp ban đầu.

So sánh SOTA (Bảng 7): VGG 2-mạng đạt 6.8% top-5 test, **giành vị trí thứ 2** ILSVRC-2014 classification (sau GoogLeNet 6.7%, dùng 7 mạng); VGG mạng đơn đạt 7.0% (tốt hơn GoogLeNet mạng đơn 0.9 điểm theo cách paper diễn giải). Cải thiện rất lớn so với kỷ lục 2012-2013: Krizhevsky et al. (2012, 1 mạng) 18.2% top-5 test → VGG giảm hơn 11 điểm tuyệt đối.

Localisation (Phụ lục A): dùng mạng D (16 tầng) với tầng cuối dự đoán bounding box (4 chiều cho SCR, hoặc $4000 = 4 \times 1000$ chiều cho PCR theo từng lớp). PCR tốt hơn SCR (34.3% vs 36.4% lỗi, Bảng 8, test trên crop giữa với nhãn lớp đúng), fine-tune toàn bộ tầng tốt hơn chỉ fine-tune 2 tầng FC đầu (33.1%). Với kịch bản đầy đủ (dense, đa tỉ lệ, ensemble, dự đoán top-5 lớp thay vì nhãn đúng): lỗi localisation top-5 **25.3%** trên test set (Bảng 9, 10), **giành giải nhất** ILSVRC-2014 localisation, vượt OverFeat (29.9%, nhà vô địch 2013).

Tổng quát hoá sang tập khác (Phụ lục B): dùng đặc trưng 4096-chiều ở tầng FC áp chót (không fine-tune, chỉ train SVM tuyến tính):

- VOC-2007/2012 (phân loại ảnh đa nhãn): VGG Net-D & Net-E kết hợp đạt mAP 89.7%/89.3%, vượt SOTA trước đó (Chatfield et al., 2014) hơn 6 điểm (Bảng 11).
- Caltech-101/256 (phân loại, đo bằng mean class recall): Net-D & Net-E kết hợp đạt 92.7%/86.2%, vượt Chatfield et al. tới 8.6 điểm trên Caltech-256.
- VOC-2012 action classification: chỉ dùng ảnh (không bbox) đạt mAP 79.2%, dùng cả ảnh và bounding box đạt **84.0%**, vượt SOTA trước đó (Bảng 12).
- Paper cũng ghi nhận cộng đồng dùng mô hình công khai này cho object detection (Girshick et al.), semantic segmentation, sinh chú thích ảnh (image captioning), nhận dạng vật liệu/texture, cho thấy khả năng tổng quát hoá rộng.

5. Phân tích / ablation

- **LRN không giúp ích:** A-LRN không tốt hơn A, lại tốn thêm bộ nhớ/thời gian; mọi cấu hình sâu hơn (B-E) bỏ LRN.
- **1x1 vs 3x3 ở cùng độ sâu (C vs D):** tách biệt được hai yếu tố "thêm phi tuyến" (ReLU của conv1x1) và "mở rộng receptive field không gian" (conv3x3); D tốt hơn C cho thấy receptive field không gian quan trọng hơn chỉ thêm phi tuyến đơn thuần.
- **Độ sâu bão hoà ở ~19 tầng:** lỗi giảm liên tục từ A (11 tầng) đến E (19 tầng) rồi "bão hoà" (saturate); tác giả suy đoán mạng sâu hơn nữa **có thể** có lợi trên tập dữ liệu lớn hơn, nhưng **không kiểm chứng** điều này (không có cấu hình >19 tầng trong paper).
- **Chiến lược gộp đặc trưng đa tỉ lệ khác nhau theo tập dữ liệu:** trên VOC, **trung bình (average)** đặc trưng đa tỉ lệ tốt bằng **ghép nối (stacking)**, vì vật thể trong VOC xuất hiện ở nhiều tỉ lệ khác nhau, không có tỉ lệ "đặc trưng" riêng; trên Caltech, **stacking tốt hơn average**, vì vật thể trong Caltech thường chiếm toàn bộ ảnh, nên đặc trưng ở các tỉ lệ khác nhau mang thông tin bổ sung khác nhau (toàn vật thể vs một phần vật thể) mà phép trung bình sẽ "xoá" đi. Đây là một ablation gắn trực tiếp với đặc điểm của từng tập dữ liệu, không phải một kết luận chung.
- **Multi-crop và dense bổ trợ nhau:** do khác cách xử lý biên khi tích chập (crop: đệm 0 ở biên crop; dense: đệm bằng pixel ảnh lân cận thật), hai cách này nắm được thông tin hơi khác nhau, nên kết hợp luôn tốt hơn từng cách riêng.

6. Giới hạn

Do tác giả nêu:

- "Lỗi bão hoà ở độ sâu 19 tầng (cho tập dữ liệu này), nhưng mô hình sâu hơn **có thể** có lợi cho tập dữ liệu lớn hơn" (suy đoán, không kiểm chứng trong paper).
- Multi-scale training **không** được chạy từ đầu (from scratch) vì lý do thời gian, chỉ fine-tune toàn bộ tầng từ một mạng single-scale $S = 384$ đã train sẵn; nên hiệu quả thật của việc train multi-scale ngay từ đầu chưa được đo riêng.
- Multi-crop evaluation tăng chi phí tính toán nhưng tác giả tự nhận "trong thực tế lợi ích tăng accuracy không chắc đủ để biện minh cho thời gian tính toán tăng thêm", chỉ báo cáo multi-crop "để tham khảo".
- Phát hiện sau khi nộp bài rằng có thể bỏ qua kỹ thuật khởi tạo "pre-train mạng A rồi truyền lại" nếu dùng khởi tạo Glorot & Bengio (2010), nhưng **không re-run** để xác nhận điều này ảnh hưởng ra sao tới toàn bộ kết quả chính.

Nhận xét:

- Phần lớn tham số (133-144M) nằm ở 3 tầng FC cuối (đặc biệt tầng FC đầu nối từ feature map $7 \times 7 \times 512$ sang 4096, tức khoảng $7 \times 7 \times 512 \times 4096 \approx 1.03 \times 10^8$ tham số chỉ riêng tầng đó, tự tính theo cấu trúc Bảng 1/mục 2.1), một thiết kế mà kiến trúc sau này (ví dụ ResNet, bài 07 cùng thư mục) bỏ hẳn bằng global average pooling, giảm tham số đáng kể.
- Paper không báo cáo trực tiếp số FLOPs của các cấu hình (số liệu FLOPs của VGG-16/19, 15.3/19.6 tỷ, được paper **khác** là ResNet (bài 07) tính và trích dẫn), nên không thể tự so sánh hiệu quả tính toán ngay trong phạm vi bài này.
- Thời gian huấn luyện 2-3 tuần mỗi mạng trên 4 GPU là rất tốn kém theo chuẩn hiện nay; paper không thảo luận các kỹ thuật tăng tốc huấn luyện hiện đại (ví dụ mixed precision) vì chưa tồn tại ở thời điểm đó.
- So sánh với GoogLeNet (nhà vô địch cùng năm, kiến trúc Inception song song nhánh) cho thấy "càng sâu với filter nhỏ" không phải là con đường duy nhất để đạt độ chính xác cao; GoogLeNet đạt độ chính xác tương đương/nhích hơn với độ sâu hiệu dụng tương tự nhưng cấu trúc nhánh song song phức tạp hơn, một hạn chế ngầm của kết luận "đơn giản là xếp sâu hơn" mà paper tự thừa nhận gián tiếp khi so sánh.

7. Liên hệ với các bài khác

- **ml basics 07 (ResNet)** (file `07_2015_resnet.md`, đọc ngay trước bài này theo lộ trình): dùng chính VGG-16/19 làm "mạng plain" mốc so sánh, chỉ ra rằng nếu xếp chồng filter nhỏ kiểu VGG tới độ sâu lớn hơn nữa (ví dụ plain-34) sẽ gặp degradation problem; residual connection của ResNet giải quyết vấn đề này mà VGG (dừng ở 19 tầng) không gặp phải ở quy mô đã thử.
- **pruning content/04_2016_pruning_filters_convnets.md**: một trong các mạng được cắt filter trực tiếp trong bài đó chính là VGG; hiểu rõ cấu trúc "cùng kích thước feature map → cùng số filter, giảm 1 nửa kích thước → tăng gấp đôi filter" ở đây giúp hiểu ràng buộc khi cắt số filter theo tầng.
- **llm content/29_2020_vit.md**: Vision Transformer thay toàn bộ chuỗi conv3x3 của VGG bằng patch embedding + self-attention, loại bỏ hoàn toàn khái niệm "receptive field tăng dần qua nhiều tầng filter nhỏ" mà bài VGG này xây dựng, một đối lập kiến trúc rõ rệt đáng tham khảo.
- **distillation content/01_2014_do_deep_nets_need_to_be_deep.md**: cùng thời kỳ, đặt câu hỏi liệu mạng nông có thể đạt độ chính xác của mạng sâu nếu "được dạy đúng cách"; đối chiếu với phát hiện của VGG rằng tăng độ sâu (ở kiến trúc đơn giản, không cần residual) tự nó cải thiện độ chính xác đáng kể, cho một góc nhìn hai chiều về vai trò độ sâu.

8. Gợi ý áp dụng

Các phép tích chập nhỏ xếp chồng cho receptive field rộng hơn và thêm phi tuyến. Khi so sánh kiến trúc, cần tính cả số tham số, chi phí suy luận và ảnh hưởng của độ sâu, vì từng yếu tố có thể thay đổi kết quả.

9. Thí nghiệm gợi ý

1. **Kiểm chứng "3 tầng 3x3 rẻ hơn 1 tầng 7x7" trên dữ liệu nhỏ:** trên CIFAR-10 hoặc một tập ảnh nhỏ, xây 2 mạng con có cùng receptive field hiệu dụng (7×7): (a) 3 tầng conv3x3 liên tiếp, (b) 1 tầng conv7x7, cùng số kênh vào/ra C. Đếm tham số theo đúng công thức ở Mục 3.3 (tự tính, kỳ vọng tỉ lệ ≈ 1.81 giữa (b) và (a)), train cả hai, so sánh accuracy và thời gian train. Kỳ vọng: (a) đạt accuracy bằng hoặc hơn (b) với ít tham số hơn.
2. **Dò độ sâu kiểu A→E ở quy mô nhỏ:** trên một tập ảnh nhỏ (CIFAR-10/subset), xây một họ mạng theo đúng quy tắc VGG (độ rộng gấp đôi sau mỗi pool) với số tầng tăng dần (ví dụ 6, 9, 12, 15 tầng ở quy mô nhỏ), vẽ accuracy theo độ sâu. Kỳ vọng: accuracy tăng rồi bão hoà (không nhất thiết đúng ở tầng 19 như ImageNet, vì tập dữ liệu nhỏ hơn nhiều).
3. **Ablation LRN:** thêm/bỏ Local Response Normalisation ở một mạng nhỏ cố định kiến trúc, đo accuracy. Kỳ vọng: không khác biệt đáng kể hoặc LRN hơi làm chậm/không giúp ích, như quan sát A vs A-LRN.
4. **So sánh gộp đặc trưng đa tỉ lệ (average vs stacking) trên 2 tập khác nhau:** với một mạng nhỏ đã train, trích đặc trưng ở 3 tỉ lệ ảnh khác nhau, thử gộp bằng trung bình vs ghép nối, đo accuracy của một classifier tuyến tính trên 2 loại tập ảnh khác nhau về "độ chiếm khung hình của vật thể" (ví dụ ảnh vật thể lớn chiếm toàn khung vs ảnh có nhiều vật thể nhỏ). Kỳ vọng: tái hiện quan sát VOC (trung bình đủ tốt) vs Caltech (ghép nối tốt hơn) ở quy mô nhỏ.
5. **Đánh giá dense vs multi-crop trên ảnh kích thước thay đổi:** với mạng nhỏ đã train ở input cố định, chuyển FC→conv, chạy "dense" trên ảnh có kích thước khác train, so với crop cố định nhiều mảnh rồi gộp; đo accuracy và thời gian suy luận. Liên quan (suy luận) tới việc xử lý trang OCR có độ phân giải thay đổi.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Receptive field của n tầng conv3x3 liên tiếp (không pooling giữa)	$(2n+1) \times (2n+1)$; $n=3 \Rightarrow 7 \times 7$
Tham số của k tầng conv3x3 (kênh C)	$k \times (3^2C^2) = 9kC^2$
Tham số 1 tầng conv$(2k+1) \times (2k+1)$ (kênh C)	$(2k+1)^2C^2$
So sánh $k=3$ (7x7 hiệu dụng)	$27C^2$ (3 tầng 3x3) vs $49C^2$ (1 tầng 7x7), chênh ≈ 81% (tự tính)
Số tầng các cấu hình A→E	11, 11, 13, 16, 16, 19 (A, A-LRN, B, C, D, E)
Số tham số A→E	133M, 133M, 133M, 134M, 138M, 144M
Độ rộng kênh	64 → 128 → 256 → 512 → 512 (gấp đôi sau mỗi max-pool)
Lỗi tốt nhất single-net (val, Bảng 4)	top-1 24.8%, top-5 7.5% (D hoặc E)
Lỗi tốt nhất ensemble 2 mạng (test, Bảng 7)	top-5 6.8% (giải nhì ILSVRC-2014 classification)
Localisation tốt nhất (test, Bảng 10)	top-5 25.3% (giải nhất ILSVRC-2014 localisation)
Dropout huấn luyện	0.5 ở 2 tầng FC đầu (4096 kênh)
Weight decay / momentum	5×10^{-4} (L2) / 0.9

11. Kết luận ngắn

VGG chứng minh một nguyên lý thiết kế đơn giản nhưng mạnh: cố định mọi thứ khác, chỉ dùng filter 3×3 và tăng dần độ sâu, độ chính xác ImageNet tăng đơn điệu tới khoảng 19 tầng, vượt các kiến trúc trước đó dùng filter lớn ở tầng đầu. Lợi ích kép của việc dùng nhiều filter nhỏ thay một filter lớn (ít tham số hơn ở cùng receptive field, nhiều phi tuyến hơn) là đóng góp khái niệm cốt lõi, cùng với bằng chứng mạnh về khả năng tổng quát hoá (transfer) của đặc trưng học được sang nhiều tập dữ liệu/bài toán khác. Hạn chế chính: bài báo không tự tìm ra giới hạn của hướng "chỉ tăng độ sâu" (19 tầng chỉ là "bão hoà" cho tập dữ liệu này, chưa phải giới hạn tối ưu hoá); chính giới hạn tối ưu hoá đó mới lộ ra ở các mạng sâu hơn nhiều, được bài 07 (ResNet) trong lộ trình này giải quyết bằng residual connection.

Nguồn gốc: https://arxiv.org/abs/1409.1556

Bài 09

Efficient Estimation of Word Representations in Vector Space (2013)

Tác giả: Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean (Google Inc., Mountain View, CA).
Venue / arXiv: arXiv:1301.3781 (v3, 07/09/2013); paper không nêu venue hội nghị trên bìa (bài 10 trong thư mục này dẫn lại nó là "ICLR Workshop, 2013").
Nguồn: https://arxiv.org/abs/1301.3781
Chủ đề: hai kiến trúc log-linear (CBOW và Skip-gram) để học vector từ liên tục ("word vector") từ tập dữ liệu rất lớn, rẻ hơn nhiều so với mô hình ngôn ngữ nơ-ron có tầng ẩn phi tuyến (NNLM, RNNLM), cùng một bộ test đo quan hệ tuyến tính ngữ nghĩa và cú pháp giữa các từ.
Vị trí trong lộ trình: Đọc trước bài 10 về negative sampling và biểu diễn cụm từ; các bài 11–12 tiếp tục với mô hình chuỗi.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Trước 2013, đa số hệ thống NLP coi từ là **đơn vị rời rạc** (atomic unit): mỗi từ là một chỉ số (index) trong một từ điển, không có khái niệm "độ giống nhau" giữa hai từ. Lựa chọn này đơn giản, bền (robust), và các mô hình N-gram đơn giản huấn luyện trên lượng dữ liệu khổng lồ (hàng nghìn tỷ từ) vẫn thắng các hệ phức tạp huấn luyện trên ít dữ liệu hơn.

Nhưng cách này gặp trần trong nhiều bài toán: dữ liệu có nhãn chất lượng cao cho nhận dạng tiếng nói thường chỉ có hàng triệu từ; corpus dịch máy song ngữ cho nhiều cặp ngôn ngữ chỉ vài tỷ từ hoặc ít hơn. Khi dữ liệu bị chặn trên, việc tăng N-gram đơn thuần không giúp nhiều; cần kỹ thuật tiên tiến hơn.

Biểu diễn phân tán của từ ("distributed representations", Hinton và cộng sự 1986) là hướng thành công nhất: thay vì one-hot, mỗi từ được gán một vector số thực liên tục, học bằng mạng nơ-ron. Các mô hình ngôn ngữ nơ-ron (neural network language model, NNLM) đã được chứng minh vượt N-gram đáng kể. Công trình trước của chính nhóm tác giả (Mikolov 2007, Mikolov và cộng sự 2009) cho thấy có thể học vector từ bằng một **mô hình đơn giản** (không cần xây NNLM đầy đủ), rồi dùng các vector đó để huấn luyện N-gram NNLM phía trên.

Một phát hiện quan trọng dùng làm động lực: với vector từ huấn luyện tốt, phép toán đại số đơn giản mang lại kết quả ngữ nghĩa, ví dụ vector("King") - vector("Man") + vector("Woman") gần nhất với vector("Queen") (Mikolov và cộng sự, "Linguistic Regularities in Continuous Space Word Representations", NAACL HLT 2013, paper trích dẫn [20]). Mục tiêu của bài này là: (a) học vector từ chất lượng cao từ tập dữ liệu hàng tỷ từ, từ điển hàng triệu từ (trước đó dimensionality thường chỉ 50-100 và dữ liệu nhỏ hơn nhiều); (b) thiết kế một bộ test toàn diện đo cả quan hệ cú pháp và ngữ nghĩa; (c) xem training time và accuracy phụ thuộc thế nào vào dimensionality của vector và lượng dữ liệu huấn luyện.

Khái niệm nền cần biết trước khi đọc (bổ sung cho bài 00 chưa viết): **one-hot encoding / 1-of-V coding** là biểu diễn một từ bằng vector dài V (kích thước từ điển) toàn số 0 trừ một vị trí bằng 1; **softmax** chuyển một vector điểm số thành phân phối xác suất; **hierarchical softmax** là một cách tính softmax gần đúng nhanh hơn bằng cây nhị phân; **stochastic gradient descent (SGD)** và **backpropagation** là thuật toán huấn luyện chuẩn của mạng nơ-ron.

2. Ý tưởng chính

Paper quan sát: phần đắt nhất về tính toán trong NNLM/RNNLM là **tầng ẩn phi tuyến** (hidden layer phi tuyến nối đầy đủ với tầng projection và tầng output). Nếu bỏ tầng ẩn phi tuyến này, mô hình học chậm hơn về biểu diễn phức tạp nhưng có thể huấn luyện trên **nhiều dữ liệu hơn rất nhiều** với cùng ngân sách tính toán, và với ngôn ngữ tự nhiên điều đó hóa ra đáng giá.

Hai kiến trúc log-linear được đề xuất:

- **CBOW (Continuous Bag-of-Words):** dự đoán từ hiện tại w_t từ trung bình (tổng rồi chia, không có thứ tự) của các vector từ ngữ cảnh xung quanh (quá khứ và tương lai). Vì không quan tâm thứ tự, gọi là "bag-of-words" (nhưng dùng biểu diễn liên tục, khác bag-of-words rời rạc cổ điển).
- **Skip-gram:** ngược lại, dùng từ hiện tại w_t để dự đoán từng từ trong một cửa sổ ngữ cảnh xung quanh nó (không phải tổng cả cửa sổ cùng lúc, mà là nhiều cặp phân loại riêng lẻ (w_t, w_{t+j})).

Cả hai bỏ tầng ẩn phi tuyến, chỉ còn một tầng projection (ánh xạ tuyến tính dùng chung trọng số) và một tầng output dùng **hierarchical softmax** (cây Huffman nhị phân, gán mã ngắn cho từ phổ biến) để tránh phải tính softmax đầy đủ trên toàn bộ từ điển V (thường 10^5-10^7 từ).

Để so sánh công bằng giữa các kiến trúc, paper định nghĩa **độ phức tạp huấn luyện** là số tham số cần truy cập để huấn luyện đầy đủ, và biểu diễn tổng chi phí là

$$O = E \times T \times Q \tag{1}$$

với E là số epoch (thường 3-50), T là số từ trong tập huấn luyện (tới hàng tỷ), và Q là chi phí trên một ví dụ huấn luyện, định nghĩa riêng cho từng kiến trúc ở mục 3. Mục tiêu là tối đa accuracy trong khi tối thiểu Q.

Kết quả cốt lõi: dùng hai kiến trúc rẻ này, có thể huấn luyện trên **6 tỷ token** (Google News corpus) trong **chưa đầy một ngày**, đạt độ chính xác state-of-the-art trên bộ test quan hệ từ tự thiết kế, vượt xa các vector từ công bố trước đó (Collobert-Weston, Turian, Mnih, RNNLM của chính nhóm) dù các mô hình đó huấn luyện hàng tuần đến hàng tháng.

3. Phương pháp chi tiết

3.1 Hai baseline: NNLM và RNNLM (mục 2.1, 2.2)

NNLM (Bengio và cộng sự 2003): tầng input mã 1-of-V cho N từ trước đó, chiếu (projection) chung một ma trận vào tầng projection P kích thước $N \times D$ (rẻ vì chỉ N input active), rồi tầng ẩn H (phi tuyến, dense, đây là nút cổ chai), rồi tầng output V (phân phối xác suất trên từ điển). Chi phí một ví dụ (công thức 2):

$$Q = N \times D + N \times D \times H + H \times V \tag{2}$$

Số hạng chiếm ưu thế là $H \times V$, nhưng dùng hierarchical softmax (cây Huffman) đưa nó về khoảng $H \times \log_2 V$. Phần còn lại $N \times D \times H$ vẫn là nút cổ chai chính của NNLM.

RNNLM (Mikolov và cộng sự 2010): không có tầng projection, chỉ input, hidden ($D = H$), output; có ma trận hồi quy nối hidden với chính nó qua bước thời gian (trễ), tạo "bộ nhớ ngắn hạn". Không cần cố định độ dài ngữ cảnh N như NNLM. Chi phí (công thức 3):

$$Q = H \times H + H \times V \tag{3}$$

với $H \times V$ cũng giảm về $H \times \log_2 V$ nhờ hierarchical softmax; phần còn lại $H \times H$.

Hierarchical softmax chi tiết: biểu diễn từ điển là cây Huffman nhị phân (gán mã ngắn cho từ tần suất cao, dựa trên quan sát trước đó rằng nhóm từ theo tần suất giúp tốc độ NNLM). Thay vì phải đánh giá V nút output, chỉ cần khoảng $\log_2(\text{Unigram perplexity}(V))$ nút. Với V = 1 triệu từ, paper nói điều này cho khoảng 2 lần tăng tốc so với cây cân bằng (balanced tree, $\log_2 V$ nút).

3.2 CBOW và Skip-gram (mục 3.1, 3.2)

CBOW: giống NNLM nhưng bỏ tầng ẩn phi tuyến, và tầng projection dùng chung cho mọi vị trí từ (không chỉ ma trận chung, mà cả vị trí trong projection: mọi từ ngữ cảnh được chiếu vào **cùng một vị trí**, tức vector của chúng được **lấy trung bình**). Dùng cả từ "tương lai" và "quá khứ": kiến trúc tốt nhất trong thí nghiệm dùng 4 từ trước và 4 từ sau để phân loại từ hiện tại (từ giữa). Chi phí (công thức 4):

$$Q = N \times D + D \times \log_2 V \tag{4}$$

Skip-gram: đối ngẫu với CBOW. Dùng từ hiện tại làm input cho một bộ phân loại log-linear có tầng projection liên tục, dự đoán các từ trong một khoảng trước và sau nó. Tăng khoảng ngữ cảnh cải thiện chất lượng vector nhưng tăng chi phí; vì từ xa thường ít liên quan hơn từ gần, paper **lấy mẫu ít hơn** ở khoảng cách xa: với C là khoảng cách tối đa, với mỗi từ huấn luyện chọn ngẫu nhiên $R \in [1, C]$, rồi dùng R từ quá khứ và R từ tương lai làm nhãn đúng, tức $R \times 2$ phép phân loại cho một từ trung tâm. Paper dùng $C = 10$ trong thí nghiệm. Chi phí (công thức 5):

$$Q = C \times (D + D \times \log_2 V) \tag{5}$$

Hình 1 trong paper minh họa: ở CBOW, các vector $w(t-2), w(t-1), w(t+1), w(t+2)$ được chiếu và **cộng** (SUM) vào một tầng projection chung, từ đó dự đoán $w(t)$; ở Skip-gram, $w(t)$ được chiếu, và từ tầng projection đó dự đoán riêng lẻ từng $w(t-2), \ldots, w(t+2)$.

3.3 Huấn luyện song song

Cả hai kiến trúc được huấn luyện trên **DistBelief**, một framework phân tán quy mô lớn (Dean và cộng sự, NIPS 2012): chạy nhiều bản sao (replica) song song, mỗi bản sao đồng bộ gradient qua một server trung tâm giữ toàn bộ tham số; tối ưu bằng **mini-batch asynchronous SGD** với **Adagrad** (tốc độ học thích nghi theo từng tham số). Thường dùng 50-100 bản sao mô hình, mỗi bản sao chạy trên nhiều CPU core tại một máy khác nhau trong trung tâm dữ liệu.

3.4 Ví dụ số tự tính: so sánh chi phí Q

Lấy các số đại diện mà paper gợi ý ("common choice"): $N = 10$, $H = 500$, giả sử $D = 100$ và từ điển $V = 2^{20} \approx 1.05$ triệu từ (để $\log_2 V = 20$, đúng bậc độ lớn paper dùng trong thí nghiệm, 1 triệu từ). Khi đó (tự tính, dùng hierarchical softmax cho cả hai để so công bằng):

$$Q_{NNLM} = ND + NDH + H \log_2 V = 1.000 + 500.000 + 10.000 = 511.000$$

$$Q_{CBOW} = ND + D \log_2 V = 1.000 + 2.000 = 3.000$$

Tỉ lệ $Q_{NNLM}/Q_{CBOW} \approx 170$ (tự tính: $511.000/3.000 \approx 170{,}3$). Đây là lý do CBOW/Skip-gram có thể dùng cùng ngân sách tính toán để huấn luyện trên lượng dữ liệu lớn hơn gấp nhiều lần so với NNLM: theo công thức (1), $O = E \times T \times Q$, nếu Q giảm 170 lần thì với cùng O ta có thể tăng T (lượng từ huấn luyện) lên gần 170 lần (xấp xỉ, bỏ qua hiệu ứng bão hòa).

3.5 Bộ test quan hệ từ (mục 4.1)

Paper xây **Semantic-Syntactic Word Relationship test set**: 5 loại quan hệ ngữ nghĩa (vd: capital-country, currency) và 9 loại quan hệ cú pháp (vd: so sánh hơn, số nhiều, thời quá khứ). Tổng cộng **8869 câu hỏi ngữ nghĩa và 10675 câu hỏi cú pháp** (paper nêu trực tiếp), tổng **19544 câu hỏi** (tự tính: $8869 + 10675 = 19544$). Mỗi câu hỏi gồm hai cặp từ cùng quan hệ, ví dụ "Athens:Greece :: Oslo:Norway". Để trả lời, tính $X = \text{vec}(\text{biggest}) - \text{vec}(\text{big}) + \text{vec}(\text{small})$, tìm từ gần X nhất theo cosine similarity (loại các từ input ra khỏi tìm kiếm), và tính là **đúng tuyệt đối** chỉ khi từ gần nhất khớp chính xác từ đáp án (từ đồng nghĩa bị tính sai). Vì vậy 100% accuracy là không khả thi (mô hình không có thông tin hình thái từ).

4. Thí nghiệm và kết quả chính

Dữ liệu chính: Google News corpus, khoảng **6 tỷ token**, từ điển hạn chế **1 triệu từ phổ biến nhất**.

Bảng 2 (độ chính xác theo dimensionality và lượng dữ liệu, CBOW, từ điển rút gọn 30K): với dimensionality 50 đến 600 và lượng dữ liệu 24M đến 783M từ, accuracy tổng tăng từ 13.4% (50 chiều, 24M từ) lên 50.4% (600 chiều, 783M từ). Paper rút ra: sau một điểm nào đó, **tăng riêng chiều hoặc riêng dữ liệu cho lợi ích giảm dần**; cần tăng cả hai cùng lúc. Theo công thức (4), tăng gấp đôi dữ liệu huấn luyện tốn độ phức tạp tính toán tương đương tăng gấp đôi kích thước vector.

Bảng 3 (so kiến trúc, cùng dữ liệu LDC 320M từ, 82K từ điển, 640 chiều):

Kiến trúc	Semantic (%)	Syntactic (%)	MSR test [20] (%)
RNNLM	9	36	35
NNLM	23	53	47
CBOW	24	64	61
Skip-gram	55	59	56

RNNLM (chỉ dùng trong [20]) yếu nhất; NNLM khá hơn vì có tầng ẩn phi tuyến; CBOW tốt hơn NNLM trên cú pháp, tương đương trên ngữ nghĩa; Skip-gram **vượt xa** trên ngữ nghĩa dù hơi kém CBOW trên cú pháp.

Bảng 4 (toàn bộ Google News 6B, so với vector công bố trước đó, từ điển đầy đủ): "Our NNLM" 100 chiều đạt 34.2/64.5/50.8 (semantic/syntactic/total, %); CBOW 300 chiều, 783M từ: 15.5/53.1/36.1; Skip-gram 300 chiều, 783M từ: 50.0/55.9/53.3; các vector Collobert-Weston, Turian, Mnih (huấn luyện trên dữ liệu ít hơn, 37M-990M từ) đều thấp hơn nhiều (ví dụ Collobert-Weston 50 chiều, 660M từ chỉ đạt 11.0% total).

Bảng 5 (1 epoch so với 3 epoch): 3-epoch CBOW mất 1 ngày huấn luyện, 3-epoch Skip-gram mất 3 ngày. Huấn luyện 1 epoch trên **gấp đôi dữ liệu** cho kết quả tương đương hoặc tốt hơn 3 epoch trên dữ liệu cũ, và nhanh hơn: ví dụ 1-epoch CBOW 783M từ đạt 33.6% total, còn 1-epoch CBOW 1.6B từ đạt 36.1% (bằng hẳn 3-epoch CBOW 783M từ, cũng 36.1%) nhưng rẻ hơn.

Bảng 6 (DistBelief, 1000 chiều, 6B từ): NNLM 100 chiều: 50.8% total, tốn 14 ngày × 180 CPU (huấn luyện NNLM 1000 chiều "sẽ tốn quá lâu để hoàn thành", không báo cáo được); CBOW 1000 chiều: 63.7% total, 2 ngày × 140 CPU; Skip-gram 1000 chiều: 65.6% total, 2.5 ngày × 125 CPU. Đây là minh chứng rõ nhất cho lợi thế tính toán: CBOW/Skip-gram đạt accuracy cao hơn NNLM nhiều với chi phí CPU-ngày thấp hơn một bậc độ lớn.

Bảng 7 (Microsoft Sentence Completion Challenge): baseline 4-gram 39%, LSA trung bình 49%, log-bilinear 54.8%, RNNLM kết hợp (SOTA trước đó) 55.4%; Skip-gram riêng chỉ 48.0% (kém RNNLM); nhưng **kết hợp Skip-gram + RNNLM** cho SOTA mới **58.9%** (59.2% dev, 58.7% test), cho thấy hai loại tín hiệu bổ sung nhau.

Bảng 8 (ví dụ chất lượng): vector(Paris) - vector(France) + vector(Italy) ≈ vector(Rome); tương tự Japan:Tokyo, Florida:Tallahassee; big:bigger, Miami:Florida → Baltimore:Maryland, copper:Cu → zinc:Zn, Microsoft:Windows → Google:Android, Japan:sushi → Germany:bratwurst. Paper tự nhận xét: vì metric exact-match quá nghiêm, các ví dụ ở Bảng 8 ước lượng chỉ đạt khoảng **60%** theo cách tính chính thức dù nhìn bằng mắt thấy hợp lý hơn. Lấy **trung bình 10 cặp ví dụ** quan hệ (thay vì 1 cặp) cải thiện accuracy thêm khoảng **10 điểm phần trăm tuyệt đối** trên mô hình tốt nhất.

5. Phân tích / ablation

- **Hierarchical softmax vs full softmax:** không có bảng số trực tiếp so sánh hai cách này trên cùng kiến trúc trong bài, nhưng mục 2.1 nêu ước lượng khoảng 2 lần tăng tốc cho $V = 10^6$ so với cây cân bằng; đây là lựa chọn kỹ thuật cố định trong mọi thí nghiệm, không phải một ablation riêng.
- **CBOW vs Skip-gram, đánh đổi tốc độ và chất lượng:** CBOW rẻ hơn Skip-gram đúng C lần theo công thức (4) so (5) (với $C = 10$); thực nghiệm Bảng 5 xác nhận: CBOW 3 epoch mất 1 ngày, Skip-gram 3 epoch mất 3 ngày trên cùng dữ liệu, nhưng Skip-gram cho semantic accuracy cao hơn nhiều (Bảng 3, 4, 6). Đây là đánh đổi tốc độ/chất lượng rõ ràng, không phải một kiến trúc thắng tuyệt đối.
- **Hiệu ứng tăng dữ liệu so với tăng kích thước (Bảng 2):** cả hai yếu tố có lợi ích giảm dần nếu tăng riêng lẻ; phải tăng đồng thời. Paper nhấn mạnh đây "có vẻ tầm thường" nhưng khi đó (2013) nhiều công trình vẫn huấn luyện trên dữ liệu lớn với dimensionality nhỏ (50-100), bỏ lỡ lợi ích.
- **Huffman tree so với gom nhóm theo tần suất khác:** paper chỉ nói ngắn "nhiều phương pháp xây cây đã được khảo sát bởi Mnih và Hinton" (tham chiếu [23]), không tự làm ablation so sánh cấu trúc cây trong bài này.
- **Follow-up work (mục 7, không phải thí nghiệm chính thức nhưng đáng ghi):** sau khi công bố bản đầu, nhóm tác giả phát hành mã C++ đa luồng, đạt tốc độ **hàng tỷ từ mỗi giờ** cho cấu hình siêu tham số thường dùng (nhanh hơn hẳn số liệu báo cáo trong bài, vốn đo trên DistBelief phân tán nhiều máy CPU dùng chung với tải khác).

6. Giới hạn

Do tác giả nêu:

- Accuracy 100% là không thể đạt được với metric exact-match (không có hiểu biết hình thái từ, đồng nghĩa luôn bị tính sai).
- Test set chỉ gồm từ đơn token (loại cụm từ nhiều từ như "New York"); đây chính là động lực cho bài 10 (học vector cho cụm từ).
- Số liệu "CPU core" trong Bảng 6 chỉ là ước lượng vì máy trung tâm dữ liệu dùng chung với tải sản xuất khác, có thể dao động.
- Các hằng số N_c, D_c trong các công thức phức tạp (không có trong bài này, nhắc lại từ scaling-laws loại khác) không áp dụng; ở bài này, giới hạn tương tự là: Bảng 3 dùng tập dữ liệu nhỏ và cũ (LDC 320M từ) chỉ để so với RNNLM công bố trước, không phải cấu hình tốt nhất của CBOW/Skip-gram.

Nhận xét:

- Phần "Kết quả Bảng 8" minh họa bằng tay rất đẹp nhưng paper tự thừa nhận chỉ đại diện khoảng 60% độ chính xác thực của mô hình tốt nhất; người đọc dễ bị ấn tượng quá mức bởi các ví dụ trình diễn.
- Mô hình không có cơ chế xử lý từ ngoài từ điển (OOV) ngoài việc loại bỏ hoàn toàn (giới hạn vocab 1M); không bàn đến subword hay byte-level tokenization (khái niệm sau này mới phổ biến).
- Không có so sánh trực tiếp hierarchical-softmax so với một baseline "không có" nào trong cùng kiến trúc CBOW/Skip-gram (chỉ so giữa các kiến trúc khác nhau, không so các lựa chọn output layer khác nhau); đây là khoảng trống mà bài 10 lấp một phần bằng negative sampling.
- Mọi thí nghiệm dùng tiếng Anh, Google News (văn bản tin tức); không rõ các con số có giữ nguyên trên domain khác hay ngôn ngữ khác.

7. Liên hệ với các bài khác

- **ml basics 10** (cùng thư mục, "Distributed Representations of Words and Phrases"): kế tục trực tiếp Skip-gram ở đây, thêm negative sampling (thay hierarchical softmax), subsampling từ tần suất cao, và học vector cho cụm từ; nên đọc ngay sau bài này.
- **llm content/01_2017_attention_is_all_you_need.md**: tầng embedding đầu vào và ma trận "unembedding" ở output của Transformer hiện đại là hậu duệ trực tiếp của ý tưởng word vector ở đây; bài toán "softmax trên từ điển lớn rẻ thế nào" (hierarchical softmax ở đây) vẫn là vấn đề thực tế với LM head của LLM (dù giải pháp hiện đại khác, ví dụ tied embedding, sampled softmax).
- **interp content/05_2022_toy_models_of_superposition.md**: tính chất "cấu trúc tuyến tính" của word vector (king - man + woman ≈ queen) là tiền đề lịch sử cho giả thuyết "biểu diễn tuyến tính" (linear representation hypothesis) được nghiên cứu sâu hơn trong superposition và interpretability hiện đại.
- **interp content/02_2021_mathematical_framework.md**: embedding matrix và unembedding matrix trong framework toán học của Transformer circuits là đúng loại đối tượng toán học (v_w, v'_w) mà bài này định nghĩa cho mô hình word2vec.

8. Gợi ý áp dụng

CBOW và Skip-gram là hai baseline hữu ích để học biểu diễn từ theo ngữ cảnh. Chất lượng embedding phụ thuộc vào dữ liệu, cửa sổ ngữ cảnh và cách đánh giá; các quan hệ vector không bảo đảm mô hình hiểu đầy đủ ngữ nghĩa.

9. Thí nghiệm gợi ý

1. **Tốc độ hierarchical softmax so với full softmax:** cài một Skip-gram nhỏ (PyTorch, vocab khoảng 20.000-50.000 từ từ một corpus tiếng Anh nhỏ như text8 cắt ngắn), huấn luyện cùng cấu hình (dim=100, $C = 5$) với (a) softmax đầy đủ và (b) hierarchical softmax dùng cây Huffman theo tần suất. Đo thời gian/epoch và loss. Kỳ vọng: (b) nhanh hơn rõ với vocab lớn hơn vài chục nghìn, khớp định tính với lập luận mục 2.1 ($\log_2 V$ so với V); diễn giải: lợi ích tăng theo V.
2. **CBOW so với Skip-gram ở quy mô nhỏ:** huấn luyện cả hai trên cùng 1-5 triệu token, dim=50, đo (a) thời gian huấn luyện, (b) độ chính xác trên một bộ 20-50 câu hỏi analogy tự viết tay (ví dụ good:better::bad:worse). Kỳ vọng tái lập định tính Bảng 3: CBOW nhanh hơn, Skip-gram có xu hướng bắt quan hệ "xa" (ngữ nghĩa) tốt hơn một chút dù ở quy mô nhỏ hiệu ứng có thể yếu và nhiều nhiễu; diễn giải cẩn trọng vì mẫu nhỏ.
3. **Hiệu ứng tăng dữ liệu và chiều cùng lúc (mini Bảng 2):** lưới nhỏ dim $\in \{25, 50, 100\}$ và kích thước subset dữ liệu $\in \{1\text{M}, 2\text{M}, 4\text{M}\}$ token, đo accuracy trên bộ analogy tự chế. Kỳ vọng: đường cong theo mỗi trục riêng bão hòa nhanh hơn đường chéo (tăng cả hai cùng lúc); nếu không thấy xu hướng, có thể do bộ test tự chế quá nhỏ để đo được, cần ghi nhận giới hạn này khi báo cáo.
4. **Đo "khoảng cách giữa exact-match và top-k":** trên mô hình đã huấn luyện ở thí nghiệm 2, với mỗi câu hỏi analogy, kiểm tra từ đúng có nằm trong top-1, top-5, top-10 gần nhất không. Kỳ vọng: accuracy top-5/top-10 cao hơn rõ top-1, minh họa định lượng nhận xét của paper rằng exact-match đánh giá thấp chất lượng thực (Bảng 8).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Chi phí huấn luyện tổng	$O = E \times T \times Q$ (1)
Q của NNLM	$ND + NDH + HV$, giảm về $+H \log_2 V$ với hierarchical softmax (2)
Q của RNNLM	$HH + HV$, giảm về $+H \log_2 V$ (3)
Q của CBOW	$ND + D \log_2 V$ (4)

Khái niệm	Công thức / giá trị
Q của Skip-gram	$C(D + D \log_2 V)$ (5)
Phép loại suy (analogy)	tìm w gần nhất (cosine) với $\text{vec}(b) - \text{vec}(a) + \text{vec}(c)$ cho câu hỏi $a : b :: c :?$
Bộ test	8869 câu ngữ nghĩa + 10675 câu cú pháp = 19544 câu (tự tính tổng)
Dữ liệu chính	Google News, khoảng 6 tỷ token, từ điển 1 triệu từ
Kết quả tốt nhất (Bảng 6)	Skip-gram 1000 chiều, 6B từ: 65.6% total accuracy, 2.5 ngày × 125 CPU

11. Kết luận ngắn

Bằng cách loại bỏ tầng ẩn phi tuyến đắt đỏ của NNLM/RNNLM, CBOW và Skip-gram học được vector từ chất lượng cao với chi phí tính toán thấp hơn hàng chục đến hàng trăm lần (tự tính minh họa: khoảng 170 lần với số liệu ví dụ ở mục 3.4), cho phép huấn luyện trên dữ liệu lớn hơn nhiều bậc độ lớn so với công trình trước. Skip-gram đặc biệt mạnh về quan hệ ngữ nghĩa, CBOW nhanh hơn và khá về cú pháp. Những hạn chế chính (metric exact-match nghiêm khắc, không xử lý cụm từ, không có cơ chế OOV) được chính bài 10 tiếp tục xem xét, đặc biệt là biểu diễn cụm từ.

Nguồn gốc: https://arxiv.org/abs/1301.3781

Bài 10

Distributed Representations of Words and Phrases and their Compositionality (2013)

Tác giả: Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, Jeffrey Dean (Google Inc., Mountain View).
Venue / arXiv: arXiv:1310.4546 (v1, 16/10/2013); paper không nêu venue trên bìa (bài 09 trong thư mục này, mục "Follow-Up Work", nói công trình này "sẽ xuất bản tại NIPS 2013").
Nguồn: https://arxiv.org/abs/1310.4546
Chủ đề: mở rộng Skip-gram (bài 09): negative sampling thay cho hierarchical softmax, subsampling từ tần suất cao để tăng tốc và cải thiện vector từ hiếm, và một phương pháp nhận diện cụm từ (phrase) để học vector cho cả cụm từ, cùng tính chất cộng vector (additive compositionality).
Vị trí trong lộ trình: Đọc sau bài 09 về CBOW và Skip-gram, trước các bài về encoder–decoder.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Bài 09 (cùng tác giả, Mikolov và cộng sự) giới thiệu Skip-gram: một mô hình log-linear học vector từ bằng cách dự đoán các từ lân cận trong câu, học được trên hàng tỷ từ với chi phí thấp nhờ bỏ tầng ẩn phi tuyến. Tuy vậy Skip-gram gốc vẫn dùng **hierarchical softmax** cho tầng output, và chưa xử lý tốt hai vấn đề:

1. Từ **rất phổ biến** ("in", "the", "a") chiếm số lần xuất hiện khổng lồ nhưng mang ít thông tin cho mỗi lần xuất hiện (ví dụ "France" đi với "Paris" hữu ích hơn nhiều so với "France" đi với "the"); đồng thời vector của từ phổ biến gần như không đổi gì nữa sau khi đã thấy vài triệu ví dụ.
2. Vector từ (word-level) không nắm được **cụm từ** mang nghĩa riêng không phải tổng các từ thành phần, ví dụ "Air Canada" không phải tổng nghĩa của "Air" và "Canada", hay "Boston Globe" (tên báo) không phải tổng nghĩa "Boston" và "Globe".

Bài báo này giải quyết cả hai vấn đề, đồng thời đề xuất một phương án thay hierarchical softmax đơn giản hơn: **negative sampling**, một biến thể đơn giản hóa của Noise Contrastive Estimation (NCE, Gutmann và Hyvärinen 2010; áp dụng vào ngôn ngữ bởi Mnih và Teh 2012).

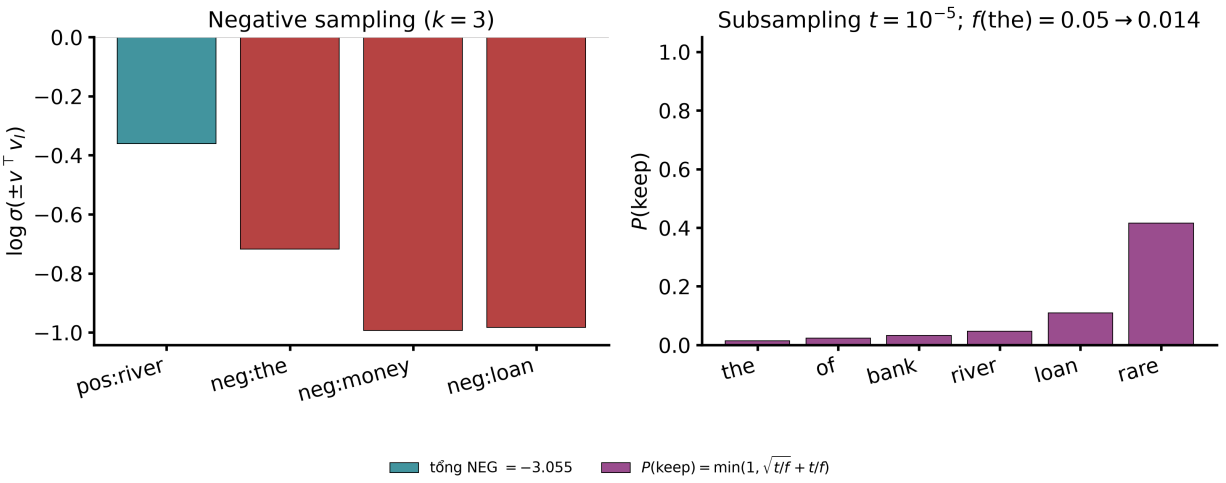
2. Ý tưởng chính

Ba cải tiến độc lập lên Skip-gram gốc:

- **Negative sampling (NEG):** thay vì tính softmax đầy đủ (hoặc hierarchical softmax) trên toàn từ điển để biết $p(w_O|w_I)$, chỉ cần phân biệt từ đích w_O thật khỏi k từ "nhiễu" lấy mẫu ngẫu nhiên, bằng hồi quy logistic (logistic regression). Rẻ hơn nhiều và cho vector tốt hơn với từ phổ biến.
- **Subsampling từ tần suất cao:** loại bớt (discard) ngẫu nhiên các lần xuất hiện của từ rất phổ biến theo một xác suất phụ thuộc tần suất, giúp tăng tốc 2-10 lần và cải thiện chất lượng vector của từ hiếm.
- **Học vector cụm từ:** nhận diện cụm từ bằng một thống kê đơn giản dựa trên tần suất song-gram so với tần suất đơn-gram, rồi coi cụm từ như một token duy nhất khi huấn luyện Skip-gram, nhờ đó học được vector cho cả cụm từ (ví dụ "New York Times", "Toronto Maple Leafs").

Ngoài ra, paper phát hiện một tính chất thú vị: vector học bởi Skip-gram có thể **cộng theo từng phần tử (element-wise)** để tạo nghĩa hợp lý, ví dụ vec("Russia") + vec("river") gần vec("Volga River"), không chỉ phép trừ-cộng phục vụ analogy như bài 09.

3. Phương pháp chi tiết



Hạng NEG $\log \sigma(\pm v^\top v_I)$ với $k = 3$ và $P(\text{keep})$ subsample ($t = 10^{-5}$) (minh hoạ, tự tính)

3.1 Mục tiêu Skip-gram (nhắc lại, mục 2)

Cho dãy từ huấn luyện $w_1, \ldots, w_T$, mục tiêu là tối đa log xác suất trung bình (công thức 1):

$$\frac{1}{T} \sum_{t=1}^{T} \sum_{-c \le j \le c, j \neq 0} \log p(w_{t+j} | w_t)$$

với c là kích thước cửa sổ ngữ cảnh. $p(w_O | w_I)$ định nghĩa bằng softmax đầy đủ (công thức 2):

$$p(w_O | w_I) = \frac{\exp(v'^\top_{w_O} v_{w_I})}{\sum_{w=1}^{W} \exp(v'^\top_w v_{w_I})}$$

với v_w, v'_w là vector "input" và "output" của từ w, W là số từ trong từ điển. Tính $\nabla \log p(w_O | w_I)$ tốn chi phí tỉ lệ W (thường 10^5-10^7), không thực tế.

3.2 Hierarchical softmax (nhắc lại, mục 2.1)

Biểu diễn từ điển bằng cây nhị phân Huffman, mỗi nút trong biểu diễn một xác suất nhánh; chi phí tính $\log p(w_O | w_I)$ và gradient của nó tỉ lệ $L(w_O)$ (độ dài đường đi từ gốc đến từ w_O), trung bình không vượt $\log W$. Công thức (3):

$$p(w | w_I) = \prod_{j=1}^{L(w)-1} \sigma\Big([\![n(w, j+1) = ch(n(w, j))]\!] \cdot v'^\top_{n(w,j)} v_{w_I} \Big)$$

Khác với softmax đầy đủ (mỗi từ có 2 vector v_w, v'_w), hierarchical softmax chỉ cần 1 vector v_w cho mỗi từ lá, cộng 1 vector v'_n cho mỗi nút trong của cây.

3.3 Negative sampling (mục 2.2, công thức 4)

Thay vì tính xác suất chuẩn hóa đầy đủ, định nghĩa một mục tiêu thay thế cho mỗi cặp (w_I, w_O):

$$\log \sigma(v'^{\top}_{w_O} v_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)} [\log \sigma(-v'^{\top}_{w_i} v_{w_I})] \tag{4}$$

dùng để thay mỗi số hạng $\log P(w_O|w_I)$ trong mục tiêu Skip-gram. Nhiệm vụ trở thành: phân biệt từ đích thật w_O khỏi k mẫu nhiễu rút từ phân phối nhiễu $P_n(w)$, bằng hồi quy logistic. Paper thử nghiệm thấy $k = 5$ **đến** 20 phù hợp với tập dữ liệu nhỏ, còn tập lớn chỉ cần $k = 2$ **đến** 5.

Khác biệt chính với NCE: NCE cần cả mẫu nhiễu và xác suất số của phân phối nhiễu; negative sampling chỉ cần mẫu. NCE xấp xỉ log xác suất của softmax đầy đủ (có ý nghĩa thống kê rõ), còn negative sampling thì không nhất thiết, nhưng paper lập luận tính chất đó không quan trọng cho mục tiêu "học vector tốt" ở đây.

Phân phối nhiễu $P_n(w)$: sau khi thử nhiều lựa chọn, paper thấy phân phối unigram lũy số mũ $3/4$, tức $U(w)^{3/4}/Z$, vượt trội rõ so với unigram thường ($U(w)$) hoặc uniform, trên mọi nhiệm vụ thử (kể cả mô hình ngôn ngữ, không báo cáo chi tiết trong bài).

3.4 Subsampling từ tần suất cao (mục 2.3, công thức 5)

Mỗi từ w_i trong tập huấn luyện bị loại bỏ (discard) với xác suất:

$$P(w_i) = 1 - \sqrt{\frac{t}{f(w_i)}} \tag{5}$$

với $f(w_i)$ là tần suất của từ (tỉ lệ số lần xuất hiện / tổng số từ), t là một ngưỡng, thường khoảng 10^{-5}. Công thức này **loại mạnh** các từ có $f(w_i) \gg t$ nhưng vẫn giữ thứ hạng tần suất (từ phổ biến hơn luôn bị loại nhiều hơn hoặc bằng từ ít phổ biến hơn). Paper nói công thức được chọn theo trực giác (heuristic) nhưng hiệu quả tốt trong thực nghiệm.

Ví dụ tự tính: với $t = 10^{-5}$ và một từ giả định có $f(\text{"the"}) = 0{,}05$ (5% số từ trong corpus, số minh họa, không phải số thật của paper):

$$P(\text{"the"}) = 1 - \sqrt{10^{-5}/0{,}05} = 1 - \sqrt{2 \times 10^{-4}} = 1 - 0{,}01414 \approx 0{,}986$$

tức khoảng 98.6% lần xuất hiện của một từ phổ biến như vậy sẽ bị loại khỏi huấn luyện (tự tính, minh họa cường độ của công thức, không phải số liệu paper đo trực tiếp cho từ "the").

3.5 Học cụm từ (mục 4, công thức 6)

Xác định cụm từ bằng điểm số đơn-giản dựa trên song-gram và đơn-gram:

$$\text{score}(w_i, w_j) = \frac{\text{count}(w_i w_j) - \delta}{\text{count}(w_i) \times \text{count}(w_j)} \tag{6}$$

với δ là hệ số chiết khấu (discount) để tránh tạo quá nhiều cụm từ gồm từ rất hiếm. Song-gram có điểm vượt ngưỡng chọn trước được coi là một cụm từ (token mới). Paper chạy **2-4 lượt** qua dữ liệu với ngưỡng giảm dần để cho phép cụm từ dài hơn (nhiều từ) hình thành dần. Ví dụ cụm "New York Times", "Toronto Maple Leafs" được gộp thành token; song-gram thường như "this is" thì không.

4. Thí nghiệm và kết quả chính

4.1 So sánh negative sampling, NCE, hierarchical softmax, subsampling (mục 3, Bảng 1)

Huấn luyện trên tập tin tức nội bộ của Google, 1 tỷ từ, loại từ xuất hiện dưới 5 lần, từ điển còn **692K từ**. Đo trên bộ test analogy của bài 09.

Phương pháp	Thời gian (phút)	Syntactic (%)	Semantic (%)	Tổng (%)
NEG-5 (không subsample)	38	63	54	59
NEG-15 (không subsample)	97	63	58	61
HS-Huffman (không subsample)	41	53	40	47
NCE-5 (không subsample)	38	60	45	53
NEG-5 (subsample 10^{-5})	14	61	58	60
NEG-15 (subsample 10^{-5})	36	61	61	61
HS-Huffman (subsample 10^{-5})	21	52	59	55

Kết luận paper rút ra: Negative Sampling vượt Hierarchical Softmax trên nhiệm vụ analogy, và nhích hơn cả NCE một chút. Subsampling cải thiện tốc độ huấn luyện (vài lần) và độ chính xác, đặc biệt rõ trên HS-Huffman (47% → 55% tổng) nhờ loại bớt các cập nhật lãng phí trên từ phổ biến.

4.2 Học cụm từ (mục 4.1, Bảng 2, 3)

Bộ test phrase-analogy mới: 3218 ví dụ, 5 loại quan hệ (báo chí theo thành phố, đội NHL, đội NBA, hãng hàng không, giám đốc công ty theo tên công ty). Với dim=300, context=5 (Bảng 3):

Phương pháp	Không subsample (%)	Subsample 10^{-5} (%)
NEG-5	24	27
NEG-15	27	42
HS-Huffman	19	47

Phát hiện đáng chú ý (paper tự gọi "surprising"): HS-Huffman **kém nhất** khi không subsample nhưng trở thành **tốt nhất** khi có subsampling 10^{-5} (47%, vượt cả NEG-15 42%); trên dữ liệu không subsample thì NEG-15 (27%) hơn HS (19%). Điều này cho thấy subsampling và phương pháp output layer tương tác không đơn giản: không có phương pháp nào thắng tuyệt đối trong mọi cấu hình.

Mô hình tốt nhất cho phrase (mục 4.1): tăng dữ liệu lên khoảng **33 tỷ từ**, dùng hierarchical softmax, dimensionality 1000, ngữ cảnh là cả câu, đạt accuracy **72%**. Giảm dữ liệu xuống 6 tỷ từ chỉ đạt **66%**, cho thấy lượng dữ liệu lớn quan trọng.

4.3 Tính chất cộng vector (mục 5, Bảng 5)

Ví dụ: Czech + currency → koruna; Vietnam + capital → Hanoi; German + airlines → airline Lufthansa; Russian + river → Moscow (hạng 1), Volga River (hạng 2); French + actress → Juliette Binoche. Giải thích định tính (không chứng minh hình thức chặt): vector từ có quan hệ tuyến tính với **input của hàm softmax**, tức liên quan log xác suất phân phối ngữ cảnh mà từ xuất hiện; tổng hai vector tương ứng gần với **tích** hai phân phối ngữ cảnh (hoạt động như AND: từ được cả hai "đồng thuận" gán xác suất cao sẽ có xác suất tổng hợp cao).

4.4 So sánh với các vector công bố trước đó (mục 6, Bảng 6)

So nearest-neighbor của các từ hiếm ("Redmond", "Havel", "ninjutsu", "graffiti", "capitulate") giữa Collobert-Weston (50 chiều, 2 tháng huấn luyện), Turian (200 chiều, vài tuần), Mnih (100 chiều, 7 ngày), và Skip-Phrase (1000 chiều, **1 ngày**, huấn luyện trên hơn 30 tỷ từ). Mô hình của paper cho neighbor hợp lý nhất (ví dụ

"Redmond" → "Redmond Wash.", "Redmond Washington", "Microsoft") dù huấn luyện nhanh hơn, nhờ dùng dữ liệu nhiều hơn 2-3 bậc độ lớn so với công trình trước.

5. Phân tích / ablation

- **Negative sampling so với NCE và hierarchical softmax (Bảng 1):** là ablation chính thức duy nhất kiểm soát cùng dữ liệu, cùng dimensionality (300); kết luận NEG > NCE > HS trên word-analogy không subsample, nhưng thứ tự đảo một phần khi có subsampling (mục 4.2 trên).
- **Số negative sample k:** $k = 15$ luôn tốt hơn $k = 5$ trong Bảng 1 và Bảng 3 (ví dụ Bảng 3: NEG-15 subsample 42% so với NEG-5 subsample 27%), nhưng tốn thời gian hơn (Bảng 1: NEG-15 không subsample 97 phút so với NEG-5 38 phút); đây là đánh đổi tốc độ/chất lượng rõ ràng theo k.
- **Subsampling là yếu tố có ảnh hưởng mạnh nhất lên tốc độ:** ví dụ NEG-5 giảm từ 38 phút (không subsample) xuống 14 phút (subsample) mà độ chính xác tổng còn tăng nhẹ (59% → 60%), tức vừa nhanh hơn gần 3 lần (tự tính: $38/14 \approx 2{,}7$) vừa không hại chất lượng.
- **Phân phối nhiễu $U(w)^{3/4}$:** paper khẳng định vượt trội unigram thường và uniform "trên mọi nhiệm vụ thử kể cả mô hình ngôn ngữ", nhưng không đưa bảng số cụ thể cho so sánh này trong bài (chỉ nêu bằng lời ở mục 2.2).
- **Lượng dữ liệu cho phrase (6B so với 33B từ):** 72% so với 66% (mục 4.1), cho thấy ở quy mô cụm từ hiếm, dữ liệu nhiều vẫn là yếu tố quyết định hơn cả kiến trúc.

6. Giới hạn

Do tác giả nêu:

- Negative sampling không có ý nghĩa thống kê rõ ràng (không xấp xỉ log-likelihood của softmax như NCE); mục tiêu chỉ là học vector tốt, không phải mô hình xác suất chính xác.
- Việc nhận diện cụm từ dùng thống kê đơn giản (công thức 6), paper tự nói "ngoài phạm vi công trình này" để so sánh với các kỹ thuật nhận diện cụm từ khác đã có.
- Chạy tối đa 2-4 lượt với ngưỡng giảm dần để tạo cụm từ dài hơn; không có cách tự động chọn số lượt hay ngưỡng tối ưu, phải chọn bằng tay.
- Chỉ huấn luyện trên tiếng Anh (tin tức Google News / tập tin tức nội bộ), không rõ các kết luận giữ nguyên trên ngôn ngữ khác.

Nhận xét:

- Kết quả Bảng 3 (HS-Huffman đảo vị trí khi có/không subsampling) là một ví dụ cho thấy kết luận "phương pháp X tốt hơn Y" trong NLP giai đoạn này khá nhạy với việc có/không một kỹ thuật phụ trợ (subsampling); người đọc cần cẩn trọng khi trích dẫn riêng Bảng 1 hoặc riêng Bảng 3 mà bỏ qua điều kiện subsampling.
- Mô hình tốt nhất cho phrase-analogy (72%, mục 4.1) dùng hierarchical softmax, không phải negative sampling, dù phần đầu bài quảng bá negative sampling là cải tiến chính; nghĩa là lựa chọn output-layer tối ưu phụ thuộc bài toán cụ thể (word-analogy so với phrase-analogy), không có một cấu hình thắng tuyệt đối trong toàn bài.
- Không có số liệu về chi phí bộ nhớ khi từ điển phình to do thêm cụm từ (mỗi cụm từ mới là một vector riêng đầy đủ dimensionality), chỉ nói định tính "không tăng kích thước từ điển quá nhiều" (mục 4).

7. Liên hệ với các bài khác

- **ml basics 09** (cùng thư mục, "Efficient Estimation of Word Representations..."): bài nền trực tiếp; mọi công thức Skip-gram (mục (1), (2) ở đây) lấy lại từ đó, bài này chỉ thay đổi tầng output (negative sampling) và thêm subsampling, phrase.

- **ml basics 11, 12** (cùng thư mục, NMT và seq2seq): cả hai dùng embedding liên tục làm tầng vào của encoder/decoder, kế thừa tinh thần "biểu diễn từ liên tục" từ dòng word2vec, dù không dùng trực tiếp negative sampling.
- **pruning content/44_2025_nemotron_h.md, 45_2025_minitron_ssm.md**: các mô hình ngôn ngữ hiện đại vẫn phải xử lý vấn đề "tầng output trên từ điển lớn tốn gì" mà negative sampling/hierarchical softmax ở đây từng giải quyết ở quy mô nhỏ hơn nhiều; LM head của các model này lớn hơn nhiều bậc nhưng bài toán gốc (ước lượng softmax rẻ) tương tự về bản chất.
- **interp content/05_2022_toy_models_of_superposition.md**: tính chất cộng vector (mục 5, vec(Russia)+vec(river) ≈ vec(Volga River)) là một minh chứng thực nghiệm sớm cho việc các đặc trưng ngữ nghĩa được mã hóa (gần) tuyến tính và có thể chồng chập (superposition) trong không gian embedding có số chiều hữu hạn.

8. Gợi ý áp dụng

Negative sampling và subsampling giúp huấn luyện biểu diễn từ hiệu quả hơn. Khi áp dụng, nên khảo sát phân phối từ âm và ngưỡng loại bớt từ phổ biến, đồng thời đánh giá trên tác vụ phù hợp với dữ liệu.

9. Thí nghiệm gợi ý

1. **So sánh negative sampling và softmax đầy đủ:** cài Skip-gram nhỏ (PyTorch) trên một corpus vài triệu token, vocab khoảng 30.000-50.000. Huấn luyện với (a) softmax đầy đủ, (b) negative sampling $k = 5$, (c) negative sampling $k = 15$. Đo thời gian/epoch và accuracy trên một bộ analogy tự chế nhỏ. Kỳ vọng: (b), (c) nhanh hơn (a) rõ, (c) chậm hơn (b) nhưng chính xác hơn, tương tự xu hướng Bảng 1.
2. **Hiệu ứng subsampling lên tốc độ và chất lượng từ hiếm:** trên cùng thiết lập trên, so sánh có/không subsampling ($t = 10^{-5}$); đo riêng accuracy trên các câu hỏi analogy chỉ chứa từ hiếm (tần suất thấp trong corpus) so với từ phổ biến. Kỳ vọng: subsampling cải thiện rõ hơn trên nhóm từ hiếm, khớp tuyên bố của paper ở mục 2.3.
3. **Tái hiện mini Bảng 3 (word so với phrase):** ghép thủ công khoảng 50-100 cụm hai từ phổ biến trong corpus nhỏ thành token đơn theo công thức (6) với δ tự chọn, huấn luyện lại Skip-gram, kiểm tra một vài ví dụ cộng vector (tương tự Bảng 5) bằng tay. Kỳ vọng: một số cặp cho kết quả hợp lý, nhiều cặp không (vì corpus nhỏ không đủ để học cụm hiếm); diễn giải cần nêu rõ giới hạn do quy mô dữ liệu nhỏ.
4. **Kiểm tra phân phối nhiễu:** huấn luyện hai mô hình negative sampling, một dùng $U(w)$ (unigram thường) làm $P_n(w)$, một dùng $U(w)^{3/4}$; so accuracy analogy. Kỳ vọng: $U(w)^{3/4}$ tốt hơn hoặc bằng, theo khẳng định mục 2.2; nếu không thấy khác biệt trên corpus nhỏ, ghi nhận có thể do quy mô chưa đủ lớn để hiệu ứng rõ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Mục tiêu Skip-gram	$\frac{1}{T}\sum_t \sum_{-c\le j\le c, j\neq 0} \log p(w_{t+j} \mid w_t)$ (1)
Softmax đầy đủ	$p(w_O \mid w_I) = \exp(v'^{\top}_{w_O} v_{w_I}) / \sum_w \exp(v'^{\top}_w v_{w_I})$ (2)
Negative sampling	$\log \sigma(v'^{\top}_{w_O} v_{w_I}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n}[\log \sigma(-v'^{\top}_{w_i} v_{w_I})]$ (4)
Số negative sample k	5-20 cho tập nhỏ, 2-5 cho tập lớn
Phân phối nhiễu tốt nhất	$U(w)^{3/4}/Z$
Subsampling	$P(w_i) = 1 - \sqrt{t/f(w_i)}$, $t \approx 10^{-5}$ (5)
Điểm số cụm từ	$\text{score}(w_i, w_j) = \big(\text{count}(w_i w_j) - \delta\big) / \big(\text{count}(w_i)\text{count}(w_j)\big)$ (6)
Tính chất cộng vector	vec(a) + vec(b) ≈ vec(cụm liên quan a và b), ví dụ Russia + river ≈ Volga River
Kết quả tốt nhất phrase-analogy	72% với dim=1000, HS, 33 tỷ từ

11. Kết luận ngắn

Bài này hoàn thiện bộ công cụ word2vec: negative sampling là một thay thế đơn giản, rẻ và hiệu quả cho hierarchical softmax (tốt hơn trên word-analogy, nhưng không phải trên mọi cấu hình như phrase-analogy); subsampling từ tần suất cao vừa tăng tốc vừa cải thiện chất lượng từ hiếm; và một thủ tục đơn giản dựa trên tỉ lệ song-gram/đơn-gram đủ để học vector cho cụm từ có nghĩa riêng, mở rộng Skip-gram từ "vector cho từ" sang "vector cho đơn vị ngôn ngữ linh hoạt". Phát hiện phụ về tính chất cộng vector theo từng phần tử là một gợi ý sớm cho cấu trúc tuyến tính của không gian biểu diễn, chủ đề được khai thác sâu hơn nhiều năm sau trong nghiên cứu interpretability.

Nguồn gốc: https://arxiv.org/abs/1310.4546

Bài 11

Neural Machine Translation by Jointly Learning to Align and Translate (2014)

Tác giả: Dzmitry Bahdanau (Jacobs University Bremen), KyungHyun Cho, Yoshua Bengio (Université de Montréal).
Venue / arXiv: arXiv:1409.0473 (v7, 19/05/2016); mỗi trang paper tự ghi "Published as a conference paper at ICLR 2015".
Nguồn: https://arxiv.org/abs/1409.0473
Chủ đề: mô hình dịch máy nơ-ron (NMT) "RNNsearch": encoder hai chiều (bidirectional RNN) tạo một dãy annotation cho câu nguồn, decoder mỗi bước tính một context vector riêng bằng phép (soft-)align có trọng số học được, giải quyết nút cổ chai "nén cả câu vào một vector cố định" của encoder-decoder cơ bản. Đây là bài gốc đưa cơ chế attention vào NMT.
Vị trí trong lộ trình: Đọc cùng bài 12 về encoder–decoder cơ bản; bài này bổ sung cơ chế attention. Bài 13 khảo sát các đơn vị hồi quy có cổng.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Dịch máy nơ-ron (neural machine translation, NMT) là hướng mới nổi năm 2014 (Kalchbrenner và Blunsom 2013; Sutskever và cộng sự 2014; Cho và cộng sự 2014b), khác hẳn dịch máy thống kê truyền thống dựa trên cụm từ (phrase-based SMT, Koehn và cộng sự 2003) vốn gồm nhiều thành phần nhỏ được tinh chỉnh riêng rẽ (mô hình dịch, mô hình ngôn ngữ, tái sắp xếp từ...). NMT thay bằng **một mạng nơ-ron lớn duy nhất**, đọc câu nguồn và sinh ra câu dịch, huấn luyện đồng thời (end-to-end).

Phần lớn mô hình NMT thời điểm đó thuộc họ **encoder-decoder**: một encoder đọc và nén câu nguồn $x = (x_1, \ldots, x_{T_x})$ thành **một vector có độ dài cố định** c, rồi decoder sinh câu dịch từ vector đó. Theo công thức tổng quát (mục 2.1, công thức 1-2):

$$h_t = f(x_t, h_{t-1}), \qquad c = q(\{h_1, \ldots, h_{T_x}\})$$

$$p(y) = \prod_{t=1}^{T} p(y_t \mid \{y_1, \ldots, y_{t-1}\}, c), \qquad p(y_t \mid \ldots) = g(y_{t-1}, s_t, c)$$

với f, q, g là các hàm phi tuyến (thường RNN/LSTM). Sutskever và cộng sự (2014, bài 12) dùng LSTM cho f và $q(\{h_1, \ldots, h_T\}) = h_T$ (lấy trạng thái ẩn cuối cùng làm c).

Vấn đề: phải nén toàn bộ thông tin câu nguồn, dù dài hay ngắn, vào **một vector kích thước cố định**. Cho và cộng sự (2014b) chỉ ra thực nghiệm rằng hiệu năng của encoder-decoder cơ bản **giảm nhanh** khi câu nguồn dài hơn, đặc biệt dài hơn các câu trong tập huấn luyện. Bài này đặt giả thuyết: chính cái "cổ chai vector cố định" này là nguyên nhân, và đề xuất một kiến trúc không ép toàn câu vào một vector, mà **giữ một dãy vector** rồi chọn linh hoạt phần liên quan khi sinh từng từ dịch.

2. Ý tưởng chính

Mô hình đề xuất, gọi là **RNNsearch**, mở rộng encoder-decoder bằng cách:

1. Encoder là một **RNN hai chiều (bidirectional RNN, BiRNN)**: đọc câu nguồn theo cả hai hướng, mỗi vị trí từ x_j nhận một **annotation** h_j tóm tắt thông tin xung quanh nó (cả từ trước và từ sau), không chỉ thông tin từ đầu câu đến vị trí đó như RNN một chiều.
2. Decoder, tại mỗi bước sinh từ y_i, không dùng một c cố định cho cả câu, mà tính một **context vector riêng** c_i bằng tổng có trọng số của toàn bộ annotation $\{h_1, \dots, h_{T_x}\}$, trọng số α_{ij} học được từ một mạng con nhỏ (gọi là **mô hình alignment**, lượng giá mức độ "khớp" giữa trạng thái decoder trước đó s_{i-1} và annotation h_j).

Vì trọng số α_{ij} được tính bằng softmax trên điểm khớp, đây chính là một phép **(soft-)align**: decoder tự "tìm" (search) phần câu nguồn liên quan nhất cho mỗi từ đích, không cần một bộ aligner rời rạc như SMT truyền thống, và toàn bộ cơ chế này **huấn luyện cùng gradient** với phần dịch. Paper gọi đây là "jointly learning to align and translate". Về sau khái niệm α_{ij} này được gọi chung là **attention**.

Vì context vector không còn cố định cho cả câu, encoder không bị ép nén hết thông tin vào một vector; thông tin "trải" trên dãy annotation và được decoder truy xuất linh hoạt theo nhu cầu từng từ đích, nên mô hình được kỳ vọng xử lý câu dài tốt hơn hẳn.

3. Phương pháp chi tiết

3.1 Decoder và context vector (mục 3.1, công thức 4-6)

Với mô hình mới, mỗi xác suất có điều kiện trong công thức (2) được định nghĩa lại:

$$p(y_i \mid y_1, \dots, y_{i-1}, x) = g(y_{i-1}, s_i, c_i), \qquad s_i = f(s_{i-1}, y_{i-1}, c_i) \tag{4}$$

khác bản gốc: xác suất giờ có điều kiện theo một **context vector riêng** c_i **cho mỗi từ đích** y_i, không phải một c chung. c_i là tổng có trọng số của các annotation (công thức 5):

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$$

Trọng số α_{ij} tính bằng softmax trên điểm khớp e_{ij} (công thức 6):

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T_x} \exp(e_{ik})}, \qquad e_{ij} = a(s_{i-1}, h_j)$$

a là **mô hình alignment**, một mạng nơ-ron feedforward nhỏ, huấn luyện cùng lúc với toàn hệ thống (không coi alignment là biến ẩn như một số công trình SMT trước đó, mà tính trực tiếp một alignment "mềm" (soft) để gradient lan truyền qua được). Có thể hiểu c_i như kỳ vọng (expectation) của annotation trên phân phối các khả năng align, với α_{ij} là xác suất "từ đích y_i được dịch từ (hoặc align với) từ nguồn x_j". Đây chính là cơ chế attention trong decoder: decoder "quyết định" phần nào của câu nguồn cần chú ý khi sinh mỗi từ, nhờ đó encoder không còn phải nén hết thông tin vào một vector cố định.

3.2 Encoder: BiRNN tạo annotation (mục 3.2, công thức 7)

RNN một chiều thông thường $h_t = f(x_t, h_{t-1})$ chỉ tóm tắt từ đầu câu đến vị trí t. Để mỗi annotation chứa cả thông tin phía sau, dùng **BiRNN** (Schuster và Paliwal 1997): một RNN thuận đọc $x_1 \to x_{T_x}$ cho dãy trạng thái thuận $(\overrightarrow{h}_1, \dots, \overrightarrow{h}_{T_x})$, một RNN nghịch đọc $x_{T_x} \to x_1$ cho dãy trạng thái nghịch $(\overleftarrow{h}_1, \dots, \overleftarrow{h}_{T_x})$. Annotation cho từ x_j là ghép hai trạng thái (công thức 7):

$$h_j = [\overrightarrow{h}_j^{\top}; \overleftarrow{h}_j^{\top}]^{\top}$$

Vì RNN có xu hướng biểu diễn tốt hơn các input gần đây, h_j sẽ "tập trung" quanh vị trí x_j nhưng vẫn mang thông tin toàn câu qua hai chiều.

3.3 Đơn vị hồi quy có cổng dùng trong decoder (Phụ lục A.1.1)

Cả encoder và decoder dùng một "gated hidden unit" do Cho và cộng sự (2014a) đề xuất (tiền thân gần của GRU, xem bài 13), không dùng tanh/sigmoid trần hay LSTM đầy đủ. Với trạng thái decoder:

$$s_i = (1 - z_i) \circ s_{i-1} + z_i \circ \tilde{s}_i$$

$$\tilde{s}_i = \tanh\left(We(y_{i-1}) + U[r_i \circ s_{i-1}] + Cc_i\right)$$

$$z_i = \sigma(W_z e(y_{i-1}) + U_z s_{i-1} + C_z c_i), \qquad r_i = \sigma(W_r e(y_{i-1}) + U_r s_{i-1} + C_r c_i)$$

với $e(\cdot)$ là tầng embedding, z_i là cổng cập nhật (update gate), r_i là cổng đặt lại (reset gate), $\circ$ là tích từng phần tử. Mô hình trở về đúng RNN Encoder-Decoder gốc (Cho và cộng sự 2014a) nếu cố định $c_i = \overrightarrow{h}_{T_x}$ (dùng trạng thái cuối cố định thay vì attention).

3.4 Mô hình alignment và kích thước tham số (Phụ lục A.1.2, A.2.3)

$$a(s_{i-1}, h_j) = v_a^\top \tanh(W_a s_{i-1} + U_a h_j)$$

với $W_a \in \mathbb{R}^{n \times n}$, $U_a \in \mathbb{R}^{n \times 2n}$, $v_a \in \mathbb{R}^n$ (n là số chiều hidden). Vì $U_a h_j$ không phụ thuộc i, có thể **tính trước (precompute)** một lần cho cả câu, giảm chi phí vì mô hình alignment phải được đánh giá $T_x \times T_y$ lần mỗi câu.

Kích thước mô hình dùng trong mọi thí nghiệm (Phụ lục A.2.3): hidden size $n = 1000$, embedding $m = 620$, tầng maxout ở output $l = 500$, hidden size của mô hình alignment $n' = 1000$.

Ví dụ số tự tính: số tham số riêng của mô hình alignment, với $n = n' = 1000$:

$$|W_a| = n' \times n = 1.000 \times 1.000 = 1 \text{ triệu}, \quad |U_a| = n' \times 2n = 1.000 \times 2.000 = 2 \text{ triệu}$$

Tổng $W_a + U_a \approx 3$ triệu tham số (tự tính, bỏ qua v_a chỉ 1000 phần tử, không đáng kể). Đây là phần tham số **thêm vào** so với RNNencdec gốc để có được khả năng attention, một tỉ lệ nhỏ so với tổng tham số encoder/decoder (các ma trận W, U, Wz, Uz, Wr, Ur của GRU-like unit, mỗi ma trận cấp $n \times n$ hoặc $n \times m$, tức hàng triệu tham số mỗi ma trận).

4. Thí nghiệm và kết quả chính

Thiết lập (mục 4): dịch Anh-Pháp, corpus song ngữ WMT'14 (Europarl 61M từ, news commentary 5.5M, UN 421M, hai corpus crawl 90M và 272.5M, tổng 850M từ), rút gọn về **348 triệu từ** bằng phương pháp chọn dữ liệu (Axelrod và cộng sự 2011), theo đúng quy trình của Cho và cộng sự (2014a) để so sánh công bằng. Không dùng dữ liệu đơn ngữ thêm. Tập dev ghép news-test-2012 + news-test-2013; tập test là news-test-2014 (**3003 câu**). Từ điển: shortlist **30.000 từ phổ biến nhất** mỗi ngôn ngữ, từ ngoài danh sách map về [UNK].

Hai kiến trúc huấn luyện mỗi loại hai lần (câu tới 30 từ, câu tới 50 từ): **RNNencdec** (Cho và cộng sự 2014a, baseline không attention) và **RNNsearch** (đề xuất). Cả hai encoder/decoder 1000 hidden units; RNNsearch encoder là BiRNN (1000 units mỗi chiều). Huấn luyện bằng minibatch SGD + **Adadelta** (minibatch 80 câu, khoảng 5 ngày mỗi mô hình); decode bằng beam search.

Bảng 1 (BLEU trên news-test-2014):

Mô hình	All	No UNK
RNNencdec-30	13.93	24.19
RNNsearch-30	21.50	31.44
RNNencdec-50	17.82	26.71
RNNsearch-50	26.75	34.16
RNNsearch-50* (huấn luyện lâu hơn)	28.45	36.15
Moses (phrase-based)	33.30	35.63

RNNsearch vượt RNNencdec trong **mọi** trường hợp; RNNsearch-30 còn vượt RNNencdec-50 (21.50 > 17.82) dù huấn luyện trên câu ngắn hơn. Trên cột "No UNK" (chỉ câu không có từ ngoài từ điển), RNNsearch-50* (36.15) thậm chí **vượt** Moses (35.63), và paper nhấn mạnh đây đáng kể vì Moses còn dùng thêm một corpus đơn ngữ 418M từ mà RNNsearch không có. Tuy nhiên trên cột "All" (toàn bộ câu test, kể cả câu có UNK), RNNsearch-50* (28.45) vẫn **kém** Moses (33.30) khá rõ: tuyên bố "hiệu năng ngang phrase-based system" của paper chỉ đúng với điều kiện loại câu có từ chưa biết, cần đọc kỹ cột nào đang được so sánh.

Hình 2 (BLEU theo độ dài câu): RNNencdec giảm mạnh khi câu dài hơn; cả RNNsearch-30 và RNNsearch-50 bền hơn nhiều theo độ dài, đặc biệt RNNsearch-50 **không suy giảm** dù câu dài 50 từ hoặc hơn.

Phân tích định tính (mục 5.2): hình 3 vẽ ma trận trọng số α_{ij} (attention map) cho 4 câu mẫu, cho thấy alignment phần lớn gần đường chéo (đơn điệu, monotonic) nhưng có những đoạn không đơn điệu rõ ràng, ví dụ cụm "European Economic Area" dịch thành "zone économique européenne": mô hình align đúng "zone" với "Area" (nhảy qua "European" và "Economic") rồi quay lại hoàn thiện cụm. Một ví dụ khác cho thấy ưu điểm của soft-alignment so với hard-alignment: từ "the" trong "the man" dịch thành "l"' (không phải "le/la/les" cố định), vì mô hình "nhìn" đồng thời cả "the" và "man" khi quyết định, điều hard-alignment (ánh xạ 1-1) không làm được tự nhiên.

Ví dụ câu dài (mục 5.2.2): với câu nguồn trên 30 từ, RNNencdec-50 dịch đúng đoạn đầu nhưng **lệch nghĩa** từ khoảng giữa câu trở đi (ví dụ thay "based on his status as a health care worker at a hospital" bằng "dựa trên tình trạng sức khỏe của ông"), còn RNNsearch-50 giữ đúng nghĩa toàn câu. Paper đưa 3 ví dụ cụ thể minh họa hiện tượng này trong Phụ lục C.

Bảng 2 (thống kê huấn luyện): số lần cập nhật, số epoch, giờ huấn luyện, loại GPU, train/dev NLL cho từng mô hình, ví dụ RNNsearch-50: $2{,}88 \times 10^5$ lần cập nhật, 2.2 epoch, 111 giờ, Quadro K-6000, train NLL 40.7, dev NLL 38.1.

5. Phân tích / ablation

- **RNNsearch so với RNNencdec là ablation chính** của bài (bật/tắt attention, mọi thứ khác giữ nguyên cùng dữ liệu, cùng quy trình huấn luyện): chênh BLEU trên All, 50-từ: $26.75 - 17.82 = 8{,}93$ điểm (tự tính), một mức cải thiện rất lớn cho một thay đổi kiến trúc đơn lẻ.
- **Độ dài câu huấn luyện (30 so với 50 từ):** cả hai kiến trúc được huấn luyện lại hoàn toàn riêng cho mỗi giới hạn độ dài (không phải cùng một mô hình test ở hai độ dài); RNNsearch-50 tốt hơn RNNsearch-30 (26.75 so với 21.50 All), cho thấy cho phép câu huấn luyện dài hơn tận dụng được khả năng của attention tốt hơn, trong khi RNNencdec cải thiện ít hơn theo tỉ lệ khi tăng độ dài cho phép (17.82 so với 13.93).
- **Huấn luyện lâu hơn (RNNsearch-50 so với RNNsearch-50*):** cùng kiến trúc, huấn luyện tới khi dev-set ngừng cải thiện (5.0 epoch, 252 giờ so với 2.2 epoch, 111 giờ) cho thêm 1.7 điểm BLEU (28.45 so với 26.75, All) nhưng tốn hơn gấp đôi thời gian huấn luyện, một đánh đổi compute/chất lượng rõ ràng.

- **Lưu ý về Bảng 2 (train NLL):** RNNenc-30 có train NLL thấp hơn (28.1) so với RNNenc-50 (44.0), nhưng chú thích bảng nói rõ "độ dài câu khác nhau" giữa hai tập, nên **không thể so sánh trực tiếp** NLL giữa hai dòng này như một chỉ số chất lượng mô hình; đây là một điểm dễ hiểu nhầm nếu đọc bảng mà bỏ qua chú thích.

6. Giới hạn

Do tác giả nêu:

- Hạn chế còn lại rõ ràng nhất: **xử lý từ hiếm/chưa biết (rare/unknown words)**; mục Kết luận nói thẳng đây là "thử thách để lại cho tương lai".
- Mô hình alignment phải đánh giá $T_x \times T_y$ lần mỗi câu; tác giả tự nhận đây là một hạn chế tiềm năng cho các bài toán khác ngoài dịch máy (nơi câu có thể rất dài), dù với dịch máy (hầu hết câu 15-40 từ) chưa phải vấn đề lớn (mục 6.1, so sánh với Graves 2013).
- So với Graves (2013, tổng hợp chữ viết tay dùng hỗn hợp Gaussian để align), cách align ở đây không bị ràng buộc đơn điệu một chiều, nhưng đổi lại tốn tính toán hơn vì phải tính trọng số cho **mọi** vị trí nguồn với **mọi** vị trí đích.

Nhận xét:

- Tuyên bố "hiệu năng ngang hệ thống phrase-based" chỉ chính xác trên tập con "No UNK" (loại câu có từ ngoài từ điển); trên toàn tập test ("All"), khoảng cách với Moses còn 4-5 điểm BLEU (33.30 so với 28.45 ở bản huấn luyện lâu nhất); người đọc cần phân biệt rõ hai cột khi trích dẫn kết quả.
- Vì mỗi cấu hình (30 từ, 50 từ) huấn luyện mô hình riêng từ đầu, bài không tách bạch được hoàn toàn đâu là lợi ích của "cho phép câu dài hơn trong huấn luyện" và đâu là lợi ích thuần của attention; hai yếu tố này nhập nhằng trong cách thiết kế thí nghiệm (dù xu hướng chung rất rõ nên không ảnh hưởng kết luận chính).
- Chỉ thử nghiệm trên một cặp ngôn ngữ (Anh-Pháp) và một kiến trúc gated-unit cụ thể; không so sánh với lựa chọn LSTM (paper tự nói ở mục 2.1 rằng LSTM "có thể dùng thay" nhưng không thử nghiệm trong bài).

7. Liên hệ với các bài khác

- **ml basics 12** (cùng thư mục, Sequence to Sequence Learning, Sutskever và cộng sự): là baseline khái niệm mà bài này cải tiến (RNNencdec cơ bản, không attention); hai bài ra đời cùng tháng (arXiv 1409.xxxx) nhưng theo hai hướng khác nhau để khắc phục cùng một vấn đề (câu dài): seq2seq đảo thứ tự câu nguồn, bài này thêm attention.
- **ml basics 13** (cùng thư mục, Chung và cộng sự, Empirical Evaluation of Gated Recurrent Neural Networks): đơn vị hồi quy có cổng dùng trong decoder ở Phụ lục A.1.1 chính là GRU (tên gọi "gated recurrent unit" xuất hiện trong bài 13, dẫn lại định nghĩa gần giống từ Cho và cộng sự 2014a mà bài này sử dụng).
- **llm content/01_2017_attention_is_all_you_need.md**: Transformer thay hoàn toàn RNN bằng self-attention, nhưng công thức softmax trọng số $\alpha_{ij} = \text{softmax}(e_{ij})$ và $c_i = \sum_j \alpha_{ij} h_j$ ở bài này chính là tiền thân trực tiếp của attention trong Transformer (khác biệt chính: Transformer dùng tích vô hướng $QK^\top/\sqrt{d}$ thay mạng feedforward $a(\cdot,\cdot)$ để tính điểm khớp).
- **llm content/14_2019_multi_query_attention.md, llm content/15_2023_gqa.md**: các biến thể tối ưu attention cho suy luận (MQA, GQA) đều xây trên nền tảng cơ chế attention được định nghĩa lần đầu ở quy mô NMT trong bài này.
- **interp content/04_2022_induction_heads.md**: hiện tượng "attention head học cách tra cứu token liên quan" trong Transformer hiện đại có cùng tinh thần với α_{ij} ở đây: một trọng số mềm cho biết vị trí nào trong input liên quan đến quyết định hiện tại.

8. Gợi ý áp dụng

Attention cho phép bộ giải mã truy cập các trạng thái nguồn thay vì chỉ dựa vào một vector cố định. Có thể quan sát trọng số attention để khảo sát căn chỉnh, nhưng không nên coi chúng tự động là lời giải thích đầy đủ cho dự đoán.

9. Thí nghiệm gợi ý

1. **Tái lập mini so sánh RNNencdec so với RNNsearch:** cài một bài toán dịch/copy chuỗi đơn giản (ví dụ "sao chép ngược" một dãy số, hoặc dịch giữa hai ngôn ngữ nhân tạo nhỏ) với độ dài chuỗi thay đổi (10, 30, 60 bước). So sánh một encoder-decoder cơ bản (context cố định = trạng thái cuối) với một bản có attention như mục 3.1-3.2. Đo accuracy theo độ dài chuỗi. Kỳ vọng: mô hình có attention giữ accuracy cao hơn hẳn trên chuỗi dài, tái lập định tính Hình 2.
2. **Vẽ attention map trên bài toán toy:** với mô hình attention ở thí nghiệm 1, lưu và vẽ ma trận α_{ij} cho vài ví dụ; kiểm tra mô hình có học được alignment hợp lý (ví dụ gần đường chéo cho bài toán sao chép thuận, hoặc đảo chéo cho bài toán sao chép ngược) không. Diễn giải: nếu attention map "nhiễu" (không có cấu trúc rõ), có thể mô hình chưa huấn luyện đủ hoặc bài toán toy chưa đủ tín hiệu.
3. **Chi phí tính attention theo độ dài:** đo thời gian forward một batch khi tăng T_x (độ dài nguồn) từ 10 đến 200, với mô hình có attention (tính $T_x \times T_y$ điểm khớp) so với không attention. Kỳ vọng: thời gian tăng gần tuyến tính hoặc hơn theo T_x cho attention, gần không đổi cho mô hình không attention; minh họa đánh đổi chi phí/khả năng nêu ở mục 6.
4. **Ablation nhỏ: alignment hidden size n'.** Với mô hình ở thí nghiệm 1, thử $n' \in \{64, 256, 1000\}$ trong mô hình alignment $a(s_{i-1}, h_j)$, đo cả accuracy và tốc độ huấn luyện. Kỳ vọng: n' nhỏ vẫn đủ cho bài toán toy đơn giản (lợi ích giảm dần), gợi ý rằng mô hình alignment không cần lớn bằng hidden chính để hoạt động tốt.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Decoder có context riêng mỗi bước	$p(y_i \mid y_{<i}, x) = g(y_{i-1}, s_i, c_i)$, $s_i = f(s_{i-1}, y_{i-1}, c_i)$ (4)
Context vector	$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j$ (5)
Trọng số attention	$\alpha_{ij} = \exp(e_{ij}) / \sum_k \exp(e_{ik})$, $e_{ij} = a(s_{i-1}, h_j)$ (6)
Mô hình alignment	$a(s_{i-1}, h_j) = v_a^\top \tanh(W_a s_{i-1} + U_a h_j)$
Annotation từ BiRNN	$h_j = [\overrightarrow{h}_j; \overleftarrow{h}_j]$ (7)
Kích thước mô hình	$n = 1000$ (hidden), $m = 620$ (embedding), $l = 500$ (maxout), $n' = 1000$ (alignment)
Tham số alignment (tự tính)	≈ 3 triệu (W_a 1M + U_a 2M)
BLEU tốt nhất (No UNK)	RNNsearch-50*: 36.15, so Moses 35.63
BLEU tốt nhất (All)	RNNsearch-50*: 28.45, so Moses 33.30

11. Kết luận ngắn

RNNsearch thay cổ chai "một vector cố định cho cả câu nguồn" bằng một cơ chế (soft-)align học được, nơi decoder tự chọn tổ hợp trọng số các annotation của encoder hai chiều khi sinh từng từ đích. Cải tiến này giúp mô hình dịch câu dài tốt hơn rõ rệt (không suy giảm theo độ dài như encoder-decoder gốc) và đạt chất lượng gần ngang hệ thống phrase-based SOTA trên tập câu không có từ lạ, dù trên toàn tập test vẫn còn khoảng cách. Cơ chế attention giới thiệu ở đây (công thức 5-6) trở thành khối xây dựng nền tảng cho toàn bộ kiến trúc Transformer sau này.

Nguồn gốc: https://arxiv.org/abs/1409.0473

Bài 12

Sequence to Sequence Learning with Neural Networks (2014)

Tác giả: Ilya Sutskever, Oriol Vinyals, Quoc V. Le (Google).
Venue / arXiv: arXiv:1409.3215 (v3, 14/12/2014); paper không nêu venue trên bìa, nhưng danh sách tài liệu tham khảo của các bài khác (ví dụ bài 11 trong thư mục này) dẫn lại nó là NIPS 2014.
Nguồn: https://arxiv.org/abs/1409.3215
Chủ đề: dùng hai LSTM nhiều tầng (một encoder, một decoder) để ánh xạ một dãy có độ dài tùy ý sang một dãy khác có độ dài tùy ý, không giả định gì về cấu trúc bài toán; áp dụng cho dịch máy Anh-Pháp, đạt BLEU vượt một hệ thống SMT dựa trên cụm từ; phát hiện quan trọng là **đảo thứ tự câu nguồn** giúp huấn luyện dễ hơn nhiều.
Vị trí trong lộ trình: Đọc cùng bài 11 về attention: bài này trình bày encoder–decoder với LSTM và vector ngữ cảnh cố định.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Mạng nơ-ron sâu (DNN) đạt hiệu năng cao trên nhiều bài toán khó (nhận dạng tiếng nói, nhận dạng vật thể) nhưng chỉ áp dụng được cho bài toán có **input và target mã hóa được bằng vector kích thước cố định**. Nhiều bài toán quan trọng không có dạng này: dịch máy, nhận dạng tiếng nói, trả lời câu hỏi đều có thể coi là ánh xạ **dãy sang dãy** với độ dài không biết trước và không cố định.

Có vài hướng liên quan trước đó: Kalchbrenner và Blunsom (2013) là người đầu tiên ánh xạ cả câu nguồn thành một vector (dùng CNN, nhưng mất thứ tự từ); Cho và cộng sự (2014, dẫn là arXiv:1406.1078, chính là "Cho et al. 2014a" trong bài 11) dùng một kiến trúc giống RNN/LSTM để ánh xạ câu thành vector và ngược lại, nhưng tập trung chính vào việc dùng mạng đó làm điểm số phụ trợ (rescoring) cho một hệ SMT có sẵn, không dịch trực tiếp. Graves (2013) giới thiệu cơ chế attention khả vi cho tổng hợp chữ viết tay; Bahdanau và cộng sự (2014, bài 11) áp dụng một biến thể tương tự cho dịch máy.

Bài này đặt câu hỏi đơn giản hơn: liệu một **LSTM lớn, nhiều tầng, không có giả định đặc biệt gì về cấu trúc bài toán**, có thể tự học ánh xạ dãy-sang-dãy tốt đến mức nào, nếu được huấn luyện trực tiếp (không qua rescoring) trên đủ dữ liệu?

2. Ý tưởng chính

Dùng **hai LSTM khác nhau**: một LSTM đọc câu nguồn $(x_1, \dots, x_T)$ từng bước, tạo ra một **vector biểu diễn kích thước cố định** v (trạng thái ẩn cuối cùng); một LSTM thứ hai nhận v làm trạng thái khởi tạo và sinh câu đích $(y_1, \dots, y_{T'})$ từng từ, giống một mô hình ngôn ngữ LSTM thông thường nhưng có điều kiện theo v. Công thức xác suất có điều kiện (công thức 1):

$$p(y_1, \dots, y_{T'} \mid x_1, \dots, x_T) = \prod_{t=1}^{T'} p(y_t \mid v, y_1, \dots, y_{t-1})$$

mỗi $p(y_t | \dots)$ là một softmax trên toàn từ điển đích. Mỗi câu phải kết thúc bằng một token đặc biệt "<EOS>" để mô hình định nghĩa được phân phối trên các dãy có độ dài khác nhau.

Ba lựa chọn thiết kế khác biệt với mô tả "thuần RNN" ở trên:

1. Dùng **hai LSTM riêng** (một cho nguồn, một cho đích) thay vì một LSTM chung, giúp tăng số tham số với chi phí tính toán không đáng kể, và cho phép huấn luyện đồng thời trên nhiều cặp ngôn ngữ dễ hơn.
2. LSTM **sâu (4 tầng)** vượt rõ LSTM nông (shallow).
3. Phát hiện quan trọng nhất: **đảo thứ tự từ của câu nguồn** (không đảo câu đích) trước khi đưa vào mô hình giúp cải thiện mạnh.

3. Phương pháp chi tiết

3.1 LSTM cho bài toán dãy-sang-dãy (mục 2)

RNN chuẩn $h_t = \text{sigm}(W^{hx}x_t + W^{hh}h_{t-1})$, $y_t = W^{yh}h_t$ chỉ dễ áp dụng khi alignment giữa input và output biết trước và độ dài bằng nhau. Với input/output độ dài khác nhau và quan hệ phức tạp, phi đơn điệu, cần một chiến lược khác: dùng một RNN để nén input thành vector cố định, một RNN khác để giải mã vector đó thành output. Vì phụ thuộc xa (long-range dependency) khó huấn luyện bằng RNN chuẩn, bài dùng **LSTM** (Hochreiter và Schmidhuber, 1997), đã biết học tốt các bài toán có phụ thuộc xa.

3.2 Đảo thứ tự câu nguồn (mục 2, mục 3.3)

Thay vì ánh xạ $a, b, c \to \alpha, \beta, \gamma$ (câu nguồn a, b, c, câu đích α, β, γ là bản dịch), mô hình được huấn luyện ánh xạ $c, b, a \to \alpha, \beta, \gamma$ (đảo thứ tự nguồn, giữ nguyên thứ tự đích). Nhờ đó a gần α, b gần β... về khoảng cách trong dãy đã nối, giảm "độ trễ thời gian tối thiểu" (minimal time lag, khái niệm từ Hochreiter và Schmidhuber 1997) giữa các từ tương ứng, giúp SGD "thiết lập liên lạc" (establish communication) giữa nguồn và đích dễ hơn. Khoảng cách **trung bình** giữa các từ tương ứng không đổi khi đảo chỉ câu nguồn, nhưng các từ **đầu câu** (vốn ảnh hưởng dự đoán sớm nhất của decoder) được đưa lại gần nhau nhất.

Kết quả thực nghiệm trực tiếp (mục 3.3): đảo câu nguồn làm **test perplexity giảm từ 5.8 xuống 4.7**, và **BLEU test tăng từ 25.9 lên 30.6**. Đáng chú ý, các tác giả ban đầu dự đoán việc đảo chỉ giúp các từ đầu câu đích được dự đoán tự tin hơn và làm hại các từ cuối câu đích, nhưng thực tế mô hình huấn luyện trên dữ liệu đảo lại **dịch câu dài tốt hơn** mô hình huấn luyện trên dữ liệu không đảo, gợi ý việc đảo giúp sử dụng "bộ nhớ" của LSTM hiệu quả hơn tổng thể, không chỉ cục bộ đầu câu.

3.3 Chi tiết huấn luyện (mục 3.1, 3.4)

- Dữ liệu: WMT'14 Anh-Pháp, tập con **12 triệu câu** (348M từ Pháp, 304M từ Anh, tập "selected" đã làm sạch).
- Từ điển: **160.000 từ phổ biến nhất** cho nguồn (Anh), **80.000 từ** cho đích (Pháp); từ ngoài danh sách thành "UNK".
- Kiến trúc: LSTM 4 tầng, mỗi tầng **1000 cell**, embedding từ **1000 chiều**; vậy LSTM sâu dùng **8000 số thực** để biểu diễn một câu (tự tính: $4 \times 1000 \times 2 = 8000$ nếu tính cả hidden state và cell state của 4 tầng, hoặc đơn giản hơn là 4 tầng × 1000 chiều hidden × 2 (h và c) theo cách paper phát biểu trực tiếp "8000 real numbers").
- LSTM sâu hơn tốt hơn: **mỗi tầng thêm giảm perplexity gần 10%**.
- Softmax ngây thơ (naive) trên 80.000 từ ở mỗi output (không dùng hierarchical softmax như bài 09, 10).
- Tổng **384 triệu tham số**, trong đó **64 triệu là liên kết hồi quy thuần (pure recurrent)**: 32M cho LSTM "encoder" và 32M cho LSTM "decoder".
- Khởi tạo tham số: uniform $[-0.08, 0.08]$.
- SGD không có momentum, learning rate cố định 0.7; sau 5 epoch, giảm nửa learning rate mỗi nửa epoch; tổng huấn luyện **7.5 epoch**.
- Batch 128 câu; gradient chia theo batch size; chặn norm gradient ở ngưỡng 5 (nếu $s = \|g\|_2 > 5$ thì $g \leftarrow 5g/s$).

- Để tránh lãng phí tính toán do câu trong một batch có độ dài rất khác nhau, các câu trong cùng một minibatch được sắp **xấp xỉ cùng độ dài**, cho tốc độ gấp 2 lần.

3.4 Song song hóa (mục 3.5)

Cài đặt C++ đơn-GPU đạt khoảng **1700 từ/giây**, quá chậm; song song trên máy **8-GPU**: mỗi tầng LSTM (4 tầng) chạy trên một GPU riêng, truyền kích hoạt (activation) sang GPU kế tiếp ngay khi tính xong; 4 GPU còn lại song song hóa softmax (mỗi GPU nhân với một ma trận 1000×20000, tức chia 80.000 từ ra làm 4 phần). Tốc độ đạt **6300 từ/giây** (cả Anh và Pháp) với batch 128; huấn luyện tốn khoảng **10 ngày**.

3.5 Giải mã và rescoring (mục 3.2)

Tìm $\hat{T} = \arg\max_T p(T|S)$ (công thức 2) bằng **beam search trái-sang-phải** đơn giản, giữ B giả thuyết (hypothesis) từng phần, mở rộng mỗi giả thuyết bằng mọi từ trong từ điển rồi giữ lại B giả thuyết có log xác suất cao nhất. Thú vị: mô hình hoạt động tốt **ngay cả với beam size 1** (tương đương lấy từ có xác suất cao nhất tại mỗi bước, greedy decoding không backtrack), và beam size 2 đã cho hầu hết lợi ích của beam search. Ngoài dịch trực tiếp, LSTM còn dùng để **rescoring**: tính log xác suất cho mỗi câu trong danh sách 1000-best của một hệ SMT baseline, lấy trung bình đều (even average) với điểm gốc của hệ đó.

4. Thí nghiệm và kết quả chính

Bảng 1 (dịch trực tiếp, test BLEU, ntst14):

Phương pháp	BLEU
Bahdanau và cộng sự [2]	28.45
Hệ SMT baseline [29]	33.30
1 LSTM thuận, beam 12	26.17
1 LSTM đảo, beam 12	30.59
Ensemble 5 LSTM đảo, beam 1	33.00
Ensemble 2 LSTM đảo, beam 12	33.27
Ensemble 5 LSTM đảo, beam 2	34.50
Ensemble 5 LSTM đảo, beam 12	**34.81**

Kết quả chính: ensemble 5 LSTM đảo, beam 12, đạt BLEU **34.81**, vượt baseline SMT (33.30), và paper nhấn mạnh đây là lần đầu một hệ thống **dịch trực tiếp thuần nơ-ron** vượt một hệ SMT dựa trên cụm từ ở quy mô lớn, dù từ điển bị giới hạn 80.000 từ (UNK bị trừng phạt khi tính BLEU, tức mô hình bị thiệt khi gặp từ ngoài danh sách).

Bảng 2 (kết hợp với hệ SMT bằng rescoring):

Phương pháp	BLEU
Hệ SMT baseline [29]	33.30
Cho và cộng sự [5]	34.54
Kết quả tốt nhất WMT'14 [9]	37.0
Rescoring 1000-best bằng 1 LSTM thuận	35.61
Rescoring 1000-best bằng 1 LSTM đảo	35.85
Rescoring bằng ensemble 5 LSTM đảo	**36.5**
Oracle rescoring của 1000-best	khoảng 45

Rescoring bằng ensemble đạt 36.5, kém kết quả tốt nhất WMT'14 (37.0) chỉ 0.5 điểm, trong khi hệ baseline tự nó chỉ 33.30; khoảng cách tới oracle (~45) cho thấy còn nhiều tiềm năng nếu cải thiện n-best list hoặc mô hình.

Hiệu năng theo độ dài câu (mục 3.7, Hình 3): paper "ngạc nhiên" khi thấy LSTM **không suy giảm** đáng kể trên câu dài: gần như không giảm chất lượng với câu dưới 35 từ, chỉ giảm nhẹ ở câu dài nhất. Điều này tương phản với các nghiên cứu trước (Cho và cộng sự 2014b; Bahdanau và cộng sự 2014, bài 11; Pouget-Abadie và cộng sự 2014) báo cáo encoder-decoder cơ bản (RNN/LSTM không đảo) suy giảm rõ theo độ dài câu. Tác giả quy cho mẹo đảo câu nguồn.

Phân tích biểu diễn (mục 3.8, Hình 2): chiếu PCA 2 chiều các trạng thái ẩn LSTM của vài câu/cụm từ cho thấy các câu gần nghĩa nhau được nhóm lại gần nhau trong không gian biểu diễn, biểu diễn **nhạy với thứ tự từ** nhưng khá **bất biến với chuyển đổi chủ động/bị động** (active/passive voice).

5. Phân tích / ablation

- **Đảo câu nguồn là ablation có số liệu rõ nhất:** test perplexity 5.8 → 4.7, BLEU 25.9 → 30.6 (mục 3.3, đo trên thiết lập không beam search lớn, có thể là greedy hoặc beam nhỏ không nêu rõ số beam ở đoạn này).
- **Độ sâu LSTM:** mỗi tầng thêm giảm perplexity gần 10% (mục 3.4), nhưng không có bảng số chi tiết so sánh 1, 2, 3, 4 tầng trong bài, chỉ phát biểu bằng lời.
- **Beam size:** Bảng 1 cho thấy beam 1 (ensemble 5, BLEU 33.00) đã gần với beam 12 (ensemble 5, BLEU 34.50 rồi 34.81 khi tăng từ 2 lên 12), và tăng từ 2 lên 12 chỉ thêm 0.31 điểm (34.50 → 34.81, tự tính), xác nhận tuyên bố "beam size 2 cho hầu hết lợi ích" ở mục 3.2.
- **Ensemble:** so 1 LSTM đảo beam 12 (30.59) với ensemble 5 LSTM đảo beam 12 (34.81), chênh 4.22 điểm BLEU (tự tính: $34.81 - 30.59 = 4{,}22$), cho thấy ensemble đóng góp lớn, tương đương hoặc hơn chính bản thân việc tăng beam size.
- **Rescoring so với dịch trực tiếp:** rescoring (36.5, ensemble) tốt hơn dịch trực tiếp tốt nhất (34.81) 1.69 điểm (tự tính), vì rescoring được hưởng lợi từ việc hệ SMT baseline đã giới hạn không gian tìm kiếm xuống 1000 câu hợp lý trước, giảm gánh nặng tìm kiếm cho LSTM.

6. Giới hạn

Do tác giả nêu:

- Không chạy được thực nghiệm kiểm chứng trực tiếp cho giả thuyết "RNN chuẩn (không LSTM) có thể huấn luyện được trên dữ liệu đã đảo": paper nói "chúng tôi tin RNN chuẩn sẽ dễ huấn luyện hơn khi câu nguồn đã đảo" nhưng **không thử nghiệm**, chỉ là suy đoán (mục Kết luận).
- Không đưa ra giải thích đầy đủ, chỉ có giải thích định tính (minimal time lag) cho hiện tượng đảo câu nguồn hiệu quả (mục 3.3: "chúng tôi không có giải thích đầy đủ cho hiện tượng này").
- Từ điển giới hạn 80.000/160.000 từ, mô hình bị **trừng phạt khi gặp UNK** trong tính BLEU, nên số 34.81 là một **giới hạn dưới** khá khắt khe của khả năng thật.

Nhận xét:

- Mâu thuẫn số liệu nhỏ: mục 3.3 nói đảo câu nguồn tăng BLEU "từ 25.9 lên 30.6", nhưng Bảng 1 ghi "1 LSTM thuận, beam 12: 26.17" và "1 LSTM đảo, beam 12: 30.59", hai cặp số gần nhưng không khớp hệt (25.9 so với 26.17; 30.6 so với 30.59); có thể do mục 3.3 đo ở điều kiện decode khác (ví dụ beam nhỏ hơn hoặc mô hình đơn lẻ khác lần chạy) với Bảng 1, paper không giải thích rõ sự khác biệt này.
- Kiến trúc "hai LSTM riêng" (một cho nguồn, một cho đích) được chọn vì lý do thực dụng (dễ huấn luyện đa ngôn ngữ, chi phí không đáng kể) hơn là có bằng chứng thực nghiệm trực tiếp so với dùng một LSTM chung; không có ablation riêng cho lựa chọn này.

- Mô hình cơ bản (không đảo, không attention) là chính kiểu kiến trúc mà bài 11 (Bahdanau và cộng sự) dùng làm baseline để chỉ ra vấn đề "cổ chai vector cố định"; bài này giải quyết vấn đề câu dài bằng một mẹo dữ liệu (đảo thứ tự) khác hẳn hướng kiến trúc (attention) của bài 11, hai hướng không loại trừ nhau nhưng paper này không thử kết hợp cả hai.

7. Liên hệ với các bài khác

- **ml basics 11** (cùng thư mục, Bahdanau và cộng sự): giải quyết đúng vấn đề "câu dài làm hại encoder-decoder cơ bản" bằng hướng khác (attention, không cần đảo câu nguồn); hai bài nên đọc như một cặp đối chứng.
- **ml basics 13** (cùng thư mục, Chung và cộng sự, so sánh GRU/LSTM/tanh unit): khảo sát trực tiếp câu hỏi "đơn vị hồi quy nào tốt hơn" mà bài này chỉ dùng LSTM mặc định không so sánh.
- **distillation content/15_2016_sequence_level_kd.md**: sequence-level knowledge distillation xây trên chính khung encoder-decoder dịch máy mà bài này giới thiệu (mô hình sinh dãy, beam search, rescoring n-best), áp dụng ý tưởng distillation ở cấp độ toàn câu thay vì từng token.
- **llm content/01_2017_attention_is_all_you_need.md**: Transformer thay cả hai LSTM (encoder, decoder) và cơ chế attention của bài 11 bằng self-attention thuần, loại bỏ hoàn toàn nhu cầu "mẹo đảo câu nguồn" vì self-attention không có thiên lệch thứ tự gần-xa như RNN.

8. Gợi ý áp dụng

Kiến trúc encoder–decoder đặt nền cho việc ánh xạ chuỗi có độ dài khác nhau. Đảo thứ tự câu nguồn và beam search là các lựa chọn thực nghiệm trong bài; hiệu quả và chi phí của chúng cần được đo lại trên tác vụ khác.

9. Thí nghiệm gợi ý

1. **Tái lập hiệu ứng đảo thứ tự trên bài toán toy:** huấn luyện một seq2seq LSTM nhỏ (2 tầng, hidden 128) cho bài toán sao chép hoặc đảo một dãy số có độ dài 20-60, so sánh có/không đảo thứ tự input trước khi đưa vào encoder. Đo loss/accuracy theo độ dài dãy. Kỳ vọng: bản đảo hội tụ nhanh hơn và/hoặc chính xác hơn trên dãy dài, tái lập định tính mục 3.3.
2. **So sánh beam size 1, 2, 5, 10 trên bài toán toy đã huấn luyện ở trên:** đo accuracy/BLEU-giả (hoặc exact-match với bài toán toy) theo beam size. Kỳ vọng: lợi ích giảm mạnh sau beam 2-4, tương tự Bảng 1.
3. **Đo ảnh hưởng độ sâu LSTM:** huấn luyện cùng bài toán toy với LSTM 1, 2, 4 tầng (giữ tổng tham số gần tương đương nếu muốn so công bằng), đo loss. Kỳ vọng: xu hướng giảm loss theo độ sâu, dù trên bài toán toy nhỏ hiệu ứng có thể bão hòa sớm hơn báo cáo "mỗi tầng giảm 10% perplexity" của paper (đo trên bài toán dịch máy thực, quy mô lớn).
4. **Quan sát biểu diễn PCA của trạng thái ẩn cuối:** với mô hình ở thí nghiệm 1, chiếu PCA 2 chiều trạng thái ẩn cuối của vài chục câu toy có cấu trúc gần giống nhau (ví dụ hoán đổi thứ tự hai phần tử), xem các câu "gần nghĩa" có nhóm lại không. Diễn giải: nếu không thấy nhóm rõ, có thể do mô hình toy chưa học đủ hoặc không gian 2D không đủ để thấy cấu trúc, cần nêu rõ hạn chế khi báo cáo.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Mục tiêu xác suất	$p(y_1, \ldots, y_{T'} \mid x_1, \ldots, x_T) = \prod_{t=1}^{T'} p(y_t \mid v, y_1, \ldots, y_{t-1})$ (1)
Giải mã	$\hat{T} = \arg\max_T p(T \mid S)$ (2), bằng beam search trái-sang-phải
Kiến trúc	2 LSTM riêng (encoder, decoder), mỗi LSTM 4 tầng × 1000 cell
Tổng tham số	384 triệu (64 triệu recurrent thuần: 32M encoder + 32M decoder)
Mẹo chính	đảo thứ tự từ câu nguồn (không đảo câu đích)

Khái niệm	Công thức / giá trị
Hiệu ứng đảo	perplexity 5.8 → 4.7; BLEU 25.9 → 30.6
BLEU tốt nhất (dịch trực tiếp)	34.81 (ensemble 5 LSTM đảo, beam 12), so SMT baseline 33.30
BLEU tốt nhất (rescoring)	36.5 (ensemble 5), so WMT'14 SOTA 37.0
Tốc độ (8-GPU)	6300 từ/giây, huấn luyện khoảng 10 ngày

11. Kết luận ngắn

Một LSTM 4 tầng đóng vai trò encoder nén câu nguồn thành một vector cố định, cộng một LSTM 4 tầng khác đóng vai trò decoder sinh câu đích có điều kiện theo vector đó, đạt BLEU 34.81 trên dịch Anh-Pháp, lần đầu vượt một hệ SMT dựa trên cụm từ (33.30) bằng dịch trực tiếp thuần nơ-ron. Phát hiện quan trọng nhất về mặt thực hành là mẹo đơn giản **đảo thứ tự câu nguồn**, giúp SGD học các phụ thuộc xa dễ hơn nhiều mà không cần thay đổi kiến trúc. Hạn chế chính (xử lý câu rất dài, cổ chai vector cố định) sau đó được bài 11 giải quyết bằng một hướng khác (attention), và về sau cả hai bị Transformer (llm 01) thay thế hoàn toàn.

Nguồn gốc: https://arxiv.org/abs/1409.3215

Bài 13

Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling (2014)

Tác giả: Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, Yoshua Bengio (Université de Montréal).
Venue / arXiv: arXiv:1412.3555 (v1, 11/12/2014); paper không nêu venue hội nghị.
Nguồn: https://arxiv.org/abs/1412.3555
Chủ đề: so sánh thực nghiệm ba loại đơn vị hồi quy (tanh thường, LSTM, GRU) trên mô hình hóa dãy (sequence modeling): nhạc đa âm (polyphonic music) và tín hiệu tiếng nói thô; đặt tên và định nghĩa rõ **GRU (gated recurrent unit)** như một biến thể của đơn vị có cổng đã dùng trong bài 11.
Vị trí trong lộ trình: Đọc sau các bài 11–12 về mô hình chuỗi để so sánh các đơn vị hồi quy có cổng.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

RNN gần đây (2014) đạt kết quả tốt trên nhiều bài toán học máy, đặc biệt khi input và/hoặc output có độ dài biến đổi (Sutskever và cộng sự, bài 12; Bahdanau và cộng sự, bài 11, cả hai đạt hiệu năng cạnh tranh với hệ dịch máy đã phát triển lâu năm). Một quan sát của nhóm tác giả: **hầu như không có thành công nào** đạt được với RNN "trần" (vanilla, chỉ tanh/sigmoid); mọi thành công lớn đều dùng một đơn vị hồi quy **có cổng (gated)**, như LSTM (Hochreiter và Schmidhuber, 1997).

Gần đây hơn, Cho và cộng sự (2014) đề xuất một đơn vị hồi quy có cổng khác, gọi là **gated recurrent unit (GRU)** (đây chính là đơn vị dùng trong decoder của bài 11, dù bài 11 không gọi tên riêng), đã dùng trong dịch máy nhưng chưa có so sánh có hệ thống với LSTM ngoài phạm vi dịch máy. LSTM đã được xác lập rõ là tốt cho các bài toán có phụ thuộc dài hạn (long-term dependency); GRU còn mới, chưa rõ nó tổng quát tốt ra sao ngoài dịch máy. Bài này đặt mục tiêu: so sánh có kiểm soát ba loại đơn vị (tanh, LSTM, GRU) trên **mô hình hóa dãy** (sequence modeling): học một phân phối xác suất trên các dãy, bằng tối đa log-likelihood.

2. Ý tưởng chính

So sánh ba loại đơn vị hồi quy, **giữ số tham số xấp xỉ bằng nhau** giữa các mô hình (để loại trừ yếu tố "mô hình lớn hơn thắng"), trên hai nhóm bài toán: mô hình hóa nhạc đa âm (symbolic, binary vector mỗi bước) và mô hình hóa tín hiệu tiếng nói thô (continuous, dùng hỗn hợp Gaussian ở output). Kết luận chính: cả LSTM và GRU **vượt rõ** đơn vị tanh truyền thống, đặc biệt trên bài toán khó hơn (tiếng nói thô); nhưng giữa LSTM và GRU, **không có kết luận dứt khoát** đơn vị nào tốt hơn nói chung, lựa chọn có vẻ phụ thuộc vào tập dữ liệu/bài toán cụ thể.

Về mặt kiến trúc, cả LSTM và GRU chia sẻ một đặc điểm cấu trúc quan trọng so với đơn vị truyền thống: **thành phần cộng (additive)** khi cập nhật trạng thái theo thời gian, tạo "đường tắt" (shortcut path) cho gradient lan truyền ngược qua nhiều bước thời gian mà không bị suy giảm quá nhanh (giảm vấn đề vanishing gradient).

3. Phương pháp chi tiết

3.1 RNN nền (mục 2)

RNN cập nhật trạng thái ẩn hồi quy theo công thức (1)-(2):

$$h_t = \begin{cases} 0 & t = 0 \\ \phi(h_{t-1}, x_t) & \text{khác} \end{cases}, \qquad h_t = g(Wx_t + Uh_{t-1})$$

với g là hàm bị chặn, mượt (sigmoid hoặc tanh). Mô hình sinh (generative) phân phối xác suất trên dãy bằng phân rã có điều kiện (công thức 3):

$$p(x_1, \ldots, x_T) = p(x_1)p(x_2|x_1) \cdots p(x_T|x_1, \ldots, x_{T-1})$$

Khó khăn kinh điển: gradient dễ **biến mất** (hầu hết trường hợp) hoặc **nổ** (hiếm, nhưng hậu quả nặng) khi huấn luyện RNN trên phụ thuộc dài hạn (Bengio và cộng sự 1994). Hai hướng khắc phục: (a) thuật toán tối ưu tốt hơn (gradient clipping, phương pháp bậc hai); (b) **thiết kế lại đơn vị hồi quy** bằng cổng (gating), hướng bài này tập trung.

3.2 LSTM (mục 3.1, công thức 4)

Mỗi đơn vị LSTM thứ j giữ một ô nhớ (memory cell) c_t^j; đầu ra $h_t^j = o_t^j \tanh(c_t^j)$ với cổng output $o_t^j = \sigma(W_o x_t + U_o h_{t-1} + V_o c_t)$ (V_o là ma trận đường chéo). Ô nhớ cập nhật bằng quên một phần nhớ cũ, cộng một phần nhớ mới (công thức 4):

$$c_t^j = f_t^j c_{t-1}^j + i_t^j \tilde{c}_t^j, \qquad \tilde{c}_t^j = \tanh(W_c x_t + U_c h_{t-1})^j$$

với cổng quên (forget gate) $f_t^j = \sigma(W_f x_t + U_f h_{t-1} + V_f c_{t-1})^j$ và cổng input $i_t^j = \sigma(W_i x_t + U_i h_{t-1} + V_i c_{t-1})^j$ (V_f, V_i cũng đường chéo). LSTM có thể **quyết định giữ nguyên** bộ nhớ cũ nhờ các cổng, nên nếu phát hiện đặc trưng quan trọng sớm, có thể mang thông tin đó đi xa mà không bị ghi đè.

3.3 GRU (mục 3.2, công thức 5)

GRU cũng có cổng điều chỉnh luồng thông tin, nhưng **không có ô nhớ riêng**: trạng thái h_t^j là nội suy tuyến tính giữa trạng thái cũ và trạng thái ứng viên mới (công thức 5):

$$h_t^j = (1 - z_t^j) h_{t-1}^j + z_t^j \tilde{h}_t^j$$

với cổng cập nhật (update gate) $z_t^j = \sigma(W_z x_t + U_z h_{t-1})^j$ quyết định mức độ cập nhật. Trạng thái ứng viên tính tương tự đơn vị truyền thống nhưng có cổng reset r_t điều chỉnh lượng trạng thái cũ đưa vào:

$$\tilde{h}_t^j = \tanh\left(Wx_t + U(r_t \odot h_{t-1})\right)^j, \qquad r_t^j = \sigma(W_r x_t + U_r h_{t-1})^j$$

Khi cổng reset gần 0, đơn vị "quên" trạng thái trước, hoạt động như đang đọc ký tự đầu tiên của một dãy mới. Paper ghi chú (footnote 1) rằng công thức reset gate ở đây **khác cách đặt vị trí gốc** của Cho và cộng sự (2014) ($\tilde{h}_t = \tanh(Wx_t + r_t \odot (Uh_{t-1}))$, reset nhân sau khi nhân U, không phải trước), nhưng hai cách thực nghiệm sơ bộ cho hiệu năng tương đương.

3.4 So sánh cấu trúc LSTM so với GRU (mục 3.3)

Cả hai chia thành phần **cộng** khi cập nhật (c_t hoặc h_t = một phần giữ lại cộng một phần mới), khác RNN truyền thống **ghi đè hoàn toàn** mỗi bước. Thành phần cộng này tạo đường tắt giúp lan truyền gradient không suy giảm quá nhanh. Khác biệt: (a) LSTM có cổng **output** riêng kiểm soát mức độ bộ nhớ được "phơi bày" (expose) ra ngoài; GRU phơi bày toàn bộ trạng thái, không kiểm soát riêng; (b) LSTM tính nội dung nhớ mới **độc lập** với cổng quên (hai cổng input/forget riêng biệt); GRU dùng **cùng một cổng** z_t cho cả việc giữ trạng

thái cũ và trộn trạng thái mới (cổng duy nhất, buộc hai quyết định "giữ bao nhiêu cũ" và "thêm bao nhiêu mới" phải bù nhau, tổng luôn bằng 1).

3.5 Thiết lập thí nghiệm (mục 4)

- **Nhạc đa âm (symbolic):** 3 tập theo văn bản chính ("ba tập dữ liệu nhạc đa âm") nhưng Bảng 2 thực tế liệt kê **4** tập: Nottingham, JSB Chorales, MuseData, Piano-midi (vector nhị phân 93, 96, 105, 108 chiều tương ứng theo thứ tự nêu trong văn bản). Output dùng sigmoid logistic.
- **Tiếng nói thô (Ubisoft, nội bộ):** mỗi bước nhìn 20 mẫu liên tiếp để dự đoán 10 mẫu kế tiếp; Ubisoft A (dãy dài 500, 7230 dãy), Ubisoft B (dãy dài 8000, 800 dãy). Output dùng hỗn hợp Gaussian 20 thành phần.
- Ba mô hình mỗi bài toán: LSTM-RNN, GRU-RNN, tanh-RNN, **kích cỡ chọn để số tham số xấp xỉ bằng nhau** (Bảng 1): nhạc đa âm khoảng 19.8-20.2K tham số (LSTM 36 units, GRU 46 units, tanh 100 units); tiếng nói khoảng 168-169K tham số (LSTM 195 units, GRU 227 units, tanh 400 units).
- Huấn luyện bằng **RMSProp**, weight noise độ lệch chuẩn cố định 0.075, rescale norm gradient về 1 nếu vượt 1, chọn learning rate tốt nhất trong 10 giá trị log-uniform mẫu từ $\mathcal{U}(-12, -6)$, early-stop theo validation.

Tự tính (minh họa chi phí tham số của cổng): với cùng ngân sách tham số cho bài toán nhạc đa âm (~20K), tanh-RNN dùng 100 unit còn LSTM chỉ dùng 36 unit, tỉ lệ $100/36 \approx 2{,}78$ (tự tính); tương tự cho bài toán tiếng nói, GRU dùng 227 unit so với LSTM 195 unit, tỉ lệ $227/195 \approx 1{,}16$ (tự tính). Hai tỉ lệ này minh họa trực tiếp lý do Bảng 1 chọn số unit khác nhau: mỗi unit LSTM/GRU "đắt" hơn một unit tanh (vì có thêm ma trận cổng $W_f, U_f, W_i, U_i, \ldots$), nên để giữ tổng số tham số bằng nhau, mô hình có cổng phải dùng ít unit hơn hẳn so với tanh, và LSTM (nhiều cổng hơn GRU: input, forget, output so với chỉ update, reset) phải dùng ít unit hơn cả GRU.

4. Thí nghiệm và kết quả chính

Bảng 2 (negative log-probability trung bình, thấp hơn tốt hơn):

Tập dữ liệu	tanh (train/test)	GRU (train/test)	LSTM (train/test)
Nottingham	3.22 / 3.13	2.79 / 3.23	3.08 / 3.20
JSB Chorales	8.82 / 9.10	6.94 / 8.54	8.15 / 8.67
MuseData	5.64 / 6.23	5.06 / 5.99	5.18 / 6.23
Piano-midi	5.64 / 9.03	4.93 / 8.82	6.49 / 9.03
Ubisoft A	6.29 / 6.44	2.31 / 3.59	1.44 / 2.70
Ubisoft B	7.61 / 7.62	0.38 / 0.88	0.80 / 1.26

Trên **nhạc đa âm**: GRU-RNN tốt nhất (test NLL thấp nhất) trên JSB Chorales, MuseData, Piano-midi; LSTM hơi tốt hơn GRU trên Nottingham (3.20 so với 3.23). Khoảng cách giữa ba loại khá hẹp trên nhạc đa âm (tất cả trong khoảng 3-9 NLL, chênh lệch vài phần mười đến một).

Trên **tiếng nói thô** (bài toán khó hơn, dãy dài hơn nhiều): cả LSTM và GRU vượt tanh **rất rõ** (ví dụ Ubisoft A test: tanh 6.44 so với LSTM 2.70, GRU 3.59; tự tính chênh lệch tanh so LSTM $6{,}44 - 2{,}70 = 3{,}74$, một khoảng cách lớn so với thang đo NLL). Giữa LSTM và GRU: LSTM tốt nhất trên Ubisoft A (test 2.70 so với GRU 3.59); GRU tốt nhất trên Ubisoft B (test 0.88 so với LSTM 1.26).

Đường học (Hình 2, 3): trên nhạc đa âm, GRU-RNN tiến bộ nhanh hơn cả về số lần cập nhật và thời gian CPU thực. Trên Ubisoft: tanh-RNN có chi phí tính toán mỗi lần cập nhật nhỏ nhất nhưng tiến bộ rất ít mỗi lần cập nhật và dừng lại ở mức tệ hơn hẳn; hai mô hình có cổng tiến bộ đều và tốt hơn nhiều dù mỗi lần cập nhật đắt hơn.

5. Phân tích / ablation

- **Đơn vị có cổng so với tanh truyền thống** là ablation chính và rõ nhất của bài: trên mọi 6 tập dữ liệu, cả LSTM và GRU đều bằng hoặc vượt tanh-RNN (ngoại trừ không có trường hợp tanh thắng rõ cả hai); khoảng cách này **lớn hơn nhiều** trên tiếng nói thô so với nhạc đa âm, gợi ý lợi ích của cổng tăng theo độ khó/độ dài phụ thuộc của bài toán.
- **LSTM so với GRU:** không có bên nào thắng tuyệt đối trên cả 6 tập (GRU thắng 2/4 tập nhạc + 1/2 tập tiếng nói; LSTM thắng 2/4 tập nhạc + 1/2 tập tiếng nói, tính theo test NLL thấp nhất mỗi dòng). Paper tự kết luận "không thể đưa ra kết luận chắc chắn", và nói lựa chọn loại cổng có thể phụ thuộc tập dữ liệu/bài toán cụ thể.
- **Số tham số được giữ cố định xấp xỉ bằng nhau** (Bảng 1) là thiết kế thí nghiệm quan trọng: nếu không làm vậy, một mô hình có thể thắng chỉ vì nhiều tham số hơn, không phải vì cấu trúc cổng tốt hơn; đây là điểm mạnh về phương pháp luận của bài, dù bản thân vẫn chỉ là một lựa chọn kích thước, không phải một quét (sweep) đầy đủ theo kích thước.
- Paper tự gọi các thí nghiệm là **"sơ bộ" (preliminary)** ở phần Kết luận, và đề xuất cần thí nghiệm kỹ hơn trong tương lai để tách riêng góp phần của từng thành phần cổng (ví dụ tắt riêng cổng output của LSTM, hoặc tắt riêng cổng reset của GRU) nhưng bài này **không tự làm** loại ablation tách thành phần đó.

6. Giới hạn

Do tác giả nêu:

- "Chúng tôi coi các thí nghiệm trong bài này là sơ bộ"; cần nghiên cứu kỹ hơn để hiểu **vì sao** đơn vị có cổng giúp học tốt hơn, và tách riêng đóng góp của từng thành phần cổng trong LSTM/GRU.
- Không đưa ra kết luận dứt khoát giữa LSTM và GRU; kết quả có thể phụ thuộc mạnh vào tập dữ liệu và bài toán cụ thể (mục 6, Kết luận).
- Các mô hình được **cố ý làm nhỏ** để tránh overfitting gây nhiễu phép so sánh (mục 4.2); nghĩa là các con số NLL ở đây không đại diện cho giới hạn trên về chất lượng có thể đạt với mô hình lớn hơn.

Nhận xét:

- Văn bản mục 4.1 viết "ba tập dữ liệu nhạc đa âm" (polyphonic music) nhưng liệt kê **bốn** tên tập (Nottingham, JSB Chorales, MuseData, Piano-midi) và Bảng 2 cũng có 4 dòng nhạc; đây là một điểm không khớp nhỏ giữa lời văn và số liệu, không ảnh hưởng kết luận nhưng cần ghi chú khi trích dẫn.
- Reset gate dùng trong bài **khác vị trí** so với định nghĩa GRU gốc của Cho và cộng sự (2014) (footnote 1); nghĩa là "GRU" trong bài này là một biến thể, không hoàn toàn giống định nghĩa đầu tiên dùng trong bài 11. Khi so sánh hai bài, cần lưu ý đây có thể là hai công thức GRU hơi khác nhau (dù paper nói "hiệu năng tương đương nhau" trong thử sơ bộ).
- Thiếu thông tin chi tiết về dataset Ubisoft (dữ liệu nội bộ, không công khai), nên khó tái lập chính xác phần tiếng nói của bài, chỉ phần nhạc đa âm là tái lập được dễ dàng (dữ liệu công khai từ Boulanger-Lewandowski và cộng sự 2012).

7. Liên hệ với các bài khác

- **ml basics 11** (cùng thư mục, Bahdanau và cộng sự): dùng chính loại đơn vị có cổng này (gọi là "gated hidden unit", không đặt tên GRU) trong decoder, nhưng không so sánh với LSTM hay tanh; bài này lấp khoảng trống đó.
- **ml basics 12** (cùng thư mục, Sutskever và cộng sự): chọn LSTM mặc định không giải thích lý do so với GRU; bài này cho thấy lựa chọn đó không hẳn tối ưu tuyệt đối (GRU thắng trên một số tập trong bài này).

- Các kiến trúc hồi quy có cổng hiện đại tiếp tục khai thác ý tưởng cập nhật trạng thái có điều khiển, dù khác đáng kể các mô hình được so sánh trong bài.

8. Gợi ý áp dụng

So sánh GRU và LSTM cần kiểm soát số tham số, dữ liệu và quy trình điều chỉnh. Kết quả trên các bài toán của bài báo không xác lập một kiến trúc luôn tốt hơn trong mọi điều kiện.

9. Thí nghiệm gợi ý

1. **Tái lập mini Bảng 2:** lấy một tập nhạc đa âm công khai nhỏ (ví dụ JSB Chorales, có sẵn trong nhiều thư viện) hoặc một bài toán mô hình hóa dãy đơn giản tự tạo (ví dụ dự đoán bước tiếp theo của một dãy sin nhiễu rời rạc hóa); huấn luyện ba mô hình tanh-RNN, LSTM, GRU với số tham số xấp xỉ bằng nhau (theo công thức tham số mỗi loại, tính tay rồi chọn số unit tương ứng). Đo train/test NLL. Kỳ vọng: cả LSTM, GRU vượt tanh rõ, khớp định tính Bảng 2.
2. **Đo tốc độ hội tụ theo thời gian CPU/GPU thực (tái lập Hình 2, 3):** trên thí nghiệm 1, vẽ learning curve theo cả số bước cập nhật và theo thời gian thực. Kỳ vọng: GRU có xu hướng hội tụ nhanh hơn theo thời gian thực nếu chi phí mỗi bước GRU thấp hơn LSTM (GRU có ít cổng hơn LSTM, ít phép nhân ma trận hơn mỗi bước).
3. **Ablation tách cổng (mở rộng việc paper tự nhận chưa làm):** từ cài đặt GRU ở thí nghiệm 1, thử một biến thể "khóa cứng" cổng reset $r_t = 1$ luôn (bỏ chức năng reset) và một biến thể khóa cứng cổng update $z_t = 0.5$ cố định (không học). Đo NLL. Kỳ vọng: cả hai biến thể "cụt cổng" đều kém hơn GRU đầy đủ, định lượng được phần đóng góp riêng của mỗi cổng mà bài gốc chỉ nêu định tính.
4. **Đo độ dài phụ thuộc chịu được:** trên một bài toán toy "ghi nhớ bit đầu tiên của dãy sau k bước" (copy task cổ điển), tăng k dần, đo tới k nào mỗi mô hình (tanh, LSTM, GRU) còn học được (accuracy trên ngưỡng, ví dụ 90%). Kỳ vọng: tanh thất bại ở k nhỏ hơn hẳn LSTM/GRU, minh họa trực tiếp vai trò chống vanishing gradient của cổng.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
RNN truyền thống	$h_t = g(Wx_t + Uh_{t-1})$ (2)
LSTM: ô nhớ	$c_t^j = f_t^j c_{t-1}^j + i_t^j \tilde{c}_t^j$ (4), $h_t^j = o_t^j \tanh(c_t^j)$
GRU: trạng thái	$h_t^j = (1 - z_t^j) h_{t-1}^j + z_t^j \tilde{h}_t^j$ (5)
GRU: ứng viên	$\tilde{h}_t^j = \tanh(Wx_t + U(r_t \odot h_{t-1}))^j$
Khác biệt chính LSTM/GRU	LSTM có cổng output riêng; GRU dùng 1 cổng cho cả giữ/trộn trạng thái
Thiết kế thí nghiệm	số tham số xấp xỉ bằng nhau giữa 3 loại đơn vị mỗi bài toán
Kết quả chính	LSTM, GRU > tanh rõ trên mọi tập; LSTM so GRU không có bên thắng tuyệt đối
Tập dữ liệu	Nottingham, JSB Chorales, MuseData, Piano-midi (nhạc); Ubisoft A, B (tiếng nói)

11. Kết luận ngắn

So sánh có kiểm soát (cùng số tham số xấp xỉ) giữa tanh-RNN, LSTM-RNN, GRU-RNN trên 6 tập dữ liệu mô hình hóa dãy cho thấy: đơn vị có cổng (LSTM, GRU) luôn bằng hoặc vượt tanh truyền thống, đặc biệt rõ trên bài toán khó (tiếng nói thô, dãy dài); nhưng giữa LSTM và GRU, kết quả chia đều, không bên nào thắng tuyệt đối trên mọi tập, nên lựa chọn nên dựa trên thực nghiệm cho từng bài toán cụ thể. Bài tự coi đây là nghiên cứu sơ bộ, mở đường cho các so sánh/ablation sâu hơn về từng thành phần cổng trong các công trình sau.

Nguồn gốc: https://arxiv.org/abs/1412.3555

Bài 14

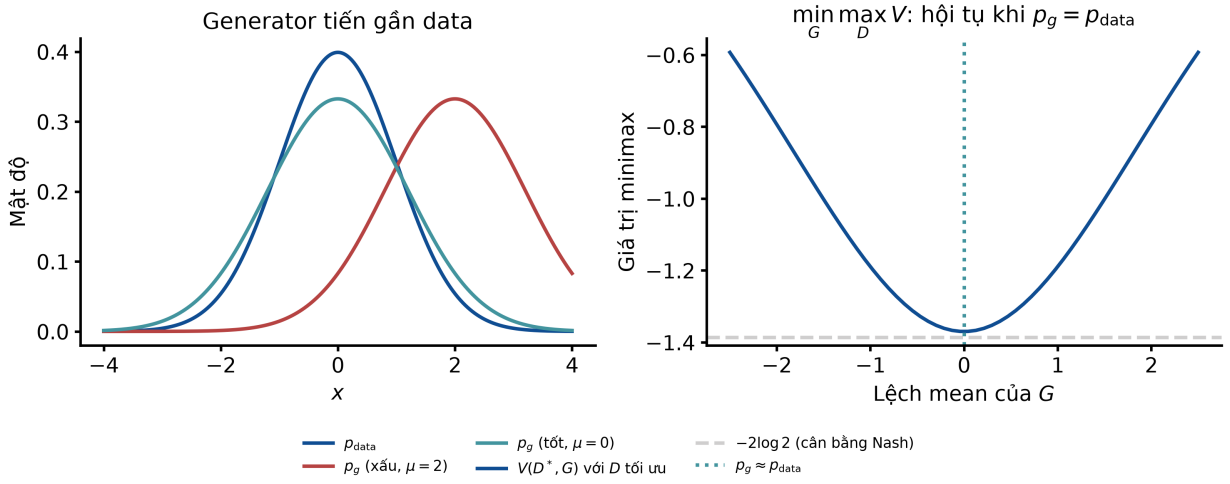
Generative Adversarial Networks (2014)

Tác giả: Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, Yoshua Bengio (Université de Montréal).
Venue / arXiv: arXiv:1406.2661 (v1, 10/06/2014); paper không nêu venue hội nghị. Tiêu đề in trên PDF là "Generative Adversarial Nets" (không có "Networks"); tên gốc chính xác trên bài là "Nets".
Nguồn: https://arxiv.org/abs/1406.2661
Chủ đề: khung huấn luyện mô hình sinh (generative model) bằng một trò chơi minimax hai người chơi giữa một mạng sinh G (cố gắng tạo mẫu giống dữ liệu thật) và một mạng phân biệt D (cố gắng phân biệt mẫu thật/giả); chứng minh điểm tối ưu toàn cục là G tái tạo đúng phân phối dữ liệu, $D \equiv 1/2$ mọi nơi.
Vị trí trong lộ trình: Đọc cùng các bài 15 và 16 để so sánh những cách tiếp cận mô hình sinh.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Deep learning đã thành công rõ rệt với **mô hình phân biệt** (discriminative model: ánh xạ input nhiều chiều về một nhãn lớp), chủ yếu nhờ backpropagation và dropout, dùng đơn vị tuyến tính từng đoạn (piecewise linear, như ReLU) có gradient tốt. Mô hình sinh (generative model) ít thành công hơn, vì hai lý do chính: (a) khó xấp xỉ các phép tính xác suất phức tạp (bất khả thi/intractable) xuất hiện trong ước lượng hợp lý cực đại (maximum likelihood) và các chiến lược liên quan; (b) khó tận dụng lợi ích của đơn vị tuyến tính từng đoạn trong bối cảnh sinh (các đơn vị này có kích hoạt không bị chặn, dễ gây vấn đề khi dùng trong vòng lặp phản hồi của mô hình sinh dựa trên Markov chain).



p_g vs p_{data} và $V(D^*, G)$; Nash $-2\log 2$ khi $p_g = p_{\text{data}}$ (minh hoạ, tự tính)

Các hướng mô hình sinh trước đó và vấn đề của chúng: mô hình đồ thị không định hướng có biến ẩn (RBM, DBM) có hàm phân hoạch (partition function) bất khả thi, phải ước lượng bằng MCMC, dễ gặp vấn đề **mixing** kém; mạng belief sâu (DBN) lai giữa có hướng và vô hướng, kế thừa khó khăn của cả hai; các tiêu chí không cần log-likelihood như score matching, noise-contrastive estimation (NCE) cần mật độ xác suất chưa chuẩn hóa được đặc tả giải tích, không khả dụng cho mô hình nhiều tầng biến ẩn; một số mô hình (denoising/contractive autoencoder) có quy tắc học gần giống score matching áp lên RBM.

Bài này đề xuất một hướng khác hẳn: **không định nghĩa xác suất $p_g(x)$ một cách giải tích**, mà huấn luyện một "máy sinh" (generative machine) chỉ để lấy mẫu từ phân phối mong muốn, bằng backpropagation thuần, không cần Markov chain hay suy luận xấp xỉ nào trong lúc huấn luyện hay sinh mẫu.

2. Ý tưởng chính

Huấn luyện đồng thời hai mô hình:

- **Mạng sinh** G: nhận noise $z \sim p_z(z)$, sinh ra $G(z;\theta_g)$, một hàm khả vi tham số hóa bằng mạng nơ-ron nhiều tầng (MLP), ngầm định một phân phối p_g trên dữ liệu.
- **Mạng phân biệt** D: nhận x, trả về một số thực trong $[0,1]$ ước lượng xác suất x là dữ liệu thật (không phải sinh ra từ G).

D được huấn luyện để tối đa xác suất gán đúng nhãn cho cả mẫu thật và mẫu sinh; G được huấn luyện **đồng thời** để tối thiểu $\log(1 - D(G(z)))$, tức để "lừa" D. Đây là một trò chơi minimax hai người chơi (công thức 1):

$$\min_G \max_D V(D,G) = \mathbb{E}_{x\sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]$$

Phép ẩn dụ của tác giả: G như một nhóm làm tiền giả, cố in tiền giả không bị phát hiện; D như cảnh sát, cố phát hiện tiền giả; sự cạnh tranh buộc cả hai cải thiện đến khi tiền giả không thể phân biệt được với tiền thật.

Về lý thuyết (không gian hàm tùy ý, năng lực vô hạn), điểm tối ưu toàn cục duy nhất là $p_g = p_{data}$, khi đó $D(x) = 1/2$ mọi nơi (mạng phân biệt không còn phân biệt được gì, vì hai phân phối đã trùng nhau).

3. Phương pháp chi tiết

3.1 Huấn luyện lặp xen kẽ (mục 3, Thuật toán 1)

Tối ưu D đến hội tụ ở mỗi bước lặp trong của G là tốn kém tính toán và dễ overfitting trên tập hữu hạn. Thay vào đó, huấn luyện **xen kẽ**: k bước cập nhật D, rồi 1 bước cập nhật G (paper dùng $k = 1$, lựa chọn rẻ nhất, trong mọi thí nghiệm). Cách này giữ D gần nghiệm tối ưu miễn G thay đổi đủ chậm, giống cách SML/PCD duy trì mẫu Markov chain giữa các bước học của Boltzmann machine để tránh phải "burn-in" lại từ đầu mỗi bước.

Thuật toán 1 (lược lại): mỗi vòng lặp, lấy minibatch m mẫu noise và m mẫu dữ liệu thật, cập nhật D theo gradient tăng của $\frac{1}{m}\sum_i[\log D(x^{(i)}) + \log(1 - D(G(z^{(i)})))]$ (k lần); rồi lấy minibatch noise mới, cập nhật G theo gradient giảm của $\frac{1}{m}\sum_i \log(1 - D(G(z^{(i)})))$.

Mục tiêu thay thế cho G (mục 3, đoạn cuối): đầu huấn luyện, khi G còn kém, D có thể bác mẫu sinh với độ tin cậy cao vì chúng rõ ràng khác dữ liệu thật; khi đó $\log(1 - D(G(z)))$ **bão hòa** (gradient rất nhỏ). Paper đề xuất thay bằng huấn luyện G để **tối đa** $\log D(G(z))$ (thay vì tối thiểu $\log(1 - D(G(z)))$); về điểm dừng động học thì tương đương, nhưng gradient mạnh hơn nhiều ở giai đoạn đầu huấn luyện. Đây là một chi tiết thực hành quan trọng, không chỉ là biến đổi đại số trung lập.

3.2 Chứng minh điểm tối ưu toàn cục (mục 4.1)

Mệnh đề 1: với G cố định, D tối ưu là

$$D^*_G(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)} \tag{2}$$

(chứng minh: hàm $y \mapsto a\log y + b\log(1-y)$ đạt cực đại tại $y = a/(a+b)$ trên $[0,1]$, áp dụng với $a = p_{data}(x)$, $b = p_g(x)$).

Định lý 1: Giá trị tối ưu toàn cục của $C(G) = \max_D V(G, D)$ đạt được khi và chỉ khi $p_g = p_{data}$, tại đó $C(G) = -\log 4$. Chứng minh dùng phân kỳ Kullback-Leibler (KL) và nhận diện biểu thức còn lại là 2 lần phân kỳ Jensen-Shannon (JSD), công thức (5)-(6):

$$C(G) = -\log(4) + KL\left(p_{data} \middle\| \frac{p_{data} + p_g}{2}\right) + KL\left(p_g \middle\| \frac{p_{data} + p_g}{2}\right) = -\log(4) + 2 \cdot JSD(p_{data} \| p_g)$$

Vì JSD luôn không âm và chỉ bằng 0 khi hai phân phối trùng nhau, $C^* = -\log 4$ là cực tiểu toàn cục, đạt duy nhất khi $p_g = p_{data}$.

Ví dụ số tự tính (minh họa Mệnh đề 1): giả sử tại một điểm x cho trước, $p_{data}(x) = 0{,}3$ và $p_g(x) = 0{,}1$ (số minh họa). Khi đó $D^*_G(x) = 0{,}3/(0{,}3 + 0{,}1) = 0{,}75$ (tự tính), nghĩa là bộ phân biệt tối ưu gán 75% xác suất "là dữ liệu thật" cho điểm đó, hợp lý vì p_{data} gấp 3 lần p_g tại điểm này. Và $-\log 4 \approx -1{,}386$ (tự tính, $\ln 4 = 1{,}3863$), là giá trị $C(G)$ khi $p_g = p_{data}$ hoàn toàn (không phải tại điểm x cụ thể trên).

Lưu ý nhỏ về Thuật toán 1: trong văn bản thuật toán, bước đầu ghi "lấy mẫu noise từ noise prior $p_g(z)$" (dùng ký hiệu p_g cho phân phối noise đầu vào), trong khi phần văn bản chính và công thức (1) đều gọi phân phối noise đầu vào là $p_z(z)$, và p_g được dùng nhất quán để chỉ phân phối **do** G **sinh ra** trên không gian dữ liệu x. Đây là một chỗ ký hiệu không khớp giữa thuật toán và phần lý thuyết, không ảnh hưởng nội dung (ý nghĩa thực tế rõ ràng là lấy mẫu noise, không phải một lỗi về thuật toán).

3.3 Hội tụ của thuật toán thực tế (mục 4.2)

Mệnh đề 2: nếu G, D có đủ năng lực, D đạt tối ưu cho mỗi G, và p_g cập nhật để cải thiện tiêu chí $\mathbb{E}_{p_{data}}[\log D^*_G(x)] + \mathbb{E}_{p_g}[\log(1 - D^*_G(x))]$, thì $p_g \to p_{data}$. Chứng minh dựa trên tính lồi (convex) của $\sup_D U(p_g, D)$ theo p_g, và việc cập nhật p_g theo gradient tại D tối ưu tương đương một bước gradient descent trên một hàm lồi có cực tiểu toàn cục duy nhất.

Paper tự lưu ý: trong thực tế, G, D bị giới hạn bởi tham số hóa MLP cụ thể (tối ưu theo θ_g, không phải trực tiếp theo p_g), nên các đảm bảo lý thuyết (không gian hàm tùy ý) không áp dụng chặt cho trường hợp thực hành, nhưng hiệu năng thực nghiệm tốt của MLP gợi ý đây vẫn là mô hình hợp lý để dùng.

4. Thí nghiệm và kết quả chính

Thiết lập: huấn luyện trên MNIST, Toronto Face Database (TFD), CIFAR-10. Mạng sinh dùng hỗn hợp kích hoạt ReLU và sigmoid; mạng phân biệt dùng kích hoạt maxout; dropout áp dụng khi huấn luyện mạng phân biệt. Noise chỉ đưa vào **tầng đáy nhất** của mạng sinh (dù khung lý thuyết cho phép noise ở các tầng giữa).

Đánh giá log-likelihood (Bảng 1): vì $p_g(x)$ không có dạng giải tích, paper dùng ước lượng **Parzen window** (khớp một cửa sổ Gauss lên mẫu sinh ra từ G, rồi tính log-likelihood của tập test dưới phân phối Parzen đó; phương pháp của Breuleux và cộng sự 2011).

Mô hình	MNIST	TFD
DBN	138 ± 2	1909 ± 66
Stacked CAE	$121 \pm 1{,}6$	2110 ± 50
Deep GSN	$214 \pm 1{,}1$	1890 ± 29
Adversarial nets (GAN)	225 ± 2	2057 ± 26

GAN đạt log-likelihood ước lượng cao nhất trên MNIST (225), và trên TFD (2057) **cao hơn** DBN và Deep GSN nhưng **thấp hơn** Stacked CAE (2110). Paper tự thừa nhận phương pháp Parzen window có **phương sai**

cao và hoạt động kém trong không gian nhiều chiều, chỉ là "phương pháp tốt nhất hiện có" chứ không hoàn hảo.

Đánh giá định tính (Hình 2, 3): mẫu sinh trên MNIST, TFD, CIFAR-10 (cả mạng kết nối đầy đủ và mạng có CNN cho phân biệt/"deconvolutional" cho sinh); cột cuối mỗi hình cho thấy ví dụ huấn luyện gần nhất với mẫu sinh, để chứng minh mô hình **không chỉ ghi nhớ** (memorize) tập huấn luyện. Hình 3 minh họa nội suy tuyến tính (linear interpolation) giữa các điểm trong không gian z, cho chuỗi ảnh số biến đổi mượt, gợi ý không gian ẩn có cấu trúc liên tục có ý nghĩa.

Bảng 2 (so sánh định tính các hướng mô hình sinh sâu): paper tổng hợp 5 khía cạnh (huấn luyện, suy luận, lấy mẫu, đánh giá $p(x)$, thiết kế mô hình) cho 4 họ mô hình: đồ thị có hướng sâu, đồ thị vô hướng sâu, autoencoder sinh, và adversarial models. Điểm paper nhấn mạnh cho GAN: không cần suy luận trong huấn luyện, không cần Markov chain khi lấy mẫu ("No difficulties"), nhưng $p(x)$ "không được biểu diễn rõ, có thể xấp xỉ bằng Parzen density estimation" (giống autoencoder sinh).

5. Phân tích / ablation

Bài này **không có bảng ablation số liệu có hệ thống** (ví dụ quét siêu tham số k, kiến trúc G/D) như các bài sau trong dòng GAN; phân tích chủ yếu định tính:

- **Lựa chọn $k = 1$ (số bước cập nhật D mỗi bước G):** chọn vì rẻ nhất, không có so sánh với $k > 1$ trong bài.
- **Mục tiêu thay thế cho G (tối đa $\log D(G(z))$ thay vì tối thiểu $\log(1 - D(G(z)))$):** được trình bày như một chi tiết thực hành quan trọng (giải quyết bão hòa gradient sớm) nhưng không có bảng số so sánh hai cách huấn luyện; chỉ lập luận bằng lý thuyết động học.
- **"Kịch bản Helvetica" (mục 6):** tác giả cảnh báo nếu G được huấn luyện quá nhiều mà không cập nhật D, G có thể **collapse** nhiều giá trị z về cùng một x (thiếu đa dạng), tương tự việc chuỗi Markov âm (negative chain) của Boltzmann machine cần được giữ cập nhật thường xuyên; đây là một quan sát định tính/cảnh báo, không có thí nghiệm đo trực tiếp mức độ collapse trong bài.
- **Bảng 2** đóng vai trò phân tích so sánh định tính với các họ mô hình khác hơn là ablation nội bộ của chính GAN.

6. Giới hạn

Do tác giả nêu (mục 6):

- **Không có biểu diễn rõ (explicit) của $p_g(x)$**; D phải được giữ "đồng bộ" tốt với G trong suốt huấn luyện (đặc biệt G không được huấn luyện quá nhiều mà thiếu cập nhật D, để tránh kịch bản Helvetica).
- Phương pháp Parzen window để ước lượng log-likelihood "có phương sai khá cao và hoạt động không tốt trong không gian nhiều chiều"; đây chỉ là phương pháp tốt nhất sẵn có lúc đó, không phải thước đo đáng tin cậy tuyệt đối.
- MLP dùng để tham số hóa G giới thiệu nhiều điểm cực trị (critical point) trong không gian tham số, nên các đảm bảo lý thuyết (không gian hàm, không gian p_g) không áp dụng chặt cho việc tối ưu θ_g thực tế; hiệu năng tốt trong thực nghiệm được paper mô tả là "gợi ý" MLP là lựa chọn hợp lý, không phải được chứng minh.

Nhận xét:

- Bảng 1 cho thấy GAN **không phải luôn tốt nhất** trong số các mô hình so sánh (kém Stacked CAE trên TFD), dù phần abstract và phần giới thiệu tạo cảm giác GAN vượt trội; cần đọc kỹ bảng số, không chỉ phần tuyên bố định tính.
- Không có thước đo định lượng về đa dạng mẫu (mode coverage/diversity) nào ngoài quan sát trực quan trên ảnh và cảnh báo lý thuyết về kịch bản Helvetica; các thước đo định lượng về mode collapse chỉ

xuất hiện trong các công trình GAN sau này.
- Thuật toán 1 dùng ký hiệu $p_g(z)$ cho phân phối noise đầu vào (thấy ở mục 3.2 trên), không khớp với định nghĩa $p_z(z)$ ở phần văn bản chính; một sơ suất ký hiệu nhỏ cần lưu ý khi đối chiếu công thức (1) với thuật toán.

7. Liên hệ với các bài khác

- **ml basics 15** (cùng thư mục, Auto-Encoding Variational Bayes/VAE): một hướng mô hình sinh khác cùng thời (VAE công bố Kingma và Welling 2014, chỉ vài tháng trước/sau GAN), dùng lower bound biến phân thay vì trò chơi minimax; phần "Related work" của bài VAE (bài 15) nhắc tới chính GAN dưới dạng "huấn luyện một máy sinh bằng backpropagate trực tiếp" như một hướng liên quan.
- **ml basics 16** (cùng thư mục, DDPM): một hướng mô hình sinh thứ ba, dựa trên khử nhiễu dần (denoising diffusion), ra đời 6 năm sau và về sau vượt GAN ở nhiều benchmark ảnh chất lượng cao; Bảng 2 "Challenges in generative modeling" ở DDPM gián tiếp kế tục cách phân loại các họ mô hình sinh mà Bảng 2 của bài này khởi xướng.
- **distillation content/05_2021_kd_survey.md**: các khảo sát về knowledge distillation đôi khi nhắc tới GAN như một dạng huấn luyện đối kháng (adversarial training) có thể kết hợp với distillation (học sinh cố "lừa" một bộ phân biệt để giống giáo viên hơn); liên hệ gián tiếp, không phải nội dung cốt lõi của cả hai bài.
- **math content/13 (Wasserstein GAN)** và **math content/17 (Spectral Normalization for GANs)**: hai công trình trực tiếp sửa các vấn đề huấn luyện của chính bài GAN gốc này, WGAN thay JSD (công thức 6 ở mục 3.2) bằng khoảng cách Wasserstein để tránh gradient bão hòa, Spectral Normalization kiểm soát Lipschitz constant của D để ổn định huấn luyện.

8. Gợi ý áp dụng

GAN đặt bài toán sinh dữ liệu thành trò chơi giữa generator và discriminator. Khi thử nghiệm, cần theo dõi cả chất lượng và độ đa dạng mẫu, vì một chỉ số đơn lẻ khó phản ánh đầy đủ hành vi của mô hình.

9. Thí nghiệm gợi ý

1. **Cài một GAN tối giản (toy 2D hoặc MNIST):** G và D là MLP nhỏ (2-3 tầng ẩn), dữ liệu là MNIST hoặc một phân phối 2D tổng hợp (ví dụ hỗn hợp Gauss nhiều mode). Huấn luyện với $k = 1$ như bài gốc, quan sát mẫu sinh qua các epoch. Kỳ vọng: với hỗn hợp Gauss nhiều mode, dễ quan sát trực tiếp hiện tượng mode collapse (một phần mode không được G sinh ra), minh họa cụ thể hóa "kịch bản Helvetica" mà paper chỉ nói định tính.
2. **So sánh hai mục tiêu huấn luyện G:** trên cùng thiết lập thí nghiệm 1, so sánh huấn luyện G bằng tối thiểu $\log(1 - D(G(z)))$ (mục tiêu gốc, công thức 1) với tối đa $\log D(G(z))$ (mục tiêu thay thế, mục 3.1). Đo tốc độ hội tụ (bao nhiêu epoch để mẫu sinh trông hợp lý) và độ ổn định huấn luyện. Kỳ vọng: mục tiêu thay thế hội tụ nhanh hơn ở giai đoạn đầu, khớp lập luận "gradient mạnh hơn" của paper.
3. **Đo $D(x)$ trung bình trên dữ liệu thật và dữ liệu sinh theo epoch:** kỳ vọng D hội tụ về gần $0{,}5$ cho cả hai loại dữ liệu nếu huấn luyện thành công (theo Định lý 1); nếu D vẫn phân biệt rõ (gần 0 hoặc 1) sau nhiều epoch, đó là dấu hiệu huấn luyện chưa hội tụ hoặc mất cân bằng G/D.
4. **Thử $k \in \{1, 2, 5\}$ (số bước cập nhật D mỗi bước G):** đo chất lượng mẫu sinh (bằng mắt hoặc một thước đo đơn giản như khoảng cách tới tập thật trong không gian pixel/feature) và độ ổn định huấn luyện. Kỳ vọng: k lớn hơn giữ D gần tối ưu hơn (đúng lý luận mục 3.1) nhưng tốn thời gian hơn; nếu k quá nhỏ có thể gây mất ổn định, lấp phần ablation mà bài gốc không tự làm.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Mục tiêu minimax	$\min_G \max_D \mathbb{E}_{p_{data}}[\log D(x)] + \mathbb{E}_{p_z}[\log(1 - D(G(z)))]$ (1)
D tối ưu cho G cố định	$D^*_G(x) = p_{data}(x)/(p_{data}(x) + p_g(x))$ (2)
Giá trị tối ưu toàn cục	$C(G) = -\log 4 \approx -1{,}386$ khi $p_g = p_{data}$
Phân rã theo JSD	$C(G) = -\log 4 + 2 \cdot JSD(p_{data} \Vert p_g)$ (6)
Mục tiêu G thay thế (thực hành)	tối đa $\log D(G(z))$ thay vì tối thiểu $\log(1 - D(G(z)))$
Số bước cập nhật D mỗi bước G	$k = 1$ trong mọi thí nghiệm
Đánh giá log-likelihood	Parzen window (xấp xỉ, phương sai cao)
Kết quả Bảng 1	MNIST: GAN 225 ± 2 (cao nhất); TFD: GAN 2057 ± 26 (kém Stacked CAE 2110 ± 50)

11. Kết luận ngắn

GAN đề xuất huấn luyện một mô hình sinh thông qua trò chơi đối kháng với một mô hình phân biệt, chỉ cần backpropagation, không cần Markov chain hay suy luận xấp xỉ, và chứng minh được điểm tối ưu lý thuyết là G tái tạo đúng phân phối dữ liệu. Thực nghiệm trên MNIST, TFD, CIFAR-10 cho kết quả cạnh tranh (dù không luôn vượt trội, ví dụ kém hơn Stacked CAE trên TFD theo ước lượng Parzen) và mẫu sinh chất lượng thị giác tốt, mở ra một dòng nghiên cứu mô hình sinh lớn kéo dài nhiều năm sau. Khung lý thuyết chặt (Định lý 1, Mệnh đề 1-2) đi kèm nhiều cảnh báo thực hành (bão hòa gradient, kịch bản Helvetica, cần giữ D đồng bộ với G) mà các công trình GAN sau tiếp tục giải quyết.

Nguồn gốc: https://arxiv.org/abs/1406.2661

Bài 15

Auto-Encoding Variational Bayes (2013)

Tác giả: Diederik P. Kingma, Max Welling (Universiteit van Amsterdam).
Venue / arXiv: arXiv:1312.6114 (v1 gốc 20/12/2013; bản đọc để viết ghi chú này là v11, 10/12/2022, không thay đổi nội dung cốt lõi so với bản gốc). Bài GAN (số 14 cùng thư mục) dẫn chiếu bài này trong danh mục tham khảo là "ICLR 2014", gợi ý venue công bố thực tế dù trang bìa arXiv không ghi rõ.
Nguồn: https://arxiv.org/abs/1312.6114
Chủ đề: suy luận biến phân (variational inference) và học biểu diễn ẩn liên tục bằng một mạng encoder-decoder huấn luyện qua một cận dưới (lower bound) khả vi của log-likelihood, nhờ "reparameterization trick" cho phép backpropagate qua bước lấy mẫu ngẫu nhiên.
Vị trí trong lộ trình: Đọc cùng các bài 14 và 16 về GAN và diffusion để đối chiếu các họ mô hình sinh.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Bài toán: cho một mô hình sinh có biến ẩn liên tục z với prior $p_\theta(z)$ và likelihood $p_\theta(x|z)$ (tham số hóa bằng mạng nơ-ron), ta muốn học θ bằng maximum likelihood, và/hoặc suy luận posterior $p_\theta(z|x)$. Vấn đề: với mạng nơ-ron phi tuyến, cả hai phép tính "hậu nghiệm đúng" (true posterior) $p_\theta(z|x)$ và "tích phân biên" (marginal likelihood) $p_\theta(x) = \int p_\theta(z)p_\theta(x|z)\,dz$ đều **bất khả thi** (intractable) ở dạng đóng (closed form), không thể tính trực tiếp hay tối ưu bằng EM thông thường (Expectation Maximization).

Các phương pháp biến phân (variational inference) truyền thống thường cần suy ra công thức cập nhật giải tích riêng cho từng mô hình (ví dụ mean-field), tốn công và không tổng quát; các phương pháp Monte Carlo biến phân trước đó (ví dụ wake-sleep) có gradient ước lượng (estimator) phương sai cao hoặc cần huấn luyện hai mạng riêng theo hai mục tiêu không trùng khớp hoàn toàn với mục tiêu thật.

Bài này đặt ra ba mục tiêu liên quan chặt với nhau: (a) ước lượng hiệu quả tham số θ của mô hình sinh; (b) suy luận xấp xỉ hiệu quả posterior $p_\theta(z|x)$ cho x quan sát được, cho các việc như mã hóa (coding) hoặc dữ liệu đại diện; (c) suy luận xấp xỉ biên x hiệu quả, cho các việc cần một prior trên x (ví dụ bài toán denoising, inpainting).

2. Ý tưởng chính

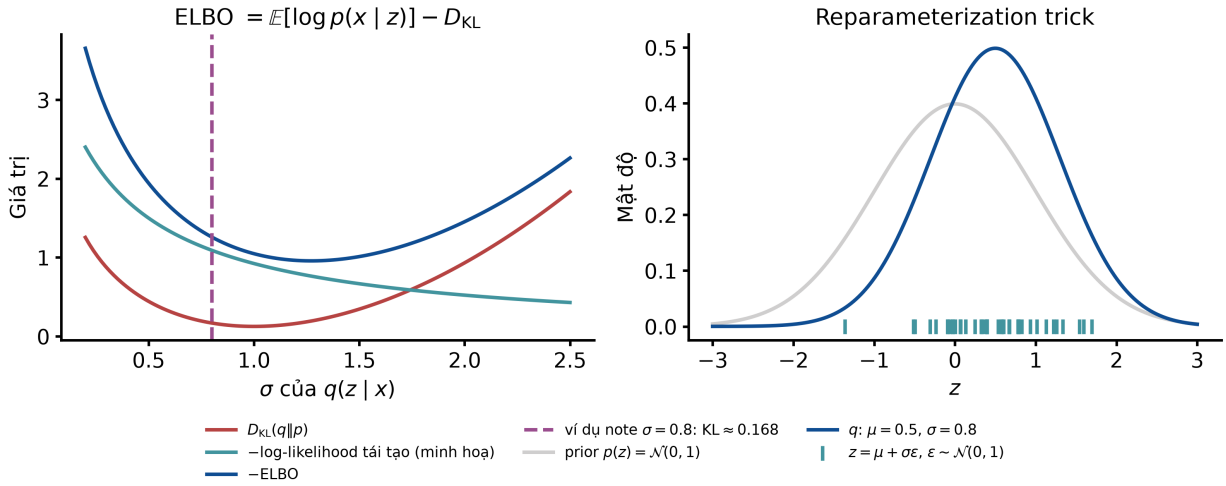
Đưa vào một **mô hình suy luận xấp xỉ** $q_\phi(z|x)$ (gọi là "encoder" - mã hóa x thành phân phối trên z), song song với mô hình sinh $p_\theta(x|z)$ (gọi là "decoder" - giải mã z thành phân phối trên x); cả hai cùng tham số hóa bằng mạng nơ-ron (MLP), và được học **đồng thời** bằng cách tối đa hóa một **cận dưới biến phân** (variational lower bound) của $\log p_\theta(x)$, thông qua gradient descent/ascent.

Khó khăn kỹ thuật chính: cận dưới chứa kỳ vọng theo $q_\phi(z|x)$, mà việc lấy mẫu $z \sim q_\phi(z|x)$ thông thường (sampling trực tiếp) không khả vi theo ϕ, nên không thể backpropagate gradient qua nó bằng cách thông thường. Đóng góp cốt lõi của bài là **reparameterization trick**: viết lại biến ngẫu nhiên $z \sim q_\phi(z|x)$ dưới dạng $z = g_\phi(\epsilon, x)$ với ϵ lấy mẫu từ một phân phối noise độc lập đơn giản $p(\epsilon)$ (ví dụ chuẩn hóa $N(0, I)$) không phụ thuộc tham số, sao cho mọi tính ngẫu nhiên nằm ở ϵ, còn phép biến đổi g_ϕ khả vi theo ϕ. Nhờ đó gradient của kỳ vọng theo z trở thành gradient của một hàm khả vi bên trong một kỳ vọng theo ϵ cố định, ước lượng được bằng Monte Carlo thông thường và cho phép backpropagation bình thường.

Thuật toán tổng thể gọi là **AEVB** (Auto-Encoding Variational Bayes): dùng ước lượng SGVB (Stochastic Gradient Variational Bayes) để tối ưu một mạng suy luận, cho phép suy luận xấp xỉ hiệu quả bằng cách dùng

trực tiếp các mẫu ancestral đơn giản (không cần MCMC lặp đi lặp lại tốn kém trong vòng lặp huấn luyện).

3. Phương pháp chi tiết



ELBO = tái tạo − KL; ví dụ $\sigma = 0.8$ cho KL≈ 0.168; reparam $z = \mu + \sigma\epsilon$ (minh hoạ, tự tính)

3.1 Cận dưới biến phân (mục 2.1-2.2)

Với tập dữ liệu $X = \{x^{(i)}\}_{i=1}^{N}$ i.i.d., mỗi điểm sinh ra từ quá trình: lấy $z^{(i)} \sim p_\theta(z)$, rồi $x^{(i)} \sim p_\theta(x|z^{(i)})$. Đưa vào $q_\phi(z|x)$ xấp xỉ posterior thật $p_\theta(z|x)$. Log-likelihood biên của mỗi điểm dữ liệu được viết (công thức 1-2):

$$\log p_\theta(x^{(i)}) = D_{KL}\big(q_\phi(z|x^{(i)}) \,\|\, p_\theta(z|x^{(i)})\big) + \mathcal{L}(\theta, \phi; x^{(i)}) \tag{2}$$

Vì $D_{KL} \geq 0$, số hạng thứ hai là một cận dưới (lower bound) của log-likelihood: $\log p_\theta(x^{(i)}) \geq \mathcal{L}(\theta, \phi; x^{(i)})$, và $\mathcal{L}$ có hai dạng tương đương (công thức 3):

$$\mathcal{L}(\theta, \phi; x^{(i)}) = -D_{KL}\big(q_\phi(z|x^{(i)}) \,\|\, p_\theta(z)\big) + \mathbb{E}_{q_\phi(z|x^{(i)})}\big[\log p_\theta(x^{(i)}|z)\big] \tag{3}$$

Dạng này sau được gọi là ELBO (Evidence Lower BOund) trong văn liệu sau đó (bài không dùng từ viết tắt này, nhưng đây chính là đại lượng đó). Số hạng đầu là một "điều hòa" (regularizer) kéo $q_\phi(z|x)$ về gần prior $p_\theta(z)$; số hạng sau là kỳ vọng log-likelihood tái tạo (reconstruction).

Muốn tối ưu $\mathcal{L}$ theo cả θ và ϕ bằng gradient, cần ước lượng gradient của $\mathbb{E}_{q_\phi(z|x)}[f(z)]$ theo ϕ. Ước lượng "naive Monte Carlo gradient" thông thường (dùng score function/REINFORCE) có phương sai rất cao, không thực tế cho ứng dụng của bài.

3.2 Reparameterization trick (mục 2.3-2.4)

Dưới một điều kiện khả vi nhẹ, có thể biểu diễn biến ngẫu nhiên liên tục $z \sim q_\phi(z|x)$ bằng một biến đổi khả vi $g_\phi(\cdot)$ của một biến noise ϵ (công thức 4):

$$\tilde{z} = g_\phi(\epsilon, x), \quad \epsilon \sim p(\epsilon)$$

Ví dụ cụ thể cho trường hợp Gauss đơn biến: nếu $z \sim N(\mu, \sigma^2)$, một cách reparameterize hợp lệ là $z = \mu + \sigma\epsilon$ với $\epsilon \sim N(0, 1)$.

Nhờ đó, kỳ vọng Monte Carlo của một hàm $f(z)$ theo $q_\phi(z|x)$ ước lượng được bằng (công thức 5): $\mathbb{E}_{q_\phi(z|x)}[f(z)] = \mathbb{E}_{p(\epsilon)}[f(g_\phi(\epsilon, x))] \simeq \frac{1}{L}\sum_{l=1}^{L} f(g_\phi(\epsilon^{(l)}, x))$, và vì g_ϕ khả vi theo ϕ, gradient qua kỳ vọng này tính được bằng backpropagation thông thường.

Ước lượng SGVB tổng quát $\tilde{\mathcal{L}}^A$ (công thức 6): áp trực tiếp công thức (5) cho cả hai số hạng của $\mathcal{L}$ (KL và reconstruction), không cần biết dạng giải tích của KL:

$$\tilde{\mathcal{L}}^A(\theta, \phi; x^{(i)}) = \frac{1}{L}\sum_{l=1}^{L} \log p_\theta(x^{(i)}, z^{(i,l)}) - \log q_\phi(z^{(i,l)}|x^{(i)})$$

Ước lượng SGVB phương sai thấp hơn $\tilde{\mathcal{L}}^B$ (công thức 7), dùng khi số hạng $D_{KL}(q_\phi(z|x)\|p_\theta(z))$ tích phân được giải tích (ví dụ cả hai là Gauss), chỉ cần Monte Carlo cho số hạng reconstruction:

$$\tilde{\mathcal{L}}^B(\theta, \phi; x^{(i)}) = -D_{KL}\big(q_\phi(z|x^{(i)}) \,\|\, p_\theta(z)\big) + \frac{1}{L}\sum_{l=1}^{L} \log p_\theta(x^{(i)}|z^{(i,l)})$$

Với **ước lượng minibatch** $\tilde{\mathcal{L}}^M$ trên M điểm dữ liệu lấy ngẫu nhiên từ toàn tập N điểm (công thức 8): $\mathcal{L}(\theta, \phi; X) \simeq \tilde{\mathcal{L}}^M(\theta, \phi; X^M) = \frac{N}{M}\sum_{i=1}^{M} \tilde{\mathcal{L}}(\theta, \phi; x^{(i)})$.

Thuật toán 1 (AEVB): lặp: lấy minibatch M điểm; lấy noise ϵ từ $p(\epsilon)$; tính gradient của $\tilde{\mathcal{L}}^M$ theo (θ, ϕ) bằng backpropagation; cập nhật (θ, ϕ) bằng SGD/Adagrad. Thí nghiệm của bài dùng $M = 100$, $L = 1$ (chỉ 1 mẫu z mỗi điểm dữ liệu mỗi bước, vì minibatch đã đủ lớn để giảm phương sai).

3.3 Trường hợp cụ thể: VAE với encoder/decoder Gauss (mục 3)

Chọn $p_\theta(z) = N(z; 0, I)$ (prior chuẩn đa biến, không tham số); $p_\theta(x|z)$ là một MLP (Bernoulli MLP cho dữ liệu nhị phân như pixel MNIST nhị phân hóa, hoặc Gaussian MLP cho dữ liệu liên tục như Frey Face); và $q_\phi(z|x)$ là một Gauss đường chéo (diagonal covariance Gauss, công thức 9):

$$\log q_\phi(z|x^{(i)}) = \log N(z; \mu^{(i)}, \sigma^{2(i)} I)$$

với $\mu^{(i)}, \sigma^{(i)}$ là output của một MLP encoder nhận $x^{(i)}$ làm input. Vì cả prior và posterior xấp xỉ đều Gauss, $D_{KL}(q_\phi(z|x)\|p_\theta(z))$ tích phân được giải tích (dẫn ở Phụ lục B), nên dùng được ước lượng $\tilde{\mathcal{L}}^B$. Công thức (10), ước lượng cho mỗi điểm dữ liệu với encoder/decoder Gauss:

$$\mathcal{L}(\theta, \phi; x^{(i)}) \simeq \frac{1}{2}\sum_{j=1}^{J}\Big(1 + \log(\sigma_j^{(i)2}) - (\mu_j^{(i)})^2 - (\sigma_j^{(i)})^2\Big) + \frac{1}{L}\sum_{l=1}^{L} \log p_\theta(x^{(i)}|z^{(i,l)})$$

với $z^{(i,l)} = \mu^{(i)} + \sigma^{(i)} \odot \epsilon^{(l)}$, $\epsilon^{(l)} \sim N(0, I)$ (reparameterization cụ thể cho J chiều latent).

Công thức KL closed-form (Phụ lục B), trường hợp $J = 1$ chiều, giữa $q_\phi(z) = N(\mu, \sigma^2)$ và prior $N(0, 1)$:

$$-D_{KL}\big(q_\phi(z) \,\|\, p(z)\big) = \frac{1}{2}\big(1 + \log(\sigma^2) - \mu^2 - \sigma^2\big)$$

Ví dụ số tự tính: lấy minh họa $\mu = 0{,}5$, $\sigma = 0{,}8$ (số minh họa, không phải số trong paper). Ta có $\sigma^2 = 0{,}64$; $\log(0{,}64) \approx -0{,}4463$ (tự tính, $\ln 0{,}64$). Tổng trong ngoặc: $1 - 0{,}4463 - 0{,}25 - 0{,}64 = -0{,}3363$ (tự tính). Vậy $-D_{KL} = \frac{1}{2}(-0{,}3363) = -0{,}1681$, nghĩa là $D_{KL} \approx 0{,}168$ nat (tự tính) cho chiều latent này, một con số dương hợp lý (KL luôn ≥ 0).

Phụ lục C định nghĩa chi tiết Bernoulli MLP decoder (công thức 11, dùng cross-entropy với output sigmoid) và Gaussian MLP encoder/decoder (công thức 12, hai tầng ẩn tanh, output mean và log-variance riêng).

3.4 Phần mở rộng: Full VB (Phụ lục F)

Bài còn trình bày một mở rộng lý thuyết: thay vì chỉ tối ưu điểm θ (MAP/ML point estimate), đặt một **phân phối biến phân đầy đủ** lên cả tham số toàn cục θ (hyperprior $p(\theta)$), suy luận xấp xỉ $q_\phi(\theta, z^{(1:N)}|x^{(1:N)})$. Phần này (công thức 13-24) chủ yếu lý thuyết, không có thí nghiệm riêng kiểm chứng trong bài; mục đích là chỉ ra khung AEVB tổng quát hơn một VAE cụ thể.

4. Thí nghiệm và kết quả chính

Dữ liệu: MNIST và Frey Face (ảnh khuôn mặt nhỏ, liên tục). **Kiến trúc:** encoder và decoder đều MLP một tầng ẩn, 500 unit ẩn cho MNIST, 200 unit ẩn cho Frey Face; số chiều latent N_z thử nghiệm $\in \{3, 5, 10, 20, 200\}$ cho MNIST và $\{2, 5, 10, 20\}$ cho Frey Face. Tối ưu bằng **Adagrad**, global stepsize chọn từ $\{0{,}01;\ 0{,}02;\ 0{,}1\}$ dựa trên giá trị ELBO đầu tiên trên tập huấn luyện; minibatch $M = 100$, $L = 1$.

So sánh (Hình 2, 3): AEVB so với hai baseline: **wake-sleep algorithm** (Hinton và cộng sự) và **Monte Carlo EM (MCEM)** dùng HMC (Hybrid Monte Carlo, Phụ lục E: 10 bước leapfrog, 5 bước cập nhật trọng số cho mỗi mẫu HMC). Hình 2 (MNIST và Frey Face, nhiều giá trị N_z) và Hình 3 (chỉ MNIST, thêm so sánh MCEM) đều vẽ đường cong lower bound theo số điểm dữ liệu đã huấn luyện (hoặc số bước lặp).

Kết luận định tính chính từ các đồ thị: AEVB hội tụ nhanh hơn rõ rệt wake-sleep ở mọi giá trị N_z thử nghiệm, và đạt lower bound tốt hơn khi hội tụ; AEVB cũng hội tụ nhanh hơn MCEM nhiều, và MCEM "hoạt động khá kém trên quy mô dữ liệu lớn" (như paper nhận xét ở Phụ lục E).

Bài **không có bảng số liệu dạng benchmark cố định** (ví dụ bit/dim hoặc NLL test trên một dòng bảng duy nhất như các bài sinh ảnh hiện đại); kết quả chính được trình bày qua đồ thị tốc độ hội tụ của lower bound, không phải một con số điểm chuẩn đơn lẻ để trích dẫn trực tiếp. Đây là điểm khác biệt về hình thức báo cáo kết quả so với bài 14 (GAN, có Bảng 1 Parzen log-likelihood) và bài 16 (DDPM, có Bảng 1 FID/IS/NLL).

5. Phân tích / ablation

- **Quét số chiều latent** N_z (Hình 2): bài thử nhiều giá trị N_z khác nhau cho cả hai bộ dữ liệu, đóng vai trò một dạng ablation về dung lượng biểu diễn của latent space; nhận xét chung là AEVB vượt trội wake-sleep ở mọi mức N_z thử.
- **Lựa chọn** $L = 1$ **(số mẫu Monte Carlo mỗi điểm dữ liệu):** bài giải thích lý do chọn $L = 1$ là vì minibatch $M = 100$ đã đủ lớn để phương sai gradient tổng thể thấp, không cần tăng L; không có bảng số so sánh trực tiếp $L = 1$ với $L > 1$ về tốc độ hội tụ, chỉ là lập luận.
- **So sánh bộ tối ưu hóa Adagrad với lựa chọn stepsize:** bài chọn stepsize theo ELBO đầu tiên quan sát được, không có ablation hệ thống giữa Adagrad và các bộ tối ưu khác.
- **MCEM baseline (Phụ lục E):** bài tự nhận xét MCEM hoạt động kém trên "quy mô lớn" dữ liệu, dùng như minh chứng gián tiếp cho việc tại sao cần một phương pháp biến phân nhanh hơn như AEVB, nhưng không có bảng số đo thời gian chạy cụ thể so sánh hai phương pháp.

6. Giới hạn

Do tác giả nêu:

- Phụ lục F (Full VB) là một mở rộng lý thuyết, không có thí nghiệm kiểm chứng thực nghiệm trong chính bài này.
- Reparameterization trick (công thức 4) chỉ áp dụng được khi biến ẩn là **liên tục** và biến đổi g_ϕ khả vi; bài không đưa ra giải pháp cho biến ẩn rời rạc (discrete latent variables) trong phạm vi chính của công thức SGVB.

- MCEM baseline được paper tự nhận xét hoạt động kém trên quy mô dữ liệu lớn, dùng nhiều bước HMC tốn kém mỗi điểm dữ liệu.

Nhận xét:

- Ngay câu mở đầu mục "Experiments" (mục đánh số 5 trong văn bản PDF, không phải mục 3), bài viết nguyên văn: "The generative model (encoder) and variational approximation (decoder) from section 3 were used" - câu này gán nhãn "(encoder)" cho mô hình sinh $p_\theta(x|z)$ và nhãn "(decoder)" cho mô hình suy luận xấp xỉ $q_\phi(z|x)$, **ngược hẳn** với định nghĩa nhất quán ở mọi nơi khác trong bài (đoạn cuối mục 2.1 định nghĩa rõ $q_\phi(z|x)$ là "probabilistic encoder" và $p_\theta(x|z)$ là "probabilistic decoder"; công thức (9) cũng gọi q_ϕ là encoder MLP). Đây là một câu bị đảo thuật ngữ trong chính văn bản gốc (không ảnh hưởng công thức toán, chỉ một câu mô tả ở đầu phần thực nghiệm), cần đọc theo định nghĩa nhất quán ở mục 2.1 và công thức (9), không theo câu văn lẻ đó.
- Kết quả thực nghiệm (Hình 2, 3) chỉ là đồ thị tốc độ hội tụ của lower bound, không có số liệu benchmark định lượng dạng bảng như test log-likelihood cụ thể theo từng N_z; muốn trích một con số chính xác (ví dụ ELBO cuối cùng tại $N_z = 20$ trên MNIST) sẽ cần đọc trực tiếp giá trị trên đồ thị, không có trong văn bản dạng số.
- Bài không thảo luận trực tiếp vấn đề "posterior collapse" (một hiện tượng VAE được văn liệu sau này hay nhắc, khi $q_\phi(z|x)$ bị kéo quá sát prior và latent trở nên vô dụng cho decoder); đây là một khoảng trống nội dung của chính bài gốc, không phải lỗi, chỉ là bài ra đời trước khi vấn đề này được đặt tên/nghiên cứu kỹ.

7. Liên hệ với các bài khác

- **ml basics 14** (cùng thư mục, GAN): một hướng mô hình sinh khác cùng thời, dùng trò chơi đối kháng thay cho cận dưới biến phân; GAN không cần định nghĩa posterior xấp xỉ $q_\phi(z|x)$ nào, trong khi VAE coi việc học posterior xấp xỉ là trọng tâm.
- **ml basics 16** (cùng thư mục, DDPM): công thức (3) của DDPM (variational bound trên negative log-likelihood, biểu diễn qua tổng các số hạng KL theo từng bước thời gian) là một mở rộng trực tiếp của ý tưởng cận dưới biến phân trình bày ở công thức (2)-(3) của bài này, áp dụng cho một chuỗi Markov nhiều bước thay vì một bước encoder-decoder đơn.
- **interp content/10_2023_towards_monosemanticity.md**: sparse autoencoder dùng trong nghiên cứu interpretability hiện đại là một hậu duệ về kiến trúc (encoder-decoder nén biểu diễn) của khung autoencoder biến phân trình bày ở bài này, dù mục tiêu huấn luyện khác hẳn (sparsity penalty thay vì KL tới prior Gauss).
- **math content/08 (Variational Inference: A Review for Statisticians)**, **math content/09 (An Introduction to Variational Autoencoders)**, **math content/14 (Deep Variational Information Bottleneck)**: bài 08 là nền tảng lý thuyết suy luận biến phân tổng quát mà mục 1-2 của bài này áp dụng cho một trường hợp cụ thể; bài 09 là một bài khảo sát/tutorial viết lại chi tiết hơn chính các công thức (1)-(10) ở đây; bài 14 dùng cùng kỹ thuật biến phân/reparameterization cho một mục tiêu khác (information bottleneck) thay vì sinh dữ liệu.

8. Gợi ý áp dụng

VAE kết hợp mô hình sinh với suy luận biến phân và cho phép tối ưu cận dưới bằng reparameterization. Khi áp dụng, cần theo dõi cả chất lượng tái tạo lẫn thành phần KL, vì tối ưu một phía không đủ mô tả chất lượng mô hình sinh.

9. Thí nghiệm gợi ý

1. **Cài một VAE tối giản trên MNIST:** encoder/decoder MLP 1 tầng ẩn như bài gốc (500 unit ẩn), $N_z \in \{2, 10, 20\}$, dùng ước lượng $\tilde{\mathcal{L}}^B$ (công thức 10). Đo ELBO trên tập validation theo epoch. Kỳ vọng:

N_z lớn hơn cho ELBO tốt hơn (ít âm hơn) nhưng tới một điểm bão hòa, minh họa trực tiếp Hình 2 của bài gốc.

2. **So sánh $\tilde{\mathcal{L}}^A$ (công thức 6, Monte Carlo cho cả KL) với $\tilde{\mathcal{L}}^B$ (công thức 7, KL closed-form):** trên cùng kiến trúc VAE nhỏ, đo phương sai của gradient ước lượng (chạy nhiều lần với cùng minibatch, đo độ lệch chuẩn gradient) và tốc độ hội tụ ELBO. Kỳ vọng: $\tilde{\mathcal{L}}^B$ hội tụ ổn định hơn và nhanh hơn, xác nhận lý do bài chọn dùng nó khi KL tích phân được.
3. **Quét L (số mẫu Monte Carlo mỗi điểm dữ liệu) $\in \{1, 5, 20\}$ với minibatch nhỏ ($M = 10$) và minibatch lớn ($M = 100$):** đo phương sai gradient và tốc độ hội tụ. Kỳ vọng: với M nhỏ, tăng L giúp ổn định hơn rõ; với $M = 100$ (như bài gốc), $L = 1$ đã đủ, khớp lập luận của bài ở mục 5.
4. **Kiểm tra posterior collapse:** huấn luyện VAE với hệ số nhân thêm β trước số hạng KL ($\beta \in \{0{,}1; 1; 10\}$, một biến thể không có trong bài gốc nhưng dễ cài thêm), đo tỉ lệ các chiều latent có $\sigma_j \to 1, \mu_j \to 0$ (tức bị "chết", không mang thông tin). Kỳ vọng: β lớn gây collapse nhiều chiều latent hơn, minh họa một vấn đề thực hành mà bài gốc (2013) chưa thảo luận nhưng văn liệu VAE sau này rất quan tâm.
5. **Tự tính lại công thức KL Gauss (Phụ lục B) bằng code, so với autograd:** code hàm `kl_gauss(mu, logvar)` theo công thức closed-form, so sánh số với KL tính bằng Monte Carlo (lấy mẫu z, tính $\log q(z) - \log p(z)$ trung bình trên nhiều mẫu). Kỳ vọng: hai cách cho cùng giá trị (sai số nhỏ do Monte Carlo), xác nhận công thức closed-form đúng và nhanh hơn nhiều so với Monte Carlo cho trường hợp Gauss-Gauss.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Phân rã log-likelihood	$\log p_\theta(x) = D_{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x)) + \mathcal{L}(\theta, \phi; x)$ (2)
Cận dưới biến phân (ELBO)	$\mathcal{L} = -D_{KL}(q_\phi(z \mid x) \| p_\theta(z)) + \mathbb{E}_{q_\phi}[\log p_\theta(x \mid z)]$ (3)
Reparameterization	$\tilde{z} = g_\phi(\epsilon, x)$, $\epsilon \sim p(\epsilon)$; ví dụ Gauss: $z = \mu + \sigma\epsilon$ (4)
SGVB tổng quát	$\tilde{\mathcal{L}}^A$ (6), Monte Carlo cho cả KL và reconstruction
SGVB phương sai thấp	$\tilde{\mathcal{L}}^B$ (7), KL closed-form + Monte Carlo cho reconstruction
Ước lượng minibatch	$\tilde{\mathcal{L}}^M = \frac{N}{M} \sum_{i=1}^{M} \tilde{\mathcal{L}}(\theta, \phi; x^{(i)})$ (8)
KL Gauss-Gauss closed-form (1 chiều)	$-D_{KL} = \frac{1}{2}(1 + \log\sigma^2 - \mu^2 - \sigma^2)$ (Phụ lục B)
Siêu tham số thí nghiệm	$M = 100$, $L = 1$, Adagrad, stepsize $\in \{0{,}01; 0{,}02; 0{,}1\}$

11. Kết luận ngắn

Auto-Encoding Variational Bayes đưa ra một thuật toán tổng quát (AEVB) để huấn luyện mô hình sinh có biến ẩn liên tục cùng một mạng suy luận xấp xỉ, bằng cách tối ưu một cận dưới biến phân khả vi qua reparameterization trick. Ước lượng SGVB cho phép backpropagation thông thường qua bước lấy mẫu ngẫu nhiên, giải quyết vấn đề phương sai cao của các ước lượng gradient biến phân trước đó. Thực nghiệm trên MNIST và Frey Face cho thấy AEVB hội tụ nhanh và ổn định hơn rõ rệt so với wake-sleep và Monte Carlo EM, dù bài không có bảng benchmark số liệu cố định mà chủ yếu trình bày qua đồ thị hội tụ. Framework này trở thành một trong hai nhánh lớn của mô hình sinh hiện đại (cùng GAN), và đặt nền cho nhiều mở rộng sau (VAE có cấu trúc latent phức tạp hơn, liên hệ tới các mô hình diffusion).

Nguồn gốc: https://arxiv.org/abs/1312.6114

Bài 16

Denoising Diffusion Probabilistic Models (2020)

Tác giả: Jonathan Ho, Ajay Jain, Pieter Abbeel (UC Berkeley).
Venue / arXiv: arXiv:2006.11239 (v2, 16/12/2020); trang bìa ghi rõ venue: 34th Conference on Neural Information Processing Systems (NeurIPS 2020), Vancouver, Canada.
Nguồn: https://arxiv.org/abs/2006.11239
Chủ đề: mô hình sinh dựa trên một quá trình khử nhiễu (denoising) Markov nhiều bước: học đảo ngược (reverse) một quá trình thuận (forward) thêm nhiễu Gauss dần vào dữ liệu, tham số hóa bằng cách dự đoán trực tiếp phần nhiễu ϵ đã thêm vào, huấn luyện bằng một mục tiêu đơn giản hóa giống denoising score matching.
Vị trí trong lộ trình: Đọc sau bài 14 về GAN và bài 15 về VAE để so sánh các mô hình sinh.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Diffusion probabilistic model (Sohl-Dickstein và cộng sự, 2015) là một mô hình sinh biến phân có dạng $p_\theta(x_0) := \int p_\theta(x_{0:T})\, dx_{1:T}$, trong đó $x_1, \ldots, x_T$ là các biến ẩn "latent" cùng chiều với dữ liệu x_0, và quá trình sinh là một **chuỗi Markov đảo ngược** (reverse chain) bắt đầu từ noise thuần $p(x_T) = N(0, I)$. Mô hình được ghép cặp với một quá trình thuận (forward/diffusion process) cố định, thêm dần nhiễu Gauss vào dữ liệu qua T bước cho tới khi tín hiệu bị phá hủy hoàn toàn.

Trước bài này, diffusion model được biết tới nhưng chưa chứng minh được khả năng sinh mẫu chất lượng cao cạnh tranh với GAN; bài này là công trình đầu tiên cho thấy diffusion model có thể sinh ảnh chất lượng cao, cạnh tranh trực tiếp với các mô hình sinh khác tại thời điểm công bố (2020).

Liên hệ đã biết trước đó (và được bài này làm rõ hơn): diffusion model có liên hệ chặt với **denoising score matching** và **Langevin dynamics** (qua các công trình như NCSN - Noise Conditional Score Networks của Song và Ermon, 2019, 2020); bài này chỉ ra một tham số hóa và một mục tiêu huấn luyện cụ thể khiến hai hướng (diffusion biến phân và score-based) gần như trùng khớp nhau về mặt thực hành.

2. Ý tưởng chính

Định nghĩa quá trình thuận q cố định (không học), thêm nhiễu Gauss nhỏ dần qua T bước theo một lịch trình phương sai $\beta_1, \ldots, \beta_T$ (công thức 2):

$$q(x_t|x_{t-1}) := N(x_t; \sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$$

và học quá trình đảo ngược p_θ (công thức 1), mỗi bước cũng là một Gauss có mean và (tùy biến thể) variance học được:

$$p_\theta(x_{t-1}|x_t) := N(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

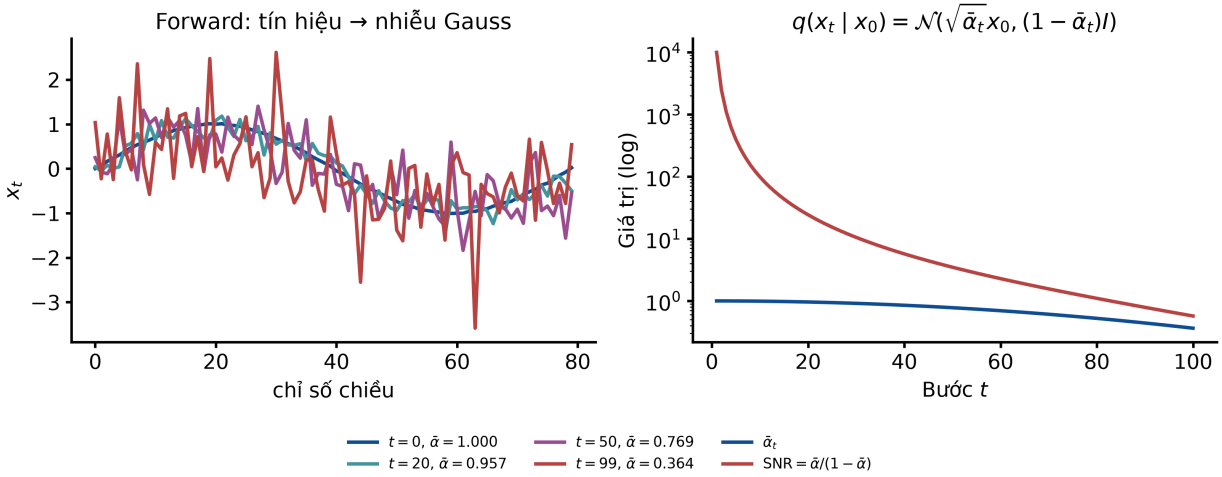
Thay vì tham số hóa μ_θ trực tiếp, đóng góp cốt lõi của bài là **tham số hóa μ_θ qua một mạng dự đoán nhiễu** $\epsilon_\theta(x_t, t)$: mạng học dự đoán chính phần nhiễu ϵ đã được thêm vào x_0 để tạo ra x_t, rồi suy ra μ_θ từ ϵ_θ bằng công thức giải tích cố định. Lựa chọn này, kết hợp với một mục tiêu huấn luyện **đơn giản hóa** (bỏ các hệ số trọng số trong cận dưới biến phân đầy đủ), biến việc huấn luyện thành gần như một bài toán hồi quy ℓ_2 đơn

giản giữa nhiễu thật và nhiễu dự đoán, và kết quả tương đương với một dạng **denoising score matching đa mức nhiễu** (multiple noise scales).

3. Phương pháp chi tiết

3.1 Cận dưới biến phân và tham số hóa forward (mục 2, 3.1)

Huấn luyện bằng tối thiểu hóa cận dưới biến phân thông thường của negative log-likelihood (công thức 3):



Forward DDPM $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ và SNR theo t (minh hoạ, tự tính)

$$\mathbb{E}\Big[-\log p_\theta(x_0)\Big] \le \mathbb{E}_q\Big[-\log p(x_T) - \sum_{t\ge 1}\log\frac{p_\theta(x_{t-1}|x_t)}{q(x_t|x_{t-1})}\Big] =: L$$

Nhờ tính chất Gauss của quá trình thuận, có công thức đóng cho $q(x_t|x_0)$ (đặt $\alpha_t := 1-\beta_t$, $\bar{\alpha}_t := \prod_{s=1}^{t}\alpha_s$, công thức 4):

$$q(x_t|x_0) = N(x_t; \sqrt{\bar{\alpha}_t}\,x_0, (1-\bar{\alpha}_t)I)$$

L được viết lại (công thức 5) thành tổng các số hạng KL dễ tính hơn: $L = L_T + \sum_{t>1} L_{t-1} + L_0$, trong đó $L_T = D_{KL}(q(x_T|x_0)\|p(x_T))$, $L_{t-1} = D_{KL}(q(x_{t-1}|x_t,x_0)\|p_\theta(x_{t-1}|x_t))$ cho $t > 1$, và $L_0 = -\log p_\theta(x_0|x_1)$. Posterior có điều kiện x_0 là một Gauss tường minh (công thức 6-7):

$$q(x_{t-1}|x_t,x_0) = N(x_{t-1}; \tilde{\mu}_t(x_t,x_0), \tilde{\beta}_t I)$$

với $\tilde{\mu}_t(x_t,x_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t$ và $\tilde{\beta}_t := \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t$.

Mục 3.1 cố định β_t là hằng số (không học qua gradient), nên L_T không phụ thuộc tham số θ và có thể bỏ qua khi huấn luyện (xấp xỉ bằng 0 nếu lịch trình β đủ tốt để $q(x_T|x_0)$ gần $N(0,I)$).

3.2 Tham số hóa L_{t-1}: dự đoán nhiễu ϵ (mục 3.2)

Mục 3.2 cố định $\Sigma_\theta(x_t,t) = \sigma_t^2 I$ (không học, hằng số theo thời gian; bài thử cả $\sigma_t^2 = \beta_t$ và $\sigma_t^2 = \tilde{\beta}_t$, cho kết quả tương tự). Khi đó L_{t-1} trở thành một sai số bình phương có trọng số giữa $\tilde{\mu}_t$ và μ_θ (công thức 8):

$$L_{t-1} = \mathbb{E}_q\Big[\frac{1}{2\sigma_t^2}\|\tilde{\mu}_t(x_t,x_0) - \mu_\theta(x_t,t)\|^2\Big] + C$$

Dùng reparameterization $x_t(x_0, \epsilon) = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ với $\epsilon \sim N(0, I)$ (công thức 9-10), thay vào $\tilde{\mu}_t$ và chọn tham số hóa (công thức 11):

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\Big(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\Big)$$

dẫn tới $L_{t-1} - C$ có dạng (công thức 12):

$$\mathbb{E}_{x_0,\epsilon}\Big[\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2\Big]$$

Bài nhận xét công thức này "giống denoising score matching qua nhiều mức nhiễu được đánh chỉ số bởi t", vì ϵ_θ về bản chất đang học một đại lượng tỷ lệ với score function $\nabla_{x_t} \log q(x_t)$.

3.3 Decoder rời rạc (mục 3.3) và mục tiêu đơn giản hóa L_{simple} (mục 3.4)

Vì dữ liệu ảnh gồm số nguyên $\{0, \dots, 255\}$ được scale tuyến tính về $[-1, 1]$, $p_\theta(x_0|x_1)$ được định nghĩa bằng cách tích phân một Gauss độc lập theo từng chiều pixel trên các "bin" rời rạc tương ứng (công thức 13), đảm bảo cận dưới biến phân là lossless codelength của dữ liệu rời rạc, không cần thêm nhiễu khử lượng tử hóa.

Mục tiêu huấn luyện thực tế, L_{simple} (công thức 14), bỏ hệ số trọng số trước $\|\cdot\|^2$ trong công thức (12):

$$L_{simple}(\theta) := \mathbb{E}_{t,x_0,\epsilon}\Big[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2\Big]$$

Bài nhận xét mục tiêu này hạ trọng số các số hạng t nhỏ (nơi nhiễu ít, nên các số hạng đó vốn có trọng số lớn trong (12) do $1/(1-\bar{\alpha}_t)$ nhỏ), khiến mạng tập trung học các nhiệm vụ khử nhiễu khó hơn ứng với t lớn; thực nghiệm cho thấy mục tiêu đơn giản hóa này cải thiện chất lượng mẫu sinh so với tối ưu đúng cận dưới biến phân có trọng số đầy đủ.

Thuật toán 1 (huấn luyện): lặp: lấy x_0 từ dữ liệu, $t \sim \text{Uniform}(\{1, \dots, T\})$, $\epsilon \sim N(0, I)$; lấy gradient của $\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2$ theo θ.

Thuật toán 2 (lấy mẫu): $x_T \sim N(0, I)$; với $t = T, \dots, 1$: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)) + \sigma_t z$, $z \sim N(0, I)$ (trừ $t = 1$ không thêm noise); trả về x_0.

3.4 Kiến trúc và siêu tham số (mục 4, Phụ lục B)

$T = 1000$ cho mọi thí nghiệm; lịch trình β tuyến tính từ $\beta_1 = 10^{-4}$ đến $\beta_T = 0{,}02$. Backbone là một **U-Net** tương tự PixelCNN++ không có mask, dùng group normalization, Transformer sinusoidal position embedding cho t, và self-attention tại độ phân giải 16×16. Tham số mô hình: CIFAR10 35,7M tham số; LSUN/CelebA-HQ (256^2) 114M tham số; biến thể LSUN Bedroom lớn hơn khoảng 256M tham số. Huấn luyện trên TPU v3-8; CIFAR chạy 21 bước/giây ở batch 128 (10,6 giờ cho 800k bước), sinh 256 ảnh tốn 17 giây; LSUN/CelebA-HQ (256^2) chạy 2,2 bước/giây ở batch 64, sinh 128 ảnh tốn 300 giây. Optimizer Adam, learning rate 2×10^{-4} (giảm xuống 2×10^{-5} cho ảnh 256×256, vì không ổn định ở lr cao hơn); EMA decay $0{,}9999$; dropout $0{,}1$ cho CIFAR10 (quét $\{0{,}1; 0{,}2; 0{,}3; 0{,}4\}$), 0 cho các bộ dữ liệu khác; lật ngang ngẫu nhiên dùng cho CIFAR10 và mọi bộ dữ liệu trừ LSUN Bedroom.

4. Thí nghiệm và kết quả chính

Bảng 1 (CIFAR10, không điều kiện/unconditional): mô hình tốt nhất của bài, "Ours (L_{simple})", đạt Inception Score (IS) $9{,}46 \pm 0{,}11$, FID $3{,}17$, NLL Test $\leq 3{,}75$ bits/dim (Train $\leq 3{,}72$). So sánh với các mô hình trước: Gated PixelCNN (IS $4{,}60$, NLL $3{,}03$), PixelIQN (IS $5{,}29$), EBM (IS $6{,}78$, FID $38{,}2$), NCSN (IS $8{,}87 \pm 0{,}12$, FID $25{,}32$), NCSNv2 (FID $10{,}87$), SNGAN (IS $8{,}22 \pm 0{,}05$), SNGAN-DDLS (IS $9{,}09 \pm 0{,}10$, FID

15,42). Mô hình của bài vượt mọi baseline unconditional này về FID, và cạnh tranh cả với một số mô hình **có điều kiện** (conditional): BigGAN FID 14,73; StyleGAN2+ADA FID 2,67 (conditional) / 3,26 (unconditional, FID thấp hơn FID 3,17 của bài này một chút).

Bảng 2 (ablation tham số hóa × mục tiêu huấn luyện): xem chi tiết đầy đủ ở mục 5.

Hình 1, 4, 11-17 (mẫu sinh định tính): ảnh CIFAR10, CelebA-HQ 256×256, LSUN Bedroom/Church/Cat 256×256.

Bảng 3 (LSUN, chỉ FID): so sánh ProgressiveGAN, StyleGAN, StyleGAN2 với "Ours (L_{simple})": Bedroom 6,36, Church 7,89, Cat 19,75; và "Ours (L_{simple}, large)" (biến thể 256M tham số, lớn hơn mô hình chuẩn): Bedroom 4,90. Số bước huấn luyện: CelebA-HQ 0,5M bước, LSUN Bedroom 2,4M bước, LSUN Cat 1,8M bước, LSUN Church 1,2M bước, biến thể LSUN Bedroom lớn 1,15M bước.

Mục 4.3 (progressive lossy compression): trên CIFAR10, mô hình tốt nhất đạt rate $= 1{,}78$ bits/dim $+$ distortion $= 1{,}97$ bits/dim (RMSE $0{,}95$ trên thang 0-255); khoảng cách train/test codelength $\leq 0{,}03$ bits/dim (cho thấy không overfitting). Thuật toán 3-4 định nghĩa một lược đồ nén/giải nén dần (progressive), nhưng bài tự nêu rõ đây **chỉ là minh chứng khái niệm** (proof of concept), dựa trên một thủ tục minimal random coding bất khả thi về mặt tính toán thực tế, không phải một hệ thống nén dùng được.

Mục 4.3 (liên hệ autoregressive): chỉ ra diffusion Gauss có thể viết lại thành một mô hình tự hồi quy tổng quát theo một thứ tự bit đặc biệt (công thức 16, dẫn chi tiết ở Phụ lục A).

Mục 4.4 (nội suy): nội suy giữa các ảnh CelebA-HQ 256×256 qua mã hóa/giải mã ngẫu nhiên (stochastic encoding qua quá trình thuận, rồi giải mã qua quá trình đảo ngược).

5. Phân tích / ablation

Bảng 2 là ablation trung tâm của bài, quét theo hai trục: tham số hóa của mean (dự đoán $\tilde{\mu}$ trực tiếp hay dự đoán ϵ) × loại variance (học được hay cố định) × mục tiêu huấn luyện (cận dưới biến phân đúng L hay mục tiêu đơn giản hóa L_{simple}):

Tham số hóa	Mục tiêu	IS	FID
$\tilde{\mu}$ dự đoán, Σ_θ học được (isotropic)	L	$7{,}28 \pm 0{,}10$	23,69
$\tilde{\mu}$ dự đoán, $\Sigma_\theta = \sigma_t^2$ cố định	L	$8{,}06 \pm 0{,}09$	13,22
$\tilde{\mu}$ dự đoán, $\|\tilde{\mu} - \tilde{\mu}_\theta\|^2$ (MSE trực tiếp trên μ)	-	(không ổn định)	(không ổn định)
ϵ dự đoán, Σ_θ học được (isotropic)	L	(không ổn định)	(không ổn định)
ϵ dự đoán, $\Sigma_\theta = \sigma_t^2$ cố định	L	$7{,}67 \pm 0{,}13$	13,51
ϵ dự đoán, $\Sigma_\theta = \sigma_t^2$ cố định	L_{simple}	$\mathbf{9{,}46 \pm 0{,}11}$	**3,17**

Kết luận ablation: tham số hóa dự đoán $\tilde{\mu}$ trực tiếp chỉ hoạt động tốt khi dùng đúng cận dưới biến phân L có trọng số đầy đủ; tham số hóa dự đoán ϵ (lựa chọn của bài) kết hợp với mục tiêu đơn giản hóa L_{simple} cho kết quả tốt nhất hẳn, cả về IS và FID; cả hai cách cho Σ_θ học được (diagonal) đều không ổn định trong huấn luyện của bài, bất kể tham số hóa mean nào.

Appendix C (so sánh với NCSN): khác biệt chính giữa DDPM và NCSN (Score Networks): kiến trúc U-Net+self-attention (DDPM) so với RefineNet+dilated convolution (NCSN); DDPM scale dữ liệu bằng hệ số $\sqrt{1-\beta_t}$ mỗi bước forward (NCSN không làm vậy); quá trình forward của DDPM phá hủy tín hiệu hoàn toàn ($D_{KL}(q(x_T|x_0)\|N(0,I)) \approx 0$), khác NCSN; các hệ số trong sampler của DDPM được suy ra chặt chẽ từ β_t (qua lý thuyết biến phân), còn NCSN đặt hệ số hậu-kiểm (post-hoc) bằng tay.

6. Giới hạn

Do tác giả nêu:

- Thuật toán nén dần (Thuật toán 3-4, mục 4.3) chỉ là minh chứng khái niệm, dựa trên thủ tục minimal random coding bất khả thi tính toán, "không phải một hệ thống nén thực tế".
- Hai biến thể cho Σ_θ học được (diagonal, isotropic) trong Bảng 2 đều không ổn định trong huấn luyện của bài (không có số liệu IS/FID báo cáo cho các dòng đó).
- Mục 3.2 nhắc có thử một tham số hóa dự đoán trực tiếp x_0 (thay vì ϵ hoặc $\tilde{\mu}$), và nhận thấy nó "dẫn tới chất lượng mẫu kém hơn ở giai đoạn đầu thí nghiệm của chúng tôi" - nhưng **thử nghiệm này không có dòng tương ứng trong Bảng 2**, chỉ được nhắc tới bằng văn xuôi, không được định lượng bằng số liệu cụ thể nào trong bài.
- Tốc độ lấy mẫu chậm vì cần $T = 1000$ bước Markov tuần tự cho mỗi mẫu sinh (ví dụ 300 giây cho 128 ảnh 256×256 như Phụ lục B nêu); đây là một giới hạn tốc độ nội tại của phương pháp được chính bài trình bày qua số liệu thời gian chạy, dù không gọi thẳng đây là "hạn chế" bằng từ ngữ đó.

Nhận xét:

- **Hình 4** ghi chú FID $= 4{,}90$ cho mẫu LSUN Bedroom; đối chiếu với **Bảng 3**, số này khớp chính xác với dòng "Ours (L_{simple}, large)" (biến thể 256M tham số), **không phải** mô hình kích thước chuẩn (có FID $= 6{,}36$ trong cùng Bảng 3). Người đọc cần cẩn thận không lẫn hai con số này khi trích dẫn "kết quả DDPM trên LSUN Bedroom".
- Rate $(1{,}78)$ + distortion $(1{,}97) = 3{,}75$ bits/dim khớp đúng với NLL Test $\leq 3{,}75$ của Bảng 1 cho mô hình L_{simple} trên CIFAR10 (tự tính, cộng hai số từ mục 4.3); đây là một điểm nhất quán nội tại tốt giữa hai phần của bài, không phải một mâu thuẫn, nhưng đáng ghi chú vì xác nhận khung rate-distortion ở mục 4.3 tương thích với con số NLL ở Bảng 1.
- StyleGAN2+ADA (unconditional) đạt FID $3{,}26$ trên CIFAR10, chỉ hơi kém FID $3{,}17$ của mô hình bài này; bài trình bày kết quả của mình là tốt nhất trong so sánh, nhưng khoảng cách với baseline mạnh nhất (StyleGAN2+ADA) là khá nhỏ, không phải một khoảng cách lớn áp đảo.

7. Liên hệ với các bài khác

- **ml basics 14** (cùng thư mục, GAN) và **ml basics 15** (cùng thư mục, VAE): hai họ mô hình sinh khác; DDPM dùng cận dưới biến phân cho một chuỗi Markov nhiều bước (mở rộng trực tiếp của công thức (2)-(3) ở bài VAE, số 15), còn GAN (bài 14) không dùng biến phân mà dùng trò chơi đối kháng. Bảng 2 (Challenges in generative modeling) của bài GAN gián tiếp đặt nền cho cách phân loại họ mô hình sinh mà DDPM kế tục.
- **math content/29_2022_understanding_diffusion_models.md**: bài tutorial "Understanding Diffusion Models: A Unified Perspective" trình bày lại chính các công thức biến phân (1)-(14) của DDPM dưới một góc nhìn thống nhất hơn (liên hệ VAE, score-based model), hữu ích để đọc song song khi cần một mạch suy luận toán học chậm hơn cho cùng nội dung.
- **optimization content/06_2014_adam.md**: DDPM dùng trực tiếp optimizer Adam cho mọi thí nghiệm (Phụ lục B, learning rate $2 \times 10^{-4}/2 \times 10^{-5}$), nên bài Adam là tài liệu nền cho chi tiết thực hành huấn luyện này.

8. Gợi ý áp dụng

DDPM cung cấp cách học quá trình khử nhiễu theo nhiều bước để sinh mẫu. Khi đánh giá, cần cân nhắc chất lượng mẫu cùng chi phí lấy mẫu tuần tự và ảnh hưởng của lịch trình nhiễu.

9. Thí nghiệm gợi ý

1. **Cài một DDPM tối giản trên MNIST hoặc CIFAR10 (ảnh nhỏ), $T = 200$ (giảm từ 1000 để chạy nhanh trên máy nhỏ):** U-Net nhỏ, mục tiêu L_{simple} (công thức 14). Đo FID xấp xỉ (hoặc chỉ đánh giá bằng mắt) sau một số epoch. Kỳ vọng: ảnh sinh ra có cấu trúc nhận ra được, minh họa cơ chế cốt lõi của bài ở quy mô nhỏ.
2. **So sánh dự đoán ϵ (công thức 11) với dự đoán $\tilde{\mu}$ trực tiếp, cùng mục tiêu L_{simple}-tương đương:** trên cùng kiến trúc nhỏ ở thí nghiệm 1, đo độ ổn định huấn luyện (loss có NaN/explode không) và chất lượng mẫu định tính. Kỳ vọng: dự đoán ϵ ổn định hơn, khớp quan sát của Bảng 2 rằng dự đoán $\tilde{\mu}$ trực tiếp cần đúng trọng số biến phân mới ổn định.
3. **Quét lịch trình β (tuyến tính với $\beta_T \in \{0{,}01; 0{,}02; 0{,}05\}$):** đo $\bar{\alpha}_T$ cuối cùng (tự tính: xấp xỉ $\exp(-\sum \beta_t)$ cho β nhỏ) và chất lượng mẫu. Kỳ vọng: β_T quá nhỏ khiến x_T chưa đủ gần noise thuần (vi phạm giả định $L_T \approx 0$), β_T quá lớn làm mất thông tin hữu ích ở các bước đầu; cần cân bằng.
4. **Đo thời gian lấy mẫu theo số bước T (hoặc số bước lấy mẫu rút gọn, nếu cài thêm DDIM-style skip, dù đây không có trong bài gốc):** đo số giây để sinh 1 ảnh với $T \in \{50, 200, 1000\}$ bước đầy đủ tuần tự. Kỳ vọng: thời gian tỉ lệ tuyến tính với T, minh họa cụ thể "giới hạn tốc độ lấy mẫu" đã nêu ở mục 6, và tại sao các công trình sau DDPM tập trung rút ngắn số bước lấy mẫu.
5. **Tự tính và kiểm tra $\bar{\alpha}_T$ cho lịch trình gốc của bài ($\beta_1 = 10^{-4}$, $\beta_T = 0{,}02$, $T = 1000$):** tính $\sum \beta_t \approx 1000 \times (10^{-4} + 0{,}02)/2 = 10{,}05$ (tự tính, trung bình cộng vì lịch trình tuyến tính), rồi $\bar{\alpha}_T = \prod(1 - \beta_t) \approx e^{-10{,}05} \approx 4{,}3 \times 10^{-5}$ (tự tính, dùng xấp xỉ $\log(1 - x) \approx -x$ cho β nhỏ). So sánh code thực tế tính chính xác $\bar{\alpha}_T$ bằng tích lũy, xem khớp xấp xỉ tay tới đâu. Kỳ vọng: giá trị rất nhỏ, xác nhận x_T gần như noise thuần, khớp phát biểu của Appendix C rằng quá trình forward "phá hủy tín hiệu hoàn toàn".

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Reverse process	$p_\theta(x_{t-1} \mid x_t) = N(x_{t-1}; \mu_\theta, \Sigma_\theta)$ (1)
Forward process	$q(x_t \mid x_{t-1}) = N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$ (2)
Forward đóng dạng	$q(x_t \mid x_0) = N(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$ (4)
Tham số hóa mean qua ϵ	$\mu_\theta = \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t))$ (11)
Mục tiêu đơn giản hóa	$L_{simple} = \mathbb{E}[\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, t)\|^2]$ (14)
T, lịch trình β	$T = 1000$; $\beta_1 = 10^{-4} \to \beta_T = 0{,}02$ (tuyến tính)
$\bar{\alpha}_T$ cho lịch trình gốc (tự tính)	$\sum \beta_t \approx 10{,}05 \Rightarrow \bar{\alpha}_T \approx e^{-10{,}05} \approx 4{,}3 \times 10^{-5}$
Kết quả tốt nhất (CIFAR10)	IS $9{,}46 \pm 0{,}11$, FID $3{,}17$, NLL $\leq 3{,}75$ bits/dim
LSUN Bedroom (mô hình chuẩn / lớn)	FID $6{,}36$ / FID $4{,}90$ (256M tham số)

11. Kết luận ngắn

DDPM huấn luyện một mô hình sinh dựa trên khử nhiễu nhiều bước bằng cách tham số hóa mạng để dự đoán nhiễu Gauss đã thêm vào, và tối ưu một mục tiêu đơn giản hóa (bỏ trọng số biến phân đầy đủ) giống denoising score matching. Kết quả thực nghiệm trên CIFAR10 (FID $3{,}17$) và LSUN (FID tốt nhất $4{,}90$ với mô hình lớn) cạnh tranh hoặc vượt các mô hình sinh ảnh mạnh nhất tại thời điểm công bố, bao gồm một số mô hình GAN có điều kiện. Bảng ablation (Bảng 2) cho thấy lựa chọn tham số hóa dự đoán ϵ kết hợp mục tiêu đơn giản hóa là then chốt cho chất lượng mẫu; hạn chế chính là tốc độ lấy mẫu chậm (cần $T = 1000$ bước Markov tuần tự), một vấn đề các công trình sau này (DDIM và các phương pháp lấy mẫu nhanh) tập trung giải quyết.

Nguồn gốc: https://arxiv.org/abs/2006.11239

Bài 17

A Simple Framework for Contrastive Learning of Visual Representations (2020)

Tác giả: Ting Chen, Simon Kornblith, Mohammad Norouzi, Geoffrey Hinton (Google Research, Brain Team).
Venue / arXiv: arXiv:2002.05709 (v3, 01/07/2020); Proceedings of the 37th International Conference on Machine Learning (ICML 2020), PMLR 119, theo paper ghi.
Nguồn: https://arxiv.org/abs/2002.05709
Chủ đề: SimCLR, một khung contrastive learning tự giám sát (self-supervised) đơn giản để học biểu diễn ảnh, không cần kiến trúc đặc biệt hay memory bank.
Vị trí trong lộ trình: Đọc cùng bài 18 về MoCo: cả hai học biểu diễn tương phản nhưng tổ chức mẫu đối chiếu khác nhau.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Trước 2020, học biểu diễn ảnh không cần nhãn (unsupervised/self-supervised representation learning) có hai nhóm hướng chính mà bài báo liệt kê: (a) **generative**, học sinh ra hoặc mô hình hoá pixel đầu vào (ví dụ deep belief net, VAE, GAN; xem ml basics 14, 15); (b) **discriminative** dựa trên pretext task thủ công (dự đoán vị trí patch, giải jigsaw, tô màu, dự đoán góc xoay). Nhóm thứ hai gần đây chuyển sang **contrastive learning**: huấn luyện encoder sao cho hai "view" (biến thể) của cùng một ảnh có biểu diễn gần nhau, còn view của ảnh khác thì xa nhau.

Khái niệm cần làm rõ trước khi đọc (người mới gặp lần đầu):

- **View/augmentation**: biến đổi ngẫu nhiên một ảnh (cắt, đổi màu, làm mờ...) để tạo ra bản "khác đi nhưng cùng nội dung" của ảnh gốc.
- **Positive pair / negative pair**: hai view từ cùng một ảnh gốc là cặp dương (positive); view từ hai ảnh khác nhau là cặp âm (negative).
- **Linear evaluation protocol**: sau khi học biểu diễn không nhãn, đóng băng (freeze) encoder, chỉ huấn luyện một lớp tuyến tính (linear classifier) ở trên bằng nhãn thật, rồi đo accuracy trên tập test. Đây là cách chuẩn để so sánh chất lượng biểu diễn giữa các phương pháp tự giám sát.
- **Memory bank**: một bộ lưu các vector biểu diễn của toàn bộ (hoặc một phần lớn) dữ liệu huấn luyện từ các bước trước, dùng làm nguồn negative mà không cần tính lại trong mỗi batch (cách làm của Wu et al. 2018, và cũng là nền cho MoCo ở ml basics 18).

Câu hỏi của bài báo: những thành phần nào trong một khung contrastive học đơn giản thực sự quan trọng, và có thể thay memory bank/kiến trúc đặc biệt bằng điều gì đơn giản hơn mà vẫn mạnh?

2. Ý tưởng chính

SimCLR học biểu diễn bằng cách **tối đa hoá sự đồng thuận (agreement)** giữa hai view tăng cường khác nhau của cùng một ảnh, dùng một hàm loss đối nghịch (contrastive) trong không gian ẩn, **không cần** kiến trúc chuyên biệt và **không cần** memory bank: mọi ảnh khác trong cùng minibatch (sau khi tăng cường) đóng vai trò negative. Ba phát hiện chính (trích từ abstract, mục 1):

1. **Tổ hợp (composition) nhiều phép tăng cường dữ liệu** là yếu tố quyết định để có predictive task tốt; contrastive learning tự giám sát cần tăng cường **mạnh hơn** so với học có giám sát.
2. Thêm một **phép biến đổi phi tuyến có thể học (projection head)** giữa biểu diễn và loss contrastive cải thiện đáng kể chất lượng biểu diễn (nhưng biểu diễn dùng cho downstream task nên lấy *trước* projection head, không phải sau).
3. Contrastive learning có lợi nhiều hơn supervised learning từ **batch size lớn và huấn luyện lâu hơn**; cũng giống supervised, nó có lợi từ mạng sâu và rộng hơn.

Kết quả: dưới linear evaluation protocol trên ImageNet ILSVRC-2012, SimCLR đạt 76.5% top-1 accuracy, cải thiện tương đối 7% so với SOTA trước đó (Hénaff et al. 2019), và khi fine-tune với chỉ 1% nhãn ImageNet đạt 85.8% top-5 accuracy (cải thiện tương đối 10%).

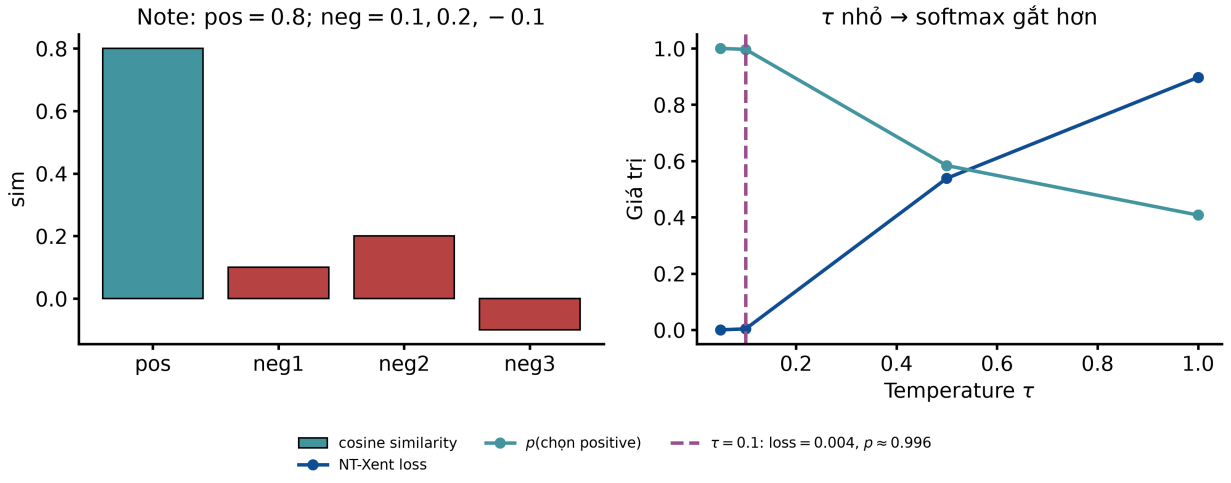
3. Phương pháp chi tiết

3.1 Bốn thành phần của khung (Hình 2, Mục 2.1)

1. **Module tăng cường dữ liệu ngẫu nhiên**: biến một ảnh x thành hai view tương quan $\tilde{x}_i$, $\tilde{x}_j$ (một "positive pair"), bằng cách lấy ngẫu nhiên hai hàm tăng cường $t \sim \mathcal{T}$, $t' \sim \mathcal{T}$ từ cùng một họ $\mathcal{T}$. Mặc định: **random crop + resize** (kèm flip ngang) rồi **random color distortion** rồi **random Gaussian blur**, áp dụng tuần tự. Paper nhấn mạnh: tổ hợp **crop + color distortion** là quan trọng nhất.
2. **Base encoder** $f(\cdot)$: trích đặc trưng $h_i = f(\tilde{x}_i) = \text{ResNet}(\tilde{x}_i) \in \mathbb{R}^d$, lấy sau lớp average pooling. Mặc định dùng ResNet-50.
3. **Projection head** $g(\cdot)$: một MLP một lớp ẩn, $z_i = g(h_i) = W^{(2)}\sigma(W^{(1)}h_i)$ với σ là ReLU, chiếu h xuống không gian 128 chiều để tính loss contrastive. Paper thấy nên tính loss trên z_i thay vì h_i (xem mục 5).
4. **Hàm loss contrastive**: cho một tập $\{\tilde{x}_k\}$ chứa một cặp dương $\tilde{x}_i, \tilde{x}_j$, bài toán là nhận ra $\tilde{x}_j$ trong số $\{\tilde{x}_k\}_{k\neq i}$ khi biết $\tilde{x}_i$.

3.2 Lấy minibatch làm nguồn negative, không memory bank (Mục 2.1, 2.2)

Lấy ngẫu nhiên minibatch N ảnh, tăng cường mỗi ảnh hai lần ra $2N$ điểm dữ liệu. Không lấy mẫu negative riêng; với một cặp dương (i, j), $2(N-1)$ **điểm tăng cường còn lại trong batch** đóng vai trò negative. Gọi $\text{sim}(u, v) = u^\top v/(\|u\|\|v\|)$ là cosine similarity (ℓ_2-normalize trước khi nhân). Loss cho một cặp dương (i, j) (phương trình 1, gọi là **NT-Xent**, Normalized Temperature-scaled Cross Entropy):



NT-Xent với sim note; $\tau = 0.1$ cho $p \approx 0.996$ — τ nhỏ làm softmax gắt (minh hoạ, tự tính)

$$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$$

với τ là temperature. Loss cuối cùng tính trên **tất cả** cặp dương (cả (i, j) và (j, i)) trong batch:

$$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} \left[\ell(2k-1, 2k) + \ell(2k, 2k-1)\right]$$

Tự tính: với batch size N = 256, mỗi anchor có $2(N-1) = 510$ negative; với N = 8192 (batch lớn nhất paper thử), mỗi anchor có $2 \times 8191 = 16382$ negative, đúng số paper nêu ("16382 negative examples per positive pair").

3.3 Huấn luyện với batch lớn (Mục 2.2)

- Không dùng memory bank; thay vào đó thay đổi batch size N từ 256 đến 8192.
- SGD/Momentum với linear LR scaling không ổn định ở batch lớn, nên dùng optimizer **LARS** (Layer-wise Adaptive Rate Scaling, You et al. 2017) cho mọi batch size. Huấn luyện trên Cloud TPU, 32-128 core tuỳ batch size (batch 4096 trên 128 TPU v3 core mất khoảng 1.5 giờ cho 100 epoch).
- **Global BN**: BatchNorm thường tổng hợp mean/variance cục bộ trên mỗi device khi huấn luyện song song dữ liệu (data parallel). Vì các cặp dương được tính trên cùng device, model có thể "rò rỉ thông tin cục bộ" (local information leakage) để đoán đúng mà không cần học biểu diễn tốt hơn. Paper sửa bằng cách tổng hợp mean/variance BN trên **toàn bộ** các device trong khi huấn luyện.

3.4 Thiết lập mặc định (Mục 2.3)

Encoder ResNet-50, projection head MLP 2 lớp chiếu về không gian 128 chiều, loss NT-Xent, optimizer LARS với learning rate $4.8 = 0.3 \times \text{BatchSize}/256$ và weight decay 10^{-6}, batch size 4096, 100 epoch, warmup 10 epoch đầu, cosine decay learning rate không restart. Đánh giá chủ yếu trên ImageNet ILSVRC-2012 bằng linear evaluation protocol; ngoài ra có vài thí nghiệm pretraining trên CIFAR-10 (Appendix B.9, không đọc chi tiết ở ghi chú này).

3.5 Augmentation: vì sao phải tổ hợp (Mục 3)

Paper thử 10 phép tăng cường đơn lẻ và từng cặp (Hình 4, 5): crop, cutout, color jitter/drop, Sobel filter, Gaussian noise, Gaussian blur, rotate. Không phép đơn lẻ nào đủ để học biểu diễn tốt (model vẫn phân biệt được cặp dương gần như hoàn hảo trong pretext task, nhưng biểu diễn kém), còn tổ hợp (đặc biệt **crop + color distortion**) cải thiện mạnh. Lý do nêu trong paper: nếu chỉ dùng random crop, các patch của cùng một ảnh có phân phối màu (histogram màu) rất giống nhau (Hình 6), nên model có thể "ăn gian" (shortcut) bằng cách so màu thay vì học đặc trưng tổng quát; thêm color distortion phá vỡ shortcut này.

Ví dụ số (tự tính minh hoạ ý tưởng, không phải số trong paper): giả sử hai view của một ảnh có cosine similarity $\text{sim}(z_i, z_j) = 0.8$, còn trung bình cosine similarity với 3 negative khác trong batch nhỏ là $0.1, 0.2, -0.1$. Với $\tau = 0.1$:

$$\ell = -\log \frac{e^{0.8/0.1}}{e^{0.8/0.1} + e^{0.1/0.1} + e^{0.2/0.1} + e^{-0.1/0.1}} = -\log \frac{e^{8}}{e^{8} + e^{1} + e^{2} + e^{-1}} \approx -\log(0.9923) \approx 0.0077$$

(tự tính: $e^8 \approx 2981$, $e^1 + e^2 + e^{-1} \approx 2.72 + 7.39 + 0.37 = 10.48$, tỉ lệ $2981/(2981 + 10.48) \approx 0.9965$; sai số nhỏ do làm tròn). Loss nhỏ khi positive similarity vượt xa negative; τ nhỏ làm hàm softmax "gắt" hơn, khuếch đại khoảng cách tương đối.

4. Thí nghiệm và kết quả chính

- **So với SOTA (Bảng 6)**: với ResNet-50, SimCLR đạt 69.3% top-1 (24M tham số), vượt MoCo (60.6%), PIRL (63.6%), CPC v2 (63.8%) cùng kích thước. Với ResNet-50 (2x) đạt 74.2%, với ResNet-50 (4x) đạt **76.5% top-1 / 93.2% top-5** (375M tham số), khớp với mô hình supervised ResNet-50 cùng kích thước theo Hình 1.
- **Semi-supervised (Bảng 7)**: fine-tune trên 1%/10% nhãn ImageNet (lấy mẫu theo lớp, khoảng 12.8/128 ảnh mỗi lớp): SimCLR ResNet-50 đạt 75.5%/87.8% top-5; ResNet-50 (4x) đạt **85.8% top-5** với 1% nhãn, cải thiện tương đối khoảng 10% so với CPC v2 (trước đó SOTA).
- **Transfer learning (Bảng 8)**: trên 12 tập ảnh tự nhiên (Food, CIFAR-10/100, Birdsnap, SUN397, Cars, Aircraft, VOC2007, DTD, Pets, Caltech-101, Flowers), ResNet-50 (4x) sau fine-tune tốt hơn có ý nghĩa thống kê (permutation test, $p > 0.05$ coi là không khác biệt) so với baseline supervised trên 5/12 tập, thua trên 2/12 (Pets, Flowers), còn lại coi như ngang nhau.
- **Mô hình lớn hơn (Hình 7)**: tăng độ sâu và độ rộng đều cải thiện; khoảng cách giữa supervised và SimCLR (linear eval) hẹp lại khi tăng kích thước model, gợi ý contrastive learning tự giám sát có lợi *nhiều hơn* từ model lớn so với supervised.
- Paper không có "benchmark lý thuyết" kiểu định lý; đây là paper thực nghiệm (empirical), toàn bộ phần 4-6 là bảng số và hình.

5. Phân tích / ablation

- **Màu mạnh hơn giúp SimCLR nhưng không giúp supervised (Bảng 1)**: ở cường độ color distortion mặc định (1 + blur), SimCLR đạt 64.5% top-1 (tốt nhất trong các mức thử), còn supervised giảm nhẹ khi tăng cường màu mạnh hơn (từ 77.0% ở độ mạnh 1/8 xuống 75.4% ở mức 1+blur). AutoAugment (chính sách tăng cường học bằng supervised) chỉ đạt 61.1% cho SimCLR, **kém hơn** tổ hợp crop + color distortion thủ công.
- **Projection head phi tuyến (Hình 8, Bảng 3)**: linear evaluation tốt hơn 3% nếu dùng projection phi tuyến so với tuyến tính, và hơn 10% so với không có projection. Quan trọng hơn: biểu diễn h (trước projection) tốt hơn $z = g(h)$ (sau projection) hơn 10%, vì z bị huấn luyện để bất biến (invariant) với phép biến đổi, nên mất thông tin như màu/hướng của vật thể mà h còn giữ (kiểm chứng bằng việc train MLP phụ để đoán biến đổi đã áp dụng từ h hay $g(h)$, Bảng 3: ví dụ đoán "màu hay xám" chính xác 99.3% từ h so với 97.4% từ $g(h)$; đoán "rotation" chính xác 67.6% từ h so với 25.6% (ngang random) từ $g(h)$).
- **Loss function (Bảng 2, 4)**: NT-Xent (có ℓ_2-norm + temperature) tốt hơn margin loss và logistic loss, dù đã áp dụng semi-hard negative mining cho hai loss sau (NT-Xent: 63.9% so với margin (sh) 57.5%, NT-Logistic (sh) 57.9%).
- **ℓ_2-norm và temperature (Bảng 5)**: không chuẩn hoá ℓ_2 cho task accuracy (entropy) cao hơn nhưng biểu diễn kém hơn dưới linear evaluation; $\tau = 0.1$ cho top-1 tốt nhất (64.4%) trong các giá trị thử với ℓ_2-norm (0.05, 0.1, 0.5, 1); không ℓ_2-norm với τ lớn (10, 100) cho kết quả còn kém hơn nữa (57.0-57.2%).
- **Batch size và số epoch (Hình 9)**: ở 100 epoch, batch lớn (như 8192) có lợi thế rõ so với batch nhỏ (256); huấn luyện lâu hơn thì khoảng cách giữa các batch size co lại hoặc biến mất, miễn là negative được lấy mẫu lại ngẫu nhiên. Phụ lục B.1-B.2 (đã đọc): batch 8192 có vẻ bão hoà hiệu năng, train lâu hơn (tới 3200 epoch) vẫn cải thiện thêm; với learning rate scaling theo căn bậc hai (LR $= 0.075\sqrt{\text{BatchSize}}$) tốt hơn linear scaling cho batch nhỏ/epoch ít khi dùng LARS (hai công thức trùng giá trị tại batch 4096: $0.3 \times 4096/256 = 4.8$ và $0.075\sqrt{4096} = 4.8$, paper nêu rõ cách quy đổi này; tự tính xác nhận cả hai cho cùng 4.8).
- **Tổ hợp augmentation rộng hơn (Appendix B.2)**: thêm Sobel, color distortion mở rộng (equalize, solarize), motion blur cải thiện thêm (ResNet-50 (4x): 76.8% so với 76.5% mặc định).

6. Giới hạn

Do tác giả nêu/ngụ ý:

- Paper tự nói "almost perfectly identify positive pairs" ở pretext task dù biểu diễn kém với augmentation yếu, nghĩa là **task accuracy của chính pretext task không phải chỉ số tốt** để đánh giá chất lượng biểu diễn; cần linear evaluation riêng.
- Kết quả tốt nhất cần huấn luyện rất lâu (1000 epoch cho Bảng 6/7/8) và batch rất lớn (4096–8192, cần TPU 32-128 core); đây là chi phí tính toán lớn so với supervised training thông thường.
- Thuật ngữ "hình dạng mô hình ít quan trọng" không được paper này thảo luận (khác với scaling laws, ml basics 25 không có trong tập này); SimCLR chỉ xét độ sâu/rộng của ResNet.

Do người ghi chú nhận thấy:

- Phạm vi đã đọc: toàn bộ phần chính (Mục 1-8) và Phụ lục A (chi tiết augmentation), B.1-B.7 (batch size/epoch, augmentation mở rộng, longer training supervised, non-linear projection, semi-supervised fine-tuning, linear evaluation, correlation linear/fine-tuning); **không đọc chi tiết** Appendix B.8 (transfer learning setup), B.9 (CIFAR-10), B.10 (tuning loss khác), C (so sánh thiết kế với các paper trước): các số liệu trong các phần này (nếu có) không được dùng ở đây.
- Toàn bộ thí nghiệm trên ảnh tự nhiên (ImageNet và vài tập ảnh chuẩn khác); không có thí nghiệm trên văn bản, tài liệu scan, hay OCR.
- SimCLR cần nhiều negative trong cùng batch để hoạt động tốt, nên phụ thuộc mạnh vào batch size lớn, một ràng buộc không phù hợp với GPU nhỏ (xem mục 8).

7. Liên hệ với các bài khác

- **ml basics 07** (Deep Residual Learning): kiến trúc ResNet-50 dùng làm base encoder $f(\cdot)$ của SimCLR.
- **ml basics 18** (MoCo, cùng thư mục): giải quyết cùng vấn đề (cần nhiều negative) bằng dictionary dạng queue + momentum encoder thay vì batch lớn; đọc liền sau bài này để so sánh hai triết lý.
- **distillation content/13_2020_contrastive_representation_distillation.md**: dùng ý tưởng contrastive loss (rất gần NT-Xent) để truyền tri thức giữa teacher và student, áp dụng trực tiếp công thức ở mục 3.2 của bài này sang bài toán distillation.
- **distillation content/22_2021_dino.md**: học tự giám sát bằng self-distillation, thừa hưởng trực giác "augmentation mạnh tạo view khác nhau của cùng ảnh" từ dòng SimCLR/MoCo.
- **llm content/30_2021_clip.md** và **llm content/31_2023_siglip.md**: cũng dùng contrastive loss dạng NT-Xent/softmax trong batch (CLIP) để khớp ảnh-văn bản; cùng gốc lý thuyết InfoNCE.

8. Gợi ý áp dụng

Trong học biểu diễn tương phản, các phép tăng cường dữ liệu xác định những biến đổi mà biểu diễn cần chịu được. Nên chọn augmentation phù hợp với tác vụ và đánh giá biểu diễn trên dữ liệu tách biệt.

9. Thí nghiệm gợi ý

1. **Đo ảnh hưởng của batch size nhỏ lên NT-Xent (toy).** Lấy một tập ảnh nhỏ (ví dụ CIFAR-10 hoặc một tập trang OCR nhỏ), huấn luyện SimCLR thu nhỏ (ResNet-18) với batch 32, 128, 512 (giới hạn bởi VRAM); đo linear evaluation accuracy. Kỳ vọng: accuracy giảm rõ khi batch nhỏ, đúng xu hướng Hình 9 của paper; diễn giải: xác nhận batch nhỏ là nút thắt thật cho NT-Xent trước khi quyết định có dùng contrastive loss kiểu này trên GPU nhỏ hay không.
2. **So sánh biểu diễn h so với $z = g(h)$ cho một downstream task nhỏ.** Huấn luyện SimCLR toy trên CIFAR-10, sau đó thử linear probe cả trên h và z cho một task phụ như "ảnh có bị rotate 90 độ không". Kỳ vọng: h cho probe accuracy cao hơn z đáng kể (giống Bảng 3 paper); diễn giải: xác nhận hiện tượng "head vứt thông tin" có xảy ra ở quy mô nhỏ, không chỉ ở ImageNet.

3. **Ablation temperature τ trên toy set.** Cố định mọi thứ, chỉ đổi $\tau \in \{0.05, 0.1, 0.5, 1\}$, đo contrastive task accuracy và linear eval accuracy. Kỳ vọng: giống Bảng 5, τ quá nhỏ hoặc quá lớn đều làm linear eval kém hơn mức trung bình (0.1 hoặc gần đó); diễn giải: minh hoạ trade-off giữa "độ gắt" loss và chất lượng biểu diễn học được.
4. **Thử tổ hợp augmentation trên ảnh scan/tài liệu toy.** Với một tập nhỏ ảnh trang tài liệu, so sánh linear eval khi dùng chỉ crop, chỉ đổi màu/độ sáng, và tổ hợp cả hai. Kỳ vọng định hướng (suy luận): tổ hợp vẫn tốt hơn từng augmentation riêng, nhưng mức cải thiện và augmentation phù hợp (ví dụ làm mờ có thể không hợp với OCR vì xoá chi tiết chữ) có thể khác hẳn ảnh tự nhiên; diễn giải: giúp quyết định augmentation phù hợp domain OCR trước khi áp dụng ý tưởng SimCLR.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Cosine similarity	$\text{sim}(u, v) = u^\top v / (\|u\|\|v\|)$
NT-Xent (loss cho một cặp)	$\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(\text{sim}(z_i, z_k)/\tau)}$
Loss toàn batch	$\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
Số negative mỗi anchor	$2(N-1)$ với N = batch size
Projection head	$z = g(h) = W^{(2)} \sigma(W^{(1)} h)$, σ = ReLU
Learning rate LARS (linear scaling)	$\text{LR} = 0.3 \times \text{BatchSize}/256$
Learning rate LARS (sqrt scaling)	$\text{LR} = 0.075\sqrt{\text{BatchSize}}$
Mặc định	ResNet-50, projection 128-D, $\tau = 0.1$, batch 4096, 100 epoch, weight decay 10^{-6}
Kết quả chính	top-1 linear eval 76.5% (R50 4x); 85.8% top-5 fine-tune với 1% nhãn

11. Kết luận ngắn

SimCLR cho thấy một khung contrastive học tự giám sát, không cần kiến trúc đặc biệt hay memory bank, có thể đạt chất lượng biểu diễn ngang supervised pretraining trên ImageNet, nhờ ba yếu tố: tổ hợp augmentation mạnh (đặc biệt crop + color distortion), projection head phi tuyến tách biệt loss contrastive khỏi biểu diễn downstream, và batch size/thời gian huấn luyện lớn. Hạn chế chính cho ứng dụng với hardware nhỏ là chính nhu cầu batch lớn này; MoCo (ml basics 18) giải quyết hạn chế đó theo hướng khác.

Nguồn gốc: https://arxiv.org/abs/2002.05709

Bài 18

Momentum Contrast for Unsupervised Visual Representation Learning (2019)

Tác giả: Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, Ross Girshick (Facebook AI Research).
Venue / arXiv: arXiv:1911.05722 (v3, 23/03/2020); paper không nêu venue hội nghị (đây chính là bản sau được công bố ở CVPR 2020, nhưng text arXiv không ghi dòng venue).
Nguồn: https://arxiv.org/abs/1911.05722
Chủ đề: MoCo, cơ chế contrastive learning tự giám sát dùng "dictionary" các key dạng hàng đợi (queue) cùng encoder key cập nhật theo trung bình động (momentum), tách kích thước dictionary khỏi kích thước minibatch.
Vị trí trong lộ trình: Đọc cùng bài 17 về SimCLR để so sánh cách xây dựng tập mẫu đối chiếu.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Paper mở đầu bằng quan sát: học không giám sát rất thành công trong NLP (GPT, BERT) nhưng vẫn kém hơn supervised pretraining trong computer vision. Lý do đề xuất: ngôn ngữ có không gian tín hiệu rời rạc (từ, sub-word) dễ xây "dictionary" (bảng từ) để học không giám sát trên đó; ảnh là tín hiệu liên tục, nhiều chiều, không có cấu trúc "từ điển" tự nhiên.

Khái niệm cần nắm trước (người mới gặp lần đầu khái niệm contrastive learning nên đọc ml basics 17 mục 1 trước):

- **Contrastive learning như "dictionary look-up"**: coi một tập biểu diễn $\{k_0, k_1, ...\}$ là các "key" của một từ điển; một "query" q (biểu diễn của view đang xét) cần được huấn luyện để giống key dương k_+ (cùng ảnh gốc) và khác các key âm còn lại.
- **InfoNCE loss**: dạng loss contrastive cụ thể dùng trong bài (giống NT-Xent ở ml basics 17 nhưng viết theo ngôn ngữ "dictionary"), công thức ở mục 3.
- **End-to-end** và **memory bank**: hai cơ chế có trước MoCo để lấy key, mỗi cơ chế có đánh đổi riêng (mục 3 bên dưới).

Giả thuyết trung tâm của bài báo: để contrastive learning học tốt, dictionary (tập key) cần **lớn** (bao quát không gian liên tục nhiều chiều của ảnh) và **nhất quán** (các key được biểu diễn bởi encoder giống hoặc gần giống nhau xuyên suốt quá trình tiến hoá).

2. Ý tưởng chính

MoCo (Momentum Contrast) xây dictionary động bằng hai cơ chế:

1. **Queue**: dictionary là một hàng đợi các biểu diễn key; minibatch hiện tại được "nhập hàng" (enqueue), minibatch cũ nhất bị "xuất hàng" (dequeue). Nhờ vậy **kích thước dictionary tách khỏi kích thước minibatch**, có thể lớn hơn minibatch nhiều lần, chỉ bị giới hạn bởi bộ nhớ lưu key (không phải bởi GPU memory cho backprop).
2. **Momentum update cho key encoder**: vì không thể backprop qua tất cả key trong queue (chúng được tính từ các bước trước), MoCo copy tham số từ query encoder θ_q sang key encoder θ_k bằng trung bình động: $\theta_k \leftarrow m\theta_k + (1-m)\theta_q$, với m lớn (mặc định $m = 0.999$) để θ_k tiến hoá **chậm và mượt**, giữ các key trong queue (dù được tính ở các bước khác nhau) nhất quán với nhau.

Pretext task dùng để minh hoạ cơ chế (không phải trọng tâm đóng góp, paper nói rõ): instance discrimination đơn giản, giống SimCLR, hai view tăng cường của cùng ảnh là cặp dương.

3. Phương pháp chi tiết

3.1 InfoNCE loss (Mục 3.1, phương trình 1)

Với query q và tập key $\{k_0, ..., k_K\}$ trong đó có một key dương k_+:

$$\mathcal{L}_q = -\log \frac{\exp(q \cdot k_+/\tau)}{\sum_{i=0}^{K} \exp(q \cdot k_i/\tau)}$$

τ là temperature. Về bản chất đây là log loss của một bộ phân loại softmax $(K+1)$-chiều cố gắng gán q vào lớp k_+. $q = f_q(x^q)$, $k = f_k(x^k)$ với f_q, f_k là hai encoder (có thể giống nhau, chia sẻ phần nào, hoặc khác nhau tuỳ cách tiếp cận).

3.2 Ba cơ chế xây dictionary (Hình 2, mục 3.2)

- **(a) End-to-end**: cả f_q và f_k cập nhật bằng backprop; key lấy từ chính minibatch hiện tại nên luôn nhất quán, nhưng kích thước dictionary bị giới hạn bởi kích thước minibatch (và bộ nhớ GPU); huấn luyện batch lớn là vấn đề mở (tham chiếu Goyal et al. 2017).
- **(b) Memory bank** (Wu et al. 2018): lưu biểu diễn của **toàn bộ** tập dữ liệu; mỗi bước lấy mẫu ngẫu nhiên dictionary từ memory bank, không backprop qua memory bank nên dictionary có thể rất lớn; nhược điểm: các key trong một dictionary được tính ở các epoch/bước khác nhau trong quá khứ nên **kém nhất quán**.
- **(c) MoCo**: dùng queue (kích thước K độc lập với batch) **và** momentum update cho f_k để vừa có dictionary lớn, vừa giữ nhất quán, mà không cần lưu biểu diễn của toàn bộ dataset như memory bank (nên nhẹ hơn, có thể scale lên tỉ ảnh).

3.3 Pseudocode (Algorithm 1, PyTorch-like, trích)

```
f_k.params = f_q.params                          # khởi tạo
for x in loader:
    x_q = aug(x); x_k = aug(x)                   # hai view tăng cường
    q = f_q.forward(x_q); k = f_k.forward(x_k).detach()
    l_pos = bmm(q.view(N,1,C), k.view(N,C,1)) # Nx1
    l_neg = mm(q.view(N,C), queue.view(C,K))  # NxK
    logits = cat([l_pos, l_neg], dim=1)
    loss = CrossEntropyLoss(logits/t, labels=zeros(N))  # positive luôn ở vị trí 0
    loss.backward(); update(f_q.params)
    f_k.params = m*f_k.params + (1-m)*f_q.params         # momentum update
    enqueue(queue, k); dequeue(queue)
```

3.4 Kỹ thuật “shuffling BN” (mục 3.3)

BatchNorm chuẩn làm “rò rỉ” thông tin giữa các mẫu trong cùng batch (giống vấn đề SimCLR gặp, ml basics 17 mục 3.3, nhưng MoCo giải quyết khác). Vì cặp dương (query, key) của MoCo được tính trên các GPU khác nhau (sample order bị xáo), BN tính độc lập theo từng GPU không còn bị “ăn gian”. Cụ thể: huấn luyện đa GPU, BN tính độc lập mỗi GPU; với key encoder f_k, xáo thứ tự mẫu trong minibatch trước khi chia GPU (và xáo lại sau khi encode); thứ tự mẫu cho query encoder f_q giữ nguyên. Nếu không làm vậy, model “cheat” pretext task (train accuracy pretext task tăng rất nhanh lên $> 99.9\%$ nhưng kNN validation accuracy giảm, Hình A.1 phụ lục).

3.5 Thiết lập huấn luyện (mục 4)

- Encoder: ResNet, lớp fully-connected cuối (sau global average pooling) có output 128-D, chuẩn hoá ℓ_2.
- Temperature $\tau = 0.07$ (theo Wu et al. 2018).
- Optimizer SGD, weight decay 0.0001, momentum 0.9.
- **ImageNet-1M (IN-1M)**: $\sim$ 1.28 triệu ảnh, batch 256 trên 8 GPU, learning rate khởi đầu 0.03, 200 epoch, LR $\times 0.1$ tại epoch 120 và 160; khoảng 53 giờ để huấn luyện ResNet-50.
- **Instagram-1B (IG-1B)**: $\sim$ 1 tỉ (940M) ảnh công khai, không cân bằng, batch 1024 trên 64 GPU, LR 0.12 giảm theo hàm mũ $\times 0.9$ mỗi 62.5k vòng lặp (64M ảnh), huấn luyện 1.25M vòng lặp ($\sim$ 1.4 epoch của IG-1B), $\sim$ 6 ngày cho ResNet-50.

4. Thí nghiệm và kết quả chính

- **So sánh 3 cơ chế trên linear classification (Hình 3, Bảng, K thay đổi)**: cả 3 đều lợi khi K tăng; memory bank kém hơn MoCo 2.6 điểm (58.0% so với 60.6% ở $K = 65536$); end-to-end ngang MoCo khi K nhỏ nhưng bị giới hạn $K \leq 1024$ (batch lớn nhất GPU 32GB Volta 8 chiếc chịu được).
- **Ablation momentum m ($K = 4096$ cố định)**: $m = 0$ (không momentum, copy trực tiếp) **thất bại hoàn toàn** (loss dao động, không hội tụ); $m = 0.9$: 55.2%; $m = 0.99$: 57.8%; $m = 0.999$ (mặc định): **59.0%**; $m = 0.9999$: 58.9%. Kết luận: momentum lớn (0.99-0.9999) là cần thiết.
- **So sánh SOTA trên linear evaluation (Bảng 1)**: MoCo ResNet-50 đạt **60.6%** top-1, tốt hơn các phương pháp cùng kích thước tham số ($\sim$ 24M: InstDisc 54.0%, LocalAgg 58.8%); MoCo ResNet-50 (4x, 375M tham số) đạt **68.6%**.
- **PASCAL VOC object detection (Bảng 2, Faster R-CNN)**: với backbone R50-C4, MoCo pretrain trên IN-1M đạt AP50 81.1 (so với supervised IN-1M 81.3, gần như ngang), AP 54.6 (+1.1 so supervised 53.5), AP75 59.9 (+1.1); MoCo pretrain trên IG-1B tốt hơn nữa: AP50 82.2 (+0.9), AP 57.2 (+3.7), AP75 63.7 (+4.9).
- **COCO detection/segmentation (Bảng 5, 2x schedule)**: MoCo IG-1B vượt supervised IN-1M trên mọi chỉ số ở cả backbone FPN và C4 (ví dụ C4: AP box 41.1 so với 40.0 supervised).
- **Tổng kết 7 task downstream** (VOC detection, COCO detection, COCO/LVIS instance segmentation, COCO keypoint detection, COCO dense pose, Cityscapes semantic/instance segmentation): MoCo **vượt** supervised ImageNet pretraining trên 7 task/benchmark; **ngang** trên Cityscapes instance segmentation; **thua** trên VOC semantic segmentation (ít nhất 0.8 điểm): đây là trường hợp tiêu cực paper tự nêu.

5. Phân tích / ablation

- **Ablation shuffling BN (Hình A.1, phụ lục)**: bỏ shuffling BN gây overfit pretext task rõ rệt (train accuracy vượt 99.9%, validation kNN accuracy giảm), xảy ra cho cả MoCo và biến thể end-to-end; memory bank không bị ảnh hưởng vì key của nó vốn đến từ minibatch khác trong quá khứ.
- **Normalization khi fine-tune downstream (mục 4.2)**: biểu diễn tự giám sát có phân phối (magnitude) khác biểu diễn supervised, nên hyperparameter tối ưu (vd. learning rate cho linear classifier: 30, weight decay 0) khác hẳn thiết lập supervised; khi fine-tune cho detection, cần BN được huấn luyện (và đồng bộ qua GPU) thay vì đóng băng.
- **Ablation backbone detector (Bảng 2)**: lợi thế MoCo so với supervised lớn hơn với backbone C4 so với dilated-C5; MoCo còn vượt rõ hơn ở các chỉ số COCO-style AP, AP75 (nghiêm ngặt hơn) so với AP50 (chỉ số VOC mặc định, dễ đạt).
- **Schedule huấn luyện dài hơn (Appendix A.8, Bảng A.1)**: ở schedule 6x ($\sim$ 72 epoch) so với 2x, khoảng cách MoCo so với supervised **tăng** (ví dụ +0.9 AP box ở 6x so với +0.5 ở 2x), cho thấy đặc trưng MoCo có lợi thế lớn hơn khi fine-tune lâu.

- **Ablation cơ chế trên downstream task (Bảng 3, Bảng 4)**: khi fine-tune cả biến thể end-to-end và memory bank (tự triển khai, cùng pretext task, cùng thiết lập fine-tune) trên VOC detection, cả hai vẫn **kém hơn MoCo trên mọi chỉ số** dù có thể vượt supervised ở vài chỉ số; chứng tỏ lợi ích không chỉ từ "có contrastive pretraining" mà từ chính cơ chế MoCo.
- **So sánh IN-1M so với IG-1B**: MoCo trên IG-1B luôn nhất quán tốt hơn IN-1M trên mọi downstream task thử nghiệm, nhưng mức cải thiện "đáng chú ý nhưng tương đối nhỏ" theo nhận xét của tác giả, gợi ý dữ liệu quy mô lớn chưa được khai thác hết với pretext task instance discrimination đơn giản.

6. Giới hạn

Do tác giả nêu (mục 5, Discussion):

- Cải thiện từ IN-1M lên IG-1B (tỉ ảnh) là nhất quán nhưng **tương đối nhỏ**, gợi ý dữ liệu quy mô lớn chưa được MoCo tận dụng hết; tác giả hy vọng pretext task tốt hơn (ví dụ masked autoencoding) sẽ cải thiện.
- MoCo thua supervised trên VOC semantic segmentation (ít nhất 0.8 điểm): một trường hợp tiêu cực được nêu rõ.
- Bài báo tập trung vào **cơ chế** contrastive learning, không khám phá các yếu tố trực giao khác (như pretext task tốt hơn) có thể cải thiện thêm; "MoCo v2" (bài tiếp theo của cùng nhóm, trích dẫn [8] trong paper) đạt 71.1% (so với 60.6%) chỉ bằng thay đổi augmentation và projection head, một cải tiến ngoài phạm vi bài này.

Do người ghi chú nhận thấy:

- Phạm vi đã đọc: toàn bộ phần chính (Mục 1-5) và hầu hết phụ lục A (A.1-A.9: chi tiết backbone detection, COCO keypoint, dense pose, LVIS, semantic segmentation, iNaturalist, fine-tuning ImageNet, COCO longer schedule, shuffling BN ablation).
- Chỉ xét ảnh tự nhiên (ImageNet, Instagram, VOC/COCO/Cityscapes); không có thí nghiệm văn bản hay OCR.
- Pretext task instance discrimination khá đơn giản (chỉ cần hai crop của cùng ảnh); không rõ MoCo hoạt động ra sao với pretext task phức tạp hơn gắn với cấu trúc dữ liệu khác (ví dụ trang tài liệu nhiều vùng văn bản).

7. Liên hệ với các bài khác

- **ml basics 17** (SimCLR): cùng chủ đề, khác cơ chế lấy negative (batch lớn so với queue + momentum); nên đọc nối tiếp để so sánh trực tiếp Bảng 6 của SimCLR với Bảng 1 ở đây (hai bài dùng đúng linear evaluation protocol nên số liệu so sánh được).
- **distillation content/22_2021_dino.md**: self-distillation dùng một "teacher" cập nhật bằng exponential moving average của student, cùng ý tưởng cập nhật trung bình động như momentum encoder của MoCo, áp dụng cho bài toán khác (self-distillation thay vì contrastive key).
- **llm content/30_2021_clip.md**: CLIP cũng dùng contrastive loss dạng softmax trong batch (giống công thức InfoNCE ở đây) nhưng giữa ảnh và văn bản, không cần dictionary/queue vì CLIP dùng batch rất lớn (gần giống hướng SimCLR hơn MoCo).
- **ml basics 07** (Deep Residual Learning): ResNet là encoder nền cho cả f_q và f_k.

8. Gợi ý áp dụng

MoCo dùng hàng đợi và momentum encoder để duy trì tập mẫu đối chiếu lớn. Khi thử trên dữ liệu mới, cần kiểm tra cả chất lượng biểu diễn downstream lẫn ảnh hưởng của augmentation và kích thước hàng đợi.

9. Thí nghiệm gợi ý

1. **Mô phỏng MoCo với queue nhỏ.** Lấy ResNet-18 hoặc CNN nhỏ, cài cơ chế queue (ví dụ $K = 4096$) và momentum update ($m = 0.999$), huấn luyện trên CIFAR-10 với batch vật lý nhỏ (32-64). Đo linear evaluation accuracy, so với cùng cấu hình chạy kiểu end-to-end (batch = K, không có queue). Kỳ vọng: MoCo đạt accuracy gần cấu hình end-to-end dù batch vật lý nhỏ hơn nhiều; diễn giải: xác nhận queue thực sự "thay thế" được batch lớn trên hardware hạn chế.
2. **Ablation hệ số momentum m ở quy mô nhỏ.** Thử $m \in \{0, 0.9, 0.99, 0.999\}$ trên cùng toy set. Kỳ vọng: giống paper, $m = 0$ không hội tụ hoặc hội tụ rất kém, m cao quanh 0.99-0.999 tốt nhất. Diễn giải: xác nhận hiện tượng không phụ thuộc vào độ lớn dữ liệu/mô hình, giúp chọn m an toàn khi áp dụng ở quy mô lớn hơn.
3. **Kiểm tra shuffling BN có cần thiết với Norm hiện tại của encoder.** Nếu encoder dùng BatchNorm ở bất kỳ lớp nào, lặp lại thí nghiệm A.9 của paper (so sánh train/validation accuracy có và không shuffle) ở quy mô nhỏ. Kỳ vọng: nếu encoder không dùng BatchNorm, hiện tượng "cheat" không xuất hiện; diễn giải: xác định có cần bận tâm kỹ thuật này hay không tuỳ kiến trúc cụ thể.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
InfoNCE loss	$\mathcal{L}_q = -\log \frac{\exp(q \cdot k_+/\tau)}{\sum_{i=0}^{K} \exp(q \cdot k_i/\tau)}$
Momentum update key encoder	$\theta_k \leftarrow m\theta_k + (1-m)\theta_q$, $m \in [0, 1)$
Temperature	$\tau = 0.07$
Momentum mặc định	$m = 0.999$
Dictionary	Queue kích thước K (độc lập batch size); mặc định $K = 65536$ cho so sánh SOTA
Encoder output	128-D, chuẩn hoá ℓ_2
Kết quả chính	linear eval ResNet-50: 60.6% (IN-1M); ResNet-50(4x): 68.6%
So sánh memory bank	MoCo 60.6% so với memory bank 58.0% ($K = 65536$)

11. Kết luận ngắn

MoCo xây dictionary lớn và nhất quán cho contrastive learning bằng queue (tách khỏi batch size) cộng momentum encoder ($m = 0.999$), giải quyết đúng điểm yếu "cần batch lớn" của các cách tiếp cận end-to-end như SimCLR. Với pretext task instance discrimination đơn giản, MoCo pretrain trên ImageNet hoặc Instagram-1B vượt supervised ImageNet pretraining trên 7 trong số các task detection/segmentation thử nghiệm, cho thấy khoảng cách giữa tự giám sát và giám sát trong computer vision có thể gần như đóng lại ở các task downstream này.

Nguồn gốc: https://arxiv.org/abs/1911.05722

Bài 19

On Calibration of Modern Neural Networks (2017)

Tác giả: Chuan Guo, Geoff Pleiss, Yu Sun, Kilian Q. Weinberger (Cornell University).
Venue / arXiv: Proceedings of the 34th International Conference on Machine Learning (ICML 2017), PMLR 70; arXiv:1706.04599 (v2, 03/08/2017).
Nguồn: https://arxiv.org/abs/1706.04599
Chủ đề: Hiện tượng miscalibration (lệch giữa độ tin cậy dự đoán và xác suất đúng thật) ở mạng neural hiện đại; ECE/MCE; temperature scaling và các phương pháp hậu xử lý calibration khác.
Vị trí trong lộ trình: Đọc trước các bài 35–37 về dự đoán và ước lượng độ bất định.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Trong các hệ thống ra quyết định dùng mạng neural (xe tự lái phát hiện người đi bộ, chẩn đoán y tế), mô hình không chỉ cần chính xác mà còn cần **biết khi nào mình có thể sai**. Paper định nghĩa: một mô hình phân loại $h(X) = (\hat{Y}, \hat{P})$ (với $\hat{Y}$ là nhãn dự đoán, $\hat{P}$ là độ tin cậy/confidence tương ứng) được gọi là **calibrated** nếu độ tin cậy đó phản ánh đúng xác suất thật của việc dự đoán đúng:

$$P(\hat{Y} = Y \mid \hat{P} = p) = p, \quad \forall p \in [0, 1]$$

Năm 2005, Niculescu-Mizil & Caruana cho thấy mạng neural (thời đó) thường calibrate tốt trên bài toán phân loại nhị phân. Bài báo này phát hiện ngược lại: mạng neural **hiện đại** (2015-2017, sâu hơn, rộng hơn) lại **kém calibrate hơn** mạng cũ, dù chính xác hơn nhiều. Hình 1 minh hoạ trực quan: LeNet 5 tầng so với ResNet 110 tầng trên CIFAR-100, biểu đồ confidence histogram và reliability diagram cho thấy ResNet tự tin thái quá (average confidence cao hơn nhiều so với accuracy thật), còn LeNet gần như calibrate hoàn hảo dù accuracy thấp hơn.

2. Ý tưởng chính

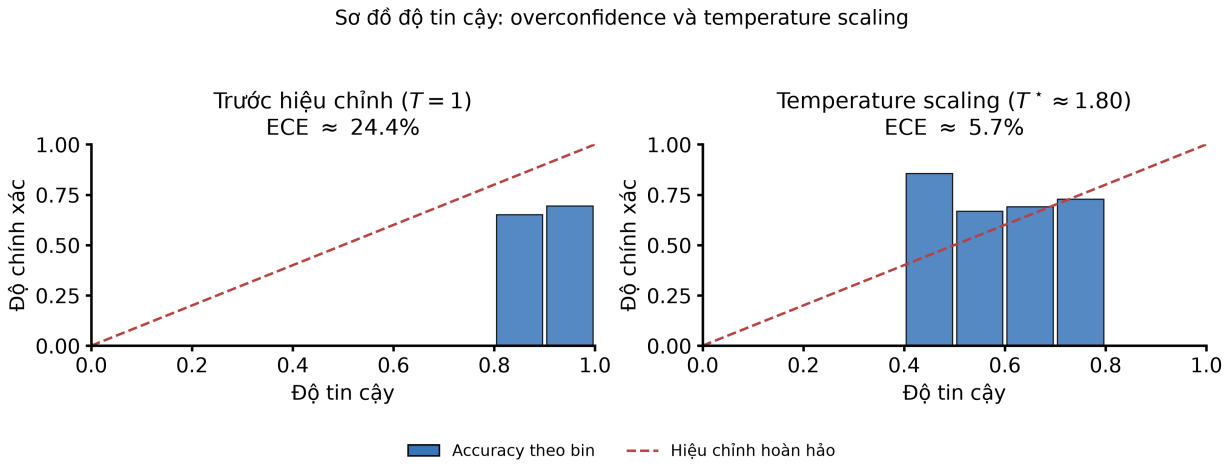
Hai đóng góp chính:

1. **Chẩn đoán**: xác định các yếu tố kiến trúc/huấn luyện (độ sâu, độ rộng, Batch Normalization, weight decay) có tương quan với mức độ miscalibration, qua thực nghiệm có kiểm soát trên ResNet/CIFAR-100 (paper nói rõ "we cannot claim causality", chỉ là tương quan quan sát được).
2. **Giải pháp**: so sánh nhiều phương pháp hậu xử lý (post-processing) để calibrate lại xác suất đầu ra **sau khi** huấn luyện xong, không cần train lại. Kết quả bất ngờ: **temperature scaling**, một biến thể một-tham-số của Platt scaling, đơn giản nhất lại **hiệu quả nhất** trên hầu hết bộ dữ liệu vision, vượt cả các phương pháp tổng quát hơn (vector scaling, matrix scaling, Bayesian Binning into Quantiles).

3. Phương pháp chi tiết

3.1 Đo lường calibration (Mục 2)

- **Reliability diagram**: chia dự đoán vào M bin theo confidence (bin B_m ứng với khoảng $\left(\frac{m-1}{M}, \frac{m}{M}\right]$), tính $\text{acc}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \mathbb{1}(\hat{y}_i = y_i)$ và $\text{conf}(B_m) = \frac{1}{|B_m|} \sum_{i \in B_m} \hat{p}_i$; vẽ acc theo conf, mô hình calibrate hoàn hảo cho đường chéo.
- **Expected Calibration Error (ECE)** (Naeini et al. 2015):



Reliability diagram trước/sau temperature scaling: mô hình overconfident có ECE cao, chia logit cho $T^\star > 1$ kéo confidence sát accuracy hơn (minh hoạ, tự tính)

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|$$

- **Maximum Calibration Error (MCE)**: $\mathrm{MCE} = \max_m \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|$, dùng cho ứng dụng rủi ro cao cần chặn sai số worst-case.
- **NLL** (negative log-likelihood, tương đương cross-entropy loss): $L = -\sum_i \log \hat{\pi}(y_i|x_i)$, cực tiểu về kỳ vọng khi và chỉ khi $\hat{\pi}$ khớp đúng phân phối thật; dùng làm thước đo calibration gián tiếp.

Tự tính (ví dụ minh hoạ ECE với 2 bin): giả sử $n = 10$ mẫu, bin B_1 (confidence 0.5-0.7) có 4 mẫu với accuracy thật 0.5 và confidence trung bình 0.6; bin B_2 (confidence 0.7-1.0) có 6 mẫu với accuracy thật 0.8 và confidence trung bình 0.9. $\mathrm{ECE} = \frac{4}{10}|0.5-0.6| + \frac{6}{10}|0.8-0.9| = 0.4\times 0.1 + 0.6\times 0.1 = 0.04 + 0.06 = 0.1$, tức ECE = 10%, nghĩa là trung bình mô hình lệch 10 điểm phần trăm giữa confidence và accuracy thật.

3.2 Các phương pháp calibration (Mục 4)

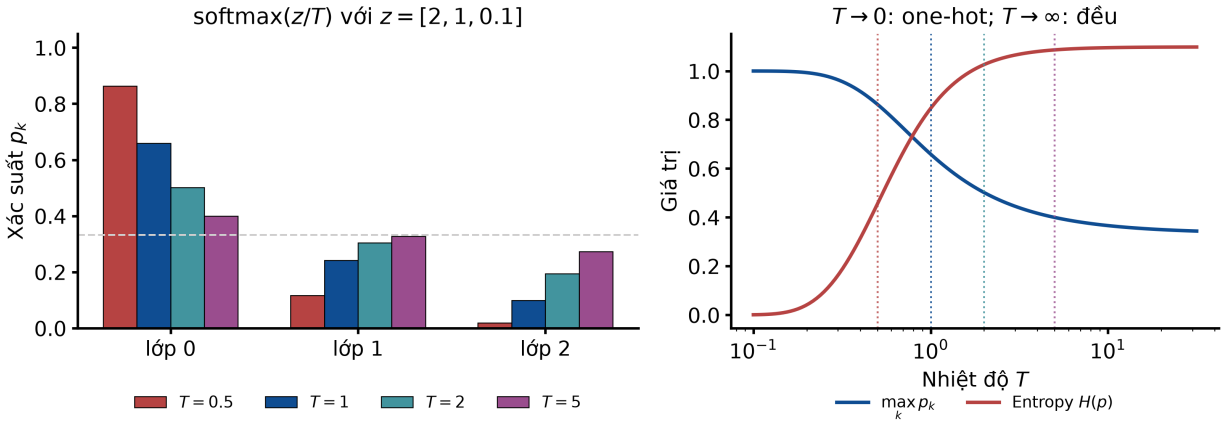
Tất cả là bước hậu xử lý (post-hoc), cần một validation set riêng, không cần sửa trọng số mạng:

- **Histogram binning**: chia confidence vào M bin cố định, gán mỗi bin một giá trị calibrated θ_m (tỉ lệ mẫu dương thật trong bin) tối thiểu hoá bình phương lỗi theo công thức (7).
- **Isotonic regression**: tổng quát hoá histogram binning, học hàm f bậc thang (piecewise constant) đơn điệu không giảm, tối ưu đồng thời biên bin và giá trị bin.
- **Bayesian Binning into Quantiles (BBQ)** (Naeini et al. 2015): lấy trung bình Bayes trên toàn bộ không gian các cách chia bin (dùng Beta prior cho θ_m), phức tạp hơn nhưng không nhất thiết tốt hơn.
- **Platt scaling**: dùng logit z_i làm đặc trưng cho hồi quy logistic $\hat{q}_i = \sigma(az_i + b)$, học a, b bằng NLL trên validation set.
- **Mở rộng cho multi-class**: *matrix scaling* $\hat{q}_i = \max_k \sigma_{SM}(Wz_i + b)^{(k)}$ (tham số tăng theo $O(K^2)$ với K số lớp); *vector scaling* (ràng buộc W là ma trận đường chéo, tham số $O(K)$); *temperature scaling*, trường hợp đặc biệt đơn giản nhất: $\hat{q}_i = \max_k \sigma_{SM}(z_i/T)^{(k)}$ với một scalar $T > 0$ duy nhất cho mọi lớp.

3.3 Temperature scaling, chi tiết

- T được học bằng cách tối thiểu hoá NLL trên validation set (bài toán lồi một chiều, hội tụ nhanh bằng conjugate gradient, khoảng 10 vòng lặp).

- Vì T chỉ "làm mềm" (soften, khi $T > 1$) hay "làm sắc" (khi $T < 1$) phân phối softmax mà **không đổi thứ tự** các logit, nó **không đổi argmax**, tức **không đổi accuracy** của mô hình, chỉ đổi độ tin cậy.
- Khi $T \to \infty$, $\hat{q}_i \to 1/K$ (bất định tối đa); khi $T \to 0$, $\hat{q}_i \to$ point mass; khi $T = 1$, giữ nguyên xác suất gốc.
- Phụ lục S2 chứng minh (Claim 1): temperature scaling là nghiệm duy nhất của bài toán cực đại hoá entropy $\max_q -\sum_{i,k} q(z_i)^{(k)} \log q(z_i)^{(k)}$ với ràng buộc $\sum_k q(z_i)^{(k)} = 1$ và ràng buộc cân bằng logit lớp đúng trung bình; nói cách khác temperature scaling tương đương việc chọn phân phối entropy cao nhất thoả một ràng buộc tuyến tính trên logit.



softmax(z/T) với $z = [2, 1, 0.1]$: T nhỏ làm sắc (gần one-hot), T lớn làm mềm về phân phối đều; kèm max p_k và entropy theo T (minh hoạ, tự tính)

3.4 Thiết lập thực nghiệm (Mục 5)

- **Vision** (6 bộ dữ liệu): Caltech-UCSD Birds (200 lớp, 5994/2897/2897 ảnh train/val/test, fine-tune từ ImageNet pretrain), Stanford Cars (196 lớp, 8041/4020/4020), ImageNet 2012 (1000 lớp, 1.3M/25k/25k), CIFAR-10/CIFAR-100 (45000/5000/10000), SVHN (598388/6000/26032).
- **Mô hình vision**: ResNet, ResNet với stochastic depth (SD), Wide ResNet, DenseNet, dùng đúng pipeline huấn luyện/hyperparameter của từng paper gốc.
- **NLP** (4 bộ): 20 Newsgroups (20 lớp, 9034/2259/7528 văn bản), Reuters (8 lớp, 4388/1097/2189), Stanford Sentiment Treebank (SST) nhị phân (6920/872/1821) và fine-grained 5 lớp (544/1101/2210).
- **Mô hình NLP**: Deep Averaging Networks (DAN, 3 tầng feed-forward + BatchNorm) cho 20 News/Reuters; TreeLSTM cho SST.
- ECE/MCE đều tính với $M = 15$ bin.

4. Thí nghiệm và kết quả chính

Bảng 1 (ECE %, $M = 15$) trên 19 cặp (dataset, model), trích một số dòng tiêu biểu:

Dataset / Model	Uncalibrated	Hist. Binning	Isotonic	BBQ	Temp. Scaling	Vector Scaling	Matrix Scaling
Birds / ResNet 50	9.19%	4.34%	5.22%	4.12%	**1.85%**	3.0%	21.13%
Cars / ResNet 50	4.3%	**1.74%**	4.29%	1.84%	2.35%	2.37%	10.5%
CIFAR-10 / ResNet 110	4.6%	0.58%	0.81%	**0.54%**	0.83%	0.88%	1.0%
CIFAR-100 / ResNet 110	16.53%	2.66%	4.99%	5.46%	**1.26%**	1.32%	25.49%
ImageNet / ResNet 152	5.48%	4.36%	4.77%	3.56%	**1.86%**	2.23%	-

Dataset / Model	Uncalibrated	Hist. Binning	Isotonic	BBQ	Temp. Scaling	Vector Scaling	Matrix Scaling
SVHN / ResNet 152 (SD)	0.44%	**0.14%**	0.28%	0.22%	0.17%	0.27%	0.17%
Reuters / DAN 3	**0.85%**	1.75%	1.15%	0.97%	0.91%	0.66%	1.58%

(In đậm là giá trị ECE thấp nhất của mỗi dòng.)

- Hầu hết dataset/model miscalibrate ở mức ECE 4-10% trước khi xử lý; hai ngoại lệ là SVHN và Reuters (ECE dưới 1% ngay từ đầu, vì error rate thấp: 1.98% và 2.97%).
- **Phát hiện chính**: temperature scaling vượt trội các phương pháp khác trên phần lớn bộ dữ liệu vision, và **ngang ngửa** (không hẳn vượt trội) trên NLP.
- **Phát hiện bất ngờ thứ hai**: temperature scaling còn tốt hơn cả vector/matrix scaling, hai phương pháp tổng quát hơn nó (có nhiều tham số hơn); tác giả giải thích "network miscalibration is intrinsically low dimensional" và quan sát rằng nghiệm vector scaling học được thường có các phần tử gần như hằng số, gần giống một phép biến đổi scalar.
- Matrix scaling thất bại rõ rệt trên các bài toán nhiều lớp (Birds 200 lớp: ECE 21.13%; CIFAR-100 100 lớp: 25.49%; không hội tụ trên ImageNet 1000 lớp, đánh dấu "-"), vì số tham số tăng bậc hai theo số lớp K nên overfit validation set nhỏ.
- Reuters là dataset duy nhất temperature scaling **không cải thiện** (0.85% $\rightarrow$ 0.91%, hơi tệ hơn): tác giả cho rằng vì ECE gốc đã rất thấp (<1%), không còn nhiều chỗ để cải thiện, và có thể do cách chia tập dữ liệu hoặc binning scheme ảnh hưởng.
- Reliability diagram (Hình 4) trên ResNet-110 (SD)/CIFAR-100: ECE giảm từ 12.67% (uncalibrated) xuống 0.96% (temperature scaling), 2.46% (histogram binning), 4.16% (isotonic regression).
- Thời gian tính toán: temperature scaling nhanh nhất (bài toán lồi 1 chiều, vài vòng lặp); vector/matrix scaling chậm hơn ít nhất 2 bậc độ lớn (do $K = 100$ ở CIFAR-100); histogram binning và isotonic chậm hơn 1 bậc; BBQ chậm hơn khoảng 3 bậc độ lớn so với temperature scaling.

5. Phân tích / ablation

- **Ảnh hưởng của depth/width/BatchNorm/weight decay lên ECE (Hình 2, ResNet trên CIFAR-100)**: tăng độ sâu (giữ 64 filter/tầng) hoặc độ rộng (giữ 14 tầng) đều làm error giảm nhưng ECE **tăng** đáng kể; mạng có Batch Normalization bị miscalibrate nhiều hơn mạng không có BN (dù accuracy nhích lên một chút) trên một ConvNet 6 tầng, kết quả này được báo là ổn định bất kể learning rate; weight decay thấp (xu hướng hiện đại từ khi có BN) tương quan với ECE cao hơn, và paper quan sát calibration **tiếp tục cải thiện** khi tăng weight decay dù đã vượt qua điểm tối ưu của accuracy (nghĩa là điểm tối ưu cho accuracy và điểm tối ưu cho calibration là khác nhau).
- **Hiện tượng "NLL overfitting" (Hình 3)**: trên ResNet 110 + stochastic depth, CIFAR-100, sau khi learning rate giảm ở epoch 250, test error và test NLL đều giảm ngay, nhưng sau đó NLL **overfit** (tăng trở lại) trong khi test error **tiếp tục giảm** từ 29% xuống 27%. Diễn giải của tác giả: mô hình học được accuracy tốt hơn bằng cách "đánh đổi" chất lượng xác suất, tức overfitting ở loss xác suất không đồng nghĩa overfitting ở độ chính xác.
- **Vector scaling "gần giống" temperature scaling**: đúng ở phần lớn trường hợp (ví dụ CIFAR-100/LeNet5: 2.02% so với 2.09%) nhưng **không hoàn toàn đúng mọi nơi**: Birds/ResNet50 có khoảng cách rõ rệt (temperature 1.85% so với vector 3.0%), cho thấy tuyên bố "vector scaling recovers essentially the same solution" (Mục 5, gần cuối) nên hiểu là xu hướng chung, không phải quy luật tuyệt đối.

6. Giới hạn

Do tác giả nêu:

- Chỉ là quan sát tương quan, không chứng minh nhân quả: "Though we cannot claim causality..." (Mục 3) khi liên hệ model capacity/BN/weight decay với miscalibration.
- Không giải thích được **tại sao** các xu hướng kiến trúc này ảnh hưởng calibration trong khi cải thiện accuracy: "It remains future work to understand why these trends affect calibration while improving accuracy" (Kết luận).
- Matrix scaling không scale được lên bài toán nhiều lớp (overfit, không hội tụ trên ImageNet).
- Temperature scaling không cải thiện được Reuters (trường hợp ngoại lệ được chính tác giả nêu).
- MCE "rất nhạy với binning scheme và kém phù hợp cho test set nhỏ" (chú thích Bảng S1).

Do người ghi chú phát hiện (mâu thuẫn nội tại cần lưu ý khi trích dẫn số liệu paper này):

- Câu văn ở Mục 5 "Temperature scaling outperforms all other methods on the vision tasks" **không khớp hoàn toàn** với chính Bảng 1: trên Cars/ResNet50, Histogram Binning (1.74%) và BBQ (1.84%) đều thấp hơn Temperature Scaling (2.35%); trên CIFAR-10/ResNet110, Histogram Binning (0.58%) và BBQ (0.54%) đều thấp hơn Temperature Scaling (0.83%); trên SVHN/ResNet152(SD), Histogram Binning (0.14%) thấp hơn Temperature Scaling (0.17%). Vậy phát biểu này đúng với đa số nhưng có ít nhất 3/11 bộ (dataset, model) vision trong Bảng 1 là ngoại lệ; người đọc nên hiểu phát biểu của tác giả là "vượt trội trên phần lớn", không phải "trên mọi" bộ dữ liệu vision.
- Phạm vi đã đọc: **toàn bộ** bài chính (Mục 1-6) và **toàn bộ** phụ lục S1-S4 (chứng minh Claim 1, Bảng S1 MCE, Bảng S2 test error, Bảng S3 NLL, các reliability diagram bổ sung cho CIFAR-10 và SST). Không có phần nào bị bỏ qua.
- Bài chỉ xét classification (ảnh và văn bản); không có thí nghiệm nào về calibration cho generative model, sequence-to-sequence, hay bài toán detection/regression.

7. Liên hệ với các bài khác

- **ml basics 01_2016_rethinking_generalization.md**: bài đó cho thấy mạng neural có thể fit hoàn toàn nhãn ngẫu nhiên (memorization), liên hệ trực tiếp tới hiện tượng "NLL overfitting" ở Hình 3 của bài này: cùng một dạng quan sát rằng mạng dư capacity có thể tối ưu loss huấn luyện/xác suất vượt xa những gì cần cho accuracy, dẫn tới các hệ quả khác nhau (ở đây là overconfidence).
- **ml basics 07_2015_resnet.md**: ResNet (và các biến thể Wide ResNet, ResNet+SD) là backbone chính được dùng trong toàn bộ thí nghiệm vision của bài calibration; nên đọc 07 trước để hiểu kiến trúc đang được đo ECE.
- **distillation content/02_2015_distilling_knowledge_hinton.md**: công thức temperature scaling ở đây ($\hat{q}_i = \max_k \sigma_{SM}(z_i/T)^{(k)}$) có dạng toán học **giống hệt** "temperature" dùng trong knowledge distillation của Hinton et al., nhưng mục đích khác nhau: ở KD, $T > 1$ dùng để làm mềm phân phối của teacher nhằm lộ thông tin "dark knowledge" cho student; ở đây, T được **học** trên validation set để sửa lại độ tin cậy của chính mô hình cho khớp với accuracy thật. Người mới học dễ nhầm hai mục đích này.

8. Gợi ý áp dụng

Temperature scaling là một cách hiệu chỉnh xác suất sau huấn luyện. Cần dùng dữ liệu hiệu chỉnh tách biệt với tập kiểm tra và xem reliability diagram cùng các thước đo, vì một giá trị ECE phụ thuộc cách chia bin.

9. Thí nghiệm gợi ý

1. **Đo ECE/reliability diagram cho một classifier nhỏ tự train.** Dùng ResNet-18/CNN nhỏ trên CIFAR-10 hoặc CIFAR-100, tính ECE theo công thức (3) với $M = 15$ bin trước và sau khi áp temperature scaling (tìm T bằng grid-search hoặc gradient descent trên NLL của validation set). Kỳ vọng: ECE

giảm mạnh sau calibrate, giống xu hướng Bảng 1; diễn giải: xác nhận hiểu đúng cách tính ECE và cách tối ưu T.

2. **Tái tạo hiện tượng "NLL overfitting" (Hình 3) ở quy mô nhỏ.** Huấn luyện một mạng vừa đủ dư capacity trên CIFAR-100 trong nhiều epoch, theo dõi test error và test NLL theo từng epoch sau khi giảm learning rate. Kỳ vọng: thấy test error tiếp tục giảm nhẹ trong khi NLL tăng trở lại ở giai đoạn cuối. Diễn giải: nếu lặp lại được, xác nhận cơ chế "overconfidence đi kèm accuracy tốt hơn" không phải hiện tượng riêng của ResNet-110 mà khá phổ biến.
3. **Thử temperature scaling theo token.** Trích logits từ một mô hình nhận dạng chuỗi, hiệu chỉnh nhiệt độ trên một tập riêng và đo ECE trên tập kiểm tra. So sánh kết quả theo nhóm ký tự để khảo sát giới hạn của một nhiệt độ chung.
4. **So sánh ECE trước/sau khi áp dụng một kỹ thuật nén (ví dụ quantization int8 đơn giản) trên classifier nhỏ.** Đo ECE của mô hình gốc so với mô hình đã nén. Kỳ vọng: không rõ trước (paper không trả lời câu hỏi này); diễn giải: đây chính là thực nghiệm khảo sát câu hỏi mà bài báo không trả lời, cho dữ liệu thực tế thay vì ngoại suy.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Định nghĩa calibration hoàn hảo	$P(\hat{Y} = Y \mid \hat{P} = p) = p,\ \forall p$
ECE	$\sum_{m=1}^{M} \frac{\lvert B_m \rvert}{n} \lvert \text{acc}(B_m) - \text{conf}(B_m) \rvert$
MCE	$\max_m \lvert \text{acc}(B_m) - \text{conf}(B_m) \rvert$
NLL	$L = -\sum_i \log \hat{\pi}(y_i \mid x_i)$
Temperature scaling	$\hat{q}_i = \max_k \sigma_{SM}(z_i/T)^{(k)}$, T học bằng NLL trên validation set
Platt scaling (nhị phân)	$\hat{q}_i = \sigma(az_i + b)$
Matrix / vector scaling	$\hat{q}_i = \max_k \sigma_{SM}(Wz_i + b)^{(k)}$; vector: W chéo
Kết quả tiêu biểu	CIFAR-100/ResNet110: ECE 16.53% $\rightarrow$ 1.26% (temp scaling); ImageNet/ResNet152: 5.48% $\rightarrow$ 1.86%
M dùng trong thí nghiệm	15 bin

11. Kết luận ngắn

Mạng neural hiện đại (sâu, rộng, ít weight decay, có BatchNorm) chính xác hơn nhưng lại kém calibrate hơn mạng cũ; nguyên nhân tương quan với model capacity và thiếu regularization, thể hiện qua hiện tượng NLL tiếp tục overfit dù accuracy vẫn cải thiện. Trong số nhiều phương pháp hậu xử lý để sửa lại độ tin cậy, temperature scaling (một tham số scalar duy nhất) đơn giản nhất lại hiệu quả nhất trên phần lớn (không phải tất cả) bộ dữ liệu vision thử nghiệm, và rẻ về compute, khiến nó trở thành lựa chọn thực dụng mặc định cho calibration hậu huấn luyện.

Nguồn gốc: https://arxiv.org/abs/1706.04599

Bài 20

Shortcut Learning in Deep Neural Networks (2020)

Tác giả: Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, Felix A. Wichmann (University of Tübingen, IMPRS-IS, University of Toronto / Vector Institute).
Venue / arXiv: Nature Machine Intelligence (bản arXiv là preprint của bài đã công bố, DOI 10.1038/s42256-020-00257-z, paper tự ghi rõ điều này ở trang 1); arXiv:2004.07780 (v5, 21/11/2023).
Nguồn: https://arxiv.org/abs/2004.07780
Chủ đề: Shortcut learning: khung khái niệm thống nhất nhiều "triệu chứng" thất bại của deep learning (dataset bias, texture bias, adversarial examples, reward hacking, bias thuật toán) dưới một nguyên nhân chung; taxonomy các loại decision rule; inductive bias; khuyến nghị kiểm tra o.o.d.
Vị trí trong lộ trình: Đọc cùng bài 21 về leakage và bài 22 về underspecification để xem xét giới hạn đánh giá mô hình.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Deep learning đạt hiệu năng "siêu người" trên nhiều benchmark (object recognition, Go, Poker) nhưng đồng thời thất bại theo những cách rất "không-con-người": thay đổi vô hình trong ảnh đổi hẳn dự đoán; mô hình caption ảnh có thể tạo câu mô tả hợp lý **mà không hề nhìn vào ảnh**; mô hình nhận diện khuôn mặt có tỉ lệ lỗi cao hơn với nhóm thiểu số; mô hình tuyển dụng thiên vị nam giới dựa trên CV. Hình 1 của bài báo liệt kê 4 ví dụ cụ thể làm motif xuyên suốt bài:

Tác vụ	Vấn đề	Shortcut bị dùng
Caption ảnh	Mô tả sườn đồi xanh là "đàn cừu đang gặm cỏ"	Dùng background để suy ra vật thể chính
Nhận diện vật thể	"Ảo giác" ra ấm trà (teapot) khi thấy một số pattern	Dùng đặc trưng con người không nhận ra được
Phát hiện viêm phổi từ X-ray	Thất bại trên ảnh từ bệnh viện mới	Nhìn token kim loại đặc trưng của bệnh viện, không nhìn vào phổi
Trả lời câu hỏi	Đổi câu trả lời khi thêm câu không liên quan	Chỉ nhìn câu cuối, bỏ qua ngữ cảnh

Ví dụ QA cụ thể (trang 1): đoạn văn gốc viết "Peython Manning..." (nguyên văn xuất hiện trong bài báo, có khả năng là lỗi chính tả của tác giả gốc ví dụ "Peyton Manning", bài báo này trích lại nguyên văn từ ví dụ gốc), câu hỏi "ai 38 tuổi ở Super Bowl XXXIII", dự đoán gốc đúng "John Elway"; sau khi chèn thêm câu nhiễu "Quarterback Jeff Dean had a jersey number 37 in Champ Bowl XXXIV" (một câu bịa, không liên quan), mô hình đổi câu trả lời thành "Jeff Dean": minh chứng mô hình chỉ bắt chữ bề mặt (từ "quarterback", số tuổi 38 gần giống) chứ không thực sự "hiểu" đoạn văn.

Thuật ngữ "shortcut learning" được bài báo đề xuất như một khung **thống nhất** nhiều hiện tượng từng được gọi bằng tên khác nhau: covariate shift, anti-causal learning, tập dữ liệu bias, "the tank legend" (giai thoại về việc một mạng phân biệt xe tăng Nga/Mỹ hoá ra chỉ phân biệt trời nắng/trời mây trong ảnh), Clever Hans effect (con ngựa Clever Hans "tính toán" được là nhờ đọc phản ứng cơ thể người hỏi, không phải biết làm toán).

2. Ý tưởng chính

Luận điểm trung tâm: rất nhiều thất bại tưởng như độc lập của deep learning (texture bias, adversarial examples, domain transfer kém, reward hacking trong RL, bias thuật toán) đều là **biểu hiện khác nhau của cùng một vấn đề gốc**: mô hình học một **decision rule** đạt hiệu năng tốt trên dữ liệu benchmark (train + test i.i.d.) nhưng **không phải** decision rule mà người thiết kế bài toán kỳ vọng (intended solution), nên thất bại khi gặp dữ liệu hơi khác (out-of-distribution, o.o.d.).

Hai ví dụ "sinh học" làm tương tự trực quan (Mục 2):

- **Chuột trong mê cung màu**: tưởng chuột học phân biệt màu bằng mắt, hoá ra chúng đánh hơi mùi sơn trên tường, không dùng thị giác chút nào.
- **Học sinh "học vẹt" (surface learning)**: Bob học thuộc lòng một chương sách và làm tốt bài thi trắc nghiệm hơn Alice (người thực sự hiểu bài), vì bài thi trắc nghiệm vô tình cho phép "shortcut" là ghi nhớ bề mặt thắng được hiểu sâu.

Thông điệp: shortcut learning không phải lỗi riêng của deep learning, mà là đặc điểm chung của **mọi hệ thống học** (sinh học và máy), bộc lộ khi có khoảng cách giữa cái môi trường huấn luyện và kiểm tra đo được và cái năng lực thật sự người ta muốn đo.

3. Phương pháp chi tiết

3.1 Taxonomy 4 tầng của decision rule (Mục 3, Hình 3)

Xuất phát từ bài toán toy "star vs moon" (Hình 2): một mạng fully-connected 3 tầng học phân loại sao/trăng; trong dữ liệu train, sao luôn ở góc trên-phải hoặc dưới-trái, trăng luôn ở góc trên-trái hoặc dưới-phải của ảnh. Mạng đạt accuracy cao trên test set cùng phân phối (i.i.d.) nhưng tụt xuống **ngẫu nhiên (chance)** khi test trên ảnh có vị trí object được xáo trộn (o.o.d.): hoá ra mạng học quy tắc "vị trí" chứ không học "hình dạng".

Từ đó, taxonomy 4 tầng (tập con lồng nhau, từ rộng tới hẹp):

1. **Tất cả decision rule khả dĩ, kể cả non-solution**: ví dụ "dự đoán sao nếu thấy pixel trắng": không giải được cả training set vì cả hai lớp đều có pixel trắng.
2. **Training solution, kể cả overfitting solution**: giải được training set nhưng không tổng quát hoá nổi tới test set i.i.d. (ghi nhớ thuần, "overfitting feature").
3. **i.i.d. test solution, kể cả shortcut**: giải được cả train và test i.i.d.; đây là tầng **không thể phân biệt được** intended solution với shortcut nếu chỉ dùng benchmark i.i.d. thông thường: ví dụ toy star/moon có ít nhất 3 cách giải đều đạt accuracy cao trên i.i.d.: theo hình dạng, theo số pixel trắng (trăng nhỏ hơn sao), hoặc theo vị trí.
4. **Intended solution**: decision rule vừa giải tốt i.i.d. vừa tổng quát hoá đúng ý định trên o.o.d. test (ví dụ phân loại theo hình dạng, vẫn đúng dù object đổi vị trí/kích thước).

Paper nhấn mạnh: "which decision rule is the intended solution is clearly in the eye of the beholder": việc xác định đâu là "intended" vốn mang tính chủ quan của người thiết kế bài toán, không phải một khái niệm toán học tuyệt đối.

3.2 Hai nguồn gốc của shortcut (Mục 4)

- **Shortcut opportunity trong dữ liệu** (Mục 4.1): quan hệ tương quan tự nhiên giữa object và ngữ cảnh (bò thường đi với cỏ, không phải nước) tạo cơ hội để mô hình "ăn gian" bằng ngữ cảnh thay vì vật thể; shortcut opportunity **không biến mất** khi tăng quy mô dữ liệu lên nhiều bậc độ lớn, vì bias hệ thống vẫn tồn tại ngay cả trong "Big Data".
- **Feature combination từ discriminative learning** (Mục 4.2): mô hình discriminative chỉ cần tìm **bất kỳ** đặc trưng đủ để phân biệt lớp trên dữ liệu đã cho, không có khái niệm thế nào là một object "thực"

trông ra sao; ví dụ nổi tiếng: DNN chuẩn phân loại ảnh mèo-với-texture-voi thành "voi" vì chỉ dựa texture, bỏ qua hình dạng tổng thể.

3.3 Bốn thành phần của inductive bias (Box II)

Inductive bias (toàn bộ giả định ảnh hưởng solution nào "dễ học") được phân rã thành 4 thành phần:

Thành phần	Ý nghĩa	Ví dụ trong paper
Structure (kiến trúc)	Convolution khiến việc dùng vị trí tuyệt đối khó hơn (so với fully-connected)	Toy star/moon: FC network học shortcut vị trí; CNN bị thiết kế ngăn điều này (footnote 3)
Experience (dữ liệu huấn luyện)	Shortcut opportunity hiện diện trong hầu hết dữ liệu thật, hiếm khi biến mất khi thêm dữ liệu	Sửa đổi dữ liệu để chặn shortcut cụ thể từng giảm được texture bias, giảm adversarial vulnerability
Goal (loss function)	Cross-entropy khuyến khích dừng học ngay khi tìm được một predictor đủ dùng	Cần regularization/loss sửa đổi để buộc mô hình dùng hết thông tin sẵn có
Learning (optimization)	SGD có xu hướng học hàm đơn giản trước; learning rate lớn → học pattern đơn giản chung, learning rate nhỏ → học pattern phức tạp/ghi nhớ	Tương tác kiến trúc-tối ưu còn ít được hiểu, chỉ có kết luận chắc cho trường hợp đơn giản

3.4 Ba bước hành động chẩn đoán (Mục 6)

1. **Diễn giải kết quả cẩn trọng**: phân biệt "dataset" với "underlying ability" nó được kỳ vọng đo (ví dụ ImageNet được thiết kế đo "object recognition" nhưng DNN có thể giải nó chủ yếu bằng "đếm texture patch"); áp dụng "Morgan's Canon cho machine learning": không gán cho mô hình khả năng bậc cao nếu shortcut bậc thấp đủ giải thích hành vi quan sát được; kiểm tra baseline mạnh nhưng "ngu" (ví dụ trả lời câu hỏi về phim mà không cần xem phim) để lộ shortcut.
2. **Phát hiện shortcut qua o.o.d. generalisation test**: một test o.o.d. tốt cần (a) có distribution shift rõ ràng, (b) có intended solution rõ ràng (test trên white noise thì không có solution nào cả nên vô nghĩa), (c) đa số mô hình hiện tại đều gặp khó trên test đó. Box I liệt kê các benchmark o.o.d. gợi ý: adversarial attacks, ARCT (loại bỏ shortcut khỏi dữ liệu), cue-conflict stimuli, ImageNet-A/C, ObjectNet, PACS, các toy dataset có kiểm soát (Shift-MNIST, biased CelebA, unfair dSprites).
3. **Hiểu tại sao shortcut dễ học**: liên hệ "Principle of Least Effort" trong Linguistics (người nói có xu hướng tối giản nỗ lực giao tiếp) với 4 thành phần inductive bias ở trên.

4. Thí nghiệm và kết quả chính

Đây là bài **perspective** (bài tổng hợp quan điểm cho Nature Machine Intelligence), không phải bài thực nghiệm chính với bảng số liệu riêng; đóng góp thực nghiệm **duy nhất và mới** của chính nhóm tác giả là thí nghiệm toy star/moon (Hình 2, do J.J. thực hiện theo ghi nhận ở "Author contributions"). Phần còn lại là tổng hợp có trích dẫn các phát hiện từ các paper khác, trải khắp 4 lĩnh vực (Mục 5):

- **Computer Vision**: dịch ảnh vài pixel, xoay vật thể, thêm noise/blur, đổi background hoặc texture (giữ nguyên hình dạng) đều có thể đổi hẳn dự đoán; adversarial example là trường hợp cực đoan (thay đổi vô hình với mắt người nhưng đổi hẳn nhãn dự đoán): paper diễn giải adversarial example là "ít nhất một phần là hệ quả của shortcut learning".
- **NLP**: BERT học được việc chỉ cần phát hiện từ "not" đã đủ để làm tốt hơn-ngẫu-nhiên trên một dataset lập luận ngôn ngữ tự nhiên, dù không hiểu nội dung câu; cả BERT và GPT-2 (có pretraining không giám sát) vẫn không giải quyết được vấn đề shortcut này.

- **Agent-based (Reinforcement) Learning**: một agent học chơi Tetris **đơn giản học cách tạm dừng game** để tránh thua, thay vì học chơi Tetris: một ví dụ "reward hacking" cổ điển; domain randomization (thêm biến thiên màu/kích thước/texture/ánh sáng trong mô phỏng) giúp giảm "reality gap" giữa training mô phỏng và world thật.
- **Fairness**: công cụ tuyển dụng của Amazon học được rằng giới tính là predictor mạnh tới mức dù xoá tên ứng viên, mô hình vẫn "tìm đường vòng" (ví dụ suy ra giới tính từ tên trường đại học chỉ-dành-cho-nữ) để tiếp tục thiên vị nam giới.

5. Phân tích / ablation

Vì không phải bài thực nghiệm định lượng, "phân tích" ở đây chủ yếu là phân rã khái niệm (không có ablation số liệu như ECE hay accuracy table):

- **Ý nghĩa thực tiễn của taxonomy 4 tầng**: cho thấy **không thể** chỉ dựa vào i.i.d. test accuracy để phân biệt intended solution với shortcut: đây chính là lập luận nền cho khuyến nghị "o.o.d. test nên là chuẩn mực, không phải ngoại lệ" (Mục 6.2).
- **Winograd Schema Challenge là case-study về benchmark "mục xuống cấp"**: ban đầu thiết kế để tránh shortcut của Turing test, nhưng khi mô hình ngôn ngữ hiện đại đạt điểm "nghi ngờ cao", việc soát lại phát hiện benchmark này vẫn chứa nhiều shortcut opportunity hơn dự kiến ban đầu: minh hoạ rằng benchmark o.o.d. cũng cần tiến hoá theo mô hình, không phải cố định.
- **Adversarial robustness được paper liên kết với causality**: vì adversarial example được xem như "counterfactual explanation" (thay đổi nhỏ nhất tạo ra output khác), mục tiêu robustness cuối cùng gắn chặt với nghiên cứu nhân quả (causality) trong machine learning (Mục 7).

6. Giới hạn

Do tác giả tự nêu:

- "Overall, our selection of examples is biased towards Computer Vision since this is one of the areas where deep learning has had its biggest successes" (Mục 1): paper tự nhận thiên lệch chọn ví dụ nghiêng về Computer Vision.
- "Which decision rule is the intended solution is clearly in the eye of the beholder" (Mục 3): khái niệm "intended solution" vốn mang tính chủ quan, không có định nghĩa hình thức tuyệt đối.
- "While overcoming shortcut learning in its entirety may potentially be impossible..." (Kết luận): bài tự nhận bài toán có thể **không giải quyết được hoàn toàn**.
- Phần "Learning: optimisation" trong Box II tự nhận "the complex interactions between training method and architecture are poorly understood so far; strong claims can only be made for simple cases."

Do người ghi chú nhận thấy:

- Đây là bài **perspective/review**, không phải bài thực nghiệm độc lập mới (trừ toy example star/moon); hầu hết "bằng chứng" là trích dẫn lại kết quả từ các paper khác ([9], [13], [14], [15], [36], [38], [67] v.v.), nên độ tin cậy của mỗi claim phụ thuộc vào paper gốc được trích, không được paper này tự kiểm chứng lại.
- Một phần đáng kể các ví dụ trích dẫn (cow/grass [9], texture-shape cue conflict [38], excessive invariance [41]) là công trình của **chính các tác giả bài này** (Geirhos, Jacobsen) ở các paper trước đó; không phải vấn đề về tính đúng đắn, nhưng người đọc nên biết đây có yếu tố tự trích dẫn (self-citation) khi đánh giá độ "độc lập" của các minh chứng.
- Ví dụ QA ở trang 1 có khả năng chứa lỗi chính tả "Peython Manning" (đúng phải là "Peyton Manning") được giữ nguyên từ ví dụ gốc (nhiều khả năng từ paper adversarial SQuAD của Jia & Liang) mà không được paper này sửa hay chú thích; đây không phải lỗi logic/khoa học của chính paper, chỉ là một điểm kỹ thuật nhỏ (typo) đáng lưu ý khi trích dẫn lại ví dụ này.

- Phạm vi đã đọc: toàn bộ Mục 1-8 (Introduction, shortcut learning ở sinh học, taxonomy, nguồn gốc shortcut, biểu hiện qua 4 lĩnh vực, chẩn đoán, các hướng vượt qua shortcut learning, kết luận) cộng Acknowledgements/Author contributions; không đọc chi tiết danh sách tài liệu tham khảo đầy đủ (References, hơn 140 mục) ngoài việc tra một số mục được trích trực tiếp trong văn bản.

7. Liên hệ với các bài khác

- **ml basics 01_2016_rethinking_generalization.md**: tầng (2) của taxonomy ("training solutions, including overfitting solutions") liên hệ trực tiếp tới hiện tượng memorization mà bài 01 chứng minh (mạng neural có thể fit hoàn toàn nhãn ngẫu nhiên); cả hai bài cùng nhắc tới việc accuracy cao trên dữ liệu quan sát không đảm bảo mô hình học đúng "khả năng" mong muốn.
- **ml basics 19_2017_calibration.md**: một mô hình dựa vào shortcut dễ bị overconfident khi gặp dữ liệu o.o.d. (vì nó "không biết" mình đang dùng một cue không còn đúng); kết hợp hai góc nhìn, một mô hình lý tưởng không chỉ cần tránh shortcut mà còn cần báo hiệu confidence thấp khi gặp input lệch khỏi phân phối train, điều paper calibration không trực tiếp bàn tới nhưng là hệ quả tự nhiên khi ghép hai bài.
- **rl content/48_2025_spurious_rewards.md**: ví dụ "reward hacking" trong RL ở Mục 5 của bài này (agent học Tetris chỉ để tạm dừng game) là chính xác loại hiện tượng mà rl content 48 phân tích sâu hơn ở ngữ cảnh RLVR/LLM hiện đại (reward giả tạo, phần thưởng "spurious" không phản ánh đúng mục tiêu); nên đọc nối tiếp để thấy vấn đề shortcut/reward hacking không chỉ là chuyện 2020 mà vẫn tái diễn trong RL cho LLM reasoning 2025.

8. Gợi ý áp dụng

Đánh giá ngoài phân phối và thử nghiệm can thiệp có thể giúp phát hiện shortcut mà điểm kiểm tra thông thường bỏ sót. Việc chọn các phép thay đổi cần phản ánh môi trường sử dụng dự kiến và giữ nguyên nhãn khi thích hợp.

9. Thí nghiệm gợi ý

1. **Tái lập thí nghiệm toy star/moon của paper (FC network so với CNN).** Tạo dataset ảnh nhỏ (ví dụ 32x32) có hai lớp hình dạng, đặt hình dạng ở vị trí tương quan với nhãn trong train nhưng ngẫu nhiên hoá vị trí ở test o.o.d. Huấn luyện một mạng fully-connected và một CNN nhỏ, so sánh accuracy i.i.d. so với o.o.d. Kỳ vọng: FC network sập xuống gần random trên o.o.d. giống Hình 2 của paper; CNN ít bị ảnh hưởng hơn nhờ convolution weight-sharing. Diễn giải: xác nhận trực tiếp vai trò của "Structure" (kiến trúc) trong Box II, chạy được dễ dàng hoặc cả CPU.
2. **Tự tính minh hoạ "tụt xuống chance level khi bỏ shortcut" cho một classifier toy.** Giả sử tập train 1000 ảnh: 500 ảnh "cow" (450 trên cỏ, 50 không), 500 ảnh "non-cow" (50 trên cỏ, 450 không); một classifier-shortcut dự đoán "cow" nếu thấy cỏ. Trên train, classifier này đúng $(450+450)/1000 = 90\%$. Trên test o.o.d. loại bỏ tương quan (mỗi 1 trong 4 tổ hợp cow/non-cow $\times$ cỏ/không-cỏ chiếm 250/1000 đều nhau): đúng khi (cow, cỏ) và (non-cow, không-cỏ), tức $(250+250)/1000 = 50\%$: đúng bằng mức ngẫu nhiên (**tự tính**, minh hoạ đúng câu "drops to chance performance" của paper). Thí nghiệm thật: lặp lại với dữ liệu tổng hợp code hoá đúng tỉ lệ trên, kiểm tra một classifier logistic regression đơn giản học được đúng gần 90%/50% như tính toán. Diễn giải: hiểu định lượng mức độ "giả" của accuracy cao khi có shortcut.
3. **Cue-conflict test cho OCR nhỏ.** Tạo ảnh text tổng hợp trong đó font/kiểu nền (cue A) và hình dạng ký tự thật (cue B) bị đặt xung đột có kiểm soát (ví dụ render cùng một ký tự bằng 2 font khác nhau nhưng dán nhãn theo nền thay vì theo ký tự ở một phần dữ liệu train). Huấn luyện một OCR classifier nhỏ (nhận diện ký tự đơn), rồi test trên ảnh cue-conflict để xem mô hình theo cue nào. Kỳ vọng: nếu mô hình theo cue nền/font nhiều hơn hình ký tự, đó là dấu hiệu shortcut giống bias texture-shape ở Mục 4.2 của paper. Diễn giải: cho biết liệu nguy cơ shortcut được bàn ở trên có thực sự xảy ra ở mô hình nhỏ.

4. **So sánh accuracy i.i.d. so với o.o.d. trước/sau pruning trên một classifier ảnh nhỏ.** Huấn luyện một CNN nhỏ, đo accuracy trên test set cùng phân phối và trên test set có shift nhẹ (ví dụ thêm nhiễu, đổi độ sáng). Prune 30-50% trọng số (ví dụ magnitude pruning đơn giản), đo lại cả hai accuracy. Kỳ vọng: không biết trước (đây đúng là khoảng trống paper không trả lời); diễn giải: nếu accuracy o.o.d. giảm nhiều hơn tỉ lệ accuracy i.i.d. giảm, đó là bằng chứng thực nghiệm nhỏ cho nhận định rằng cần test o.o.d. riêng khi đánh giá pruning.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Shortcut	Decision rule tốt trên i.i.d. benchmark, thất bại trên o.o.d. test
Taxonomy 4 tầng	(1) mọi decision rule kể cả non-solution $\supset$ (2) training solution kể cả overfitting $\supset$ (3) i.i.d. test solution kể cả shortcut $\supset$ (4) intended solution
4 thành phần inductive bias	Structure (kiến trúc), Experience (dữ liệu), Goal (loss), Learning (optimization)
Morgan's Canon cho ML	"Never attribute to high-level abilities that which can be adequately explained by shortcut learning"
4 khuyến nghị kết luận	(1) connecting the dots, (2) diễn giải cẩn trọng, (3) test o.o.d., (4) hiểu điều gì khiến solution "dễ học"
Ví dụ tự tính (giả định cỏ-bò)	Train accuracy shortcut $90\% \to$ o.o.d. accuracy sau khi bỏ tương quan 50% (chance level)

11. Kết luận ngắn

Shortcut learning là khung khái niệm thống nhất giải thích vì sao deep learning vừa đạt hiệu năng "siêu người" trên benchmark vừa thất bại khó hiểu trong thực tế: mô hình chỉ học decision rule đủ tốt cho dữ liệu i.i.d. đã thấy, không nhất thiết là decision rule "intended". Bài báo không đề xuất một thuật toán cụ thể để giải quyết triệt để vấn đề này (và tự nhận có thể không giải quyết triệt để được), nhưng đưa ra khung chẩn đoán (taxonomy, inductive bias, Morgan's Canon) và khuyến nghị thực dụng nhất: biến o.o.d. generalisation test thành chuẩn mực đánh giá, thay vì chỉ dựa vào i.i.d. test accuracy.

Nguồn gốc: https://arxiv.org/abs/2004.07780

Bài 21

Leakage and the Reproducibility Crisis in ML-based Science (2022)

Tác giả: Sayash Kapoor, Arvind Narayanan (Department of Computer Science và Center for Information Technology Policy, Princeton University).
Venue / arXiv: paper không nêu venue hội nghị/tạp chí cụ thể trong văn bản arXiv; arXiv:2207.07048 (v1, 14/07/2022).
Nguồn: https://arxiv.org/abs/2207.07048
Chủ đề: Data leakage trong "ML-based science" (dùng hiệu năng mô hình ML làm bằng chứng cho một claim khoa học); khảo sát 20 paper/17 lĩnh vực; taxonomy 8 loại leakage (L1-L3); model info sheet; case study civil war prediction.
Vị trí trong lộ trình: Đọc cùng bài 20 về shortcut learning và bài 34 về cross-validation.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Nhiều ngành khoa học định lượng (chính trị học, y học, sinh học, bảo mật máy tính...) đã chuyển sang dùng mô hình ML để dự đoán và coi hiệu năng dự đoán đó là **bằng chứng khoa học** (ví dụ: "mô hình phức tạp dự đoán nội chiến tốt hơn hồi quy logistic, vậy quan hệ giữa các biến này phức tạp hơn tuyến tính"). Paper gọi phạm vi này là **"ML-based science"**, phân biệt rõ với 4 phạm vi khác dễ nhầm: methods research (tạo/cải thiện thuật toán ML tổng quát), ethics research, engineering application (xây sản phẩm/dịch vụ), và modeling contest (thi dự đoán trên tập dữ liệu cố định do bên thứ ba độc lập tạo).

Vấn đề: **data leakage** (mô hình "nhìn thấy" thông tin mà nó không nên có ở thời điểm dự đoán thật) đã được biết tới từ lâu trong ML ứng dụng (Kaufman et al. 2012), nhưng bài báo này cho thấy nó **chưa được khảo sát hệ thống** trong bối cảnh khoa học (ML-based science), và các biện pháp giảm thiểu leakage từ modeling contest/engineering application **không áp dụng thẳng** được sang khoa học.

2. Ý tưởng chính

Ba đóng góp chính:

1. **Khảo sát + taxonomy**: tổng hợp 20 paper trên 17 lĩnh vực (Y học, Neuroimaging, Autism Diagnostics, Bioinformatics, Nutrition, Software Engineering, Toxicology, Satellite Imaging, Tractography, Clinical Epidemiology, Brain-computer Interface, Histopathology, Neuropsychiatry, Radiology, IT Operations, Genomics, Computer Security), mỗi paper chỉ ra lỗi leakage ảnh hưởng tổng cộng **329 paper khác**. Mọi lĩnh vực khảo sát đều có leakage ở ít nhất một paper ("a pitfall in every single case").
2. **Model info sheet**: một template báo cáo (lấy cảm hứng từ "model card" của Mitchell et al. 2019) yêu cầu tác giả trình bày 3 luận cứ tường minh để chứng minh không có leakage, nhằm bắt lỗi leakage **trước khi** xuất bản.
3. **Case study thực nghiệm**: tái phân tích lĩnh vực dự đoán nội chiến (civil war prediction), nơi nhiều paper tuyên bố mô hình ML phức tạp (Random Forests, AdaBoost, Gradient Boosted Trees) vượt trội hồi quy logistic (LR) hàng chục năm tuổi; sau khi sửa lỗi leakage, **tất cả** các tuyên bố này đều không còn đứng vững.

3. Phương pháp chi tiết

3.1 Định nghĩa

> "Data leakage is a spurious relationship between the independent variables and the target variable that arises as an artifact of the data collection, sampling, or pre-processing strategy."

Vì quan hệ này là "giả" (artifact), nó **không tồn tại** trong phân phối mà claim khoa học thực sự nhắm tới, nên leakage thường dẫn tới **ước lượng hiệu năng bị thổi phồng** (inflated).

Paper cũng định nghĩa lại "reproducibility" rộng hơn định nghĩa hẹp thường dùng: một finding được coi là reproducible nếu code/data công khai **và** dữ liệu được phân tích đúng; định nghĩa hẹp hơn ("computational reproducibility", chỉ cần chạy lại đúng code/data ra đúng kết quả) không đủ, vì nó không bắt được trường hợp code chạy đúng nhưng có bug logic (Phụ lục A).

3.2 Taxonomy 8 loại leakage (Mục 2.4)

[L1] Thiếu tách biệt sạch giữa train và test:

- **L1.1 No test set**: dùng cùng dữ liệu để train và test (lỗi "sách giáo khoa" kinh điển).
- **L1.2 Pre-processing trên cả train và test**: ví dụ impute giá trị thiếu hoặc oversample **trước khi** chia train/test, khiến thông tin từ test rò vào quá trình xử lý.
- **L1.3 Feature selection trên cả train và test**: chọn feature dựa trên hiệu năng đo trên toàn bộ dữ liệu (bao gồm test) rồi mới train.
- **L1.4 Duplicates trong dataset**: nếu dataset có bản sao/gần giống nhau mà không loại trước khi chia, các bản sao có thể rơi vào cả train và test.

[L2] Mô hình dùng feature không hợp lệ (illegitimate features): feature đóng vai trò **proxy** cho chính biến mục tiêu, nên tại thời điểm dự đoán thật sự thông tin đó không có sẵn (ví dụ: dùng "đang dùng thuốc chống tăng huyết áp" để dự đoán "tăng huyết áp": tầm thường hoá bài toán). Loại này không có sub-category vì đánh giá "hợp lệ hay không" cần kiến thức miền, mang tính case-by-case.

[L3] Test set không được lấy từ đúng phân phối khoa học quan tâm:

- **L3.1 Temporal leakage**: test set chứa dữ liệu từ **trước** thời điểm của training set (mô hình được xây bằng dữ liệu "từ tương lai" so với một phần test).
- **L3.2 Nonindependence giữa mẫu train và test**: ví dụ cùng một bệnh nhân xuất hiện ở cả train và test (các lần đo khác nhau của cùng người) trong khi claim khoa học là dự đoán cho **bệnh nhân mới**.
- **L3.3 Sampling bias trong test distribution**: ví dụ bias không gian (lấy test từ một vùng địa lý nhưng claim áp dụng cho vùng khác), hoặc selection bias (loại các trường hợp "khó/biên" ra khỏi test set, làm test không đại diện quần thể thật).

Tổng: 4 (L1.x) $+1$ (L2) $+3$ (L3.x) $=8$ loại, đúng như abstract nêu "8 types of leakage".

3.3 Model info sheet (Mục 3)

Ba luận cứ (argument) cốt lõi tác giả phải trình bày:

1. **[L1] Clean train-test separation**: giải trình vì sao test set không tương tác với dữ liệu train ở bất kỳ bước preprocessing/modeling/evaluation nào.
2. **[L2] Mỗi feature đều hợp lệ**: giải trình vì sao mỗi feature dùng trong mô hình có ý nghĩa khoa học hợp lệ (không phải proxy ngầm của outcome).
3. **[L3] Test set được lấy từ đúng phân phối khoa học quan tâm**: giải trình không có selection/sampling bias, và nếu claim liên quan thời gian, test luôn ở timestamp sau train.

Model info sheet được paper tuyên bố là đủ để phát hiện **mọi** loại trong 8 loại leakage đã khảo sát (Mục 3.2), và được minh hoạ cụ thể cho từng loại ở Phụ lục C (ví dụ: L1.2 cần câu hỏi Q12-13 về cách tách train/test

trong bước preprocessing; L2 cần câu hỏi Q21 giải trình từng feature).

3.4 Case study: civil war prediction (Mục 4, Phụ lục B)

- Tìm kiếm hệ thống (chi tiết ở Phụ lục B.1) cho 124 paper có chứa "civil" AND "war" AND ("prediction"/"predicting"/"forecast") trong tiêu đề/abstract, công bố 01/01/2016 đến 14/05/2021.
- Thu hẹp còn **15 paper** thoả 2 tiêu chí (tập trung dự đoán nội chiến + đánh giá bằng train-test split): số liệu này nằm ở Phụ lục B.1.
- Trong 15 paper đó, **12 paper** chia sẻ đầy đủ code và data, được dùng cho phân tích chi tiết.
- Trong 12 paper, tìm thấy lỗi ở **đúng 4 paper**: đây chính là 4 paper duy nhất tuyên bố mô hình ML phức tạp vượt trội LR (Muchlinski et al. 2016, Colaresi & Mahmood 2017, Wang 2019, Kaufman et al. 2019).

4. Thí nghiệm và kết quả chính

Finding 1 (Hình 1): Data leakage khiến kết quả không tái lập được.

Paper	Claim gốc	Loại lỗi	Tác động sau khi sửa
Muchlinski et al. (2016)	Random Forests vượt trội LR	L1.2 (impute train+test chung)	RF không tốt hơn LR
Colaresi & Mahmood (2017)	Random Forests vượt trội LR	L1.2 (tái dùng dataset đã impute sai)	RF không tốt hơn LR
Wang (2019)	AdaBoost/GBT vượt trội các mô hình khác	L1.2 + L3.1 (k-fold CV trên dữ liệu có cấu trúc thời gian)	Khoảng cách AUC giữa AdaBoost và LR giảm từ 0.14 xuống 0.01
Kaufman et al. (2019)	AdaBoost vượt trội các mô hình khác	L2 (proxy variable cho outcome) + L3.1	Không mô hình nào vượt baseline dự đoán "giữ nguyên như năm trước"

Sau khi sửa lỗi, mô hình ML phức tạp **không tốt hơn LR một cách có ý nghĩa** trong mọi trường hợp, trừ Wang (2019) còn một khoảng cách rất nhỏ (0.01 AUC, được paper coi là "không đáng kể" dù khác 0 tuyệt đối). Không trường hợp nào trong 4 lỗi này **có thể phát hiện chỉ bằng đọc văn bản paper** (phải đọc code), nhưng model info sheet (nếu được điền đúng) sẽ bắt được cả 4 (Phụ lục C).

Finding 2: Thiếu kiểm định ý nghĩa thống kê (significance testing). 9 trong 12 paper có code/data đầy đủ **không** báo cáo significance test hay uncertainty quantification cho so sánh hiệu năng. Ví dụ minh hoạ: Blair & Sambanis (2020) có test set chỉ **11** sự kiện nội chiến khởi phát (civil war onset); AUC làm trơn (smoothed) báo cáo là 0.85, nhưng khoảng tin cậy 95% tính bằng bootstrap resampling test set là $[0.66, 0.95]$: rất rộng. Khi so sánh mô hình đề xuất (escalation model) với 4 baseline khác bằng one-tailed significance test (Robin et al. method cho ROC curve), kết quả: $Z = 0.64, 1.09, 0.42, 0.67$; $p = 0.26, 0.14, 0.34, 0.25$: **không có trường hợp nào đạt ý nghĩa thống kê**, và paper còn ghi rõ chưa hiệu chỉnh cho multiple comparison (nếu hiệu chỉnh, ý nghĩa thống kê sẽ càng thấp hơn).

5. Phân tích / ablation

Thay cho ablation số liệu, paper phân tích theo 2 hướng:

- **Vì sao mitigation từ modeling contest/engineering không áp dụng cho khoa học (Mục 2.2)**: trong modeling contest, bên thứ ba độc lập tạo dataset và giữ test set bí mật tới khi hết thi; không ai truy cập test set trước khi nộp. Trong khoa học, nhà nghiên cứu **có quyền truy cập toàn bộ dataset** suốt quá trình modeling, đây là nguồn gốc chính của leakage trong ML-based science. Trong engineering, biện

pháp phổ biến là "deploy ở quy mô nhỏ rồi quan sát": không áp dụng được cho khoa học vì mục tiêu không phải deploy sản phẩm mà là rút ra insight khoa học, và khoa học nhạy với sai khác hiệu năng **nhỏ** (trong khi engineering với dataset lớn thường không quan tâm khác biệt nhỏ).

- **5 chẩn đoán (D) kèm khuyến nghị (R) cho khủng hoảng tái lập (Mục 5)**:

Chẩn đoán	Khuyến nghị
D1: Thiếu hiểu biết về giới hạn khả năng dự đoán	R1: Nghiên cứu/truyền đạt rõ giới hạn dự đoán (tránh nhầm "ML vượt người ở nhận diện ảnh" với "ML vượt người ở mọi tác vụ dự đoán xã hội")
D2: Hype, overoptimism, publication bias	R2: Xem kết quả ML-based science là "tạm thời" (tentative), cần thận trọng; paper trích dẫn rằng finding không tái lập được bị trích dẫn **nhiều hơn** finding tái lập được, tạo vòng lặp phản hồi tăng overoptimism
D3: Thiếu chuyên môn phù hợp	R3: Hợp tác đa ngành (có cả người hiểu miền khoa học và người hiểu ML)
D4: Thiếu tiêu chuẩn hoá	R4: Áp dụng "common task framework" (bên thứ ba độc lập giữ test set bí mật) khi khả thi
D5: Thiếu computational reproducibility	R5: Dùng hạ tầng đảm bảo tái lập môi trường tính toán (ví dụ CodeOcean)

6. Giới hạn

Do tác giả tự nêu:

- Khảo sát "không phải một meta-review hệ thống từ một mẫu paper nhất quán", chỉ là "lower bound" của vấn đề reproducibility trong ML-based science (Mục 2.1), vì các lĩnh vực khác nhau dùng thuật ngữ khác nhau cho cùng một lỗi, khiến tìm kiếm hệ thống khó khăn.
- Đa số review chỉ xem xét **nội dung văn bản** của paper, không xem code/data, nên số paper bị ảnh hưởng thực tế có thể bị **đếm thiếu** (undercounting).
- Model info sheet tự nhận 3 giới hạn (Mục 3.4): (1) không thể kiểm chứng claim trong info sheet nếu thiếu computational reproducibility; (2) claim sai trong info sheet có thể tạo "an tâm giả" cho reviewer; (3) điền/đánh giá info sheet cần chuyên môn ML, lĩnh vực thiếu chuyên môn có thể vẫn bỏ sót leakage tinh vi dù có info sheet.
- Case study dự đoán nội chiến: "chúng tôi cảnh báo rằng việc kết quả đã sửa cho ước lượng chính xác hơn không có nghĩa là chúng tôi ủng hộ mọi lựa chọn phương pháp khác trong các paper đó" (Phụ lục B): ví dụ việc sửa Muchlinski et al. vẫn phải dùng một out-of-sample dataset có 95% giá trị thiếu, không lý tưởng.

Do người ghi chú phát hiện (mâu thuẫn số liệu giữa phần chính và phụ lục):

- Mục 4 (phần chính văn bản) viết: "We narrowed this list to the **12** papers that focused on predicting civil war, evaluated performance using a train-test split, and **shared the complete code and data**": câu này đọc như thể 3 tiêu chí (chủ đề, train-test split, chia sẻ code/data) được áp dụng **đồng thời trong một bước** để từ 124 paper ra ngay 12 paper. Tuy nhiên Phụ lục B.1 cho biết quá trình thực tế là **hai bước**: 124 paper → **15 paper** thoả 2 tiêu chí đầu (chủ đề + train-test split) → trong 15 paper đó chỉ **12** chia sẻ đầy đủ code/data. Số "15" hoàn toàn không xuất hiện ở phần chính văn bản, chỉ lộ ra khi đọc phụ lục. Đây không phải sai số nghiêm trọng (12 là số cuối cùng đúng ở cả hai nơi), nhưng cách trình bày ở phần chính dễ khiến người đọc nghĩ nhầm quy trình chỉ có một bước lọc.
- Phạm vi đã đọc: toàn bộ Mục 1-6 (Overview, Evidence of a reproducibility crisis, Model info sheets, Case study civil war, Beyond leakage, Conclusion); Phụ lục A (định nghĩa reproducibility) đọc đầy đủ; Phụ lục B chỉ đọc B.1 (phương pháp chọn paper) và phần mở đầu B.2, **không đọc chi tiết** B.2-B.7 (mô

tả từng paper lỗi cụ thể) và Bảng A6; Phụ lục C đọc đầy đủ (bảng đối chiếu loại leakage với câu hỏi trong model info sheet). Phần "Materials and methods" cuối Mục 6 (trước Appendix) không đọc chi tiết nội dung link website.

7. Liên hệ với các bài khác

- **ml basics 20_2020_shortcut_learning.md**: paper này **trực tiếp trích dẫn** bài Shortcut Learning (Geirhos et al., 2020) hai lần trong Mục 2.4 ("Robustness to distribution shift is an area of ongoing research in ML methods, and is as such an open problem (Geirhos et al., 2020)") và Mục 2.5 ("best practices to deal with distribution shift currently include testing the ML models on the data from the distribution we want to make claims about (Geirhos et al., 2020)"). Về bản chất, data leakage (nhất là L3: test set không đúng phân phối khoa học) là một trường hợp cụ thể, có thể định lượng, của hiện tượng shortcut learning: mô hình "ăn gian" nhờ một mối quan hệ chỉ tồn tại do cách thu thập/xử lý dữ liệu, không tồn tại trong phân phối thật.
- **ml basics 19_2017_calibration.md**: cả hai bài cùng cảnh báo về việc **tin quá mức** vào một con số hiệu năng duy nhất; "Finding 2" (thiếu significance testing, khoảng tin cậy AUC rộng tới $[0.66, 0.95]$) tương tự như tinh thần ECE/calibration: một mô hình có thể "trông" chính xác (AUC cao, hoặc confidence cao) nhưng độ không chắc chắn thật sự lớn hơn nhiều so với con số điểm (point estimate) thể hiện.
- **ml basics 01_2016_rethinking_generalization.md**: loại leakage L1.1 (No test set) chính là "lỗi sách giáo khoa" overfitting mà bài 01 minh hoạ ở quy mô cực đoan (mạng fit hoàn toàn nhãn ngẫu nhiên); cả hai bài cùng nhấn mạnh train performance (hoặc train+test lẫn nhau) không phản ánh khả năng tổng quát hoá thật.

8. Gợi ý áp dụng

Để hạn chế leakage, mọi bước học từ dữ liệu, kể cả tiền xử lý và chọn đặc trưng, cần được đặt đúng trong quy trình chia tập. Khi so sánh kết quả, nên công bố cách chia, quy trình chọn mô hình và độ bất định của ước lượng.

9. Thí nghiệm gợi ý

1. **Tự tính minh hoạ leakage loại L2 (proxy feature).** Giả sử một tập 200 mẫu "tăng huyết áp": 100 mẫu dương (trong đó 95 có dùng thuốc chống tăng huyết áp, 5 không), 100 mẫu âm (trong đó 5 có dùng thuốc, 95 không). Một classifier chỉ dựa feature "có dùng thuốc hay không" sẽ đúng 95 (dương, có thuốc) $+95$ (âm, không thuốc) $= 190/200 = 95\%$ (**tự tính**, mô phỏng đúng tình huống Filho et al. 2021 bị paper trích dẫn ở Mục 2.4 [L2]). Thí nghiệm thật: code hoá dataset tổng hợp đúng tỉ lệ trên, huấn luyện logistic regression, kiểm tra accuracy có khớp khoảng 95% như tính tay; sau đó loại bỏ feature proxy, đo lại accuracy thật dựa trên các feature hợp lệ còn lại. Diễn giải: định lượng trực quan mức độ "giả" của accuracy khi có proxy leakage, chạy được trên CPU, không cần GPU.
2. **Mô phỏng L1.2 (preprocessing rò rỉ) trên toy dataset ảnh nhỏ.** Tạo dataset ảnh nhỏ có missing pixel ngẫu nhiên; so sánh accuracy của một classifier nhỏ khi (a) impute giá trị thiếu bằng mean tính trên **toàn bộ** dữ liệu (train+test) so với (b) impute bằng mean tính **chỉ trên train** rồi áp dụng cho test. Kỳ vọng: (a) cho accuracy cao hơn (a) một cách giả tạo so với (b), đặc biệt rõ khi tỉ lệ thiếu dữ liệu lớn. Diễn giải: tái lập trực tiếp cơ chế lỗi của Muchlinski et al. (2016) ở quy mô nhỏ.
3. **Kiểm tra trùng lặp giữa các tập.** Dùng metadata để tìm tài liệu hoặc mẫu có liên hệ xuất hiện ở cả tập huấn luyện và tập kiểm tra; đánh giá nguy cơ leakage theo cách chia dữ liệu.
4. **Ước lượng khoảng tin cậy bằng bootstrap.** Lấy mẫu lại các đơn vị kiểm tra và tính lại chỉ số để khảo sát độ bất định của kết quả; khi so sánh hai mô hình trên cùng mẫu, dùng chênh lệch theo cặp.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Data leakage (định nghĩa)	Quan hệ giả giữa biến độc lập và biến mục tiêu, sinh ra do cách thu thập/sampling/tiền xử lý dữ liệu
Reproducibility (định nghĩa rộng)	Code + data công khai **và** dữ liệu được phân tích đúng (không chỉ "chạy lại ra đúng số")
Taxonomy 8 loại leakage	L1.1 No test set; L1.2 Preprocessing train+test; L1.3 Feature selection train+test; L1.4 Duplicates; L2 Feature không hợp lệ (proxy); L3.1 Temporal; L3.2 Nonindependence; L3.3 Sampling bias
Model info sheet	3 luận cứ: [L1] clean train-test separation, [L2] mỗi feature hợp lệ, [L3] test set đúng phân phối khoa học
Quy mô khảo sát	20 paper, 17 lĩnh vực, ảnh hưởng 329 paper khác; leakage có mặt trong mọi lĩnh vực khảo sát
Case study civil war	124 $\rightarrow$ 15 (chủ đề + train-test split) $\rightarrow$ 12 (chia sẻ code/data) $\rightarrow$ lỗi ở đúng 4 paper
Kết quả sau sửa lỗi	Khoảng cách AUC AdaBoost so với LR (Wang 2019): $0.14 \rightarrow 0.01$
5 chẩn đoán	D1 giới hạn dự đoán, D2 hype/publication bias, D3 thiếu chuyên môn, D4 thiếu chuẩn hoá, D5 thiếu computational reproducibility

11. Kết luận ngắn

Data leakage là lỗi phổ biến và có hệ thống trong "ML-based science", xuất hiện ở mọi lĩnh vực được khảo sát và đủ nghiêm trọng để đảo ngược hoàn toàn các claim nổi bật (mô hình ML phức tạp "vượt trội" các mô hình thống kê cũ). Taxonomy 8 loại leakage (L1-L3) cùng model info sheet (3 luận cứ: tách train-test sạch, feature hợp lệ, test set đúng phân phối) là công cụ cụ thể để phát hiện và ngăn chặn leakage trước khi xuất bản, dù bản thân công cụ này vẫn phụ thuộc vào computational reproducibility và chuyên môn của người điền/review để thực sự hiệu quả.

Nguồn gốc: https://arxiv.org/abs/2207.07048

Bài 22

Underspecification Presents Challenges for Credibility in Modern Machine Learning (2020)

Tác giả: Alexander D'Amour, Katherine Heller, Dan Moldovan (đóng góp ngang nhau), cùng hơn 30 đồng tác giả khác tại Google Research/Google Health, Stanford, UCSD, và cộng tác viên lâm sàng (VA, Aravind Eye Hospital, Sankara Nethralaya).
Venue / arXiv: paper không nêu venue hội nghị/tạp chí trong văn bản arXiv; arXiv:2011.03395 (v2, 24/11/2020).
Nguồn: https://arxiv.org/abs/2011.03395
Chủ đề: Underspecification: hiện tượng một ML pipeline có thể trả về nhiều predictor khác nhau cùng đạt hiệu năng iid gần-tối-ưu tương đương, nhưng các predictor này lại hành xử rất khác nhau khi gặp distribution shift; minh chứng qua 3 ví dụ toán đơn giản và 4 case study deep learning (computer vision, ảnh y tế, NLP, EHR).
Vị trí trong lộ trình: Đọc cùng các bài 20–21 về shortcut và leakage để khảo sát những giới hạn của đánh giá mô hình.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

ML pipeline chuẩn chỉ xác nhận một predictor f bằng cách đo rủi ro (risk) trên một test set i.i.d. (rút từ cùng phân phối P với training data). Cách làm này **không nhạy** với inductive bias cụ thể mà mô hình đã học được, miễn là rủi ro i.i.d. thấp. Lo ngại về "spurious correlation" và "shortcut learning" (xem ml basics 20) đã chỉ ra một lỗ hổng: đôi khi predictor tối ưu i.i.d. **buộc phải** dùng đặc trưng giả (spurious), vì có xung đột cấu trúc (structural conflict) giữa phân phối train và deploy.

Bài báo này chỉ ra đây **không phải toàn bộ câu chuyện**. Theo cách nhìn "structural conflict" thuần túy, ta sẽ kỳ vọng hai predictor huấn luyện giống nhau sẽ lộ **cùng một lỗi** khi deploy. Quan sát trung tâm của bài báo: trên thực tế, các predictor huấn luyện tới cùng mức độ tổng quát hoá i.i.d. lại thường hành xử **rất khác nhau** khi áp vào tình huống thực tế, dù chỉ khác nhau ở một lựa chọn tưởng như không quan trọng (ví dụ random seed).

2. Ý tưởng chính

Định nghĩa underspecification: một ML pipeline là underspecified nếu có nhiều cách khác nhau (ví dụ cấu hình trọng số khác nhau) để mô hình đạt hiệu năng held-out gần-như-tương đương trên dữ liệu i.i.d., ngay cả khi model specification và training data giữ cố định. Gọi tập các predictor "gần-tối-ưu, rủi ro tương đương" này là $F^* \subset F$.

Hai claim chính:

1. **Claim 1**: underspecification là trở ngại chính để huấn luyện tin cậy predictor hành xử đúng khi deploy. Pipeline chỉ nhạy với hiệu năng i.i.d. sẽ trả về **một predictor tuỳ ý** trong F^*; dù F^* có chứa predictor với inductive bias đúng, pipeline không có cách gì đảm bảo chọn đúng predictor đó. Paper minh chứng bằng 3 ví dụ đơn giản (mô phỏng dịch bệnh, mô hình random feature lý thuyết, mô hình tuyến tính di truyền thực tế).

2. **Claim 2**: underspecification **phổ biến** trong các ứng dụng ML hiện đại và có hệ quả thực tiễn đáng kể, minh chứng qua một protocol thực nghiệm áp dụng cho pipeline deep learning cấp sản phẩm ở 4 lĩnh vực (computer vision, ảnh y tế, NLP, dự đoán từ hồ sơ bệnh án điện tử EHR).

Phân biệt quan trọng (Mục 2.2) giữa 2 chế độ thất bại:

- **Structural failure mode** (shortcut learning kiểu Geirhos 2020): predictor tối ưu i.i.d. **bắt buộc** phải dùng đặc trưng giả vì có xung đột cấu trúc; mọi predictor gần-tối-ưu sẽ lộ **cùng một kiểu lỗi**.
- **Underspecified failure mode** (claim của bài này): không có xung đột cấu trúc, chỉ đơn giản là **không đủ thông tin** trong phân phối train để phân biệt inductive bias "đúng" với inductive bias "giả" (paper diễn giải bằng ngôn ngữ nhân quả: đây là thiếu "positivity", không phải lỗi cấu trúc); các predictor gần-tối-ưu khác nhau lộ ra **các lỗi khác nhau**, không đồng nhất.

3. Phương pháp chi tiết

3.1 Ba loại stress test (Mục 2.3)

Loại	Cách làm	Ví dụ trong paper
Stratified evaluation	Chia test set theo một đặc trưng A, so sánh metric giữa các nhóm	Accuracy theo skin type (Fitzpatrick scale)
Shifted evaluation	Đánh giá trên phân phối $P' \neq P$ (biến đổi có kiểm soát hoặc thu thập khác)	ImageNet-C (nhiễu tổng hợp), ObjectNet, camera type mới
Contrastive evaluation	So sánh output trên tập các input "khớp cặp" (matched set) bị biến đổi, kiểm tra tính nhất quán	Fairness counterfactual, robustness check trong NLP

3.2 Protocol thực nghiệm chung (Mục 4)

1. Tạo **ensemble** predictor từ cùng một pipeline, chỉ khác một thành phần nhỏ (random seed, scheme khởi tạo, loại RNN cell...).
2. Xác nhận các predictor trong ensemble có hiệu năng i.i.d. **gần như tương đương** (tức đều thuộc F^*).
3. Đánh giá ensemble trên stress test; đo 3 tiêu chí để xác nhận đây đúng là underspecification (không phải nhiễu thống kê thường):
   - **Biên độ biến thiên** so với biến thiên i.i.d. (hoặc so với benchmark ngoài).
   - **Tính không thể đoán được** từ hiệu năng i.i.d.: dùng Spearman rank correlation giữa metric i.i.d. và metric stress test; tương quan yếu nghĩa là không thể chọn predictor tốt trên stress test chỉ dựa vào hiệu năng i.i.d.
   - **Tính hệ thống** của sự khác biệt (không chỉ do nhiễu mẫu): dùng permutation test hoặc phân tích định tính bổ sung.

3.3 Ba ví dụ "khởi động" (Mục 3, mô hình đơn giản, phân tích được)

- **Mô hình dịch bệnh SIR**: với dữ liệu quan sát sớm (giai đoạn T_{obs} nhỏ), tốc độ tăng trưởng quan sát được chỉ xác định $\beta - 1/D$ (β = tốc độ lây, D = thời gian trung bình còn lây nhiễm), không xác định riêng β và D. **Tự tính**: giả sử tốc độ tăng trưởng quan sát $r = \beta - 1/D = 0.1$/ngày; nghiệm A: $D = 5 \Rightarrow \beta = 0.1 + 1/5 = 0.3$, cho hệ số lây cơ bản $R_0 = \beta D = 1.5$; nghiệm B: $D = 20 \Rightarrow \beta = 0.1 + 1/20 = 0.15$, cho $R_0 = \beta D = 3.0$. Hai nghiệm khớp **chính xác** cùng một $r = 0.1$ quan sát được, nhưng R_0 (quyết định quy mô dịch bệnh ở đỉnh điểm) khác nhau **gấp đôi** (1.5 so với 3.0): minh hoạ đúng hiện tượng paper mô tả ở Hình 1 (hai quỹ đạo dự báo khác nhau hàng chục lần dù khớp dữ liệu quan sát giống nhau).
- **Mô hình random feature (lý thuyết, xấp xỉ mạng vô hạn rộng)**: ở chế độ overparameterized ($N/n \geq 1$), nghiệm min-norm interpolation từ hai random feature matrix W_1, W_2 độc lập có rủi ro i.i.d. **không**

thể phân biệt, nhưng hàm dự đoán f_{W_1}, f_{W_2} gần như **trực giao**. Với một phân phối shift P_{Δ,W_0} được chọn "đối kháng" riêng cho W_0: rủi ro của W_0 tăng gấp 3 lần, còn rủi ro của một W độc lập hầu như không đổi.

- **Polygenic Risk Score (PRS) tuyến tính, UK Biobank, dự đoán intraocular pressure (IOP)**: do các biến thể di truyền tương quan cao (collinear), bước tiền xử lý chuẩn ("LD clumping") chỉ chọn 1 đại diện mỗi cụm biến thể tương quan; đổi đại diện trong cụm hầu như không đổi rủi ro i.i.d. Ensemble 1000 predictor PRS (khác nhau ở lựa chọn đại diện) có hiệu năng gần như nhau trên tập train/test Anh Quốc, nhưng phân tán rất rộng khi transfer sang tập test "non-British"; tương quan Spearman giữa hiệu năng trên 2 tập chỉ $\rho = 0.135$ (95% CI 0.070-0.20); heuristic chuẩn của công cụ PLINK (phổ biến trong di truyền học) **không** tốt hơn việc chọn đại diện ngẫu nhiên trên tập shifted.

4. Thí nghiệm và kết quả chính

Bốn case study deep learning (Mục 5-8), mỗi case study là một lĩnh vực sản phẩm thực tế:

Lĩnh vực	Mô hình / ensemble	Hiệu năng i.i.d.	Stress test	Kết quả chính
Computer Vision (Mục 5)	ResNet-50 (50 seed) và BiT ResNet-101x3 (30 lần fine-tune, pretrain JFT-300M)	ResNet-50: $75.9\% \pm 0.11$; BiT: $86.2\% \pm 0.09$ top-1 ImageNet	ImageNet-C (corruption mức 5), ObjectNet	Biến thiên trên một số task ImageNet-C lớn hơn biến thiên i.i.d. **cả một bậc độ lớn**; 95% CI hệ số tương quan Pearson (i.i.d. so với stress test) chứa số 0 cho cả 2 model; SD trên ObjectNet gấp 2x (ResNet-50) và 5x (BiT) so với test chuẩn
Ảnh y tế - nhãn khoa (Mục 6.1)	Inception-V4 fine-tune, 10 seed, dự đoán diabetic retinopathy	AUC ổn định trên camera type 1-4 (dùng để train)	Camera type 5 (chưa từng gặp khi train)	Z-test so sánh SD AUC: $z = 2.47$, $p = 0.007$ (một phía); đường calibration gần giống nhau ở camera 1-4 nhưng khác biệt rõ ở camera 5 (Hình 7)
Ảnh y tế - da liễu (Mục 6.2)	Inception-V4 fine-tune, 10 seed, phân loại tình trạng da	Accuracy tổng thể ổn định	Stratify theo loại da Fitzpatrick (I-V)	Da loại IV ($n = 798$, 19.6% test set): permutation test $p = 0.03$ (biến thiên có tính hệ thống); da loại III, V: $p = 0.54$, $p = 0.42$ (giải thích được bằng nhiễu mẫu)

Lĩnh vực	Mô hình / ensemble	Hiệu năng i.i.d.	Stress test	Kết quả chính
NLP (Mục 7)	BERT Large Cased, 5 pretraining seed $\times$ 20 fine-tuning seed (tới 100 model)	STS-B 0.87-0.90; pronoun resolution 0.960-0.965; MNLI matched 83.4-84.4%	STS/pronoun templates giới tính (Webster 2020, Rudinger 2018); StereoSet; HANS + Naik StressTest	Tương quan gender-correlation với STS/pronoun accuracy yếu ($\rho = 0.21$, $\rho = 0.08$); ICAT StereoSet dao động 3.35 điểm giữa 5 checkpoint pretrain giống nhau, **lớn hơn** khoảng cách giữa 6 model top bảng xếp hạng công khai (khác hẳn kiến trúc/kích thước/dữ liệu); biến thiên trên HANS/StressTest nhạy cả với seed fine-tuning **và** pretraining
EHR (Mục 8)	RNN (SRU/LSTM/UGRNN) $\times$ 5 seed $= 15$ model, dự đoán AKI từ hồ sơ VA (703,782 bệnh nhân)	Normalized PRAUC 34.59-36.61	Time-shift tín hiệu + giới hạn lab panel (mô phỏng loại bỏ tín hiệu "operational")	Hiệu năng trên dữ liệu bị can thiệp vừa giảm vừa phân tán rộng hơn; 2 model LSTM chỉ khác seed cho số lượng quyết định "flip" khác nhau, và phần lớn các flip này **không trùng** patient-timepoint giữa 2 model

5. Phân tích / ablation

- **Scaling không giải quyết được underspecification**: BiT (model lớn, pretrain trên JFT-300M, SOTA ImageNet) vẫn thể hiện underspecification rõ không kém ResNet-50, dù scaling đã được biết giúp cải thiện đồng thời cả robustness và i.i.d. accuracy (Djolonga 2020, Taori 2020). Paper suy đoán: khi model scale lên, phần "headroom" cải thiện OOD còn lại có thể càng do underspecification quyết định.
- **PCA soi cấu trúc biến thiên NLP (Hình 9)**: thành phần chính thứ nhất (22% phương sai) của độ lệch similarity-score giữa 20 lần fine-tune cùng 1 checkpoint BERT tương quan trực tiếp với tỉ lệ nữ giới tham gia nghề nghiệp (BLS); các model có loading dương/âm cực trị trên trục này biểu hiện rõ "mâu thuẫn giới tính-nghề nghiệp" khác nhau (nam-y tá/lễ tân so với nữ-thợ máy/thợ mộc/bác sĩ).
- **Pretraining seed ảnh hưởng độc lập với fine-tuning seed**: ở cả StereoSet và NLI stress test, các checkpoint pretrain khác nhau tạo ra **cụm** hành vi khác nhau rõ rệt (Hình 10, 11), cho thấy một phần robustness/bias được "đóng khung" (bake in) ngay từ giai đoạn pretrain, không chỉ do fine-tuning.
- **Tương quan giữa các stress test với nhau cũng yếu (Hình 12)**: ngoại trừ vài cặp stress test được thiết kế đo cùng loại inductive bias, phần lớn các stress test tương quan yếu với nhau, gợi ý không gian các "inductive bias bị underspecified" có **nhiều chiều** độc lập, không thể gộp về một chỉ số duy nhất.
- **Ablation timestamp feature cho EHR (nêu ở Phụ lục, không trình bày chi tiết trong main text đã đọc)**: một model huấn luyện với feature timestamp bị loại bỏ hoàn toàn vẫn đạt hiệu năng i.i.d. **giống**

hệt bản gốc, cho thấy việc giảm phụ thuộc vào tín hiệu "operational" có thể không phải đánh đổi với hiệu năng i.i.d.

6. Giới hạn

Do tác giả tự nêu (Mục 9, Discussion):

- Phương pháp của bài **đánh giá thấp hơn thực tế** mức độ underspecification: chỉ thăm dò bằng cách đổi random seed, trong khi pipeline thực tế còn vô số lựa chọn tuỳ ý khác (scheme khởi tạo, cách tham số hoá, optimizer, cách biểu diễn dữ liệu, batch size, learning rate...) mỗi lựa chọn đều có thể mang "implicit regularization" riêng.
- Tiêu chí lựa chọn/ràng buộc để thu hẹp F^* về đúng inductive bias mong muốn **vẫn là bài toán mở**, và các ràng buộc này về bản chất phải đặc thù theo ứng dụng, khó tổng quát hoá thành phương pháp chung.
- Marginalize/ensemble toàn bộ F^* (một chiến lược phổ biến để cải thiện hiệu năng i.i.d.) **không được đảm bảo** cải thiện hành vi OOD, vì một số thành viên của F^* vốn đã tổng quát hoá kém.

Do người ghi chú phát hiện (lỗi/không nhất quán nội tại trong văn bản):

- **Định nghĩa F-statistic bị viết ngược trong cả 2 bảng**: chú thích Bảng 3 viết "the F statistic quantifies how systematic differences are between pretrainings using the ratio of **within-pretraining variance to between-pretraining variance**" và chú thích Bảng 4 lặp lại nguyên văn công thức này. Theo quy ước ANOVA chuẩn, F = between-group variance/within-group variance (F lớn = khác biệt hệ thống giữa nhóm lớn so với nhiễu nội nhóm); nếu đúng như câu chữ "within/between" thì F lớn phải có nghĩa **ngược lại** (nhiễu nội nhóm lớn so với khác biệt giữa nhóm), mâu thuẫn trực tiếp với câu ngay sau đó trong Bảng 3: "F-values of this magnitude are consistent with systematic **between-group** variation." Nhiều khả năng đây là lỗi đánh máy lặp lại ở cả hai bảng (ý định đúng là between/within), nhưng về mặt văn bản, câu chữ hiện tại tự mâu thuẫn với chính diễn giải đi kèm.
- **Một lỗi số liệu nhỏ, có khả năng là typo, ở Mục 5.2**: khoảng tin cậy Spearman ρ cho BiT trên ObjectNet được viết là "$0.47\ (0.13, 71)$" (dòng văn bản, Mục 5.2): cận trên "71" rất có khả năng thiếu phần thập phân, đúng ra phải là "0.71" (giống định dạng cận trên của các khoảng tin cậy khác trong bài, luôn trong đoạn $[-1, 1]$). Số liệu này không ảnh hưởng tới luận điểm chính (vẫn cho thấy tương quan yếu-trung bình), nhưng là một lỗi trình bày cụ thể, trích dẫn được.
- **Khoảng tin cậy Spearman ρ không khớp hoàn toàn giữa văn xuôi và Bảng 3 (lệch 0.01)**: văn bản Mục 7.1.3 viết "STS-B: Spearman $\rho = 0.21$; 95% CI $= (0.00, 0.39)$" và "Pronoun resolution: Spearman $\rho = 0.08$; 95% CI $= (-0.13, 0.29)$", nhưng Bảng 3 lại ghi "$0.21\ (-0.00, 0.40)$" và "$0.08\ (-0.13, 0.28)$": cận trên lệch 0.01 ở cả hai dòng (có thể do làm tròn số khác nhau giữa bản thảo và bảng, không ảnh hưởng kết luận nhưng là một điểm không khớp số liệu cụ thể).
- Phạm vi đã đọc: toàn bộ Mục 1-9 (Introduction, Preliminaries, 3 ví dụ khởi động, 4 case study deep learning, Discussion) và Acknowledgements; **không đọc chi tiết** các Phụ lục A-E (A: ví dụ bổ sung về marginalization, B: chi tiết Hình 1, C: kết quả ablation timestamp EHR, D: chi tiết thí nghiệm PRS, E: suy diễn toán cho mô hình random feature) ngoài các chỗ được trích dẫn trực tiếp trong main text.

7. Liên hệ với các bài khác

- **ml basics 20_2020_shortcut_learning.md**: paper này trích dẫn trực tiếp Geirhos et al. (2020) ở Mục 2.2 để phân biệt rõ "structural failure mode" (shortcut learning, mọi predictor gần-tối-ưu lộ **cùng lỗi**) với "underspecified failure mode" (các predictor gần-tối-ưu lộ **lỗi khác nhau**); đọc 2 bài liên tiếp giúp hiểu đây là 2 góc nhìn bổ sung, không loại trừ nhau, cho cùng hiện tượng "i.i.d. accuracy cao không đảm bảo hành vi đúng khi deploy".
- **ml basics 21_2022_leakage_reproducibility.md**: cả hai bài cùng cảnh báo việc tin tưởng một con số hiệu năng duy nhất trên một test set cố định; loại leakage L3 (test set không đúng phân phối khoa học

quan tâm) và khái niệm "stress test shifted evaluation" ở đây đều nhắm tới cùng vấn đề: validation set hiện tại có thể không đo đúng thứ cần đo cho ứng dụng thật.

- **ml basics 19_2017_calibration.md**: case study nhãn khoa (Mục 6.1, Hình 7) dùng chính **reliability/calibration plot** (khái niệm từ bài 19) làm công cụ phát hiện underspecification: hai model chỉ khác random seed có đường calibration giống nhau trên dữ liệu train nhưng khác biệt rõ trên camera type mới, chứng minh calibration cũng là một đại lượng bị underspecified.
- **pruning content/08_2018_lottery_ticket_hypothesis.md**: paper này trích dẫn trực tiếp (Mục 2.1) công trình về "network pruning" (Frankle et al., 2020, cùng tác giả Lottery Ticket Hypothesis) khi thảo luận về connectivity giữa các global mode của loss landscape; gợi ý rằng các "ticket" hay sub-network khác nhau tìm được bởi pruning cũng là những điểm khác nhau trong tập F^* các predictor near-optimal, tiềm năng mang inductive bias khác nhau dù cùng đạt sparsity/accuracy mục tiêu.

8. Gợi ý áp dụng

Nhiều mô hình có kết quả tương đương trên dữ liệu kiểm định vẫn có thể khác nhau khi gặp dịch chuyển phân phối. Huấn luyện nhiều lần và thiết kế stress test theo các điều kiện có ý nghĩa giúp quan sát sự biến thiên đó.

9. Thí nghiệm gợi ý

1. **Tái lập protocol cốt lõi ở quy mô nhỏ cho một OCR/classifier đơn giản.** Huấn luyện 5-10 model nhỏ (hoặc fine-tune 5-10 lần) trên cùng pipeline, chỉ khác random seed, đo accuracy/CER trên validation set (xác nhận gần như bằng nhau) rồi đo trên 1 stress test (ví dụ ảnh có noise/blur tổng hợp, hoặc font khác). Kỳ vọng: biến thiên trên stress test lớn hơn rõ rệt biến thiên trên validation, giống Hình 4-5 của paper. Diễn giải: xác nhận trực tiếp underspecification có xảy ra ở quy mô nhỏ hay không, chạy được vì model/batch nhỏ.
2. **Tính Spearman ρ giữa validation accuracy và stress-test accuracy trên ensemble nhỏ đó.** Kỳ vọng: ρ yếu (tương tự 0.08-0.47 trong các case study của paper), nghĩa là không thể dùng validation accuracy để đoán trước model nào sẽ tốt trên stress test. Diễn giải: nếu ρ quả thực yếu, đây là cảnh báo cụ thể rằng chọn checkpoint pruning "tốt nhất" theo validation accuracy không đảm bảo chọn đúng checkpoint tổng quát hoá tốt nhất.
3. **Tự tính mở rộng ví dụ SIR cho một bài toán 2 tham số đơn giản khác (ví dụ fit đường mũ $y = ae^{(b-c)t}$).** Chọn 2 cặp (b, c) cho cùng $b - c = 0.1$ (ví dụ $(0.3, 0.2)$ và $(0.5, 0.4)$), vẽ dự báo y tại t lớn: với $a = 1$, $t = 50$: cặp 1 cho $y = e^5 \approx 148.4$; cặp 2 cũng cho $y = e^5 \approx 148.4$ (vì cùng $b - c$): nhưng nếu đổi c lệch đi dù giữ gần-đúng tốc độ tăng quan sát ở vùng dữ liệu ít, dự báo dài hạn lệch nhanh. Thí nghiệm thật: dựng toy regression 2 tham số, chỉ quan sát dữ liệu sớm, dùng gradient descent với các khởi tạo khác nhau, đo độ phân tán dự báo dài hạn. Diễn giải: xây trực giác về non-identifiability trước khi áp dụng khái niệm cho mô hình phức tạp hơn.
4. **Contrastive evaluation cho OCR nhỏ: cùng 1 chuỗi ký tự, 2 font render khác nhau.** Tạo bộ test "matched pairs" (cùng text, khác font), đo tỉ lệ model dự đoán **khác nhau** giữa 2 phiên bản của cùng text trên từng checkpoint trong ensemble ở thí nghiệm 1. Kỳ vọng: tỉ lệ bất nhất quán khác nhau giữa các checkpoint (một số checkpoint nhạy cảm với font hơn checkpoint khác dù CER tổng thể bằng nhau). Diễn giải: minh chứng cụ thể cho khả năng stress test loại contrastive có thể lộ sự khác biệt mà CER tổng không lộ ra.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung
Underspecification (định nghĩa)	Pipeline có tập F^* (nhiều predictor) đạt rủi ro i.i.d. gần-tối-ưu tương đương, nhưng khác nhau ở inductive bias/hành vi OOD

Khái niệm	Nội dung
Structural failure mode	Mọi predictor gần-tối-ưu buộc phải dùng đặc trưng giả (do xung đột cấu trúc train/deploy), lộ **cùng** lỗi
Underspecified failure mode	Các predictor gần-tối-ưu lộ **lỗi khác nhau**, do thiếu thông tin phân biệt (không phải xung đột cấu trúc)
3 loại stress test	Stratified (theo subgroup), Shifted (theo phân phối khác), Contrastive (theo cặp input khớp nhau)
3 tiêu chí xác nhận underspecification	Biên độ biến thiên lớn, tương quan yếu với i.i.d. (Spearman ρ), tính hệ thống (permutation test)
Ví dụ tự tính (SIR)	$r = \beta - 1/D = 0.1$: nghiệm $D = 5, \beta = 0.3 \Rightarrow R_0 = 1.5$; nghiệm $D = 20, \beta = 0.15 \Rightarrow R_0 = 3.0$ (cùng r, R_0 gấp đôi)
Kết quả tiêu biểu CV	ResNet-50/BiT: biến thiên ImageNet-C lớn hơn biến thiên i.i.d. tới 1 bậc độ lớn
Kết quả tiêu biểu NLP	ICAT StereoSet dao động 3.35 điểm giữa 5 pretraining-seed giống nhau, lớn hơn khoảng cách giữa 6 kiến trúc khác nhau trên leaderboard

11. Kết luận ngắn

Underspecification là một chế độ thất bại riêng biệt, khác với shortcut learning kiểu "xung đột cấu trúc": một ML pipeline hiện đại thường có nhiều predictor đạt hiệu năng i.i.d. gần-tối-ưu tương đương, nhưng các predictor này mã hoá inductive bias khác nhau và hành xử khác nhau đáng kể khi gặp distribution shift, dù chỉ khác nhau ở một lựa chọn tuỳ ý như random seed. Hiện tượng này được xác nhận có mặt ở cả mô hình toán đơn giản và 4 lĩnh vực deep learning sản phẩm thực tế (computer vision, ảnh y tế, NLP, EHR), và không biến mất khi scale model lên. Hệ quả thực tiễn: cần stress test gắn với yêu cầu ứng dụng cụ thể, và cần cơ chế ràng buộc rõ ràng (không chỉ đo i.i.d. accuracy) để chọn đúng predictor có inductive bias đáng tin cậy trong tập F^*.

Nguồn gốc: https://arxiv.org/abs/2011.03395

Bài 23

Explaining and Harnessing Adversarial Examples (2014)

Tác giả: Ian J. Goodfellow, Jonathon Shlens, Christian Szegedy (Google Inc.).
Venue / arXiv: Published as a conference paper at ICLR 2015 (ghi rõ ngay đầu bài); arXiv:1412.6572 (v3, 20/03/2015).
Nguồn: https://arxiv.org/abs/1412.6572
Chủ đề: Giải thích nguyên nhân tồn tại của adversarial example bằng tính **tuyến tính** (không phải phi tuyến/overfitting như giả thuyết trước đó); Fast Gradient Sign Method (FGSM); adversarial training như một kỹ thuật regularization.
Vị trí trong lộ trình: Đọc cùng bài 24 về adversarial examples và bài 39 về adversarial training.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Szegedy et al. (2014b, chính là ml basics 24) phát hiện: nhiều mô hình ML hiện đại, bao gồm neural network SOTA, bị đánh lừa bởi **adversarial example**: input gần như giống hệt một ví dụ phân loại đúng nhưng bị gán nhãn sai, thường với độ tin cậy cao. Hiện tượng gây ngạc nhiên hơn nữa: cùng một adversarial example thường đánh lừa được **nhiều model khác nhau** (kiến trúc khác nhau, huấn luyện trên tập con dữ liệu khác nhau), và các model bị đánh lừa thường **đồng thuận** về nhãn sai đó. Các giải thích ban đầu đổ lỗi cho tính phi tuyến cực độ của deep network, kết hợp với thiếu model averaging và thiếu regularization.

2. Ý tưởng chính

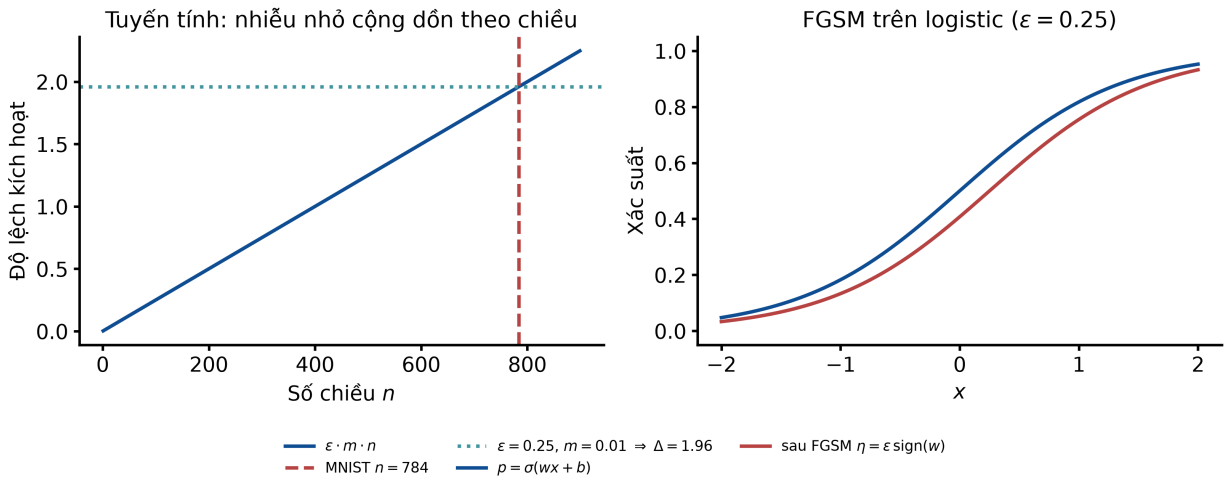
Paper phản bác giả thuyết "phi tuyến": lập luận rằng **chính tính tuyến tính** của các thành phần hiện đại (ReLU, maxout, LSTM, hay sigmoid được tinh chỉnh để hoạt động ở vùng gần-tuyến-tính) là nguyên nhân chính. Lập luận này:

- Đơn giản hơn giả thuyết cũ, và còn giải thích được vì sao **softmax regression** (mô hình tuyến tính thuần) cũng dễ bị tấn công.
- Giải thích được lần đầu tiên tại sao adversarial example **tổng quát hoá qua kiến trúc và tập train khác nhau**: các model tuyến tính học trên cùng nhiệm vụ có xu hướng học trọng số tương tự nhau, nên cùng nhạy với cùng một hướng nhiễu.
- Dẫn tới một phương pháp sinh adversarial example **nhanh và rẻ**: Fast Gradient Sign Method (FGSM), biến adversarial training từ một kỹ thuật tốn kém (cần L-BFGS có ràng buộc ở mỗi bước) thành khả thi để chạy thường quy.

3. Phương pháp chi tiết

3.1 Giải thích tuyến tính (Mục 3)

Với model tuyến tính, xét $w^T\tilde{x} = w^Tx + w^T\eta$. Với ràng buộc $\|\eta\|_\infty \leq \epsilon$, giá trị η tối đa hoá $w^T\eta$ là $\eta = \text{sign}(w)$ (nhân thêm ϵ), khi đó kích hoạt tăng thêm $\epsilon \cdot m \cdot n$ (n = số chiều, m = độ lớn trung bình một phần tử của w). Vì $\|\eta\|_\infty$ **không tăng** theo số chiều nhưng tổng thay đổi kích hoạt tăng **tuyến tính** theo n, trong không gian nhiều chiều, rất nhiều thay đổi nhỏ cộng lại thành một thay đổi lớn ở output. Paper gọi đây là "accidental steganography": model tuyến tính buộc phải chú ý đến tín hiệu khớp hướng với trọng số của nó, dù biên độ tín hiệu đó nhỏ hơn các tín hiệu khác nhiều.



Tích luỹ $\varepsilon \cdot m \cdot n$ (MNIST $\Delta = 1.96$); FGSM dịch sigmoid logistic (minh hoạ, tự tính)

Tự tính: giả sử w có $n = 784$ chiều (giống kích thước ảnh MNIST 28×28), độ lớn trung bình mỗi phần tử $m = 0.01$, $\epsilon = 0.25$. Thay đổi kích hoạt $= \epsilon \cdot m \cdot n = 0.25 \times 0.01 \times 784 = 1.96$: một con số đủ lớn để đảo ngược quyết định phân loại của một unit tuyến tính nếu margin ban đầu nhỏ hơn giá trị này, dù mỗi phần tử của η chỉ thay đổi input tối đa 0.25 (nhỏ so với range $[0, 1]$ của pixel).

3.2 Fast Gradient Sign Method (FGSM, Mục 4)

$$\eta = \text{sign}(\nabla_x J(\theta, x, y))$$

Đây là nhiễu loạn tối ưu (max-norm constrained) khi tuyến tính hoá hàm cost J xung quanh θ hiện tại; gradient có thể tính hiệu quả bằng backprop (1 lần forward + 1 lần backward, rẻ hơn hẳn L-BFGS có ràng buộc).

3.3 Adversarial training của mô hình tuyến tính so với weight decay (Mục 5)

Với logistic regression, FGSM là **chính xác** (không phải xấp xỉ) nhiễu tệ nhất trong max-norm box. Vì $\text{sign}(\nabla) = -\text{sign}(w)$ và $w^T \text{sign}(w) = \|w\|_1$, mục tiêu adversarial trở thành tối thiểu hoá $\mathbb{E}[\zeta(y(\|w\|_1 - w^T x - b))]$: giống **L1 regularization** nhưng khác biệt quan trọng: hạng tử $\|w\|_1$ bị **trừ** khỏi activation trong lúc train (không phải cộng vào cost), nên nó có thể "tự tắt" khi model đủ tự tin (ζ bão hoà); L1 weight decay không có cơ chế tự tắt này nên "worst-case" hơn adversarial training. Thực nghiệm: hệ số weight decay 0.0025 ở tầng đầu đã quá lớn, khiến model mắc kẹt với lỗi train $> 5\%$; hệ số nhỏ hơn train được nhưng không mang lợi ích regularization nào.

3.4 Adversarial training của deep network (Mục 6)

Hàm mục tiêu:

$$\tilde{J}(\theta, x, y) = \alpha J(\theta, x, y) + (1 - \alpha) J(\theta, x + \text{sign}(\nabla_x J(\theta, x, y)), y), \quad \alpha = 0.5$$

Universal approximator theorem (Hornik 1989) đảm bảo network có ≥ 1 hidden layer **có thể** biểu diễn hàm chống nhiễu, nhưng không đảm bảo thuật toán train sẽ **tìm ra** hàm đó; cần mã hoá rõ yêu cầu này vào hàm mục tiêu train (chính là lý do thêm số hạng adversarial).

4. Thí nghiệm và kết quả chính

Thí nghiệm	ϵ	Kết quả
GoogLeNet/ImageNet, demo định tính (Hình 1)	0.007 (= đúng bằng bit nhỏ nhất của ảnh 8-bit)	"panda" 57.7% $\rightarrow$ "gibbon" 99.3%
Softmax shallow / MNIST	0.25	error 99.9%, confidence trung bình 79.3%
Maxout network / MNIST (chưa train adversarial)	0.25	error 89.4%, confidence 97.6%
Maxout conv / CIFAR-10 (đã tiền xử lý)	0.1	error 87.15%, xác suất gán nhãn sai trung bình 96.6%
Logistic regression (3 so với 7) / MNIST	0.25	error $1.6\% \rightarrow 99\%$ (clean so với adversarial)
RBF network (0 hidden layer) / MNIST	0.25	error 55.4% nhưng confidence khi sai chỉ 1.2% (so với 60.6% confidence trên clean test)

Adversarial training trên maxout + dropout (MNIST, mạng 240 unit/layer): error giảm $0.94\% \rightarrow 0.84\%$.

Sau khi tăng kích thước mạng (1600 unit/layer) và đổi tiêu chí early stopping (dùng adversarial validation error thay vì validation error thường): 5 lần train với seed khác nhau cho 4 lần error 0.77% và 1 lần 0.83%, trung bình 0.782%: paper tự nhận đây là "kết quả tốt nhất được báo cáo trên MNIST permutation-invariant" nhưng **"statistically indistinguishable"** so với kết quả 0.79% của fine-tune DBM với dropout (Srivastava 2014, không dùng adversarial training). Error trên chính adversarial example giảm từ 89.4% (không train adversarial) xuống 17.9% (có train adversarial), nhưng khi model đã train adversarial vẫn sai, confidence trung bình vẫn cao (81.4%).

Transfer giữa 2 model (gốc so với đã adversarial-trained): adversarial example sinh từ model gốc gây error 19.6% trên model đã train adversarial; ngược lại, adversarial example sinh từ model đã train adversarial gây error 40.9% trên model gốc: bất đối xứng rõ rệt (model đã "cứng" hơn thì ít bị ảnh hưởng từ adversarial example cũ, nhưng vẫn tạo được adversarial example mới gây hại mạnh cho model gốc).

Thí nghiệm kiểm chứng giả thuyết thay thế (Mục 9):

- **Giả thuyết generative training giúp chống nhiễu**: dùng MP-DBM (Goodfellow 2013a, error clean 0.88%, inference differentiable): vẫn bị lừa với error 97.5% ở $\epsilon = 0.25$. **Bị bác bỏ.**
- **Giả thuyết ensembling "trung hoà" các quirk riêng của từng model**: ensemble 12 maxout network (chỉ khác seed): vẫn error 91.1% (tấn công cả ensemble) hoặc 87.9% (tấn công 1 thành viên rồi lan sang ensemble). **Bị bác bỏ, chỉ có tác dụng hạn chế.**

5. Phân tích / ablation

- **Vì sao adversarial example tổng quát hoá qua model khác nhau (Mục 8)**: dưới góc nhìn tuyến tính, adversarial example tồn tại trong **vùng liên tục rộng** (không phải "túi" rải rác tinh vi như giả thuyết trước), vì chỉ cần η có dot-product dương với gradient cost và ϵ đủ lớn. Hình 4 minh hoạ: dò theo các giá trị ϵ khác nhau cho thấy nhãn sai **ổn định trên một dải rộng** của ϵ, không phải chỉ tại điểm cô lập.
- **Kiểm định định lượng giả thuyết "các model học trọng số tương tự"**: sinh adversarial example từ 1 maxout network, phân loại lại bằng softmax shallow và RBF shallow. Loại trừ các trường hợp cả 2 model so sánh đều sai: softmax dự đoán đúng nhãn của maxout 84.6% số lần, RBF chỉ 54.3% số lần; để so sánh, RBF dự đoán đúng nhãn của softmax 53.6% số lần (tức RBF vẫn có một phần hành vi tuyến tính trong chính nó). Paper tự nhận không giải thích được **toàn bộ** lỗi của maxout, nhưng một phần đáng kể phù hợp với giả thuyết tuyến tính.
- **Control experiment với nhiễu ngẫu nhiên (không có hướng)**: train maxout với nhiễu $\pm\epsilon$ ngẫu nhiên từng pixel hoặc $U(-\epsilon, \epsilon)$: cho error 86.2% (confidence 97.3%) và 90.4% (confidence 97.8%) trên

FGSM adversarial example, **không khác biệt đáng kể** so với không train thêm gì. Kết luận: nhiễu zero-mean không có hướng hầu như vô dụng để chống adversarial example có hướng, vì kỳ vọng dot-product giữa nhiễu ngẫu nhiên và một vector tham chiếu bất kỳ bằng 0.

- **Nên perturb input hay hidden layer?**: kết quả "không nhất quán" (paper tự dùng từ này): Szegedy et al. (2014b) báo cáo perturb hidden layer tốt hơn (trên sigmoid network); ở đây, với FGSM trên network có activation không chặn trên, perturb input thường tốt hơn (vì hidden unit tự khuếch đại hoạt động để "lấn át" perturbation cộng thêm); trên model bão hoà (Rust model) perturb input và hidden layer cho hiệu quả tương đương. Không nên perturb **tầng ẩn cuối** (trước lớp linear-softmax cuối) vì lớp cuối không phải universal approximator của tầng ẩn cuối, dễ gây underfitting.
- **RBF/quadratic network để chống nhiễu**: RBF miễn nhiễm với adversarial example (vì confidence tụt khi xa dữ liệu train) nhưng đổi lại mất khả năng tổng quát hoá (không bất biến với biến đổi). Paper thử mở rộng sang các unit bậc hai (quadratic) nói chung, nhưng "mọi model có đủ 'quadratic inhibition' để chống nhiễu lại cho lỗi train cao khi dùng SGD": không tìm được điểm cân bằng tốt bằng SGD thông thường.

6. Giới hạn

Do tác giả tự nêu:

- Kết quả MNIST "tốt nhất" (0.782%) chỉ **"statistically indistinguishable"** so với kết quả không dùng adversarial training đã có từ trước (0.79%, dropout DBM fine-tune): nghĩa là cải thiện headline của chính bài có thể không có ý nghĩa thống kê rõ rệt so với baseline mạnh nhất đã biết.
- Giả thuyết tuyến tính **không giải thích hết** mọi lỗi của maxout network hay mọi trường hợp tổng quát hoá qua model (tỉ lệ dự đoán khớp nhãn giữa model chỉ 54-85% tuỳ phép đo, không phải 100%).
- Adversarial training perturb tầng ẩn cuối gây underfitting; không có cách rõ ràng để huấn luyện thành công deep RBF/quadratic network bằng SGD.
- Mô hình đã train adversarial, khi vẫn sai, vẫn tự tin cao (81.4%): adversarial training không giải quyết triệt để vấn đề overconfidence, chỉ giảm tần suất sai.

Do người ghi chú phát hiện:

- **Mức độ "imperceptible" không nhất quán giữa phần minh hoạ và phần thực nghiệm chính**: Hình 1 (panda/gibbon) dùng $\epsilon = 0.007$, được giải thích chặt chẽ bằng lập luận lượng tử hoá 8-bit ở Mục 3 (nhiễu nhỏ hơn 1 bit thì cảm biến/bộ nhớ vốn đã bỏ qua, nên về lý thuyết **không thể phân biệt** bằng mắt người). Nhưng phần lớn kết quả định lượng chính trên MNIST/CIFAR-10 (Mục 4-6) dùng $\epsilon = 0.25$ (MNIST, dải pixel $[0, 1]$) hoặc $\epsilon = 0.1$ (CIFAR-10 đã chuẩn hoá): lớn hơn nhiều so với "1 bit lượng tử hoá", và không còn được biện minh bằng đúng lập luận lượng tử hoá đó. Với $\epsilon = 0.25$, paper phải dùng một lập luận khác (chú thích 1): ảnh MNIST "gần như nhị phân" nên người vẫn đọc được dù bị nhiễu ở biên độ này. Hai lập luận ("không thể phân biệt bằng mắt" so với "người vẫn đọc được dù nhiễu rõ") là hai tiêu chuẩn khác nhau, và bài không nói rõ ràng khi chuyển từ ví dụ minh hoạ ($\epsilon = 0.007$) sang thực nghiệm định lượng ($\epsilon = 0.25$) thì tiêu chuẩn "adversarial" đã chuyển từ "vô hình" sang "vẫn nhận dạng được nhưng gây sai lệch mạnh": một sự trượt định nghĩa cần lưu ý khi trích dẫn số liệu cụ thể của bài.
- Mục 7 có lỗi đánh máy nhỏ trong câu "very model with sufficient quadratic inhibition..." (nhiều khả năng ý là "every model"), không ảnh hưởng nội dung nhưng là lỗi văn bản cụ thể.
- Phạm vi đã đọc: toàn bộ Mục 1-10 (Introduction, Related Work, giải thích tuyến tính, FGSM, adversarial training logistic regression, adversarial training deep network, model capacity, vì sao adversarial example tổng quát hoá, giả thuyết thay thế, Summary) cộng Acknowledgements và Phụ lục A (rubbish class examples): đọc đầy đủ, không có phần nào bị bỏ qua (bài tương đối ngắn, 11 trang kể cả phụ lục).

7. Liên hệ với các bài khác

- **ml basics 20_2020_shortcut_learning.md**: adversarial example được chính bài Shortcut Learning liệt kê như "trường hợp cực đoan" của shortcut learning (thay đổi vô hình với người nhưng đổi hẳn dự đoán máy); bài FGSM cho một cơ chế toán học cụ thể (tuyến tính + nhiều chiều) giải thích *tại sao* loại shortcut cực đoan này tồn tại và vì sao nó tổng quát hoá qua model.
- **ml basics 22_2020_underspecification.md**: hiện tượng "adversarial example tổng quát hoá qua model khác nhau nhưng không hoàn toàn" (chỉ 54-85% khớp nhãn tuỳ cách đo, Mục 8 của bài này) có thể đọc như một dạng bằng chứng thực nghiệm sớm (2014) cho khái niệm underspecification: các model "tương đương" về train/test accuracy vẫn có inductive bias (ở đây là "hướng nhạy cảm tuyến tính") không hoàn toàn giống nhau.
- **ml basics 01_2016_rethinking_generalization.md**: cả hai bài cùng thách thức trực giác "model fit tốt dữ liệu train/test nghĩa là đã học đúng khái niệm"; bài 01 chứng minh bằng khả năng fit nhãn ngẫu nhiên, bài FGSM chứng minh bằng khả năng bị đánh lừa có hệ thống ngay trên các điểm gần dữ liệu thật.
- **ml basics 24_2013_intriguing_properties.md**: đây chính là bài gốc (Szegedy et al. 2014b) phát hiện ra adversarial example bằng tối ưu hoá L-BFGS có ràng buộc hộp, giải thích hiện tượng bằng giả thuyết "manifold xác suất thấp nhưng dày đặc"; FGSM thay giả thuyết đó bằng lập luận tuyến tính đơn giản hơn, và thay thủ tục L-BFGS đa bước bằng một bước gradient-sign rẻ hơn hẳn (FGSM), nên đọc nối tiếp hai bài theo thứ tự này.

8. Gợi ý áp dụng

FGSM là một baseline đơn giản để khảo sát độ nhạy với nhiễu đối kháng có giới hạn. Kết quả phụ thuộc threat model và cường độ tấn công; một tấn công đơn lẻ không đủ chứng minh độ bền tổng quát.

9. Thí nghiệm gợi ý

1. **Tái lập FGSM trên một OCR/classifier nhỏ.** Huấn luyện một classifier ký tự nhỏ (ví dụ CNN nhỏ trên MNIST hoặc một bộ ký tự tổng hợp giống OCR), sinh adversarial example bằng $\eta = \epsilon \operatorname{sign}(\nabla_x J)$ ở vài mức ϵ (ví dụ $0.05, 0.1, 0.25$), đo accuracy và confidence trung bình trên example sai. Kỳ vọng: accuracy giảm mạnh, confidence trên các dự đoán sai vẫn cao, giống mẫu hình paper báo cáo (ví dụ 97.6% confidence khi sai với maxout). Diễn giải: xác nhận hiện tượng "sai nhưng tự tin cao" cũng xảy ra ở model OCR nhỏ.
2. **Adversarial training kiểu FGSM ($\alpha = 0.5$) trên cùng model, đo đánh đổi clean so với robust accuracy.** Huấn luyện lại model ở thí nghiệm 1 với hàm mục tiêu trộn $\tilde{J}$. Kỳ vọng: accuracy trên adversarial example tăng rõ (từ rất thấp lên trung bình), accuracy trên clean test giữ nguyên hoặc giảm rất ít, giống mẫu hình $89.4\% \rightarrow 17.9\%$ error của paper. Diễn giải: định lượng chi phí/lợi ích cụ thể của kỹ thuật trước khi áp dụng cho pruning recovery thật.
3. **Đo độ nhạy adversarial trước/sau khi pruning model nhỏ.** Lấy một classifier nhỏ, prune 30-50% trọng số (ví dụ magnitude pruning), so sánh accuracy dưới FGSM ở cùng ϵ giữa bản gốc và bản đã prune, dù cả hai có accuracy clean gần bằng nhau. Kỳ vọng: không biết trước (đây là khoảng trống paper không trả lời trực tiếp, chỉ gợi ý theo lý thuyết); diễn giải: nếu bản đã prune nhạy hơn rõ rệt, đó là bằng chứng cụ thể ủng hộ suy luận "pruning có thể làm tăng độ nhạy nhiễu", có thể thử thêm robust fine-tuning.
4. **Kiểm tra transfer của adversarial example giữa 2 biến thể pruning khác nhau.** Sinh adversarial example bằng FGSM từ biến thể pruning A, kiểm tra tỉ lệ đánh lừa thành công khi áp lên biến thể pruning B (kiến trúc giống nhau, pruning mask khác nhau hoặc seed fine-tune khác nhau). Kỳ vọng: tỉ lệ transfer thành công đáng kể nhưng không tuyệt đối, giống mẫu hình "19.6% so với 40.9%" bất đối xứng giữa 2 model gốc/đã-robust trong paper. Diễn giải: cho biết liệu các cấu hình pruning khác

nhau của cùng một model có chia sẻ cùng điểm yếu nhiễu hay không, liên hệ trực tiếp tới câu hỏi underspecification.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Fast Gradient Sign Method	$\eta = \epsilon \operatorname{sign}(\nabla_x J(\theta, x, y))$
Tăng kích hoạt tuyến tính do nhiễu	$\Delta = \epsilon \cdot m \cdot n$ (n = số chiều, m = độ lớn trung bình phần tử w)
Hàm mục tiêu adversarial training	$\tilde{J} = \alpha J(\theta, x, y) + (1-\alpha) J(\theta, x+\eta, y)$, $\alpha = 0.5$
Adversarial logistic regression	tối thiểu $\mathbb{E}[\zeta(y(\|w\|_1 - w^T x - b))]$, $\zeta(z) = \log(1+e^z)$
Kết quả tiêu biểu	Maxout/MNIST: error $89.4\% \to 17.9\%$ sau adversarial training ($\epsilon = 0.25$)
RBF so với linear	RBF: confidence thấp khi bị lừa (1.2%) nhưng tổng quát hoá kém; linear: recall cao, precision thấp ở vùng lạ
Kết luận tổng hợp	Adversarial example là hệ quả của model quá tuyến tính trong không gian nhiều chiều, không phải quá phi tuyến

11. Kết luận ngắn

Trái với giả thuyết ban đầu đổ lỗi cho tính phi tuyến, bài này cho thấy adversarial example là hệ quả tự nhiên của việc các model hiện đại (ReLU, maxout, LSTM...) được thiết kế để hành xử **gần tuyến tính** nhằm dễ tối ưu: trong không gian nhiều chiều, một nhiễu có hướng đúng dù biên độ mỗi phần tử rất nhỏ vẫn có thể cộng dồn thành thay đổi lớn ở output. Cách nhìn này giải thích được cả việc softmax tuyến tính thuần cũng bị lừa, cả việc adversarial example tổng quát hoá qua kiến trúc/tập train khác nhau, và dẫn tới FGSM, một phương pháp sinh adversarial example rẻ đủ để dùng adversarial training như một kỹ thuật regularization thực dụng, dù hiệu quả cải thiện cuối cùng trên MNIST chỉ "thống kê không phân biệt được" so với kỹ thuật regularization khác đã có từ trước.

Nguồn gốc: https://arxiv.org/abs/1412.6572

Bài 24

Intriguing Properties of Neural Networks (2013)

Tác giả: Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, Rob Fergus (Google Inc., New York University, University of Montreal, Facebook Inc.).
Venue / arXiv: Không ghi rõ venue trong phần text đã đọc (chỉ có arXiv header); được biết rộng rãi là ICLR 2014, nhưng dòng "Published as a conference paper" như ở ml basics 23 không xuất hiện trong bản text này. arXiv:1312.6199 (v4, 19/02/2014).
Nguồn: https://arxiv.org/abs/1312.6199
Chủ đề: Hai tính chất "kỳ lạ" của deep neural network: (1) thông tin ngữ nghĩa nằm trong toàn bộ không gian activation, không riêng từng unit; (2) tồn tại adversarial example, nhiễu nhỏ không thể nhận biết nhưng đảo ngược dự đoán, và nhiễu này tổng quát hoá qua model/tập train khác nhau.
Vị trí trong lộ trình: Đọc cùng bài 23 về FGSM và bài 39 về adversarial training.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Deep network đạt hiệu năng cao nhờ tính biểu diễn phong phú (expressiveness), nhưng chính điều này khiến hàm học được khó diễn giải và có thể mang tính chất "phi trực giác". Các công trình trước đó (Girshick 2013, Zeiler và Fergus 2013, Goodfellow et al. 2009) phân tích ý nghĩa ngữ nghĩa của từng unit bằng cách tìm input làm cực đại hoá activation của unit đó, ngầm giả định cơ sở tự nhiên (natural basis, tức từng toạ độ riêng của tầng feature cuối) là đặc biệt hữu ích để trích xuất ngữ nghĩa. Bài này đặt câu hỏi: cơ sở tự nhiên đó có thực sự đặc biệt, hay bất kỳ hướng nào trong không gian activation cũng cho kết quả tương tự?

2. Ý tưởng chính

Hai phát hiện phản trực giác:

1. **Không có sự khác biệt giữa unit riêng lẻ và tổ hợp tuyến tính ngẫu nhiên của các unit ở tầng cao**: ảnh làm cực đại activation theo một hướng ngẫu nhiên $v \in \mathbb{R}^n$ cũng mang các thuộc tính ngữ nghĩa dễ nhận ra giống như ảnh làm cực đại một unit đơn lẻ e_i. Suy ra: thông tin ngữ nghĩa nằm trong **toàn bộ không gian** activation, không phải ở từng toạ độ/unit riêng biệt; cơ sở tự nhiên không "đặc biệt" hơn cơ sở ngẫu nhiên.
2. **Deep network học một hàm input-output khá "gián đoạn" (discontinuous)** ở mức độ đáng kể: có thể khiến network phân loại sai một ảnh bằng cách thêm một nhiễu nhỏ, "hầu như không thể nhận biết" (hardly perceptible), tìm được bằng cách **cực đại hoá lỗi dự đoán** của network (ngược hướng với train thông thường). Quan trọng hơn: nhiễu này **không phải artifact ngẫu nhiên** của quá trình học: cùng một nhiễu có thể khiến **một network khác**, được train trên **tập con dữ liệu khác**, cũng phân loại sai cùng input đó.

3. Phương pháp chi tiết

3.1 Kiểm tra cơ sở tự nhiên so với cơ sở ngẫu nhiên (Mục 3)

So sánh hai cách tìm ảnh "đại diện ngữ nghĩa" của một hướng trong không gian activation $\phi(x)$:

$$x_0 = \arg\max_{x \in I} \langle \phi(x), e_i \rangle \quad \text{(unit tự nhiên)} \qquad \text{so với} \qquad x_0 = \arg\max_{x \in I} \langle \phi(x), v \rangle \quad \text{(hướng ngẫu nhiên } v\text{)}$$

với I là tập ảnh held-out (test set MNIST hoặc validation set ImageNet). Nếu các ảnh x_0 tìm được theo hai cách đều có nhóm ảnh trong cùng một hàng chia sẻ thuộc tính ngữ nghĩa tương tự (ví dụ "nét cong dưới", "hoa trắng"), thì cơ sở tự nhiên không có gì đặc biệt hơn cơ sở ngẫu nhiên.

3.2 Sinh adversarial example bằng tối ưu hoá có ràng buộc hộp (Mục 4.1)

Với classifier $f : \mathbb{R}^m \to \{1, \dots, k\}$ và loss liên tục loss_f, với ảnh x và nhãn mục tiêu $l \neq f(x)$, bài toán gốc là:

$$\text{minimize } \|r\|_2 \quad \text{với } f(x+r) = l,\ x + r \in [0,1]^m$$

Đây là NP-hard/non-convex nói chung, nên bài xấp xỉ bằng **box-constrained L-BFGS**: tìm $c > 0$ nhỏ nhất (line-search) sao cho nghiệm r của

$$\text{minimize } c|r| + \text{loss}_f(x+r, l) \quad \text{với } x + r \in [0,1]^m$$

thoả $f(x+r) = l$. Với loss lồi, phương pháp penalty này cho nghiệm chính xác; với neural network (non-convex) chỉ là xấp xỉ. Phân biệt rõ với FGSM (ml basics 23): đây là một bài toán tối ưu **đa bước, tìm nhiễu tối thiểu theo chuẩn L_2 để đạt một nhãn mục tiêu cụ thể**, tốn kém hơn hẳn so với 1 bước gradient-sign của FGSM, nhưng không giới hạn vào hướng $\text{sign}(\nabla_x J)$ nên có thể tìm nhiễu nhỏ hơn.

3.3 Phân tích phổ độ bất ổn (Spectral analysis, Mục 4.3)

Với network K tầng $\phi(x) = \phi_K(\phi_{K-1}(\dots \phi_1(x; W_1); W_2) \dots; W_K)$, định nghĩa hằng số Lipschitz trên L_k của mỗi tầng: $\|\phi_k(x) - \phi_k(x+r)\| \leq L_k \|r\|$. Khi đó $\|\phi(x) - \phi(x+r)\| \leq L\|r\|$ với $L = \prod_{k=1}^{K} L_k$. Với tầng rectify (ReLU) $\phi_k(x) = \max(0, W_k x + b_k)$, vì $\max(0, \cdot)$ là co (contractive), $L_k \leq \|W_k\|$ (chuẩn operator, tức singular value lớn nhất). Tầng max-pooling và contrast-normalization cũng co ($L_k \leq 1$ hoặc $\leq \gamma^{-1}$). Suy ra: chỉ cần tính chuẩn operator của từng tầng fully-connected/convolutional là có **chặn trên bảo toàn (conservative)** cho độ bất ổn toàn network, **không cần sinh adversarial example thật**. Bài nhấn mạnh: chặn trên lớn **không** đảm bảo tồn tại adversarial example; nhưng chặn trên nhỏ **đảm bảo không có** adversarial example nào tồn tại trong bán kính đó.

4. Thí nghiệm và kết quả chính

4.1 Unit tự nhiên so với hướng ngẫu nhiên

Trên MNIST (CNN) và ImageNet (AlexNet), cả Hình 1/3 (unit tự nhiên) và Hình 2/4 (hướng ngẫu nhiên) đều cho các nhóm ảnh "chia sẻ nhiều thuộc tính mức cao" trong cùng một hàng. Kết luận định tính: không phân biệt được chất lượng ngữ nghĩa giữa hai cơ sở.

4.2 Adversarial example trên MNIST (Bảng 1-2)

Model	Training error	Test error	Min. distortion TB
FC10(10^{-4}) (softmax, $\lambda = 10^{-4}$)	6.7%	7.4%	0.062
FC10(10^{-2})	10%	9.4%	0.1
FC10(1) ($\lambda = 1$, cực lớn)	21.2%	20%	0.14
FC100-100-10 (sigmoid 2 tầng ẩn)	0%	1.64%	0.058
FC200-200-10	0%	1.54%	0.065
AE400-10 (autoencoder + softmax)	0.57%	1.9%	0.086

Mọi model, kể cả FC10(1) với weight decay cực lớn, **vẫn tìm được** adversarial example đạt 0% accuracy trên train set với nhiễu nhỏ (distortion trung bình 0.058 đến 0.14 trên thang pixel $[0, 1]$, $n = 784$ chiều).

Cross-model generalization (Bảng 2): adversarial example sinh cho model A, khi đưa vào model B, gây lỗi đáng kể dù thấp hơn trong-model (ví dụ example sinh cho FC10(10^{-2}) gây lỗi 87.1% trên FC10(10^{-4}), nhưng chỉ 35.9% trên FC100-100-10). So với nhiễu Gaussian cùng độ lớn (stddev 0.1: lỗi chỉ 0-18.3% tuỳ model), adversarial example luôn gây lỗi cao hơn hẳn nhiễu ngẫu nhiên cùng biên độ.

4.3 Cross-training-set generalization (Bảng 3-4)

Chia MNIST train thành P_1, P_2 (30000 ảnh mỗi phần), train FC100-100-10 và FC123-456-10 trên P_1, FC100-100-10' trên P_2. Adversarial example sinh cho FC100-100-10 (distortion gốc, stddev ≈ 0.062) gây lỗi 26.2% trên FC123-456-10 (cùng tập train, kiến trúc khác) và 5.9% trên FC100-100-10' (tập train khác hẳn), so với baseline nhiễu Gaussian cùng stddev chỉ gây lỗi 2.2-2.6%. Sau khi khuếch đại distortion thêm 40% (stddev $0.062 \to 0.1$), lỗi cross-training-set tăng mạnh lên 22-98%.

4.4 Huấn luyện adversarial sơ bộ

Train lại FC 100-100-10 bằng cách duy trì một pool adversarial example, liên tục thay mới, trộn vào tập train: đạt test error $< 1.2\%$, so với 1.6% (chỉ weight decay) và 1.3% (dropout). Chi tiết quan trọng: chỉ có cải thiện khi sinh adversarial example **cho từng tầng output riêng** (không chỉ input) để train các tầng phía trên; adversarial example ở tầng cao có ích hơn tầng input/tầng thấp (theo quan sát ban đầu, chưa so sánh hệ thống).

5. Phân tích / ablation

- **Chặn trên Lipschitz thực nghiệm (Bảng 5, AlexNet)**: dao động 2.75 (Conv 1) đến 11 (Conv 5), cho thấy "bất ổn có thể xuất hiện ngay từ tầng conv đầu tiên". Các chặn trên này **nhất quán** với sự tồn tại thực nghiệm của blind spot ở Mục 4, nhưng bài tự nhận **không** giải thích được vì sao các adversarial example này tổng quát hoá qua hyperparameter/tập train khác nhau (đây là một câu hỏi mở, không phải điều phương pháp Lipschitz trả lời được).
- **So sánh adversarial so với nhiễu Gaussian**: ở mọi mức distortion thử nghiệm, adversarial example luôn gây lỗi cao hơn hẳn nhiễu ngẫu nhiên cùng chuẩn L_2, xác nhận đây không đơn thuần là hiệu ứng "nhiễu đủ lớn thì gây lỗi" mà là một **hướng cụ thể** trong không gian input.
- **Autoencoder (AE400-10) có vẻ bền hơn nhưng không miễn nhiễm**: trong Bảng 2, AE400-10 là cột có lỗi thấp nhất khi nhận adversarial example từ các model khác ($2.7\%, 9.8\%, 34.4\%, 2\%, 2.7\%$), nhưng vẫn có adversarial example riêng cho chính nó (100% lỗi khi tự tấn công, distortion 0.086) nên "không miễn nhiễm hoàn toàn" như bài tự nói.
- **Vai trò của weight decay cực lớn (FC10(1))**: dù train error đã rất cao (21.2%, mô hình bị "ép" đơn giản hoá mạnh), model này **vẫn** có adversarial example (distortion 0.14, lớn hơn các model khác nhưng không phải vô hạn), cho thấy regularization mạnh làm giảm nhưng không loại bỏ hiện tượng.

6. Giới hạn

Do tác giả tự nêu:

- Mục Discussion tự nhận: "chúng tôi không có hiểu biết sâu về việc adversarial negative xuất hiện thường xuyên đến mức nào, và vấn đề này cần được giải quyết trong nghiên cứu tương lai": tức lời giải thích "manifold xác suất thấp nhưng dày đặc (như số hữu tỉ)" ở cuối bài chỉ là **giả thuyết chưa kiểm chứng**, không phải kết luận thực nghiệm.
- Với MNIST, bài chỉ có kết quả cho model không-convolutional; với model convolutional chỉ có "quan sát định tính sơ bộ" trên AlexNet, chưa có bảng số liệu định lượng tương đương Bảng 1-2.
- Chặn trên Lipschitz chỉ là **chặn bảo toàn một chiều**: chặn lớn không suy ra tồn tại adversarial example, nên Bảng 5 không thể dùng để "chứng minh" sự tồn tại của blind spot, chỉ "nhất quán" với nó.

- Quan sát "adversarial example ở tầng cao hữu ích hơn tầng input" được tác giả tự gọi là "theo quan sát ban đầu" (initial observations) và nói rõ sẽ so sánh hệ thống hơn trong công việc tương lai, tức **chưa được kiểm chứng kỹ** tại thời điểm bài công bố.

Do người ghi chú phát hiện:

- **Số liệu không khớp hoàn toàn với câu chữ mô tả (Mục 4.2, ngay dưới Bảng 2)**: bài viết "Note that even the noise with stddev 0.1 is greater than the stddev of our adversarial noise for all but one of the models" (nhiễu Gaussian stddev 0.1 lớn hơn distortion trung bình của adversarial noise, trừ đúng một model). Nhưng đối chiếu với cột "Av. distortion" của chính Bảng 1/2: FC10(10^{-2}) có distortion $= 0.1$ (bằng, không nhỏ hơn 0.1) và FC10(1) có distortion $= 0.14$ (> 0.1). Như vậy có **hai** model, không phải một, mà nhiễu Gaussian stddev 0.1 không lớn hơn thực sự (một bằng, một nhỏ hơn). Đây là một điểm số liệu nhỏ không khớp hoàn toàn với câu mô tả bằng lời, cần cẩn trọng khi trích dẫn nguyên văn câu này.
- **Giọng văn Abstract mạnh hơn số liệu chi tiết ở Mục 4.3**: Abstract viết "the same perturbation can cause a different network, that was trained on a different subset of the dataset, to misclassify the same input", đọc tự nhiên như một phát hiện chắc chắn và mạnh. Nhưng Bảng 4 cho thấy ở độ lớn distortion gốc, lỗi cross-training-set chỉ 5.1-8.2% (so với baseline nhiễu ngẫu nhiên $\approx$ 2-2.6%): tức hiệu ứng **có thật nhưng khá yếu** ở độ lớn distortion ban đầu, chỉ trở nên mạnh (22-98%) sau khi tác giả **chủ động khuếch đại** distortion thêm 40%. Bài có nêu rõ chi tiết này trong thân bài ("their effectiveness decreases considerably"), nhưng Abstract không truyền tải mức độ suy giảm này; người đọc chỉ xem Abstract có thể đánh giá quá cao độ mạnh của hiệu ứng cross-training-set so với số liệu thực.
- Vài lỗi đánh máy nhỏ, không ảnh hưởng nội dung: "senstive" (lặp lại ở chú thích Hình 1 và Hình 3, đúng ra là "sensitive"), "Plase" (chú thích Hình 5, đúng ra là "Please").
- Phạm vi đã đọc: toàn bộ bài (588 dòng file text đầy đủ: Abstract, Mục 1 Introduction, Mục 2 Framework, Mục 3 Units of $\phi(x)$, Mục 4 Blind Spots gồm 4.1-4.3, Mục 5 Discussion, References): đọc hết, không bỏ qua phần nào vì bài khá ngắn (9 trang nội dung).

7. Liên hệ với các bài khác

Bài	Liên hệ
ml basics 23 (FGSM, 2014)	Giải thích lại chính hiện tượng bài này phát hiện: thay giả thuyết "manifold xác suất thấp, dày đặc" bằng giả thuyết tuyến tính đơn giản hơn; FGSM thay L-BFGS tốn kém bằng 1 bước gradient-sign, và chỉ ra adversarial example tồn tại trên **vùng liên tục rộng** chứ không phải "túi" rải rác như ngụ ý ở đây.
ml basics 20 (Shortcut Learning)	Liệt kê adversarial example như ví dụ cực đoan của shortcut learning; bài này (2013) là nguồn gốc lịch sử của chính ví dụ đó.
ml basics 01 (Rethinking Generalization)	Cùng chủ đề "deep network có thể hành xử phản trực giác so với kỳ vọng về generalization": bài 01 cho thấy network fit được nhãn ngẫu nhiên hoàn toàn, bài này cho thấy network "gián đoạn" ngay gần dữ liệu train thật; cả hai cùng thách thức niềm tin ngây thơ rằng train/test accuracy cao đồng nghĩa với việc học "đúng khái niệm".
ml basics 06 (Dropout)	Mục 4.2 so sánh trực tiếp lợi ích của adversarial training pool ($< 1.2\%$ lỗi) với dropout (1.3%) và chỉ weight decay (1.6%) trên cùng kiến trúc FC100-100-10, cùng hướng so sánh "regularization nào tốt hơn" mà FGSM (ml basics 23) cũng lặp lại sau đó.

8. Gợi ý áp dụng

Các ví dụ đối kháng cho thấy dự đoán có thể thay đổi mạnh trước nhiễu nhỏ theo một chuẩn được chọn. Đánh giá có ý nghĩa cần nêu rõ chuẩn, ngân sách nhiễu và phương pháp tấn công.

9. Thí nghiệm gợi ý

1. **Tái lập thí nghiệm "unit tự nhiên so với hướng ngẫu nhiên" trên một OCR encoder nhỏ.** Lấy activation tầng cuối của một backbone nhỏ (ví dụ CNN nhỏ cho ký tự), tìm ảnh cực đại hoá theo một unit cụ thể so với theo một hướng ngẫu nhiên chuẩn hoá $v/\|v\|$; so sánh định tính xem nhóm ảnh thu được có cùng chia sẻ đặc điểm (nét chữ, font) hay không. Diễn giải: kiểm tra xem kết luận "ngữ nghĩa nằm trong không gian, không riêng unit" của bài có còn đúng cho OCR encoder nhỏ.
2. **So sánh L-BFGS minimum-distortion so với FGSM một bước trên cùng classifier nhỏ.** Dùng cùng model đã dùng ở ml basics 23 (gợi ý thí nghiệm 1 của bài đó), sinh adversarial example bằng cả hai phương pháp, đo chuẩn L_2 của nhiễu cần thiết để đổi nhãn. Kỳ vọng: L-BFGS cho nhiễu nhỏ hơn (vì tối ưu đúng hướng tối thiểu) nhưng chậm hơn nhiều; FGSM nhanh nhưng nhiễu lớn hơn. Diễn giải: định lượng đánh đổi tốc độ/chất lượng giữa 2 họ phương pháp sinh adversarial example, liên hệ trực tiếp tới việc chọn phương pháp kiểm tra robustness khả thi trên phần cứng thử nghiệm.
3. **Tự tính chặn trên Lipschitz cho một tầng linear nhỏ, trước và sau khi prune.** Giả sử một tầng linear 4×4 có singular value lớn nhất (chuẩn operator) $\|W\| = 2.3$ trước khi prune; sau khi prune 50% trọng số nhỏ nhất và fine-tune lại, giả sử $\|W'\| = 3.1$ (tự tính, số liệu minh hoạ). Chặn trên Lipschitz của riêng tầng này tăng từ 2.3 lên 3.1, tức tăng $\frac{3.1-2.3}{2.3} \approx 34.8\%$ (tự tính): một dấu hiệu định lượng cụ thể cho thấy pruning có thể làm tầng đó nhạy cảm hơn với nhiễu input.
4. **Cross-model generalization giữa 2 kiến trúc OCR nhỏ khác kích thước ẩn.** Train 2 classifier nhỏ cùng nhiệm vụ nhận diện ký tự nhưng khác số hidden unit (giống cặp FC100-100-10 so với FC123-456-10 trong Bảng 3-4), sinh adversarial example cho model A, đo tỉ lệ gây lỗi trên model B. Diễn giải: ước lượng mức độ "blind spot" có tính universal qua kiến trúc trong bài toán OCR cụ thể, không chỉ trên MNIST như bài gốc.
5. **Cross-training-set split cho dữ liệu OCR tổng hợp.** Chia một tập dữ liệu ký tự/trang tổng hợp nhỏ thành 2 nửa ngẫu nhiên, train 2 model cùng kiến trúc trên từng nửa, kiểm tra tỉ lệ transfer của adversarial example giữa chúng so với baseline nhiễu Gaussian cùng biên độ. Kỳ vọng: nếu lặp lại mẫu hình Bảng 4 (transfer thấp hơn hẳn trong-model nhưng cao hơn baseline ngẫu nhiên), đó là bằng chứng cụ thể cho bài toán OCR rằng blind spot phần nào phụ thuộc dữ liệu train, không hoàn toàn universal.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Tìm hướng đại diện ngữ nghĩa	$x_0 = \arg\max_{x \in I} \langle \phi(x), v \rangle$, v = unit tự nhiên hoặc hướng ngẫu nhiên
Bài toán adversarial gốc	minimize $\|r\|_2$ với $f(x+r) = l$, $x + r \in [0,1]^m$
Xấp xỉ bằng L-BFGS	minimize $c\|r\| + \text{loss}_f(x+r, l)$ với $x + r \in [0,1]^m$, line-search c
Hằng số Lipschitz từng tầng	$\|\phi_k(x) - \phi_k(x+r)\| \leq L_k \|r\|$, tầng ReLU: $L_k \leq \|W_k\|$
Chặn trên toàn network	$L = \prod_{k=1}^{K} L_k$
Kết quả tiêu biểu	Distortion trung bình để đạt 0% accuracy: 0.058-0.14 (thang $[0,1]$, $n = 784$)
Kết quả adversarial training sơ bộ	Test error: 1.6% (weight decay) $\rightarrow$ 1.3% (dropout) $\rightarrow <$ 1.2% (pool adversarial)

11. Kết luận ngắn

Bài này là nguồn gốc của khái niệm adversarial example trong deep learning hiện đại, đưa ra hai phát hiện: ngữ nghĩa phân tán trong toàn không gian activation (không riêng từng unit), và sự tồn tại của nhiễu nhỏ, có hướng, tổng quát hoá qua model/tập train khác nhau, khiến network "gián đoạn" ở gần dữ liệu train thật. Phương pháp sinh adversarial example (L-BFGS có ràng buộc hộp) tốn kém nhưng cho nhiễu nhỏ và chính xác hơn; phân tích Lipschitz cho một cách rẻ để chặn trên độ bất ổn mà không cần sinh adversarial example thật. Bài tự nhận chưa hiểu sâu cơ chế vì sao hiện tượng này tổng quát hoá, để ngỏ cho các công trình sau (trong đó có ml basics 23) giải thích lại bằng góc nhìn đơn giản hơn: tính tuyến tính.

Nguồn gốc: https://arxiv.org/abs/1312.6199

Bài 25

Deep Learning in Neural Networks: An Overview (2014)

Tác giả: Jürgen Schmidhuber (The Swiss AI Lab IDSIA, Istituto Dalle Molle di Studi sull'Intelligenza Artificiale, University of Lugano & SUPSI).
Venue / arXiv: arXiv:1404.7828 (v4, 08/10/2014); paper không nêu venue hội nghị/tạp chí, chỉ ghi "Technical Report IDSIA-03-14" và tự gọi là "an invited Deep Learning (DL) overview" (Preface).
Nguồn: https://arxiv.org/abs/1404.7828
Chủ đề: tổng quan lịch sử Deep Learning (DL) trong neural network (NN) cho Supervised Learning (SL), Unsupervised Learning (UL) và Reinforcement Learning (RL); dùng khái niệm tự định nghĩa "Credit Assignment Path" (CAP) để phân biệt học "sâu" (deep) và học "nông" (shallow).
Vị trí trong lộ trình: Bài tổng quan đặt các chủ đề trong lộ trình vào lịch sử phát triển của deep learning.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận. Bài tổng quan dài 88 trang với 888 tài liệu tham khảo; ghi chú tóm tắt các mục con của phần 5 và không liệt kê toàn bộ tài liệu tham khảo.

1. Bối cảnh

Năm 2014, các deep NN (kể cả recurrent) đã thắng nhiều cuộc thi pattern recognition. Câu hỏi nền tảng mà Schmidhuber đặt ra ngay từ câu đầu bài (trích Minsky 1963): "thành phần nào của một hệ thống học có thể thay đổi được chịu trách nhiệm cho thành công hay thất bại của nó? Thay đổi gì ở chúng sẽ cải thiện hiệu năng?" - đây gọi là **bài toán credit assignment** (gán công/gán lỗi).

Một NN là tập các "neuron" đơn giản được kết nối, mỗi neuron phát ra một chuỗi giá trị kích hoạt (activation) thực. Input neuron được kích hoạt bởi cảm biến từ môi trường; neuron khác được kích hoạt qua kết nối có trọng số từ các neuron đã kích hoạt trước đó. Một số neuron có thể ảnh hưởng tới môi trường qua hành động. Học, hay gán công, là tìm trọng số làm NN thể hiện hành vi mong muốn.

Mô hình "nông" (ít tầng tính toán nối tiếp) đã tồn tại hàng thập kỷ, thậm chí hàng thế kỷ (hồi quy tuyến tính từ đầu thế kỷ 19). Mô hình nhiều tầng phi tuyến liên tiếp có từ ít nhất thập niên 1960-1970. Backpropagation (BP, giải thích chi tiết ở Mục 3) ra đời về mặt lý thuyết thập niên 1960-1970, áp dụng cho NN năm 1981, nhưng huấn luyện NN nhiều tầng bằng BP bị phát hiện là khó trong thực tế vào cuối thập niên 1980, và trở thành đề tài nghiên cứu công khai từ đầu thập niên 1990. DL trở nên khả thi một phần nhờ Unsupervised Learning (UL) hỗ trợ (1991, 2006). Từ 2009, deep NN có giám sát (supervised) đã thắng nhiều cuộc thi quốc tế chính thức, đạt kết quả "siêu nhân" (superhuman) trong một số miền hạn chế (2011). DL cũng trở nên quan trọng cho Reinforcement Learning (RL), nơi không có giáo viên (teacher) giám sát trực tiếp.

Paper khẳng định cả FNN (feedforward, không có chu trình) và RNN (recurrent, có chu trình) đều đã thắng các cuộc thi; theo một nghĩa nào đó RNN là "sâu nhất" trong mọi NN vì chúng là máy tính tổng quát (general computer) mạnh hơn FNN, có thể tạo và xử lý bộ nhớ của chuỗi input dài tuỳ ý.

2. Ý tưởng chính

Ý tưởng tổ chức xuyên suốt bài là khái niệm **Credit Assignment Path (CAP)**: chuỗi liên kết nhân quả (có thể học được) từ input đến output (ở FNN) hoặc qua các biến đổi theo thời gian (ở RNN). Độ sâu (depth) của một CAP đo khoảng cách mà việc gán công có thể "lùi lại" qua chuỗi nhân quả để tìm trọng số thay đổi được. Tác giả **tự định nghĩa** ngưỡng: bài toán có độ sâu CAP > 10 cần "Very Deep Learning" (không có lý do lý thuyết chặt cho số 10 này, chỉ là quy ước thực dụng của tác giả, thừa nhận "các chuyên gia DL chưa đạt đồng thuận").

Dựa trên khung CAP này, paper tổ chức toàn bộ lịch sử NN (từ 1800s đến 2014) theo dòng thời gian, chỉ ra 5 "chủ đề lặp lại" (recurring themes, Mục 4): Dynamic Programming (DP) hỗ trợ gán công; UL hỗ trợ SL và RL; học biểu diễn theo tầng bậc (hierarchical representation); nguyên lý Occam's razor/Minimum Description Length (MDL) cho việc tìm NN đơn giản, ít overfit; và vai trò của GPU trong việc làm DL khả thi về mặt tính toán. Phần thân chính (Mục 5, SL/UL; Mục 6, RL) kể lại câu chuyện lịch sử theo các chủ đề này, từ GMDH (1965), Neocognitron (1979), qua vấn đề vanishing/exploding gradient (1991), LSTM (1995), Deep Belief Network (2006), GPU-CNN (2006-2011), đến các cuộc thi ImageNet/object detection/segmentation (2012-2013) và các "mẹo" huấn luyện hiện đại như dropout, ReLU, maxout (2012-2014).

3. Phương pháp chi tiết

Phạm vi đọc: toàn bộ Mục 2, 3, 4 (khung khái niệm); Mục 5.1 đến 5.26 (lịch sử SL/UL, tóm ý chính mỗi mục con); Mục 6.1 đến 6.8 (RL); Mục 7 (kết luận). Không liệt kê lại toàn bộ danh sách trích dẫn trong mỗi mục con.

3.1 Notation sự kiện (Mục 2) và CAP (Mục 3)

Paper dùng một notation "event-oriented" rất tổng quát: trong một episode, có một chuỗi giá trị thực x_t ($t = 1, \ldots, T$) gọi là **event** (sự kiện). Mỗi x_t là input do môi trường đặt, hoặc là kích hoạt của một unit phụ thuộc vào các x_k ($k < t$) khác qua hàm f_t:

$$x_t = f_t(\text{net}_t), \qquad \text{net}_t = \sum_{k \in in_t} x_k \, w_{v(k,t)} \text{ (trường hợp cộng)}$$

với in_t là tập chỉ số các kết nối nhân quả đi vào t, w là trọng số, v là hàm map cặp (k, t) sang chỉ số trọng số. Một số NN hiện đại dùng $x_t = \max_{k \in in_t}(x_k)$ (chính là max-pooling, Mục 3.3 bên dưới).

Hai event x_p, x_q ($p < q$) có **Potential Direct Causal Connection (PDCC)** nếu $p \in in_q$. Khi đó (p, q) là một CAP (tối giản) từ p đến q. **Potential Causal Connection (PCC)** tổng quát hơn, định nghĩa đệ quy: đúng nếu có PDCC, hoặc nếu có PCC từ p đến k và PDCC từ k đến q. Nếu một CAP có dạng $(\ldots, k, t, \ldots, q)$ với $w_{v(k,t)}$ là trọng số **thay đổi được** gần input nhất trên chuỗi, thì **độ sâu (depth)** của CAP là độ dài của đoạn hậu tố $(t, \ldots, q)$.

Ví dụ tự tính: xét một FNN đơn giản, kết nối đầy đủ, không có shortcut, gồm 1 tầng input và 3 tầng không-input (hidden1, hidden2, output). Theo định nghĩa của paper ("SL FNNs with fixed topology imply a problem-independent maximal problem depth bounded by the number of non-input layers"), CAP dài nhất đi input $\rightarrow$ hidden1 $\rightarrow$ hidden2 $\rightarrow$ output có độ sâu bằng số tầng không-input, tức **độ sâu tối đa = 3** (tự tính, bằng đúng số tầng không-input). Với ngưỡng "problem depth > 10 cần Very Deep Learning" mà paper tự đặt, một FNN cố định topology muốn được gọi là "Very Deep" theo cách tính thuần số tầng này cần **ít nhất 11 tầng không-input** (tự tính: 3 tầng ở ví dụ trên còn cách ngưỡng 10 khá xa).

3.2 Các chủ đề lặp lại (Mục 4)

- **DP (4.1):** BP có thể xem là một phương pháp suy ra từ Dynamic Programming (Bellman 1957); trong RL truyền thống dựa giả định Markov, DP giúp giảm mạnh độ sâu CAP (Mục 6.2).
- **UL hỗ trợ SL/RL (4.2):** UL mã hoá dữ liệu thô (video, âm thanh) thành dạng gọn hơn, ít dư thừa hơn, giúp không gian tìm kiếm của SL/RL nhỏ lại (CAP nông hơn).
- **Học biểu diễn theo tầng bậc (4.3):** nhiều phương pháp AI cũ (GOFAI) và DL hiện đại đều học hệ thống biểu diễn ngày càng trừu tượng; đây là hệ quả tự nhiên của việc nén dữ liệu.
- **Occam's razor / MDL (4.4):** MDL đo độ phức tạp của một giải pháp bằng độ dài chương trình ngắn nhất tính ra nó; weight decay, optimal brain damage/surgeon, Flat Minimum Search (FMS) đều là các cách tìm NN "đơn giản" (ít bit mô tả) để giảm overfitting.

- **GPU (4.5):** GPU giá rẻ, nhiều lõi, tăng tốc tính ma trận/vector tới trên 50 lần so với CPU, là yếu tố quyết định cho các kỷ lục 2009-2014.

3.3 Dòng thời gian Mục 5 (SL, một phần UL)

Năm	Mốc	Ý chính
1965	GMDH	Lưới nhiều tầng đầu tiên kiểu feedforward multilayer perceptron (theo paper, "có thể là DL system đầu tiên"); unit có hàm kích hoạt đa thức Kolmogorov-Gabor; tầng được thêm dần và cắt bớt bằng validation set (ý tưởng pruning sớm). Một mạng GMDH 8 tầng đã có từ 1971.
1979	Neocognitron (Fukushima)	"Có thể là NN đầu tiên xứng đáng gọi là deep"; giới thiệu CNN: filter trượt trên mảng 2D, cộng lớp subsampling; lấy cảm hứng neurophysiology (simple/complex cell ở vỏ não thị giác, Hubel & Wiesel ~1960). Không dùng BP, dùng rule học cục bộ WTA hoặc pre-wiring.
1960-1981	Lịch sử BP	Giảm gradient trong hệ nhiều tầng khả vi được thảo luận từ đầu thập niên 1960 (Kelley, Bryson, Pontryagin...) trong khung Calculus of Variations; BP tổng quát, hiệu quả đầu tiên mô tả trong luận văn thạc sĩ 1970 (Linnainmaa, không gắn với NN); áp dụng cho NN lần đầu năm 1981 (Werbos); bài 1986 (Rumelhart et al.) phổ biến hoá BP cho NN, cho thấy biểu diễn ẩn hữu ích xuất hiện tự nhiên.
Cuối 1980s-2000	Cải tiến BP	Vấn đề: thêm tầng ẩn thường không giúp gì về thực nghiệm; nhiều người an tâm với định lý "1 tầng ẩn đủ rộng xấp xỉ được mọi hàm liên tục" (Kolmogorov/Hornik). Các cải tiến: momentum, R-prop (chỉ dùng dấu đạo hàm), quasi-Newton/BFGS (quá đắt cho NN lớn), weight decay (suy ra từ Bayes với prior Gauss/Laplace), Flat Minimum Search (tìm vùng cực tiểu "phẳng", ít bit mô tả trọng số), optimal brain damage và optimal brain surgeon (cắt trọng số).
1987	Autoencoder (AE) hierarchies	Xếp tầng các AE huấn luyện UL (mã hoá input thành chính nó qua bottleneck hẹp); tiền thân của DBN/AE-stack 2006.
1989	BP cho CNN	BP áp vào Neocognitron-like CNN có trọng số thích ứng; giới thiệu bộ MNIST (sau này là benchmark nổi tiếng nhất của ML); CAP depth 5.
1991	Vấn đề nền tảng của DL	Luận văn Hochreiter (1991) chứng minh: sai số lan truyền ngược tích lũy co lại hoặc nổ ra theo cấp số nhân theo số tầng/độ sâu CAP (vanishing/exploding gradient).
1991	History Compressor	Ngăn xếp (stack) RNN huấn luyện UL để dự đoán input kế tiếp; chỉ input "bất ngờ" (lỗi dự đoán) được đưa lên RNN cấp cao hơn, chạy ở nhịp thời gian chậm hơn - nén dữ liệu theo không gian và thời gian; gán công qua hàng trăm tầng/toán tử phi tuyến.

Năm	Mốc	Ý chính
1992	Max-Pooling (MP)	Cresceptron dùng MP (chọn unit kích hoạt cao nhất trong một vùng nhỏ) thay cho các cơ chế WTA/subsampling khác; tiền đề cho MPCNN hiện đại.
1994	Cuộc thi đầu tiên	NN với delay line thắng cuộc thi Santa Fe time-series (laser hỗn loạn); CAP không sâu.
1995	LSTM	Giải thích ở Mục 3.4 dưới.
2006	Deep Belief Network (DBN)	Xếp tầng Restricted Boltzmann Machine (RBM), huấn luyện UL từng tầng rồi fine-tune bằng BP; thuật ngữ "Deep Learning" được đặt ra khoảng thời điểm này.
2006-2007	GPU-CNN, BP cho MPCNN	CNN chạy GPU đầu tiên (nhanh gấp 4 lần CPU); BP lần đầu áp cho MPCNN (xen lớp convolution và max-pooling); ngăn xếp LSTM huấn luyện bằng Connectionist Temporal Classification (CTC).
2009	Thắng cuộc thi chính thức đầu tiên	Ngăn xếp LSTM + CTC thắng 3 cuộc thi nhận dạng chữ viết tay ICDAR (Pháp, Ả Rập, Farsi) không cần kiến thức ngôn ngữ; CNN 3D thắng 3 cuộc thi TRECVID.
2010	Kỷ lục MNIST bằng BP thuần	BP + biến dạng dữ liệu (distortion) trên GPU đạt 0.35% lỗi, không cần UL pre-training, không cần convolution; chỉ nhờ GPU nhanh hơn CPU tới 50 lần.
2011	MPCNN trên GPU: hiệu năng siêu nhân	Ensemble GPU-MPCNN thắng cuộc thi nhận dạng biển báo giao thông IJCNN 2011.
2011	Hessian-free cho RNN	Tối ưu Hessian-free giúp giảm vấn đề vanishing gradient cho RNN, vượt LSTM chuẩn trên một số task.
2012	ImageNet, object detection, segmentation	Ensemble GPU-MPCNN đạt kết quả tốt nhất trên benchmark ImageNet (Krizhevsky et al. 2012); NN lớn nhất thời điểm đó (10^9 tham số) huấn luyện UL trên 16.000 lõi CPU; GPU-MPCNN thắng cuộc thi phát hiện u (mitosis) và phân đoạn ảnh thần kinh.
2013-	Thêm nhiều kỷ lục	LSTM+CTC phá kỷ lục TIMIT phoneme; LSTM vượt HMM trong keyword spotting, nhận dạng chữ Hán, dịch máy (rerank câu dịch Anh-Pháp).
2014 (hiện tại của paper)	Các "mẹo" gần đây	Dropout, ReLU, maxout, RmsProp - giải thích ở Mục 3.5.

3.4 LSTM (Mục 5.13) và vanishing gradient (Mục 5.9)

Ý tưởng cốt lõi: một số unit gọi là **Constant Error Carousel (CEC)** dùng hàm kích hoạt đồng nhất $f(x) = x$ và có kết nối tự hồi với trọng số cố định 1.0. Vì $f' = 1.0$ không đổi, sai số lan truyền ngược qua một CEC **không co lại cũng không nổ ra**, giữ nguyên giá trị (trừ khi "chảy ra" khỏi CEC tới các phần thích ứng khác của NN). CEC kết nối tới các unit phi tuyến thích ứng khác (một số có hàm kích hoạt nhân) để học hành vi phi tuyến.

Minh hoạ tự tính vấn đề vanishing gradient mà CEC giải quyết: với một RNN thường dùng hàm kích hoạt sigmoid, đạo hàm cực đại của sigmoid là 0.25 (tại điểm giữa). Nếu đạo hàm cục bộ trung bình mỗi bước khoảng 0.25 và tín hiệu lỗi phải lan truyền qua CAP depth $d = 10$ bước, hệ số suy giảm tích lũy là $0.25^{10} \approx 9.5 \times 10^{-7}$ (tự tính), tức tín hiệu học hầu như biến mất. Ngược lại nếu dùng tanh với đạo hàm

trung bình ≈ 0.9 qua 20 bước thì hệ số còn lại là $0.9^{20} \approx 0.12$ (tự tính), vẫn suy giảm đáng kể dù chậm hơn sigmoid. Với CEC ($f' = 1$), hệ số sau d bước là $1^d = 1$ (tự tính), không suy giảm. Đây chính xác là cơ chế paper mô tả ở Mục 5.9/5.13 (không phải số liệu trong paper, chỉ là minh hoạ bằng số của người ghi chú dựa trên công thức đạo hàm sigmoid/tanh/identity).

3.5 RL (Mục 6) và các "mẹo" SL gần đây (Mục 5.24)

- **RL qua world model (6.1):** một FNN/RNN M học dự đoán input tiếp theo của một controller C (world model); khi M đủ chính xác, M và C ghép thành một RNN, BP chuẩn (Mục 3.1) lan truyền gradient từ reward mong muốn qua M xuống C (trọng số M giữ cố định).
- **RL truyền thống/MDP (6.2):** giả định Markov cho phép định nghĩa value function đệ quy $V(x_t) = r_t + V(x_k)$ với x_k là event input kế tiếp, giảm mạnh độ sâu CAP cần thiết; ví dụ nêu: Atari 2600 chơi trực tiếp từ pixel 84x84 bằng GPU-CNN + experience replay (chính là DQN, Mnih et al. 2013).
- **POMDP (6.3):** không có giả định Markov, dùng RNN để nén lịch sử quan sát thành state feature.
- **HRL (6.5), Direct search/policy gradient/evolution (6.6), Indirect/compressed search (6.7), Universal RL/Gödel Machine (6.8):** các hướng mở rộng khi không có gradient tốt hoặc cần tìm kiếm trong không gian NN nén (ví dụ mã hoá trọng số RNN bằng hệ số DCT).
- **Dropout** (Hinton et al. 2012b): loại bỏ ngẫu nhiên một số unit khi huấn luyện để cải thiện tổng quát hoá; có thể xem là ensemble ngầm.
- **ReLU:** $f(x) = x$ nếu $x > 0$, $f(x) = 0$ nếu ngược lại; vượt sigmoid trên nhiều benchmark.
- **Maxout:** kết hợp unit cạnh tranh tuyến tính và dropout.
- **RmsProp:** tăng tốc gradient descent bậc một, họ với vario-η, Adagrad, Adadelta.

4. Thí nghiệm và kết quả chính

Đây là một **survey**, không có thí nghiệm riêng của tác giả; mọi số liệu dưới đây là kết quả tác giả **trích dẫn từ các paper khác** mà ông tổng hợp theo dòng thời gian (paper không tự chạy benchmark). Các con số cụ thể nêu trong văn bản:

Benchmark	Năm / phương pháp	Kết quả
MNIST	1990 (CNN đầu tiên, BP)	CAP depth 5, không ghi % lỗi cụ thể
MNIST	2003 (CNN + distortion, Simard et al.)	0.4% lỗi (so với 0.7% của BP net chuẩn)
MNIST	2006 (CNN + distortion, Ranzato et al.)	0.39% lỗi
MNIST	2006 (DBN fine-tune BP, không distortion)	1.2% lỗi
MNIST	2010 (BP thuần + distortion trên GPU)	0.35% lỗi (kỷ lục mới); 0.95% nếu chỉ có dịch chuyển nhỏ kiểu saccade, không distortion khác
MNIST	2012 (ensemble GPU-MPCNN)	khoảng 0.2% lỗi, lần đầu đạt mức người (human-competitive)
TIMIT phoneme	2010 (DBN, Mohamed & Hinton)	26.7% lỗi (test core set)
TIMIT phoneme	2013 (bi-directional LSTM stack + CTC)	17.7% lỗi, phá kỷ lục sau hàng nghìn năm-người nghiên cứu HMM
IJCNN 2011 traffic sign (chung kết)	Ensemble GPU-MPCNN	0.56% lỗi: gấp 2 lần tốt hơn người, gấp 3 lần tốt hơn đối thủ NN gần nhất, gấp 6 lần tốt hơn phương pháp phi-neural tốt nhất
IJCNN 2011 traffic sign (vòng loại online)	Ensemble GPU-MPCNN vs đối thủ	1.02% vs 1.03% (đối thủ); lỗi của người trên test set: 1.19% (công bố sau)

Benchmark	Năm / phương pháp	Kết quả
ImageNet classification 2012	Ensemble GPU-MPCNN (Krizhevsky et al.)	kết quả tốt nhất thời điểm đó (không có số % cụ thể trong văn bản paper)

Các "chiến thắng" khác được nêu định tính (không kèm số liệu phần trăm trong văn bản): ICPR 2012 Mitosis Detection, ISBI 2012 Segmentation of Neuronal Structures, MICCAI 2013 Grand Challenge on Mitosis Detection, CASP 2012 protein contact map, các cuộc thi TRECVID 2009, ICDAR 2009 (3 ngôn ngữ), dịch máy Anh-Pháp (rerank hypothesis bằng LSTM lớn), nhận dạng chữ Hán ICDAR.

5. Phân tích / ablation

Paper không có ablation theo nghĩa thực nghiệm (không phải paper phương pháp). Các điểm phân tích/so sánh mà tác giả tự rút ra xuyên suốt lịch sử:

- **Phần cứng vs thuật toán (Mục 5.18):** tác giả nhận xét rằng vì BP đã "3-5 thập kỷ tuổi" và biến dạng dữ liệu (pattern distortion) đã "2 thập kỷ tuổi" khi kỷ lục MNIST 2010 được lập chỉ bằng GPU nhanh hơn CPU 50 lần (không thuật toán mới), điều này "gợi ý rằng tiến bộ trong khai thác phần cứng tính toán hiện đại quan trọng hơn tiến bộ thuật toán" cho trường hợp cụ thể này.
- **Vấn đề overfitting lên test set công khai (Mục 5.19):** tác giả chỉ ra một vấn đề chung của benchmark có test set công khai hoặc có thể "probe" (dò): trong vòng loại IJCNN 2011, các đội liên tục cải thiện kết quả qua nhiều lần nộp lại (ban tổ chức sau đó phải giới hạn 10 lần nộp), cho thấy nguy cơ "ngầm overfit" lên test set dù không dùng trực tiếp để train.
- **Hai câu chuyện lịch sử song song (Mục 5.23):** tác giả nhận thấy một mẫu hình lặp lại: thập niên 1990 có xu hướng đi từ RNN stack bán-giám-sát (1991, History Compressor) sang LSTM thuần giám sát (1995); thập niên 2000 có xu hướng tương tự từ FNN stack bán-giám-sát (DBN 2006) sang MPCNN thuần giám sát (2006-2012). Tác giả tự nhận xét: dù vậy, kết hợp UL pre-training với SL fine-tuning vẫn có thể có lợi trong một số ứng dụng.
- **Ngôn ngữ dè dặt lặp lại nhiều lần:** các cụm như "perhaps the first", "to my knowledge", "arguably" xuất hiện liên tục khi gán nhãn "đầu tiên" cho GMDH (1965, "perhaps the first DL system"), Neocognitron (1979, "perhaps the first NN that deserved the attribute deep"), Linnainmaa (1970, "apparently was first described"). Đây là điểm tác giả tự thừa nhận trong Preface: có "selection bias" do chuyên môn và quen thuộc đặc biệt với công trình của nhóm nghiên cứu riêng của mình trong 1/4 thế kỷ, nên các mốc "đầu tiên" nên được đọc với mức độ dè dặt tương ứng, không phải chân lý tuyệt đối.
- **Kết luận tổng hợp (Mục 7):** tác giả tự so sánh 3 nhánh SL/UL/RL và chỉ ra common theme xuyên suốt là 5 chủ đề ở Mục 4 (DP, UL hỗ trợ, hierarchical representation, MDL/Occam, GPU); đồng thời nêu một quan sát mang tính phê bình: các deep RNN thành công nhất thời điểm viết bài "không giống não" theo 2 tiêu chí tiết kiệm năng lượng (chỉ một phần nhỏ neuron kích hoạt tại một thời điểm nhờ winner-take-all; kết nối thưa trong khối 3D với phần lớn kết nối ngắn) - các RNN SOTA lúc đó thường kích hoạt gần hết unit và kết nối dày đặc.

6. Giới hạn

Do tác giả tự nêu:

- Preface nói rõ: "bài viết này nên được xem như một snapshot của một quá trình gán công đang diễn ra" (not final/exhaustive); có "selection bias chuyên gia" và "bias do quen thuộc đặc biệt với công trình của nhóm nghiên cứu riêng tác giả trong 1/4 thế kỷ qua".
- Ngưỡng "problem depth > 10 là Very Deep Learning" (Mục 3) được tác giả thừa nhận là tự đặt ra, không có đồng thuận chuyên gia, chỉ "để phục vụ mục đích của overview này".

- Mục 6 (RL) được tác giả nói thẳng là ngắn hơn Mục 5 (SL/UL) "phản ánh quy mô hiện tại của từng lĩnh vực", và "các mục con này không thể đóng vai trò một survey RL tổng quát".
- Universal RL/Gödel Machine (6.8) được tác giả tự nhận là "vượt quá cái thường gọi là DL", chỉ nhắc qua.

Người ghi chú nhận thấy:

- Đây là paper tổng quan lịch sử tới tháng 10/2014; nó hoàn toàn không thể và không có ý định đề cập tới các bước ngoặt định hình DL hiện đại xảy ra **sau** thời điểm viết (ResNet 2015, Batch Normalization 2015, Transformer/attention 2017, pre-training quy mô lớn...). Đây không phải lỗi của paper, nhưng người đọc hôm nay cần hiểu bài này là ảnh chụp lịch sử tới 2014, không phải bản đồ đầy đủ của DL hiện tại.
- Nhiều số liệu benchmark được trích theo kiểu liệt kê rời rạc (một câu, một con số, không có bảng thống nhất đơn vị đo hay protocol đánh giá xuyên suốt paper); việc so sánh định lượng giữa các mốc thời gian khác nhau (ví dụ MNIST 1990 vs 2012) khó chuẩn hoá nếu chỉ dựa trên văn bản paper.
- Paper không phân biệt rõ ràng "shallow learning kết thúc, deep learning bắt đầu" ở đâu (tự thừa nhận ở Mục 3), nên khung CAP tuy hữu ích về mặt khái niệm nhưng không đưa ra một quy tắc định lượng chắc chắn để phân loại một hệ thống cụ thể là "sâu" hay "nông" ngoài ngưỡng tuỳ ý 10.

7. Liên hệ với các bài khác

- **ml basics 06** (Dropout): đặt kỹ thuật dropout trong bối cảnh các phương pháp giảm overfitting được khảo sát ở đây.
- **ml basics 07** (ResNet 2015, Deep Residual Learning): paper này **không thể** nhắc tới ResNet vì ra đời sau (2015), nhưng là bước tiếp theo tự nhiên của dòng "CNN ngày càng sâu" mà Mục 5.4, 5.8, 5.16, 5.19, 5.21 kể lại; đọc ResNet sau bài này để thấy vấn đề vanishing gradient (Mục 5.9 ở đây) tiếp tục được giải quyết bằng skip connection, một hướng khác với CEC của LSTM.
- **rl content/01_2013_dqn.md** (DQN, Mnih et al. 2013): được trích dẫn trực tiếp ở Mục 6.2 của bài này ("a rather deep GPU-CNN was used in a traditional RL framework to play several Atari 2600 computer games directly from 84x84 pixel ... video input (Mnih et al., 2013), using experience replay"). Đây là một trong rất ít kết quả RL "hiện đại" (tại thời điểm 2014) mà Schmidhuber trích dẫn, cho thấy khung MDP/DP (Mục 6.2) vẫn là nền cho DQN.
- **pruning content/01_1989_1993_optimal_brain_damage_surgeon.md** (Optimal Brain Damage/Surgeon): được trích dẫn trực tiếp ở Mục 5.6.3 trong danh sách các phương pháp "tìm NN đơn giản, ít overfit" ("weight pruning, e.g., optimal brain damage (LeCun et al., 1990b), and optimal brain surgeon (Hassibi and Stork, 1993)"). Cho thấy pruning không phải ý tưởng mới của kỷ nguyên LLM, mà đã được liệt vào "recurring theme" Occam's razor/MDL (Mục 4.4) từ rất sớm.
- **ml basics 26** (Representation Learning: A Review and New Perspectives, Bengio/Courville/Vincent 2012): một survey gần cùng thời điểm (2012-2013), tập trung hẹp hơn vào UL/representation learning; nên đọc nối tiếp bài này để có góc nhìn thứ hai, ít theo kiểu "biên niên sử" hơn, về cùng giai đoạn lịch sử DL.

8. Gợi ý áp dụng

Bài tổng quan hữu ích để đặt các kiến trúc và thuật toán vào lịch sử phát triển của deep learning. Khi dùng nó làm điểm xuất phát, cần phân biệt các kết quả lịch sử với tình hình nghiên cứu ở thời điểm hiện tại.

9. Thí nghiệm gợi ý

Tất cả chạy được bằng PyTorch, dùng mô hình toy (không cần mô hình lớn), nhằm minh hoạ trực tiếp các claim lịch sử của paper:

1. **Tái hiện vanishing gradient (Mục 5.9):** huấn luyện một MLP toy 15-20 tầng (vài trăm unit mỗi tầng) trên một bài phân loại tổng hợp đơn giản (ví dụ 2 class, dữ liệu sinh ngẫu nhiên có thể phân tách tuyến tính được một phần), hai phiên bản: kích hoạt sigmoid và kích hoạt ReLU. Đo norm của gradient tại mỗi tầng sau vài bước huấn luyện đầu tiên, vẽ theo chỉ số tầng (log scale). Kỳ vọng: gradient ở tầng gần input của mạng sigmoid nhỏ hơn nhiều bậc độ lớn so với tầng gần output, khớp với mô tả "decay exponentially in the number of layers" (Mục 5.9); mạng ReLU suy giảm ít hơn. Diễn giải: nếu không thấy sự khác biệt, kiểm tra lại khởi tạo trọng số (cần đủ "xấu" để hiện tượng rõ, ví dụ std nhỏ hoặc lớn bất thường).
2. **So sánh CEC-như-LSTM vs vanilla RNN trên task nhớ dài (Mục 5.13):** tạo task toy "nhớ bit đầu tiên của chuỗi, trả lời ở bước T" với $T \in \{10, 50, 100, 300\}$. Huấn luyện một vanilla RNN (tanh) và một LSTM nhỏ (1 tầng, vài chục hidden unit) trên mỗi T, đo tỉ lệ thành công. Kỳ vọng: vanilla RNN thành công tốt ở T nhỏ nhưng sụp nhanh khi T tăng; LSTM giữ được thành công ở T lớn hơn nhiều, minh hoạ trực tiếp cơ chế CEC (Mục 3.4).
3. **Dropout và generalization gap (Mục 5.24):** huấn luyện một MLP nhỏ (2-3 tầng ẩn) trên một tập dữ liệu nhỏ (ví dụ vài nghìn mẫu, dễ overfit), có và không có dropout ($p = 0.5$) ở tầng ẩn. Đo khoảng cách train accuracy - test accuracy qua các epoch. Kỳ vọng: không dropout, train accuracy tiến gần 100% còn test accuracy dừng sớm và khoảng cách nới rộng (overfit rõ); có dropout, train accuracy thấp hơn nhưng test accuracy cao hơn hoặc ổn định hơn, khoảng cách hẹp lại.
4. **Max-pooling vs average-pooling (Mục 5.11):** huấn luyện cùng một CNN nhỏ (2-3 tầng conv) trên MNIST (hoặc tập con MNIST để tiết kiệm thời gian), một bản dùng max-pooling, một bản dùng average-pooling, giữ mọi thứ khác giống nhau. So sánh test accuracy sau cùng số epoch. Kỳ vọng: max-pooling nhích hơn một chút hoặc hội tụ nhanh hơn, phản ánh lý do lịch sử MP trở thành lựa chọn chuẩn (Mục 5.11, 5.19) dù khoảng cách trên MNIST hiện đại có thể rất nhỏ.
5. **ReLU vs sigmoid, tốc độ hội tụ (Mục 5.24):** cùng MLP 5-6 tầng, 2 bản kích hoạt khác nhau, cùng learning rate, đo số epoch cần để đạt một ngưỡng loss cố định trên tập train nhỏ. Kỳ vọng: bản ReLU hội tụ nhanh hơn rõ rệt (ít epoch hơn) nhờ không bị bão hoà đạo hàm ở vùng giá trị lớn, khớp với claim "ReLU outperformed sigmoidal activation functions in deep NNs" (Mục 5.24, Glorot et al. 2011).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn trong paper
Event, activation	$x_t = f_t(\text{net}_t)$, $\text{net}_t = \sum_{k \in in_t} x_k w_{v(k,t)}$	Mục 2
PDCC	$pdcc(p, q)$ đúng khi $p \in in_q$	Mục 3
CAP, depth	chuỗi $(\ldots, k, t, \ldots, q)$; depth = độ dài hậu tố từ trọng số thay đổi được cuối cùng tới q	Mục 3
Very Deep Learning	problem depth > 10 (ngưỡng tự đặt)	Mục 3
BP (gradient qua chain rule)	$\frac{\partial E}{\partial w_i} = \sum_t \frac{\partial E}{\partial \text{net}_t} \frac{\partial \text{net}_t}{\partial w_i}$, Algorithm 5.5.1	Mục 5.5.1
CEC (LSTM)	$f(x) = x$, self-loop weight 1.0, $f' = 1$ nên sai số không co/nổ qua CEC	Mục 5.13
ReLU	$f(x) = x$ nếu $x > 0$; $f(x) = 0$ nếu ngược lại	Mục 5.24
Value function MDP	$V(x_t) = r_t + V(x_k)$, $V(x_k) = r_k$ nếu x_k là input cuối	Mục 6.2
5 chủ đề lặp lại	DP; UL hỗ trợ SL/RL; hierarchical representation; MDL/Occam; GPU	Mục 4
2 kỹ thuật SL thắng cuộc thi nhiều nhất tới 2014	LSTM (1997) + CTC (2006); GPU-MPCNN (2011) dựa trên CNN (1979) + MP (1992) + BP (1989-2007)	Mục 5.23

11. Kết luận ngắn

Bài này là một bản biên niên sử rất dài (88 trang, 888 trích dẫn) về Deep Learning trong neural network từ thế kỷ 19 tới tháng 10/2014, tổ chức xuyên suốt bằng khái niệm tự định nghĩa Credit Assignment Path (CAP) và ngưỡng tuỳ ý "problem depth > 10 là Very Deep Learning". Giá trị chính của bài không nằm ở một kết quả thực nghiệm hay thuật toán mới, mà ở việc hệ thống hoá 5 chủ đề lặp lại (DP, UL hỗ trợ SL/RL, học biểu diễn theo tầng bậc, MDL/Occam's razor, GPU) xuyên suốt ba nhánh SL, UL, RL, và chỉ ra rằng tới 2014 hai kỹ thuật thắng hầu hết cuộc thi là LSTM+CTC (cho dữ liệu tuần tự) và GPU-MPCNN (cho ảnh). Vì xuất bản trước 2015, bài hoàn toàn không đề cập ResNet, Batch Normalization, Transformer hay pre-training quy mô lớn - người đọc cần bổ sung các bài sau trong lộ trình để có bức tranh đầy đủ tới hiện tại; tác giả cũng tự thừa nhận bias do quen thuộc với công trình của chính nhóm mình, nên các mốc "đầu tiên" trong bài nên được đọc với một mức dè dặt hợp lý.

Nguồn gốc: https://arxiv.org/abs/1404.7828

Bài 26

Representation Learning: A Review and New Perspectives (2012)

Tác giả: Yoshua Bengio, Aaron Courville, Pascal Vincent (Département d'informatique et de recherche opérationnelle, Université de Montréal; Bengio và Vincent cũng thuộc CIFAR).
Venue / arXiv: arXiv:1206.5538 (v3, 23/04/2014); paper không nêu venue hội nghị/tạp chí trên trang đầu, chỉ có nhãn arXiv. (Ngoài paper, kiến thức chung: bản hoàn chỉnh sau đó xuất bản ở IEEE TPAMI, không được ghi trong văn bản PDF đọc ở đây.)
Nguồn: https://arxiv.org/abs/1206.5538
Chủ đề: tổng quan (review) ba hướng tiếp cận representation learning tới năm 2012: mô hình xác suất có biến ẩn (probabilistic models: sparse coding, RBM/Boltzmann machine), auto-encoder (và các biến thể regularized: denoising, contractive, PSD), và manifold learning; đặt ra các "prior" tổng quát cho AI-task, bàn vai trò của độ sâu/disentangling, huấn luyện toàn cục (global training) và cách đưa bất biến (invariance) vào mô hình (convolution, pooling, temporal coherence).
Vị trí trong lộ trình: Đọc cùng bài 25 về lịch sử deep learning và các bài 27–28 về transfer learning.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận. Bài tổng quan này được tóm tắt theo phần nội dung chính; danh mục tài liệu tham khảo không được trình bày đầy đủ.

1. Bối cảnh

Hiệu suất của các thuật toán học máy phụ thuộc rất nặng vào cách biểu diễn (representation/feature) của dữ liệu đầu vào. Trước 2012, phần lớn công sức triển khai học máy nằm ở "feature engineering" (thiết kế tay tiền xử lý/đặc trưng), vì các thuật toán học khi đó không tự trích xuất và tổ chức được thông tin phân biệt từ dữ liệu thô. Bài báo đặt câu hỏi: có thể làm cho thuật toán học ít phụ thuộc feature engineering hơn, tiến gần hơn tới AI tổng quát, bằng cách học chính các representation đó không?

Đến năm 2012, representation learning (gồm cả "deep learning"/"feature learning") đã thành một lĩnh vực riêng với hội nghị ICLR, các workshop tại NIPS/ICML, và một loạt kết quả thực nghiệm nổi bật mà bài báo liệt kê làm bối cảnh (tất cả là trích dẫn từ công trình khác, không phải của chính bài báo này):

- **Nhận dạng giọng nói:** Seide et al. (2011a) giảm word error rate khoảng 30% trên 4 benchmark lớn (ví dụ RT03S: 27.4% xuống 18.5%) với 309 giờ giọng nói, dùng deep learning thay Gaussian mixture. Dahl et al. (2012) giảm 16-23% trên benchmark Bing mobile (40 giờ).
- **Nhận dạng đối tượng (object recognition):** từ 2006, deep learning phá kỷ lục MNIST (SVM 1.4% lỗi); tới 2012, Ciresan et al. (2012) đạt 0.27% lỗi (có convolutional), Rifai et al. (2011c) đạt 0.81% lỗi ở thể loại "không dùng prior ảnh" (không convolution/deformation). Trên ImageNet 1000 lớp, Krizhevsky et al. (2012) giảm lỗi từ 26.1% xuống 15.3%.
- **NLP:** word embedding (Hinton 1986; Bengio et al. 2003) dùng trong neural net language model; mô hình hồi quy (Mikolov et al. 2011) giảm perplexity từ 140 xuống 102 và word error rate liên quan từ 17.2%/16.9% xuống 14.4% trên Wall Street Journal; hệ SENNA (Collobert et al. 2011) chia sẻ representation cho nhiều task NLP; word sense disambiguation tăng từ 67.8% lên 70.2% trên tập con Senseval-3 (Bordes et al. 2012).
- **Transfer learning:** hai Transfer Learning Challenge năm 2011 đều do thuật toán representation learning thắng (unsupervised layer-wise pre-training; spike-and-slab sparse coding S3C).

Từ các quan sát này, paper đặt mục tiêu tổng hợp lại: điều gì làm một representation "tốt"? Làm sao tính được

representation đó (inference)? Mục tiêu huấn luyện nào là phù hợp?

2. Ý tưởng chính

Paper tổ chức lại ba dòng nghiên cứu representation learning tưởng như tách biệt thành một khung chung:

1. **Mô hình xác suất (probabilistic models):** biểu diễn là phân phối hậu nghiệm $p(h \mid x)$ của biến ẩn h cho dữ liệu quan sát x; gồm mô hình có hướng (directed, ví dụ PCA xác suất, sparse coding) và mô hình vô hướng (undirected, ví dụ RBM/Boltzmann machine).
2. **Auto-encoder:** tham số hoá trực tiếp một hàm encoder f_θ (và decoder g_θ) ánh xạ input sang representation, huấn luyện bằng tối thiểu hoá lỗi tái tạo (có regularization để tránh học hàm identity).
3. **Manifold learning:** góc nhìn hình học, giả định dữ liệu tập trung gần một "manifold" (đa tạp) chiều thấp hơn nhiều so với không gian đầu vào; representation tốt gắn với toạ độ nội tại (intrinsic coordinate) trên manifold đó, và đạo hàm bậc một của encoder cho biết không gian tiếp xúc (tangent space) của manifold tại một điểm.

Paper lập luận ba góc nhìn này **liên kết chặt với nhau** (ví dụ: PCA đồng thời là mô hình xác suất, auto-encoder tuyến tính, và manifold tuyến tính; RBM huấn luyện bằng score matching tương đương một auto-encoder có regularization; auto-encoder có regularization ước lượng đạo hàm bậc một và hai của log-density, tức ước lượng cả vị trí và hướng tiếp xúc của manifold).

Các "prior" tổng quát cho AI-task (mục 3.1) mà một representation tốt nên nắm bắt: smoothness (hàm mục tiêu trơn), nhiều yếu tố giải thích độc lập (multiple explanatory factors), tổ chức theo thứ bậc (hierarchy, lý do cần depth), semi-supervised learning (chia sẻ thống kê giữa học không giám sát trên X và học có giám sát $P(Y \mid X)$), chia sẻ yếu tố giữa nhiều task, manifold, natural clustering (các lớp tương ứng các manifold riêng, không chồng lên nhau), coherence theo thời gian/không gian, sparsity, và sự đơn giản của quan hệ giữa các yếu tố (thường tuyến tính ở mức biểu diễn cao). Ý chính xuyên suốt: **distributed representation** (nhiều feature cùng "bật" độc lập) có sức biểu diễn vượt trội so với representation one-hot (clustering, cây quyết định, k-NN): với k thành phần không-zero trong N chiều, có thể phân biệt tới $O(2^k)$ vùng input chỉ với $O(N)$ tham số, trong khi one-hot cần $O(N)$ tham số cho $O(N)$ vùng.

3. Phương pháp chi tiết

3.1 PCA như điểm chung của ba góc nhìn (mục 5)

PCA học $h = f(x) = W^T x + b$ với cột của W là các hướng phương sai lớn nhất. Paper dùng PCA làm ví dụ xuyên suốt vì nó đồng thời là: (a) mô hình xác suất (factor analysis, công thức 1: $p(h) = \mathcal{N}(h; 0, \sigma_h^2 I)$, $p(x \mid h) = \mathcal{N}(x; Wh + \mu_x, \sigma_x^2 I)$); (b) auto-encoder tuyến tính cơ bản; (c) manifold tuyến tính (đặc trưng hoá một vùng chiều thấp nơi mật độ dữ liệu đạt đỉnh). Nhược điểm: feature tuyến tính không thể xếp lớp (compose thành sâu) vì tổ hợp các phép tuyến tính vẫn là tuyến tính, nên các phần sau tập trung vào feature phi tuyến.

3.2 Mô hình xác suất có hướng: sparse coding (mục 6.1)

Sparse coding học từ điển (dictionary) W sao cho mỗi x được biểu diễn bởi mã thưa h:

$$h^* = f(x) = \arg\min_h \|x - Wh\|_2^2 + \lambda \|h\|_1 \quad (2)$$

$$J_{SC} = \sum_t \|x^{(t)} - W h^{*(t)}\|_2^2 \quad (3)$$

Diễn giải xác suất: prior Laplace trên h (tương ứng phạt L1), $p(x \mid h) = \mathcal{N}(x; Wh + \mu_x, \sigma_x^2 I)$ (công thức 4); học W bằng ước lượng MAP của h (không chuẩn trong lý thuyết xác suất đồ hoạ, nhưng hoạt động tốt thực

nghiệm, giống k-means so với Gaussian mixture). Sparse coding minh hoạ rõ hiệu ứng **explaining away**: dù từ điển "overcomplete" (nhiều chiều h hơn x) và các cột tương quan cao, bước suy luận MAP vẫn chọn được tập con phù hợp nhất và đặt các thành phần còn lại về 0. Paper nêu biến thể spike-and-slab sparse coding (S3C, Goodfellow et al. 2012): tách biến "spike" nhị phân (quyết định sparsity pattern) và biến "slab" thực; S3C thắng NIPS'2011 Transfer Learning Challenge và vượt sparse coding trên CIFAR-10/100 khi có ít (<1000) ảnh có nhãn mỗi lớp.

3.3 Mô hình xác suất vô hướng: Boltzmann machine và RBM (mục 6.2-6.4)

Hàm năng lượng Boltzmann machine đầy đủ (công thức 7):

$$E_\theta^{BM}(x,h) = -\tfrac{1}{2}x^T U x - \tfrac{1}{2}h^T V h - x^T W h - b^T x - d^T h$$

RBM giới hạn tương tác chỉ giữa x và h ($U = V = 0$), cho hai phân phối điều kiện tách biệt (công thức 12-13):

$$P(h_i = 1 \mid x) = \text{sigmoid}\Big(\sum_j W_{ji} x_j + d_i\Big), \quad P(x_j = 1 \mid h) = \text{sigmoid}\Big(\sum_i W_{ji} h_i + b_j\Big)$$

giúp suy luận $P(h \mid x)$ tractable, nhưng partition function Z_θ vẫn intractable. Gradient log-likelihood (công thức 15) có pha "clamped" (positive phase, dễ tính) và pha "unclamped" (negative phase, cần MCMC). Ba thuật toán xấp xỉ chính: **Contrastive Divergence (CD)** (chạy Gibbs rất ngắn, thường 1 bước, khởi tạo từ data), **Stochastic Maximum Likelihood/Persistent CD (SML/PCD)** (Gibbs chain chạy liên tục qua các bước cập nhật), và **Fast-weights PCD (FPCD, công thức 17)**: $\theta^*_{t+1} = (1-\eta)\theta^*_{t+1} + \eta\theta^*_t + \epsilon^* \, \partial/\partial\theta_i \sum_t \log p(x^{(t)})$ với $\epsilon^* > \epsilon$, giúp mixing nhanh hơn bằng cách lấy mẫu từ một bản nhiễu loạn cục bộ của mô hình. Paper cũng nêu Gaussian RBM và các mở rộng cho dữ liệu thực (mcRBM, mPoT, spike-and-slab RBM/ssRBM) nhằm mô hình hoá covariance (không chỉ mean) của ảnh tự nhiên.

3.4 Auto-encoder và các biến thể regularized (mục 7)

Auto-encoder cơ bản: $h^{(t)} = f_\theta(x^{(t)})$, tái tạo $r = g_\theta(h)$, tối thiểu hoá (công thức 19):

$$J_{AE}(\theta) = \sum_t L(x^{(t)}, g_\theta(f_\theta(x^{(t)}))), \quad f_\theta(x) = s_f(b + Wx),\ g_\theta(h) = s_g(d + W'h)$$

Denoising Auto-Encoder (DAE, Vincent et al. 2008/2010**,** công thức 22**):**

$$J_{DAE} = \sum_t \mathbb{E}_{q(\tilde{x}|x^{(t)})}[L(x^{(t)}, g_\theta(f_\theta(\tilde{x})))]$$

học tái tạo bản sạch từ bản bị nhiễu (Gaussian, salt-and-pepper, hoặc masking noise); học hàm identity không còn đủ, buộc mô hình nắm cấu trúc phân phối dữ liệu. Vincent (2011) chỉ ra DAE học một vector field hướng về vùng mật độ cao, gắn với score (đạo hàm log-density); DAE tuyến tính + lỗi bình phương tương đương một energy-based model gần GRBM, huấn luyện bằng "denoising score matching".

Contractive Auto-Encoder (CAE, Rifai et al. 2011a**,** công thức 23-24**):**

$$J_{CAE} = \sum_t L(x^{(t)}, g_\theta(f_\theta(x^{(t)}))) + \lambda \|J(x^{(t)})\|_F^2, \qquad \|J(x)\|_F^2 = \sum_j (f_\theta(x)_j(1 - f_\theta(x)_j))^2 \|W_j\|^2$$

phạt trực tiếp chuẩn Frobenius của Jacobian encoder (độ nhạy representation với thay đổi input nhỏ) thay vì phạt độ nhạy của reconstruction như DAE. Ba khác biệt với DAE mà paper nêu: (a) phạt độ nhạy của feature, không phải reconstruction; (b) penalty có công thức đóng (giải tích), không cần lấy mẫu nhiễu; (c) có λ riêng để cân bằng reconstruction/robustness tách khỏi nhiễu. DAE với nhiễu nhỏ có thể xem là một dạng CAE phạt trên toàn hàm reconstruction (Alain & Bengio 2012). Biến thể CAE+H (công thức 25) phạt cả đạo hàm bậc cao hơn bằng cách ép $J(x)$ và $J(x+\epsilon)$ gần nhau.

Predictive Sparse Decomposition (PSD, Kavukcuoglu et al. 2008**,** công thức 26-27**):**

$$J_{PSD} = \sum_t \lambda \|h^{(t)}\|_1 + \|x^{(t)} - Wh^{(t)}\|_2^2 + \|h^{(t)} - f_\alpha(x^{(t)})\|_2^2, \quad f_\alpha(x) = \tanh(b + W^T x)$$

tối ưu đồng thời mã thưa h (như sparse coding) **và** một encoder tham số nhanh f_α xấp xỉ h, để lúc nhận dạng (inference) không cần giải lại bài toán tối ưu tốn kém của sparse coding. Paper diễn giải PSD như một "biến thể" nằm giữa mô hình xác suất (có decoder sinh $p(x \mid h)$) và direct-encoding (có encoder tham số), tương tự vai trò "kép" của RBM.

3.5 Manifold learning và Local Coordinate Coding (mục 8)

Local Coordinate Coding (LCC, Yu et al. 2009, công thức 28) sửa sparse coding bằng cách thêm trọng số khoảng cách tới điểm neo (anchor):

$$J_{LCC} = \sum_t \left(\|x^{(t)} - Wh^{(t)}\|_2^2 + \lambda \sum_j |h_j^{(t)}| \, \|W_{:,j} - x^{(t)}\|_1^{1+p} \right)$$

($p = -1$ cho lại sparse coding thường). Định lý của Yu et al. (2009): một hàm Lipschitz-trơn trên manifold M có thể được xấp xỉ tốt bởi một hàm tuyến tính của mã h, với độ chính xác phụ thuộc chiều nội tại d_M (không phải chiều input d_x). Với auto-encoder/PSD, các hướng mà encoder "nhạy" (singular vector lớn của Jacobian) được xem như span không gian tiếp xúc (tangent space) tại điểm đó (Hình 4 của paper, phân tích CAE trên MNIST: phổ singular value giảm mạnh, chỉ vài hướng nhạy, phù hợp giả thuyết manifold chiều thấp).

3.6 Huấn luyện toàn cục: Deep Boltzmann Machine (mục 10.2)

Với 2 tầng ẩn, năng lượng DBM (công thức 29):

$$E_\theta^{DBM}(v, h^{(1)}, h^{(2)}) = -v^T W h^{(1)} - h^{(1)T} V h^{(2)} - d^{(1)T} h^{(1)} - d^{(2)T} h^{(2)} - b^T v$$

Posterior không còn tractable (khác RBM), nên dùng xấp xỉ mean-field $Q_v(h^{(1)}, h^{(2)})$ tối đa hoá cận dưới biến phân (công thức 30, 33) qua các phương trình cập nhật lặp (công thức 31-32), rồi huấn luyện bằng SML cho negative phase. Salakhutdinov & Hinton (2009) nhận thấy huấn luyện SML trực tiếp từ đầu dễ rơi vào cực tiểu cục bộ xấu (nhiều unit "chết"), nên đề xuất tiền huấn luyện từng tầng kiểu Deep Belief Network rồi mới "fine-tune" chung bằng SML.

3.7 Đưa bất biến vào mô hình (mục 11)

Convolution + pooling: chia sẻ tham số của một bộ trích đặc trưng cục bộ (local receptive field) quét qua toàn ảnh, sau đó pooling (max hoặc sum, hoặc L2-pooling) theo vị trí lân cận để tạo bất biến dịch chuyển cục bộ; đây là nền tảng CNN/Neocognitron/HMAX. **Patch-based training:** chạy thuật toán unsupervised bất kỳ (kể cả k-means đơn giản) trên các patch nhỏ rút ngẫu nhiên, rồi quét (sweep) bộ trích đặc trưng đã học qua ảnh để tạo feature map. **Temporal/spatial coherence:** phạt sự thay đổi (bình phương hoặc L1) của feature theo thời gian, dựa trên giả định các yếu tố giải thích thay đổi chậm.

Ví dụ số (tự tính), minh hoạ ưu thế distributed representation (mục 3.3 của paper): xét $N = 6$ đơn vị nhị phân. Một representation one-hot (clustering) với 6 trạng thái chỉ phân biệt được 6 cấu hình. Một distributed code cho phép đúng $k = 3$ trong 6 đơn vị "bật" cùng lúc phân biệt được $\binom{6}{3} = 20$ cấu hình khác nhau (tự tính: $6!/(3! \cdot 3!) = 20$), tức khoảng $3,3\times$ số cấu hình so với one-hot dù dùng cùng $N = 6$ "chỗ chứa". Nếu cho tất cả $2^6 = 64$ tổ hợp bật/tắt đều hợp lệ (không ép đúng 3 bit), số cấu hình phân biệt được là $2^6 = 64$, tức $O(2^N)$ so với $O(N) = 6$ của one-hot, đúng như claim "$O(2^k)$ vùng với $O(N)$ tham số" của paper.

4. Thí nghiệm và kết quả chính

Đây là bài **review lý thuyết**: paper không chạy thí nghiệm mới của riêng mình, mà tổng hợp và trích dẫn kết quả từ các công trình khác để minh hoạ các luận điểm. Các số liệu đã nêu ở mục 1 (nhận dạng giọng nói, ImageNet, NLP, Transfer Learning Challenge) là ví dụ chính. Một số điểm bổ sung được trích trong phần phương pháp:

Kết quả trích dẫn	Số liệu	Nguồn trong paper
MNIST, có prior ảnh (convolution)	0.27% lỗi	Ciresan et al. (2012)
MNIST, không prior ảnh	0.81% lỗi	Rifai et al. (2011c), dùng Manifold Tangent Classifier
ImageNet 1000 lớp	26.1% → 15.3% lỗi	Krizhevsky et al. (2012)
MNIST biến dạng, affine+elastic	0.32% lỗi	Ciresan et al. (2010), MLP lớn + GPU 40x speedup
CIFAR-10/100, <1000 nhãn/lớp	sparse coding/S3C vượt các encoding khác	Coates & Ng (2011a); Goodfellow et al. (2012)
Transfer Learning Challenge 2011 (2 cuộc)	cả hai do representation learning thắng	unsupervised layer-wise pre-training; Goodfellow et al. (2011)

Paper không đưa bảng so sánh định lượng tổng hợp của riêng mình (không có Table thực nghiệm gốc); mọi số liệu trên là số liệu "thứ cấp" (second-hand) dùng để minh hoạ xu hướng, không nên coi là "kết quả của bài 26" khi trích lại trong các báo cáo khác.

5. Phân tích / ablation

Vì không có thí nghiệm gốc, phần này tổng hợp các **so sánh/định tính** mà paper đưa ra giữa các phương pháp (ablation "khái niệm" hơn là ablation số liệu):

- **CD vs SML vs FPCD:** CD có bias (vì không chờ Gibbis chain hội tụ) nhưng phương sai thấp; SML không bias tiệm cận nhưng phụ thuộc mixing tốt của Gibbs chain, và mixing xấu đi khi trọng số lớn dần (weights lớn → chain khó "nhảy mode"); FPCD cải thiện mixing bằng weight nhiễu loạn tạm thời nhưng chưa hiểu rõ lý thuyết hội tụ, và có thể sinh mẫu "giả" (spurious) giữa hai mode. Paper trích Marlin et al. (2010): SML cho kết hợp tốt nhất giữa hiệu năng và chi phí tính, nhưng Swersky et al. (2011) sau đó cho rằng denoising score matching cạnh tranh được cả về classification và tốc độ.
- **DAE vs CAE:** CAE thực nghiệm (theo paper tổng hợp) cho feature tốt hơn DAE nhờ ba khác biệt (phạt sensitivity trực tiếp, công thức đóng, λ tách biệt), nhưng CAE chỉ phạt độ nhạy **vô cùng nhỏ** (infinitesimal), còn DAE "robust" với nhiễu có biên độ hữu hạn hơn; CAE+H khắc phục bằng phạt đạo hàm bậc cao.
- **Patch-based k-means vs sparse coding phức tạp hơn:** Coates & Ng (2011a) thấy hiệu năng gần tương đương dù dùng thuật toán rất đơn giản; paper suy đoán (của tác giả, không chứng minh) là do patch nhỏ (ví dụ 6×6) đã đủ chiều thấp để không cần distributed representation phức tạp, hoặc vì histogram pooling của cluster count tự nó đã là một distributed representation.

- **Huấn luyện DBM từ đầu (SML thuần) vs greedy layer-wise + fine-tune:** phương án đầu thất bại (nhiều unit "chết", Desjardins et al. 2012), phương án sau (Salakhutdinov & Hinton 2009) hoạt động được; đây là minh chứng cụ thể cho lợi ích của "guidance từng tầng" được bàn ở mục 10.1.
- **Unsupervised pre-training có cần khi dữ liệu nhãn rất lớn?** Paper trích Krizhevsky et al. (2012, ImageNet) là ví dụ "very deep purely supervised networks can be trained successfully without any layerwise pre-training" khi có đủ dữ liệu nhãn, khởi tạo tốt, và phi tuyến phù hợp (rectifier). Paper diễn giải điều này ủng hộ giả thuyết pre-training đóng vai trò **regularization** (ít cần khi dữ liệu nhãn đủ lớn), nhưng cũng đặt câu hỏi mở "vì sao điều này không được phát hiện sớm hơn".

6. Giới hạn

Do tác giả nêu (rải trong mục 3.6, 9, 10, 11, 12):

- Objective "tốt" cho representation learning không rõ ràng như trong classification (không có mục tiêu đơn giản kiểu "giảm số lỗi"); việc tối ưu một mục tiêu gián tiếp (reconstruction, likelihood) không đảm bảo tối ưu mục tiêu cuối (classifier downstream).
- **Sampling/mixing** trong Boltzmann machine ngày càng khó khi huấn luyện tiến triển: entropy mô hình giảm, các mode (manifold) trở nên cách biệt bởi vùng mật độ thấp rộng, MCMC bước nhỏ khó nhảy giữa mode (Hình 7 của paper).
- Chưa có phương pháp rõ ràng để **disentangle** các yếu tố giải thích một cách tổng quát (không giả định trước thời điểm thay đổi của từng yếu tố như transforming auto-encoder của Hinton et al. 2011 phải giả định).
- Pooling làm mất thông tin không thể phục hồi (không biết "credit" cho feature cấp thấp nào khi chỉ biết giá trị pooled), và không có cách đảm bảo tầng cao hơn thực sự bất biến hơn tầng thấp (chỉ có bằng chứng thực nghiệm yếu, Erhan et al. 2010a).
- Vấn đề tối ưu (optimization) của deep architecture **chưa được hiểu rõ**: không rõ khó khăn chủ yếu do cực tiểu cục bộ hay do ill-conditioning; paper nói thẳng "these basic questions remain unanswered and deserve much more study".
- Inference xấp xỉ (MAP, variational, MCMC) có thể không nắm hết tính đa-mode của posterior; paper đề xuất ý tưởng "học inference" thay vì dùng cơ chế suy luận chung, nhưng chỉ là hướng gợi mở, chưa có giải pháp.

Người ghi chú nhận xét:

- Paper viết ngay trước giai đoạn bản lề của deep learning (2012-2014): chính trong phần 10.1, paper đã trích dẫn kết quả AlexNet/ImageNet (Krizhevsky et al. 2012) như một dấu hiệu "pre-training không nhất thiết cần khi đủ dữ liệu nhãn", nhưng chưa thể dự đoán được rằng xu hướng 2012-2017 sẽ chuyển hẳn sang purely-supervised training (ReLU, batch norm, Adam, dropout) và "layerwise unsupervised pre-training" (RBM, DBN, stacked auto-encoder, trọng tâm chính của 2/3 bài viết) gần như biến mất khỏi thực hành chủ đạo, chỉ hồi sinh dưới dạng khác (contrastive/self-supervised learning, "ml basics 17-18") sau 2018-2020.
- Phần RBM/Boltzmann machine (mục 6, chiếm tỉ lệ lớn của bài) hiện ít còn dùng trực tiếp trong các pipeline hiện đại; giá trị còn lại chủ yếu là khái niệm (explaining away, energy-based model, liên hệ CD với auto-encoder) hơn là công thức để áp dụng trực tiếp.
- Một số tuyên bố mang tính suy đoán được chính tác giả đánh dấu rõ là giả thuyết chưa kiểm chứng (ví dụ lý do k-means đơn giản hoạt động tốt ở patch nhỏ, mục 11.2); cần đọc đúng với mức độ chắc chắn mà paper gán cho từng claim, không nên coi mọi câu trong bài là kết luận đã được chứng minh.

7. Liên hệ với các bài khác

- **ml basics 14** (Generative Adversarial Networks, 1406.2661): cách tiếp cận sinh hoàn toàn khác (không log-likelihood, không MCMC) cho cùng câu hỏi "học representation sinh được dữ liệu", xuất hiện 2

năm sau bài này, giải quyết trực tiếp vấn đề mixing/sampling (mục 9.4) mà RBM/DBM gặp phải.

- **ml basics 15** (Auto-Encoding Variational Bayes, 1312.6114): VAE hình thức hoá chính xác ý tưởng PSD nêu ở mục 9.1 ("encoder là xấp xỉ tham số cho MAP/posterior của một mô hình có biến ẩn", "biến phân lower bound" như công thức 30/33 ở DBM) nhưng dùng reparameterization trick để lan truyền gradient qua bước lấy mẫu, thay cho mean-field hoặc MCMC.
- **ml basics 17, 18** (SimCLR, MoCo): hiện thực hoá trực tiếp prior "natural clustering" và ý tưởng pooling/bất biến (mục 3.1, mục 11) bằng contrastive learning, không cần decoder hay mô hình sinh, khác hẳn hướng auto-encoder/RBM chiếm phần lớn bài này.
- **math 06** (New insights and perspectives on the natural gradient method): natural gradient (Amari 1998) được chính paper này nhắc tới ở mục 10.1 như một hướng giải quyết ill-conditioning khi huấn luyện deep architecture (trích Le Roux et al. 2008b; Pascanu & Bengio 2013), tức hai bài bàn cùng vấn đề tối ưu ở hai mức độ chi tiết khác nhau.
- **math 09** (An Introduction to Variational Autoencoders): mở rộng trực tiếp phần biến phân (variational lower bound, mean-field) được giới thiệu sơ ở mục 9 và 10.2 của bài 26.

8. Gợi ý áp dụng

Khung representation learning giúp phân tích những yếu tố biến thiên mà biểu diễn cần giữ lại hoặc bỏ qua. Các đặc tính như disentanglement cần được kiểm tra theo tác vụ và phép đo cụ thể, không thể suy ra chỉ từ một hình chiếu trực quan.

9. Thí nghiệm gợi ý

1. **So sánh DAE vs CAE trên toy data:** huấn luyện hai auto-encoder nhỏ (encoder 1-2 lớp, ~256 chiều ẩn) trên MNIST hoặc một tập con ảnh trang OCR cắt nhỏ; đo reconstruction error trên tập test sạch và tập test có nhiễu nhân tạo (masking). Kỳ vọng: CAE có Jacobian norm nhỏ hơn rõ (đo trực tiếp $\|J(x)\|_F$), DAE robust hơn với nhiễu biên độ lớn hơn do được huấn luyện trực tiếp trên nhiễu đó; diễn giải kết quả theo đúng 3 khác biệt (a)(b)(c) nêu ở mục 3.4.
2. **Đếm "vùng phân biệt được" theo kiểu one-hot vs distributed (mô phỏng nhỏ):** sinh dữ liệu toy với $N = 8$ chiều ẩn nhị phân; so hiệu năng phân loại của một linear probe khi ép representation one-hot (one unit active) so với khi cho phép $k = 3$ unit active đồng thời, trên một bài toán phân loại tổng hợp có đúng số lớp tương ứng; đo accuracy và số tham số cần. Kỳ vọng: distributed cho accuracy ngang hoặc hơn với ít tham số hơn, minh hoạ trực tiếp lập luận mục 3.3 (không chỉ là lý thuyết).
3. **Sparse coding trên patch ảnh OCR nhỏ:** học dictionary sparse coding (công thức 2-3) trên các patch 8×8 cắt từ vài trang scan OCR; visualize các cột W đã học (kỳ vọng giống edge/stroke detector như trong ảnh tự nhiên); đo tỉ lệ thưa trung bình của h^* và so với việc dùng patch embedding hiện có của vision encoder (nếu trích được) để xem patch embedding hiện tại có "thưa" tương tự không.
4. **RBM nhỏ: CD-1 vs PCD trên dữ liệu nhị phân nhân tạo:** RBM với ~20 visible, ~10 hidden units; huấn luyện bằng CD-1 và PCD trên cùng dữ liệu, theo dõi log-likelihood ước lượng (bằng liệt kê trực tiếp vì mô hình nhỏ, đúng gợi ý "toy RBM 25 units" ở mục 9.5 của paper) qua các epoch. Kỳ vọng: PCD hội tụ gần hơn log-likelihood thật nhưng có thể kém ổn định hơn khi trọng số lớn (minh hoạ đúng vấn đề mixing nêu ở mục 5).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý chính
PCA xác suất	$p(h) = \mathcal{N}(h; 0, \sigma_h^2 I)$, $p(x \mid h) = \mathcal{N}(x; Wh + \mu_x, \sigma_x^2 I)$
Sparse coding	$h^* = \arg\min_h \|x - Wh\|_2^2 + \lambda \|h\|_1$; $J_{SC} = \sum_t \|x^{(t)} - Wh^{*(t)}\|_2^2$
Năng lượng Boltzmann machine	$E_\theta^{BM} = -\frac{1}{2} x^T U x - \frac{1}{2} h^T V h - x^T W h - b^T x - d^T h$

Khái niệm	Công thức / ý chính
RBM, posterior	$P(h_i = 1 \mid x) = \text{sigmoid}(\sum_j W_{ji} x_j + d_i)$ (tractable, factorized)
Gradient log-likelihood RBM	positive phase (clamped, tractable) trừ negative phase (unclamped, cần MCMC)
FPCD	$\theta^*_{t+1} = (1-\eta)\theta^*_{t+1} + \eta\theta^*_t + \epsilon^* \, \partial \sum \log p(x^{(t)})/\partial\theta_i$
Auto-encoder cơ bản	$J_{AE} = \sum_t L(x^{(t)}, g_\theta(f_\theta(x^{(t)})))$
Denoising AE (DAE)	$J_{DAE} = \sum_t \mathbb{E}_{q(\tilde{x}\mid x^{(t)})}[L(x^{(t)}, g_\theta(f_\theta(\tilde{x})))]$
Contractive AE (CAE)	$J_{CAE} = \sum_t L(\cdot) + \lambda \lVert J(x^{(t)}) \rVert_F^2$
Predictive Sparse Decomposition	$J_{PSD} = \sum_t \lambda \lVert h^{(t)} \rVert_1 + \lVert x^{(t)} - W h^{(t)} \rVert_2^2 + \lVert h^{(t)} - f_\alpha(x^{(t)}) \rVert_2^2$
Local Coordinate Coding	thêm trọng số $\lVert W_{:,j} - x^{(t)} \rVert_1^{1+p}$ vào sparse coding, gắn với chiều manifold d_M
Deep Boltzmann Machine	$E_\theta^{DBM} = -v^T W h^{(1)} - h^{(1)T} V h^{(2)} - \ldots$; suy luận bằng mean-field, học bằng SML trên cận biến phân
Phân biệt vùng input	distributed/sparse: $O(2^k)$ vùng với $O(N)$ tham số; one-hot: $O(N)$ vùng với $O(N)$ tham số

11. Kết luận ngắn

Bài review này hợp nhất ba hướng representation learning tới 2012 (probabilistic latent-variable model, auto-encoder, manifold learning) dưới một bộ "prior" chung cho AI-task (distributed, sparse, phân tầng, disentangled), và chỉ ra các cầu nối toán học cụ thể giữa chúng (PCA là điểm giao; DAE/CAE liên hệ score matching; PSD/RBM nằm giữa mô hình xác suất và direct-encoding). Bài báo không có thí nghiệm riêng, chỉ tổng hợp số liệu từ nơi khác để minh hoạ xu hướng (ImageNet, MNIST, speech, NLP). Giá trị bền của bài nằm ở khung khái niệm (distributed representation, disentangling, manifold tangent, các prior AI-task) hơn là các công thức RBM/DBM cụ thể, phần lớn đã được thay thế trong thực hành hiện đại bởi purely-supervised training và sau đó bởi self-supervised/contrastive learning.

Nguồn gốc: https://arxiv.org/abs/1206.5538

Bài 27

How Transferable Are Features in Deep Neural Networks? (2014)

Tác giả: Jason Yosinski (Cornell), Jeff Clune (University of Wyoming), Yoshua Bengio (Université de Montréal), Hod Lipson (Cornell).
Venue / arXiv: arXiv:1411.1792 (v1, 06/11/2014); venue chính thức ghi ở trang đầu bản preprint: "Advances in Neural Information Processing Systems 27 (NIPS '14)".
Nguồn: https://arxiv.org/abs/1411.1792
Chủ đề: đo định lượng, theo từng tầng, mức độ "general" (dùng được cho task khác) hay "specific" (chỉ hợp với task gốc) của feature trong một mạng CNN tám tầng huấn luyện trên ImageNet, qua thí nghiệm cắt-ghép mạng (transfer vs selffer, đóng băng vs fine-tune).
Vị trí trong lộ trình: Đọc trước bài 28 để đối chiếu khả năng chuyển giao đặc trưng với vai trò của pretraining.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Một hiện tượng đã được quan sát nhiều lần: mạng CNN huấn luyện trên ảnh tự nhiên, bất kể tập dữ liệu hay hàm mục tiêu (supervised classification như Krizhevsky et al. 2012, unsupervised density learning như Lee et al. 2009, hay sparse representation không giám sát như Le et al. 2011), đều học được **tầng đầu** gần giống bộ lọc Gabor và "color blob". Paper gọi đây là feature **general** (dùng được cho nhiều tập dữ liệu/task). Ngược lại, **tầng cuối** của một mạng classification (softmax N chiều) chắc chắn phải **specific** cho đúng task đó (mỗi unit gắn với một lớp cụ thể). Vậy giữa hai đầu "general" và "specific" đó, sự chuyển tiếp diễn ra ở đâu, đột ngột ở một tầng hay trải dài qua nhiều tầng?

Bối cảnh ứng dụng: transfer learning (Caruana 1995; Bengio et al. 2011; Bengio 2011) thường copy n tầng đầu của một "base network" đã huấn luyện sang một "target network", rồi huấn luyện tiếp các tầng còn lại (ngẫu nhiên hoá) trên task đích; n tầng copy có thể bị đóng băng (frozen) hoặc cho fine-tune tiếp. Lựa chọn đóng băng hay fine-tune phụ thuộc kích thước tập dữ liệu đích và số tham số của n tầng đó: tập dữ liệu nhỏ + nhiều tham số dễ overfit nên thường đóng băng; tập dữ liệu lớn hoặc ít tham số thì fine-tune có lợi. Các nghiên cứu trước (Donahue et al. 2013a; Zeiler & Fergus 2013; Sermanet et al. 2014) đã dùng feature tầng cao cho SOTA ở nhiều task, gợi ý các tầng đó khá "general", nhưng chưa ai đo định lượng theo từng tầng một cách có hệ thống. Đây chính là khoảng trống paper lấp vào.

2. Ý tưởng chính

Định nghĩa **độ general của feature học trên task A** là mức độ feature đó dùng tốt cho task B. Paper thiết kế một bộ thí nghiệm đối chứng chặt: với mỗi tầng $n \in \{1, \dots, 7\}$ (mạng 8 tầng, không xét $n = 8$ vì đó chính là lớp output), tạo 4 cấu hình mạng xuất phát từ hai mạng gốc baseA và baseB (mỗi cái huấn luyện trên một nửa ngẫu nhiên 500 lớp ImageNet):

- **BnB** (selffer, đóng băng): copy n tầng đầu từ baseB, đóng băng, random-init và huấn luyện $8 - n$ tầng còn lại **trên cùng tập dữ liệu B**. Đây là đối chứng (control), không phải transfer thật.
- **AnB** (transfer, đóng băng): copy n tầng đầu từ baseA, đóng băng, huấn luyện phần còn lại **trên tập dữ liệu B**. Đây là phép đo transfer thật sự.
- **BnB+**, **AnB+**: như trên nhưng toàn bộ mạng được fine-tune (không đóng băng n tầng copy).

So BnB với baseB cho biết: việc "cắt rồi dán lại" bản thân nó (dù không đổi task) có làm hại hiệu năng không, qua đó phát hiện hiện tượng **fragile co-adaptation** (các neuron ở tầng liên tiếp phối hợp với nhau một cách "mong manh" trong quá trình huấn luyện đồng thời; cắt rời một tầng ra rồi để các tầng trên tự học lại có thể không khôi phục được phối hợp đó). So AnB với BnB cho biết phần giảm hiệu năng nào là do mất đặc trưng chuyên biệt (specificity) so với phần do khó tối ưu (fragile co-adaptation) khi cắt mạng.

3. Phương pháp chi tiết

3.1 Dataset và kiến trúc

ImageNet (ILSVRC2012, Deng et al. 2009): 1.281.167 ảnh train, 50.000 ảnh test, 1000 lớp. Chia ngẫu nhiên 1000 lớp thành hai nửa A và B, mỗi nửa 500 lớp, khoảng 645.000 ảnh mỗi nửa (xấp xỉ một nửa dữ liệu). Kiến trúc: mạng CNN 8 tầng tham chiếu của Caffe (Jia et al. 2014), gần với AlexNet (Krizhevsky et al. 2012) nhưng có vài khác biệt nhỏ theo Donahue et al. (2013): bỏ data augmentation PCA-color, warp ảnh về 256×256 thay vì scale-giữ-tỉ-lệ rồi crop, đặt Local Response Normalization sau pooling (thay vì trước), dùng dropout trên các tầng fully-connected (Hinton et al. 2012, "Improving neural networks by preventing co-adaptation of feature detectors", chính là nguồn gốc thuật ngữ "co-adaptation" mà paper này dùng).

3.2 Siêu tham số huấn luyện (Phụ lục A)

SGD với momentum 0.9, batch size 256, weight decay 0.0005/iteration (chỉ áp dụng cho trọng số không bị đóng băng). Learning rate khởi đầu 0.01, giảm theo hệ số 10 mỗi 100.000 iteration, dừng sau 450.000 iteration. Mỗi iteration ~1.7 giây trên GPU NVidia K20, toàn bộ quá trình huấn luyện một mạng ~9.5 ngày. Mạng base đạt top-1 error 42.5% trên validation (so với 42.9% của Donahue et al. 2013 và kém 1.8% so với 40.7% của Krizhevsky et al. 2012; paper nói sự khác biệt này đến từ các thay đổi huấn luyện nhỏ nêu trên, không phải mục tiêu của bài là đạt SOTA).

3.3 Hai thí nghiệm bổ sung

Man-made vs Natural split (Phụ lục C): để tạo A/B khác biệt ngữ nghĩa tối đa, paper gán nhãn mỗi node trong đồ thị WordNet bằng số lớp ImageNet có thể truy tới từ nó (theo chiều cha→con), rồi chọn hai tập con lớn nhất, loại trừ nhau: "artifact" (522 lớp, nhân tạo) và "organism" (410 lớp, tự nhiên). 68 lớp còn lại ($1000 - 522 - 410 = 68$, tự tính khớp với số paper nêu) được gán tay (ví dụ pizza, cup, bagel vào nhóm nhân tạo; strawberry, volcano, banana vào nhóm tự nhiên), cho tổng cộng 551 lớp nhân tạo và 449 lớp tự nhiên.

Random weights (Section 4.3): thay n tầng đầu bằng trọng số ngẫu nhiên chưa huấn luyện (không copy từ base network nào), để kiểm tra xem bộ lọc ngẫu nhiên có hoạt động gần-tối-ưu như Jarrett et al. (2009) từng báo cáo trên mạng nhỏ hơn (2-3 tầng) và dataset nhỏ hơn (Caltech-101) hay không.

3.4 Kiểm tra mức overfit của AlexNet trên ImageNet (Phụ lục B)

Để loại trừ khả năng kết quả "random weights kém" chỉ do AlexNet overfit mạnh trên ImageNet (khác với Caltech-101 nhỏ), paper huấn luyện lại cùng kiến trúc với số ảnh/lớp giảm dần: $n \in \{1300, 1000, 750, 500, 250, 100, 50, 25, 10, 5, 2, 1\}$ (siêu tham số nhanh hơn: lr 0.0125, giảm sau 64.000 iteration, dừng sau 200.000 iteration, ~4 ngày/lần chạy trên K20).

4. Thí nghiệm và kết quả chính

Thí nghiệm chính, A/B ngẫu nhiên (tương tự, Section 4.1, Hình 2, 4 lần chia A/B ngẫu nhiên khác nhau):

- baseB đạt top-1 accuracy 0.625 (error 37.5%), thấp hơn error 42.5% của mạng 1000-lớp vì chỉ còn 500 lớp để nhầm (ít "cách để sai" hơn), dù dataset bị chia đôi có thể làm tăng overfit.
- **BnB** (selffer, đóng băng): tầng 1, 2 bằng baseB; tầng 3-6 (rõ nhất ở 4, 5) **giảm hẳn**, bằng chứng của fragile co-adaptation; tầng 6, 7 gần trở lại mức base (càng gần output 8 chiều 500-way, càng ít phải "học lại").

- **BnB+** (selffer, fine-tune): gần bằng baseB ở mọi tầng, tức fine-tune khắc phục được co-adaptation bị mất.
- **AnB** (transfer, đóng băng): tầng 1, 2 chuyển gần như hoàn hảo từ A sang B; tầng 3 giảm nhẹ; tầng 4-7 giảm rõ hơn. So với BnB, paper tách được: ở tầng 3, 4, 5 phần giảm chủ yếu do **mất co-adaptation** (vì BnB cũng giảm tương tự ở các tầng này); ở tầng 6, 7, phần giảm chủ yếu do **tính specific của representation** (vì BnB đã hồi phục ở các tầng này nhưng AnB vẫn giảm).
- **AnB+** (transfer, fine-tune): tốt hơn cả baseB, hiệu ứng "bất ngờ" chính của bài. Bảng 1 (trung bình qua các khoảng tầng được gộp vì chi phí tính mỗi điểm ~9.5 ngày GPU):

Tầng gộp	Boost trung bình so với baseB	Boost trung bình so với selffer BnB+
1-7	1.6%	1.4%
3-7	1.8%	1.4%
5-7	2.1%	1.7%

Paper loại trừ khả năng boost chỉ do tổng thời gian huấn luyện dài hơn (AnB+ có 450k iteration huấn luyện base + 450k fine-tune, nhưng BnB+ cũng huấn luyện 450k+450k mà không có boost tương tự).

Man-made vs Natural (Section 4.2, Hình 3 trái): mạng hướng tới nhóm "tự nhiên" (baseB và AnB nhắm natural) đạt accuracy cao hơn mạng hướng tới "nhân tạo" (có thể vì chỉ 449 lớp thay vì 551, hoặc task tự nhiên "dễ" hơn, hoặc cả hai). So sánh chuẩn hoá (trừ đi baseline riêng từng loại, Hình 3 dưới): khoảng cách transferability ở tầng cuối là **8%** cho task tương tự (random split) so với **25%** cho task khác biệt (man-made/natural) (tự tính: chênh lệch $25 - 8 = 17$ điểm phần trăm).

Random weights (Section 4.3, Hình 3 phải): hiệu năng giảm nhanh ngay từ tầng 1, 2 và gần mức ngẫu nhiên (chance level) từ tầng 3 trở đi, kém hẳn so với kết quả Jarrett et al. (2009) trên mạng nhỏ/Caltech-101. Nhưng dù vậy, **transfer từ task xa (man-made/natural) vẫn tốt hơn random weights** ở mọi tầng.

Kiểm tra overfit (Phụ lục B, Bảng S1): accuracy giảm dần đều khi giảm số ảnh/lớp từ 1300 xuống 250 (0.55176 → 0.51470 → 0.47568 → 0.38428), rồi **sụp hẳn về gần 0** (~0.001) ở 100, 50, 25, 10, 5 ảnh/lớp (không hội tụ, chỉ đạt mức ngẫu nhiên $1/1000 = 0.001$, tự tính khớp với 0.00110...0.00108 trong bảng), và tăng bất thường nhẹ ở 2 và 1 ảnh/lớp (0.00444, 0.00379, cao hơn mức ngẫu nhiên nhưng network gần như không học được gì có ý nghĩa ở vùng này). Giữa 1000 và 1300 ảnh/lớp, độ dốc gần 0 (accuracy tăng chỉ 0.010820, từ 0.54094 lên 0.55176), paper dùng điều này làm bằng chứng AlexNet trên ImageNet **không overfit nặng**, nên kết quả "random weights kém" ở Section 4.3 không chỉ là hệ quả của overfit khác biệt giữa ImageNet và Caltech-101.

5. Phân tích / ablation

Vì bản chất bài là một "ablation theo tầng" có hệ thống, phần phân tích chính đã nằm trong mục 4; ở đây tổng hợp lại các **suy luận nhân-quả** mà paper rút ra:

- **Hai nguồn gây mất transferability độc lập nhau, và mỗi nguồn trội ở vùng tầng khác nhau:** (1) specificity (đặc trưng quá chuyên biệt cho task gốc), trội ở tầng cao (6, 7); (2) optimization difficulty do cắt network ở giữa các neuron co-adapted "mong manh", trội ở tầng giữa (3, 4, 5). Đây là phát hiện mà paper nhấn mạnh là **chưa từng được quan sát trước đó**: khó khăn tối ưu có thể tệ hơn ở **giữa** mạng so với đầu hoặc cuối.
- **Fine-tune luôn phục hồi được mất mát do co-adaptation** (BnB+ ≈ baseB ở mọi tầng), nhưng với AnB+, vẫn còn một "boost" dương không giải thích được bằng co-adaptation đơn thuần, tức có thêm hiệu ứng tích cực từ việc đã "thấy" dataset A trước đó, kéo dài qua toàn bộ 450k bước fine-tune.

- **Khoảng cách task ảnh hưởng đơn điệu:** man-made/natural (khác xa) có gap tệ hơn random-split (tương tự) rõ rệt (25% vs 8% ở tầng cuối), nhưng quan hệ này chỉ được đo ở **hai điểm** (tương tự, khác biệt tối đa trong ImageNet) chứ không phải một đường cong liên tục theo "độ xa" được định lượng rõ.
- **Random weights là baseline rất yếu ở mạng lớn/dataset lớn**, ngược với quan sát trước đó của Jarrett et al. (2009) trên mạng nhỏ/Caltech-101; paper không khẳng định chắc nguyên nhân (nêu 3 khác biệt: phi tuyến relu thay abs(tanh), kích thước tầng, chỉ thử 2 tầng random ở nghiên cứu trước), chỉ cung cấp "một điểm dữ liệu mới" cho câu hỏi này.

6. Giới hạn

Do tác giả nêu:

- Mỗi điểm trong Hình 2 rất đắt (9.5 ngày GPU/mạng), nên số lần lặp lại mỗi tầng ít; để có ước lượng ổn định hơn, paper phải **gộp** nhiều tầng lại khi tính boost trung bình ở Bảng 1 (tầng 1-7, 3-7, 5-7), nghĩa là không có độ lệch chuẩn đáng tin cậy cho từng tầng riêng lẻ.
- Thí nghiệm random-weights ở tầng 3 không converge tốt (loss huấn luyện không giảm, chỉ đạt mức ngẫu nhiên), nên kết luận "random weights kém ở mạng lớn" có thể một phần do **chưa dò đúng siêu tham số** cho trường hợp đó, paper tự ghi "much better convergence may be possible with different hyperparameters".
- So sánh với Jarrett et al. (2009) không hoàn toàn công bằng: khác nonlinearity (relu vs abs(tanh)), khác kích thước tầng/số tầng random, khác dataset, nên "mâu thuẫn" giữa hai kết quả chỉ là gợi ý, chưa được kiểm chứng isolating từng yếu tố.
- Bài chỉ thử **một kiến trúc** (CNN 8 tầng kiểu AlexNet) và **một dataset gốc** (ImageNet); không có bằng chứng các kết luận (vị trí transition, mức gộp 2 hiệu ứng) tổng quát hoá sang kiến trúc/domain khác.

Người ghi chú nhận xét:

- Phép chia A/B "ngẫu nhiên" (Section 4.1) và "man-made/natural" (Section 4.2) chỉ là hai điểm rời rạc trên một "trục độ tương tự task" không được định lượng bằng số liệu cụ thể (không có thước đo khoảng cách task kiểu KL-divergence hay feature-distance); gọi là "quantify" độ giảm theo khoảng cách task (abstract của paper) có phần mạnh hơn mức paper thực sự đo được (chỉ 2 điểm, không phải một đường cong).
- Kết quả "AnB+ tốt hơn cả baseB" (boost 1.6-2.1%) được paper mô tả là "surprising" nhưng không có cơ chế giải thích rõ ràng (chỉ là giả thuyết "hiệu ứng của việc đã thấy dataset A còn lưu lại"); đây là một hiện tượng thực nghiệm cần các bài sau (ví dụ bài 28 trong lộ trình) kiểm chứng lại ở quy mô/kiến trúc khác.

7. Liên hệ với các bài khác

- **ml basics 06** (Improving neural networks by preventing co-adaptation of feature detectors / Dropout, Hinton et al. 2012): chính là nguồn thuật ngữ "co-adaptation" mà bài 27 dùng để giải thích vì sao đóng băng một phần mạng có thể phá vỡ sự phối hợp giữa các tầng; paper 27 cũng dùng dropout trong huấn luyện base network (Phụ lục A).
- **ml basics 07, 08** (ResNet, VGG): các kiến trúc CNN sâu hơn AlexNet ra đời ngay sau bài này; câu hỏi "tầng nào general/specific" mà bài 27 đặt ra tiếp tục được hỏi lại (và thường có câu trả lời khác) với các kiến trúc đó trong thực hành transfer learning sau 2015.
- **ml basics 28** (Rethinking ImageNet Pre-training, 1811.08883): bốn năm sau, He, Girshick, Dollár đặt lại chính câu hỏi "pretrain ImageNet rồi transfer có thực sự cần không" nhưng ở task detection/segmentation (Mask R-CNN/COCO) và với đủ compute/data để train from scratch; kết quả của bài 28 (train from scratch cạnh tranh được, miễn đủ iteration/normalization phù hợp) nên được đọc như một phản biện/mở rộng trực tiếp cho tiền đề "transfer luôn có lợi" mà bài 27 phần nào gợi ý qua kết quả AnB+.

- **distillation content/21_2021_deit.md** (DeiT): một trường hợp hiện đại của transfer/fine-tune cho Vision Transformer, nơi câu hỏi "đóng băng hay fine-tune bao nhiêu tầng" vẫn còn nguyên giá trị nhưng với kiến trúc attention thay CNN.

8. Gợi ý áp dụng

Khả năng chuyển giao đặc trưng phụ thuộc khoảng cách giữa tác vụ nguồn và đích cũng như tầng được chuyển. Nên so sánh đóng băng và fine-tune các tầng dưới cùng một quy trình đánh giá.

9. Thí nghiệm gợi ý

1. **BnB vs AnB thu nhỏ:** dùng một CNN nhỏ (ví dụ 5-6 tầng) trên CIFAR-100 chia thành hai nửa 50 lớp A/B (tương tự Hình 1 của paper nhưng quy mô nhỏ); đo accuracy khi cắt tại từng tầng, đóng băng vs fine-tune, tái tạo đúng 4 đường baseB/BnB/BnB+/AnB/AnB+ và kiểm tra có xuất hiện "vùng giữa giảm mạnh" tương tự tầng 3-5 của paper không.
2. **Boost của transfer+fine-tune so với train from scratch:** với một checkpoint OCR nhỏ đã có, so sánh fine-tune toàn bộ (giống AnB+) với train from scratch trên đúng tập OCR đích (cùng tổng số bước); đo CER/accuracy cuối, kỳ vọng: nếu có hiệu ứng tương tự bài 27, bản fine-tune từ checkpoint khác domain vẫn nhích hơn train from scratch dù dữ liệu cuối giống nhau.
3. **Random-weights baseline cho tầng cắt:** với một vài tầng của encoder OCR, thay trọng số đã train bằng trọng số ngẫu nhiên (cùng kiến trúc), đóng băng, rồi train lại phần còn lại; so với giữ trọng số gốc đóng băng. Đo khoảng cách hiệu năng; kỳ vọng theo paper: khoảng cách này lớn, xác nhận trọng số đã học (dù từ task khác) tốt hơn hẳn ngẫu nhiên.

10. Tóm tắt công thức / khái niệm

Khái niệm	Ý nghĩa / số liệu
BnB (selffer, frozen)	copy n tầng từ baseB, đóng băng, train phần còn lại trên B; đối chứng
AnB (transfer, frozen)	copy n tầng từ baseA, đóng băng, train phần còn lại trên B; đo transferability
BnB+, AnB+	như trên nhưng fine-tune toàn mạng
2 nguồn giảm hiệu năng	(1) specificity của feature; (2) fragile co-adaptation khi cắt mạng
Vùng trội	specificity trội ở tầng cao (6,7); co-adaptation trội ở tầng giữa (3-5)
Boost AnB+ so với baseB	1.6% (tầng 1-7), 1.8% (3-7), 2.1% (5-7), trung bình (Bảng 1)
Gap tầng cuối, task tương tự vs khác biệt	8% (random A/B split) vs 25% (man-made/natural split)
Random weights	gần chance level từ tầng 3+ trên AlexNet/ImageNet, kém hơn transferred weights ở mọi tầng
Ngưỡng overfit kiểm tra	accuracy gần như bằng 0 khi còn ≤100 ảnh/lớp; tăng chậm ($+0.0108$) giữa 1000 và 1300 ảnh/lớp
Man-made/natural split	522 lớp "artifact" + 410 lớp "organism" (WordNet) + 68 lớp gán tay = 551/449

11. Kết luận ngắn

Bằng một thiết kế đối chứng cẩn thận (BnB/AnB, đóng băng/fine-tune), paper tách được hai nguyên nhân độc lập khiến transfer feature kém hiệu quả: tính chuyên biệt (specificity, trội ở tầng cao) và khó tối ưu do phá vỡ co-adaptation giữa các tầng liên tiếp (trội ở tầng giữa). Transfer kém hiệu quả hơn khi task nguồn/đích khác biệt nhiều (gap 25% so với 8% ở tầng cuối), nhưng vẫn tốt hơn hẳn trọng số ngẫu nhiên ở mọi tầng. Phát

hiện bất ngờ nhất: fine-tune sau khi transfer, dù từ gần như bất kỳ số tầng nào, cho generalization tốt hơn train from scratch (boost 1.6-2.1%), một hiệu ứng lưu lại qua toàn bộ quá trình fine-tune. Các kết luận này đặt nền cho câu hỏi "khi nào cần pretrain, khi nào train from scratch đủ tốt" mà bài 28 (Rethinking ImageNet Pre-training) khảo sát lại ở quy mô/task khác bốn năm sau.

Nguồn gốc: https://arxiv.org/abs/1411.1792

Bài 28

Rethinking ImageNet Pre-training (2018)

Tác giả: Kaiming He, Ross Girshick, Piotr Dollár (Facebook AI Research, FAIR).
Venue / arXiv: arXiv:1811.08883 (v1, 21/11/2018); paper không nêu venue hội nghị/tạp chí trên trang đầu (chỉ có nhãn arXiv). (Ngoài paper, kiến thức chung: bản hoàn chỉnh sau đó công bố tại ICCV 2019, không được ghi trong văn bản PDF đọc ở đây.)
Nguồn: https://arxiv.org/abs/1811.08883
Chủ đề: kiểm định lại bằng thực nghiệm có kiểm soát câu hỏi "có cần ImageNet pre-training để đạt độ chính xác cao trên object detection/instance segmentation/keypoint detection không?", bằng cách huấn luyện Mask R-CNN/Faster R-CNN từ khởi tạo ngẫu nhiên trên COCO/VOC, chỉ thay đổi normalization và độ dài lịch huấn luyện so với baseline pre-trained.
Vị trí trong lộ trình: Đọc sau bài 27 về transferability để khảo sát khi nào pretraining hữu ích.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Từ khi R-CNN (Girshick et al. 2014) và OverFeat chứng minh fine-tune từ mạng pre-train trên ImageNet classification giúp object detection, "pre-train rồi fine-tune" trở thành paradigm mặc định cho hầu hết thị giác máy tính (detection, segmentation, action recognition). Xu hướng gần đây còn đẩy quy mô pre-training lớn hơn: ImageNet-5k (6x ImageNet), JFT (300x), và Instagram data của Mahajan et al. (2018, 3000x ImageNet). Nhưng paper chỉ ra: lợi ích của pre-training quy mô cực lớn này trên object detection **nhỏ và giảm dần** (ví dụ chỉ +1.5 AP trên COCO khi dùng dữ liệu pre-train lớn hơn ImageNet 3000 lần); "liệu con đường này có 'giải quyết' được computer vision hay không" là điều còn phải hoài nghi.

Từ đó, paper đặt câu hỏi ngược hẳn: thay vì tiếp tục pre-train trên dữ liệu ngày càng lớn, điều gì xảy ra nếu **bỏ hẳn** pre-training và huấn luyện trực tiếp trên COCO từ khởi tạo ngẫu nhiên ("from scratch")? Trước paradigm pre-train/fine-tune, các detector cũ (ví dụ Rowley et al. 1996 cho face detection, Matan et al. 1992) vốn đã được train from scratch, một sự thật mà bài báo nói "ngày nay bị bỏ quên phần nào"; nghiên cứu sau đó của Agrawal et al. (2014, [1]) từng cho thấy train from scratch trên nhiều dữ liệu detection hơn có thể đạt 90% độ chính xác của fine-tuning, báo hiệu trước kết quả của bài này. Một số công trình (DSOD của Shen et al. 2017, CornerNet của Law & Deng 2018, DetNet) đã thiết kế kiến trúc **chuyên biệt** cho train-from-scratch, nhưng bài báo này khác: tập trung vào các kiến trúc **không chuyên biệt hoá** (Mask R-CNN, Faster R-CNN nguyên bản, không được thiết kế riêng cho train from scratch).

2. Ý tưởng chính

Bài báo cho thấy chỉ cần **hai thay đổi tối thiểu** so với baseline Mask R-CNN chuẩn (vốn được tối ưu cho fine-tuning từ pre-trained weights), ta có thể train from scratch đạt độ chính xác **không kém** (thậm chí có lúc tốt hơn) so với bản pre-trained, trên nhiều kịch bản khác nhau:

1. **Normalization phù hợp:** thay Batch Normalization "đóng băng" (frozen BN, cách dùng phổ biến khi fine-tune vì batch nhỏ do ảnh detection độ phân giải cao) bằng Group Normalization (GN) hoặc Synchronized BN (SyncBN) để huấn luyện ổn định ngay từ đầu không cần thống kê BN đã học trước.
2. **Huấn luyện đủ lâu:** vì mô hình from-scratch phải học cả đặc trưng mức thấp/trung (biên, texture, vốn đã "có sẵn" nhờ pre-training) lẫn mức cao, nó cần nhiều iteration hơn để hội tụ (ví dụ lịch 5x-6x so với lịch 2x chuẩn cho fine-tuning).

Với hai thay đổi đó, paper chứng minh train-from-scratch cạnh tranh được qua rất nhiều điều kiện: nhiều kiến trúc backbone (ResNet-50/101, ResNeXt-152, cả VGG-16 không cần BN/GN), nhiều task/metric (bbox AP, mask AP, keypoint AP, nhiều ngưỡng IoU), mô hình lớn hơn (không overfit), và dữ liệu ít hơn (tới 10% COCO). Kết luận ngược hẳn "common wisdom": **ImageNet pre-training giúp hội tụ nhanh hơn ở giai đoạn đầu, nhưng không tự động cải thiện regularization hay độ chính xác cuối cùng**, trừ khi dữ liệu target quá nhỏ.

3. Phương pháp chi tiết

3.1 Normalization (mục 3.1)

Detector huấn luyện với ảnh độ phân giải cao, nên batch size (số ảnh/GPU) nhỏ hơn nhiều so với classifier; BN với batch nhỏ suy giảm độ chính xác nặng. Khi fine-tune từ pre-trained, người ta thường "đóng băng" BN (dùng lại batch statistics đã học từ ImageNet) để lách vấn đề này, nhưng đóng băng BN **không hợp lệ** khi train from scratch (không có thống kê nào để đóng băng). Paper dùng hai giải pháp thay thế:

- **Group Normalization (GN,** Wu & He 2018**):** tính toán độc lập với chiều batch, nên độ chính xác không nhạy với batch size.
- **Synchronized BN (SyncBN):** BN với batch statistics đồng bộ trên nhiều GPU, giúp tăng batch size hiệu dụng.

Cả hai đều giúp detector train from scratch được. Với VGG-16 (không cần BN/GN như ResNet), chỉ cần khởi tạo chuẩn hoá đúng cách (MSRA initialization, He et al. 2015) là đủ để train từ đầu ổn định.

3.2 Độ dài lịch huấn luyện (mục 3.2)

Pre-training ImageNet điển hình dùng hơn một triệu ảnh, lặp 100 epoch; ngoài thông tin ngữ nghĩa, mô hình pre-trained còn "miễn học lại" các đặc trưng mức thấp (biên, texture) khi fine-tune (paper ghi rõ: thực hành phổ biến [8, 36] là đóng băng vài tầng conv đầu khi fine-tune). Mô hình from-scratch phải học cả mức thấp và mức cao nên cần nhiều iteration hơn để hội tụ. Paper định lượng "số lượng mẫu đã thấy" theo ba cách: số ảnh, số instance, số pixel đã xử lý qua toàn bộ quá trình huấn luyện; dù from-scratch train nhiều hơn gấp 3 lần số iteration so với fine-tuning trên COCO, nếu tính theo số ảnh thì vẫn ít hơn fine-tuning (vì 1.28 triệu ảnh ImageNet x 100 epoch áp đảo), chỉ khi tính theo **số pixel** thì hai bên mới gần nhau hơn (do detector dùng ảnh độ phân giải cao hơn ImageNet classifier).

3.3 Thiết lập thực nghiệm (mục 4)

Baseline theo đúng Mask R-CNN trong codebase Detectron, chỉ thay đổi normalization và số iteration. Backbone: ResNet hoặc ResNeXt + Feature Pyramid Network (FPN); Region Proposal Network huấn luyện end-to-end cùng Mask R-CNN. Lịch gốc: "1x schedule" = 90k iteration, "2x" = 180k; bài này mở rộng tới "6x" = 540k iteration, luôn giảm learning rate 10 lần ở 60k iteration cuối và 20k iteration cuối (bất kể tổng độ dài). Siêu tham số khác: learning rate khởi đầu 0.02 (có linear warm-up), weight decay 0.0001, momentum 0.9, 8 GPU, mini-batch 2 ảnh/GPU. **Tự tính:** batch hiệu dụng $= 8 \times 2 = 16$ ảnh/iteration. Augmentation mặc định: chỉ lật ngang (horizontal flip) lúc train, không augment lúc test; ảnh scale cạnh ngắn 800px. Một số thí nghiệm sau dùng thêm scale augmentation train-time (cạnh ngắn lấy ngẫu nhiên trong $[640, 800]$ hoặc $[512, 800]$) và test-time multi-scale augmentation.

3.4 Quy ước "schedule" dùng cho VGG-16

Với VGG-16 (mục 5.1 cuối), paper báo cáo lịch "9x" cho pre-trained và "11x" cho from-scratch. **Tự tính** (theo định nghĩa 1x = 90k iteration ở mục 3.3): 9x $= 9 \times 90$k $= 810.000$ iteration; 11x $= 11 \times 90$k $= 990.000$ iteration.

4. Thí nghiệm và kết quả chính

Train from scratch bắt kịp accuracy (mục 5.1, Bảng 1, Hình 1/3/4): với Mask R-CNN + GN, bbox AP trên COCO val2017 theo lịch 2x-6x:

Backbone	Khởi tạo	2x	3x	4x	5x	6x
R50	random init	36.8	39.5	40.6	40.7	41.3
R50	w/ pre-train	40.3	40.8	40.9	40.9	41.1
R101	random init	38.2	41.0	41.8	42.2	42.7
R101	w/ pre-train	41.8	42.3	42.3	41.9	42.2

Ở lịch 2x, random init luôn kém hẳn pre-train (chưa hội tụ); từ 5x-6x, random init **bắt kịp hoặc vượt** pre-train. Với SyncBN (R50, 6x): random init 39.3 vs pre-train 39.0.

So nhiều metric cùng lúc, lấy lịch tốt nhất mỗi mô hình (Bảng 2):

Backbone	Khởi tạo	AP(bbox)	AP50	AP75	AP(mask)	AP50(mask)	AP75(mask)
R50	random init	41.3	61.8	45.6	36.6	59.0	38.9
R50	pre-train	41.1	61.7	44.6	36.4	58.5	38.7
R50	chênh lệch	+0.2	+0.1	**+1.0**	+0.2	+0.5	+0.2
R101	random init	42.7	62.9	47.0	37.6	59.9	39.7
R101	pre-train	42.3	62.6	46.2	37.2	59.7	39.7
R101	chênh lệch	+0.4	+0.3	**+0.8**	+0.4	+0.2	0.0

Random init nhích hơn pre-train ở hầu hết metric, đặc biệt rõ ở AP_{75} (ngưỡng IoU cao, đòi hỏi định vị chính xác): chênh lệch +1.0 (R50) và +0.8 (R101) AP. **Tự tính** (tăng tương đối): với R50, AP_{75} tăng $(45.6 - 44.6)/44.6 \approx 2.24\%$ so với pre-train.

Mô hình lớn hơn, tăng cường (Bảng 3, Hình 5), ResNeXt-152 (X152, ~4x FLOPs của R101), train from scratch:

Cấu hình	bbox AP	AP50	AP75	mask AP	AP50(mask)	AP75(mask)
R101, +train aug	45.0	65.7	49.3	39.5	62.5	42.1
X152, +train aug	46.4	67.1	51.1	40.5	63.9	43.4
X152, +cascade	48.6	66.8	52.9	41.4	64.2	44.6
X152, +test aug	50.9	68.7	55.4	43.2	66.1	46.8

X152 random-init không overfit dù lớn hơn hẳn R101; với pre-training, cùng X152 chỉ đạt 50.3/42.5 (bbox/mask AP) trên val2017, **kém hơn** bản from-scratch (50.9/43.2). Nộp bài thi COCO 2018: 51.3 bbox AP / 43.6 mask AP (test-challenge), ngang mức của người thắng COCO 2017 (50.5 bbox AP), và là con số cao nhất trong nhóm "không dùng ImageNet pre-training, một mô hình đơn" (so với DSOD 29.3 AP dùng kiến trúc chuyên biệt, CornerNet 42.1 AP).

Keypoint detection (Hình 6): random init bắt kịp **nhanh hơn**, không cần tăng iteration (chỉ 2x-3x, không cần 5x-6x như detection); kết quả 2x: 65.6 (random init) vs 65.5 (pre-train) keypoint AP, tức pre-training "có rất ít lợi ích" cho task nhạy với định vị không gian chi tiết này.

VGG-16, không BN/GN (mục 5.1 cuối): pre-train đạt tối đa 35.6 bbox AP sau lịch 9x (810k iteration, tự tính); random init đạt tối đa 35.2 bbox AP sau lịch 11x (990k iteration, tự tính); huấn luyện dài hơn nữa

làm giảm nhẹ AP ở cả hai trường hợp. Xác nhận kết luận tổng quát hoá sang kiến trúc không cần activation normalization.

Train from scratch với ít dữ liệu hơn (mục 5.2):

Số ảnh train COCO	random init	pre-train	Ghi chú
35k (~1/3 COCO)	36.3 AP (lịch 6x, dùng hyperparam tối ưu cho pre-train)	36.1 AP	cả hai gần tương đương, kể cả khi hyperparam bị "thiên vị" cho bên pre-train
10k (~1/10 COCO)	25.9 AP (220k iteration)	26.0 AP (60k iteration, giảm nhẹ nếu train thêm)	tương đương
3.5k	9.3 AP	16.0 AP	pre-train vượt rõ
1k	3.5 AP (hoặc 5.4 AP nếu grid-search riêng cho from-scratch)	9.9 AP	pre-train vượt rõ; loss train thì from-scratch vẫn bắt kịp, nhưng validation AP thấp hơn nhiều (dấu hiệu overfit nặng)

Điểm "gãy" (breakdown point) nằm trong khoảng 3.5k-10k ảnh COCO. Với **PASCAL VOC** (15k ảnh train, trung bình 2.3 instance/ảnh và 20 lớp, so với COCO ~7 instance/ảnh và 80 lớp): pre-train đạt 82.7 mAP (18k iteration); from-scratch chỉ đạt 77.6 mAP (144k iteration) và **không bắt kịp** dù train lâu hơn. Paper cho rằng số instance/lớp ít tương đương hiệu ứng "dữ liệu không đủ", giải thích vì sao VOC không bắt kịp được như COCO.

5. Phân tích / ablation

- **Hai hiện tượng độc lập phân biệt rõ theo lượng dữ liệu:** với đủ dữ liệu (≥10k ảnh COCO), pre-training chỉ còn vai trò "tăng tốc hội tụ sớm", không cải thiện regularization hay accuracy cuối; với dữ liệu quá ít (≤3.5k ảnh, hoặc VOC với mật độ instance thấp), pre-training vẫn mang lợi ích rõ ràng không bù được bằng train lâu hơn.
- **Độ nhạy theo loại metric/task:** AP_{75} (độ chính xác định vị cao) và keypoint AP (cần định vị không gian mịn) là hai nơi train-from-scratch có lợi thế hoặc bắt kịp nhanh nhất; paper diễn giải: pre-training phân loại (classification) không cung cấp thông tin định vị không gian rõ ràng, nên ít giúp các task/metric nhạy với localization.
- **Augmentation và kiến trúc mạnh hơn không xoá bỏ hiện tượng:** scale augmentation, Cascade R-CNN, test-time augmentation, và cả việc tăng kích thước backbone (X152, ~4x FLOPs của R101) đều giữ nguyên hoặc củng cố kết luận "from-scratch không kém", chứng tỏ hiện tượng không phải do backbone quá nhỏ hoặc thiếu tăng cường dữ liệu.
- **Grid-search hyperparameter là yếu tố quan trọng để so sánh công bằng:** ở thí nghiệm 35k ảnh, dùng thẳng hyperparameter tối ưu cho 118k ảnh khiến cả pre-train lẫn from-scratch đều overfit (Hình 7 trái); chỉ sau khi re-grid-search riêng cho trường hợp pre-train (và áp lại chính hyperparameter đó cho from-scratch, vốn **thiên vị** cho bên pre-train) thì mới thấy from-scratch vẫn bắt kịp (36.3 vs 36.1 AP) mặc dù bị đặt vào thế bất lợi hơn.

6. Giới hạn

Do tác giả nêu:

- Không kỳ vọng xu hướng "train from scratch bắt kịp" kéo dài tới dữ liệu target cực nhỏ: breakdown point rõ ràng tồn tại giữa 3.5k và 10k ảnh COCO (và với VOC dù 15k ảnh nhưng ít instance/lớp).

- ImageNet pre-training vẫn "chắc chắn có ích" (phần Discussion, mục 6) với các lý do khác ngoài accuracy cuối: tăng tốc nghiên cứu (không phải trả chi phí pre-train lại mỗi lần), giúp vượt qua khó khăn tối ưu khi thiếu kỹ thuật normalization/initialization tốt, và đã cho phép cộng đồng thấy tiến bộ trước khi có dữ liệu quy mô lớn (ví dụ trên VOC một thời gian dài).
- Kiến trúc dùng trong bài **không được thiết kế riêng** cho train from scratch (khác DSOD/CornerNet/DetNet); các kiến trúc chuyên biệt khác có thể train from scratch tốt hơn nữa, nhưng đó không phải trọng tâm so sánh của bài.
- Lợi ích của pre-training quy mô cực lớn (JFT 300x, Instagram 3000x) trên chính task classification là có thật và được paper ghi nhận; bài chỉ tập trung vào detection/segmentation/keypoint trên COCO/VOC, không mở rộng sang các bài toán khác (ví dụ chính classification, hay domain ảnh phi tự nhiên).

Người ghi chú nhận xét:

- Có một chỗ số liệu lệch nhẹ giữa văn bản và chú thích hình trong thí nghiệm 1k ảnh (mục 5.2): đoạn văn chính viết "the from scratch model's **3.5** AP", nhưng chú thích Hình 8 viết "lower validation accuracy (**3.4** AP) than the pre-training counterpart (9.9 AP)". Đây là một mâu thuẫn số liệu nhỏ trong chính paper (3.5 vs 3.4), không ảnh hưởng tới kết luận chung (cả hai đều thấp hơn hẳn 9.9 AP của pre-train) nhưng nên nêu rõ khi trích dẫn số chính xác.
- Figure 2 (biểu đồ số ảnh/instance/pixel đã thấy) bị lỗi định dạng nặng khi trích xuất văn bản PDF (các số liệu bị xáo trộn vị trí), nên phần 3.2 chỉ mô tả **định tính** kết luận của hình đó (from-scratch train nhiều iteration hơn nhưng vẫn thấy ít ảnh hơn, trừ khi tính theo pixel) chứ không trích số liệu cụ thể từ hình, để tránh suy diễn sai số liệu đã bị xáo trộn.

7. Liên hệ với các bài khác

- **ml basics 27** (How transferable are features in deep neural networks?, 1411.1792): tiền đề trực tiếp; bài 27 đo "tầng nào general/specific" trên classification ImageNet A/B split, còn bài 28 hỏi "có cần transfer/pretrain gì không" trên detection/segmentation COCO; kết quả bài 28 (train from scratch bắt kịp nếu đủ data) là một phản ví dụ tinh vi cho giả định ngầm "transfer luôn có lợi" mà nhiều kết quả AnB+ của bài 27 gợi ý (dù hai bài đo trên task khác nhau nên không mâu thuẫn trực tiếp).
- **ml basics 07** (Deep Residual Learning, ResNet): backbone chính (R50/R101) dùng trong toàn bộ thí nghiệm của bài 28, kèm ResNeXt-152 (Xie et al. 2017) là bản mở rộng nhóm (group) của ResNet.
- **ml basics 29** (Self-training with Noisy Student, 1911.04252): cùng câu hỏi lớn "vai trò của ImageNet/dữ liệu quy mô lớn trong việc cải thiện mô hình thị giác", nhưng theo hướng ngược lại: bài 29 dùng pseudo-label từ dữ liệu không nhãn quy mô cực lớn để **cải thiện thêm** mô hình đã pre-train, còn bài 28 cho thấy chính việc pre-train có thể bỏ qua được nếu đủ dữ liệu có nhãn của target task.
- **opt 38** (Batch Normalization) và **opt 39** (Layer Normalization) (nếu đã có trong "optimization content/"): nền tảng kỹ thuật cho vấn đề normalization (BN với batch nhỏ) mà mục 3.1 của bài 28 giải quyết bằng Group Normalization.

8. Gợi ý áp dụng

Pretraining là một lựa chọn thực nghiệm chứ không phải mặc định luôn cải thiện kết quả. Khi so sánh với huấn luyện từ đầu, cần kiểm soát lịch huấn luyện, dữ liệu và ngân sách tính toán.

9. Thí nghiệm gợi ý

1. **Tái tạo nhỏ "normalization + train dài hơn" trên bài toán OCR:** lấy một mô hình nhận dạng ảnh nhỏ (ví dụ CNN/ViT nhỏ cho phân loại ký tự hoặc dòng văn bản), so sánh train from scratch (với LayerNorm/GroupNorm đúng) vs fine-tune từ checkpoint pretrain, ở hai độ dài lịch (ngắn và dài gấp 3-5 lần); đo accuracy theo số iteration, kỳ vọng tái tạo đúng dạng đường cong Hình 1 của paper (from-scratch chậm hội tụ ban đầu nhưng bắt kịp nếu đủ lâu).

2. **So sánh metric "nội dung" vs metric "định vị" sau cắt tỉa:** nếu OCR pipeline có cả bước nhận diện ký tự (giống classification) và bước định vị dòng/bounding-box (giống localization), đo riêng mức suy giảm của hai loại metric này khi mô hình bị cắt rồi phục hồi với vs không với trọng số gốc đóng băng; kỳ vọng theo bài 28: metric định vị ít hưởng lợi từ giữ trọng số pretrain gốc hơn metric nội dung.

10. Tóm tắt công thức / khái niệm

Khái niệm	Nội dung / số liệu
Hai thay đổi tối thiểu	(1) GN hoặc SyncBN thay frozen BN; (2) huấn luyện dài hơn (5x-6x so với 2x chuẩn)
Lịch "kx"	$k \times 90$k iteration (1x = 90k; ví dụ 6x = 540k, 9x = 810k tự tính, 11x = 990k tự tính)
Batch hiệu dụng	8 GPU × 2 ảnh/GPU = 16 ảnh/iteration (tự tính)
Bbox AP tốt nhất (Bảng 2)	R50 random init 41.3 vs pre-train 41.1; R101 random init 42.7 vs pre-train 42.3
AP_{75} chênh lệch rõ nhất	R50: +1.0 AP (random hơn pre-train); tăng tương đối ~2.24% (tự tính)
X152 (COCO 2018)	từ scratch: 50.9/43.2 (val), 51.3/43.6 (test-challenge); pre-train cùng model: 50.3/42.5 (val, kém hơn)
Keypoint AP, 2x schedule	random init 65.6 vs pre-train 65.5 (gần như không khác biệt)
Breakdown điểm dữ liệu COCO	giữa 3.5k ảnh (scratch 9.3 vs pre-train 16.0 AP) và 10k ảnh (scratch 25.9 ≈ pre-train 26.0 AP)
VOC (15k ảnh, ít instance/lớp)	pre-train 82.7 mAP vs scratch 77.6 mAP, scratch không bắt kịp
Trả lời 4 câu hỏi của paper	pretrain không cần thiết nếu đủ data; pretrain vẫn hữu ích (research cycle, optimization); vẫn cần "big data" nhưng nên là data của target task; vẫn nên theo đuổi universal representation

11. Kết luận ngắn

Chỉ với hai thay đổi (normalization phù hợp cho batch nhỏ, và huấn luyện đủ lâu để bù phần đặc trưng mức thấp/trung không còn được "cho sẵn" từ pre-training), Mask R-CNN/Faster R-CNN huấn luyện từ khởi tạo ngẫu nhiên trên COCO đạt độ chính xác không kém, và ở một số metric/mô hình còn vượt, bản fine-tune từ ImageNet pre-training, qua nhiều backbone, nhiều task (detection, segmentation, keypoint), và dữ liệu ít tới 10% COCO. Lợi ích thật của ImageNet pre-training, theo bài báo, chủ yếu là tốc độ hội tụ sớm và tiện lợi nghiên cứu, không phải accuracy trần hay regularization, trừ khi dữ liệu target quá khan hiếm (dưới khoảng 10k ảnh COCO, hoặc tập dữ liệu có mật độ instance/lớp thấp như VOC). Kết quả này thúc đẩy suy nghĩ lại việc đầu tư thu thập dữ liệu/compute cho đúng target task thay vì luôn mặc định cần một bước pre-training quy mô lớn trên dữ liệu tổng quát.

Nguồn gốc: https://arxiv.org/abs/1811.08883

Bài 29

Self-training with Noisy Student improves ImageNet classification (2019)

Tác giả: Qizhe Xie, Minh-Thang Luong, Eduard Hovy, Quoc V. Le (Google Research, Brain Team; Eduard Hovy thuộc Carnegie Mellon University).
Venue / arXiv: arXiv:1911.04252 (v4, 19/06/2020); paper không nêu venue hội nghị/tạp chí trong văn bản đọc ở đây (chỉ có nhãn arXiv). (Ngoài paper, kiến thức chung: bản hoàn chỉnh công bố tại CVPR 2020, không nằm trong văn bản PDF đọc ở đây.)
Nguồn: https://arxiv.org/abs/1911.04252
Chủ đề: self-training/semi-supervised learning quy mô lớn cho ImageNet classification: dùng một teacher (EfficientNet) sinh pseudo-label trên 300 triệu ảnh JFT không nhãn, sau đó train một student lớn hơn-hoặc-bằng với noise injection (RandAugment, dropout, stochastic depth) trên dữ liệu có nhãn cộng pseudo-nhãn, lặp lại nhiều vòng bằng cách biến student thành teacher mới.
Vị trí trong lộ trình: Đọc cùng các bài 30–33 về những lựa chọn dữ liệu và mục tiêu huấn luyện.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Các mô hình thị giác máy tính SOTA (state-of-the-art) vẫn chủ yếu dùng supervised learning, cần một corpus ảnh có nhãn rất lớn để đạt độ chính xác cao. Điều này bỏ lỡ lượng ảnh không nhãn lớn hơn nhiều đang có sẵn, lượng ảnh có thể giúp cải thiện cả accuracy và robustness (khả năng tổng quát hoá khi dữ liệu kiểm thử bị lệch phân phối, out-of-distribution).

Trước bài báo này, SOTA trên ImageNet (ResNeXt-101 WSL, FixRes ResNeXt-101 WSL) đạt được bằng cách pretrain trên 3,5 tỷ (3.5B) ảnh Instagram được gắn tag yếu (weakly labeled). Bài báo đặt câu hỏi: có thể đạt hoặc vượt kết quả đó bằng một lượng ảnh không nhãn nhỏ hơn nhiều (300 triệu ảnh JFT, không có nhãn gắn kèm được sử dụng) hay không, thông qua self-training thay vì weakly-supervised pretraining?

Self-training là một kỹ thuật cũ (paper dẫn Scudder 1965, Yarowsky 1995, Riloff 1996): dùng teacher train trên dữ liệu có nhãn để sinh pseudo-label cho dữ liệu không nhãn, rồi dùng cả hai loại dữ liệu để train tiếp. Một hướng liên quan khác là consistency training/pseudo-labeling (ép mô hình dự đoán nhất quán qua các phiên bản augment của cùng một ảnh, ví dụ UDA của cùng nhóm tác giả) và distillation (Hinton et al. 2015, dùng soft target từ một mô hình đã train để "dạy" mô hình khác). Theo paper, các phương pháp dựa trên consistency regularization và pseudo-labeling hoạt động kém hơn trên ImageNet trong các thử nghiệm sơ bộ của nhóm, vì chúng dùng chính mô hình đang train (độ chính xác thấp, entropy cao ở giai đoạn đầu) để sinh pseudo-label, khiến regularization kéo mô hình về phía dự đoán entropy cao, cản trở việc đạt độ chính xác tốt; cách khắc phục thường cần thêm siêu tham số (entropy minimization, ramping schedule, confidence filtering) làm phương pháp khó dùng ở quy mô lớn. Khung teacher-student self-training (có teacher riêng, train hội tụ trước) được bài báo cho là phù hợp hơn với ImageNet vì có thể train một teacher tốt trên dữ liệu có nhãn sẵn có.

2. Ý tưởng chính

Noisy Student Training mở rộng self-training và distillation theo hai hướng mới:

1. **Student lớn hơn-hoặc-bằng teacher** (ngược với distillation thường dùng student nhỏ hơn để nhanh hơn teacher): mục tiêu không phải compress mà "mở rộng tri thức" (knowledge expansion), cho student

đủ capacity để học từ một tập dữ liệu lớn hơn (có nhãn + pseudo-nhãn).

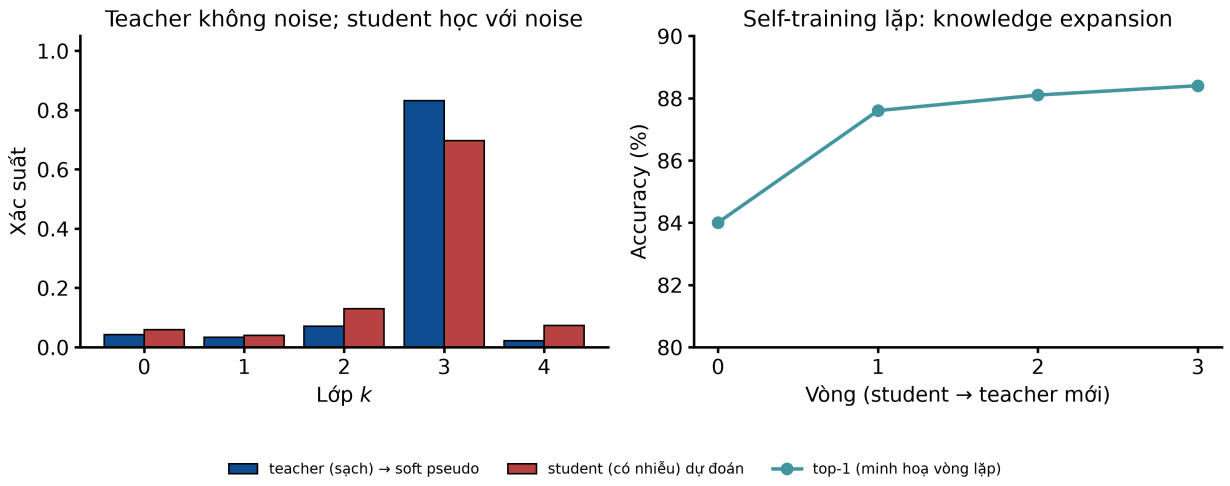
2. **Thêm noise vào student khi train** (input noise: RandAugment; model noise: dropout, stochastic depth), nhưng **không** thêm noise vào teacher khi teacher sinh pseudo-label (teacher "sạch", nhận ảnh sạch, sinh nhãn sạch). Nhờ vậy student bị buộc phải dự đoán đúng nhãn ngay cả khi đầu vào bị augment mạnh hoặc một phần mạng bị dropout/stochastic-depth, tức là học "khó" hơn thay vì chỉ copy y nguyên teacher.

Trực giác vì sao noise cần thiết: nếu dùng soft pseudo-label và student học y hệt teacher, cross-entropy loss trên dữ liệu không nhãn sẽ tiến về 0, tín hiệu học biến mất, student không thể học tốt hơn teacher. Paper giả thuyết rằng noise ngăn điều này theo hai cách: (a) data augmentation ép student dự đoán nhất quán qua các phiên bản bị biến đổi của cùng ảnh (giống UDA); (b) khi teacher sinh nhãn, do không bị dropout/stochastic-depth, nó hành xử như một ensemble ở thời điểm suy luận, còn student (có noise) hành xử như một mô hình đơn, nên bị buộc phải "bắt chước" một ensemble mạnh hơn chính nó.

Một kỹ thuật phụ quan trọng là **lọc và cân bằng dữ liệu** (data filtering và balancing): chỉ giữ các ảnh không nhãn mà teacher tự tin (vì ảnh tự tin thấp thường là ảnh ngoài-miền, out-of-domain so với ImageNet), và cân bằng số ảnh mỗi lớp (nhân bản ảnh ở lớp thiếu, giữ ảnh tự tin cao nhất ở lớp dư) để phân phối lớp giống dữ liệu có nhãn.

Toàn bộ quá trình được lặp lại: student vừa train xong trở thành teacher mới, sinh pseudo-label mới, train một student mới (thường lớn hơn hoặc cùng cỡ), tạo một chuỗi cải thiện tăng dần.

3. Phương pháp chi tiết



Teacher sạch → soft pseudo; student có nhiễu; vòng self-training minh hoạ tăng accuracy (minh hoạ, tự tính)

3.1. Thuật toán (Algorithm 1 trong paper)

Cho dữ liệu có nhãn $\{(x_1, y_1), \ldots, (x_n, y_n)\}$ và dữ liệu không nhãn $\{\tilde{x}_1, \ldots, \tilde{x}_m\}$:

1. Học teacher θ_t^* tối thiểu hoá cross-entropy trên dữ liệu có nhãn:

$$\frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f^{\text{noised}}(x_i, \theta_t))$$

   (ở bước huấn luyện teacher, noise vẫn được dùng như augmentation thông thường cho dữ liệu có nhãn, nhưng khi *sinh* pseudo-label ở bước 2 thì không dùng noise).

2. Dùng teacher *không noise* để sinh pseudo-label (soft hoặc hard) cho ảnh không nhãn *sạch* (không bị

biến đổi):

$$\tilde{y}_i = f(\tilde{x}_i, \theta_t^*), \quad \forall i = 1, \ldots, m$$

3. Học một student θ_s^* lớn hơn-hoặc-bằng teacher, tối thiểu hoá tổng cross-entropy trên dữ liệu có nhãn và không nhãn, nhưng **noise chỉ thêm vào student**:

$$\frac{1}{n}\sum_{i=1}^{n} \ell(y_i, f^{\text{noised}}(x_i, \theta_s)) + \frac{1}{m}\sum_{i=1}^{m} \ell(\tilde{y}_i, f^{\text{noised}}(\tilde{x}_i, \theta_s))$$

4. Huấn luyện lặp: dùng student vừa học làm teacher mới, quay lại bước 2.

Paper nhận thấy cả pseudo-label dạng soft (phân phối liên tục) và hard (one-hot) đều hoạt động tốt; soft label có lợi hơn một chút cho dữ liệu không nhãn ngoài-miền, nên paper mặc định báo cáo kết quả với soft pseudo-label.

3.2. Dữ liệu

- **Có nhãn:** ImageNet 2012 ILSVRC classification (1000 lớp).
- **Không nhãn:** JFT, khoảng 300 triệu ảnh (có nhãn gốc nhưng paper bỏ qua, coi như không nhãn); loại bỏ các ảnh trùng với ImageNet validation set.
- **Pipeline lọc/cân bằng:** dùng EfficientNet-B0 train trên ImageNet để dự đoán nhãn cho ảnh JFT; chỉ giữ ảnh có độ tự tin (confidence) của nhãn dự đoán lớn hơn 0.3; với mỗi lớp giữ tối đa 130.000 ảnh tự tin nhất; lớp nào thiếu thì nhân bản ngẫu nhiên để đủ 130K. Kết quả: tổng 130 triệu ảnh dùng để train student (có trùng lặp do nhân bản), trong đó chỉ có 81 triệu ảnh là duy nhất (không trùng lặp) trong số 130 triệu đó. Paper nói rõ không tune kỹ các hyperparameter này (ngưỡng 0.3, giới hạn 130K) vì phương pháp "rất bền (robust)" với chúng, nhưng không đưa thí nghiệm định lượng cho khẳng định "robust" này.
- Paper cũng thử một dataset công khai thay thế, YFCC100M, để các nhà nghiên cứu khác có thể so sánh công bằng (JFT không công khai).

3.3. Kiến trúc và chi tiết huấn luyện

Kiến trúc nền là EfficientNet (Tan & Le, 2019). Paper scale EfficientNet-B7 lên một biến thể mới gọi là **EfficientNet-L2**: rộng hơn và sâu hơn B7 nhưng dùng resolution train thấp hơn, cho nhiều tham số hơn để fit lượng dữ liệu không nhãn khổng lồ.

Kiến trúc	w (width)	d (depth)	Train res.	Test res.	# Params
EfficientNet-B7	2.0	3.1	600	600	66M
EfficientNet-L2	4.3	5.3	475	800	480M

(Tự tính: tỷ lệ tham số L2/B7 = $480/66 \approx 7{,}27$ lần; thời gian train L2 theo paper khoảng 5 lần thời gian train B7.)

Chi tiết huấn luyện: batch size cho ảnh có nhãn mặc định 2048 (giảm nếu không đủ nhớ; paper nói 512, 1024, 2048 cho cùng hiệu năng). Số epoch: 350 cho model lớn hơn B4 (gồm L2), 700 cho model nhỏ hơn. Learning rate khởi đầu 0.128 (với batch 2048), decay 0.97 mỗi 2.4 epoch (khi train 350 epoch) hoặc mỗi 4.8 epoch (khi train 700 epoch). Batch size cho ảnh không nhãn được đặt lớn (đặc biệt với model lớn) để khai thác hết lượng ảnh không nhãn; ảnh có nhãn và không nhãn được nối lại để tính trung bình cross-entropy loss chung. Với EfficientNet-L2, paper áp dụng kỹ thuật fix train-test resolution discrepancy (tương tự FixRes, Touvron et al. 2019): train ở resolution nhỏ 350 epoch, sau đó finetune 1.5 epoch ở resolution lớn hơn trên ảnh có nhãn không augment, cố định các layer nông trong khi finetune.

EfficientNet-L2 cần train 6 ngày trên một Cloud TPU v3 Pod (2048 core) khi tỷ lệ batch không nhãn : batch có nhãn là 14:1.

Noise cho student: stochastic depth (survival probability 0.8 ở layer cuối, giảm tuyến tính cho các layer khác), dropout (tỷ lệ 0.5 ở layer cuối), RandAugment (2 phép biến đổi ngẫu nhiên, magnitude 27). Các hyperparameter noise này giống nhau cho cả B7 và L2.

3.4. Huấn luyện lặp (iterative training)

Mô hình tốt nhất đạt được qua 3 vòng lặp đưa student trở lại làm teacher:

Vòng	Model	Tỷ lệ batch (unlabeled:labeled)	Top-1 Acc.
1	EfficientNet-L2 (teacher = B7)	14:1	87.6%
2	EfficientNet-L2 (teacher = L2 vòng 1)	14:1	88.1%
3	EfficientNet-L2 (teacher = L2 vòng 2)	28:1	88.4%

(Tự tính minh hoạ ý "trained on unlabeled data only for one epoch every time it is trained on labeled data for a hundred epochs" mà paper nói ở Mục 4.3/Study #7: với 130.000.000 ảnh không nhãn và 1.300.000 ảnh có nhãn, nếu dùng cùng batch size cho hai loại dữ liệu thì tỷ lệ số bước train trên unlabeled so với labeled đúng bằng $130{,}000{,}000/1{,}300{,}000 = 100$, khớp với phát biểu "một trăm epoch" của paper, đây là lý do paper cần tăng batch size cho ảnh không nhãn lên nhiều lần. Ở vòng lặp cuối cùng, với tỷ lệ 28:1 và batch có nhãn mặc định 2048, batch không nhãn tương ứng $2048 \times 28 = 57.344$ ảnh mỗi bước, tự tính.)

4. Thí nghiệm và kết quả chính

4.1. Kết quả chính trên ImageNet

Bảng tổng hợp (Table 1 của paper):

	ImageNet top-1	ImageNet-A top-1	ImageNet-C mCE	ImageNet-P mFR
SOTA trước đó	86.4%	61.0%	45.7	27.8
Noisy Student Training	88.4%	83.7%	28.3	12.2

(mCE: mean corruption error, thấp hơn tốt hơn; mFR: mean flip rate, thấp hơn tốt hơn.)

EfficientNet-L2 với Noisy Student Training đạt 88.4% top-1 trên ImageNet, tốt hơn 2.0% so với SOTA trước đó (FixRes ResNeXt-101 WSL, 86.4%, cần 3.5 tỷ ảnh Instagram gắn tag yếu); mô hình của paper chỉ dùng 300 triệu ảnh JFT không nhãn và nhỏ hơn khoảng 2 lần về số tham số so với ResNeXt-101 WSL (829M tham số).

So với EfficientNet-B7 gốc (85.0%, không dùng dữ liệu phụ), tổng mức tăng là 3.4%, được paper phân tách thành hai nguồn: +0.5% từ việc dùng kiến trúc lớn hơn (EfficientNet-L2 khi train một mình, không có NST, đạt 85.5%) và +2.9% từ chính Noisy Student Training ($85.5\% \rightarrow 88.4\%$). Paper nhấn mạnh phần đóng góp của NST lớn hơn nhiều so với việc đổi kiến trúc.

Table 2 so sánh với nhiều phương pháp SOTA trước đó (ResNet-50 76.0%, ResNet-152 77.8%, Inception-v4 80.0%, ResNeXt-101 80.9%, SENet 82.7%, AmoebaNet-C 83.5%, GPipe 84.3%, EfficientNet-B7 85.0%, EfficientNet-L2 85.5% không dùng dữ liệu phụ; ResNet-50/ResNeXt-101 Billion-scale 81.2%/84.8%, ResNeXt-101 WSL 85.4%, FixRes ResNeXt-101 WSL 86.4% dùng 3.5B ảnh Instagram; Big Transfer BiT-L 87.5% dùng 300M ảnh JFT weakly-labeled; Noisy Student Training (EfficientNet-L2) 88.4% dùng 300M ảnh JFT **không nhãn**).

4.2. Model-size study (không iterative training)

Thử NST trên toàn bộ dãy EfficientNet-B0 đến B7 (cùng model cho teacher và student, không lặp), batch không nhãn = 3x batch có nhãn (trừ B0 dùng 1x). Kết quả cải thiện nhất quán khoảng 0.8% ở mọi kích cỡ model, ví dụ: B0 77.3%→78.1%, B2 80.0%→81.1%, B5 84.0%→85.1%, B7 85.0%→86.4%.

4.3. Robustness trên ImageNet-A, C, P

- **ImageNet-A** (ảnh khó, gây lỗi lớn cho SOTA): top-1 từ 61.0% (ResNeXt-101 WSL) lên 83.7% (NST L2); EfficientNet-L2 baseline không NST chỉ đạt 49.6% top-1/78.6% top-5, cho thấy phần lớn cải thiện đến từ NST chứ không chỉ từ kiến trúc L2.
- **ImageNet-C** (ảnh bị corruption nhân tạo): mCE giảm từ 45.7 (ResNeXt-101 WSL) xuống 28.3 (NST L2, resolution 299); ở resolution 224, NST L2 đạt 30.0 mCE so với baseline L2 (không NST) 47.5 mCE.
- **ImageNet-P** (ảnh bị perturbation liên tục): mFR giảm từ 27.8 xuống 12.2 (resolution 299) hoặc 14.2 (resolution 224, so sánh trực tiếp với baseline cùng resolution).
- **Đối kháng (adversarial):** dưới tấn công FGSM, NST cải thiện đáng kể accuracy (paper nói tăng 11% ở $\epsilon = 2$ và tăng thêm khi ϵ lớn hơn); dưới PGD 10 bước tại $\epsilon = 16$, accuracy tăng từ 1.1% lên 4.4%.

Paper tự nhận các cải thiện robustness này là "đáng ngạc nhiên" (surprising) vì phương pháp không được thiết kế có chủ đích để tối ưu robustness.

5. Phân tích / ablation

5.1. Vai trò của noise (Mục 4.1, Table 6)

Dùng EfficientNet-B5 làm teacher, thử hai kịch bản (1.3M và 130M ảnh không nhãn), loại dần augmentation/stochastic depth/dropout trên ảnh không nhãn (vẫn giữ cho ảnh có nhãn):

Cấu hình	1.3M unlabeled	130M unlabeled
Supervised baseline (EfficientNet-B5)	83.3%	84.0%
Noisy Student Training (B5)	83.9%	85.1%
student bỏ Aug	83.6%	84.6%
student bỏ Aug, SD, Dropout	83.2%	84.3%
teacher có noise (Aug, SD, Dropout)	83.7%	84.4%

Lưu ý: hai cột 1.3M và 130M dùng augmentation chuẩn khác nhau cho baseline (cột 1.3M dùng augmentation chuẩn gồm random translate/flip, cột 130M dùng RandAugment), nên hai giá trị baseline 83.3% và 84.0% không mâu thuẫn với nhau, chỉ là hai thiết lập thí nghiệm khác nhau (ghi rõ ở đây để người đọc không hiểu lầm thành lỗi số liệu).

Hiệu năng giảm dần khi bỏ noise, nhưng ngay cả khi bỏ hết noise trên ảnh không nhãn (130M), kết quả vẫn tốt hơn baseline thuần supervised (84.3% so với 84.0%); paper giả thuyết phần cải thiện còn lại này đến từ tính ngẫu nhiên vốn có của SGD. Thêm noise vào **teacher** khi sinh nhãn làm giảm hiệu năng, cho thấy cần một teacher "sạch", mạnh, không nhiễu khi tạo pseudo-label.

5.2. Tám phát hiện từ ablation ở Appendix A.2 (Study #1-8)

1. **Teacher lớn/tốt hơn → kết quả tốt hơn** (Study #1): dùng NST (L2, 88.4%) làm teacher cho các student B0-B7 cho cải thiện thêm 0.5%-1.6% so với dùng cùng cỡ model làm teacher, ví dụ B4 đạt 85.3% khi teacher là L2 (so với 84.4% khi teacher cùng cỡ B4).
2. **Lượng dữ liệu không nhãn lớn là cần thiết** (Study #2): giảm dần từ 130M xuống 1/128 (~1M) dữ liệu, accuracy giảm từ 84.0% xuống 83.4%.

3. **Soft pseudo-label tốt hơn hard pseudo-label cho dữ liệu ngoài-miền** (Study #3, Figure 5): với ảnh tự tin thấp (ngoài-miền), hard label có thể làm giảm hiệu năng còn soft label vẫn ổn định; nhưng paper cũng tự hạ bớt mức độ chắc chắn của phát hiện này, nói việc chọn soft hay hard "có thể cần quyết định theo từng trường hợp cụ thể".
4. **Student lớn quan trọng** (Study #4, Table 11): với cùng teacher, student lớn hơn luôn cho kết quả tốt hơn.
5. **Cân bằng dữ liệu có lợi cho model nhỏ** (Study #5, Table 12): lợi ích của data balancing rõ với B0/B1, giảm dần với B2/B3 (model lớn hơn học tốt từ dữ liệu không cân bằng).
6. **Joint training tốt hơn pretrain-rồi-finetune** (Study #6, Table 13): train đồng thời trên dữ liệu có nhãn và pseudo-nhãn vượt trội cách pretrain trên pseudo-nhãn rồi mới finetune trên dữ liệu có nhãn (cách làm của Yalniz et al. 2019); pretrain-chỉ-trên-pseudo-nhãn cho kết quả kém hơn cả supervised-chỉ-trên-nhãn, gợi ý phân phối dữ liệu không nhãn khác đáng kể so với dữ liệu có nhãn.
7. **Tỷ lệ batch không nhãn:có nhãn lớn giúp model lớn train lâu hơn trên dữ liệu không nhãn, cho accuracy cao hơn** (Study #7, Table 14): với EfficientNet-L2 (87.4% ở tỷ lệ 3:1, tăng lên 87.9% ở tỷ lệ 6:1 trong một thiết lập teacher khác); hiệu ứng này không rõ với model nhỏ (B4) vì bão hoà capacity.
8. **Train từ đầu đôi khi tốt hơn warm-start từ teacher** (Study #8, Table 15): dù warm-start tiết kiệm thời gian, student khởi tạo từ teacher vẫn cần ít nhất 140 epoch để đạt hiệu năng tốt; ví dụ cụ thể hơn, với teacher B7 (86.4%), student warm-start bị "kẹt" ở khoảng 86.4% sau 210 epoch, còn student train từ đầu đạt 86.9%.

5.3. Kết quả bổ sung trên kiến trúc/dataset khác (Appendix A.3-A.4)

- **ResNet-50** (không phải EfficientNet): NST cải thiện từ 77.6% lên 78.9%, cho thấy phương pháp không chỉ hiệu quả với riêng EfficientNet.
- **SVHN** (dataset nhỏ hơn hẳn ImageNet, 73K ảnh train có nhãn, 531K ảnh "extra" dùng làm không nhãn): NST cải thiện baseline (RandAugment EfficientNet-B0, 98.1%) lên 98.6%, vượt SOTA trước đó (RandAugment với Wide-ResNet-28-10, 98.3%).
- **YFCC100M** (thay thế công khai cho JFT): sau lọc/cân bằng, đạt 130M ảnh nhưng chỉ có 34M ảnh duy nhất (so với JFT 81M ảnh duy nhất trước cân bằng). NST vẫn cải thiện đáng kể với YFCC100M, nhưng JFT vẫn cho kết quả tốt hơn ở hầu hết kích cỡ model (trừ B0, B1); paper cho rằng khác biệt "có lẽ" do khác biệt kích cỡ dataset, nhưng không kiểm chứng thêm giả thuyết này.

6. Giới hạn

Các giới hạn tự thừa nhận hoặc có thể suy ra trực tiếp từ văn bản paper:

- **JFT không công khai**, làm giảm khả năng tái lập độc lập của kết quả chính; thí nghiệm thay thế với YFCC100M (công khai) cho kết quả thấp hơn, và lời giải thích "có lẽ do khác biệt kích cỡ dataset" chỉ là suy đoán của chính các tác giả, chưa được kiểm chứng bằng một thí nghiệm kiểm soát kích cỡ.
- **Kết quả đối kháng (adversarial) không so sánh trực tiếp được với công trình trước**, vì paper dùng resolution đầu vào lớn hơn nhiều (800x800), và độ dễ bị tấn công đối kháng được biết là co giãn theo số chiều đầu vào; đây là giới hạn do paper tự nêu rõ trong văn bản (footnote mục 3.4).
- **Khả năng gây nhiễu (confound) giữa RandAugment và các corruption/perturbation của ImageNet-C/P**: paper tự thừa nhận cả model của họ và ResNeXt-101 WSL (baseline so sánh) đều dùng augmentation có phần trùng lặp với các corruption trong benchmark robustness (RandAugment trùng Brightness, Contrast, Sharpness với ImageNet-C; trùng Brightness, Translate, Rotate với ImageNet-P), điều này "có thể" khiến kết quả tốt hơn một phần vì lý do không liên quan tới cơ chế self-training; paper lập luận phần trùng lặp "nhỏ" nhưng không định lượng mức ảnh hưởng cụ thể.
- **Không dùng cùng một cấu hình model cho đánh giá chuẩn và đánh giá robustness**: với ImageNet-C/P, paper phải dùng bản EfficientNet-L2 *chưa* áp dụng kỹ thuật finetune resolution (FixRes-style) vì bản đã finetune ở resolution lớn hơn lại cho hiệu năng *kém hơn* trên các bộ này (do sai khác resolution

giữa lúc train và lúc test theo một kiểu khác với lúc đánh giá ImageNet chuẩn); đây là một điểm không nhất quán về cấu hình đánh giá mà paper tự nêu ở footnote, không che giấu, nhưng người đọc cần lưu ý khi so sánh các bảng số với nhau.

- **Hyperparameter lọc dữ liệu (ngưỡng confidence 0.3, giới hạn 130K ảnh/lớp) không được paper tune kỹ**; khẳng định "phương pháp rất bền với các hyperparameter này" không kèm thí nghiệm định lượng trong phần chính (chỉ có một ghi chú ở Appendix nói lợi ích của data balancing khác nhau theo cỡ model, Study #5).
- **So sánh B7 và L2 làm student (Table 9) được paper tự nhận là "không hoàn toàn công bằng"**, vì hai cấu hình dùng tỷ lệ batch không nhãn:có nhãn khác nhau (3x với B7 so với 7x được coi là tốt hơn khi train B7, nhưng thí nghiệm thực tế dùng tỷ lệ khác cho L2).
- **Tìm kiếm soft-vs-hard pseudo-label (finding #3) không đưa ra một quy tắc tổng quát**: paper viết rõ lựa chọn này "có thể cần quyết định theo từng trường hợp" (case-by-case), nghĩa là không có kết luận chắc chắn, phổ quát.
- **Chi phí tính toán rất lớn** (model 480 triệu tham số, 300 triệu ảnh JFT, 6 ngày trên TPU v3 Pod 2048 core cho một lần train L2), khiến việc tái lập toàn bộ pipeline ngoài tầm của hầu hết nhóm nghiên cứu nhỏ; đây là nhận xét của người ghi chú (không phải câu paper tự viết), nhưng được rút trực tiếp từ các số liệu paper công bố.

7. Liên hệ với các bài khác

- **ml basics 26** (Representation Learning: A Review): nền tảng lý thuyết về semi-supervised learning và dùng dữ liệu không nhãn để học biểu diễn, một trong các dòng mà self-training thuộc về.
- **ml basics 27, ml basics 28** (How transferable are features; Rethinking ImageNet Pre-training): cùng dòng câu hỏi "cần bao nhiêu dữ liệu/pretraining để đạt độ chính xác tốt trên task đích", nhưng bài 28 trả lời bằng cách bỏ hẳn pretraining (train from scratch), còn bài này trả lời bằng cách thêm dữ liệu không nhãn số lượng lớn; cả ba bài cùng thuộc chủ đề "vai trò thật sự của pretraining/dữ liệu phụ".
- **ml basics 06** (Dropout, co-adaptation): dropout là một trong các nguồn noise chính được dùng cho student ở đây; paper này ứng dụng dropout theo một vai trò mới (làm teacher-ở-chế-độ-inference hành xử như ensemble).
- **distillation 02** (Hinton, Distilling the Knowledge in a Neural Network): paper tự so sánh trực tiếp với Knowledge Distillation trong Mục 5 (Related Works): KD thường không dùng noise và dùng student nhỏ hơn để nén mô hình, không tận dụng dữ liệu không nhãn; Noisy Student Training ngược lại ở cả ba điểm đó (có noise, student lớn hơn-hoặc-bằng, tận dụng dữ liệu không nhãn, mục tiêu mở rộng tri thức chứ không compress).
- **ml basics 19** (On Calibration of Modern Neural Networks): liên quan gián tiếp qua khái niệm độ tự tin (confidence) của mô hình, dùng làm tiêu chí lọc ảnh không nhãn (confidence > 0.3) ở bài này; nếu confidence bị miscalibrated, bước lọc này có thể chọn sai ảnh.
- **ml basics 17, ml basics 18** (SimCLR, MoCo): một hướng khác để dùng dữ liệu không nhãn (self-supervised contrastive pretraining) thay vì self-training với pseudo-label có giám sát từ teacher; cả hai hướng đều nhằm khai thác dữ liệu không nhãn nhưng theo nguyên lý khác nhau.
- **ml basics 33** (class imbalance trong CNN, theo quy ước số của thư mục): kỹ thuật cân bằng dữ liệu không nhãn theo lớp (duplicate/giữ ảnh tự tin nhất) ở bài này có chung mục tiêu với các kỹ thuật xử lý mất cân bằng lớp, dù áp dụng trên pseudo-label thay vì trên nhãn gốc.

8. Gợi ý áp dụng

Noisy Student minh họa cách dùng dữ liệu không nhãn với pseudo-label và nhiễu trong huấn luyện student. Chất lượng nhãn giả, lượng dữ liệu và quy trình đánh giá độc lập đều ảnh hưởng kết quả.

9. Thí nghiệm gợi ý

Tất cả đều chạy được, dùng mô hình/tập dữ liệu nhỏ (toy), batch 1-vài chục:

1. **Tái hiện mini self-training trên MNIST/CIFAR-10:** chia dữ liệu train thành 10% "có nhãn" và 90% "không nhãn" (bỏ nhãn thật). Train teacher nhỏ (CNN vài layer) trên 10% có nhãn, sinh pseudo-label (soft) cho 90% còn lại, train một student cùng kích cỡ với/không noise (random crop/flip cho input, dropout cho model) trên 10% có nhãn + 90% pseudo-nhãn. Đo accuracy trên test set thật. Kỳ vọng: student có noise > student không noise > teacher-chỉ-học-trên-10%, minh hoạ trực tiếp claim chính của paper ở quy mô nhỏ.
2. **Ablation loại noise:** lặp thí nghiệm 1 với ba cấu hình (a) không noise, (b) chỉ input augmentation, (c) input augmentation + dropout. Đo accuracy mỗi cấu hình, so khoảng cách giữa (a)-(b) và (b)-(c) để xem phần lớn lợi ích đến từ augmentation hay từ model noise, so sánh với nhận định của paper ở Mục 5.1 rằng augmentation là nguồn noise quan trọng nhất.
3. **Soft vs hard pseudo-label trên dữ liệu giả-OOD:** lấy một phần ảnh "không nhãn" và làm mờ/nhiễu mạnh để mô phỏng ảnh ngoài-miền, train hai student (một dùng soft label, một dùng hard label/one-hot từ teacher) trên cùng dữ liệu. Đo accuracy trên tập test gốc và trên một tập test cũng bị làm mờ tương tự (giả lập robustness). Kỳ vọng theo paper: soft label ổn định hơn trên phần ảnh giống-OOD.
4. **Tỷ lệ batch unlabeled:labeled:** cố định batch có nhãn (ví dụ 32 ảnh MNIST), thử các batch không nhãn 32 (tỷ lệ 1:1) và 128 (tỷ lệ 4:1) với cùng tổng số bước train. Đo accuracy cuối, minh hoạ (ở quy mô rất nhỏ, không cần TPU) hiệu ứng Finding #7: tỷ lệ lớn hơn giúp model "thấy" dữ liệu không nhãn nhiều hơn mỗi epoch dữ liệu có nhãn.
5. **Warm-start vs train từ đầu:** train một CNN nhỏ teacher trên 20% dữ liệu có nhãn, sau đó train hai student: (a) khởi tạo ngẫu nhiên, (b) khởi tạo từ trọng số teacher, cả hai cùng số epoch cố định trên dữ liệu có nhãn + pseudo-nhãn. Theo dõi training curve để xem (b) có bị "kẹt" ở một mức accuracy gần teacher trong khi (a) vượt qua không, đối chiếu với Finding #8.

10. Tóm tắt công thức / khái niệm

Thuật toán (3 bước, lặp lại):

$$\text{(1)}\ \min_{\theta_t} \frac{1}{n}\sum_i \ell(y_i, f^{\text{noised}}(x_i, \theta_t))$$

$$\text{(2)}\ \tilde{y}_i = f(\tilde{x}_i, \theta_t^*)$$

$$\text{(3)}\ \min_{\theta_s} \frac{1}{n}\sum_i \ell(y_i, f^{\text{noised}}(x_i, \theta_s)) + \frac{1}{m}\sum_i \ell(\tilde{y}_i, f^{\text{noised}}(\tilde{x}_i, \theta_s))$$

Khái niệm chính:

Thuật ngữ	Ý nghĩa trong bài
Teacher	Model train trên dữ liệu có nhãn, dùng *không* noise để sinh pseudo-label
Student	Model train trên dữ liệu có nhãn + pseudo-nhãn, kích cỡ lớn hơn-hoặc-bằng teacher, *có* noise
Pseudo-label (soft/hard)	Nhãn do teacher sinh ra cho ảnh không nhãn: phân phối xác suất (soft) hoặc one-hot (hard)
Input noise	RandAugment (2 phép biến đổi ngẫu nhiên, magnitude 27)
Model noise	Dropout (rate 0.5, layer cuối), stochastic depth (survival probability 0.8, layer cuối, decay tuyến tính)
Data filtering/balancing	Giữ ảnh confidence > 0.3; tối đa 130K ảnh/lớp; nhân bản ảnh ở lớp thiếu
Iterative training	Student vòng trước trở thành teacher vòng sau

Số liệu chính (tự tính một phần, phần còn lại lấy trực tiếp từ paper):

Mốc	Giá trị
ImageNet top-1 (NST, EfficientNet-L2)	88.4%
Phân rã mức tăng so với B7 (85.0%)	+0.5% (kiến trúc L2) + 2.9% (NST) = 3.4%
Tỷ lệ tham số L2/B7	$480\text{M}/66\text{M} \approx 7{,}27$ (tự tính)
Tỷ lệ "epoch tương đương" unlabeled/labeled (130M/1.3M)	100 (tự tính, khớp phát biểu paper)
Batch không nhãn ở tỷ lệ 28:1, batch có nhãn 2048	$2048 \times 28 = 57.344$ ảnh/bước (tự tính)
ImageNet-A top-1	$61.0\% \rightarrow 83.7\%$
ImageNet-C mCE (res 299)	$45.7 \rightarrow 28.3$
ImageNet-P mFR (res 299)	$27.8 \rightarrow 12.2$

11. Kết luận ngắn

Noisy Student Training cho thấy self-training, khi được cải tiến bằng cách bắt student lớn hơn-hoặc-bằng teacher và thêm noise mạnh (RandAugment, dropout, stochastic depth) trong lúc student học từ pseudo-label của một teacher sạch, có thể khai thác dữ liệu không nhãn quy mô lớn để đạt SOTA mới trên ImageNet (88.4% top-1) với ít dữ liệu phụ hơn nhiều so với phương pháp weakly-supervised trước đó (300M so với 3.5B ảnh), đồng thời cải thiện mạnh robustness trên ImageNet-A/C/P và dưới tấn công đối kháng, dù phương pháp không được thiết kế riêng cho mục tiêu robustness. Đổi lại, phương pháp đòi hỏi hạ tầng tính toán rất lớn (model hàng trăm triệu tham số, hàng trăm triệu ảnh không nhãn, nhiều ngày trên TPU Pod), và một số kết luận phụ (lựa chọn soft/hard label, mức độ "robust" với siêu tham số lọc dữ liệu) được bài báo tự nhận là chưa dứt khoán, cần xét theo từng trường hợp cụ thể.

Nguồn gốc: https://arxiv.org/abs/1911.04252

Bài 30

When Does Label Smoothing Help? (2019)

Tác giả: Rafael Müller, Simon Kornblith, Geoffrey Hinton (Google Brain, Toronto).
Venue / arXiv: arXiv:1906.02629 (v3, 10/06/2020); paper có nêu rõ venue ngay trong văn bản (chân trang trang 1): “33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada”.
Nguồn: https://arxiv.org/abs/1906.02629
Chủ đề: phân tích thực nghiệm khi nào và vì sao label smoothing (làm “mềm” nhãn one-hot bằng cách trộn với phân phối đều) giúp cải thiện generalization và calibration, nhưng lại làm hỏng knowledge distillation; lý giải bằng hình học biểu diễn ở penultimate layer và bằng một phép đo mutual information xấp xỉ.
Vị trí trong lộ trình: Đọc cùng bài 31 về mixup và bài 32 về focal loss để đối chiếu các cách thay đổi mục tiêu huấn luyện.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu “tự tính” là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Việc train mạng neural rất nhạy với loss function cụ thể được tối thiểu hoá. Không lâu sau khi backpropagation ra đời, nhiều nghiên cứu đã chỉ ra cross-entropy cho hội tụ nhanh và kết quả tốt hơn so với loss bình phương thông thường, và ngay từ sớm đã có gợi ý rằng những objective “lạ” hơn cross-entropy chuẩn có thể còn tốt hơn nữa. Szegedy et al. (2016, Rethinking the Inception Architecture) đưa ra **label smoothing**: thay vì tính cross-entropy với nhãn “cứng” (hard target, one-hot) từ tập dữ liệu, tính với một hỗn hợp có trọng số giữa nhãn cứng và phân phối đều (uniform) trên các lớp.

Kỹ thuật này đã được dùng thành công để cải thiện accuracy trên nhiều tác vụ khác nhau. Paper tổng hợp lại trong Bảng khảo sát literature (Table 1 của bài báo):

Dataset	Kiến trúc	Metric	Không LS	Có LS
ImageNet	Inception-v2	Top-1 error	23.1	22.8
ImageNet	Inception-v2	Top-5 error	6.3	6.1
EN-DE (dịch máy)	Transformer	BLEU	25.3	25.8
EN-DE (dịch máy)	Transformer	Perplexity	4.67	4.92
WSJ (nhận dạng giọng nói)	BiLSTM+Attention	WER	8.9	7.0/6.7

Điểm đáng chú ý tự thân paper nêu: với Transformer dịch máy, BLEU score được cải thiện **dù perplexity trở nên xấu hơn** ($4.67 \rightarrow 4.92$, càng cao càng xấu), một nghịch lý mà bài báo sẽ bàn tới ở phần calibration. Mặc dù được dùng rộng rãi như một “trick”, lý do vì sao và khi nào label smoothing hiệu quả vẫn chưa được hiểu rõ; đây là động lực chính của bài báo.

2. Ý tưởng chính

Paper có ba đóng góp chính (paper tự liệt kê trong Introduction):

1. **Một phương pháp visualization mới** dựa trên chiếu tuyến tính (linear projection) các activation ở penultimate layer (layer ngay trước lớp output), cho thấy trực giác về cách biểu diễn khác nhau giữa mạng có và không có label smoothing.

2. **Label smoothing calibrate mạng một cách ngầm định (implicit)**: làm cho độ tự tin (confidence) của dự đoán khớp với độ chính xác (accuracy) thực tế hơn, một hiệu ứng tương tự temperature scaling (Guo et al. 2017) nhưng không cần bước hậu xử lý riêng.
3. **Label smoothing làm hỏng knowledge distillation**: khi teacher được train với label smoothing, student được distill từ teacher đó kém hơn hẳn, dù chính teacher đó có accuracy cao hơn. Paper lý giải điều này bằng việc label smoothing xoá bớt thông tin trong logit về mức độ "giống nhau" giữa các lớp khác nhau (ví dụ một "con chó giống bao nhiêu % một con sói"), thông tin không cần thiết cho generalization/calibration nhưng lại cần thiết cho distillation.

Trực giác hình học trung tâm: label smoothing khuyến khích các activation của ví dụ cùng lớp co cụm lại thành các cluster rất chặt (tight), cách đều các "template" (trọng số) của mọi lớp sai khác, thay vì cluster rộng, không đều như khi train với nhãn cứng.

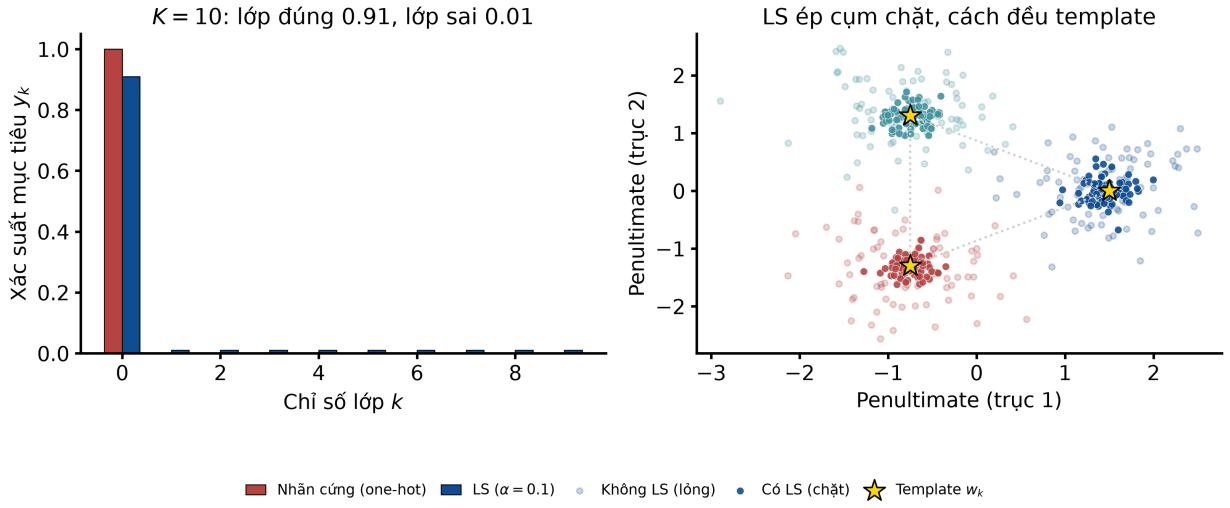
3. Phương pháp chi tiết

3.1. Định nghĩa label smoothing

Biểu diễn dự đoán của mạng bằng softmax ở layer cuối:

$$p_k = \frac{e^{x^T w_k}}{\sum_{l=1}^{K} e^{x^T w_l}}$$

trong đó p_k là xác suất gán cho lớp k, w_k là trọng số (và bias) của layer cuối ứng với lớp k, x là vector activation của penultimate layer (nối thêm "1" để biểu diễn bias), K là số lớp.



So sánh nhãn one-hot và label smoothing $\alpha = 0.1$ ($K = 10$: 0.91/0.01); minh hoạ cụm penultimate chặt hơn và cách đều template khi có LS (minh hoạ, tự tính)

Với nhãn cứng, tối thiểu hoá cross-entropy kỳ vọng:

$$H(y, p) = \sum_{k=1}^{K} -y_k \log(p_k)$$

với $y_k = 1$ cho lớp đúng, 0 cho các lớp còn lại.

Với label smoothing hệ số α, nhãn mục tiêu được sửa thành:

$$y_k^{LS} = y_k(1 - \alpha) + \alpha/K$$

(Tự tính ví dụ số: với $K = 10$ lớp, $\alpha = 0{,}1$, lớp đúng có $y_k^{LS} = 1 \times (1 - 0{,}1) + 0{,}1/10 = 0{,}9 + 0{,}01 = 0{,}91$; mỗi lớp sai có $y_k^{LS} = 0 \times 0{,}9 + 0{,}01 = 0{,}01$. Kiểm tra tổng: $0{,}91 + 9 \times 0{,}01 = 0{,}91 + 0{,}09 = 1{,}0$, đúng là một phân phối xác suất hợp lệ.)

3.2. Hình học penultimate layer và phương pháp visualization

Logit của lớp k, $x^T w_k$, có thể hiểu như một biến đổi của khoảng cách Euclid bình phương giữa activation x và một "template" w_k của lớp đó:

$$\|x - w_k\|^2 = x^T x - 2x^T w_k + w_k^T w_k$$

Vì $x^T x$ bị loại khi tính softmax (hằng số chung cho mọi lớp) và $w_k^T w_k$ thường gần như hằng số giữa các lớp, logit gần tương đương với $-2x^T w_k$ (âm của khoảng cách tới template, lệch hằng số). Do đó label smoothing (ép khoảng cách giữa logit đúng và logit sai là một hằng số phụ thuộc α) tương đương ép activation x gần template lớp đúng và cách đều các template lớp sai.

Phương pháp visualization paper đề xuất: (1) chọn 3 lớp; (2) tìm một basis trực chuẩn (orthonormal) của mặt phẳng đi qua 3 template w_k của 3 lớp đó; (3) chiếu các activation penultimate layer của ví dụ thuộc 3 lớp này lên mặt phẳng đó. Kết quả cho thấy (bằng hình, Figure 1 của paper, trên AlexNet/CIFAR-10, ResNet-56/CIFAR-100, Inception-v4/ImageNet) các cluster co chặt và tạo hình tam giác đều rõ nét hơn khi có label smoothing.

3.3. Đo calibration (ECE)

Paper dùng expected calibration error (ECE, định nghĩa gốc theo Guo et al. 2017, (Ngoài paper, kiến thức chung: ECE thường tính bằng cách chia dự đoán vào các bin theo confidence, rồi lấy trung bình có trọng số của $|\text{accuracy}_{bin} - \text{confidence}_{bin}|$), không được paper này viết lại công thức chi tiết) để đo độ lệch giữa confidence dự đoán và accuracy thực tế, và so sánh với temperature scaling (chia logit cho một hệ số nhiệt độ T trước softmax) như một cách calibrate hậu xử lý.

3.4. Knowledge distillation và "equivalent label smoothing factor"

Loss distillation tổng quát mà paper dùng:

$$\mathcal{L} = (1 - \beta) H(y, p) + \beta H(p^t(T), p(T))$$

trong đó $p_k(T)$ và $p_k^t(T)$ là output (sau softmax) của student và teacher với temperature T, β cân bằng giữa việc khớp nhãn cứng và việc khớp teacher đã làm mềm.

Để so sánh các kịch bản (train student trực tiếp với label smoothing α, hay distill từ teacher hard-target với nhiệt độ T, hay distill từ teacher đã train với label smoothing), paper định nghĩa một **hệ số label smoothing tương đương** γ: với kịch bản train trực tiếp, $\gamma = \alpha$; với kịch bản distillation,

$$\gamma = \mathbb{E}\left[\sum_{k=1}^{K} (1 - y_k)\, p_k^t(T)\, \frac{K}{K-1}\right]$$

tức là khối lượng xác suất mà teacher gán cho các lớp sai, chuẩn hoá theo số lớp, tính trung bình trên tập train. Cách định nghĩa này cho phép đặt mọi kịch bản trên cùng một "thước đo độ mềm" để so sánh công bằng.

3.5. Đo mutual information giữa input và chênh lệch logit

Để định lượng hiện tượng "mất thông tin" trong logit, paper đo mutual information $I(X; Y)$ giữa X (chỉ số của ví dụ training, biến rời rạc) và Y (chênh lệch giữa hai logit của hai lớp, biến liên tục). Nguồn ngẫu nhiên của Y là data augmentation: $y = f(d(z_x))$, với z_x là ảnh đầu vào ứng với chỉ số x, $d(\cdot)$ là một phép

augmentation ngẫu nhiên (ví dụ dịch ảnh ngẫu nhiên), $f(\cdot)$ là mạng đã train. Paper xấp xỉ phân phối của Y là Gaussian và ước lượng trung bình/phương sai bằng Monte Carlo:

$$I(X;Y) = \mathbb{E}_{X,Y}[\log p(y|x) - \log \textstyle\sum_x p(y|x)p(x)]$$

$$\hat{I}(X;Y) = \frac{1}{N}\sum_{x=1}^{N}\Big[-\frac{(f(d(z_x)) - \mu_x)^2}{2\sigma^2} - \log\Big(\frac{1}{N}\sum_{x=1}^{N} e^{-(f(d(z_x))-\mu_x)^2/(2\sigma^2)}\Big)\Big]$$

với μ_x, σ^2 ước lượng từ L mẫu Monte Carlo cho mỗi ví dụ, N là số ví dụ dùng để ước lượng. Mutual information theo định nghĩa này nằm trong khoảng $[0, \log(N)]$.

3.6. Chi tiết thực nghiệm (Appendix)

Paper dùng 4 kiến trúc/dataset khác nhau cho các phần thí nghiệm khác nhau, chi tiết chủ yếu lấy từ Appendix A:

- **AlexNet/CIFAR-10** (visualization, distillation): chia train/val 40k/10k, batch 128, train 390k iteration, SGD, LR khởi đầu 0.1 giảm 10 lần mỗi 130k bước, nhân cross-entropy term với 3, weight decay 0.04 ở hai dense layer cuối.
- **ResNet-56/CIFAR-10 và CIFAR-100** (visualization, calibration, distillation): chia 40k/10k, batch 128, 64k iteration, SGD + Nesterov momentum 0.9, LR 0.1 giảm 10 lần tại bước 32k và 48k, gradient clipping 1.0, kernel size 3, weight decay 0.0001.
- **Transformer/EN-DE WMT32k** (calibration): dùng hyperparameter gốc `transformer_base` (label smoothing 0.1) và biến thể không smoothing, train 300k bước (thay vì 100k ở paper gốc), không dùng weight averaging của checkpoint.
- **Inception-v4/ImageNet** (visualization, calibration): batch 4096, SGD + Nesterov momentum 0.9, weight decay 8×10^{-5}, exponential moving average trọng số với decay 0.9999.
- **Fully-connected 2 layer/MNIST** (distillation, thí nghiệm mở đầu Mục 4): teacher 1200 neuron/layer train với $\alpha = 0{,}1$ và dịch ảnh ngẫu nhiên ± 2 pixel (dropout trong thí nghiệm gốc của Hinton et al. 2015 được thay bằng label smoothing ở biến thể mới); student 800 neuron/layer, train với $\beta = 0{,}6$ (0.4 trọng số cho cross-entropy nhãn cứng, 0.6 trọng số cho khớp logit teacher bằng nửa bình phương sai số, không dùng data augmentation cho student).

4. Thí nghiệm và kết quả chính

4.1. Visualization và accuracy (Table 2 của paper)

Dataset	Kiến trúc	Accuracy ($\alpha = 0{,}0$)	Accuracy ($\alpha = 0{,}1$)
CIFAR-10	AlexNet	$86{,}8 \pm 0{,}2$	$86{,}7 \pm 0{,}3$
CIFAR-100	ResNet-56	$72{,}1 \pm 0{,}3$	$72{,}7 \pm 0{,}3$
ImageNet	Inception-v4	$80{,}9$	$80{,}9$

Với CIFAR-10/AlexNet, cluster co chặt rõ với label smoothing nhưng accuracy gần như không đổi. Với CIFAR-100/ResNet-56, label smoothing cải thiện accuracy và cluster cũng co chặt hơn. Với ImageNet/Inception-v4, hai lớp ngữ nghĩa gần nhau (toy poodle, miniature poodle) cộng một lớp khác xa (tench): không có label smoothing, hai lớp giống nhau nằm cụm gần nhau theo kiểu lan toả isotropic (gợi ý có thể đo "một con poodle giống một con cá tench đến mức nào"); có label smoothing, hai lớp giống nhau nằm trên một đường cung (arc), và thông tin về mức độ giống nhau đó "gần như bị xoá".

4.2. Calibration (Table 3 của paper)

Dataset	Kiến trúc	ECE baseline ($T = 1, \alpha = 0$)	ECE/temp. scaling	ECE/label smoothing
CIFAR-100	ResNet-56	0.150	0.021 ($T = 1{,}9$)	0.024 ($\alpha = 0{,}05$)
ImageNet	Inception-v4	0.071	0.022 ($T = 1{,}4$)	0.035 ($\alpha = 0{,}1$)
EN-DE	Transformer	0.056	0.018 ($T = 1{,}13$)	0.019 ($\alpha = 0{,}1$)

Cả temperature scaling và label smoothing đều giảm ECE đáng kể so với mô hình train bằng nhãn cứng không calibrate; hai cách cho kết quả tương đương nhau về bậc độ lớn. Với ImageNet, paper tự ghi "không tune α kỹ" (did not tune α extensively) khi chọn $\alpha = 0{,}1$ cho kết quả 0.035.

Với dịch máy (Transformer EN-DE): label smoothing cải thiện BLEU dù NLL (negative log-likelihood) kém hơn so với nhãn cứng ở mọi mức temperature scaling thử nghiệm. Temperature scaling áp lên mô hình train bằng nhãn cứng **không đủ** để đạt mức BLEU của mô hình train bằng label smoothing; mô hình label-smoothing được paper gọi là "tự động được calibrate" (automatically calibrated), và việc chỉnh nhiệt độ trên mô hình đó chỉ làm cả calibration và BLEU xấu đi. Paper tự kết luận: "sự khác biệt về BLEU chỉ được giải thích một phần bởi calibration" (chưa có lý giải đầy đủ).

4.3. Knowledge distillation (phần kết quả gây bất ngờ nhất của paper)

Thí nghiệm mở đầu, MNIST, fully-connected: teacher (không convolution) train bằng nhãn cứng + dropout trên MNIST đã dịch ngẫu nhiên (randomly translated), đạt 0.67% test error; distill sang một student hẹp hơn, không regularize, train trên ảnh gốc không dịch, student đạt 0.74% test error. Nếu thay dropout bằng label smoothing cho teacher: teacher train nhanh hơn và có kết quả tốt hơn một chút (0.59% error), **nhưng** distillation từ teacher này cho ra student kém hơn hẳn: 0.91% test error. Đây là ví dụ số rõ nhất cho thấy "có gì đó sai nghiêm trọng" khi distill từ một teacher được train bằng label smoothing, dù chính teacher đó tốt hơn.

Thí nghiệm CIFAR-10, ResNet-56 → AlexNet (Figure 5 của paper): 4 đường kết quả được so sánh trên cùng thước đo γ (hệ số smoothing tương đương):

- Accuracy của teacher tăng theo α tới $\alpha = 0{,}6$ rồi mới giảm.
- Accuracy của student train trực tiếp (không distill) với label smoothing giảm nhẹ so với không smoothing.
- Student distill từ teacher train bằng nhãn cứng, ở các nhiệt độ T khác nhau: tất cả đều vượt accuracy của student-trực-tiếp-với-label-smoothing.
- Student distill từ teacher train bằng label smoothing ($\alpha > 0$, các teacher này có accuracy tốt hơn): kết quả **không tốt hơn, thậm chí đôi khi xấu hơn** so với train student trực tiếp bằng label smoothing.

Kết luận paper rút ra trực tiếp: "một teacher có accuracy tốt hơn không nhất thiết là teacher distill tốt hơn".

4.4. Mutual information (Figure 6 của paper)

Ước lượng mutual information giữa chỉ số ví dụ ($N = 600$ từ 2 lớp) và chênh lệch logit của hai lớp đó qua các bước train: mutual information tăng nhanh lúc đầu, sau đó giảm dần, đặc biệt rõ với mạng train bằng label smoothing. Ở giai đoạn train muộn, mutual information của mạng label-smoothing giữ ở mức hơi trên $\log(2)$, tương ứng trường hợp cực đoan mọi ví dụ co về đúng 2 cluster (one bit thông tin, chỉ còn phân biệt được lớp, không còn thông tin phụ nào khác trong logit so với chính nhãn).

5. Phân tích / ablation

- **Teacher tốt hơn không đảm bảo distill tốt hơn** (Mục 4.3): đây là phát hiện trung tâm, đối lập với giả định ngầm phổ biến khi chọn teacher cho distillation (thường chọn teacher accuracy cao nhất có thể).

- **Cơ chế giải thích bằng hình học**: với teacher nhãn cứng, các ví dụ cùng lớp phân tán rộng, nghĩa là các ví dụ khác nhau của cùng lớp có thể có độ "giống" rất khác nhau với các lớp khác (thông tin hữu ích cho student học "cấu trúc tương tự giữa lớp"); với teacher label smoothing, mọi ví dụ bị ép cách đều mọi lớp sai, nên thông tin tương tự liên-lớp gần như đồng nhất, không còn gì đặc biệt để student học thêm so với chính nhãn gốc.
- **Mutual information giảm dần theo thời gian train, nhanh hơn với label smoothing**, là bằng chứng định lượng (dù là ước lượng xấp xỉ) hỗ trợ trực tiếp cho cơ chế hình học nói trên, không chỉ là suy luận định tính từ hình vẽ.
- **Hiệu ứng khác nhau giữa tác vụ**: CIFAR-10/AlexNet gần như không đổi accuracy khi thêm label smoothing (86.8 → 86.7, nằm trong sai số), nhưng CIFAR-100/ResNet-56 cải thiện rõ (72.1 → 72.7); paper không giải thích sâu vì sao có khác biệt này giữa hai bộ dữ liệu, chỉ quan sát và trình bày cả hai trường hợp.
- **Related work liên hệ chặt**: Pereyra et al. (2017) cho thấy label smoothing tương đương với "confidence penalty" (một regularizer phạt dự đoán entropy thấp) nếu đảo thứ tự KL-divergence giữa phân phối đều và output mô hình; họ cũng đề xuất unigram label smoothing (dùng phân phối theo tần suất lớp thay vì đều, có lợi khi nhãn mất cân bằng). DisturbLabel (Xie et al. 2016) liên hệ như "label dropout", còn label smoothing là phiên bản marginalize (lấy kỳ vọng) của label dropout đó.
- **Liên hệ information bottleneck**: Chelombiev et al. (2019) cho thấy tương quan giữa mức độ compress của representation ở softmax layer và generalization, phù hợp với lý thuyết information bottleneck (Tishby et al.) mà paper này dùng để gợi ý hướng nghiên cứu tiếp theo, nhưng paper không tự chứng minh mối liên hệ này một cách chặt chẽ, chỉ nêu như gợi ý.
- **Kornblith et al. (2018)**, cùng đồng tác giả Simon Kornblith với paper này, từng chỉ ra label smoothing làm giảm chất lượng transfer learning, với cùng cơ chế nghi vấn (mất thông tin không liên quan đến lớp ở các layer cuối); paper coi đây là bằng chứng gián tiếp ủng hộ cơ chế "xoá thông tin" mà paper đề xuất cho distillation.

6. Giới hạn

Các giới hạn tự thừa nhận hoặc được chính paper nêu rõ, cùng một vài quan sát của người ghi chú:

- **Chưa có giải thích đầy đủ cho kết quả dịch máy**: paper tự viết "sự khác biệt về BLEU chỉ được giải thích *một phần* bởi calibration" (chỉ *partly explained*), nghĩa là bản thân tác giả cũng không khẳng định đã hiểu hết cơ chế vì sao label smoothing cải thiện BLEU trong khi NLL xấu hơn.
- **Không mở rộng phân tích distillation sang dịch máy**: ở Appendix B, paper viết rõ "chúng tôi để lại phân tích distillation cho dịch máy cho công việc tương lai" (leave analysis of distillation for translation for future work). Các khác biệt giữa classification và translation mà paper tự liệt kê khiến việc suy rộng trực tiếp kết luận chính (label smoothing hại distillation) sang mô hình sinh chuỗi là *chưa được kiểm chứng* trong bài, chỉ là khả năng hợp lý: (a) phân phối token trong dịch máy mất cân bằng (không giống phân phối lớp cân bằng của ảnh), paper gợi ý unigram label smoothing "có thể" giải quyết nhưng không thử nghiệm; (b) training accuracy của next-token prediction chỉ khoảng 80% (so với gần như hoàn hảo ở classification ảnh), nên representation lúc train lẫn validation đều có lỗi, và paper tự nhận "không rõ [distillation] sẽ hoạt động tốt hơn với teacher nào" (it is unclear which will perform best) trong trường hợp này; (c) không gian embedding (khoảng 512 chiều) nhỏ hơn số token (~30 nghìn), nên không thể tạo simplex đều như ở classification ảnh (penultimate dimension > số lớp), làm trực giác hình học ở Mục 3.2 không áp dụng được trực tiếp.
- **Phép đo mutual information là một xấp xỉ, không phải giá trị chính xác**: paper tự nói "tính mutual information ở chiều cao với phân phối chưa biết là thách thức, nhưng ở đây chúng tôi đơn giản hoá vấn đề theo vài cách" (simplify the problem in several ways), dùng xấp xỉ Gaussian cho một biến đơn chiều (chênh lệch hai logit) và chỉ ước lượng trên một tập con nhỏ (600 ví dụ, 2 lớp); đây là bằng chứng hỗ trợ, không phải chứng minh chặt cho giả thuyết "xoá thông tin".

- **Phạm vi thực nghiệm còn nhỏ theo chuẩn hiện tại**: các mô hình dùng để chứng minh cơ chế chính (MNIST FC 2 layer, AlexNet/CIFAR-10, ResNet-56/CIFAR-100) là các mô hình nhỏ; Inception-v4/ImageNet chỉ dùng cho visualization/calibration, không dùng trong thí nghiệm distillation chính, nên chưa có bằng chứng trực tiếp trong bài rằng hiệu ứng "label smoothing hại distillation" vẫn giữ nguyên ở quy mô model lớn hơn nhiều (ví dụ các mạng ImageNet SOTA).
- **Không tune hyperparameter kỹ cho một số thí nghiệm**: paper tự nói không tune α kỹ cho thí nghiệm calibration trên ImageNet; do đó giá trị ECE 0.035 ở $\alpha = 0{,}1$ không chắc là mức tốt nhất có thể đạt bằng label smoothing trên ImageNet.
- **Quan sát của người ghi chú (không phải paper tự nói)**: Table 2 báo accuracy ImageNet/Inception-v4 dưới dạng một giá trị đơn (80.9 so với 80.9, không kèm sai số), khác với hai dòng CIFAR có kèm $\pm$ sai số từ nhiều seed; đây chỉ là khác biệt cách báo cáo số liệu giữa các thí nghiệm, không phải mâu thuẫn trong paper, nhưng người đọc nên lưu ý khi so sánh độ tin cậy giữa các dòng của bảng.

7. Liên hệ với các bài khác

- **distillation 02** (Hinton, Vinyals, Dean, Distilling the Knowledge in a Neural Network, 2015): liên hệ trực tiếp và quan trọng nhất. Loss distillation ở Mục 3.4 của bài này chính là công thức tổng quát hoá loss gốc trong bài đó; và vì Geoffrey Hinton là đồng tác giả của cả hai bài, paper này có thể coi là lời cảnh báo "ngược" từ chính tác giả gốc của distillation: nếu teacher được train với label smoothing, phương pháp distillation kinh điển có thể thất bại.
- **ml basics 29** (Noisy Student): cả hai bài đều dùng khung teacher-student và soft target/pseudo-label, nhưng theo hướng đối lập: Noisy Student dùng soft pseudo-label để *mở rộng* khả năng của student (student lớn hơn/bằng, học thêm từ dữ liệu không nhãn), còn bài này cho thấy một loại soft target khác (label smoothing) có thể *phá hỏng* việc chuyển giao tri thức qua distillation.
- **ml basics 19** (On Calibration of Modern Neural Networks, Guo et al. 2017): được trích dẫn trực tiếp làm nền cho khái niệm ECE và temperature scaling; bài này mở rộng bằng cách cho thấy label smoothing cũng calibrate mô hình, theo một cách khác với temperature scaling.
- **ml basics 27** (How transferable are features in deep neural networks?): liên hệ qua Kornblith et al. (2018, "Do better imagenet models transfer better?", cùng đồng tác giả Simon Kornblith) được trích dẫn trong Related Work, cho thấy label smoothing cũng làm giảm chất lượng transfer learning, cùng cơ chế mất thông tin không-liên-quan-đến-lớp ở layer cuối mà bài 27 phân tích theo một góc khác (fragile co-adaptation giữa các layer).
- **math 30** (Opening the Black Box of Deep Neural Networks via Information, Tishby & Zaslavsky): được trích dẫn trực tiếp trong Related Work như nền tảng cho information bottleneck theory, lý thuyết mà paper này gợi ý có thể giải thích vì sao mạng train bằng label smoothing vẫn generalize tốt dù representation bị "compress" mạnh.
- **ml basics 31** (mixup, theo quy ước số của thư mục): cùng là kỹ thuật làm "mềm" mục tiêu huấn luyện (mixup trộn cả input và label của hai ví dụ, label smoothing chỉ trộn label với phân phối đều) ảnh hưởng tới generalization và calibration; một so sánh trực tiếp giữa hai kỹ thuật là hướng đọc tiếp tự nhiên.

8. Gợi ý áp dụng

Label smoothing thay đổi mục tiêu huấn luyện và có thể ảnh hưởng cả accuracy, calibration lẫn distillation. Vì các tác động này khác nhau theo tác vụ, cần đánh giá riêng từng mục tiêu thay vì chỉ dựa vào một chỉ số.

9. Thí nghiệm gợi ý

Tất cả chạy được với model/dataset nhỏ:

1. **Tái hiện thí nghiệm MNIST FC teacher-student** (gần sao chép Mục 4.3 của paper, nhẹ tới mức có thể chạy cả trên CPU): train teacher FC 2 layer (ví dụ 256 neuron/layer thay vì 1200 để nhanh hơn) với

(a) dropout, nhãn cứng, và (b) không dropout, label smoothing $\alpha = 0{,}1$; distill cả hai sang cùng một student FC nhỏ hơn (ví dụ 128 neuron/layer). Đo test error của teacher và student mỗi trường hợp. Kỳ vọng: teacher (b) có error thấp hơn hoặc bằng teacher (a), nhưng student distill từ (b) có error cao hơn hẳn student distill từ (a), lặp lại hiện tượng chính của paper ở quy mô rất nhỏ.

2. **Quét α trên CIFAR-10 với một CNN nhỏ** (ví dụ ResNet-8 hoặc một CNN 4-5 layer): train teacher với $\alpha \in \{0, 0{,}05, 0{,}1, 0{,}2, 0{,}4\}$, đo accuracy teacher; sau đó distill từng teacher sang một student cố định (CNN nhỏ hơn), đo accuracy student. Vẽ đường accuracy-teacher và accuracy-student-sau-distill theo α, kiểm tra có xuất hiện xu hướng giống Figure 5 của paper (student không tăng theo α dù teacher tăng) không.
3. **Đo ECE/reliability diagram nhỏ**: train một CNN nhỏ trên CIFAR-10 với $\alpha = 0$ và $\alpha = 0{,}1$, tính ECE 10-15 bin trên validation set, vẽ reliability diagram, so sánh với bản $\alpha = 0$ đã temperature-scale (tìm T tối ưu bằng grid search đơn giản trên validation). Kỳ vọng theo paper: cả hai cách (label smoothing và temperature scaling) giảm ECE về mức tương đương.
4. **Proxy đơn giản cho mutual information**: chọn 2 lớp từ CIFAR-10 (ví dụ "airplane" và "bird"), train một CNN nhỏ với $\alpha = 0$ và $\alpha = 0{,}1$, mỗi vài trăm bước train lưu lại chênh lệch logit của khoảng 100-200 ví dụ cố định (dùng cùng augmentation ngẫu nhiên nhiều lần để có nhiều mẫu Monte Carlo mỗi ví dụ). Tính phương sai trung bình giữa các ví dụ của chênh lệch logit (proxy đơn giản hơn công thức mutual information đầy đủ của paper) theo thời gian train, so sánh xu hướng giữa $\alpha = 0$ và $\alpha = 0{,}1$; kỳ vọng phương sai giữa-ví-dụ giảm nhanh và mạnh hơn với label smoothing, phản ánh ý "co cụm" ở Mục 3.2.
5. **Tái hiện visualization hình tam giác**: train một CNN nhỏ trên 3 lớp CIFAR-10 (ví dụ "airplane", "automobile", "bird") với $\alpha = 0$ và $\alpha = 0{,}1$, lấy vector trọng số 3 lớp ở layer cuối làm 3 template, tìm basis trực chuẩn của mặt phẳng qua 3 template đó (ví dụ bằng Gram-Schmidt), chiếu activation penultimate layer của các ví dụ train/validation lên mặt phẳng này và vẽ scatter plot. Kỳ vọng quan sát trực tiếp: cluster co chặt hơn, có dạng tam giác đều rõ hơn khi có label smoothing, giống Figure 1 của paper.

10. Tóm tắt công thức / khái niệm

Softmax và cross-entropy:

$$p_k = \frac{e^{x^T w_k}}{\sum_{l=1}^{K} e^{x^T w_l}}, \qquad H(y, p) = \sum_{k=1}^{K} -y_k \log(p_k)$$

Label smoothing:

$$y_k^{LS} = y_k(1 - \alpha) + \alpha/K \qquad \text{(tự tính, } K{=}10,\ \alpha{=}0{,}1\text{: lớp đúng } 0{,}91,\ \text{lớp sai } 0{,}01 \text{ mỗi lớp)}$$

Khoảng cách tới template (giải thích hình học của logit):

$$\|x - w_k\|^2 = x^T x - 2x^T w_k + w_k^T w_k$$

Loss distillation tổng quát:

$$\mathcal{L} = (1 - \beta)H(y, p) + \beta H\big(p^t(T), p(T)\big)$$

Hệ số label smoothing tương đương (so sánh công bằng giữa train-trực-tiếp và distillation):

$$\gamma_{\text{train trực tiếp}} = \alpha, \qquad \gamma_{\text{distillation}} = \mathbb{E}\Big[\sum_{k=1}^{K}(1-y_k)\,p_k^t(T)\,\frac{K}{K-1}\Big]$$

Mutual information xấp xỉ giữa ví dụ và chênh lệch logit:

$$\hat{I}(X;Y) = \frac{1}{N}\sum_{x=1}^{N}\Big[-\frac{(f(d(z_x))-\mu_x)^2}{2\sigma^2} - \log\Big(\frac{1}{N}\sum_{x=1}^{N} e^{-(f(d(z_x))-\mu_x)^2/(2\sigma^2)}\Big)\Big], \quad I \in [0, \log N]$$

Bảng số liệu chính cần nhớ:

Hiện tượng	Số liệu
Teacher hard+dropout (MNIST)	0.67% test error
Student distill từ teacher đó	0.74% test error
Teacher label-smoothing (MNIST)	0.59% test error (tốt hơn)
Student distill từ teacher LS đó	0.91% test error (tệ hơn nhiều)
ECE CIFAR-100/ResNet-56	$0{,}150 \to 0{,}024$ (LS $\alpha = 0{,}05$) hoặc $\to 0{,}021$ (temp. scaling)
BLEU Transformer EN-DE	$25{,}3 \to 25{,}8$ (LS), dù perplexity xấu hơn $4{,}67 \to 4{,}92$

11. Kết luận ngắn

Label smoothing cải thiện generalization và calibration trên nhiều tác vụ (image classification, dịch máy, nhận dạng giọng nói) nhờ ép các activation penultimate layer của cùng một lớp co cụm chặt, cách đều template các lớp khác; nhưng chính cơ chế co cụm đó xoá đi thông tin về mức độ "giống nhau" giữa các lớp vốn tiềm ẩn trong logit, thông tin không cần cho calibration/generalization nhưng lại thiết yếu cho knowledge distillation. Hệ quả thực tiễn quan trọng: nếu teacher được train bằng label smoothing, dù teacher đó có accuracy tốt hơn, việc distill tri thức sang một student khác có thể cho kết quả kém hơn cả việc train student trực tiếp bằng label smoothing. Phát hiện này, cùng với phép đo mutual information hỗ trợ (dù chỉ là một xấp xỉ), buộc người thiết kế pipeline distillation phải kiểm tra lại giả định "chọn teacher tốt nhất luôn luôn tốt cho distillation", và paper tự nhận còn nhiều câu hỏi mở, đặc biệt là liệu hiệu ứng tương tự có xảy ra với mô hình sinh chuỗi (dịch máy, và có thể cả OCR) hay không, vì paper chỉ kiểm chứng trên classification ảnh.

Nguồn gốc: https://arxiv.org/abs/1906.02629

Bài 31

mixup: Beyond Empirical Risk Minimization (2017)

Tác giả: Hongyi Zhang (MIT); Moustapha Cisse, Yann N. Dauphin, David Lopez-Paz (Facebook AI Research, FAIR). (Paper tự ghi chú tên tác giả xếp theo thứ tự alphabet, không phản ánh mức đóng góp.)
Venue / arXiv: arXiv:1710.09412 (v2, 27/04/2018); paper có nêu rõ venue ngay trên mỗi trang: "Published as a conference paper at ICLR 2018".
Nguồn: https://arxiv.org/abs/1710.09412
Chủ đề: đề xuất mixup, một nguyên lý data augmentation đơn giản, không phụ thuộc domain, thay thế Empirical Risk Minimization (ERM) bằng Vicinal Risk Minimization (VRM): train trên tổ hợp lồi (convex combination) của cặp ví dụ ngẫu nhiên và nhãn one-hot tương ứng của chúng, nhằm cải thiện generalization, giảm memorization nhãn nhiễu, tăng robustness trước adversarial example, và ổn định huấn luyện GAN.
Vị trí trong lộ trình: Đọc cùng bài 30 về label smoothing và bài 32 về focal loss.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Hầu hết mạng neural sâu hiện đại được train bằng nguyên lý Empirical Risk Minimization (ERM, Vapnik 1998): tối thiểu hoá sai số trung bình trên dữ liệu train. Đồng thời, kích thước các mạng SOTA tăng tuyến tính theo số lượng ví dụ train (paper dẫn ví dụ: Springenberg et al. dùng 10^6 tham số cho 5×10^4 ảnh CIFAR-10; Simonyan & Zisserman dùng 10^8 tham số cho 10^6 ảnh ImageNet; Chelba et al. dùng 2×10^{10} tham số cho 10^9 từ trong One Billion Word).

Bài báo chỉ ra một mâu thuẫn lý thuyết: một kết quả cổ điển (Vapnik & Chervonenkis 1971) đảm bảo ERM hội tụ chỉ khi kích thước learning machine **không** tăng theo số lượng dữ liệu train (đo bằng số tham số hoặc VC-complexity). Thực tế các mạng hiện đại đi ngược hoàn toàn giả định này. Hệ quả quan sát được: (a) ERM cho phép mạng lớn ghi nhớ (memorize) dữ liệu train thay vì tổng quát hoá, thậm chí khi có regularization mạnh hoặc khi nhãn bị gán ngẫu nhiên (Zhang et al. 2017, "Understanding deep learning requires rethinking generalization"); (b) mạng train bằng ERM đổi dự đoán rất mạnh khi gặp ví dụ chỉ lệch nhẹ khỏi phân phối train (adversarial example, Szegedy et al. 2014).

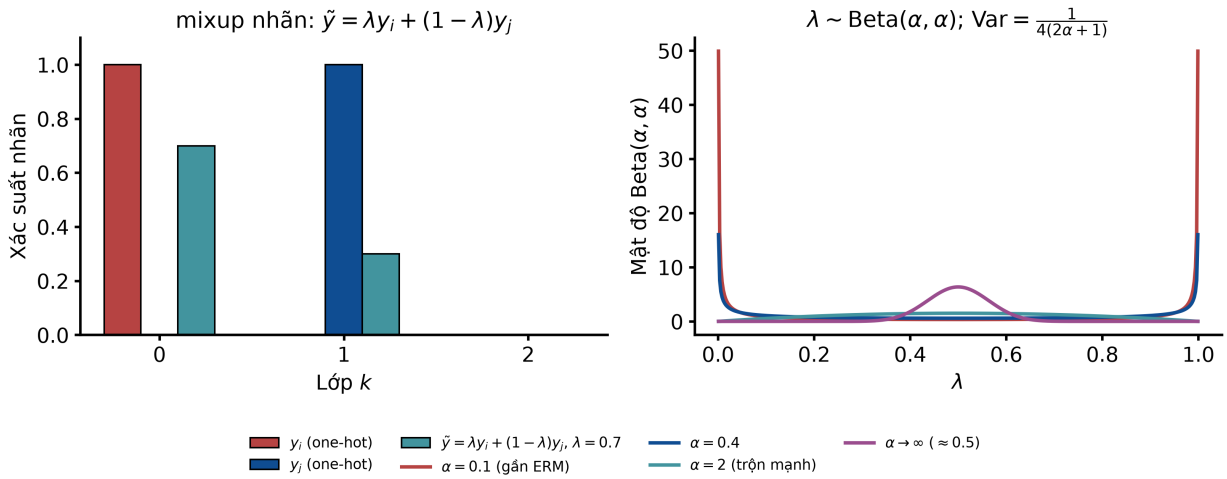
Một hướng thay thế là data augmentation, được hình thức hoá bởi nguyên lý Vicinal Risk Minimization (VRM, Chapelle et al. 2000): định nghĩa một "vùng lân cận" (vicinity) xung quanh mỗi ví dụ train dựa trên tri thức con người (ví dụ với ảnh: lật ngang, xoay nhẹ, scale nhẹ), rồi lấy thêm ví dụ "ảo" từ phân phối lân cận đó để mở rộng support của phân phối train. Nhược điểm paper chỉ ra: (1) cách định nghĩa vicinity phụ thuộc tập dữ liệu, cần tri thức chuyên gia; (2) vicinity truyền thống giả định các ví dụ lân cận **cùng lớp** với ví dụ gốc, không mô hình hoá quan hệ lân cận **giữa các lớp khác nhau**.

2. Ý tưởng chính

mixup là một phân phối vicinal tổng quát, không phụ thuộc domain: xây dựng ví dụ train ảo bằng tổ hợp lồi của **cả input và label** của hai ví dụ ngẫu nhiên:

$$\tilde{x} = \lambda x_i + (1-\lambda) x_j, \qquad \tilde{y} = \lambda y_i + (1-\lambda) y_j$$

với (x_i, y_i), (x_j, y_j) là hai ví dụ lấy ngẫu nhiên từ dữ liệu train, y_i, y_j là nhãn one-hot, và $\lambda \in [0, 1]$.



mixup nhãn $\tilde{y} = \lambda y_i + (1-\lambda) y_j$ và Beta(α, α): α nhỏ gần ERM (minh hoạ, tự tính)

mixup đưa vào tri thức tiên nghiệm: nội suy tuyến tính giữa hai feature vector nên dẫn tới nội suy tuyến tính giữa nhãn tương ứng của chúng. Hiệu ứng mong muốn: khuyến khích mô hình hành xử **tuyến tính** ở vùng giữa các ví dụ train, giảm dao động (oscillation) không mong muốn khi dự đoán ở vùng ngoài các điểm train cụ thể; đây cũng là một thiên kiến quy nạp (inductive bias) hợp lý theo nguyên lý "dao cạo Occam" (hành vi tuyến tính là một trong các hành vi đơn giản nhất có thể).

mixup cài đặt được trong vài dòng code, overhead tính toán gần như không đáng kể. Theo thực nghiệm của bài báo, mixup cải thiện SOTA trên ImageNet-2012, CIFAR-10, CIFAR-100; cải thiện generalization trên dữ liệu giọng nói (Google commands) và dữ liệu dạng bảng (UCI); giảm memorization nhãn nhiễu; tăng robustness trước adversarial example; và ổn định huấn luyện GAN.

3. Phương pháp chi tiết

3.1. Từ ERM tới VRM

Trong supervised learning, ta muốn học hàm f xấp xỉ quan hệ giữa X và Y theo phân phối chung $P(X, Y)$, bằng cách tối thiểu hoá expected risk:

$$R(f) = \int \ell(f(x), y)\, dP(x, y)$$

Vì P không biết trước, ta xấp xỉ bằng phân phối thực nghiệm (empirical distribution, Dirac mass tại mỗi điểm dữ liệu):

$$P_\delta(x, y) = \frac{1}{n} \sum_{i=1}^{n} \delta(x = x_i, y = y_i)$$

dẫn tới empirical risk, chính là ERM:

$$R_\delta(f) = \frac{1}{n} \sum_{i=1}^{n} \ell(f(x_i), y_i) \qquad (1)$$

Vấn đề: với f có số tham số tương đương n (mạng lớn), một cách tối thiểu hoá (1) là ghi nhớ (memorize) toàn bộ dữ liệu train, dẫn tới hành vi xấu ngoài tập train.

VRM thay P_δ bằng:

$$P_\nu(\tilde{x}, \tilde{y}) = \frac{1}{n} \sum_{i=1}^{n} \nu(\tilde{x}, \tilde{y} \mid x_i, y_i)$$

với ν là một "vicinity distribution" đo xác suất tìm thấy cặp feature-target ảo $(\tilde{x}, \tilde{y})$ ở gần (x_i, y_i). Ví dụ cổ điển (Chapelle et al. 2000): vicinity Gaussian $\nu(\tilde{x}, \tilde{y}|x_i, y_i) = \mathcal{N}(\tilde{x} - x_i, \sigma^2)\,\delta(\tilde{y} = y_i)$, tương đương cộng nhiễu Gaussian vào input và **giữ nguyên nhãn**. Risk tương ứng (empirical vicinal risk):

$$R_\nu(f) = \frac{1}{m}\sum_{i=1}^{m} \ell(f(\tilde{x}_i), \tilde{y}_i)$$

3.2. Phân phối vicinal mixup

Paper đề xuất phân phối vicinal tổng quát sau, gọi là mixup:

$$\mu(\tilde{x}, \tilde{y} \mid x_i, y_i) = \frac{1}{n}\sum_{j} \mathbb{E}_\lambda[\delta(\tilde{x} = \lambda x_i + (1-\lambda)x_j,\ \tilde{y} = \lambda y_i + (1-\lambda)y_j)]$$

với $\lambda \sim \text{Beta}(\alpha, \alpha)$, $\alpha \in (0, \infty)$. Lấy mẫu từ phân phối này cho ra đúng công thức $\tilde{x}, \tilde{y}$ ở Mục 2. Hyperparameter α điều khiển độ mạnh của nội suy; $\alpha \to 0$ thì phân phối $\text{Beta}(\alpha, \alpha)$ dồn xác suất về hai đầu $\{0, 1\}$ (hầu như luôn lấy nguyên một trong hai ví dụ gốc), khôi phục lại ERM.

(Tự tính: phân phối $\text{Beta}(\alpha, \alpha)$ luôn có trung bình $0{,}5$ (do đối xứng) và phương sai $\text{Var}(\lambda) = \dfrac{1}{4(2\alpha+1)}$. Khi $\alpha \to 0$, phương sai $\to \dfrac{1}{4} = 0{,}25$, giá trị phương sai lớn nhất có thể của một biến trong $[0, 1]$, khớp với việc phân phối dồn về hai đầu mút 0 và 1 (tức gần như không trộn, giữ nguyên một ví dụ gốc, đúng như paper nói "$\alpha \to 0$ khôi phục ERM"). Khi $\alpha \to \infty$, phương sai $\to 0$, nghĩa là λ gần như luôn bằng $0{,}5$ (trộn đều 50/50 hai ví dụ bất kể ví dụ nào), đây là mức trộn **mạnh nhất** có thể, không phải ERM, và giải thích vì sao paper quan sát "α lớn dẫn tới underfitting": ví dụ ảo lúc đó gần như luôn là trung bình của hai ảnh ngẫu nhiên, rất khác ảnh gốc.)

Cài đặt (Figure 1a của paper, PyTorch, một epoch mixup):

```
for (x1, y1), (x2, y2) in zip(loader1, loader2):
    lam = numpy.random.beta(alpha, alpha)
    x = Variable(lam * x1 + (1. - lam) * x2)
    y = Variable(lam * y1 + (1. - lam) * y2)
    optimizer.zero_grad()
    loss(net(x), y).backward()
    optimizer.step()
```

(y1, y2 phải là vector one-hot.)

Một số lựa chọn thiết kế khác đã thử nhưng **không** mang lại lợi ích (paper tự nêu ở Mục 2, kết quả từ thí nghiệm sơ bộ, không có bảng số chi tiết):

- Tổ hợp lồi của 3 ví dụ hoặc nhiều hơn, với trọng số lấy từ phân phối Dirichlet: không cho thêm lợi ích, chỉ tăng chi phí tính toán.
- Chỉ trộn trong cùng một minibatch (shuffle rồi trộn với chính minibatch đó) thay vì dùng 2 data loader riêng: hoạt động tương đương, giảm yêu cầu I/O (đây là lựa chọn được paper giữ lại dùng chính thức).
- Chỉ nội suy giữa các input **cùng nhãn**: không cho lợi ích như mixup đầy đủ (nội suy giữa các lớp khác nhau).

3.3. Mở rộng cho GAN

Huấn luyện GAN chuẩn là bài toán min-max:

$$\max_g \min_d\ \mathbb{E}_{x,z}[\ell(d(x), 1) + \ell(d(g(z)), 0)]$$

với ℓ là binary cross-entropy. Paper lập luận mixup giúp ổn định việc train GAN vì đóng vai trò regularizer trên gradient của discriminator (tương tự hiệu ứng làm mượt gradient quan sát ở Figure 2 cho bài toán phân loại), cho generator một nguồn gradient ổn định hơn. Công thức mixup-GAN:

$$\max_{g} \min_{d} \; \mathbb{E}_{x,z,\lambda}[\ell(d(\lambda x + (1-\lambda)g(z)),\, \lambda)]$$

4. Thí nghiệm và kết quả chính

4.1. ImageNet-2012 (Table 1)

Train trên 1,3 triệu ảnh, 1000 lớp, đánh giá trên 50.000 ảnh validation, dùng augmentation chuẩn (scale/aspect distortion, random crop, horizontal flip), batch 1024, lịch learning rate theo Goyal et al. 2017 (tăng tuyến tính 0.1 lên 0.4 trong 5 epoch đầu, chia 10 sau mỗi cột mốc epoch).

Model	Method	Epochs	Top-1 Error	Top-5 Error
ResNet-50	ERM	90	23.5	-
ResNet-50	mixup α=0.2	90	23.3	6.6
ResNet-101	ERM	90	22.1	-
ResNet-101	mixup α=0.2	90	21.5	5.6
ResNeXt-101 32x4d	ERM	90	21.2	5.6
ResNeXt-101 32x4d	mixup α=0.4	90	20.7	5.3
ResNeXt-101 64x4d	ERM (Xie et al., 100ep)	100	20.4	5.3
ResNeXt-101 64x4d	mixup α=0.4	90	19.8	4.9
ResNet-50	ERM	200	23.6	7.0
ResNet-50	mixup α=0.2	200	22.1	6.1
ResNet-101	ERM	200	22.0	6.1
ResNet-101	mixup α=0.2	200	20.8	5.4
ResNeXt-101 32x4d	ERM	200	21.3	5.9
ResNeXt-101 32x4d	mixup α=0.4	200	20.1	5.0

(Tự tính, kiểm tra lại các claim định lượng mà paper nêu bằng lời: ResNet-101 90 epoch cải thiện $22{,}1 - 21{,}5 = 0{,}6$ điểm phần trăm, ResNet-50 90 epoch cải thiện $23{,}5 - 23{,}3 = 0{,}2$ điểm phần trăm, đúng khớp với câu "mixup cho ResNet-101/ResNeXt-101 cải thiện 0,5-0,6% còn ResNet-50 chỉ 0,2%" của paper. ResNet-50 mixup ở 200 epoch (22.1) so với ResNet-50 mixup ở 90 epoch (23.3): cải thiện thêm $23{,}3 - 22{,}1 = 1{,}2$ điểm phần trăm, khớp đúng số "1.2%" paper nêu; còn ResNet-50 ERM 200 epoch (23.6) so với 90 epoch (23.5) hầu như không đổi, khớp với câu "ERM giữ nguyên".)

Phạm vi α tốt: $\alpha \in [0{,}1, 0{,}4]$; α lớn hơn dẫn tới underfitting. Model dung lượng lớn hơn và/hoặc train lâu hơn hưởng lợi nhiều hơn từ mixup.

4.2. CIFAR-10 và CIFAR-100 (Figure 3)

Dataset	Model	ERM	mixup
CIFAR-10	PreAct ResNet-18	5.6	4.2
CIFAR-10	WideResNet-28-10	3.8	2.7
CIFAR-10	DenseNet-BC-190	3.7	2.7
CIFAR-100	PreAct ResNet-18	25.6	21.1
CIFAR-100	WideResNet-28-10	19.4	17.5

Dataset	Model	ERM	mixup
CIFAR-100	DenseNet-BC-190	19.0	16.8

$\alpha = 1$ cố định (λ phân phối đều trên $[0, 1]$); train 200 epoch, batch 128, 1 GPU Tesla P100, weight decay 10^{-4}, không dùng dropout. mixup và ERM hội tụ về test error tốt nhất với tốc độ tương tự nhau (mixup không làm chậm hội tụ dù regularization mạnh hơn).

4.3. Dữ liệu giọng nói (Google commands, Figure 4)

Dataset: 65.000 utterance, mỗi utterance khoảng 1 giây, 30 lớp lệnh giọng nói (yes, no, down, left, v.v.), nhiều nghìn người nói khác nhau; tiền xử lý thành spectrogram chuẩn hoá 160×101, mixup áp dụng ở mức spectrogram. Hai kiến trúc: LeNet và VGG-11 (mỗi kiến trúc gồm 2 lớp convolution và 2 lớp fully-connected theo mô tả của paper), train 30 epoch, Adam, warm-up 5 epoch không mixup.

Model	Method	Validation	Test
LeNet	ERM	9.8	10.3
LeNet	mixup α=0.1	10.1	10.8
LeNet	mixup α=0.2	10.2	11.3
VGG-11	ERM	5.0	4.6
VGG-11	mixup α=0.1	4.0	3.8
VGG-11	mixup α=0.2	3.9	3.4

Lưu ý quan trọng (số liệu trực tiếp từ paper, không phải diễn giải thêm): với LeNet, mixup làm kết quả **xấu hơn** ERM ở cả hai giá trị α; với VGG-11 (dung lượng lớn hơn), mixup cải thiện rõ rệt.

4.4. Memorization nhãn nhiễu (Table 2)

Theo phương pháp của Zhang et al. 2017, làm nhiễu 20%/50%/80% nhãn CIFAR-10 (random), giữ nguyên nhãn test. So sánh ERM, ERM+dropout, mixup, mixup+dropout. Ví dụ tại 20% nhiễu: ERM test error tốt nhất/cuối cùng 12.7/16.6 (train error trên nhãn thật/nhãn nhiễu: 0.05/0.28); ERM+dropout (p=0.7): 8.8/10.4 (5.26/83.55); mixup (α=8): 5.9/6.4 (2.27/86.32); mixup+dropout (α=4, p=0.1): 6.2/6.2 (1.92/85.02). Mẫu hình tương tự tiếp tục ở 50% và 80% nhiễu (xem Mục 5 để phân tích).

4.5. Robustness trước adversarial example (Table 3)

Dùng 3 model ResNet-101/ImageNet (2 train ERM, 1 train mixup), tấn công FGSM và I-FGSM ($\epsilon = 4$/pixel, I-FGSM 10 bước):

Kịch bản	Metric	ERM	mixup
White-box FGSM	Top-1 error	90.7	75.2
White-box FGSM	Top-5 error	63.1	49.1
White-box I-FGSM	Top-1 error	99.9	99.6
White-box I-FGSM	Top-5 error	93.4	95.8
Black-box FGSM	Top-1 error	57.0	46.0
Black-box FGSM	Top-5 error	24.8	17.4
Black-box I-FGSM	Top-1 error	57.3	40.9
Black-box I-FGSM	Top-5 error	18.1	11.8

Paper phát biểu các tỷ lệ "mạnh hơn X lần". (Tự tính, suy ra cách tính các tỷ lệ này từ bảng số: với FGSM white-box, phần "còn đúng" là $100-90{,}7=9{,}3$ (ERM) và $100-75{,}2=24{,}8$ (mixup); tỷ lệ $24{,}8/9{,}3\approx 2{,}67$, khớp với con số "2.7 lần" paper nêu. Với FGSM black-box: $100-57{,}0=43{,}0$ và $100-46{,}0=54{,}0$, tỷ lệ $54{,}0/43{,}0\approx 1{,}26$, khớp "1.25 lần". Với I-FGSM black-box: $100-57{,}3=42{,}7$ và $100-40{,}9=59{,}1$, tỷ lệ $59{,}1/42{,}7\approx 1{,}38$, khớp "khoảng 40% mạnh hơn" (tăng 38%).) Với white-box I-FGSM, paper tự nhận cả hai mô hình "không robust" (lỗi gần 100%); và ở đúng ô này, Top-5 error của mixup (95.8) thực ra **cao hơn** (tệ hơn) ERM (93.4), một ngoại lệ so với xu hướng chung mà paper không bàn riêng.

4.6. Dữ liệu dạng bảng (UCI, Table 4)

Mạng fully-connected 2 lớp ẩn, 128 ReLU unit mỗi lớp, Adam mặc định, 10 epoch, batch 16:

Dataset	ERM	mixup	Dataset	ERM	mixup
Abalone	74.0	73.6	Htru2	2.0	2.0
Arcene	57.6	48.0	Iris	21.3	17.3
Arrhythmia	56.6	46.3	Phishing	16.3	15.2

mixup cải thiện 4/6 dataset, không có trường hợp nào tệ hơn ERM (Htru2 hoà).

4.7. Ổn định huấn luyện GAN

Thử nghiệm định tính (Figure 5) trên 2 dataset toy 2D, mạng fully-connected 3 lớp ẩn 512 ReLU unit, train 20.000 minibatch (128 mẫu/batch), Adam, discriminator train 5 bước mỗi bước generator. Paper quan sát (không có số liệu định lượng kiểu FID hay tương tự, chỉ hình ảnh mẫu qua các bước 10/100/1000/10000/20000) rằng mixup-GAN (α=0.2) ổn định hơn GAN thường theo thời gian train.

5. Phân tích / ablation

5.1. Ablation study chi tiết (Table 5, PreAct ResNet-18/CIFAR-10)

So sánh mixup với các biến thể thiết kế khác, hai mức weight decay (10^{-4} và 5×10^{-4}):

Phương pháp	wd 10^{-4}	wd 5×10^{-4}
ERM	5.53	5.18
mixup (AC+RP, chuẩn)	4.24	4.68
mixup (AC+KNN)	4.98	5.26
mix label+latent, Layer 1	4.44	4.51
mix label+latent, Layer 5 (gần output nhất)	5.39	5.15
mix input only (AC+RP, gán nhãn của mẫu trội hơn)	5.17	5.72
label smoothing ($\epsilon=0{,}1$)	5.33	5.17
mix input + label smoothing ($\epsilon=0{,}2$, AC+RP)	4.98	5.06
thêm Gaussian noise ($\sigma=0{,}2$)	7.16	7.24

(AC: trộn giữa mọi lớp; SC: trộn trong cùng lớp; RP: trộn cặp ngẫu nhiên; KNN: trộn với 200-nearest-neighbor.)

Quan sát chính của paper: (1) mixup chuẩn (trộn cả input và label, ở mức raw input, cặp ngẫu nhiên từ mọi lớp) là tốt nhất, hơn hẳn phương pháp tốt thứ hai (mix input + label smoothing). (2) ERM thích weight decay lớn (5×10^{-4} tốt hơn 10^{-4}: 5.18 so với 5.53), còn mixup thích weight decay nhỏ (10^{-4} tốt hơn: 4.24 so với 4.68), xác nhận mixup tự thân đã có hiệu ứng regularization mạnh, làm giảm nhu cầu weight decay. (3) Trộn

ở layer latent càng cao (gần output) thì hiệu ứng regularization càng yếu đi (thể hiện qua lợi thế của weight decay lớn tăng dần từ Layer 1 đến Layer 5). (4) Trộn chỉ input (không trộn label) là regularizer mạnh nhất trong các biến thể "chỉ input"; label smoothing và thêm Gaussian noise có hiệu ứng regularization tương đối nhỏ. (5) SMOTE (Chawla et al. 2002, tương tự SC+KNN) không cho lợi ích đáng kể.

5.2. Thảo luận về α, dung lượng model, và bias-variance (Mục 5 của paper)

Paper quan sát một xu hướng nhất quán: α tăng thì training error trên dữ liệu thật tăng, còn generalization gap giảm, ủng hộ giả thuyết mixup kiểm soát độ phức tạp mô hình một cách ngầm định. Nhưng paper tự nhận **chưa có lý thuyết tốt** để xác định "điểm ngọt" (sweet spot) của trade-off bias-variance này.

Đáng chú ý, hành vi này **khác nhau giữa dataset**: trên CIFAR-10, training error trên dữ liệu thật vẫn rất thấp dù $\alpha \to \infty$; trên ImageNet, training error tăng đáng kể khi $\alpha \to \infty$. Từ các thí nghiệm ImageNet và Google commands với nhiều kiến trúc khác nhau, paper **suy đoán (conjecture, chưa kiểm chứng trực tiếp bằng một thí nghiệm có kiểm soát biến dung lượng)** rằng model dung lượng lớn hơn sẽ ít nhạy với α lớn hơn, từ đó hưởng lợi nhiều hơn từ mixup. Kết quả LeNet-kém/VGG-11-tốt ở Mục 4.3 khớp với suy đoán này (LeNet dung lượng nhỏ, bị mixup làm xấu đi; VGG-11 dung lượng lớn hơn, được lợi).

5.3. Ngoại lệ trong kết quả adversarial

Như đã nêu ở Mục 4.5, kết quả Top-5 error dưới white-box I-FGSM cho thấy mixup (95.8) tệ hơn ERM (93.4), đi ngược xu hướng "mixup robust hơn" ở mọi ô khác của Table 3; paper không phân tích riêng trường hợp này, chỉ gộp chung vào câu "cả hai mô hình đều không robust trước white-box I-FGSM" mà không giải thích vì sao đúng ở ô Top-5 lại có chiều ngược lại.

6. Giới hạn

- **Chưa có lý thuyết giải thích "điểm ngọt" của α**: paper tự viết rõ "chúng tôi chưa có lý thuyết tốt để hiểu điểm ngọt của trade-off bias-variance này" (Mục 5), nghĩa là việc chọn α trong thực tế vẫn dựa trên dò thử (grid trong khoảng $[0{,}1, 0{,}4]$ cho ImageNet, cố định $\alpha = 1$ cho CIFAR) hơn là một nguyên tắc suy ra từ lý thuyết.
- **Giả thuyết "dung lượng model lớn hơn hưởng lợi nhiều hơn" chỉ là suy đoán (conjecture)**, không có thí nghiệm kiểm soát biến dung lượng một cách hệ thống (ví dụ quét nhiều kích cỡ model cùng kiến trúc trên cùng dataset để kiểm chứng xu hướng liên tục); bằng chứng hiện có chỉ là so sánh gián tiếp giữa các kiến trúc khác nhau trên các dataset khác nhau.
- **Kết quả trên giọng nói cho thấy mixup có thể làm xấu đi model dung lượng thấp** (LeNet, Mục 4.3): đây là bằng chứng ngược với headline "mixup luôn cải thiện generalization", paper không phân tích sâu nguyên nhân, chỉ nêu số liệu.
- **Ngoại lệ Top-5/white-box I-FGSM** (Mục 5.3): mixup cho kết quả tệ hơn ERM ở chính ô này, không được paper giải thích hay thậm chí nêu riêng, dù nó mâu thuẫn trực tiếp với tinh thần chung của Mục 3.5 ("mixup cải thiện robustness"); đây là điểm người đọc cần tự lưu ý khi trích dẫn kết quả adversarial của paper.
- **Thí nghiệm GAN chỉ định tính, không có metric định lượng**: Mục 3.7 chỉ trình bày hình ảnh mẫu qua các bước train (Figure 5), không có chỉ số định lượng (ví dụ FID hay Inception Score) để so sánh mức độ ổn định hay chất lượng mẫu một cách khách quan; đây là phần thực nghiệm yếu nhất trong bài so với các mục khác đều có bảng số liệu rõ ràng.
- **Không đo lường tác động của mixup lên calibration (ECE)**: dù có liên hệ rõ với label smoothing (bài "ml basics 30", cũng tạo soft target và cũng được paper này so sánh trực tiếp trong ablation), paper không đo expected calibration error cho mô hình train bằng mixup ở bất kỳ thí nghiệm nào; đây là một khoảng trống rõ ràng khi đặt cạnh bài label smoothing.
- **Khác biệt lịch train với bản gốc DenseNet**: paper tự nêu DenseNet trong Huang et al. 2017 được train 300 epoch (decay ở epoch 150, 225) trong khi thí nghiệm ở đây chỉ train 200 epoch (decay ở 100,

150), và paper tự nhận đây "có thể giải thích" sự khác biệt giữa kết quả DenseNet ở Figure 3a và kết quả gốc; đây là một điểm minh bạch tốt của paper (tự giải thích khác biệt thay vì im lặng), nhưng cũng nghĩa là so sánh DenseNet trong bài này với con số gốc của Huang et al. không hoàn toàn công bằng.
- **Mở rộng sang structured prediction/semi-supervised/RL/feature-label extrapolation đều được paper tự xếp là "còn mang tính suy đoán" (still speculative)**: paper thẳng thắn nói việc mở rộng mixup sang bài toán structured prediction như segmentation "còn chưa rõ ràng" (less obvious), và các hướng ngoài supervised learning chỉ là gợi ý chưa kiểm chứng trong bài.

7. Liên hệ với các bài khác

- **distillation 01** (Understanding deep learning requires rethinking generalization, Zhang et al. 2017): là động lực trực tiếp cho vấn đề memorization mà mixup nhắm tới giải quyết (Mục 1, Mục 4.4 dùng chính phương pháp làm nhiễu nhãn của bài đó để đánh giá).
- **ml basics 06** (Dropout): được so sánh và kết hợp trực tiếp trong Mục 4.4 (mixup+dropout cho kết quả tốt nhất trong thí nghiệm memorization, hai phương pháp "tương thích" theo lời paper).
- **ml basics 30** (When Does Label Smoothing Help?): liên hệ gần nhất về mặt ý tưởng ("soft target"), được paper này so sánh trực tiếp trong ablation Table 5 (mixup tốt hơn label smoothing, và tốt hơn cả khi kết hợp mix input + label smoothing); khác biệt cốt lõi mà paper nêu: label smoothing làm mềm nhãn độc lập với giá trị feature cụ thể, còn mixup gắn trực tiếp mức độ làm mềm nhãn với mức độ nội suy feature.
- **ml basics 19** (On Calibration of Modern Neural Networks): liên hệ gián tiếp qua chủ đề làm mượt decision boundary/soft target ảnh hưởng calibration, nhưng như đã nêu ở Mục 6, paper mixup không trực tiếp đo ECE, nên đây là một hướng so sánh còn thiếu dữ liệu.
- **ml basics 32** (Focal Loss): cùng nhóm "sửa đổi loss/mục tiêu huấn luyện để xử lý vấn đề dữ liệu", nhưng theo hướng khác: Focal Loss nhắm tới mất cân bằng lớp bằng cách hạ trọng số các ví dụ dễ, còn mixup nhắm tới memorization/adversarial robustness bằng cách tạo thêm ví dụ ảo nội suy.
- **ml basics 27, 28, 29** (transferability, pretraining, Noisy Student): cùng thuộc nhóm chủ đề lớn hơn về cách khai thác/tăng cường dữ liệu và nhãn (pretraining, pseudo-label, data augmentation) để cải thiện generalization của model ảnh; mixup bổ sung một góc nhìn khác (tổ hợp tuyến tính cặp ví dụ) so với self-training (bài 29) hay transfer learning (bài 27, 28).

8. Gợi ý áp dụng

Mixup tạo ví dụ và nhãn nội suy cho bài toán phân loại. Khi áp dụng, cần kiểm tra giả định về phép trộn có phù hợp với dữ liệu và nhãn; mở rộng sang các đầu ra có cấu trúc đòi hỏi thiết kế riêng.

9. Thí nghiệm gợi ý

Tất cả chạy được:

1. **Tái hiện mixup vs ERM trên CIFAR-10 quy mô nhỏ**: train một CNN nhỏ (ví dụ PreAct ResNet-18 rút gọn hoặc một CNN 4-6 layer) với ERM và với mixup $\alpha = 1$, cùng số epoch (50-100 đủ để thấy xu hướng), đo test error. Kỳ vọng: mixup cho test error thấp hơn, đúng chiều hướng Figure 3a dù quy mô nhỏ hơn.
2. **Quét α và quan sát trade-off training-error/generalization-gap**: trên cùng CNN nhỏ và CIFAR-10 (có thể dùng 20-50% dữ liệu để tiết kiệm thời gian), thử $\alpha \in \{0,0{,}1, 0{,}4, 1, 4, 8\}$, đo cả training accuracy trên nhãn thật và test accuracy. Kỳ vọng: vẽ được xu hướng paper nêu ở Mục 5.2 (train error tăng, generalization gap giảm theo α), xác nhận hoặc bác bỏ quan sát này ở quy mô nhỏ.
3. **Memorization nhãn nhiễu quy mô nhỏ**: lấy một subset CIFAR-10 (hoặc MNIST để nhanh hơn), làm nhiễu 50% nhãn train ngẫu nhiên, so sánh ERM, dropout, và mixup (α lớn, ví dụ 8-32) về (a) train accuracy trên nhãn nhiễu (muốn thấp, tức không "nhớ" nhiễu) và (b) test accuracy trên nhãn thật (muốn

cao). Kỳ vọng theo Table 2: mixup cho train-accuracy-trên-nhiễu thấp hơn ERM (ít memorize hơn) và test accuracy cao hơn cả ERM và dropout.

4. **Trực quan hoá phân phối Beta(α,α)**: không cần GPU, chỉ cần vẽ histogram mẫu $\lambda \sim \text{Beta}(\alpha, \alpha)$ với vài giá trị α (0.1, 1, 4, 32), đối chiếu với công thức phương sai $1/(4(2\alpha+1))$ (tự tính ở Mục 3.2), để hình dung trực quan "mức trộn" thực tế ở mỗi α trước khi chọn α cho thí nghiệm 1-3.
5. **Tái hiện hiệu ứng dung lượng model (như LeNet vs VGG-11 ở Mục 4.3)**: trên cùng một bài toán nhỏ (ví dụ CIFAR-10 subset hoặc MNIST), so sánh một CNN rất nhỏ (dưới 50 nghìn tham số) với một CNN lớn hơn vài lần, cả hai train với ERM và với mixup cùng α; kiểm tra xem mixup có làm model nhỏ **xấu đi** (như LeNet trong paper) trong khi giúp model lớn hơn **tốt lên** không, để tự kiểm chứng giả thuyết "dung lượng lớn hơn hưởng lợi nhiều hơn từ mixup" mà paper chỉ nêu dưới dạng suy đoán.

10. Tóm tắt công thức / khái niệm

Risk và ERM:

$$R(f) = \int \ell(f(x), y)\, dP(x, y), \qquad R_\delta(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(x_i), y_i)$$

VRM tổng quát và trường hợp Gaussian vicinity:

$$P_\nu(\tilde{x}, \tilde{y}) = \frac{1}{n}\sum_i \nu(\tilde{x}, \tilde{y} \mid x_i, y_i), \qquad \nu_{\text{Gauss}} = \mathcal{N}(\tilde{x} - x_i, \sigma^2)\, \delta(\tilde{y} = y_i)$$

Công thức mixup:

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \qquad \tilde{y} = \lambda y_i + (1-\lambda)y_j, \qquad \lambda \sim \text{Beta}(\alpha, \alpha)$$

Trung bình và phương sai của λ (tự tính):

$$\mathbb{E}[\lambda] = 0{,}5, \qquad \text{Var}(\lambda) = \frac{1}{4(2\alpha+1)} \quad \big(\to 0{,}25 \text{ khi } \alpha \to 0;\ \to 0 \text{ khi } \alpha \to \infty\big)$$

mixup cho GAN:

$$\max_g \min_d\ \mathbb{E}_{x,z,\lambda}\big[\ell(d(\lambda x + (1-\lambda)g(z)), \lambda)\big]$$

Bảng số liệu cần nhớ:

Hiện tượng	Số liệu
ResNet-50 ImageNet 90ep, ERM → mixup α=0.2	23,5% → 23,3% top-1
ResNeXt-101 64x4d 90ep, mixup α=0.4	19,8% top-1 (SOTA thời điểm paper)
CIFAR-10 WideResNet-28-10, ERM → mixup α=1	3,8% → 2,7%
Memorization CIFAR-10, 20% nhiễu, test error cuối	ERM 16,6 → mixup(α=8) 6,4
Robustness FGSM white-box top-1, ERM → mixup	tỷ lệ "còn đúng" tăng ≈ 2,7 lần (tự tính)
LeNet (Google commands)	mixup làm **xấu đi** test error (10,3 → 10,8/11,3)

11. Kết luận ngắn

mixup thay ERM bằng một dạng Vicinal Risk Minimization tổng quát, không cần tri thức domain: tổ hợp lồi tuyến tính của cả input và nhãn one-hot của hai ví dụ ngẫu nhiên. Với chi phí cài đặt/tính toán gần như không đáng kể, mixup cải thiện generalization trên nhiều loại dữ liệu (ảnh, giọng nói, bảng), giảm memorization

nhãn nhiễu, tăng robustness trước nhiều dạng tấn công adversarial, và ổn định huấn luyện GAN, đồng thời vượt qua label smoothing và các biến thể nội suy khác trong ablation study trực tiếp. Tuy nhiên paper tự nhận chưa có lý thuyết giải thích "điểm ngọt" của siêu tham số α, quan sát một ngoại lệ chưa giải thích (mixup làm model dung lượng thấp xấu đi trên dữ liệu giọng nói, và một ô kết quả adversarial đi ngược xu hướng chung), và tự xếp việc mở rộng sang bài toán structured prediction hay ngoài supervised learning vẫn còn ở mức suy đoán, chưa kiểm chứng.

Nguồn gốc: https://arxiv.org/abs/1710.09412

Bài 32

Focal Loss for Dense Object Detection (2017)

Tác giả: Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, Piotr Dollár (Facebook AI Research, FAIR).
Venue / arXiv: arXiv:1708.02002 (v2, 07/02/2018); paper không nêu rõ venue hội nghị trong văn bản đọc được (chỉ có nhãn arXiv và tên FAIR). (Ngoài paper, kiến thức chung: công bố tại ICCV 2017, đạt Best Student Paper Award, không nằm trong văn bản PDF đọc ở đây.)
Nguồn: https://arxiv.org/abs/1708.02002
Chủ đề: phát hiện và xử lý nguyên nhân chính khiến object detector một giai đoạn (one-stage) kém chính xác hơn detector hai giai đoạn (two-stage): mất cân bằng cực độ giữa lớp foreground/background khi train trên tập anchor dày đặc; đề xuất Focal Loss, một biến thể động của cross-entropy tự động hạ trọng số các ví dụ dễ, và một detector một giai đoạn đơn giản gọi là RetinaNet để kiểm chứng.
Vị trí trong lộ trình: Đọc cùng bài 33 về mất cân bằng lớp; bài này xét dense detection và thay đổi hàm mất mát.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Detector chính xác nhất tính tới thời điểm paper đều dựa trên cơ chế hai giai đoạn (two-stage), khởi xướng bởi R-CNN: giai đoạn một sinh một tập ứng viên vị trí vật thể thưa (sparse, thường 1.000-2.000 vị trí), giai đoạn hai phân loại từng ứng viên. Ngược lại, detector một giai đoạn (one-stage, ví dụ YOLO, SSD) áp dụng trực tiếp trên một lưới lấy mẫu dày đặc (dense) các vị trí/tỷ lệ/khung hình có thể có, nên nhanh và đơn giản hơn, nhưng độ chính xác luôn thua two-stage khoảng 10-40% tương đối.

Bài báo đặt câu hỏi: vì sao one-stage kém chính xác hơn, và liệu có thể khắc phục? Phát hiện chính: nguyên nhân trung tâm là **mất cân bằng cực độ giữa lớp foreground (có vật thể) và background** khi train trên lưới anchor dày đặc. Một ảnh thường có khoảng 10^4-10^5 vị trí ứng viên (paper ví dụ khoảng 100 nghìn), nhưng chỉ vài vị trí chứa vật thể thật; tỷ lệ mất cân bằng có thể tới khoảng 1:1000.

Two-stage xử lý mất cân bằng này qua hai cơ chế: (1) giai đoạn đề xuất (proposal, ví dụ Selective Search, RPN) lọc tập ứng viên gần như vô hạn xuống 1-2 nghìn, loại bỏ phần lớn background dễ; (2) lấy mẫu minibatch có thiên lệch (biased sampling), ví dụ giữ tỷ lệ cố định foreground:background 1:3, hoặc online hard example mining (OHEM). One-stage phải xử lý toàn bộ lưới dày đặc; áp heuristic lấy mẫu tương tự vẫn kém hiệu quả vì quá trình train vẫn bị chi phối bởi số lượng khổng lồ background dễ phân loại. Đây là vấn đề cổ điển trong object detection, từng được xử lý bằng bootstrapping hoặc hard example mining.

Paper cũng đối chiếu khái niệm với robust loss function (ví dụ Huber loss): robust loss hạ trọng số các **outlier** (ví dụ khó, lỗi lớn) để giảm ảnh hưởng của chúng; Focal Loss làm **ngược lại**: hạ trọng số các **inlier** (ví dụ dễ) để dù số lượng rất lớn, tổng đóng góp của chúng vào loss vẫn nhỏ, từ đó tập trung việc học vào một tập thưa các ví dụ khó.

2. Ý tưởng chính

Paper xác định mất cân bằng lớp, không phải thiết kế kiến trúc, là rào cản chính khiến one-stage detector chưa đạt độ chính xác SOTA. Giải pháp: **Focal Loss**, một cross-entropy được "co giãn động" (dynamically scaled), với hệ số co giãn giảm về 0 khi độ tự tin vào lớp đúng tăng lên. Hệ số này tự động hạ trọng số đóng góp của các ví dụ dễ trong lúc train, nhanh chóng tập trung mô hình vào các ví dụ khó.

Để kiểm chứng, paper thiết kế một detector một giai đoạn đơn giản gọi là **RetinaNet** (tên gợi từ việc lấy mẫu dày đặc các vị trí vật thể trong ảnh đầu vào, giống cách mắt người/retina lấy mẫu). Paper nhấn mạnh rõ: RetinaNet đạt kết quả top không nhờ cải tiến thiết kế mạng, mà nhờ hàm loss mới; kiến trúc mạng kế thừa nhiều ý tưởng có sẵn (Feature Pyramid Network, anchor box kiểu RPN).

3. Phương pháp chi tiết

3.1. Từ cross-entropy tới Focal Loss

Với bài toán phân loại nhị phân, cross-entropy (CE) viết theo $y \in \{\pm 1\}$ và $p \in [0,1]$ (xác suất mô hình ước lượng cho lớp $y = 1$):

$$CE(p,y) = \begin{cases} -\log(p) & \text{nếu } y = 1 \\ -\log(1-p) & \text{nếu khác} \end{cases} \qquad (1)$$

Định nghĩa p_t để viết gọn:

$$p_t = \begin{cases} p & \text{nếu } y = 1 \\ 1-p & \text{nếu khác} \end{cases} \qquad (2)$$

nên $CE(p,y) = CE(p_t) = -\log(p_t)$. Vấn đề cốt lõi: dù ví dụ được phân loại dễ ($p_t \gg 0{,}5$), loss vẫn có độ lớn không hề nhỏ; cộng trên số lượng khổng lồ ví dụ dễ, các giá trị loss nhỏ này có thể lấn át hoàn toàn đóng góp của lớp hiếm.

Balanced CE (baseline thực nghiệm, Mục 3.1 của paper): thêm hệ số trọng số $\alpha \in [0,1]$ cho lớp 1 và $1-\alpha$ cho lớp -1 (có thể đặt theo nghịch đảo tần suất lớp hoặc tune bằng cross-validation); định nghĩa α_t tương tự p_t:

$$CE(p_t) = -\alpha_t \log(p_t) \qquad (3)$$

α-balanced CE cân bằng độ quan trọng giữa ví dụ dương/âm, nhưng **không** phân biệt được ví dụ dễ/khó.

Focal Loss (Mục 3.2): thêm hệ số điều biến (modulating factor) $(1-p_t)^\gamma$ vào CE, với tham số focusing $\gamma \geq 0$:

$$FL(p_t) = -(1-p_t)^\gamma \log(p_t) \qquad (4)$$

Hai tính chất: (1) khi ví dụ bị phân loại sai, p_t nhỏ, hệ số điều biến gần 1, loss gần như không đổi; khi $p_t \to 1$ (phân loại đúng, tự tin), hệ số $\to 0$, loss của ví dụ dễ bị hạ mạnh. (2) γ điều chỉnh mượt tốc độ hạ trọng số; $\gamma = 0$ thì $FL = CE$; γ lớn hơn thì hiệu ứng hạ trọng số mạnh hơn (paper tìm ra $\gamma = 2$ hoạt động tốt nhất).

Ví dụ số minh hoạ ngay trong paper: với $\gamma = 2$, một ví dụ có $p_t = 0{,}9$ sẽ có loss thấp hơn CE khoảng 100 lần; với $p_t \approx 0{,}968$, thấp hơn khoảng 1000 lần. Ngược lại, loss của ví dụ bị phân loại sai (ứng với $p_t \leq 0{,}5$) chỉ bị hạ tối đa 4 lần khi $\gamma = 2$.

(Tự tính, kiểm chứng và bổ sung các con số trên bằng công thức tỷ lệ $FL/CE = (1-p_t)^\gamma$: tại $p_t = 0{,}9, \gamma = 2$: $(1-0{,}9)^2 = 0{,}01$, tức thấp hơn $1/0{,}01 = 100$ lần, khớp. Tại $p_t \approx 0{,}968$: $(1-0{,}968)^2 = (0{,}032)^2 \approx 0{,}001$, thấp hơn khoảng 1000 lần, khớp. Tại $p_t = 0{,}5$: $(0{,}5)^2 = 0{,}25$, thấp hơn 4 lần, khớp với giới hạn "tối đa 4 lần cho $p_t \leq 0{,}5$". Thêm một điểm paper không nêu: tại $p_t = 0{,}7, \gamma = 2$: $(1-0{,}7)^2 = 0{,}09$, tức thấp hơn khoảng $1/0{,}09 \approx 11{,}1$ lần.)

Dạng dùng trong thực nghiệm chính của paper là **α-balanced Focal Loss**, cho accuracy nhích hơn một chút so với bản không α:

$$FL(p_t) = -\alpha_t (1-p_t)^\gamma \log(p_t) \qquad (5)$$

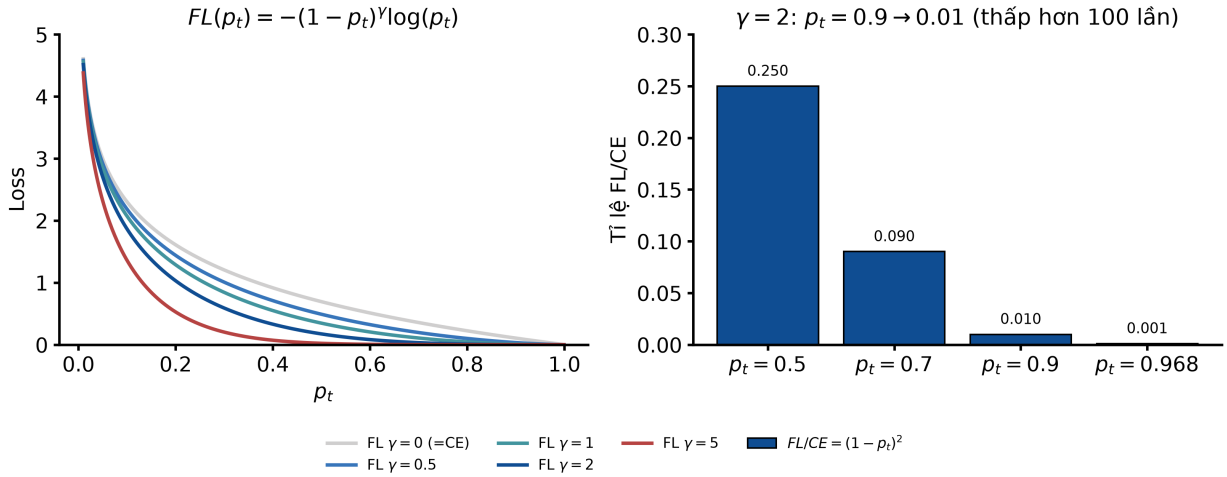
Paper cũng ghi chú: cài đặt kết hợp phép sigmoid tính p với phép tính loss trong cùng một layer để tăng ổn định số học, và dạng cụ thể của Focal Loss không quan trọng tuyệt đối (một biến thể thay thế FL^* được trình bày ở Appendix A, cho kết quả tương đương).

3.2. Mất cân bằng lớp và khởi tạo mô hình (Mục 3.3)

Mô hình phân loại nhị phân thường khởi tạo để xác suất ra $y = -1$ hay 1 là bằng nhau. Khi dữ liệu mất cân bằng mạnh, loss từ lớp phổ biến (background) có thể lấn át tổng loss ngay từ đầu, gây bất ổn huấn luyện. Paper đưa khái niệm "prior" π cho xác suất ước lượng p của lớp hiếm (foreground) lúc bắt đầu train, ví dụ $\pi = 0{,}01$, bằng cách khởi tạo bias của layer cuối cùng (không phải thay đổi hàm loss):

$$b = -\log\left(\frac{1-\pi}{\pi}\right)$$

(Tự tính với $\pi = 0{,}01$: $b = -\log(0{,}99/0{,}01) = -\log(99) \approx -4{,}60$.) Paper nhận thấy cách khởi tạo này cải thiện ổn định train cho cả CE và Focal Loss trong trường hợp mất cân bằng nặng.



$FL = (1-p_t)^{\gamma}(-\log p_t)$; $\gamma = 2$, $p_t = 0.9$ thì $FL/CE = 0.01$ (minh hoạ, tự tính)

3.3. Mất cân bằng lớp ở two-stage detector (Mục 3.4)

Two-stage detector thường train bằng CE thường, không cần α-balancing hay Focal Loss, nhờ hai cơ chế: (1) cascade hai giai đoạn (proposal lọc gần-vô-hạn ứng viên xuống 1-2 nghìn, các ứng viên được giữ lại nhiều khả năng đúng là vị trí vật thể thật, loại bỏ hầu hết background dễ); (2) lấy mẫu minibatch thiên lệch ở giai đoạn hai (ví dụ tỷ lệ 1:3 dương:âm), có vai trò như một hệ số α-balancing ngầm thực hiện qua sampling. Focal Loss nhắm xử lý trực tiếp hai cơ chế này bằng hàm loss, cho hệ thống one-stage.

3.4. Kiến trúc RetinaNet

RetinaNet = backbone (trích đặc trưng) + hai subnetwork chuyên biệt (phân loại, hồi quy box):

- **Backbone FPN:** Feature Pyramid Network trên ResNet, tạo các mức pyramid P_3 đến P_7 (độ phân giải P_l thấp hơn input 2^l lần), mỗi mức $C = 256$ kênh. P_3-P_5 tính từ các stage C_3-C_5 của ResNet qua kết nối top-down/lateral (như FPN gốc); P_6 bằng conv 3×3 stride 2 trên C_5; P_7 bằng ReLU rồi conv 3×3 stride 2 trên P_6 (khác FPN gốc ở 3 điểm: không dùng mức P_2 độ phân giải cao vì lý do tính toán, P_6 dùng strided conv thay vì downsample, và thêm P_7 để cải thiện phát hiện vật lớn).
- **Anchor:** diện tích từ 32^2 đến 512^2 trên P_3-P_7, 3 tỷ lệ khung hình $\{1{:}2, 1{:}1, 2{:}1\}$ mỗi mức, cộng thêm 3 scale $\{2^0, 2^{1/3}, 2^{2/3}\}$ cho bao phủ dày hơn (tổng $A = 9$ anchor/mức), bao phủ scale 32-813 pixel so với ảnh input. Mỗi anchor gán một vector nhãn phân loại one-hot độ dài K (K = số lớp vật thể) và một vector 4 chiều hồi quy box, theo quy tắc gán của RPN chỉnh lại cho multi-class: IoU $\geq 0{,}5$ với ground-truth thì gán foreground; IoU trong $[0, 0{,}4)$ thì gán background; IoU trong $[0{,}4, 0{,}5)$ thì bỏ qua (không tính loss) khi train.
- **Classification subnet:** FCN nhỏ gắn vào mỗi mức FPN, tham số chia sẻ giữa các mức; 4 lớp conv 3×3

(C filter, ReLU) rồi 1 lớp conv 3×3 ra KA filter, sigmoid cho KA dự đoán nhị phân mỗi vị trí. Khác RPN: sâu hơn, chỉ dùng conv 3×3, không chia sẻ tham số với box subnet.

- **Box regression subnet:** thiết kế giống classification subnet nhưng kết thúc bằng $4A$ output tuyến tính mỗi vị trí (tham số hoá box theo kiểu R-CNN); dùng bộ hồi quy **không phân biệt lớp** (class-agnostic), ít tham số hơn nhưng paper thấy hiệu quả tương đương cách phổ biến hơn là hồi quy riêng theo từng lớp.
- **Suy luận (inference):** forward một lần qua mạng; chỉ decode tối đa 1.000 dự đoán có điểm cao nhất mỗi mức FPN sau khi lọc theo ngưỡng confidence 0.05; hợp nhất dự đoán từ mọi mức, áp Non-Maximum Suppression (NMS) ngưỡng 0.5 để ra kết quả cuối.
- **Áp dụng Focal Loss:** dùng cho output của classification subnet; $\gamma = 2$ hoạt động tốt, RetinaNet khá bền (robust) với $\gamma \in [0{,}5, 5]$. Điểm quan trọng: Focal Loss áp dụng cho **toàn bộ** khoảng 100 nghìn anchor mỗi ảnh (khác với thực hành phổ biến chỉ chọn một tập nhỏ anchor, ví dụ 256, qua sampling heuristic của RPN hoặc hard example mining của OHEM/SSD). Tổng Focal Loss một ảnh = tổng trên toàn bộ $\sim$ 100 nghìn anchor, **chuẩn hoá theo số anchor được gán cho một ground-truth box** (không chuẩn hoá theo tổng số anchor), vì phần lớn anchor là background dễ, nhận loss gần như bằng 0 dưới Focal Loss. α cũng có khoảng ổn định nhưng tương tác với γ (cần chọn cùng nhau): α nên giảm nhẹ khi γ tăng (với $\gamma = 2$, $\alpha = 0{,}25$ tốt nhất).

3.5. Khởi tạo và tối ưu hoá

Backbone ResNet-50/ResNet-101 pretrain trên ImageNet1k. Layer conv mới thêm (trừ layer cuối các subnet) khởi tạo bias $b = 0$, trọng số Gaussian $\sigma = 0{,}01$; layer conv cuối của classification subnet khởi tạo bias theo công thức π ở Mục 3.2, $\pi = 0{,}01$. SGD đồng bộ trên 8 GPU, 16 ảnh/minibatch (2/GPU); 90 nghìn iteration, learning rate ban đầu 0.01, chia 10 tại iteration 60k và 80k; chỉ dùng lật ngang (horizontal flip) làm augmentation trừ khi nêu khác; weight decay 0.0001, momentum 0.9; training loss = Focal Loss (phân loại) + smooth L1 loss chuẩn (hồi quy box); thời gian train 10-35 giờ cho các model ở Table 1e.

4. Thí nghiệm và kết quả chính

Benchmark: COCO bounding-box detection. Train trên trainval35k (80k ảnh train + 35k ảnh ngẫu nhiên từ tập val 40k); ablation/sensitivity đánh giá trên minival (5k ảnh val còn lại); kết quả chính báo cáo trên test-dev (không có nhãn công khai, cần evaluation server).

4.1. Huấn luyện dense detector (Mục 5.1)

- **CE thường không kèm thay đổi khởi tạo/learning**: train thất bại nhanh, mô hình phân kỳ (diverge).
- Chỉ cần khởi tạo bias layer cuối theo prior $\pi = 0{,}01$ (không đổi gì ở hàm loss) đã giúp train ổn định: ResNet-50 + khởi tạo này đạt ngay AP 30.2 trên COCO (CE không α-balance).
- **Balanced CE** (Table 1a, $\gamma = 0$): $\alpha = 0{,}10$: AP 0.0 (hỏng hoàn toàn); $\alpha = 0{,}25$: 10.8; $\alpha = 0{,}50$: 30.2 (khớp con số ở trên); $\alpha = 0{,}75$: 31.1 (tốt nhất); $\alpha = 0{,}90$: 30.8; $\alpha = 0{,}99$: 28.7; $\alpha = 0{,}999$: 25.1. Mức tăng từ $\alpha = 0{,}75$ so với $\alpha = 0{,}50$ là $31{,}1 - 30{,}2 = 0{,}9$ điểm AP (tự tính, khớp câu paper "gain of 0.9 points AP").
- **Focal Loss** (Table 1b, α tốt nhất cho mỗi γ): $\gamma = 0, \alpha = .75$: 31.1 (đúng bằng CE-balanced tốt nhất, hợp lý vì $\gamma = 0 \Rightarrow FL = CE$); $\gamma = 0{,}1$: 31.4; $\gamma = 0{,}2$: 31.9; $\gamma = 0{,}5, \alpha = .50$: 32.9; $\gamma = 1{,}0, \alpha = .25$: 33.7; $\gamma = 2{,}0, \alpha = .25$: 34.0 (tốt nhất); $\gamma = 5{,}0, \alpha = .25$: 32.2 (giảm lại). Mức tăng của $\gamma = 2$ so với CE-balanced tốt nhất là $34{,}0 - 31{,}1 = 2{,}9$ điểm AP (tự tính, khớp "2.9 AP gain"). Paper dùng mặc định $\gamma = 2{,}0, \alpha = 0{,}25$ cho mọi thực nghiệm khác; $\alpha = 0{,}5$ cho kết quả gần tương đương (thấp hơn 0.4 AP, tức khoảng 33.6 AP, tự tính).
- **Phân tích phân phối loss** (Figure 4): lấy model ResNet-101, scale train/test 600px, $\gamma = 2$ (36.0 AP), áp lên nhiều ảnh ngẫu nhiên, lấy mẫu $\sim 10^7$ cửa sổ âm (negative) và $\sim 10^5$ cửa sổ dương (positive), tính Focal Loss, chuẩn hoá tổng về 1, vẽ hàm phân phối tích luỹ (CDF) theo nhiều giá trị γ (dù model chỉ train ở $\gamma = 2$). Với ví dụ dương: CDF khá giống nhau giữa các γ, hiệu ứng nhỏ (khoảng 20% ví

dụ dương khó nhất chiếm khoảng nửa tổng loss dương, gần như không đổi theo γ). Với ví dụ âm: hiệu ứng rất khác biệt; γ tăng thì loss tập trung mạnh vào một phần rất nhỏ các ví dụ âm khó; ở $\gamma = 2$ (mặc định), phần lớn tổng loss đến từ một phần nhỏ mẫu.

- **So sánh OHEM** (Table 1d, ResNet-101): OHEM tốt nhất (batch 128, nms 0.5, không ép tỷ lệ 1:3): AP 32.8. Focal Loss cùng thiết lập: AP 36.0. Khoảng cách $36{,}0 - 32{,}8 = 3{,}2$ điểm AP (tự tính, khớp "gap of 3.2 AP"). Biến thể "OHEM 1:3" (ép tỷ lệ dương:âm) cho kết quả còn thấp hơn ở mọi batch size thử.
- **Hinge loss**: thử train với hinge loss trên p_t (loss về 0 trên một ngưỡng p_t), nhưng không ổn định, không đạt kết quả có ý nghĩa.

4.2. Thiết kế kiến trúc (Mục 5.2)

- **Mật độ anchor** (Table 1c, ResNet-50): 1 anchor vuông/vị trí đã đạt AP 30.3 (khá cao bất ngờ). Dùng 3 scale $\times$ 3 tỷ lệ (9 anchor/vị trí, mặc định của paper): AP 34.0, tăng gần 4 điểm. Tăng thêm (4 scale $\times$ 3 tỷ lệ, 12 anchor/vị trí): AP 33.8, không tăng thêm (bão hoà). (Quan sát của người ghi chú: dòng "2 scale $\times$ 3 tỷ lệ" trong cùng Table 1c cho AP 34.2, cao hơn 0.2 điểm so với chính thiết lập 3 scale $\times$ 3 tỷ lệ mà paper chọn làm chuẩn cho mọi thực nghiệm khác; paper không giải thích lựa chọn này, chênh lệch 0.2 AP có thể nằm trong nhiễu đo nhưng đáng ghi chú.)
- **Tốc độ/độ chính xác** (Table 1e, Figure 2): backbone và scale ảnh lớn hơn cho AP cao hơn nhưng chậm hơn. RetinaNet-101 scale 600px khớp độ chính xác của Faster R-CNN FPN đã công bố, nhưng chạy 122ms/ảnh so với 172ms (đo trên GPU Nvidia M40) của Faster R-CNN FPN. Scale lớn hơn giúp RetinaNet vượt mọi phương pháp two-stage về AP mà vẫn nhanh hơn. Paper tự ghi chú: sau khi công bố, một biến thể khác của Faster R-CNN đã đạt kết quả nhanh và chính xác hơn, nằm ngoài phạm vi bài này.
- **So sánh SOTA** (Table 2, test-dev): RetinaNet-101-800 (train với scale jitter, lâu hơn 1.5 lần so với Table 1e, cho thêm 1.3 AP) đạt 39.1 AP (ResNet-101-FPN), vượt one-stage tốt nhất (DSSD513, 33.2 AP) $39{,}1 - 33{,}2 = 5{,}9$ điểm (tự tính), vượt two-stage tốt nhất trong bảng (Faster R-CNN w/ TDM, 36.8 AP) $39{,}1 - 36{,}8 = 2{,}3$ điểm (tự tính). Đổi backbone sang ResNeXt-101-FPN: 40.8 AP, thêm $40{,}8 - 39{,}1 = 1{,}7$ điểm (tự tính), vượt mốc 40 AP.

5. Phân tích / ablation

- γ **quan trọng hơn** α: dù quét $\alpha \in [0{,}01, 0{,}999]$, α tốt nhất tìm được chỉ nằm trong $[0{,}25, 0{,}75]$ ở mọi γ; lợi ích từ việc tăng γ lớn hơn nhiều so với lợi ích tinh chỉnh α. α tối ưu giảm khi γ tăng, vì γ đã tự hạ trọng số ví dụ âm dễ, giảm nhu cầu tăng trọng số ví dụ dương qua α.
- α **cực trị làm hỏng hoàn toàn việc train** ($\alpha = 0{,}10 \Rightarrow$ AP 0.0): cho thấy dù α-balancing là ý tưởng hợp lý, chọn sai cực trị có thể làm việc train thất bại tuyệt đối, không chỉ kém hơn một chút.
- **Cơ chế Focal Loss nhìn từ góc CDF**: hiệu ứng hạ trọng số gần như không ảnh hưởng ví dụ dương (vốn đã ít), nhưng định hình lại mạnh mẽ phân phối loss của ví dụ âm, dồn hầu hết "trọng lượng học" vào một phần rất nhỏ ví dụ âm khó; đây là bằng chứng định lượng (không chỉ suy luận từ công thức) cho cơ chế "focus vào ví dụ khó" mà paper đề xuất.
- **FL* (Appendix A), một dạng thay thế**: định nghĩa $x_t = yx$ (6) (y là nhãn ± 1, x là logit), $p_t = \sigma(x_t)$ (tương thích định nghĩa p_t ở (2)); dạng thay thế $p_t^* = \sigma(\gamma x_t + \beta)$ (7), $FL^* = -\log(p_t^*)/\gamma$ (8), với 2 tham số γ, β kiểm soát độ dốc/độ lệch của đường loss. Train RetinaNet-50-600 với FL^* thay FL, cùng mọi thiết lập khác: CE 31.1/49.4/33.0 (AP/AP50/AP75); $FL(\gamma = 2)$ 34.0/52.5/36.5; $FL^*(\gamma = 2, \beta = 1)$ 33.8/52.7/36.3; $FL^*(\gamma = 4, \beta = 0)$ 33.9/51.8/36.4. Kết quả gần tương đương FL gốc, xác nhận "dạng chính xác của Focal Loss không quan trọng", miễn giữ đúng tính chất hạ trọng số ví dụ dễ ($x_t > 0$). Quét rộng γ, β (Figure 7, $\alpha = 0{,}25$ cố định) cho thấy các thiết lập hiệu quả (AP $> 33{,}5$) đều là các thiết lập hạ trọng số ví dụ dễ.
- **Phân tích đạo hàm** (Appendix B): $\frac{dCE}{dx} = y(p_t - 1)$ (9); $\frac{dFL}{dx} = y(1 - p_t)^\gamma(\gamma p_t \log(p_t) + p_t - 1)$ (10); $\frac{dFL^*}{dx} = y(p_t^* - 1)$ (11). Với mọi hàm loss, đạo hàm tiến về -1 hoặc 0 ở vùng dự đoán tự tin cao;

nhưng khác CE, với thiết lập FL/FL^* hiệu quả, đạo hàm đã nhỏ ngay khi $x_t > 0$ (vừa qua ngưỡng phân loại đúng), cho thấy hiệu ứng "focus vào ví dụ khó" thể hiện cả ở mức gradient, không chỉ ở giá trị loss.

6. Giới hạn

- **Lưới quét γ còn thô**: γ chỉ thử tại $\{0,0{,}1, 0{,}2, 0{,}5, 1, 2, 5\}$, không có điểm nào giữa 2 và 5; AP đã giảm rõ tại $\gamma = 5$ (32.2, so với đỉnh 34.0 tại $\gamma = 2$), nên khẳng định "$\gamma = 2$ là tốt nhất" dựa trên lưới thô, chưa phải tìm kiếm liên tục/triệt để.
- **Chênh lệch nhỏ chưa giải thích trong Table 1c**: như nêu ở Mục 5.2, thiết lập "2 scale $\times$ 3 tỷ lệ" (34.2 AP) nhích cao hơn thiết lập chuẩn "3 scale $\times$ 3 tỷ lệ" (34.0 AP) mà paper chọn dùng cho mọi thực nghiệm khác, không có lời giải thích rõ cho lựa chọn này trong văn bản.
- **Thất bại với hinge loss không được phân tích sâu**: paper chỉ ghi "không ổn định, không đạt kết quả có ý nghĩa", không có số liệu hay phân tích nguyên nhân cụ thể, khác với mức độ chi tiết cho các thí nghiệm khác.
- **Khẳng định tổng quát ở Appendix A** ("bất kỳ hàm loss nào có tính chất tương tự FL hoặc FL* cũng sẽ hiệu quả tương đương") là một suy rộng (generalization) chỉ dựa trên đúng 2 dạng loss đã thử (FL và FL*), chưa có thêm instantiation thứ ba để kiểm chứng độc lập trong bài.
- **Tốc độ đo trên một GPU cụ thể (Nvidia M40) tại một thời điểm**: paper tự ghi "sau khi công bố, một biến thể của Faster R-CNN đã nhanh và chính xác hơn", nghĩa là chính tác giả cũng xác nhận bảng so sánh tốc độ/độ chính xác (Table 1e, Figure 2) đã lạc hậu một phần ngay ở các bản chỉnh sửa sau của chính bài báo.
- **Phân tích chính thức chỉ ở bài toán nhị phân**: paper tự ghi ở chú thích "mở rộng Focal Loss sang multi-class là đơn giản và hoạt động tốt, nhưng để đơn giản chúng tôi chỉ tập trung vào loss nhị phân trong bài này"; RetinaNet thực tế dùng K đầu ra sigmoid độc lập (nhị phân hoá theo từng lớp) để lách qua softmax multi-class, nhưng không có ablation riêng cho một dạng Focal Loss multi-class thực sự (softmax) trong bài.
- **Phạm vi tốc độ rất cao (very high frame rate) được paper tự nhận là ngoài phạm vi**: "giải quyết vùng frame rate cao có thể cần thiết kế mạng chuyên biệt...và nằm ngoài phạm vi công trình này".
- **Kiến trúc không phải đóng góp chính, kế thừa gần như nguyên vẹn từ công trình trước** (FPN, anchor kiểu RPN): điều này được paper nói rõ là chủ đích (để tách biệt rõ hiệu ứng của loss khỏi hiệu ứng kiến trúc), nhưng cũng nghĩa là các lựa chọn kiến trúc cụ thể (số lớp conv, kênh, v.v.) không được paper này tối ưu/kiểm chứng độc lập, chỉ mượn từ nơi khác.

7. Liên hệ với các bài khác

- **ml basics 28** (Rethinking ImageNet Pre-training, Kaiming He, Ross Girshick, Piotr Dollár): cùng nhóm tác giả (3/5 tác giả trùng nhau), cùng dùng framework đánh giá COCO AP và backbone ResNet/ResNeXt-FPN; bài 28 đặt câu hỏi "pretraining ImageNet có cần cho detection không", còn bài này tập trung vào "loss function nào xử lý tốt mất cân bằng lớp trong detection", hai câu hỏi bổ sung cho nhau trên cùng bài toán COCO detection.
- **ml basics 07** (ResNet): backbone ResNet-50/ResNet-101 (và biến thể ResNeXt) được dùng trực tiếp làm backbone cho RetinaNet trong mọi thí nghiệm.
- **ml basics 30** (When Does Label Smoothing Help?): cùng là kỹ thuật sửa đổi cách cross-entropy được tính để cải thiện hành vi huấn luyện, nhưng theo hướng đối lập: label smoothing làm *mềm nhãn mục tiêu* bất kể độ khó của ví dụ, còn Focal Loss giữ nguyên nhãn cứng nhưng *hạ trọng số loss* theo độ tự tin dự đoán của chính ví dụ đó.
- **ml basics 31** (mixup): cùng thuộc nhóm "sửa đổi loss/dữ liệu train để xử lý một bệnh lý cụ thể của dữ liệu" (mixup: memorization nhãn nhiễu, adversarial robustness; Focal Loss: mất cân bằng lớp cực độ); cả hai đều được paper gốc trình bày kèm ablation study chi tiết so sánh với các baseline cùng mục tiêu.

- **ml basics 06** (Dropout): không liên hệ trực tiếp về nội dung kỹ thuật, nhưng cùng là một trong các "trick" huấn luyện chuẩn mực được dùng rộng rãi trong các pipeline detection/classification hiện đại; paper Focal Loss không dùng dropout trong RetinaNet, phụ thuộc hoàn toàn vào Focal Loss + weight decay để kiểm soát overfitting.

8. Gợi ý áp dụng

Focal loss điều chỉnh đóng góp của các ví dụ dễ và khó, đặc biệt hữu ích để khảo sát mất cân bằng trong dense detection. Siêu tham số và kết quả cần được đánh giá cùng các baseline lấy mẫu hoặc đánh trọng số.

9. Thí nghiệm gợi ý

Tất cả chạy được:

1. **Toy classification mất cân bằng**: tạo dữ liệu tổng hợp 2 lớp với tỷ lệ mất cân bằng 1:100 hoặc 1:1000 (ví dụ 2 cụm Gaussian), train một MLP/CNN nhỏ với (a) CE thường, (b) α-balanced CE với vài α, (c) Focal Loss với $\gamma \in \{0, 0{,}5, 1, 2, 5\}$ (giữ α tốt nhất tương ứng). Đo precision/recall hoặc AP trên lớp hiếm, tái hiện xu hướng Table 1a/1b ở quy mô nhỏ.
2. **Kiểm chứng hiệu ứng khởi tạo prior** π: trên bài toán phân loại mất cân bằng của thí nghiệm 1, train hai lần: (a) khởi tạo bias mặc định (0, tương ứng xác suất 0.5/0.5), (b) khởi tạo bias theo công thức $b = -\log((1-\pi)/\pi)$ với π bằng đúng tỷ lệ lớp hiếm thật (dùng lại ví dụ tự tính $\pi = 0{,}01 \Rightarrow b \approx -4{,}60$ ở Mục 3.2). So sánh đường loss vài trăm bước đầu, kiểm tra xem (a) có dấu hiệu bất ổn/phân kỳ mà (b) không có, mô phỏng lại quan sát "CE không khởi tạo prior thì phân kỳ, có thì ổn định ngay cả chưa dùng Focal Loss".
3. **Tái hiện phân tích CDF loss (Figure 4)**: trên bài toán của thí nghiệm 1, sau khi train với $\gamma = 2$, lấy mẫu một lượng lớn ví dụ dương/âm, tính Focal Loss từng ví dụ, chuẩn hoá tổng về 1, vẽ CDF riêng cho dương/âm. Kiểm tra xem phần lớn loss có dồn vào một tỷ lệ nhỏ ví dụ âm khó, giống phát hiện của paper ở quy mô COCO, hay không.
4. **Mini dense detection**: dán một vài chữ số MNIST rải rác lên một canvas trắng lớn hơn, chia canvas thành lưới patch nhỏ, gán nhãn "có chữ số" / "background" cho mỗi patch (đa số patch sẽ là background). Train một classifier dày đặc nhỏ với CE, α-CE, và Focal Loss ($\gamma = 2, \alpha = 0{,}25$), đo AP/F1 trên lớp "có chữ số", mô phỏng trực tiếp bài toán anchor foreground/background của RetinaNet ở quy mô rất nhỏ.
5. **Kiểm tra** FL^* **so với** FL: trên bài toán của thí nghiệm 1 hoặc 4, cài cả Focal Loss gốc (công thức 4/5) và FL^* (công thức 7/8) với các cặp (γ, β) tương tự Table 3 của paper, so sánh kết quả cuối, kiểm chứng tuyên bố "dạng chính xác của Focal Loss không quan trọng, miễn giữ đúng tính chất hạ trọng số ví dụ dễ".

10. Tóm tắt công thức / khái niệm

Cross-entropy nhị phân và p_t:

$$CE(p, y) = \begin{cases} -\log(p) & y = 1 \\ -\log(1-p) & \text{khác} \end{cases}, \qquad p_t = \begin{cases} p & y = 1 \\ 1-p & \text{khác} \end{cases}$$

Balanced CE và Focal Loss:

$$CE(p_t) = -\alpha_t \log(p_t), \qquad FL(p_t) = -(1-p_t)^\gamma \log(p_t), \qquad FL_\alpha(p_t) = -\alpha_t (1-p_t)^\gamma \log(p_t)$$

Khởi tạo bias theo prior (tự tính minh hoạ $\pi = 0{,}01$):

$$b = -\log\left(\frac{1-\pi}{\pi}\right) \approx -4{,}60$$

Tỷ lệ FL/CE theo p_t tại $\gamma = 2$ (một phần paper nêu, một phần tự tính):

p_t	$(1-p_t)^2$	Loss thấp hơn CE khoảng
0.5	0.25	4 lần
0.7	0.09	11.1 lần (tự tính)
0.9	0.01	100 lần
0.968	0.001	1000 lần

FL^* thay thế (Appendix A):

$$x_t = yx, \quad p_t^* = \sigma(\gamma x_t + \beta), \quad FL^* = -\log(p_t^*)/\gamma$$

Bảng số liệu chính cần nhớ:

Mốc	Giá trị
CE không khởi tạo prior	phân kỳ, không train được
ResNet-50 + khởi tạo prior, CE không balance	AP 30.2
Balanced CE tốt nhất ($\alpha = 0{,}75$)	AP 31.1
Focal Loss tốt nhất ($\gamma = 2, \alpha = 0{,}25$)	AP 34.0 (hơn balanced CE 2.9 AP)
Focal Loss vs OHEM tốt nhất	36.0 vs 32.8 AP (hơn 3.2 AP)
RetinaNet-101-800 trên test-dev	39.1 AP (hơn one-stage tốt nhất 5.9 AP, hơn two-stage tốt nhất trong bảng 2.3 AP)
RetinaNeXt-101-FPN	40.8 AP

11. Kết luận ngắn

Bài báo chỉ ra nguyên nhân chính khiến detector một giai đoạn kém chính xác hơn hai giai đoạn không nằm ở kiến trúc mà ở mất cân bằng lớp cực độ khi train trên lưới anchor dày đặc, và giải quyết bằng Focal Loss, một cross-entropy co giãn động hạ trọng số ví dụ dễ theo hệ số $(1-p_t)^\gamma$. Với một detector đơn giản (RetinaNet) không có cải tiến kiến trúc gì đặc biệt ngoài việc kế thừa FPN và anchor kiểu RPN, chỉ nhờ đổi loss và một cách khởi tạo bias hợp lý, paper đạt độ chính xác vượt cả detector hai giai đoạn SOTA trong khi vẫn giữ tốc độ nhanh của one-stage. Các ablation (phân tích CDF loss, so sánh OHEM, biến thể FL^*) cho thấy cơ chế hoạt động khá rõ ràng và không quá nhạy với dạng công thức chính xác, nhưng một số lựa chọn (mật độ anchor chuẩn, lưới quét γ) chỉ được bài báo xác nhận ở mức thô, và việc mở rộng Focal Loss sang multi-class hay sang các bài toán ngoài detection (như sinh chuỗi) chưa được bài báo này kiểm chứng trực tiếp.

Nguồn gốc: https://arxiv.org/abs/1708.02002

Bài 33

A systematic study of the class imbalance problem in convolutional neural networks (2017)

Tác giả: Mateusz Buda (Duke University, KTH), Atsuto Maki (KTH), Maciej A. Mazurowski (Duke University).
Venue / arXiv: arXiv:1710.05381 (v2, 13/10/2018); dòng chú dẫn ở đầu bản PDF ghi rõ venue xuất bản: "Neural Networks, 106:249-259, 2018" (Elsevier).
Nguồn: https://arxiv.org/abs/1710.05381
Chủ đề: khảo sát thực nghiệm có hệ thống (trên MNIST, CIFAR-10, ImageNet) về ảnh hưởng của mất cân bằng lớp (class imbalance) lên hiệu năng phân loại của CNN, và so sánh các phương pháp xử lý: oversampling, undersampling, two-phase training, thresholding.
Vị trí trong lộ trình: Đọc cùng bài 32 về focal loss để đối chiếu xử lý mất cân bằng ở mức dữ liệu và hàm mất mát.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Class imbalance là tình huống một số lớp (class) trong tập dữ liệu huấn luyện có số lượng ví dụ (examples) nhiều hơn hẳn các lớp khác. Ví dụ paper nêu: trong chẩn đoán y khoa, tần suất một lớp (ung thư) có thể ít hơn lớp khác (bệnh nhân khoẻ) tới 1000 lần. Vấn đề này đã được nghiên cứu kỹ với các mô hình machine learning cổ điển (cây quyết định, SVM, mạng neural nhiều lớp - MLP), nhưng tới thời điểm bài báo này (2017) **chưa có nghiên cứu hệ thống nào cho CNN** (convolutional neural network), dù CNN đang là kiến trúc chủ đạo cho bài toán ảnh. Các kỹ thuật dùng trong thực tế với CNN khi gặp dữ liệu mất cân bằng chủ yếu dựa trên trực giác hoặc kinh nghiệm từ machine learning cổ điển, chưa được kiểm chứng có còn đúng với CNN hay không.

Hai khái niệm cần nắm trước khi đọc:

- **Oversampling / Undersampling**: hai kỹ thuật ở mức dữ liệu (data level). Oversampling nhân bản (lặp lại) ngẫu nhiên các ví dụ từ lớp thiểu số (minority class) để tăng số lượng; undersampling loại bỏ ngẫu nhiên một phần ví dụ từ lớp đa số (majority class) để giảm số lượng, mục tiêu cả hai là làm các lớp có số lượng bằng nhau.
- **Thresholding (threshold moving / post scaling)**: kỹ thuật ở mức bộ phân loại (classifier level), áp dụng lúc kiểm thử (test time), điều chỉnh lại ngưỡng/xác suất đầu ra của mạng để bù lại việc mạng đã học lệch theo tần suất lớp trong tập huấn luyện.

Câu hỏi nghiên cứu của bài báo: (1) mất cân bằng lớp ảnh hưởng thế nào đến hiệu năng phân loại của CNN theo các mức độ mất cân bằng khác nhau? (2) phương pháp nào (trong các phương pháp phổ biến) xử lý tốt nhất vấn đề này cho CNN, và có khác gì so với kết luận đã biết ở machine learning cổ điển không?

2. Ý tưởng chính

Bài báo không đề xuất phương pháp mới, mà làm một **nghiên cứu thực nghiệm có hệ thống**: tạo ra các phiên bản mất cân bằng nhân tạo (artificially imbalanced) của ba bộ dữ liệu benchmark có độ phức tạp tăng dần (MNIST, CIFAR-10, ImageNet/ILSVRC-2012), huấn luyện CNN tương ứng (LeNet-5, All-CNN, ResNet-10) trên hàng nghìn cấu hình mất cân bằng khác nhau, rồi so sánh 7 cách xử lý mất cân bằng (oversam-

pling, undersampling, hai biến thể two-phase training, thresholding, và kết hợp thresholding với oversampling/undersampling).

Kết luận cốt lõi (đã được bài báo kiểm chứng qua rất nhiều cấu hình): **oversampling là phương pháp vượt trội trong gần như mọi tình huống**, nên được áp dụng **tới mức xoá hoàn toàn mất cân bằng** (không phải một mức trung gian); và **oversampling không gây overfitting trên CNN**, trái với hiểu biết phổ biến từ machine learning cổ điển rằng lặp lại dữ liệu thiểu số sẽ làm mô hình học vẹt (overfit) lớp đó. Đây là phát hiện quan trọng nhất của bài: một kết luận "đã biết" ở lĩnh vực cổ điển **không chuyển thẳng sang CNN**.

3. Phương pháp chi tiết

3.1 Định nghĩa hai dạng mất cân bằng

Paper định nghĩa hai dạng mất cân bằng tham số hoá được, dùng xuyên suốt thí nghiệm:

- **Step imbalance** (mất cân bằng dạng bậc): các lớp thiểu số có số lượng bằng nhau, các lớp đa số có số lượng bằng nhau, nhưng hai nhóm khác nhau. Tham số hoá bằng hai đại lượng:

$$\mu = \frac{|\{i \in \{1, \ldots, N\} : C_i \text{ là thiểu số}\}|}{N} \tag{1}$$

$$\rho = \frac{\max_i |C_i|}{\min_i |C_i|} \tag{2}$$

trong đó C_i là tập ví dụ của lớp i, N là tổng số lớp. μ là tỉ lệ phần lớp bị coi là thiểu số, ρ là tỉ số số lượng giữa lớp đông nhất và lớp ít nhất. Ví dụ minh hoạ trong paper: 10 lớp, 5 lớp có 500 ảnh huấn luyện, 5 lớp có 5.000 ảnh, khi đó $\rho = 10$ và $\mu = 0{,}5$.

- **Linear imbalance** (mất cân bằng tuyến tính): chỉ dùng một tham số ρ như công thức (2), số lượng ví dụ ở các lớp còn lại được nội suy tuyến tính sao cho hiệu số giữa hai lớp liên tiếp là hằng số (không có "bậc" rõ ràng như step).

Ví dụ tính tay (tự tính): với ví dụ 10 lớp ở trên ($\mu = 0{,}5$, $\rho = 10$, 5 lớp 500 ảnh và 5 lớp 5.000 ảnh), tổng số ảnh huấn luyện là $5 \times 500 + 5 \times 5000 = 2500 + 25000 = 27500$. Xác suất tiên nghiệm (prior) của một lớp thiểu số là $p(\text{minority}) = 500/27500 \approx 0{,}01818$, của một lớp đa số là $p(\text{majority}) = 5000/27500 \approx 0{,}18182$. Tỉ số hai prior này đúng bằng $\rho = 10$, như kỳ vọng. Đây là minh hoạ cho công thức thresholding ở mục 3.2 dưới đây: nếu không hiệu chỉnh, đầu ra mạng cho lớp đa số sẽ bị "thiên vị" gấp khoảng 10 lần so với lớp thiểu số chỉ vì tần suất huấn luyện, không phải vì đặc trưng ảnh.

3.2 Các phương pháp xử lý mất cân bằng (Mục 2 của paper)

Paper chia các phương pháp hiện có trong literature thành hai nhóm chính, rồi chọn ra 7 biến thể để so sánh trực tiếp.

Nhóm data level (sửa trên dữ liệu, giữ nguyên thuật toán học):

- *Oversampling*: "random minority oversampling" - nhân bản ngẫu nhiên các ví dụ từ lớp thiểu số. Paper liệt kê các biến thể "thông minh" hơn trong literature (SMOTE - tạo ví dụ tổng hợp bằng nội suy giữa các điểm lân cận; cluster-based oversampling; DataBoost-IM; class-aware sampling cho SGD) nhưng **chỉ dùng bản random đơn giản nhất** trong thí nghiệm của mình.
- *Undersampling*: "random majority undersampling" - xoá ngẫu nhiên ví dụ từ lớp đa số cho tới khi các lớp bằng nhau. Paper cũng liệt kê biến thể "one-sided selection" (chỉ xoá các ví dụ gần biên giữa các lớp) nhưng không dùng trong thí nghiệm chính.

Nhóm classifier level (giữ nguyên dữ liệu, sửa thuật toán học hoặc suy luận):

- *Thresholding* (threshold moving / post scaling): áp dụng ở giai đoạn test, điều chỉnh ngưỡng quyết định. Paper trích dẫn kết quả rằng mạng neural ước lượng xác suất hậu nghiệm Bayes: với một điểm dữ liệu x, đầu ra của mạng cho lớp i xấp xỉ

$$y_i(x) = p(i \mid x) = \frac{p(i) \cdot p(x \mid i)}{p(x)} \tag{3}$$

Do đó có thể lấy lại xác suất "đúng" (không bị lệch theo tần suất huấn luyện) bằng cách chia đầu ra của mỗi lớp cho prior ước lượng của nó $p(i) = |i| / \sum_k |k|$ (tần suất lớp i trong tập huấn luyện mất cân bằng, tính **trước khi** áp dụng sampling).

- *Cost sensitive learning*: gán chi phí (cost) khác nhau cho lỗi phân loại các lớp khác nhau; paper lưu ý một cách cài đặt (minimize trực tiếp cost thay vì loss chuẩn) cho kết quả **tương đương oversampling**, nên không triển khai riêng trong thí nghiệm.
- *One-class classification* (novelty detection): dùng autoencoder học ánh xạ đồng nhất (identity mapping), phân loại dựa trên lỗi tái tạo (reconstruction error); paper xếp đây vào trường hợp đặc biệt cho mất cân bằng cực đoan (anomaly detection), nằm ngoài phạm vi nghiên cứu chính.

Nhóm hybrid (kết hợp): ensembling (EasyEnsemble, BalanceCascade - huấn luyện nhiều classifier trên các tập con đã undersample; SMOTEBoost - kết hợp boosting với SMOTE), và đặc biệt **two-phase training**: huấn luyện mạng trên dữ liệu đã cân bằng (bằng oversampling hoặc undersampling) ở phase 1, sau đó fine-tune riêng các lớp đầu ra cuối (trước softmax) trên dữ liệu gốc (mất cân bằng) ở phase 2, giữ nguyên các siêu tham số và lịch suy giảm learning rate của phase 1 nhưng nhân learning rate gốc với 10^{-1}.

Từ các nhóm trên, paper chọn **7 phương pháp cụ thể** để so sánh có hệ thống trên CNN:

1. Random minority oversampling
2. Random majority undersampling
3. Two-phase training (pre-train trên tập đã oversample)
4. Two-phase training (pre-train trên tập đã undersample)
5. Thresholding với prior class probabilities
6. Oversampling + thresholding
7. Undersampling + thresholding

Lý do loại các phương pháp khác: ensemble cần huấn luyện nhiều mạng, quá tốn thời gian với deep network; one-class chỉ phù hợp mất cân bằng cực đoan kiểu anomaly detection, ngoài phạm vi; cost-sensitive-trực-tiếp cho kết quả tương đương oversampling nên bỏ.

3.3 Thiết lập thí nghiệm: 3 bộ dữ liệu, 3 kiến trúc

Dataset	Kích thước ảnh (W×H×D)	Số lớp	Ảnh/lớp (train/test, bản cân bằng)	CNN dùng
MNIST	28×28×1	10	5.000 / 1.000	LeNet-5 (bản hiện đại)
CIFAR-10	32×32×3	10	5.000 / 1.000	All-CNN
ILSVRC-2012 (ImageNet)	$\geq$ 256×$\geq$ 256×3	1.000	1.000 / 50	ResNet-10

(Bảng 1 của paper.) Cả ba bộ đều được subsample để có **bản cân bằng hoàn hảo** làm gốc, rồi từ đó tạo các phiên bản mất cân bằng nhân tạo theo step/linear ở trên. Tất cả mạng cùng một dataset được huấn luyện với **cùng số bước lặp (iterations)** cố định, tức số epoch khác nhau giữa các phiên bản mất cân bằng (vì tổng số

ảnh khác nhau), để giữ số lần cập nhật trọng số không đổi. Không dùng pretraining, trọng số khởi tạo ngẫu nhiên. Toàn bộ thí nghiệm cài bằng framework Caffe.

MNIST (LeNet-5 hiện đại, Bảng 2): kiến trúc gồm 2 lớp conv (kernel 5, không padding) xen max-pooling (kernel 2, stride 2), rồi 2 fully-connected (500 unit có ReLU, 10 unit), softmax. Huấn luyện 10.000 iteration, SGD với momentum $\mu = 0{,}9$, lịch learning rate $\eta_t = \eta_0 \cdot (1 + \gamma \cdot t)^{-\alpha}$ với $\eta_0 = 0{,}01$, $\gamma = 0{,}0001$, $\alpha = 0{,}75$; batch size 64; weight decay $\lambda = 0{,}0005$; khởi tạo Xavier (uniform) cho trọng số, 0 cho bias; không data augmentation. Test error của baseline (không mất cân bằng) dưới 1%.

Không gian tham số mất cân bằng cho MNIST: linear thử $\rho \in \{10, 25, 50, 100, 250, 500, 1000, 2500, 5000\}$; step dùng cùng dải ρ, với mọi số lớp thiểu số từ 1 đến 9 (tức $\mu \in \{0{,}1, \ldots, 0{,}9\}$). Mỗi tổ hợp lặp lại **50 lần** (ngẫu nhiên lại tập lớp thiểu số mỗi lần), tạo ra 4.050 tập mất cân bằng dạng step và 450 tập dạng linear (tổng 4.500 tập dữ liệu). Với 4 phương pháp cần huấn luyện riêng (cộng baseline), tổng số mạng đã huấn luyện trên MNIST là **22.500**.

CIFAR-10 (All-CNN, Bảng 3): kiến trúc toàn convolution (không fully-connected), có dropout 0,2/0,5 xen giữa, kết thúc bằng average pooling và softmax. Huấn luyện 70.000 iteration, SGD momentum 0,9, learning rate nhân 0,1 sau 40.000, 50.000, 60.000 iteration; batch 256; weight decay $\lambda = 0{,}001$; khởi tạo Xavier. Test error baseline **9,75%**. Vì nhạy với learning rate gốc, paper dò trong tập $\eta_0 \in \{0{,}05;\ 0{,}005;\ 0{,}0005;\ 0{,}00005\}$, mỗi cấu hình lặp 2 lần, chọn kết quả tốt nhất trên 8 lần chạy (4 lr $\times$ 2 lần). Không gian mất cân bằng (sparser hơn MNIST vì tốn thời gian hơn): $\rho \in \{2, 10, 20, 50\}$ cho cả linear và step, với step dùng $\mu \in \{0{,}2; 0{,}5; 0{,}8\}$ (tương ứng 2, 5, 8 lớp thiểu số). Tổng cộng **640 mạng** đã huấn luyện trên CIFAR-10.

ImageNet/ILSVRC-2012 (ResNet-10): số ví dụ lớp đa số giảm từ 1.200 xuống 1.000; tiền xử lý: resize cạnh ngắn về 256 pixel, train crop ngẫu nhiên 224×224, test center crop, lật ngang ngẫu nhiên lúc train (không màu/scale/aspect augmentation). ResNet-10 (4 residual block, 9 conv layer + 1 fully-connected, batch norm). Huấn luyện 320.000 iteration, SGD momentum 0,9, $\eta_0 = 0{,}1$ suy giảm tuyến tính về 0; batch 256; weight decay 0,0001; khởi tạo Kaiming (MSRA). Baseline: top-1 error **62,56%**, multi-class ROC AUC **99,50** (centered crop đơn). Do chi phí huấn luyện (paper ghi chú: **5 ngày** cho một mạng ResNet-10 trên GPU Nvidia GTX 1070), chỉ thử một trường hợp mất cân bằng nhẹ và hai trường hợp nặng, trên baseline, undersampling, oversampling: $(\mu, \rho) = (0{,}1, 10)$, $(0{,}8, 50)$, $(0{,}9, 100)$ - tương ứng 100 lớp thiểu số với tỉ số 10, 800 lớp thiểu số với tỉ số 50, 900 lớp thiểu số với tỉ số 100. Trường hợp mất cân bằng cao nhất được lặp 3 lần (chọn lại lớp thiểu số và subsample ngẫu nhiên) để ước lượng độ biến động. Tổng cộng **15 mạng ResNet-10** trên 5 tập con mất cân bằng nhân tạo.

3.4 Thước đo đánh giá: multi-class ROC AUC

Paper lập luận **accuracy tổng thể** (tỉ lệ dự đoán đúng) có nhược điểm nghiêm trọng khi dữ liệu mất cân bằng: nếu một lớp chiếm 99% số ví dụ test, một bộ phân loại luôn đoán lớp đó sẽ đạt accuracy "ảo" 99% dù vô dụng. Vì vậy thước đo chính là **ROC AUC** (area under receiver operating characteristic curve) mở rộng cho đa lớp: tính AUC cho từng lớp theo kiểu one-vs-rest (phân biệt lớp đó với tất cả lớp còn lại), rồi lấy trung bình các AUC đó. Dùng cài đặt trong scikit-learn (quy tắc hình thang - trapezoid rule). Tập test ở mọi dataset giữ nguyên cân bằng (không chỉnh theo tập train mất cân bằng), để các điểm số có thể so sánh trực tiếp giữa các cấu hình.

4. Thí nghiệm và kết quả chính

4.1 Hiệu ứng của mất cân bằng và so sánh phương pháp (MNIST, CIFAR-10, Hình 3-5 của paper). Mức giảm hiệu năng do mất cân bằng là **đáng kể**, tăng theo cả tỉ số ρ và số lớp thiểu số μ. Quan sát quan trọng: **độ phức tạp của bài toán khuếch đại hiệu ứng mất cân bằng**: một mức giảm hiệu năng tương đương trên MNIST và CIFAR-10 tương ứng với mức mất cân bằng ở MNIST **mạnh hơn khoảng 100 lần** so với CIFAR-10 (nguyên văn ý paper, không có một hệ số chính xác hơn trong text). Về phương pháp: **oversampling** là phương pháp tốt nhất trong gần như mọi tình huống, luôn cải thiện so với baseline (không xử lý) và **không bao**

giờ làm giảm hiệu năng đáng kể trên hai dataset này. **Undersampling** nói chung kém: giảm hiệu năng so với baseline trong nhiều trường hợp, chỉ cải thiện ở các trường hợp có tỉ lệ lớp thiểu số cao, nhưng vẫn không vượt được oversampling. **Two-phase training** (cả hai biến thể) cho kết quả nằm **giữa** baseline và phương pháp sampling tương ứng của nó: nếu baseline tốt hơn sampling, fine-tuning cải thiện phương pháp gốc; ngược lại (trường hợp phổ biến với oversampling) fine-tuning luôn làm **giảm** điểm số so với oversampling thuần.

4.2 Kết quả trên ImageNet (Bảng 4):

Phương pháp	$\mu = 0{,}1, \rho = 10$	$\mu = 0{,}8, \rho = 50$	$\mu = 0{,}9, \rho = 100$ (3 lần chạy)
Baseline	99,41	96,31	90,74 / 90,46 / 90,05
Oversampling	99,35	95,06	88,38 / 88,39 / 88,17
Undersampling	96,85	94,98	88,35 / 84,08 / 83,74

Mức giảm điểm ROC AUC lớn nhất quan sát được (mất cân bằng cao nhất) là **từ khoảng 99 xuống khoảng 90** (nguyên văn paper). **Ví dụ tính tay (tự tính):** lấy trung bình 3 lần chạy baseline ở mức mất cân bằng cao nhất, $(90{,}74 + 90{,}46 + 90{,}05)/3 = 90{,}417$; so với baseline ở mức mất cân bằng nhẹ nhất $99{,}41$, độ giảm tuyệt đối là $99{,}41 - 90{,}417 \approx 9{,}0$ điểm ROC AUC, khớp với mô tả định tính "từ 99 xuống 90" của paper. Khác với MNIST/CIFAR-10, trên ImageNet **oversampling giảm nhẹ so với baseline** ở mức mất cân bằng cực đoan nhất (paper nhấn mạnh: kết quả này nên được nhìn với sự cẩn trọng, vì tập mất cân bằng cao nhất chỉ còn khoảng 10% kích thước ILSVRC-2012 gốc, sinh ra các vấn đề "confound" khác như learning rate tối ưu cho tập đã đổi). Dù vậy, oversampling vẫn **nhất quán tốt hơn undersampling** trong mọi trường hợp trên ImageNet; độ lệch tuyệt đối giữa 3 lần chạy của undersampling ở mức cao nhất vượt quá 4 điểm ROC AUC, lớn hơn chênh lệch 1-2 điểm giữa một số phương pháp, nên các khác biệt nhỏ cần được diễn giải cẩn trọng vì biến động giữa các lần chạy.

5. Phân tích / ablation

5.1 Tách biệt hiệu ứng "ít dữ liệu hơn" và "mất cân bằng" (Mục 4.3). Câu hỏi: liệu hiệu năng giảm chỉ vì tập mất cân bằng có **ít ví dụ tổng** hơn, hay thực sự vì **phân phối lệch**? Ba luận điểm paper đưa ra: (1) oversampling dùng **đúng lượng dữ liệu bằng baseline** (chỉ nhân bản) nhưng vẫn cải thiện hiệu năng trong hầu hết trường hợp, dù không đạt lại hiệu năng của tập cân bằng gốc, chứng tỏ hiệu ứng mất cân bằng không hề nhỏ; (2) ở một số trường hợp, undersampling (dùng **ít dữ liệu hơn** baseline) vẫn tốt hơn baseline, và đôi khi ngang oversampling, cho thấy giữa hai phương pháp cùng xoá mất cân bằng, dùng ít dữ liệu hơn vẫn có thể chấp nhận được; (3) so sánh linear imbalance và step imbalance với $\mu = 0{,}5$ ở cùng ρ (cùng tổng số ví dụ huấn luyện), mức giảm hiệu năng của step **lớn hơn hẳn** linear, cho thấy không chỉ tổng số ví dụ mà **cách phân bố giữa các lớp** mới là yếu tố quyết định.

5.2 Thresholding cải thiện accuracy, không đổi ROC AUC (Mục 4.4). Vì nhân đầu ra quyết định với một số dương bất kỳ không làm đổi ROC AUC (bất biến theo scale), thresholding **không có tác dụng lên khả năng phân biệt** giữa các lớp đo bằng ROC AUC, nhưng giúp tìm điểm vận hành (operating point) tối ưu trên đường ROC, cải thiện rõ accuracy tổng thể, đặc biệt khi **kết hợp với oversampling**. Paper khuyến nghị cụ thể: nên kết hợp thresholding với baseline hoặc oversampling, **không nên** kết hợp với undersampling.

5.3 Mức độ sampling trung gian (Mục 4.5, MNIST $\rho = 1000$). Thử giảm mất cân bằng từng phần (không nhất thiết xoá hoàn toàn): với oversampling, **bản đầy đủ (xoá hoàn toàn mất cân bằng) luôn là tốt nhất**, mọi mức giảm nhẹ hơn đều cải thiện so với không xử lý nhưng kém hơn bản đầy đủ. Với undersampling, một số trường hợp có số lớp thiểu số trung bình, mức **trung gian** lại tốt hơn cả undersampling đầy đủ và baseline - nghĩa là với undersampling, "giảm mất cân bằng tới mức nào" cần dò theo bài toán cụ thể, không có quy tắc "luôn làm hết mức" như oversampling.

5.4 Oversampling và overfitting (Mục 4.6), phát hiện lệch với machine learning cổ điển. Ở machine learning cổ điển, oversampling được biết là có thể gây overfitting (do lặp lại một số ít ví dụ thiểu số nhiều lần, mô hình học vẹt chúng). Dựa trên hiểu biết này, undersampling được kỳ vọng tổng quát hoá tốt hơn. Thực nghiệm của paper trên CNN (so sánh đường cong train/test accuracy theo iteration, Hình 8, trường hợp CIFAR-10 step imbalance 5 lớp thiểu số, $\rho = 50$) **không xác nhận** điều này: khoảng cách giữa accuracy train và test **không tăng** theo iteration khi dùng oversampling, nghĩa là CNN **không bị overfit** do oversampling trong các kịch bản đã thử; hiện tượng này được paper xác nhận lại trên nhiều kịch bản khác (MNIST, ImageNet, các mức mất cân bằng khác, phụ lục A) và không quan sát overfitting ở bất kỳ kịch bản nào.

6. Giới hạn

Paper tự nêu:

- Kết quả trên ImageNet ở mức mất cân bằng cực đoan nhất nên được "xem xét cẩn trọng, không phải bằng chứng mạnh" rằng oversampling kém hiệu quả ở bài toán phức tạp cao, vì tập mất cân bằng đó chỉ còn ~10% kích thước gốc, kéo theo các yếu tố nhiễu khác (ví dụ learning rate tối ưu đã thay đổi theo kích thước tập dữ liệu mới).
- Chỉ dùng biến thể **đơn giản nhất** của oversampling (random) và undersampling (random), không thử SMOTE, cluster-based oversampling, one-sided selection, hay ensemble method, dù các phương pháp này được liệt kê trong phần tổng quan.
- Không gian tham số mất cân bằng cho CIFAR-10 và đặc biệt ImageNet "sparser" (ít điểm thử) hẳn so với MNIST, chỉ vì lý do chi phí tính toán (một mạng ResNet-10 mất 5 ngày trên GTX 1070), nên các kết luận ở quy mô ImageNet dựa trên ít điểm dữ liệu hơn hẳn.
- Lớp chọn làm thiểu số trên ImageNet luôn là các lớp có nhãn số hiệu thấp nhất (không ngẫu nhiên hoá đầy đủ như MNIST/CIFAR-10 với 50 lần lặp), nên kết quả ImageNet ít được kiểm chứng về độ biến động hơn.

Người ghi chú nhận xét:

- Toàn bộ kết luận dựa trên **ba kiến trúc CNN cụ thể** (LeNet-5, All-CNN, ResNet-10) huấn luyện từ đầu, không pretrain; không rõ kết luận "oversampling không gây overfit" có giữ nguyên với mô hình pretrain-rồi-finetune (cách làm phổ biến hiện nay) hay với kiến trúc Transformer/attention, vì cơ chế overfitting có thể khác khi phần lớn tham số đã được khởi tạo từ tri thức có sẵn.
- Toàn bộ thí nghiệm là **phân loại ảnh đơn nhãn** (single-label classification); không rõ các kết luận có áp dụng cho bài toán sequence-to-sequence hoặc token-level classification (như OCR) hay không, vì ở đó "lớp" có thể là từng ký tự/token xuất hiện lặp lại trong một mẫu, khác hẳn với một ảnh một nhãn.
- Paper dùng framework Caffe (2017), các con số về thời gian huấn luyện (5 ngày/mạng ResNet-10 trên GTX 1070) đã lạc hậu nhiều so với phần cứng/framework hiện tại, chỉ nên dùng để hiểu **quy mô tương đối** của chi phí thí nghiệm khi đọc bài, không dùng làm tham chiếu tuyệt đối.

7. Liên hệ với các bài khác

- **ml basics 30** (When Does Label Smoothing Help?): một kỹ thuật thực hành khác tác động lên phân phối xác suất đầu ra/nhãn; cùng nhóm "mẹo huấn luyện CNN", có thể tương tác với mất cân bằng lớp (nhãn mềm làm giảm độ tự tin tuyệt đối vào lớp đa số).
- **ml basics 31** (mixup: Beyond Empirical Risk Minimization): một cách tăng cường dữ liệu bằng nội suy, có thể xem như một hướng "oversampling thông minh hơn" khác với random oversampling thuần của bài này.
- **ml basics 32** (Focal Loss for Dense Object Detection): xử lý mất cân bằng lớp (đặc biệt mất cân bằng foreground/background trong detection) ***ở mức hàm loss***, khác hướng data-level/classifier-level-threshold của bài này; hai bài bổ sung cho nhau về các "tầng" có thể can thiệp khi dữ liệu mất cân bằng (dữ liệu, loss, hoặc hậu xử lý đầu ra).

- **ml basics 19** (On Calibration of Modern Neural Networks): thresholding sửa lại xác suất đầu ra theo prior lớp, có liên hệ gần với khái niệm **calibration** (xác suất đầu ra có khớp với tần suất thật không); bài này cho thấy một nguồn gây "miscalibration" cụ thể là mất cân bằng lớp trong tập huấn luyện.

8. Gợi ý áp dụng

Với dữ liệu mất cân bằng, nên so sánh oversampling, undersampling và baseline bằng các chỉ số phản ánh cả lớp hiếm. Kết luận thực nghiệm của bài được rút ra trên các CNN và tập dữ liệu đã xét, không tự động áp dụng cho mọi mô hình.

9. Thí nghiệm gợi ý

1. **Tái tạo mini step-imbalance trên MNIST/CIFAR-10 nhỏ:** với một CNN nhỏ (vài lớp conv), tạo 2-3 mức (μ, ρ) (ví dụ $\mu = 0{,}5, \rho \in \{10, 100\}$), huấn luyện baseline/oversampling/undersampling, đo multi-class ROC AUC. Kỳ vọng: oversampling $\geq$ baseline $>$ undersampling, khớp chiều hướng của paper; diễn giải: nếu không khớp, kiểm tra lại learning rate (paper nhấn mạnh CIFAR-10 nhạy với lr khi mất cân bằng tăng).
2. **Kiểm tra overfitting của oversampling (tái tạo Mục 4.6):** vẽ accuracy train/test theo epoch cho baseline và oversampling trên cùng tập mất cân bằng nhỏ. Đo: khoảng cách train-test accuracy có tăng theo thời gian với oversampling không. Kỳ vọng theo paper: không tăng rõ rệt; nếu một kiến trúc hoặc cách huấn luyện khác cho kết quả khác, cần xem xét giới hạn tổng quát hóa của kết luận.
3. **So sánh thresholding ảnh hưởng ROC AUC vs accuracy:** trên cùng một mô hình baseline (không sampling) huấn luyện trên tập mất cân bằng, áp post-hoc thresholding (chia đầu ra cho prior ước lượng theo công thức (3) ở mục 3.2), đo lại cả ROC AUC và accuracy trước/sau. Kỳ vọng: ROC AUC gần như không đổi (do bất biến scale theo lý thuyết), accuracy tăng. Nếu ROC AUC đổi đáng kể, kiểm tra lại cách tính AUC (có thể đã tính nhầm theo accuracy-based threshold thay vì toàn bộ đường ROC).
4. **Mức sampling trung gian (tái tạo Mục 4.5) ở quy mô nhỏ:** với một mức mất cân bằng cố định, thử 3-4 mức oversampling/undersampling trung gian (ví dụ chỉ bù 25%/50%/75%/100% mất cân bằng) và đo ROC AUC. Kỳ vọng: với oversampling, 100% (bù hết) luôn tốt nhất (đường đơn điệu); với undersampling, có thể thấy một điểm giữa tốt hơn cả hai đầu, như paper quan sát trên MNIST $\rho = 1000$.
5. **Thử trên bài toán sequence-level giả lập**: tạo một bài toán phân loại ký tự đơn giản (ví dụ phân loại ký tự viết tay từ nhiều bộ font, mô phỏng OCR) với một vài lớp ký tự hiếm, áp oversampling ở mức mẫu/ảnh, xem kết luận "không overfit" có còn giữ hay không khi mô hình nhỏ hơn và lớp ít dữ liệu hơn nhiều.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Step imbalance	$\mu = \frac{\lvert\{\text{lớp thiểu số}\}\rvert}{N}$, $\rho = \frac{\max_i \lvert C_i\rvert}{\min_i \lvert C_i\rvert}$	công thức (1)-(2)
Linear imbalance	chỉ dùng ρ, nội suy tuyến tính số lượng giữa các lớp	
Thresholding	$y_i(x) = p(i \mid x) = \frac{p(i)p(x \mid i)}{p(x)}$, chia đầu ra cho $p(i) = \lvert i\rvert / \sum_k \lvert k\rvert$	công thức (3)
7 phương pháp so sánh	oversampling, undersampling, 2 biến thể two-phase, thresholding, oversampling+threshold, undersampling+threshold	Mục 3.2 paper
Thước đo	multi-class ROC AUC (trung bình AUC one-vs-rest từng lớp)	bất biến theo scale dương
MNIST	LeNet-5, baseline error $< 1\%$, 22.500 mạng huấn luyện	

Khái niệm	Công thức / giá trị	Ghi chú
CIFAR-10	All-CNN, baseline error 9,75%, 640 mạng	
ImageNet	ResNet-10, baseline top-1 error 62,56%, ROC AUC 99,50; 15 mạng	
Kết luận chính	oversampling tốt nhất, áp dụng tới mức xoá hết mất cân bằng; không gây overfit trên CNN	khác với hiểu biết ở machine learning cổ điển

11. Kết luận ngắn

Qua hàng chục nghìn mạng CNN huấn luyện trên MNIST, CIFAR-10 và ImageNet với hàng nghìn cấu hình mất cân bằng nhân tạo (step và linear), paper kết luận: mất cân bằng lớp luôn gây hại cho hiệu năng CNN và hại nặng hơn khi bài toán phức tạp hơn; trong 7 phương pháp xử lý được so sánh, **oversampling (bản đơn giản nhất, nhân bản ngẫu nhiên)** vượt trội gần như luôn đúng và nên áp dụng **tới mức xoá hoàn toàn mất cân bằng**, trong khi mức undersampling tối ưu phải dò theo từng bài toán; **thresholding** (chia đầu ra cho prior lớp) không đổi ROC AUC nhưng cải thiện accuracy, nên kết hợp với oversampling chứ không với undersampling; và phát hiện quan trọng nhất là **oversampling không gây overfitting trên CNN**, trái với lo ngại phổ biến dựa trên kinh nghiệm machine learning cổ điển. Các kết luận này là thực nghiệm (không có chứng minh lý thuyết kèm theo) và giới hạn ở ba kiến trúc CNN huấn luyện từ đầu cho bài toán phân loại ảnh đơn nhãn.

Nguồn gốc: https://arxiv.org/abs/1710.05381

Bài 34

A survey of cross-validation procedures for model selection (2009)

Tác giả: Sylvain Arlot (CNRS, nhóm Willow, Laboratoire d'Informatique de l'École Normale Supérieure), Alain Celisse (Laboratoire Paul Painlevé, Université des Sciences et Technologies de Lille 1).
Venue / arXiv: arXiv:0907.4728 (v1, nộp 27/07/2009; chỉ có một phiên bản trên arXiv). Trang đầu bản PDF có in dòng "October 22, 2018", nhiều khả năng là ngày build lại file (không phải ngày nộp mới, vì không có v2 trên arXiv). Bản PDF arXiv không ghi tên venue/tạp chí; paper không nêu venue trong văn bản (ngoài paper, kiến thức chung: bài sau này được in ở tạp chí Statistics Surveys, nhưng dòng này không xuất hiện trong bản PDF đã tải).
Nguồn: https://arxiv.org/abs/0907.4728
Chủ đề: khảo sát lý thuyết và thực nghiệm (survey, không có thí nghiệm deep learning mới) về các thủ tục cross-validation (CV) dùng để chọn mô hình (model selection), phân biệt rõ hai mục tiêu "estimation" và "identification", và tổng hợp các kết quả đã biết về bias, variance, độ phức tạp tính toán của từng loại CV.
Vị trí trong lộ trình: Đọc trước các bài 35–37 về độ bất định; cross-validation là công cụ chọn và đánh giá mô hình.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận. Bản trích xuất PDF có lỗi font và bố cục ở một số công thức; những công thức không chắc chắn được diễn giải định tính.

1. Bối cảnh

Model selection (chọn mô hình) là bài toán: khi có nhiều thuật toán/mô hình ứng viên (ví dụ nhiều bậc đa thức, nhiều giá trị regularization, nhiều kiến trúc), ta cần chọn một trong số đó **chỉ dựa trên dữ liệu**, trong khi chính dữ liệu đó cũng được dùng để huấn luyện các mô hình ứng viên. Khó khăn cốt lõi: nếu đánh giá một mô hình trên đúng dữ liệu đã huấn luyện nó (gọi là **resubstitution error**), kết quả sẽ "quá lạc quan" (overoptimistic) vì mô hình có thể học vẹt dữ liệu đó.

Cross-validation (CV) là chiến lược phổ biến nhất để giải quyết vấn đề này: chia dữ liệu (một hoặc nhiều lần) thành phần huấn luyện (training sample) và phần kiểm định (validation sample); huấn luyện trên phần đầu, ước lượng rủi ro (risk) trên phần sau (phần kiểm định đóng vai trò "dữ liệu mới" vì độc lập với phần huấn luyện khi dữ liệu i.i.d.). CV phổ biến vì **tính tổng quát**: chỉ cần giả định dữ liệu độc lập cùng phân phối (i.i.d.), không cần biết gì về mô hình hay phân phối cụ thể, nên áp dụng được cho hầu như mọi bài toán (regression, phân loại, density estimation...). Tuy vậy, paper nhấn mạnh ngay từ đầu: **tính tổng quát này không đồng nghĩa CV luôn hoạt động tốt** - nhiều thủ tục CV đã được chứng minh **thất bại** với một số bài toán chọn mô hình, tuỳ theo **mục tiêu** của việc chọn mô hình là gì.

Khái niệm nền tảng cần nắm trước khi đọc:

- **Loss function** $L(t)$ và **contrast function** γ: đo "t khớp với dữ liệu tốt đến đâu"; ví dụ contrast bình phương sai số $\gamma(t;(x,y)) = (t(x)-y)^2$ trong regression.
- **Risk** của một thuật toán/ước lượng: kỳ vọng của "excess loss" (chênh lệch loss so với loss tối ưu) trên dữ liệu mới.
- **Mô hình (model)** S_m: một tập con các hàm ứng viên; **oracle model** $S_{m^\star}$: mô hình cho excess loss nhỏ nhất **nếu biết trước phân phối thật** (không thể tính được trong thực tế, chỉ dùng làm chuẩn so sánh lý thuyết).

Câu hỏi nghiên cứu của survey: CV thực chất đang làm gì? Khi nào CV hoạt động tốt cho model selection (nhớ rằng "tốt" có thể nghĩa khác nhau tuỳ mục tiêu)? Với một bài toán cụ thể, nên chọn thủ tục CV nào?

2. Ý tưởng chính

Ý tưởng cốt lõi, tách biệt rạch ròi hai **mục tiêu khác nhau** của model selection, là đóng góp tổ chức quan trọng nhất của survey:

1. **Model selection cho estimation**: mục tiêu là mô hình được chọn cho **loss nhỏ nhất** (không quan tâm mô hình đó có "đúng" theo nghĩa cấu trúc hay không). Chất lượng đo bằng **hiệu quả tiệm cận (efficiency)** hoặc **bất đẳng thức oracle (oracle inequality)**: so sánh loss của mô hình được chọn với loss của oracle.
2. **Model selection cho identification**: mục tiêu là tìm đúng **mô hình thật** S_{m_0} (giả định có tồn tại một mô hình thật, nhỏ nhất, chứa s). Chất lượng đo bằng **tính nhất quán (consistency)**: xác suất chọn đúng m_0 tiến về 1 khi $n \to \infty$.

Hai mục tiêu này **mâu thuẫn nhau về mặt lý thuyết**: Yang (2005) chứng minh không thủ tục nào vừa model-consistent vừa minimax-rate-optimal cùng lúc (gọi là **AIC-BIC dilemma**, vì AIC tối ưu cho estimation còn BIC nhất quán cho identification). CV, vì chỉ dựa trên một nguyên lý chia dữ liệu duy nhất, **thể hiện cả hai xu hướng** tuỳ vào cách chọn kích thước tập huấn luyện n_t: lấy n_t gần n (dùng nhiều dữ liệu để train) có lợi cho estimation; lấy n_t nhỏ hơn hẳn n (dùng nhiều dữ liệu để validate) có lợi cho identification. Đây là nguồn gốc của cái gọi là **"cross-validation paradox"** (Yang, 2006): trực giác thông thường "dùng nhiều dữ liệu train hơn thì tốt hơn" chỉ đúng cho mục tiêu estimation, sai cho mục tiêu identification.

Kết luận tổng quát của survey (mục 10): không có một thủ tục CV "tốt nhất" duy nhất; lựa chọn phải dựa trên ba yếu tố **bias, variance (độ biến động), và chi phí tính toán**, cân nhắc theo mục tiêu cụ thể (estimation hay identification) và đặc điểm bài toán (tỉ số signal-to-noise, có nhiều mô hình ứng viên không, dữ liệu có phụ thuộc không...).

3. Phương pháp chi tiết

3.1 Khung thống kê và bài toán model selection (Mục 1-2 của paper)

Với dữ liệu $\xi_1, \ldots, \xi_n$ i.i.d. theo phân phối P chưa biết, mục tiêu là ước lượng một đặc trưng s của P (ví dụ hàm hồi quy, hàm mật độ, bộ phân loại Bayes). Loss được định nghĩa qua contrast:

$$L(t) = L_P(t) := \mathbb{E}_{\xi \sim P}[\gamma(t; \xi)] \tag{1}$$

Ví dụ cụ thể paper nêu: density estimation dùng $\gamma(t; x) = -\ln(t(x))$ (excess loss là Kullback-Leibler divergence); regression dùng least-squares $\gamma(t; (x, y)) = (t(x) - y)^2$ (excess loss là bình phương khoảng cách L^2); classification nhị phân dùng 0-1 contrast $\gamma(t; (x, y)) = \mathbb{1}_{t(x) \neq y}$ (bộ phân loại tối ưu là **Bayes classifier**).

Với một mô hình S_m (tập con ứng viên), ước lượng minimum-contrast $\hat{s}_m$ minimizing $L_{P_n}(t) = \frac{1}{n}\sum_i \gamma(t; \xi_i)$ trên S_m. Mô hình nhỏ có **bias** lớn (nếu s xa S_m); mô hình lớn có bias nhỏ nhưng $\hat{s}_m$ dễ overfit. Với mô hình là không gian vector chiều D_m, trong nhiều khung thống kê cổ điển:

$$\mathbb{E}[\ell(s, \hat{s}_m(D_n))] \approx \ell(s, S_m) + \lambda D_m \tag{2}$$

với $\lambda > 0$ không phụ thuộc m ($\lambda = 1/(2n)$ cho density estimation dùng log-likelihood contrast; $\lambda = \sigma^2/n$ cho regression dùng least-squares, nếu $\mathrm{var}(Y \mid X) = \sigma^2$ không phụ thuộc X). Đây chính là **bias-variance trade-off**: cần cân bằng $\ell(s, S_m)$ (bias) và λD_m (variance/estimation error).

Ví dụ tính tay (tự tính): áp dụng công thức (2) cho density estimation với $n = 100$ quan sát, một mô hình có $D_m = 5$ (ví dụ histogram 5 bin): số hạng variance $\lambda D_m = \dfrac{1}{2 \times 100} \times 5 = 0{,}025$. Nếu tăng n lên 1000 (giữ $D_m = 5$), số hạng này giảm xuống $0{,}0025$, tức giảm đúng 10 lần, khớp với việc $\lambda \propto 1/n$. Đây minh hoạ trực quan: cùng độ phức tạp mô hình, có nhiều dữ liệu hơn luôn giảm phần variance của bias-variance trade-off.

Hai mục tiêu (Mục 2.2-2.4):

- *Estimation*: mô hình tốt nhất có thể đạt là **oracle** $m^\star = \arg\min_{m \in M_n} \{\ell(s, \hat{s}_m(D_n))\}$ (công thức 3, phụ thuộc P thật nên không tính được). Một thủ tục $\hat{m}$ được gọi **hiệu quả (efficient)** nếu $\ell(s, \hat{s}_{\hat{m}(D_n)}) / \inf_m \{\ell(s, \hat{s}_m)\} \to 1$ hầu chắc chắn; hoặc thoả **oracle inequality** (công thức 4): $\ell(s, \hat{s}_{\hat{m}}) \le C_n \inf_m \{\ell(s, \hat{s}_m)\} + R_n$ với $C_n \to 1$.
- *Identification*: giả định tồn tại mô hình thật nhỏ nhất S_{m_0} chứa s; thủ tục $\hat{m}$ được gọi **(model) consistent** nếu $P(\hat{m}(D_n) = m_0) \to 1$.
- **AIC-BIC dilemma**: Yang (2005) chứng minh trong khung regression, không thủ tục nào đồng thời model-consistent (như BIC) và minimax-rate-optimal (như AIC).

3.2 Tổng quan các thủ tục model selection khác (Mục 3, làm nền để hiểu vị trí của CV)

Mọi thủ tục trong mục này chọn $\hat{m}(D_n) \in \arg\min_m \{\text{crit}(m; D_n)\}$ (công thức 5); dạng đặc biệt là **penalization**: $\hat{m} \in \arg\min_m \{L_{P_n}(\hat{s}_m) + \text{pen}(m; D_n)\}$ (công thức 6). Paper nhóm các thủ tục thành 5 loại (theo cách phân loại của Shao 1997 cho hồi quy tuyến tính):

1. **Nguyên lý ước lượng rủi ro không chệch (unbiased risk estimation)**: $\text{crit}(m)$ ước lượng không chệch (ít nhất tiệm cận) $L_P(\hat{s}_m)$. Ví dụ: AIC (dùng log-likelihood contrast), Mallows' C_p (dùng least-squares contrast), FPE, GCV, và chính CV (Leave-one-out) thuộc nhóm này. Các thủ tục này tối ưu cho estimation khi số mô hình $\text{Card}(M_n)$ không tăng quá nhanh (tối đa đa thức theo n).
2. **Ước lượng rủi ro có chệch (biased, "overpenalizing")**: trọng số phần variance trong $\text{crit}(m)$ lớn hơn (2) một chút. Ví dụ FPE_α với $\alpha > 2$, GIC_λ với $\lambda > 2$, Leave-p-out với p/n cố định trong $(0, 1)$. Hữu ích khi signal-to-noise ratio (SNR) nhỏ, hoặc khi số mô hình tăng theo cấp hàm mũ $e^{\kappa n}$ (lúc đó nguyên lý không chệch thất bại, theo Birgé và Massart 2007).
3. **Thủ tục xây riêng cho identification**: ví dụ BIC (trùng với $\text{GIC}_{\ln(n)}$); các thủ tục Minimum Description Length (MDL); Lasso cũng được xếp vào nhóm này cho bài toán chọn biến (variable selection).
4. **Structural risk minimization (Vapnik và Chervonenkis, 1974)**: phạt bằng chặn trên của sai số ước lượng (VC dimension, Rademacher complexity...); thường "quá bi quan" (pessimistic) trong thực tế.
5. **Penalization ad hoc**: phạt theo chuẩn ℓ^p của $\hat{s}_m$ (như Lasso, ridge, spline smoothing).

CV thuộc cả nhóm (1) và (2) tuỳ cách chọn n_t (kích thước tập huấn luyện trong mỗi lần chia), và khác các thủ tục khác ở điểm: CV **không trực tiếp dùng thông tin tiên nghiệm** về mô hình; muốn thêm thông tin đó phải kết hợp CV với một penalty riêng hoặc tiền chọn một tập con mô hình.

3.3 Định nghĩa các thủ tục cross-validation cổ điển (Mục 4)

Hold-out (validation đơn giản). Chia một lần dữ liệu thành $I^{(t)}$ (train, kích thước n_t) và $I^{(v)} = \overline{I^{(t)}}$ (validation, kích thước $n_v = n - n_t$):

$$\hat{L}^{H-O}(A; D_n; I^{(t)}) := \frac{1}{n_v} \sum_{i \in D_n^{(v)}} \gamma(A(D_n^{(t)}); (X_i, Y_i)) \tag{8}$$

CV tổng quát (Geisser, 1975). Lặp lại hold-out B lần với các tập train $I_1^{(t)}, \ldots, I_B^{(t)}$ khác nhau, lấy trung bình:

$$\hat{L}^{CV}(A; D_n; (I_j^{(t)})_{1\le j\le B}) := \frac{1}{B}\sum_{j=1}^{B} \hat{L}^{H-O}(A; D_n; I_j^{(t)}) \tag{9}$$

Mọi thủ tục CV cổ điển là một trường hợp riêng của (9), khác nhau ở cách chọn chuỗi $(I_j^{(t)})$. Paper cũng nhắc biến thể **CV-voting** (Yang 2006/2007): khi so hai thuật toán, chọn thuật toán thắng ở **đa số** các lần chia, khác với **CV-averaging** (công thức 9) là so **trung bình** điểm số.

Các thủ tục cụ thể (Mục 4.3):

Thủ tục	n_t (kích thước train)	B (số lần chia)	Ghi chú
Leave-one-out (LOO)	$n-1$	n	mỗi điểm lần lượt làm validation; công thức (10): $\hat{L}^{LOO} = \frac{1}{n}\sum_j \gamma(A(D_n^{(-j)}); \xi_j)$
Leave-p-out (LPO)	$n-p$	$\binom{n}{p}$	xét **mọi** tập con p điểm làm validation (exhaustive); LPO với $p=1$ là LOO
V-fold CV (VFCV)	$n(1-1/V)$	V	chia dữ liệu thành V phần gần bằng nhau, mỗi phần lần lượt làm validation; công thức (11); VFCV với $V=n$ là LOO
Balanced Incomplete CV (BICV)	n_t tuỳ chọn	B (theo balanced incomplete block design)	mỗi điểm/cặp điểm xuất hiện với tần suất đều nhau trong các lần chia
Repeated Learning-Testing (RLT)	n_t tuỳ chọn	B tuỳ chọn	B tập con **ngẫu nhiên độc lập**, không lặp; gần giống LPO khi B lớn
Monte-Carlo CV (MCCV)	n_t tuỳ chọn	B tuỳ chọn	giống RLT nhưng **cho phép** chọn lại cùng một cách chia nhiều lần

Các biến thể khác được nhắc tới: **GCV** (Generalized CV, Craven và Wahba 1979) là một phiên bản bất biến-xoay của LOO cho ước lượng tuyến tính trong regression, thực chất gần với C_L (Mallows) hơn là CV đúng nghĩa; **.632 bootstrap** và **.632+ bootstrap** (Efron 1983; Efron và Tibshirani 1997) dùng resampling kiểu bootstrap thay cho chia dữ liệu trực tiếp, có cơ sở lý thuyết yếu hơn nhưng được hỗ trợ bằng thực nghiệm; **V-fold penalization** (Arlot) là một phiên bản đã hiệu chỉnh bias của VFCV.

Lịch sử (Mục 4.4): hold-out xuất phát từ lĩnh vực tâm lý học (Larson, 1931); LOO được giới thiệu độc lập bởi Stone, Allen, và Geisser (1974-1975); ý tưởng dùng CV cho model selection (thay vì chỉ ước lượng risk của một thuật toán cố định) xuất hiện lần đầu quanh 1973-1974 (Efron và Morris; Geisser).

4. Thí nghiệm và kết quả chính

Đây là một survey lý thuyết thống kê, không có thí nghiệm deep learning mới do nhóm tác giả thực hiện. "Kết quả chính" ở đây là các **định lý, bất đẳng thức oracle, và kết luận khảo sát** tổng hợp từ nhiều bài báo khác (một số có thực nghiệm mô phỏng nhỏ được paper trích dẫn lại, không phải của chính survey này).

4.1 Bias của các ước lượng CV (Mục 5.1). Vì độc lập giữa tập train/validation:

$$\mathbb{E}\big[\hat{L}^{CV}(A; D_n; (I_j^{(t)}))\big] = \mathbb{E}[L_P(A(D_{n_t}))] \tag{12}$$

tức bias của CV chỉ phụ thuộc n_t: đúng bằng chênh lệch giữa risk của A khi huấn luyện với n_t và với n điểm. Vì $n_t < n$, bias này **thường không âm** (dương), chặt chẽ khi risk của A là hàm giảm theo n ("thuật toán khôn" - smart rule theo thuật ngữ paper); paper lưu ý **1-nearest-neighbour trong classification không phải một "smart rule"** (risk không đơn điệu giảm theo n trong mọi trường hợp). Kết quả thực nghiệm cụ thể paper trích dẫn (Davison và Hall, 1992): với bài toán phân biệt hai phân phối lệch nhau một khoảng $n^{-1/2}$, bias của LOO giữ bậc n^{-1} khi $n \to \infty$, còn bias của bootstrap **xấu đi** tới bậc $n^{-1/2}$ (LOO vượt trội). Một so sánh thực nghiệm khác (Molinaro et al., 2005, trên dữ liệu sinh học thật và tổng hợp): bias giảm theo n_t và **thường nhỏ nhất ở LOO**, nhưng **10-fold CV có bias gần như nhỏ nhất đồng loạt** trên các thí nghiệm của họ; .632+ bootstrap có bias nhỏ nhất khi mẫu nhỏ và SNR nhỏ nhưng bias lớn hơn hẳn trong các trường hợp khác.

4.2 Variance của các ước lượng CV (Mục 5.2). Với hold-out, phương sai phân tách thành hai phần:

$$\operatorname{var}\big[\hat{L}^{H-O}(A; D_n; I^{(t)})\big] = \mathbb{E}\Big[\operatorname{var}\big(L_{P^{(v)}}(A(D_n^{(t)})) \mid D_n^{(t)}\big)\Big] + \operatorname{var}[L_P(A(D_{n_t}))] \tag{13}$$

Số hạng đầu, tỉ lệ với $1/n_v$, giảm khi tập validation lớn hơn (ước lượng rủi ro chính xác hơn); số hạng hai phụ thuộc **độ ổn định (stability)** của thuật toán A. Khi A **không ổn định**, LOO thường có phương sai lớn (được ghi nhận nhiều nơi, kể cả sách Hastie et al. 2001); khi A ổn định, hiện tượng này biến mất (Molinaro et al. 2005, bằng mô phỏng). Trong khung hồi quy tuyến tính với dữ liệu homoscedastic, Burman (1989) chứng minh khai triển tiệm cận phương sai của VFCV: phương sai **giảm theo** V (số fold), ngụ ý LOO ($V = n$) có phương sai tiệm cận nhỏ nhất **trong khung này cụ thể** (kết quả không tổng quát hoá cho mọi khung, xem mục 5 dưới). Một kết quả tiêu cực quan trọng: **không tồn tại ước lượng không chệch phổ quát cho phương sai của RLT** (Nadeau và Bengio, 2003) **hay của VFCV** (Bengio và Grandvalet, 2004); nếu không tính đến cấu trúc tương quan giữa các lỗi test, phương sai của CV có thể bị **ước lượng thấp hơn thực tế rất nhiều**.

4.3 CV cho estimation (Mục 6). Phân loại của Shao (1997) trong hồi quy tuyến tính: khi $n_t \sim n$, CV **tiệm cận tương đương** Mallows' C_p (hiệu quả/efficient); khi $n_t \sim \lambda n$ với $\lambda \in (0, 1)$, CV tương đương GIC_κ với

$$\kappa = 1 + \lambda^{-1}$$

tức CV "overpenalize" (phạt nặng hơn AIC) với hệ số $(1 + \lambda)/(2\lambda) > 1$. Một quan sát quan trọng (Breiman và Spector, 1992): dù LOO là ước lượng rủi ro tốt nhất (ít bias/variance nhất) **đồng đều trên mọi mô hình**, **10-fold CV lại chính xác hơn khi dùng để model selection** - lý do: điều quyết định hiệu năng model selection không phải bias/variance tuyệt đối của mỗi ước lượng risk, mà là xác suất **giữ đúng dấu** của so sánh $\mathrm{crit}(m_1) - \mathrm{crit}(m_2)$ so với $L_P(\hat{s}_{m_1}) - L_P(\hat{s}_{m_2})$ cho các mô hình gần oracle.

4.4 CV cho identification (Mục 7). Shao (1993) chứng minh LOO, LPO, và BICV **không nhất quán (inconsistent)** cho bài toán chọn biến (variable selection) trong hồi quy tuyến tính khi $\liminf_{n\to\infty}(n_t/n) > 0$ (dù các thủ tục này chọn đúng mọi biến thật với xác suất 1 khi $n \to \infty$, xác suất chọn **thêm** biến dư không tiến về 0). Ngược lại, CV **nhất quán** khi $n \gg n_t \to \infty$ (ví dụ RLT, BICV, hoặc LPO với $p = p_n \sim n$ và $n - p_n \to \infty$). Đây dẫn tới **"cross-validation paradox"** (Yang, 2006): cần tập validation **lớn hơn hẳn** tập train để identification nhất quán, trái ngược khuyến nghị cho estimation. Yang (2006, 2007) tổng quát hoá cho bài toán so sánh 2 thuật toán bất kỳ (không cần giả định "mô hình thật" tồn tại): điều kiện đủ để CV chọn đúng thuật toán hội tụ nhanh hơn liên quan tới $n_v \max_i r_{n_t,i} \to \infty$ (công thức 14, hồi quy) hay $n_v \max_i r_{n_t,i}^2 / s_{n_t} \to \infty$ (công thức 15, classification), với $r_{n,i}$ là tốc độ hội tụ của thuật toán i. Hệ quả thực nghiệm được paper trích dẫn (Dietterich 1998; Alpaydin 1999): khi dùng VFCV để **so sánh hai quy trình học**, $V = 2$ là lựa chọn tốt nhất (ngược hẳn khuyến nghị V lớn cho bài toán estimation thông thường).

5. Phân tích / ablation

5.1 "CV-calibrated algorithms" và nguy cơ đánh giá lạc quan khi tái sử dụng dữ liệu (Mục 5.1.1). Khi một siêu tham số $\hat{\lambda}$ được chọn bằng cách minimize $\hat{L}^{CV}(A_\lambda; D_n)$ theo λ, thì chính $\hat{L}^{CV}(A_{\hat{\lambda}}; D_n)$ **bị chệch (lạc quan)** khi dùng làm ước lượng risk của $A_{\hat{\lambda}}(D_n)$ - vì $\hat{\lambda}$ đã được chọn dựa trên **chính dữ liệu đó**. Đây là hiện tượng tương tự optimism của loss huấn luyện. Cách làm đúng (paper gọi là **"double cross"**, theo Stone 1974): coi toàn bộ quy trình "chọn λ rồi fit" là một thuật toán A' duy nhất, và áp CV lên A' (tức CV lồng nhau / nested CV) để có ước lượng risk không lạc quan. Đây là một cạm bẫy thực hành rất phổ biến (data leakage khi tune hyperparameter bằng CV rồi báo cáo luôn điểm CV đó làm kết quả cuối).

5.2 Partial splitting làm tăng biến động (Mục 5.2.1). Với (n_t, n_v) cố định, các phương pháp **partial data splitting** (VFCV, RLT, MCCV, hold-out) có biến động lớn hơn LPO (exhaustive) vì chỉ xét một số nhỏ trong tổng số $\binom{n}{n_t}$ cách chia khả dĩ, thường chọn ngẫu nhiên. Với MCCV, biến động phụ thêm này giảm theo $1/B$ (vì các lần chia độc lập); với RLT/VFCV phức tạp hơn vì các lần chia **không độc lập**. Biến động phụ thêm là **lớn nhất ở hold-out** (chỉ 1 lần chia) và **bằng không (triệt tiêu)** ở LOO/LPO (vì đã lấy trung bình trên **mọi** cách chia có thể).

5.3 Không có "một con số V đúng cho mọi trường hợp" (Mục 10.3). Khuyến nghị dân gian "V giữa 5 và 10" chỉ đúng trong nhiều trường hợp thực tế (vì hiệu năng thống kê không tăng nhiều với V lớn hơn, còn chi phí tính toán tăng tuyến tính theo V), nhưng paper nhấn mạnh đây **không phải quy tắc tổng quát**: khi SNR lớn, so sánh tiệm cận ở mục 6 áp dụng được, LOO tối ưu còn VFCV với $V = O(1)$ là "suboptimal"; khi SNR nhỏ, phạt nặng hơn (như VFCV với V nhỏ) lại tốt hơn LOO nhờ bù lại bias bằng việc giảm variance. V lớn nhất **không phải luôn là lựa chọn tốt nhất cho regression** (ghi nhận thực nghiệm của Breiman và Spector, 1992): V tối ưu phụ thuộc bài toán.

5.4 Mất chi phí tính toán cho tính "exhaustive" (Mục 9). Chi phí tính toán thô của CV tỉ lệ với số lần chia B: 1 (hold-out), V (VFCV), B (RLT/MCCV), n (LOO), $\binom{n}{p}$ (LPO) lần huấn luyện lại thuật toán. **Ví dụ tính tay (tự tính):** với $n = 20$ và $p = 3$, số tập con cần xét cho LPO là $\binom{20}{3} = 1140$, so với chỉ 20 lần cho LOO ($p = 1$) hay 10 lần cho 10-fold VFCV; chi phí LPO với $p = 3$ đã lớn hơn LOO khoảng $1140/20 = 57$ lần (tự tính), minh hoạ tại sao LPO với $p > 1$ hiếm khi dùng trực tiếp trong thực hành dù có tính chất lý thuyết tốt (exhaustive). Với một số mô hình đặc biệt (regressogram, kernel/projection estimator trong density estimation, least-squares linear), paper liệt kê các **công thức dạng đóng (closed-form)** cho phép tính LOO/LPO/GCV mà không cần huấn luyện lại B lần, giảm mạnh chi phí này.

6. Giới hạn

Paper tự nêu:

- Không có **một kết luận tổng quát duy nhất** cho mọi khung thống kê; hành vi của CV (bias, variance) khác nhau rõ rệt giữa regression, density estimation, classification, và đôi khi **mâu thuẫn nhau** giữa các bài báo được trích dẫn (ví dụ giá trị V cho phương sai cực tiểu của VFCV).
- Không tồn tại **ước lượng không chệch phổ quát** cho phương sai của CV (RLT, VFCV) trong mọi phân phối; các ước lượng phương sai đã có đều **có chệch** và cần giả định thêm.
- Nhiều kết quả lý thuyết chỉ **phân biệt được hold-out và CV cùng n_t ở bậc nhất** (first order); các khác biệt bậc hai (second order), vốn quan trọng ở n thực tế, hầu như chưa có lý thuyết đầy đủ - paper liệt kê đây là **hướng nghiên cứu tương lai quan trọng nhất** (Mục 10.4).
- Việc chọn V cho VFCV "vẫn là câu hỏi mở", dù có vài gợi ý theo trường hợp cụ thể.
- Dữ liệu phụ thuộc (time series) cần CV sửa đổi (modified CV, h-block CV...) với các giả định mixing; CV "thuần" có thể overfit khi chọn bandwidth cho dữ liệu tương quan dương (Hart và Wehrly, 1986).

Người ghi chú nhận xét:

- Toàn bộ các kết quả định lượng (tốc độ bias/variance, điều kiện nhất quán) được chứng minh trong các

khung thống kê **cổ điển, tham số thấp** (hồi quy tuyến tính, histogram/kernel density estimation, phân loại nhị phân đơn giản) **từ trước 2009**; survey hoàn toàn **không đề cập tới deep learning**, mạng neural hiện đại, hay dữ liệu chiều cao (ảnh, văn bản). Việc suy rộng trực tiếp các con số cụ thể (ví dụ "LOO có variance nhỏ nhất") sang mô hình deep learning hiện đại là **không có cơ sở trong chính paper**, chỉ có thể dùng các **nguyên lý định tính** (bias-variance trade-off, cross-validation paradox, cảnh báo về double-dipping khi tune hyperparameter bằng CV).

- Vì là bản arXiv trích xuất bằng pdftotext có lỗi font (xem lưu ý ở đầu bài), một số công thức có cấu trúc phân số phức tạp (ví dụ công thức phương sai tiệm cận của VFCV ở Mục 5.2.2) bị xáo trộn thứ tự khi trích xuất văn bản thô; người ghi chú đã diễn giải định tính (xu hướng giảm theo V) thay vì chép lại công thức số có khả năng sai ký hiệu.

7. Liên hệ với các bài khác

- **ml basics 33** (A systematic study of the class imbalance problem in CNNs): bài trước, cùng nhóm "thực hành huấn luyện/đánh giá mô hình"; paper đó đánh giá bằng train/test split cố định (không CV), có thể xem CV survey này là phần mở rộng lý thuyết cho câu hỏi "chia dữ liệu thế nào mới đáng tin".
- **ml basics 35** (A Gentle Introduction to Conformal Prediction): một hướng khác cho đánh giá/ước lượng độ tin cậy mô hình **không cần giả định phân phối cụ thể**, nhưng (khác CV cổ điển ở đây) hướng tới cung cấp **tập dự đoán có bảo đảm coverage**, không chỉ ước lượng risk trung bình; cả hai đều dựa trên nguyên lý chia dữ liệu (data splitting) làm nền tảng.
- **ml basics 21** (Leakage and the Reproducibility Crisis in ML-based Science): "double cross"/data leakage khi tune hyperparameter bằng CV rồi báo cáo điểm CV đó (mục 5.1 bài này) là một dạng cụ thể, kinh điển của leakage mà bài 21 khảo sát ở quy mô lớn hơn trong khoa học dựa trên ML.
- **ml basics 22** (Underspecification Presents Challenges for Credibility in Modern ML): CV chỉ đánh giá được **một** mô hình trên **một** cách chia dữ liệu cụ thể; nếu bài toán underspecified (nhiều mô hình cùng đạt điểm CV tốt nhưng hành xử rất khác nhau ngoài phân phối train), bản thân chỉ số CV không đủ để phân biệt, liên hệ trực tiếp tới giới hạn "không suy rộng ngoài khung đã kiểm chứng" nêu ở mục 6.

8. Gợi ý áp dụng

Cross-validation hỗ trợ lựa chọn và đánh giá mô hình, nhưng cách chia phải phản ánh cấu trúc dữ liệu và mục tiêu sử dụng. Cần giữ tập test độc lập hoặc dùng nested cross-validation khi muốn đánh giá quy trình chọn siêu tham số.

9. Thí nghiệm gợi ý

1. **Tái tạo bias/variance của k-fold theo k (Mục 5, quy mô nhỏ, CPU đủ):** trên một bài toán hồi quy tuyến tính toy (sinh dữ liệu giả với σ biết trước), đo bias và variance của ước lượng risk bằng k-fold với $k \in \{2, 5, 10, n\}$ (LOO), lặp lại nhiều lần Monte Carlo để ước lượng variance thực nghiệm của chính ước lượng CV. Kỳ vọng: bias giảm theo k (theo mục 4.1); so sánh xu hướng variance với mô tả định tính ở mục 4.2 (có thể không đơn điệu, tuỳ cấu hình).
2. **Minh hoạ "cross-validation paradox" khi so sánh 2 mô hình gần nhau:** sinh hai mô hình hồi quy có risk thật gần bằng nhau (hiệu nhỏ, biết trước), dùng hold-out với n_v/n_t khác nhau (ví dụ 10/90, 50/50, 90/10) để chọn mô hình tốt hơn nhiều lần Monte Carlo, đo tỉ lệ chọn đúng. Kỳ vọng theo mục 4.4: tỉ lệ chọn đúng tăng khi n_v lớn hơn (ngược với khuyến nghị cho bài toán estimation thuần).
3. **Minh hoạ "double cross" / optimism khi tune hyperparameter bằng CV (Mục 5.1):** trên ridge regression toy, chọn λ bằng k-fold CV, so sánh điểm CV tại $\hat{\lambda}$ đã chọn với risk thật trên một tập test độc lập lớn (biết trước phân phối). Đo khoảng cách giữa hai giá trị này qua nhiều lần lặp; kỳ vọng: điểm CV tại $\hat{\lambda}$ lạc quan hơn (thấp hơn) risk thật một lượng hệ thống, minh hoạ trực tiếp nguy cơ đánh giá lạc quan khi chọn siêu tham số.

4. **So sánh chi phí LOO closed-form vs brute-force (Mục 9):** với hồi quy tuyến tính (least-squares), cài công thức LOO dạng đóng (dùng hat matrix, $\hat{L}^{LOO} = \frac{1}{n}\sum_i \left(\frac{y_i - \hat{y}_i}{1-h_{ii}}\right)^2$, một dạng công thức quen biết liên quan tới GCV) và so thời gian chạy với LOO brute-force (huấn luyện lại n lần) trên n tăng dần; đo thời điểm brute-force trở nên chậm hơn hẳn, minh hoạ động lực cho các "closed-form formulas" mục 9 của paper.
5. **Kiểm tra tính không nhất quán của LOO cho variable selection (Mục 4.4/7.1, quy mô nhỏ):** sinh hồi quy tuyến tính với một số biến thật và một số biến dư (nhiễu, không liên quan), dùng LOO để chọn tập biến tốt nhất khi n tăng, đo xác suất **chọn thừa biến dư** qua nhiều lần lặp. Kỳ vọng theo Shao (1993): xác suất này **không tiến về 0** khi n_t/n giữ cố định tỉ lệ dương, minh hoạ trực tiếp việc LOO không nhất quán cho identification dù nó tốt cho estimation.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý chính	Nguồn (mục trong paper)
Loss, excess loss	$L(t) = \mathbb{E}_\xi[\gamma(t;\xi)]$; $\ell(s,t) = L(t) - L(s) \geq 0$	Mục 1.1, công thức (1)
Bias-variance trade-off	$\mathbb{E}[\ell(s,\hat{s}_m)] \approx \ell(s,S_m) + \lambda D_m$	Mục 2.1, công thức (2)
Oracle, oracle inequality	$m^\star = \arg\min_m\{\ell(s,\hat{s}_m)\}$; $\ell(s,\hat{s}_{\hat{m}}) \leq C_n \inf_m \ell(s,\hat{s}_m) + R_n$	Mục 2.2, công thức (3)-(4)
Hold-out	chia 1 lần, $\hat{L}^{H-O}$	Mục 4.2.1, công thức (8)
CV tổng quát	trung bình B lần hold-out	Mục 4.2.2, công thức (9)
LOO	$n_t = n-1$, $B = n$	Mục 4.3.1, công thức (10)
LPO	$n_t = n-p$, $B = \binom{n}{p}$ (exhaustive)	Mục 4.3.1
VFCV	$n_t = n(1-1/V)$, $B = V$	Mục 4.3.2, công thức (11)
Bias của CV	$\mathbb{E}[\hat{L}^{CV}] = \mathbb{E}[L_P(A(D_{n_t}))]$, thường không âm, giảm theo n_t	Mục 5.1, công thức (12)
Variance của hold-out	$\mathbb{E}[\text{var}(L_{P^{(v)}} \mid D_n^{(t)})] + \text{var}[L_P(A(D_{n_t}))]$	Mục 5.2.1, công thức (13)
Shao (1997), estimation	$n_t \sim n$: CV $\approx C_p$ (hiệu quả); $n_t \sim \lambda n$: CV $\approx$ GIC$_\kappa$, $\kappa = 1 + \lambda^{-1}$	Mục 6.2
Shao (1993), identification	LOO/LPO/BICV không nhất quán nếu $\liminf(n_t/n) > 0$; cần $n \gg n_t \to \infty$	Mục 7.1
Cross-validation paradox	cần $n_v \gg n_t$ cho identification, ngược với estimation	Mục 7.1-7.2
"Double cross"	CV lồng nhau để tránh đánh giá lạc quan khi đã dùng CV chọn siêu tham số	Mục 5.1.1
Quy tắc thực hành V	thường 5-10 cho estimation; $V = 2$ khi so sánh 2 thuật toán	Mục 10.3

11. Kết luận ngắn

Survey tổng hợp có hệ thống các kết quả (chủ yếu lý thuyết, một phần thực nghiệm mô phỏng từ các bài báo khác) về cross-validation, với đóng góp tổ chức quan trọng nhất là tách bạch hai mục tiêu **estimation** (tối thiểu hoá loss, đo bằng efficiency/oracle inequality) và **identification** (tìm đúng mô hình thật, đo bằng consistency), hai mục tiêu này đòi hỏi cách chọn kích thước tập huấn luyện n_t **ngược hướng nhau** (n_t gần n tốt cho estimation, n_t nhỏ hơn hẳn n tốt cho identification), tạo ra "cross-validation paradox". Paper hệ thống hoá các thủ tục CV cổ điển (hold-out, LOO, LPO, VFCV, BICV, RLT, MCCV cùng các biến thể GCV, bootstrap), phân tích bias và variance của chúng theo nhiều khung thống kê, và kết luận **không có một thủ tục CV tối ưu phổ quát**: lựa chọn phải cân nhắc bias, variance, chi phí tính toán theo đặc điểm bài toán cụ thể (SNR, mục tiêu estimation hay identification, số mô hình ứng viên, dữ liệu độc lập hay phụ thuộc). Một

cảnh báo thực hành bền vững theo thời gian là nguy cơ đánh giá lạc quan (“double cross”) khi tái sử dụng chính điểm CV đã dùng để chọn siêu tham số làm thước đo hiệu năng cuối cùng.

Nguồn gốc: https://arxiv.org/abs/0907.4728

Bài 35

A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification (2021)

Tác giả: Anastasios N. Angelopoulos, Stephen Bates.
Venue / arXiv: arXiv:2107.07511 (v6, 07/12/2022; nộp lần đầu 07/2021); paper không nêu venue hội nghị/journal cụ thể trên trang đầu (là một bản "gentle introduction" dạng tutorial/survey tự xuất bản, dài như một cuốn sách nhỏ, có kèm code Python và Jupyter notebook).
Nguồn: https://arxiv.org/abs/2107.07511
Chủ đề: giới thiệu conformal prediction, một phương pháp tạo prediction set/interval có bảo đảm xác suất bao phủ (coverage) đúng theo nghĩa phân phối-tự do (distribution-free), áp được lên bất kỳ mô hình đã huấn luyện (black-box) mà không cần giả định gì về mô hình hay phân phối dữ liệu; đồng thời khảo sát các mở rộng (risk control, outlier detection, covariate shift, time-series) và lịch sử phát triển của lĩnh vực.
Vị trí trong lộ trình: Đọc cùng các bài 36–37 để đối chiếu conformal prediction với ensemble và MC dropout.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận. Ghi chú trình bày thân bài và các phụ lục kỹ thuật, không liệt kê toàn bộ danh mục tài liệu tham khảo.

1. Bối cảnh

Các mô hình học máy "hộp đen" (black-box, ví dụ neural network) ngày càng được dùng trong các bối cảnh rủi ro cao (chẩn đoán y tế, lái xe tự hành, tư pháp), nơi cần biết **mức độ chắc chắn** của một dự đoán, không chỉ giá trị dự đoán điểm. Các công cụ heuristic có sẵn (softmax output, độ lệch chuẩn ước lượng từ một mạng Gaussian, variance của ensemble, variance của MC dropout...) **không có bảo đảm gì**: chúng có thể bị overfit hoặc sai lệch tuỳ ý, và không tự động phản ánh đúng xác suất thật.

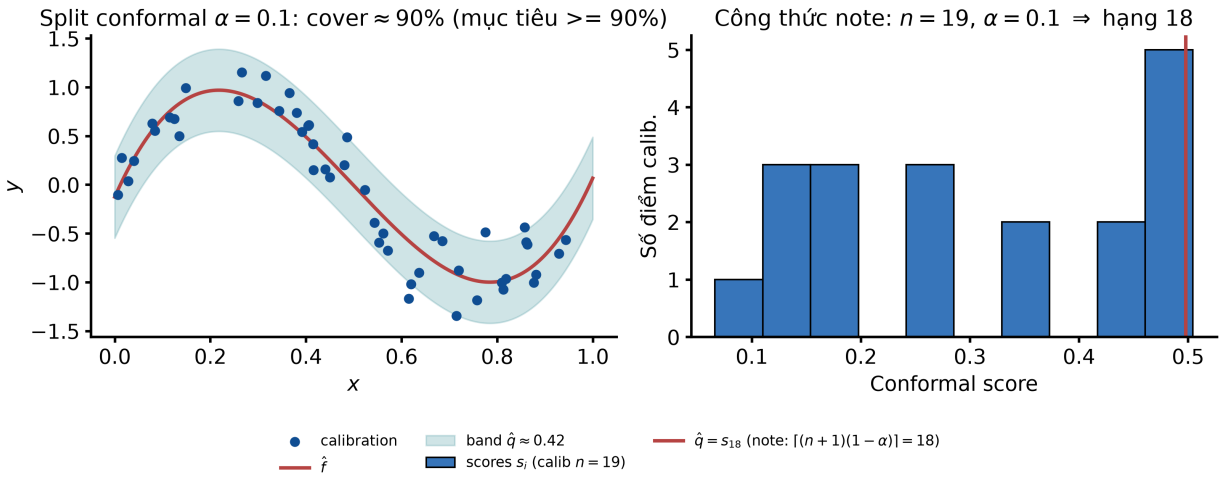
Conformal prediction (hay conformal inference, paper trích dẫn các công trình [1-3] là nền tảng) là một kỹ thuật biến bất kỳ ước lượng uncertainty heuristic nào thành một **prediction set** có bảo đảm xác suất hình thức, **không cần giả định về mô hình hay phân phối dữ liệu** (distribution-free), chỉ cần một giả định tối thiểu: dữ liệu calibration và dữ liệu test là **exchangeable** (một giả định tổng quát hơn i.i.d. cần được xem xét trong từng ứng dụng). Bảo đảm này đúng với **cỡ mẫu hữu hạn** (non-asymptotic), không cần $n \to \infty$.

Khái niệm cần nắm trước:

- **Exchangeable**: một dãy biến ngẫu nhiên có phân phối không đổi khi hoán vị thứ tự (yếu hơn i.i.d. nhưng đủ cho các chứng minh trong bài).
- **Coverage (độ bao phủ)**: xác suất prediction set chứa đúng giá trị thật.
- **Score function / nonconformity score** $s(x, y)$: một hàm đo "mức độ không khớp" giữa x và y, giá trị lớn nghĩa là (x, y) càng "lạc quẻ" so với dữ liệu calibration.

2. Ý tưởng chính

Ý tưởng cốt lõi: xem conformal prediction như một **bộ chuyển đổi tổng quát**, nhận đầu vào là "heuristic uncertainty" của bất kỳ mô hình đã huấn luyện sẵn, và trả về "rigorous uncertainty" (uncertainty có bảo đảm xác suất hình thức):



Split conformal band đạt coverage gần $1 - \alpha$; công thức $\lceil (n + 1)(1 - \alpha) \rceil$ với $n = 19$, $\alpha = 0.1$ (minh hoạ, tự tính)

$$\text{Heuristic uncertainty (per input)} \xrightarrow{\text{conformal prediction}} \text{Rigorous uncertainty (per input)}$$

Cách làm (minh hoạ bằng ví dụ classification ảnh, K lớp, mô hình $\hat{f}(x) \in [0, 1]^K$ là softmax output): dành ra n điểm dữ liệu "calibration" chưa dùng để huấn luyện ($\approx$ 500-1000 điểm là đủ, xem Mục 5). Với mỗi điểm calibration, tính **conformal score** $s_i = 1 - \hat{f}(X_i)_{Y_i}$ (1 trừ softmax của đúng lớp; lớn khi mô hình sai nhiều). Tính $\hat{q}$ là phân vị $\lceil (n + 1)(1 - \alpha) \rceil / n$ của các s_i (phân vị $1 - \alpha$ có điều chỉnh nhỏ). Với điểm test mới, trả về tập dự đoán $C(X_{test}) = \{y : \hat{f}(X_{test})_y \geq 1 - \hat{q}\}$ (mọi lớp có softmax đủ cao). Kết quả **bảo đảm** (công thức 1):

$$1 - \alpha \;\leq\; P(Y_{test} \in C(X_{test})) \;\leq\; 1 - \alpha + \frac{1}{n+1} \tag{1}$$

bất kể mô hình $\hat{f}$ tốt hay dở, bất kể phân phối dữ liệu là gì. Đây gọi là **marginal coverage** (bao phủ biên, lấy trung bình trên toàn bộ ngẫu nhiên của calibration và test). Set lớn hơn khi mô hình bất định hoặc input khó, nhỏ hơn khi dễ, nên kích thước của $C(x)$ tự nhiên phản ánh độ chắc chắn của mô hình tại điểm đó. Điều đáng ngạc nhiên nhất: bảo đảm này **vẫn đúng** dù $\hat{f}$ tệ đến đâu (ví dụ score hoàn toàn ngẫu nhiên), lúc đó set chỉ trở nên **vô dụng** (to, không chọn lọc) nhưng vẫn **đúng về coverage**. Vì vậy, chất lượng thực dụng của conformal prediction phụ thuộc hoàn toàn vào việc chọn **hàm điểm (score function)** tốt, không phụ thuộc vào bảo đảm (bảo đảm luôn đúng).

3. Phương pháp chi tiết

3.1 Thuật toán split conformal tổng quát và định lý bao phủ

Với input x và output y tổng quát (không cần rời rạc), quy trình 4 bước:

1. Xác định một notion heuristic về uncertainty từ mô hình đã huấn luyện sẵn.
2. Định nghĩa hàm điểm $s(x, y) \in \mathbb{R}$ (điểm lớn hơn = x, y khớp nhau kém hơn).
3. Tính $\hat{q}$ là phân vị $\lceil (n + 1)(1 - \alpha) \rceil / n$ của các điểm calibration $s_1, \ldots, s_n$.
4. Dùng $\hat{q}$ để tạo tập dự đoán cho điểm mới: $C(X_{test}) = \{y : s(X_{test}, y) \leq \hat{q}\}$ (công thức 2).

Định lý 1 (Vovk, Gammerman, Saunders). Nếu $(X_i, Y_i)_{i=1}^n$ và (X_{test}, Y_{test}) i.i.d. (hoặc exchangeable), $\hat{q}, C$ định nghĩa như trên, thì $P(Y_{test} \in C(X_{test})) \geq 1 - \alpha$.

Phác thảo chứng minh (Phụ lục D, paper trình bày đầy đủ, không chỉ trích dẫn): giả sử các điểm $s_1, \dots, s_n$ khác nhau từng đôi (bỏ qua ties), sắp tăng dần $s_1 < \cdots < s_n$. Khi đó $\hat{q} = s_{\lceil (n+1)(1-\alpha) \rceil}$ (nếu $\alpha \geq 1/(n+1)$; ngược lại $\hat{q} = \infty$ và set luôn chứa toàn bộ $\mathcal{Y}$, coverage hiển nhiên đúng). Sự kiện bao phủ $\{Y_{test} \in C(X_{test})\}$ tương đương $\{s_{test} \leq \hat{q}\} = \{s_{test} \leq s_{\lceil (n+1)(1-\alpha) \rceil}\}$. Do exchangeability của $n+1$ điểm $(X_1, Y_1), \dots, (X_{test}, Y_{test})$, với mọi số nguyên k: $P(s_{test} \leq s_k) = k/(n+1)$ (vì s_{test} có hạng đồng khả năng trong $n+1$ giá trị). Suy ra $P(s_{test} \leq s_{\lceil (n+1)(1-\alpha) \rceil}) = \lceil (n+1)(1-\alpha) \rceil / (n+1) \geq 1-\alpha$. Chặn trên $1-\alpha+1/(n+1)$ (Định lý D.2) chỉ đúng khi phân phối của score **liên tục** (không có ties); paper dẫn chứng minh này sang một tài liệu khác ([83]), không tự chứng minh trong văn bản chính.

Ví dụ tính tay minh hoạ chính cơ chế chứng minh (tự tính): lấy $n = 19$ điểm calibration, $\alpha = 0{,}1$. $\lceil (n+1)(1-\alpha) \rceil = \lceil 20 \times 0{,}9 \rceil = \lceil 18 \rceil = 18$, vậy $\hat{q} = s_{18}$ (giá trị lớn thứ nhì trong 19 điểm calibration, vì s_{19} là giá trị lớn nhất bị loại). Trong $n+1 = 20$ điểm exchangeable (19 calib + 1 test), hạng của s_{test} đều như nhau trên $\{1, \dots, 20\}$; sự kiện "không bị phủ" ($s_{test} > s_{18}$) xảy ra khi và chỉ khi s_{test} thuộc 2 giá trị lớn nhất trong 20 (gồm s_{19} và giá trị lớn hơn giữa s_{test}, s_{19}... nói đơn giản: s_{test} nằm trong top-2 của 20 giá trị), xác suất $= 2/20 = 0{,}1 = \alpha$ **chính xác**, khớp hoàn toàn với công thức $1 - 18/20 = 0{,}1$. Đây là lý do con số $\lceil (n+1)(1-\alpha) \rceil$ "có vẻ kỳ lạ" (không phải đơn giản $\lceil n(1-\alpha) \rceil$) lại cho coverage đúng khớp.

3.2 Các hàm điểm cụ thể (ví dụ Mục 2 của paper)

Bài toán	Hàm điểm $s(x, y)$	Tập dự đoán $C(x)$
Classification cơ bản	$1 - \hat{f}(x)_y$	$\{y : \hat{f}(x)_y \geq 1 - \hat{q}\}$
Adaptive Prediction Sets (APS, công thức 3)	$\sum_{j=1}^{k} \hat{f}(x)_{\pi_j(x)}$ với $y = \pi_k(x)$ (tổng softmax các lớp xếp hạng cao hơn hoặc bằng y)	gộp lớp theo thứ tự xác suất giảm dần tới khi tổng vượt $\hat{q}$
Conformalized Quantile Regression (CQR, công thức 4)	$\max(\hat{t}_{\alpha/2}(x) - y,\ y - \hat{t}_{1-\alpha/2}(x))$	$[\hat{t}_{\alpha/2}(x) - \hat{q},\ \hat{t}_{1-\alpha/2}(x) + \hat{q}]$
Uncertainty scalar (ví dụ $\hat{\sigma}(x)$, công thức 5)	$\dfrac{y - \hat{f}(x)}{u(x)}$	$[\hat{f}(x) - u(x)\hat{q},\ \hat{f}(x) + u(x)\hat{q}]$
Conformalized Bayes (công thức 6)	$-\hat{f}(y \mid x)$ (âm posterior predictive density)	$\{y : \hat{f}(y \mid x) > -\hat{q}\}$ (superlevel set)

APS (Mục 2.1) sửa lỗi của score cơ bản là **undercover ở nhóm khó, overcover ở nhóm dễ** (vì score cơ bản chỉ nhìn softmax của đúng lớp, bỏ qua hình dạng toàn bộ phân phối). CQR (Mục 2.2) dùng hai mô hình quantile regression ($\hat{t}_{\alpha/2}, \hat{t}_{1-\alpha/2}$, huấn luyện bằng pinball loss $L_\gamma(\hat{t}_\gamma, y) = (y - \hat{t}_\gamma)\gamma \mathbb{1}[y > \hat{t}_\gamma] + (\hat{t}_\gamma - y)(1-\gamma)\mathbb{1}[y \leq \hat{t}_\gamma]$), rồi nới/co khoảng theo $\hat{q}$ để đạt coverage đúng; paper khuyến nghị CQR khi có thể, vì quantile regression đã có coverage xấp xỉ đúng ngay cả không conformalize, và có "conditional coverage xấp xỉ đúng tiệm cận" (asymptotically valid). Uncertainty scalar (Mục 2.3) là cách phổ biến hơn nhưng kém chặt hơn CQR về lý thuyết (không có lý do $\hat{\sigma}$ scale đúng theo α). Conformalized Bayes (Mục 2.4) chứng minh được **Bayes-optimal** (kích thước trung bình nhỏ nhất trong số mọi tập conformal đạt coverage $1-\alpha$) nếu mô hình Bayesian và giả định kỹ thuật đúng (dẫn [11]), gợi nhớ Neyman-Pearson lemma, nhưng chứng minh đầy đủ không có trong văn bản chính.

3.3 Mở rộng của conformal prediction (Mục 4 của paper)

Group-balanced CP (công thức 8): calibrate riêng từng nhóm $g \in \{1, \dots, G\}$ (ví dụ nhóm chủng tộc), bảo đảm $P(Y_{test} \in C(X_{test}) \mid X_{test,1} = g) \geq 1 - \alpha$ cho mọi nhóm, không chỉ trung bình toàn bộ (Proposition 1, không chứng minh chi tiết).

Class-conditional CP (công thức 9): tương tự nhưng theo **lớp đầu ra** y (ví dụ đảm bảo coverage riêng cho lớp "ung thư" và lớp "bình thường"), calibrate $\hat{q}^{(k)}$ riêng từng lớp, $C(x) = \{y : s(x, y) \leq \hat{q}^{(y)}\}$ (Proposition

2).

Conformal risk control (Mục 4.3, công thức 11-12, Định lý 2): tổng quát hoá từ bảo đảm miscoverage $P(Y \notin C(X)) \leq \alpha$ sang bảo đảm $\mathbb{E}[\ell(C(X_{test}), Y_{test})] \leq \alpha$ với **bất kỳ hàm loss ℓ bị chặn và đơn điệu giảm theo một tham số ngưỡng** λ (ví dụ false negative rate trong multilabel classification). Chọn $\hat{\lambda} = \inf\{\lambda : \hat{R}(\lambda) \leq \alpha - (B - \alpha)/n\}$ với $\hat{R}(\lambda)$ là risk thực nghiệm trên dữ liệu calibration, B là chặn trên của loss. Ví dụ paper cho: $B = 1, \alpha = 0{,}1, n = 1000$ thì $\hat{\lambda}$ được chọn ở mức risk thực nghiệm $0{,}0991$ (hơi nghiêm hơn $0{,}1$) chứ không phải đúng $0{,}1$, để bù trừ sai số hữu hạn mẫu.

Learn-Then-Test (LTT, Phụ lục A): tổng quát hoá hơn nữa, kiểm soát **bất kỳ risk nào** (không cần đơn điệu), kể cả nhiều risk đồng thời. Với mỗi λ trong lưới Λ (thường rời rạc hoá $N \approx 1000$ điểm), gán giả thuyết không $H_\lambda : R(\lambda) > \alpha$; tính p-value cho mỗi H_λ bằng một concentration inequality (Hoeffding: $p_\lambda = e^{-2n(\alpha - \hat{R}(\lambda))_+^2}$, hoặc Hoeffding-Bentkus mạnh hơn); rồi kết hợp các p-value bằng một thuật toán kiểm soát **familywise error rate (FWER)** (Bonferroni: $\hat{\Lambda} = \{\lambda : p_\lambda < \delta/|\Lambda|\}$, hoặc fixed-sequence testing mạnh hơn khi risk gần-đơn điệu). Định lý A.1: chọn $\hat{\lambda}$ bất kỳ trong $\hat{\Lambda}$ cho ta (α, δ)-risk-controlling prediction: $P(R(\hat{\lambda}) \leq \alpha) \geq 1 - \delta$ (bảo đảm xác suất cao, khác với bảo đảm kỳ vọng ở conformal risk control).

Outlier detection (Mục 4.4, công thức 14): chỉ cần dữ liệu "sạch" $X_1, \ldots, X_n$ (không nhãn), tính score $s(X_i)$, $\hat{q}$ là phân vị thường, test point là outlier nếu $s(x) > \hat{q}$; bảo đảm $P(C(X_{test}) = \text{outlier}) \leq \alpha$ trên dữ liệu sạch (Proposition 3). Có thể diễn giải như kiểm định giả thuyết, điểm bị gắn outlier là điểm có p-value $< \alpha$ cho giả thuyết "exchangeable với calibration data" (liên quan permutation test).

Covariate shift (Mục 4.5, Định lý 3): khi $X_{test} \sim P_{test} \neq P$ (phân phối X đổi) nhưng $Y \mid X$ không đổi, và biết **tỉ số khả năng** (likelihood ratio) $w(x) = dP_{test}(x)/dP(x)$, dùng **weighted conformal prediction**: tính lại $\hat{q}(x)$ là phân vị có trọng số (trọng số lớn hơn cho điểm calibration có $w(X_i)$ lớn, tức "giống" phân phối test hơn). Vẫn giữ bảo đảm $P(Y_{test} \in C(X_{test})) \geq 1 - \alpha$ (Định lý 3), nhưng đòi hỏi biết w chính xác, một điều kiện hạn chế trong thực tế.

Distribution drift (Mục 4.6, Định lý 4): dữ liệu lệch dần theo thời gian (time-series), dùng trọng số w_i (ví dụ cửa sổ trượt K điểm gần nhất, hoặc decay $0{,}99^{n-i+1}$). Bảo đảm chỉ còn xấp xỉ: $P(Y_{test} \in C(X_{test})) \geq 1 - \alpha - 2\sum_i \tilde{w}_i \epsilon_i$, với ϵ_i là khoảng cách total-variation giữa điểm i và điểm test; nếu không có drift ($\epsilon_i = 0$) thì không mất coverage dù chọn trọng số thế nào. Có một đại lượng quan trọng: **effective sample size** $n_{eff} = (\sum w_i)/(\sum w_i^2)$; phương sai của coverage có trọng số tăng theo $1/\sqrt{n_{eff}}$, nên trọng số quá lệch (một vài điểm áp đảo) làm n_{eff} nhỏ dù n lớn.

Ví dụ tính tay minh hoạ n_{eff} (tự tính, số liệu minh hoạ): giả sử 9 điểm calibration, một điểm có trọng số $w_1 = 1$ (gần test nhất), 8 điểm còn lại có trọng số rất nhỏ $w_2 = \cdots = w_9 = 0{,}01$ (xa test, gần như bị bỏ qua). $\sum w_i = 1 + 8 \times 0{,}01 = 1{,}08$. $\sum w_i^2 = 1 + 8 \times 0{,}0001 = 1{,}0008$. $n_{eff} = 1{,}08/1{,}0008 \approx 1{,}08$. Dù có 9 điểm calibration "trên danh nghĩa", $n_{eff} \approx 1$, tức về mặt thống kê gần như chỉ có **một** điểm calibration thật sự đóng góp; theo quy tắc $1/\sqrt{n_{eff}}$, phương sai coverage ở đây sẽ rất lớn, một cảnh báo cụ thể cho việc chọn trọng số quá "nhọn" khi dữ liệu trôi nhanh.

3.4 Full conformal prediction và các biến thể trung gian (Mục 6)

Split conformal (toàn bộ nội dung trên) chỉ cần huấn luyện mô hình **một lần**, nhưng "hy sinh" một phần dữ liệu cho calibration (giảm hiệu quả thống kê). **Full conformal prediction** (lịch sử phát triển trước, split conformal được nhận ra là trường hợp đặc biệt sau) không chia dữ liệu: với mỗi giá trị ứng viên $y \in \mathcal{Y}$, huấn luyện lại mô hình $\hat{f}^y$ trên dữ liệu augmented $(X_1, Y_1), \ldots, (X_{n+1}, y)$ (mô hình phải **invariant với hoán vị** dữ liệu), tính score, rồi giữ y vào tập dự đoán nếu $s_{n+1}^y \leq \hat{q}^y$ (công thức 16, Định lý 5, bảo đảm giống Định lý 1). Chi phí: $(n+1) \cdot K$ lần huấn luyện lại nếu $|\mathcal{Y}| = K$, rất tốn; một số trường hợp đặc biệt có công thức closed-form tránh vòng lặp này. **Cross-conformal, CV+, Jackknife+** (Mục 6.2, chỉ dẫn chiếu [41, 42], không trình bày chi tiết thuật toán) là các điểm trung gian: dùng ít lần huấn luyện lại hơn full conformal nhưng vẫn dùng toàn bộ dữ liệu cho cả fit và calibrate, đổi lại hiệu quả thống kê cao hơn split conformal.

4. Thí nghiệm và kết quả chính

Đây là một tutorial nên "thí nghiệm" chủ yếu là các **ví dụ áp dụng minh hoạ (worked examples, Mục 5)** trên dữ liệu thật, kèm code, không phải benchmark cạnh tranh giữa nhiều phương pháp.

4.1 Multilabel classification với FDR control (Mục 5.1, Hình 13). Dataset MS-COCO (ảnh đa nhãn), dùng conformal risk control để chọn ngưỡng $\hat{\lambda}$ kiểm soát false negative rate (tỉ lệ nhãn đúng bị bỏ sót) ở $\alpha = 0{,}1$: bảo đảm trung bình $\mathbb{E}[\text{FNR}] \leq 0{,}1$, tức tập dự đoán giữ được trung bình $\geq 90\%$ nhãn đúng của mỗi ảnh.

4.2 Tumor segmentation (Mục 5.2, Hình 14). Dataset ảnh polyp đường ruột (gut polyps); mô hình phân đoạn xuất xác suất mỗi pixel là tumor; dùng risk control với loss là tỉ lệ pixel u bướu bị bỏ sót (FNR), $\alpha = 0{,}1$, cùng cơ chế với ví dụ 4.1 nhưng ở cấp độ pixel.

4.3 Weather prediction với distribution shift theo thời gian (Mục 5.3, Hình 15). Dataset Yandex Weather Prediction (một phần của Shifts Project), dùng ensemble 10 mô hình CatBoost đã huấn luyện sẵn: $\hat{f}$ = trung bình dự đoán của 10 mô hình, $\hat{u}$ = trung bình variance nội tại của 10 mô hình. Nối dữ liệu in-distribution và out-of-distribution theo thời gian tạo một **change-point** rõ rệt. Kết quả: weighted conformal (cửa sổ K điểm gần nhất) giữ coverage ổn định quanh $0{,}9$ cả trước và sau change-point, và **hồi phục nhanh** sau change-point; baseline unweighted (dùng toàn bộ score từ đầu tới hiện tại, không trọng số) bị **undercover có hệ thống** trước change-point (quan sát rõ hơn khi dùng trailing average 5000 điểm). Paper tự nêu: vì không đo được chính xác khoảng cách TV giữa các điểm liên tiếp, **không có bảo đảm hình thức** cho trường hợp này (Định lý 4 không áp dụng được đầy đủ), chỉ là một heuristic "hoạt động tốt trên thực tế".

4.4 Phát hiện bình luận độc hại qua outlier detection (Mục 5.4, Hình 16). Dataset Jigsaw Multilingual Toxic Comment Classification (WILDS benchmark), mô hình Detoxify (BERT) cho điểm độc hại $\hat{f}(x) \in [0, 1]$. Dùng n bình luận **không độc hại** làm calibration, $\hat{q}$ là phân vị $\lceil (n+1)(1-\alpha) \rceil$ của các điểm đó. Kết quả cụ thể: **với type-1 error** $\alpha = 10\%$**, hệ thống gắn cờ đúng 70% tổng số bình luận độc hại thật** (và theo thiết kế, không gắn cờ sai quá 10% bình luận không độc hại).

4.5 Selective classification (Mục 5.5, Hình 17). ImageNet, $\alpha = 0{,}1$: hệ thống chỉ trả lời khi $\hat{P}(x) = \max_y \hat{f}(x)_y \geq \hat{\lambda}$ (ngưỡng chọn qua Learn-Then-Test vì **selective accuracy không đơn điệu theo ngưỡng**, phải dùng chặn trên Binomial-CDF và quét λ, Proposition 4), bảo đảm $P(Y_{test} = \hat{Y}(X_{test}) \mid \hat{P}(X_{test}) \geq \hat{\lambda}) \geq 1 - \alpha$ với xác suất cao $1 - \delta$ qua calibration data. Kết quả định tính (Hình 17): accuracy trên các điểm "giữ lại" (không abstain) tăng rõ khi fraction giữ lại giảm, đúng như kỳ vọng của một hệ thống biết khi nào nên "bỏ cuộc".

4.6 Cỡ calibration set cần thiết (Bảng 1, liên quan Mục 3.2). Với $\delta = 0{,}1, \alpha = 0{,}1$, số điểm calibration $n(\epsilon)$ cần để coverage nằm trong $1 - \alpha \pm \epsilon$ với xác suất $1 - \delta$:

ϵ	0,1	0,05	0,01	0,005	0,001
$n(\epsilon)$	22	102	2.491	9.812	244.390

Chỉ cần **22 điểm** để coverage dao động trong $\pm 0{,}1$, nhưng cần **244.390 điểm** để siết chặt tới $\pm 0{,}001$: chi phí tăng rất nhanh (tăng gần 100 lần số điểm khi ϵ giảm 10 lần), phù hợp hướng dẫn thực dụng của paper là "$n \approx 1000$ đủ cho hầu hết mục đích" (dao động coverage khoảng $[0{,}88, 0{,}92]$ với $\alpha = 0{,}1$, Hình 11).

5. Phân tích / ablation

Adaptivity (set size và conditional coverage, Mục 3.1). Một thủ tục conformal cho set size nhỏ nhất trung bình **không nhất thiết là tốt nhất**: nó có thể undercover nhóm khó và overcover nhóm dễ mà vẫn đạt đúng marginal coverage. Ví dụ minh hoạ bằng số cụ thể (Hình 10 của paper, hai nhóm tần suất 90%/10%): nếu mọi lỗi dồn vào nhóm nhỏ, ta có thể thấy **"100% coverage / 80% coverage"** (marginal trung bình vẫn $\approx 90\%$+

hợp lệ) hoặc thậm chí **"5% coverage / 14% coverage"** (nếu thủ tục tệ, marginal coverage cũng tệ luôn), trong khi **conditional coverage đúng nghĩa** phải cho **"90% coverage / 90% coverage"** ở cả hai nhóm. Conditional coverage (công thức 7) là một thuộc tính mạnh hơn nhiều so với marginal coverage mà conformal prediction đảm bảo, và **nói chung là bất khả thi** để đạt được chính xác với dữ liệu liên tục (dẫn [14]). Hai thước đo thực dụng để đánh giá gần đúng: **FSC (feature-stratified coverage)** = coverage thực nghiệm nhỏ nhất trên các nhóm đặc trưng rời rạc đã biết trước; **SSC (size-stratified coverage)** = coverage thực nghiệm nhỏ nhất trên các bin kích thước tập dự đoán (không cần biết trước đặc trưng nhóm, tổng quát hơn FSC). Giá trị gần $1-\alpha$ hơn nghĩa là gần conditional coverage hơn.

Ảnh hưởng của cỡ calibration set n (Mục 3.2). Phân phối coverage (với validation set vô hạn, calibration cố định) có công thức đóng: $P(Y_{test} \in C(X_{test}) \mid \text{calib data}) \sim \text{Beta}(n+1-l, l)$ với $l = \lfloor (n+1)\alpha \rfloor$. n nhỏ (ví dụ 100) cho phân phối rộng (coverage có thể lệch khá xa $1-\alpha$ với một bộ calibration cụ thể); n lớn (10.000) cho phân phối hẹp quanh $1-\alpha$. Tốc độ hội tụ $O(n^{-1/2})$.

Kiểm tra coverage đúng cách (Mục 3.3, Phụ lục C). Chạy R lần với calibration/validation ngẫu nhiên khác nhau (thường bằng cách cache toàn bộ score rồi shuffle, tránh chi phí tính lại score mỗi lần), histogram của coverage thực nghiệm C_j phải tâm quanh $1-\alpha$. Phụ lục C cho công thức đóng chính xác: C_j là biến **beta-binomial** (do n_{val} hữu hạn cộng với n hữu hạn), và trung bình/phương sai của $\bar{C}$ (trung bình trên R lần) là:

$$\mathbb{E}[\bar{C}] = 1 - \frac{l}{n+1}, \qquad \text{Var}(\bar{C}) = \sqrt{\frac{l(n+1-l)(n+n_{val}+1)}{n_{val}R(n+1)^2(n+2)}} = O\left(\frac{1}{\sqrt{R\min(n, n_{val})}}\right)$$

Nếu coverage thực nghiệm lệch khỏi $1-\alpha$ nhiều hơn mức công thức này cho phép, đó là dấu hiệu **lỗi cài đặt**, không phải biến động tự nhiên.

Trade-off giữa các score function (liên hệ Mục 2 và 3.1). Score cơ bản (chỉ nhìn softmax đúng lớp) tối thiểu hoá kích thước set trung bình nhưng kém adaptivity; APS hy sinh một phần kích thước set trung bình để cải thiện conditional coverage/adaptivity (paper dẫn [4], [7] cho các cải tiến thực dụng thêm, không trình bày chi tiết công thức trong văn bản chính). Đây là một đánh đổi thiết kế thực sự, không có "điểm số tối ưu tuyệt đối" cho mọi mục đích.

6. Giới hạn

Paper tự nêu:

- Conditional coverage **nói chung bất khả thi** để đạt chính xác (dẫn [14]) với dữ liệu liên tục; mọi thủ tục conformal chỉ đảm bảo marginal coverage, FSC/SSC chỉ là chẩn đoán thực nghiệm, không phải bảo đảm hình thức.
- Weighted conformal cho covariate shift (Mục 4.5) yêu cầu biết chính xác tỉ số khả năng $w(x) = dP_{test}/dP$, một "trường hợp khá hạn chế (somewhat restricted case) - đó là covariate shift đã biết trước", paper tự thừa nhận.
- Với distribution drift (Mục 4.6), ϵ_i (khoảng cách TV) "không bao giờ biết chính xác trước"; ví dụ thực nghiệm thời tiết (Mục 5.3) **không có bảo đảm hình thức** vì không đo được TV distance, chỉ dùng trọng số heuristic.
- Chặn trên của coverage ($1-\alpha+1/(n+1)$, công thức 1) chỉ đúng khi phân phối score liên tục (không ties); paper có một footnote (chú thích 1) nói rõ cần một "modification involving tie-breaking" để đạt chặn trên đầy đủ, và "sẽ bỏ qua tie-breaking này trong phần còn lại" của bài, tức bản thuật toán thực dụng trình bày xuyên suốt **không hoàn toàn đảm bảo chặn trên** trong trường hợp có ties (một giới hạn kỹ thuật được flag công khai, không phải lỗi ẩn).

- Full conformal prediction tốn $(n+1) \cdot K$ lần huấn luyện lại mô hình, "nói chung tốn kém về mặt tính toán" (generally computationally costly).
- Thủ tục FWER Bonferroni trong Learn-Then-Test là **bảo thủ** (conservative) khi $|\Lambda|$ lớn; các cải tiến mạnh hơn (sequential graphical testing) được dẫn sang tài liệu khác ([18], [132]), không trình bày chi tiết trong văn bản chính được đọc.

Người ghi chú nhận xét:

- Nhiều định lý quan trọng (Định lý 2, 3, 4, A.1) chỉ được **trích dẫn** từ các paper khác của chính nhóm tác giả hoặc đồng nghiệp (ví dụ [17], [25], [26], [18]), không có chứng minh đầy đủ trong văn bản chính; chỉ Định lý 1 (bảo đảm coverage cơ bản) được chứng minh trọn vẹn ở Phụ lục D của chính bài này. Người đọc muốn kiểm chứng chặt các mở rộng cần tìm thêm các paper gốc đó.
- Phần lịch sử (Mục 7) dựa nhiều vào trả lời phỏng vấn/hồi ức cá nhân của Vladimir Vovk do chính tác giả thu thập ("oral history related by Vovk"); đây là một nguồn giá trị nhưng mang tính kể chuyện một chiều, không phải tư liệu lưu trữ độc lập, nên các mốc thời gian/nhân vật có thể phản ánh góc nhìn của riêng Vovk hơn là một tổng kết khách quan toàn ngành.
- Các ví dụ thực nghiệm (Mục 5) đều là minh hoạ định tính (hình ảnh, một vài con số tổng hợp), không có bảng so sánh định lượng nhiều phương pháp như các paper benchmark thông thường; phù hợp với mục đích "gentle introduction" nhưng người đọc cần paper gốc được dẫn (ví dụ [4], [17], [18], [25], [26]) nếu cần số liệu đầy đủ để so sánh.

7. Liên hệ với các bài khác

- **ml basics 34** (Cross-Validation survey): cùng là công cụ thống kê distribution-free dùng sample-splitting (chia dữ liệu calibration/validation) để đánh giá/hiệu chỉnh mô hình mà không cần giả định phân phối, nhưng CV nhằm ước lượng risk/chọn mô hình, còn conformal prediction nhằm tạo prediction set có bảo đảm coverage cho từng điểm dữ liệu.
- **ml basics 36** (Deep Ensembles) và **ml basics 37** (MC Dropout): paper này **tự liệt kê trực tiếp** hai kỹ thuật này là ví dụ của "uncertainty scalar" $u(x)$ có thể conformalize (Mục 2.3.2, liệt kê "đo variance của $\hat{f}(x)$ qua một ensemble mô hình" và "đo variance của $\hat{f}(x)$ khi random dropout một phần node", đúng là Deep Ensembles và MC Dropout); nghĩa là conformal prediction có thể dùng **làm lớp calibrate phía trên** hai phương pháp kia để biến uncertainty heuristic của chúng thành bảo đảm coverage hình thức.
- **ml basics 19** (On Calibration of Modern Neural Networks): paper nhắc tới "calibrating a scalar notion of uncertainty... like a rigorous version of Platt scaling or isotonic regression" (Mục 7, phần Current Trends), cùng chủ đề calibration nhưng conformal prediction cho bảo đảm hình thức mạnh hơn (phân phối-tự do, hữu hạn mẫu) so với calibration truyền thống (thường chỉ đúng tiệm cận hoặc theo một mô hình tham số).

8. Gợi ý áp dụng

Conformal prediction tạo tập dự đoán với bảo đảm bao phủ dưới các giả định được nêu, đặc biệt là tính trao đổi. Cần phân biệt bao phủ biên với bao phủ theo từng nhóm và xem xét tác động của dịch chuyển phân phối.

9. Thí nghiệm gợi ý

1. **Tái tạo thuật toán classification cơ bản (Mục 1) trên MNIST/CIFAR-10 với CNN nhỏ:** $n = 500$ hoặc 1000 điểm calibration, $\alpha = 0{,}1$; đo coverage thực nghiệm trên nhiều lần random split (theo mẫu code Hình 12 của paper), kiểm tra phân phối coverage có khớp công thức Beta/beta-binomial (Phụ lục C) không.
2. **So sánh score cơ bản vs APS (Mục 2.1):** cùng một mô hình, đo kích thước set trung bình và SSC metric (Mục 3.1) theo bin kích thước set; kỳ vọng APS có SSC cao hơn (gần $1-\alpha$ hơn) ở các bin, dù

kích thước set trung bình có thể lớn hơn score cơ bản.

3. **Conformalized Quantile Regression trên dữ liệu tổng hợp 1D:** tạo $y = \sin(x) + \epsilon(x)$ với $\epsilon(x)$ có variance thay đổi theo x (heteroskedastic), huấn luyện mạng nhỏ với pinball loss cho hai quantile $0{,}05$ và $0{,}95$; so coverage của khoảng thô (chưa conformalize) với khoảng đã conformalize (công thức 4); kỳ vọng khoảng thô lệch khỏi 90% còn khoảng CQR bám sát 90%.
4. **Hiệu ứng cỡ calibration set:** lặp thí nghiệm 1 với $n \in \{20, 100, 1000\}$, vẽ histogram coverage qua $R = 200$ lần random split cho mỗi n; đo độ lệch chuẩn của coverage và so với công thức $O(1/\sqrt{R\min(n, n_{val})})$ ở Phụ lục C.
5. **Toy distribution drift:** mô phỏng chuỗi thời gian 1D hồi quy với một change-point giữa chuỗi (variance nhiễu đổi đột ngột), so coverage cuốn chiếu (rolling) của unweighted conformal vs weighted conformal (cửa sổ K điểm gần nhất), tái tạo định tính Hình 15 của paper; thí nghiệm này rất rẻ (không cần GPU, chỉ cần một mô hình hồi quy nhỏ).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Ghi chú
Bảo đảm coverage cơ bản	$1-\alpha \le P(Y_{test} \in C(X_{test})) \le 1-\alpha+\frac{1}{n+1}$	công thức (1), Định lý 1
Phân vị calibration	$\hat{q} = \text{Quantile}(s_1, \ldots, s_n;\ \lceil (n+1)(1-\alpha)\rceil/n)$	bước trung tâm của mọi thủ tục conformal
Tập dự đoán tổng quát	$C(x) = \{y : s(x,y) \le \hat{q}\}$	công thức (2)
APS	$s(x,y) = \sum_{j\le k} \hat{f}(x)_{\pi_j(x)},\ y = \pi_k(x)$	công thức (3)
CQR	$C(x) = [\hat{t}_{\alpha/2}(x) - \hat{q},\ \hat{t}_{1-\alpha/2}(x) + \hat{q}]$	công thức (4)
Risk control	$\hat{\lambda} = \inf\{\lambda : \hat{R}(\lambda) \le \alpha - (B-\alpha)/n\}$	công thức (12), Định lý 2
Effective sample size	$n_{eff} = \dfrac{\sum w_i}{\sum w_i^2}$	phương sai coverage $\sim 1/\sqrt{n_{eff}}$
Phân phối coverage	$P(\cdot \mid \text{calib}) \sim \text{Beta}(n+1-l,\ l)$, $l = \lfloor (n+1)\alpha \rfloor$	Hình 11
$n(\epsilon)$ với $\alpha = \delta = 0{,}1$	$22 \to 102 \to 2.491 \to 9.812 \to 244.390$ (cho $\epsilon = 0{,}1 \to 0{,}001$)	Bảng 1
Kết quả nổi bật	Toxic comment: 70% bình luận độc hại bị gắn cờ đúng ở $\alpha = 10\%$	Mục 5.4

11. Kết luận ngắn

Conformal prediction biến bất kỳ ước lượng uncertainty heuristic của một mô hình đã huấn luyện sẵn thành một prediction set có bảo đảm coverage hình thức, phân phối-tự do, đúng với cỡ mẫu hữu hạn, chỉ cần dữ liệu exchangeable. Bài "gentle introduction" này trình bày thuật toán split conformal cơ bản kèm chứng minh, nhiều hàm điểm thực dụng (APS, CQR, Bayes), các mở rộng quan trọng (group/class-balanced, risk control qua Learn-Then-Test, outlier detection, covariate shift, distribution drift), cách đánh giá và kiểm tra coverage đúng cách, cùng lịch sử phát triển của lĩnh vực từ Kolmogorov/Vovk tới các ứng dụng hiện đại (bầu cử, y tế, robot). Khác với "ml basics 36/37" (Deep Ensembles, MC Dropout, dựa trên giả định ensemble/Bayesian), điểm mạnh riêng của conformal prediction là bảo đảm bao phủ biên dưới giả định trao đổi, bất kể mô hình nền tốt hay tệ, đổi lại chất lượng thực dụng (kích thước set) phụ thuộc hoàn toàn vào việc chọn hàm điểm tốt, một lựa chọn thiết kế mà bản thân phương pháp không tự động giải quyết.

Nguồn gốc: https://arxiv.org/abs/2107.07511

Bài 36

Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles (2016)

Tác giả: Balaji Lakshminarayanan, Alexander Pritzel, Charles Blundell (DeepMind).
Venue / arXiv: arXiv:1612.01474 (v3, 04/11/2017; nộp lần đầu 12/2016 theo mã arXiv); paper tự nêu venue ở trang đầu: "31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA."
Nguồn: https://arxiv.org/abs/1612.01474
Chủ đề: một phương pháp non-Bayesian, đơn giản và dễ scale để ước lượng predictive uncertainty cho neural network, dùng ba thành phần: proper scoring rule làm loss, adversarial training để làm mượt phân phối dự đoán, và ensemble nhiều mạng độc lập; so sánh trực tiếp với MC-dropout.
Vị trí trong lộ trình: Đọc cùng bài 35 về conformal prediction và bài 37 về MC dropout để so sánh các cách lượng hóa độ bất định.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Neural network tiêu chuẩn là "hộp đen" dự đoán tốt nhưng **không biết mình không biết**: khi gặp đầu vào lạ hoặc mơ hồ, mạng vẫn thường đưa ra xác suất rất tự tin (overconfident), dù sai. Paper đặt vấn đề cần **lượng hoá predictive uncertainty** (độ bất định của dự đoán) theo hai tiêu chí thực dụng:

- **Calibration** (hiệu chỉnh, theo quan điểm frequentist): độ lệch giữa xác suất mạng tự báo và tần suất thực nghiệm dài hạn. Ví dụ nếu mạng nói "90% tự tin" cho một nhóm dự đoán, thì trong nhóm đó khoảng 90% phải đúng thật. Calibration là mối quan tâm **độc lập với accuracy**: một mạng có thể chính xác nhưng miscalibrated (hoặc ngược lại).
- **Tổng quát hoá sang out-of-distribution (OOD)**: khi dữ liệu test khác hẳn phân phối huấn luyện (domain shift), mạng tốt phải "biết mình không biết", tức báo uncertainty cao, không chỉ đoán bừa với độ tự tin cao.

Hướng tiếp cận chính trước đó là **Bayesian neural network**: đặt một phân phối tiên nghiệm (prior) lên trọng số, suy ra posterior sau khi thấy dữ liệu, rồi dùng posterior này để lượng hoá uncertainty. Vì suy luận Bayes chính xác không khả thi với NN, người ta dùng các xấp xỉ: Laplace approximation, MCMC, biến phân (variational, bao gồm MC-dropout của Gal & Ghahramani, "ml basics 37"), assumed density filtering, expectation propagation, SGLD/Hamiltonian MCMC. Nhược điểm chung: cần sửa đổi đáng kể pipeline huấn luyện, chạy chậm hơn NN thường, và chất lượng uncertainty phụ thuộc mạnh vào mức xấp xỉ và việc chọn prior "đúng" (prior sai có thể cho uncertainty vô lý).

Khái niệm cần nắm trước:

- **Proper scoring rule**: một hàm $S(p_\theta, (y, x))$ chấm điểm chất lượng của phân phối dự đoán $p_\theta(y \mid x)$ so với phân phối thật q; "proper" nghĩa là kỳ vọng điểm số $S(p_\theta, q) = \mathbb{E}_q[S(p_\theta, (y, x))]$ đạt cực đại duy nhất khi $p_\theta = q$. Tối thiểu hoá loss $L(\theta) = -S(p_\theta, q)$ do đó khuyến khích mạng học đúng cả phân phối, không chỉ đúng nhãn.
- **Ensemble**: huấn luyện nhiều mạng độc lập (thường từ khởi tạo khác nhau), rồi kết hợp dự đoán của chúng, khác với Bayesian model averaging (giả định mô hình đúng nằm trong không gian giả thuyết của prior và làm model *selection* mềm); ensemble làm model *combination*, hữu ích hơn khi mô hình thật nằm **ngoài** không gian giả thuyết.

2. Ý tưởng chính

Bài báo đề xuất một "công thức" (recipe) ba bước, hoàn toàn **non-Bayesian** nhưng vẫn cho uncertainty chất lượng cao, đơn giản hơn hẳn các phương pháp Bayesian xấp xỉ:

1. Huấn luyện mỗi mạng để output một **phân phối xác suất** (không chỉ một giá trị điểm), dùng **proper scoring rule** (log-likelihood / NLL cho classification và regression, hoặc Brier score) làm hàm loss, để khuyến khích calibration ngay từ cách huấn luyện.
2. (Tuỳ chọn) dùng **adversarial training** (theo kiểu FGSM của Goodfellow và đồng nghiệp) như một cách làm mượt phân phối dự đoán trong một vùng lân cận nhỏ quanh mỗi điểm huấn luyện, với chi phí tính toán rẻ hơn hẳn làm mượt theo mọi hướng.
3. Huấn luyện M **mạng độc lập** (song song, không chia sẻ tham số, không cần bagging) và kết hợp dự đoán như một **mixture model** đồng trọng số.

Kết quả chính: phương pháp này **vượt hoặc ngang** MC-dropout trên hầu hết tiêu chí (NLL, Brier score, accuracy, chất lượng OOD detection), dễ cài đặt (không cần sửa kiến trúc, không cần dropout), dễ song song hoá, và là nhóm đầu tiên (theo bài báo tự nhận) đánh giá predictive uncertainty ở quy mô ImageNet.

3. Phương pháp chi tiết

3.1 Proper scoring rule làm loss

Với phân loại K lớp, hàm **softmax cross-entropy** chuẩn chính là log-likelihood, và paper chỉ ra (qua bất đẳng thức Gibbs) đây **là** một proper scoring rule: $S(p_\theta, q) = \mathbb{E}_q[\log p_\theta(y \mid x)] \le S(q, q)$, đẳng thức khi $p_\theta = q$. Thú vị hơn, **Brier score** (tối thiểu hoá bình phương sai số giữa xác suất dự đoán và one-hot nhãn đúng) cũng là một proper scoring rule hợp lệ:

$$L(\theta) = K^{-1} \sum_{k=1}^{K} (\delta_{k=y} - p_\theta(y = k \mid x))^2$$

Với **regression**, mạng xuất ra hai giá trị ở lớp cuối: trung bình $\mu_\theta(x)$ và phương sai $\sigma^2_\theta(x) > 0$ (ép dương bằng softplus $\log(1 + \exp(\cdot))$, cộng thêm phương sai tối thiểu, ví dụ 10^{-6}, để ổn định số học). Coi giá trị quan sát là một mẫu Gaussian (heteroscedastic) với mean/variance đó, loss là negative log-likelihood:

$$-\log p_\theta(y_n \mid x_n) = \frac{\log \sigma^2_\theta(x)}{2} + \frac{(y - \mu_\theta(x))^2}{2\sigma^2_\theta(x)} + \text{const} \tag{1}$$

3.2 Adversarial training để làm mượt

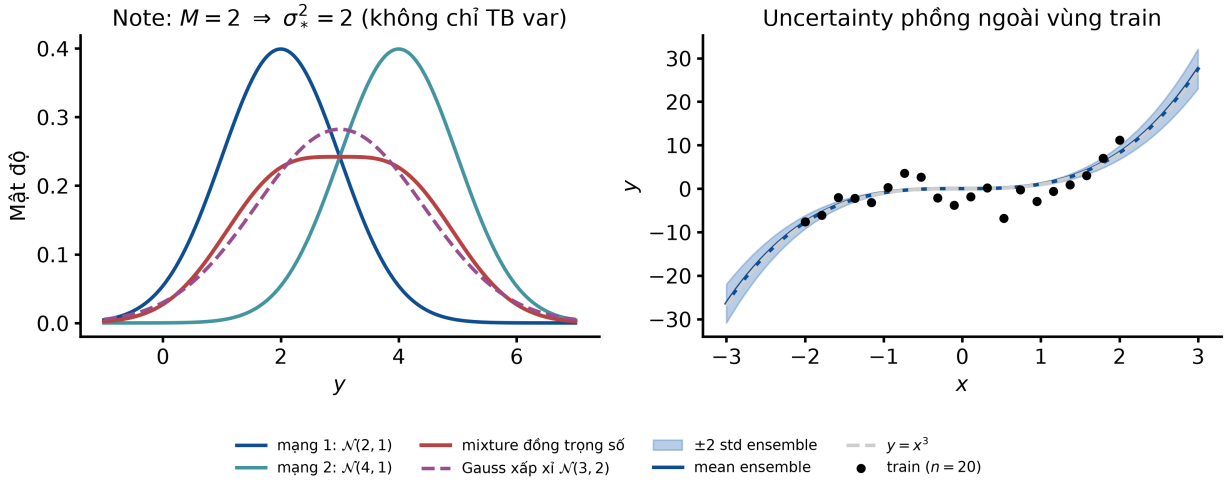
Lấy ý tưởng **fast gradient sign method (FGSM)** của Goodfellow và đồng nghiệp: với input x, nhãn y, loss $\ell(\theta, x, y)$ (ví dụ $-\log p_\theta(y \mid x)$), tạo ví dụ đối nghịch

$$x' = x + \epsilon \cdot \text{sign}(\nabla_x \ell(\theta, x, y))$$

với ϵ nhỏ để giới hạn max-norm của nhiễu. Paper diễn giải: thêm (x', y) làm ví dụ huấn luyện bổ sung tương đương **làm mượt phân phối dự đoán** theo hướng mà loss tăng nhanh nhất quanh mỗi điểm huấn luyện, trong một ϵ-lân cận, rẻ hơn hẳn việc làm mượt theo toàn bộ 2^D hướng $\{1, -1\}^D$ có thể. Khác với FGSM gốc (dùng ϵ cố định cho mọi chiều), paper đặt ϵ bằng **1% khoảng giá trị (range) của dữ liệu huấn luyện theo từng chiều** (ví dụ $\epsilon = 2{,}55$ nếu range đầu vào là $[0, 255]$), để công bằng giữa các chiều có thang đo khác nhau.

3.3 Huấn luyện và kết hợp ensemble

Paper chỉ dùng kiểu ensemble **randomization-based** (song song, không tương tác, khác boosting), vì phù hợp tính toán phân tán. Khác với random forest (thường cần bagging/bootstrap để tạo đa dạng), với NN tác giả quan sát **bagging làm giảm hiệu năng** trong thí nghiệm của họ (một bootstrap sample chỉ chứa trung bình $1-(1-1/N)^N \to 1-e^{-1} \approx 63{,}2\%$ số điểm gốc khi N lớn), nên chỉ cần **khởi tạo ngẫu nhiên tham số + xáo trộn ngẫu nhiên thứ tự dữ liệu**, huấn luyện trên **toàn bộ** tập train cho mỗi mạng, là đủ tạo đa dạng cần thiết.



Ensemble $M = 2$: $\sigma_*^2 = 2$ gồm bất đồng mạng; uncertainty phồng ngoài train trên $y = x^3$ (minh hoạ, tự tính)

Algorithm 1 (paper):

```
Mỗi mạng tham số hoá p_theta(y|x); loss l(theta,x,y) là một proper scoring rule.
Mặc định đề xuất: M = 5, epsilon = 1% range đầu vào mỗi chiều.
Khởi tạo theta_1,...,theta_M ngẫu nhiên.
for m = 1..M (song song, độc lập):
    lấy một (minibatch) điểm dữ liệu
    tạo ví dụ đối nghịch x' = x + epsilon * sign(grad_x l(theta_m, x, y))
    tối thiểu hoá l(theta_m, x, y) + l(theta_m, x', y)   # phần adversarial training là
    ↪ tuỳ chọn
```

Kết hợp dự đoán: coi ensemble là mixture đồng trọng số $p(y \mid x) = M^{-1} \sum_{m=1}^{M} p_{\theta_m}(y \mid x)$. Với classification, đây chính là **trung bình các xác suất softmax**. Với regression, đây là một **mixture of Gaussian**; để dễ tính quantile/xác suất dự đoán, paper xấp xỉ mixture này bằng **một Gaussian duy nhất** có mean và variance:

$$\mu_*(x) = M^{-1} \sum_m \mu_{\theta_m}(x), \qquad \sigma_*^2(x) = M^{-1} \sum_m \left(\sigma_{\theta_m}^2(x) + \mu_{\theta_m}^2(x)\right) - \mu_*^2(x)$$

Ví dụ tính tay (tự tính): giả sử $M = 2$ mạng, mạng 1 dự đoán $\mu_1 = 2$, $\sigma_1^2 = 1$, mạng 2 dự đoán $\mu_2 = 4$, $\sigma_2^2 = 1$ (hai mạng **bất đồng** về giá trị trung bình nhưng tự tin tương đương về phương sai riêng). Áp công thức trên: $\mu_* = (2+4)/2 = 3$. $\sigma_*^2 = \frac{1}{2}\left[(1+2^2) + (1+4^2)\right] - 3^2 = \frac{1}{2}(5+17) - 9 = 11 - 9 = 2$. Chú ý: trung bình đơn giản của hai phương sai riêng lẻ chỉ là 1, nhưng phương sai **tổng hợp** của ensemble là 2, lớn hơn hẳn, vì công thức tự động cộng thêm phần "bất đồng giữa các mạng" (giống phương sai giữa các thành phần của mixture) vào phương sai cuối. Đây chính là cách ensemble bắt được "model uncertainty": khi các mạng đồng ý, phương sai tổng hợp gần với trung bình phương sai riêng; khi các mạng bất đồng nhiều, phương sai tổng hợp tăng vọt, phản ánh đúng việc ta kém chắc chắn hơn.

3.4 Thước đo đánh giá

Classification: NLL (proper scoring rule, thước đo chính), accuracy, **Brier score** $BS = K^{-1}\sum_k (t_k^* - p(y = k \mid x^*))^2$ với $t_k^* = 1$ nếu $k = y^*$. Regression: NLL và RMSE. Thiết lập mặc định: batch size 100, Adam với learning rate cố định $0{,}1$ (trừ khi nói khác), ϵ cho adversarial training bằng $0{,}01\times$ range dữ liệu huấn luyện theo mỗi chiều, khởi tạo trọng số mặc định của Torch.

4. Thí nghiệm và kết quả chính

4.1 Regression toy 1D (Hình 1). Bài toán $y = x^3 + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 3^2)$, 20 điểm huấn luyện (lấy từ thiết lập của Hernández-Lobato & Adams). So 4 biến thể: (a) ensemble 5 mạng huấn luyện tối thiểu hoá MSE, dùng **phương sai thực nghiệm** giữa các mạng làm ước lượng uncertainty (cách làm phổ biến trong thực hành); (b) một mạng học cả mean và variance bằng NLL; (c) thêm adversarial training; (d) ensemble 5 mạng học mean/variance bằng NLL. Quan sát: (b) học variance bằng scoring rule tốt hơn hẳn (a) chỉ lấy phương sai thực nghiệm của ensemble-MSE; (d) kết hợp ensemble cải thiện thêm, đặc biệt ***ở xa vùng dữ liệu quan sát*** (uncertainty mở rộng hợp lý ra xa, không bó hẹp).

4.2 Regression trên 10 bộ dữ liệu UCI thật (Bảng 1, so PBP / MC-dropout / Deep Ensembles). Kiến trúc giống nhau: NN 1 lớp ẩn ReLU, 50 unit (100 cho Protein và Year Prediction MSD); 40 epoch; $M = 5$. 20 fold train-test (5 fold cho Protein, 1 split cho Year Prediction MSD vì quá lớn).

Dataset	RMSE (PBP / MC-drop / DeepEns)	NLL (PBP / MC-drop / DeepEns)
Boston housing	3,01 / 2,97 / 3,28	2,57 / 2,46 / 2,41
Energy	1,80 / 1,66 / 2,09	2,04 / 1,99 / **1,38**
Kin8nm	0,10 / 0,10 / 0,09	-0,90 / -0,95 / -1,20
Naval propulsion plant	0,01 / 0,01 / 0,00	-3,73 / -3,80 / **-5,63**
Protein	4,73 / 4,36 / 4,71	2,97 / 2,89 / 2,83
Year Prediction MSD	8,88 / 8,85 / 8,89	3,60 / 3,59 / 3,35

(Bảng đầy đủ có 10 dòng; đây là các dòng tiêu biểu.) Deep Ensembles **thắng hoặc ngang** về NLL trên hầu hết dataset, đặc biệt vượt rõ ở Energy và Naval propulsion plant; về RMSE đôi khi **kém hơn** nhẹ, vì phương pháp tối ưu cho NLL (bắt uncertainty) không phải MSE thuần.

4.3 Classification: MNIST, SVHN, ImageNet (Mục 3.4). MNIST: MLP 3 lớp ẩn, 200 unit/lớp, ReLU + batch norm; MC-dropout thêm dropout 0,1 sau mỗi non-linearity (0,5 thử nhưng kém hơn). Theo số mạng/số mẫu MC M tăng, cả ba thước đo (classification error, NLL, Brier score) cải thiện; **adversarial training tốt hơn** so với augment bằng hướng ngẫu nhiên; **ensemble vượt rõ MC-dropout** ở mọi M. SVHN (VGG-style convnet): ensemble vẫn vượt MC-dropout; adversarial training chỉ giúp rõ khi $M = 1$, hiệu ứng giảm khi M tăng.

ImageNet (Inception network, chỉ test ensemble vì hạn chế tính toán, Hình 4):

M	Top-1 error (%)	Top-5 error (%)	NLL	Brier score
1	22,166	6,129	0,959	$0{,}317 \times 10^{-3}$
2	20,462	5,274	0,867	$0{,}294 \times 10^{-3}$
5	19,104	4,637	0,809	$0{,}280 \times 10^{-3}$
10	18,675	4,364	0,789	$0{,}275 \times 10^{-3}$

(Ghi chú cách đọc: trong bảng gốc, nhãn $\times 10^{-3}$ của paper gắn ở cột Brier score; cột NLL trình bày trực tiếp theo đơn vị nats, không scale, theo cách người ghi chú đọc bố cục bảng.) Tăng M từ 1 lên 10 giảm top-1 error

từ 22,166% xuống 18,675% và NLL từ 0,959 xuống 0,789, cho thấy ensemble vừa cải thiện accuracy vừa cải thiện chất lượng uncertainty đồng thời.

4.4 Uncertainty trên known vs unknown classes (OOD, Mục 3.5). Train MLP trên MNIST, test trên (a) MNIST test (known) và (b) **NotMNIST** (chữ cái, cùng kích thước ảnh nhưng nhãn khác hẳn, unknown). Đo **entropy** phân phối dự đoán: với known classes, entropy thấp cho cả ensemble và MC-dropout (như kỳ vọng); với **unknown classes**, entropy của deep ensembles tăng nhanh hơn hẳn MC-dropout khi M tăng; MC-dropout vẫn cho nhiều dự đoán tự tin cao (entropy gần 0) dù input hoàn toàn lạ. Thí nghiệm tương tự SVHN (known) / CIFAR-10 (unknown) và ImageNet chia dogs (known) / non-dogs (unknown) cho kết luận nhất quán.

4.5 Accuracy theo ngưỡng tin cậy (Hình 6). Lọc các ví dụ test có confidence $\geq \tau$, đo accuracy trên tập đã lọc, trộn MNIST (known) và NotMNIST (unknown). Nếu uncertainty tốt, accuracy phải tăng đơn điệu theo τ. MC-dropout cho accuracy **thấp** ngay cả ở τ cao (dự đoán tự tin nhưng sai nhiều trên input lạ); deep ensembles robust hơn hẳn, accuracy tăng đúng theo kỳ vọng khi τ tăng.

5. Phân tích / ablation

5.1 Vai trò riêng của việc học variance vs ensemble (Phụ lục A.1, Bảng 2). So 5 biến thể trên cùng bộ regression: Ensemble-5/10 (MSE, lấy phương sai thực nghiệm) **tệ hơn hẳn** ML-1 (một mạng học mean+variance bằng NLL) về cả RMSE và NLL (ví dụ Boston housing NLL: Ensemble-5(MSE) $17{,}28 \pm 6{,}17$ so với ML-1 $2{,}55 \pm 0{,}36$, một khoảng cách rất lớn). ML-1+AT không khác biệt đáng kể so với ML-1. ML-5 (chính là "Deep Ensembles" ở Bảng 1) tốt nhất hoặc ngang ML-1. Kết luận: **học variance qua proper scoring rule quan trọng hơn hẳn** so với chỉ lấy phương sai thực nghiệm giữa các mạng huấn luyện bằng MSE; ensemble là bước cải thiện **thêm vào**, không thay thế được bước học variance.

5.2 Calibration curve trên Year Prediction MSD (Hình 7, Phụ lục A.2). Với khoảng dự đoán z% tính từ quantile Gaussian (mean/variance dự đoán), đo tỉ lệ điểm test thực sự rơi vào khoảng đó. Phương pháp học variance (đường chéo lý tưởng) khớp tốt; ngược lại, nếu dùng **phương sai thực nghiệm** của 5 mạng huấn luyện bằng MSE, khoảng dự đoán 80% chỉ chứa **khoảng** 20% số điểm test thực tế, chứng tỏ phương sai thực nghiệm **đánh giá thấp nghiêm trọng** uncertainty thật.

5.3 Adversarial training không phải lúc nào cũng cần. Trên SVHN, hiệu ứng của adversarial training giảm dần khi M tăng; paper suy đoán nếu các lớp đã tách biệt rõ (well-separated), làm mượt theo hướng adversarial không đổi biên quyết định nhiều; paper nói rõ cần điều tra thêm, không khẳng định chắc.

5.4 Disagreement giữa các mạng như một thước đo uncertainty định tính (Hình 10-12). Định nghĩa disagreement $= \sum_{m=1}^{M} \mathrm{KL}(p_{\theta_m}(y \mid x) \,\|\, p_E(y \mid x))$. Trên NotMNIST, ảnh chữ "I" (giống số 1 trong MNIST) có **đồng thuận cao nhất** giữa các mạng ensemble (ensemble "nhận lầm" một cách có hệ thống, hợp lý vì hình dạng giống), còn các chữ khác biệt rõ với chữ số thì disagreement cao hơn, đúng với trực giác.

6. Giới hạn

Paper tự nêu (Mục 4, Discussion):

- Phương pháp tốn M **lần tham số và** M **lần tính toán inference** so với một mạng đơn; với ứng dụng hạn chế bộ nhớ, tác giả gợi ý có thể **distill ensemble thành một mô hình đơn giản hơn** (trích dẫn Bucila et al. 2006 và Hinton et al. 2015), nhưng không tự làm thí nghiệm đó trong bài.
- Chỉ huấn luyện các mạng **độc lập hoàn toàn**; chưa thử các cách khuyến khích đa dạng tường minh hơn (ví dụ decorrelate dự đoán, như Lee et al.), hay tối ưu trọng số kết hợp (stacking, mixture-of-experts thích ứng) - đều để lại cho "future work".
- Trên SVHN, hiệu ứng adversarial training giảm khi tăng M và paper **không chắc nguyên nhân** (giả thuyết lớp đã tách biệt rõ), cần điều tra thêm.

- Trên ImageNet chỉ thử biến thể **ensemble thuần** (không test MC-dropout hay adversarial training) "do hạn chế tính toán", nên so sánh đầy đủ ba yếu tố ở quy mô lớn chưa có.

Người ghi chú nhận xét:

- Bảng kết quả ImageNet (Hình 4) có nhãn cột "$\times 10^{-3}$" đặt ở vị trí mơ hồ trong bản PDF (có thể do lỗi dàn trang khi chuyển PDF sang text); người ghi chú đã diễn giải nó thuộc về cột Brier score dựa trên thứ tự độ lớn hợp lý của Brier score cho 1.000 lớp, nhưng đây là một suy luận đọc bảng, không phải trích dẫn chắc chắn 100% từ bố cục gốc.
- Tất cả so sánh với MC-dropout dùng **dropout rate cố định nhỏ (0,1)** sau khi thử 0,5 thấy kém hơn; không rõ MC-dropout có thể cải thiện tới mức nào nếu dò kỹ hơn dropout rate hoặc dùng các biến thể MC-dropout tinh chỉnh hơn (ví dụ Concrete Dropout, không có trong paper này).
- $M = 5$ là mặc định khuyến nghị nhưng hầu hết lợi ích đã đạt được ở M nhỏ (theo các đường cong Hình 2, 4: độ dốc cải thiện giảm rõ sau $M \approx 5$-10), gợi ý chi phí/lợi ích nên được cân nhắc theo bài toán cụ thể, paper không đưa ra quy tắc chọn M tối ưu tổng quát.

7. Liên hệ với các bài khác

- **ml basics 37** (Dropout as a Bayesian Approximation, Gal & Ghahramani 2015): **chính là baseline MC-dropout** được so sánh trực tiếp trong mọi thí nghiệm classification/regression của bài này; đây là liên hệ trực tiếp nhất.
- **ml basics 19** (On Calibration of Modern Neural Networks, Guo et al. 2017): paper trích dẫn [20] chính là bài này, nêu rõ "concurrent with our work, ... Guo et al. have also independently shown that non-Bayesian solutions can produce good predictive uncertainty estimates" - hai bài ra đời cùng thời điểm, cùng chủ đề calibration cho NN hiện đại.
- **ml basics 23** (Explaining and Harnessing Adversarial Examples, Goodfellow et al. 2015): nguồn trực tiếp của kỹ thuật FGSM mà Mục 3.2 của bài này dùng làm bước "adversarial training" (trích dẫn [18] trong paper).
- **distillation content/02_2015_distilling_knowledge_hinton.md**: paper gợi ý trực tiếp ở Discussion rằng ensemble (tốn M lần tham số) "can be distilled into a simpler model" và trích chính bài Hinton 2015 này ([26]) làm hướng giải quyết.
- **ml basics 35** (Conformal Prediction): cùng nhóm chủ đề "ước lượng uncertainty không hoàn toàn Bayesian", nhưng theo hướng khác hẳn (coverage guarantee hình thức, không cần proper scoring rule hay ensemble).

8. Gợi ý áp dụng

Deep ensembles cung cấp một cách ước lượng bất định dựa trên nhiều mô hình độc lập. Khi áp dụng, cần cân nhắc chi phí huấn luyện và suy luận, đồng thời đánh giá calibration và hiệu năng dưới dịch chuyển phân phối.

9. Thí nghiệm gợi ý

1. **Tái tạo Hình 1 (toy regression $x^3 + \varepsilon$):** trên CPU hoặc GPU nhỏ, huấn luyện 4 biến thể (ensemble-MSE lấy phương sai thực nghiệm, 1 mạng học NLL, +adversarial training, ensemble-5 học NLL), vẽ dải $\pm 3\sigma$ dự đoán. Kỳ vọng: dải uncertainty mở rộng hợp lý khi ra xa vùng có dữ liệu chỉ với các biến thể học NLL/ensemble, không phải biến thể MSE-thực-nghiệm.
2. **MNIST, so ensemble vs MC-dropout theo $M \in \{1, 3, 5, 10\}$:** đo classification error, NLL, Brier score trên một MLP nhỏ. Kỳ vọng: cả hai cải thiện theo M nhưng ensemble luôn tốt hơn hoặc ngang MC-dropout ở cùng M (tái tạo xu hướng Hình 2a).
3. **OOD đơn giản (MNIST vs ảnh nhiễu/rotate mạnh thay cho NotMNIST):** đo entropy dự đoán trên ảnh trong-phân-phối và ảnh đã biến dạng mạnh (coi như "unknown"). Kỳ vọng: entropy tăng rõ với ensemble khi M tăng, tăng chậm hơn với MC-dropout, tái tạo định tính Hình 3.

4. **Accuracy vs confidence threshold (tái tạo Hình 6):** trên cùng mô hình MNIST, lọc theo τ và vẽ accuracy. Kỳ vọng: đường ensemble đơn điệu tăng rõ hơn, MC-dropout có thể "phẳng" hoặc sai ở τ cao do overconfident.
5. **Distill ensemble 5 mô hình nhỏ thành 1 mô hình:** huấn luyện ensemble nhỏ trên MNIST, dùng xác suất trung bình của ensemble làm soft target để distill vào một mạng đơn (tương tự "ml basics content" phần distillation). Đo: NLL/Brier/entropy OOD của mô hình đã distill so với ensemble gốc và so với mô hình huấn luyện trực tiếp trên nhãn cứng. Diễn giải: nếu mô hình distill giữ được phần lớn chất lượng uncertainty của ensemble với chi phí $1\times$ thay vì $M\times$, đây là một đánh đổi đáng khảo sát.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Proper scoring rule	$S(p_\theta, q) \le S(q, q)$, bằng khi $p_\theta = q$	NLL và Brier score đều thoả
NLL regression	$\frac{\log \sigma_\theta^2(x)}{2} + \frac{(y-\mu_\theta(x))^2}{2\sigma_\theta^2(x)} + \text{const}$	công thức (1)
Brier score	$K^{-1}\sum_k (\delta_{k=y} - p_\theta(y{=}k \mid x))^2$	proper scoring rule
Adversarial example (FGSM)	$x' = x + \epsilon\,\text{sign}(\nabla_x \ell(\theta, x, y))$	$\epsilon = 1\%$ range mỗi chiều
Kết hợp ensemble (classification)	$p(y \mid x) = M^{-1}\sum_m p_{\theta_m}(y \mid x)$	trung bình softmax
Kết hợp ensemble (regression)	$\mu_* = M^{-1}\sum_m \mu_m$; $\sigma_*^2 = M^{-1}\sum_m(\sigma_m^2 + \mu_m^2) - \mu_*^2$	xấp xỉ mixture bằng 1 Gaussian
Default khuyến nghị	$M = 5$	Algorithm 1
Disagreement	$\sum_m \text{KL}(p_{\theta_m} \Vert p_E)$	thước đo định tính OOD
Kết quả nổi bật	NLL Naval propulsion: PBP $-3{,}73$, MC-drop $-3{,}80$, DeepEns $-5{,}63$	Bảng 1

11. Kết luận ngắn

Deep Ensembles là một công thức ba bước (proper scoring rule, adversarial training tuỳ chọn, ensemble $M = 5$ mạng độc lập) cho predictive uncertainty, hoàn toàn non-Bayesian nhưng vượt hoặc ngang MC-dropout trên hầu hết tiêu chí: NLL, Brier score, accuracy trên cả classification (MNIST, SVHN, ImageNet) và regression (10 bộ UCI), đồng thời cho uncertainty tốt hơn rõ rệt trên out-of-distribution (NotMNIST, CIFAR-10 trên mô hình train SVHN, non-dogs trên mô hình train dogs trong ImageNet). Phát hiện quan trọng đi kèm: học variance bằng một scoring rule đúng đắn quan trọng hơn hẳn chỉ lấy phương sai thực nghiệm giữa các mạng huấn luyện bằng MSE. Giới hạn chính là chi phí M lần tham số/tính toán, mà bài báo tự gợi ý có thể giảm bằng distillation ensemble vào một mô hình đơn, hướng chưa được chính paper thử nghiệm.

Nguồn gốc: https://arxiv.org/abs/1612.01474

Bài 37

Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning (2015)

Tác giả: Yarin Gal, Zoubin Ghahramani (University of Cambridge).
Venue / arXiv: arXiv:1506.02142 (v6, 04/10/2016; nộp lần đầu 06/2015); paper tự nêu venue ở trang đầu: "Proceedings of the 33rd International Conference on Machine Learning, New York, NY, USA, 2016. JMLR: W&CP volume 48" (ICML 2016).
Nguồn: https://arxiv.org/abs/1506.02142
Chủ đề: chứng minh huấn luyện dropout tương đương một dạng xấp xỉ biến phân (variational inference) cho deep Gaussian process, từ đó đề xuất Monte Carlo dropout (MC dropout: giữ dropout bật lúc test, lấy nhiều lượt forward pass) để rút ra predictive uncertainty từ chính các mô hình dropout đã có, không cần đổi kiến trúc hay cách huấn luyện.
Vị trí trong lộ trình: Đọc cùng bài 36 về deep ensembles để so sánh các cách ước lượng độ bất định.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận. Các chứng minh chi tiết được dẫn tới bản phụ lục riêng arXiv:1506.02157; ghi chú này dựa trên nội dung bài chính.

1. Bối cảnh

Các công cụ deep learning tiêu chuẩn cho regression và classification **không** lượng hoá được model uncertainty (độ bất định về chính mô hình, khác với nhiễu quan sát). Trong classification, người ta thường **hiểu sai** rằng giá trị softmax cao đồng nghĩa mô hình "tự tin đúng"; paper chỉ ra một điểm dữ liệu có thể có softmax rất cao (ví dụ 1) nhưng mô hình **vẫn bất định**, nếu điểm đó nằm xa vùng dữ liệu huấn luyện (xem Hình 1 của bài báo: hàm điểm ước lượng qua softmax ngoại suy ra một giá trị tự tin tuyệt đối cho điểm x^* xa dữ liệu, trong khi một phân phối đầy đủ qua softmax phải cho uncertainty cao ở đó).

Bayesian neural network (đặt phân phối lên trọng số, infer posterior) là công cụ có cơ sở toán học để lượng hoá uncertainty, nhưng tốn kém: các phương pháp biến phân trước đó (Hinton & Van Camp 1993; Barber & Bishop 1998; Graves 2011) "limited success"; các kỹ thuật biến phân mới hơn (Blundell et al. 2015) cho chất lượng tốt nhưng **gấp đôi số tham số** để biểu diễn uncertainty, hội tụ chậm hơn, và không cải thiện so với kỹ thuật hiện có; expectation propagation (Hernández-Lobato & Adams 2015, PBP) tốt hơn về RMSE/uncertainty so với VI nhưng vẫn là một mô hình riêng, không tận dụng được mô hình dropout NN thông thường đã có sẵn.

Khái niệm cần nắm:

- **Gaussian process (GP)**: một mô hình xác suất đặt phân phối lên không gian hàm; dự đoán của GP tại một điểm đi kèm cả mean và variance, biểu diễn trực tiếp uncertainty.
- **Biến phân (variational inference)**: thay vì tính posterior $p(\omega \mid X, Y)$ chính xác (bất khả thi), xấp xỉ nó bằng một phân phối $q(\omega)$ dễ xử lý, chọn q bằng cách tối thiểu hoá KL-divergence $\mathrm{KL}(q(\omega) \,\|\, p(\omega \mid X, Y))$.
- **Dropout**: kỹ thuật regularization phổ biến (Srivastava et al. 2014), tại mỗi forward pass lúc **huấn luyện**, mỗi unit bị "tắt" (set 0) với xác suất $1 - p_i$ độc lập.

Câu hỏi nghiên cứu: dropout training có thể được diễn giải lại dưới một khung Bayesian hình thức không, và nếu có, ta rút ra được công cụ thực dụng gì để lấy uncertainty từ chính các mạng dropout đã huấn luyện, **mà không cần thay đổi gì** mô hình hay quy trình huấn luyện?

2. Ý tưởng chính

Paper chứng minh: một neural network **độ sâu và non-linearity tuỳ ý**, có dropout áp trước mỗi lớp trọng số, **tương đương về mặt toán học** với một xấp xỉ biến phân cho **deep Gaussian process** (marginalise theo tham số của hàm hiệp phương sai). Cụ thể, các mặt nạ dropout ngẫu nhiên (biến Bernoulli) đóng vai trò tham số biến phân $q(\omega)$ xấp xỉ posterior của GP; tối thiểu hoá loss dropout chuẩn (cross-entropy/squared-error cộng weight decay) **chính là** tối thiểu hoá một chặn trên của KL-divergence giữa $q(\omega)$ và posterior thật.

Hệ quả thực dụng quan trọng nhất: vì dropout NN **đã là** một dạng mô hình Bayesian xấp xỉ, ta có thể lấy predictive uncertainty bằng cách **giữ dropout bật lúc test**, chạy T lượt forward pass độc lập (mỗi lượt lấy mẫu mặt nạ dropout khác nhau), rồi lấy trung bình/phương sai thực nghiệm của T kết quả. Đây gọi là **MC dropout** (Monte Carlo dropout). Điểm mạnh: không cần đổi kiến trúc, không cần huấn luyện lại, không tăng chi phí huấn luyện, có thể áp ngay lên các mô hình dropout đã có; chi phí thêm chỉ là T lần forward pass lúc suy luận (có thể chạy song song).

3. Phương pháp chi tiết

3.1 Dropout NN như xấp xỉ biến phân cho deep GP

Ký hiệu mạng L lớp, hàm loss $E(\cdot,\cdot)$ (ví dụ softmax loss hoặc squared loss), trọng số W_i (kích thước $K_i \times K_{i-1}$), bias b_i. Mục tiêu dropout chuẩn (công thức 1):

$$L_{\text{dropout}} := \frac{1}{N}\sum_{i=1}^{N} E(y_i, \hat{y}_i) + \lambda \sum_{i=1}^{L} \left(\|W_i\|_2^2 + \|b_i\|_2^2\right) \tag{1}$$

Xét một **deep Gaussian process** L lớp với hàm hiệp phương sai $K(x,y) = \int p(w)p(b)\,\sigma(w^T x+b)\,\sigma(w^T y+b)\,dw\,db$ (phi tuyến σ, phân phối $p(w), p(b)$). Paper chỉ ra (dẫn chi tiết ở phụ lục riêng, không có trong văn bản chính) hàm hiệp phương sai này có thể xấp xỉ bằng cách đặt một phân phối biến phân lên mỗi thành phần của khai triển phổ (spectral decomposition) của GP, ánh xạ mỗi lớp GP sang một lớp hidden unit tường minh:

$$p(y \mid x, X, Y) = \int p(y \mid x, \omega)\, p(\omega \mid X, Y)\, d\omega, \qquad p(y \mid x, \omega) = \mathcal{N}\big(y; \hat{y}(x,\omega),\ \tau^{-1} I_D\big) \tag{2}$$

với $\omega = \{W_i\}_{i=1}^{L}$, $\hat{y}(x,\omega) = W_L\,\sigma(\cdots\sigma(W_1 x + m_1)\cdots)$ (có chuẩn hoá $1/\sqrt{K_i}$ mỗi lớp), $\tau > 0$ là precision. Posterior $p(\omega \mid X, Y)$ bất khả thi; paper chọn một họ $q(\omega)$ với cấu trúc dropout:

$$W_i = M_i \cdot \text{diag}([z_{i,j}]_{j=1}^{K_{i-1}}), \qquad z_{i,j} \sim \text{Bernoulli}(p_i) \tag{q}$$

tức mỗi cột của M_i (tham số biến phân) bị "tắt" ngẫu nhiên theo đúng cách dropout vẫn làm. Tối thiểu hoá $\text{KL}(q(\omega)\|p(\omega \mid X, Y))$, viết lại thành:

$$-\int q(\omega) \log p(Y \mid X, \omega)\, d\omega + \text{KL}(q(\omega)\|p(\omega)) \tag{3}$$

Xấp xỉ Monte Carlo số hạng đầu bằng **một mẫu** $\hat{\omega}_n \sim q(\omega)$ mỗi điểm dữ liệu (không lệch - unbiased), và xấp xỉ số hạng KL thứ hai (phụ lục), ta được mục tiêu (công thức 4):

$$L_{\text{GP-MC}} \propto \frac{1}{N}\sum_{n=1}^{N} \frac{-\log p(y_n \mid x_n, \hat{\omega}_n)}{\tau} + \sum_{i=1}^{L} \left(\frac{p_i l^2}{2\tau N}\|M_i\|_2^2 + \frac{l^2}{2\tau N}\|m_i\|_2^2\right) \tag{4}$$

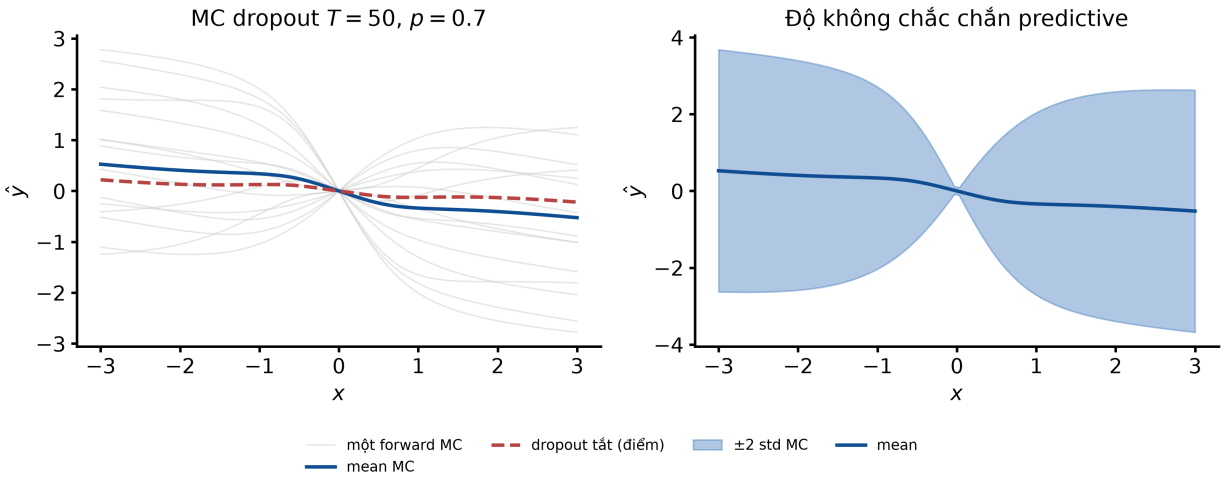
với l là **prior length-scale**. So (4) với (1): nếu đặt $E(y_n, \hat{y}_n) = -\log p(y_n \mid x_n, \hat{\omega}_n)/\tau$, hai mục tiêu **trùng nhau** với một cách đặt tương ứng của τ, l. Việc lấy mẫu $\hat{\omega}_n$ theo Bernoulli chính xác là cơ chế dropout đang làm. Hệ quả: **weight decay λ trong huấn luyện dropout thông thường tương ứng với một lựa chọn cụ thể của precision τ và length-scale l**, theo quan hệ (công thức 7):

$$\tau = \frac{p\, l^2}{2N\lambda} \tag{7}$$

Ví dụ tính tay minh hoạ công thức (7) (tự tính, số liệu tự chọn để minh hoạ, không phải số liệu thực nghiệm của paper): giả sử dropout rate $p = 0{,}1$, prior length-scale $l = 1$, $N = 1000$ điểm huấn luyện, weight decay $\lambda = 0{,}0001$. Áp (7): $\tau = \dfrac{0{,}1 \times 1^2}{2 \times 1000 \times 0{,}0001} = \dfrac{0{,}1}{0{,}2} = 0{,}5$. Nghĩa là với các siêu tham số dropout/weight-decay đã chọn khi huấn luyện mô hình như thường lệ, ta **suy ra ngược** được một precision $\tau = 0{,}5$ cho mô hình Gaussian giả định tương đương, dùng để tính predictive log-likelihood ở công thức (8) dưới đây, **không cần** train lại gì thêm.

3.2 MC dropout: ước lượng predictive mean/variance

Phân phối dự đoán xấp xỉ (công thức 5): $q(y^* \mid x^*) = \int p(y^* \mid x^*, \omega) q(\omega)\, d\omega$. Lấy T bộ mặt nạ Bernoulli độc lập $\{z_1^t, \ldots, z_L^t\}_{t=1}^T$ (tương ứng T lượt forward pass dropout bình thường lúc test), ước lượng mean (công thức 6):



MC dropout ($T = 50$, $p = 0.7$): mean/std predictive so với điểm ước lượng tắt dropout (minh hoạ, tự tính)

$$\mathbb{E}_{q(y^* \mid x^*)}(y^*) \approx \frac{1}{T} \sum_{t=1}^{T} \hat{y}(x^*, W_1^t, \ldots, W_L^t) \tag{6}$$

đây chính là **MC dropout**: trung bình của T lượt forward pass dropout (tương đương "model averaging" đã được biết tới trong literature, nhưng paper cho một suy diễn Bayesian hình thức mới). Ước lượng predictive variance:

$$\mathrm{Var}_{q(y^* \mid x^*)}(y^*) \approx \tau^{-1} I_D + \frac{1}{T} \sum_{t=1}^{T} \hat{y}(x^*, \cdot)^T \hat{y}(x^*, \cdot) - \mathbb{E}(y^*)^T \mathbb{E}(y^*)$$

tức **phương sai mẫu của T lượt forward pass, cộng thêm nghịch đảo model precision** τ^{-1} (phần nhiễu quan sát cố định). Predictive log-likelihood cho regression (công thức 8):

$$\log p(y^* \mid x^*, X, Y) \approx \text{logsumexp}\Big(-\tfrac{1}{2}\tau\|y^* - \hat{y}_t\|^2\Big) - \log T - \tfrac{1}{2}\log 2\pi - \tfrac{1}{2}\log \tau^{-1} \tag{8}$$

Ví dụ tính tay (tự tính, số liệu minh hoạ): giả sử $T = 2$ lượt forward pass cho giá trị $\hat{y}_1 = 1{,}0$ và $\hat{y}_2 = 1{,}4$ với nhãn thật $y^* = 1{,}2$ và $\tau = 10$. Tính từng số hạng trong logsumexp: $-\frac{1}{2} \cdot 10 \cdot (1{,}2 - 1{,}0)^2 = -0{,}2$ và $-\frac{1}{2} \cdot 10 \cdot (1{,}2 - 1{,}4)^2 = -0{,}2$. logsumexp$(-0{,}2, -0{,}2) = \ln(2e^{-0{,}2}) = \ln 2 - 0{,}2 \approx 0{,}493$. Trừ $\log T = \ln 2 \approx 0{,}693$, trừ $\frac{1}{2}\log(2\pi) \approx 0{,}919$, trừ $\frac{1}{2}\log(\tau^{-1}) = \frac{1}{2}\log(0{,}1) \approx -1{,}151$ (tức **cộng** $1{,}151$). Tổng: $0{,}493 - 0{,}693 - 0{,}919 + 1{,}151 \approx 0{,}032$ nats. Log predictive likelihood dương nhỏ ($\approx 0{,}032$) tương ứng mật độ dự đoán tại điểm đúng $\approx e^{0{,}032} \approx 1{,}03$, hợp lý vì cả hai forward pass đều gần giá trị thật $1{,}2$.

Mạng dropout **không hề bị thay đổi**: chỉ cần thu thập kết quả của các lượt forward pass dropout thông thường (có thể chạy đồng thời), nên tổng thời gian suy luận không đổi nếu tính song song.

3.3 Mở rộng sang classification

Theo một ghi chú (footnote 2) ở thân bài (chi tiết đầy đủ ở phụ lục riêng không có trong văn bản chính), paper mở rộng cùng suy diễn sang classification bằng cách đặt $E(\cdot)$ là softmax loss và $\tau = 1$. Thực nghiệm classification ở Mục 5.2 dùng chính cách mở rộng này.

4. Thí nghiệm và kết quả chính

4.1 Regression ngoại suy (extrapolation), dữ liệu CO2 Mauna Loa (Mục 5.1, Hình 2-3). Dùng tập con nồng độ CO2 khí quyển (~200 điểm), huấn luyện NN 4 hoặc 5 lớp ẩn, 1024 unit/lớp, non-linearity ReLU hoặc TanH, dropout rate 0,1 hoặc 0,2. So 4 biến thể trên cùng mạng 5 lớp ReLU: (a) dropout chuẩn với weight-averaging lúc test (không đánh giá uncertainty); (b) Gaussian process với hiệp phương sai squared-exponential (tham chiếu lý tưởng); (c) MC dropout (ReLU); (d) MC dropout (TanH, 5 lớp). Kết quả định tính: tại điểm x^* xa dữ liệu huấn luyện, **dropout chuẩn (a) dự đoán tự tin một giá trị vô nghĩa** (giá trị 0); GP (b) và MC dropout (c, d) đều biểu diễn đúng rằng mô hình **bất định** ở điểm đó (độ lệch chuẩn tăng mạnh). Khác biệt giữa ReLU và TanH: **uncertainty của ReLU tăng không giới hạn** khi ra xa dữ liệu, còn **TanH bị chặn** (bão hoà), khớp với tính chất của các hàm hiệp phương sai GP tương ứng mà hai non-linearity này xấp xỉ. Số lượt forward pass $T = 1000$ dùng để vẽ hình chính; $T = 10$ (Hình 3) đã cho ước lượng mean/uncertainty "hợp lý" (reasonable), không cần T lớn.

4.2 Classification trên MNIST, ảnh số "1" xoay liên tục (Mục 5.2, Hình 4). LeNet convnet, dropout trước lớp fully-connected cuối, dropout rate 0,5, huấn luyện 10^6 iteration (Caffe), lịch learning rate $\gamma = 0{,}0001$, $p = 0{,}75$ (cùng dạng inverse-time decay như ở "ml basics 33"). Lấy 100 lượt forward pass dropout trên 12 ảnh số 1 xoay dần góc; lớp dự đoán cho 12 ảnh là $[1, 1, 1, 1, 1, 5, 5, 7, 7, 7, 7, 7]$. Quan sát chính: khi "đám" (scatter) 100 lượt forward pass của softmax-input cho lớp đang thắng **tách biệt rõ** khỏi các lớp khác, dự đoán chắc chắn; nhưng khi đám của lớp thắng **giao nhau** với lớp khác (ảnh ở giữa quá trình xoay), softmax output vẫn có thể gần 1 (tự tin tuyệt đối theo giá trị điểm), nhưng **độ bất động của chính giá trị softmax output đó** (quan sát qua 100 lượt) có thể trải khắp cả khoảng $[0, 1]$: minh chứng rằng "softmax cao" không đồng nghĩa "mô hình chắc chắn" nếu chỉ nhìn một lượt forward pass; cần nhìn đám MC dropout, hoặc tính entropy/variation ratio, để biết thật sự có bất định hay không.

4.3 So sánh predictive performance trên 10 bộ UCI (Mục 5.3, Bảng 1). So VI (Graves 2011), PBP (Hernández-Lobato & Adams 2015), và Dropout (MC dropout, bài này) trên RMSE và predictive log-likelihood, trung bình trên 20 random split (Protein: 5 split; Year Prediction MSD: 1 split). Mạng nhỏ: 50 hidden unit (theo đúng kích thước PBP dùng), dropout rate **chỉ 0,05 hoặc 0,005** (rất nhỏ, vì mạng và dataset nhỏ); dò precision τ bằng Bayesian Optimization (BO) trên validation log-likelihood; prior length-scale $l = 10^{-2}$ cho hầu hết dataset; minibatch 32, Adam.

Dataset (N, Q)	RMSE (VI / PBP / Dropout)	LL (VI / PBP / Dropout)
Boston Housing (506, 13)	4,32 / 3,01 / **2,97**	-2,90 / -2,57 / **-2,46**
Energy Efficiency (768, 8)	2,65 / 1,80 / **1,66**	-2,39 / -2,04 / **-1,99**
Kin8nm (8.192, 8)	0,10 / 0,10 / 0,10	0,90 / 0,90 / **0,95**
Naval Propulsion (11.934, 16)	0,01 / 0,01 / 0,01	3,73 / 3,73 / **3,80**
Protein Structure (45.730, 9)	4,84 / 4,73 / **4,36**	-2,99 / -2,97 / **-2,89**
Yacht Hydrodynamics (308, 6)	6,89 / **1,02** / 1,11	-3,43 / -1,63 / **-1,55**
Year Prediction MSD (515.345, 90)	9,034 / 8,879 / **8,849**	-3,622 / -3,603 / **-3,588**

(7/10 dòng tiêu biểu trong Bảng 1 đầy đủ.) **Dropout (MC dropout) thắng hoặc ngang cả về RMSE và LL trên mọi dataset, trừ Yacht** (PBP tốt hơn về RMSE, dù Dropout vẫn thắng LL). Paper nhấn mạnh: trên Boston Housing, **median** (không phải mean) cho kết quả tốt hơn: RMSE median 2,68 (IQR $[2{,}45, 3{,}35]$), LL median $-2{,}34$ (IQR $[-2{,}54, -2{,}29]$) - mean bị kéo lệch bởi vài lần chạy ngoại lai.

Footnote cập nhật quan trọng (10/2016): phiên bản đầu của paper từng báo **sai lệch chuẩn (standard error) bị nhân nhầm gấp 4,5 lần** (ví dụ RMSE Boston báo SE $0{,}85$ thay vì $0{,}19$ đúng) - một lỗi đã được tác giả tự sửa và ghi chú công khai trong bản v6 đang đọc. Cùng footnote: thử với **100 lần epoch nhiều hơn** PBP (gấp 10 lần số epoch dùng trong Bảng 1) và thêm một biến thể **2 lớp ẩn**, cho cải thiện đáng kể cả RMSE và LL (Bảng 2 ở cuối paper, ví dụ Naval Propulsion LL tăng từ 3,80 lên 4,32 với 2 lớp).

Chi phí thời gian chạy: trên Naval Propulsion, mô hình của paper (gồm BO dò τ) tốn 276 giây/split trung bình, so với PBP 220 giây; nếu bỏ BO chỉ còn 95 giây. Trên Kin8nm: 188 giây (có BO) hoặc 65 giây (không BO), so với PBP 156 giây.

4.4 Dropout rate ảnh hưởng rõ dù rất nhỏ: lặp lại thí nghiệm Boston Housing với dropout rate $= 0$ (tức không dropout) cho RMSE $3{,}07$ và LL $-2{,}59$, **kém hơn rõ** so với dùng dropout rate nhỏ $0{,}05$ (RMSE $2{,}97$, LL $-2{,}46$), cho thấy dù dropout rate rất nhỏ, nó vẫn ảnh hưởng đáng kể tới cả predictive mean và uncertainty.

4.5 Reinforcement learning: dropout Q-network + Thompson sampling (Mục 5.4, Hình 5-6). Agent 2D (9 "mắt" cảm nhận pixel, 5 action điều khiển 2 động cơ, vòng tròn đỏ/xanh là thưởng/phạt), mô phỏng lại thiết lập đơn giản hoá của Mnih et al. 2015 (deep RL trên Atari). Huấn luyện Q-network gốc (epsilon-greedy) và một bản có dropout rate 0,1 trước mỗi lớp trọng số, dùng **Thompson sampling** (một lượt forward pass dropout ngẫu nhiên để chọn hành động, dùng lại đúng mặt nạ đó khi backprop ở replay) thay cho epsilon-greedy. Kết quả: Thompson sampling đạt phần thưởng > 1 chỉ sau **25 batch** tính từ lúc hết "burn-in" (25 batch di chuyển ngẫu nhiên không tính); epsilon-greedy cần **175 batch** để đạt cùng mức. Ghi chú thú vị: sau khoảng 1.000 batch, phương pháp Thompson có vẻ **ngừng cải thiện**, vì vẫn luôn lấy mẫu ngẫu nhiên (một lượt forward pass dropout mỗi lần), trong khi epsilon-greedy ở giai đoạn đó chỉ khai thác (exploit) hoàn toàn.

5. Phân tích / ablation

- **ReLU vs TanH (Mục 5.1):** khác biệt hành vi uncertainty (không giới hạn vs bị chặn) khi ra xa dữ liệu phản ánh trực tiếp các hàm hiệp phương sai GP khác nhau mà mỗi non-linearity tương ứng xấp xỉ (chi tiết dẫn ở phụ lục riêng không có trong văn bản đọc được).
- **Dropout rate 0,1 vs 0,2 (TanH):** lúc khởi tạo, model với dropout 0,1 cho uncertainty nhỏ hơn model dropout 0,2, nhưng **sau khi hội tụ** cả hai cho uncertainty gần như không phân biệt được - gợi ý các moment của mô hình dropout "hội tụ về" đúng các moment của GP xấp xỉ tương ứng, không phụ thuộc điểm khởi đầu.
- **Số lớp ẩn:** paper thử (không thành công, "unsuccessfully") khớp dữ liệu CO2 với **số lớp nhỏ hơn**, mô hình không lớn thì không fit tốt, cho thấy kết quả về uncertainty ở đây gắn với một kiến trúc đủ sâu cụ thể, không tổng quát "mọi kích thước mạng cho uncertainty tốt như nhau".

- T **(số lượt forward pass):** $T = 1000$ chỉ để vẽ hình mượt; $T = 10$ đã cho ước lượng "hợp lý" (Hình 3), gợi ý chi phí suy luận thêm của MC dropout có thể rất nhỏ trong thực tế (không cần hàng trăm/nghìn lượt).
- **Epoch và độ sâu (Footnote cập nhật, Bảng 2):** tăng epoch gấp 10 và thêm một lớp ẩn cải thiện cả RMSE và LL trên mọi dataset UCI đã thử lại, cho thấy kết quả Bảng 1 (chỉ 10x epoch so với PBP's 40, 1 lớp) **chưa phải giới hạn trên** của chất lượng MC dropout, chỉ là một điểm so sánh "công bằng về thời gian chạy" với PBP.

6. Giới hạn

Paper tự nêu:

- Phần dẫn chứng minh đầy đủ (deep GP, các Proposition C/D cho công thức mean/variance) nằm ở **một bản arXiv phụ lục riêng** (1506.02157), không nằm trong thân bài chính; nhiều khẳng định trong Mục 3-4 ("following proposition C in the appendix", "section 4.1 in the appendix") không tự chứng minh được trong văn bản đã đọc, phải tin vào tham chiếu.
- Mục Kết luận tự nhận: Bernoulli dropout chỉ là **một ví dụ** trong họ các kỹ thuật regularization có thể diễn giải theo khung biến phân này; các biến thể khác (drop-connect, multiplicative Gaussian noise) cho uncertainty khác, việc khảo sát chúng được để lại cho "follow-up work", chưa làm trong bài.
- Có mối liên hệ một-một giữa hàm hiệp phương sai GP và tổ hợp (non-linearity, kiểu regularization) của NN, nhưng việc chọn tổ hợp phù hợp theo giả định tiên nghiệm về dữ liệu (ví dụ non-linearity cosine + L2 cho hàm trơn, uncertainty tăng xa dữ liệu) mới chỉ là hướng "subject of current research", chưa có kết quả đầy đủ trong bài.
- Tự công khai và sửa một lỗi tính sai standard error (gấp 4,5 lần) ở phiên bản trước, một điểm minh bạch khoa học đáng ghi nhận nhưng cũng cho thấy số liệu ở Bảng 1 **đã từng sai** trước khi được sửa ở v6.

Người ghi chú nhận xét:

- Toàn bộ thí nghiệm UCI dùng mạng **rất nhỏ** (50 unit, 1 lớp ẩn) và dropout rate **rất nhỏ** (0,05/0,005); không rõ các kết luận (Dropout thắng VI/PBP) có giữ nguyên ở quy mô mạng lớn hiện đại (hàng trăm triệu đến hàng tỷ tham số) hay không, vì bản chất xấp xỉ GP có thể suy biến khác đi khi mạng rất rộng/sâu.
- Thí nghiệm classification (Mục 5.2) chỉ mang tính **minh họa định tính** (12 ảnh xoay của một chữ số), không có bảng số định lượng (ví dụ NLL/Brier) như "ml basics 36" (Deep Ensembles) đã làm kỹ hơn khi so sánh hai phương pháp trên cùng MNIST với các thước đo định lượng đầy đủ.
- Dropout áp dụng **trước mỗi lớp trọng số** là điều kiện lý thuyết để suy diễn GP áp dụng đúng; nếu mô hình chỉ có dropout ở một vài lớp (cách dùng rất phổ biến trong thực hành, như ví dụ chính Mục 5.2 chỉ dropout trước lớp FC cuối), suy diễn Bayesian đầy đủ không còn áp dụng nguyên vẹn cho toàn mạng, paper không bàn rõ mức độ sai lệch khi dùng "một phần" dropout như vậy.

7. Liên hệ với các bài khác

- **ml basics 36** (Deep Ensembles): so sánh trực tiếp và toàn diện nhất với MC dropout trên cùng các benchmark (MNIST, SVHN, ImageNet, 10 bộ UCI); Deep Ensembles báo cáo vượt MC dropout trên hầu hết tiêu chí, đặc biệt về robustness cho out-of-distribution.
- **ml basics 06** (Improving neural networks by preventing co-adaptation of feature detectors, Hinton và đồng nghiệp): bài gốc đề xuất dropout như kỹ thuật regularization; bài này diễn giải lại đúng cơ chế đó dưới góc nhìn Bayesian, không đổi cách dùng dropout lúc huấn luyện.
- **ml basics 19** (On Calibration of Modern Neural Networks): cùng quan tâm tới việc softmax output có phản ánh đúng độ tin cậy thật hay không; MC dropout là một công cụ khả dĩ để bổ sung uncertainty ngoài xác suất điểm (point estimate) mà bài Calibration phân tích.

- **ml basics 35** (Conformal Prediction): cùng nhóm chủ đề uncertainty quantification trong batch này, nhưng theo hướng hoàn toàn khác (đảm bảo coverage hình thức, không cần giả định Bayesian/GP).

8. Gợi ý áp dụng

MC dropout dùng nhiều lượt dự đoán ngẫu nhiên từ cùng bộ trọng số để ước lượng phân phối dự đoán. Chi phí suy luận tăng theo số lượt, và diễn giải Bayesian phụ thuộc các giả định của phương pháp.

9. Thí nghiệm gợi ý

1. **Tái tạo Hình 2 (CO2 toy, hoặc một hàm 1D tương tự) trên GPU nhỏ:** huấn luyện NN vài lớp ẩn với dropout 0,1-0,2, so sánh dropout chuẩn (weight-averaging) với MC dropout ($T = 10, 100, 1000$) ở vùng ngoại suy xa dữ liệu. Đo: độ lệch chuẩn dự đoán tăng theo khoảng cách ra xa dữ liệu với MC dropout nhưng không với dropout chuẩn.
2. **So sánh ReLU vs TanH về hành vi uncertainty ngoại suy:** cùng kiến trúc, chỉ đổi non-linearity, đo xem variance MC dropout có "mở rộng không giới hạn" (ReLU) hay "bị chặn" (TanH) khi x ra xa miền train, kiểm tra claim định tính của Mục 5.1.
3. **MNIST với T nhỏ:** đo NLL/Brier score (định lượng, bổ sung cho thí nghiệm định tính gốc của paper) theo $T \in \{1, 5, 10, 50\}$ trên một LeNet nhỏ, dropout trước lớp FC cuối. Kỳ vọng: cải thiện nhanh rồi bão hoà sau một T nhỏ, như "ml basics 36" quan sát với ensemble size.
4. **Kiểm tra điều kiện "dropout trước mọi lớp":** huấn luyện hai mô hình CNN nhỏ, một chỉ dropout ở lớp cuối (cách dùng phổ biến), một dropout ở mọi lớp trọng số (đúng giả định lý thuyết của paper); so sánh chất lượng uncertainty (entropy trên ảnh nhiễu/rotate mạnh) giữa hai cách, xem khác biệt có lớn không.
5. **Ước lượng chi phí latency thực tế của MC dropout cho một mô hình nhỏ dạng encoder-decoder** (mô phỏng OCR): đo thời gian suy luận theo T, batch 1, xác định T tối đa chấp nhận được cho một ứng dụng cần tốc độ, làm cơ sở thực nghiệm cho suy luận ở mục 8.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị	Ghi chú
Dropout objective	$L_{\text{dropout}} = \frac{1}{N}\sum E(y_i, \hat{y}_i) + \lambda \sum (\|W_i\|_2^2 + \|b_i\|_2^2)$	công thức (1)
$q(\omega)$ (biến phân = dropout)	$W_i = M_i \,\text{diag}(z_{i,j})$, $z_{i,j} \sim \text{Bernoulli}(p_i)$	mặt nạ dropout = tham số biến phân
Mục tiêu GP-MC	công thức (4), trùng công thức (1) với $E = -\log p(y_n \mid x_n, \hat{\omega}_n)/\tau$	
Quan hệ precision	$\tau = \dfrac{p\, l^2}{2N\lambda}$	công thức (7)
MC dropout mean	$\mathbb{E}(y^*) \approx \frac{1}{T}\sum_t \hat{y}(x^*, \omega_t)$	công thức (6), T lượt forward pass
MC dropout variance	$\text{Var}(y^*) \approx \tau^{-1} I_D +$ phương sai mẫu T lượt	
Predictive log-likelihood	$\text{logsumexp}(-\frac{1}{2}\tau\|y - \hat{y}_t\|^2) - \log T - \frac{1}{2}\log 2\pi - \frac{1}{2}\log \tau^{-1}$	công thức (8)
Kết quả nổi bật	Dropout thắng VI/PBP trên 9/10 dataset UCI (RMSE và LL)	Bảng 1
Thompson sampling (RL)	đạt reward mục tiêu ở batch 25 so với 175 của epsilon-greedy	Mục 5.4

11. Kết luận ngắn

Gal và Ghahramani chứng minh huấn luyện dropout tương đương một xấp xỉ biến phân cho deep Gaussian process, từ đó đưa ra **MC dropout**: giữ dropout bật lúc test, lấy trung bình/phương sai của T lượt forward pass, để rút uncertainty từ bất kỳ mô hình dropout đã huấn luyện mà không cần đổi kiến trúc hay huấn luyện lại. Thực nghiệm trên regression toy (CO2), classification (MNIST), 10 bộ UCI (so với VI và PBP), và reinforcement learning (Thompson sampling) đều cho thấy MC dropout cải thiện rõ predictive log-likelihood/RMSE so với các phương pháp Bayesian xấp xỉ trước đó, với chi phí thêm chỉ là vài lượt forward pass. Phần dẫn chứng minh chi tiết nằm ở một bản arXiv phụ lục riêng không có trong văn bản đã đọc; so với "ml basics 36" (Deep Ensembles, ra sau và báo cáo vượt MC dropout trên nhiều tiêu chí, đặc biệt out-of-distribution), MC dropout vẫn có lợi thế rõ về chi phí bộ nhớ (chỉ 1 mô hình), là điểm đáng cân nhắc khi tài nguyên GPU hạn chế.

Nguồn gốc: https://arxiv.org/abs/1506.02142

Bài 38

A ConvNet for the 2020s (2022)

Tác giả: Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, Saining Xie (Facebook AI Research và UC Berkeley).
Venue / arXiv: arXiv:2201.03545 (v2, 02/03/2022; nộp lần đầu 01/2022); paper không nêu dòng venue hội nghị cụ thể trong phần văn bản đã trích xuất (không thấy ghi "CVPR" hay tên hội nghị nào trên trang đầu của bản PDF đọc được).
Nguồn: https://arxiv.org/abs/2201.03545
Chủ đề: "hiện đại hoá" từng bước một ResNet thông thường theo hướng thiết kế của Swin Transformer (một vision Transformer dạng hierarchical), không thêm bất kỳ cơ chế self-attention nào, để tạo ra họ mô hình thuần ConvNet gọi là ConvNeXt, cạnh tranh ngang ngửa hoặc vượt Swin Transformer về độ chính xác, khả năng scale và robustness trên nhiều benchmark.
Vị trí trong lộ trình: Đọc cùng bài 07 về ResNet và bài về Vision Transformer để đối chiếu các lựa chọn kiến trúc.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Đầu những năm 2020, Vision Transformer (ViT) nhanh chóng vượt ConvNet để trở thành mô hình state-of-the-art cho phân loại ảnh (ImageNet classification). Nhưng ViT "thuần" (vanilla) gặp khó khi áp cho các bài toán computer vision tổng quát hơn (object detection, semantic segmentation), vì thiết kế global attention của nó có độ phức tạp **bậc hai** theo kích thước input, khó chấp nhận với ảnh độ phân giải cao, và ViT gần như không có inductive bias đặc thù cho ảnh (ngoài lớp "patchify" ban đầu).

Hierarchical Transformer (ví dụ Swin Transformer) giải quyết vấn đề này bằng cách **mang lại một số ConvNet prior**: chiến lược "sliding window" (attention trong các window cục bộ) được đưa lại vào Transformer, giúp nó hoạt động giống ConvNet hơn, trở thành backbone tổng quát khả dụng thực tế, đạt SOTA trên nhiều task. Thành công của Swin cho thấy: bản chất của convolution **không hề mất giá trị**, nó vẫn được mong muốn và tái sử dụng dưới một hình thức khác.

Góc nhìn phổ biến khi đó là gán hiệu quả vượt trội này cho "sự ưu việt nội tại của Transformer, hơn là inductive bias vốn có của convolution". Câu hỏi nghiên cứu của bài báo: **có bao nhiêu phần của lợi thế đó thực ra đến từ kỹ thuật huấn luyện và lựa chọn thiết kế kiến trúc (không gắn riêng với self-attention), và liệu một ConvNet "thuần" (không có attention) có thể đạt mức hiệu năng tương đương khi áp dụng đúng các kỹ thuật đó không?**

2. Ý tưởng chính

Bắt đầu từ một ResNet-50/ResNet-200 chuẩn, paper "hiện đại hoá" dần kiến trúc theo hướng thiết kế của Swin Transformer, **từng bước một, không bao giờ thêm cơ chế attention nào**, theo 5 nhóm thay đổi: (1) macro design, (2) ResNeXt-ify, (3) inverted bottleneck, (4) large kernel size, (5) micro design (activation, normalization). Mỗi bước được giữ lại nếu cải thiện hiệu năng, loại bỏ nếu không, và FLOPs được kiểm soát tương đối ổn định xuyên suốt (roughly controlled) để so sánh công bằng.

Kết quả cuối: một họ mô hình thuần ConvNet gọi là **ConvNeXt**, xây dựng hoàn toàn từ các module ConvNet tiêu chuẩn (conv thường, depthwise conv, LayerNorm, GELU), **cạnh tranh ngang ngửa hoặc vượt** Swin Transformer về độ chính xác, khả năng scale, và robustness trên classification (ImageNet),

detection/segmentation (COCO), và semantic segmentation (ADE20K), đạt tới **87,8% top-1 accuracy trên ImageNet-1K** (với pretraining trên ImageNet-22K), đồng thời vẫn giữ được sự đơn giản, tính fully-convolutional, và tốc độ suy luận cao của ConvNet chuẩn. Kết luận ngầm: phần lớn lợi thế của Transformer trong các so sánh trước đó nằm ở **lựa chọn thiết kế kiến trúc + kỹ thuật huấn luyện**, không nằm ở chính cơ chế self-attention.

3. Phương pháp chi tiết

Tất cả thí nghiệm "hiện đại hoá" huấn luyện và đánh giá trên ImageNet-1K, ở 2 mức FLOPs: chế độ ResNet-50/Swin-T ($\approx 4{,}5 \times 10^9$ FLOPs) và ResNet-200/Swin-B ($\approx 15{,}0 \times 10^9$ FLOPs); phần chính trình bày chế độ ResNet-50/Swin-T, chế độ lớn hơn cho kết quả nhất quán (Phụ lục C).

3.1 Công thức huấn luyện (training recipe, không đổi kiến trúc)

Trước khi đổi bất kỳ chi tiết kiến trúc nào, paper huấn luyện ResNet-50 **gốc** với một công thức huấn luyện hiện đại giống DeiT/Swin Transformer: AdamW, huấn luyện 300 epoch (thay 90 epoch gốc), augmentation Mixup, Cutmix, RandAugment, Random Erasing, cùng regularization Stochastic Depth và Label Smoothing. Chỉ riêng thay đổi này đã nâng accuracy từ **76,1%** (ResNet-50 gốc, theo PyTorch) lên **78,8%** (**+2,7 điểm**). Đây là baseline "công bằng" được **giữ cố định cùng hyperparameter** suốt toàn bộ quá trình hiện đại hoá kiến trúc phía sau, để cách ly đúng phần đóng góp của kiến trúc.

3.2 Macro design

- **Đổi stage compute ratio**: ResNet-50 gốc phân bổ block $(3, 4, 6, 3)$ qua 4 stage (chủ yếu theo kinh nghiệm). Swin-T dùng tỉ lệ 1:1:3:1 (Swin lớn hơn dùng 1:1:9:1). Đổi thành $(3, 3, 9, 3)$ khớp FLOPs với Swin-T: $78{,}8\% \rightarrow 79{,}4\%$.
- **Đổi stem sang "patchify"**: thay lớp stem gốc (conv 7×7 stride 2 + max pool 3×3 stride 2, tổng downsample $4\times$) bằng một lớp conv 4×4 stride 4 không chồng lấp (giống patchify của ViT/Swin, cũng cho downsample $4\times$): $79{,}4\% \rightarrow 79{,}5\%$.

3.3 ResNeXt-ify

Dùng **depthwise convolution** (trường hợp đặc biệt của grouped convolution, số group = số channel, phổ biến hoá bởi MobileNet/Xception) thay cho conv 3×3 thường trong bottleneck block, tương tự ý tưởng ResNeXt (nhiều group, mở rộng width). Depthwise conv giống phép weighted-sum theo channel của self-attention: chỉ trộn thông tin theo chiều không gian, kết hợp với conv 1×1 để tách biệt trộn-không-gian và trộn-channel, một đặc điểm Transformer cũng có. Tăng width từ 64 lên 96 (khớp Swin-T) để bù lại giảm capacity: $79{,}5\% \rightarrow 80{,}5\%$ (FLOPs tăng lên 5,3G).

3.4 Inverted bottleneck

Mọi block Transformer có "inverted bottleneck": hidden dimension của MLP rộng gấp 4 lần input dimension (ý tưởng phổ biến hoá bởi MobileNetV2). Áp dụng tương tự cho ConvNet (Hình 3a→3b): tuy tăng FLOPs ở lớp depthwise conv, nhưng **giảm mạnh FLOPs tổng thể** (do giảm FLOPs ở lớp conv 1×1 shortcut của khối downsampling), kết quả FLOPs giảm xuống 4,6G, và accuracy tăng nhẹ: $80{,}5\% \rightarrow 80{,}6\%$ (ở chế độ ResNet-200/Swin-B, bước này mang lợi ích lớn hơn: $81{,}9\% \rightarrow 82{,}6\%$, đồng thời giảm FLOPs).

3.5 Kernel lớn

Đặc điểm nổi bật của self-attention là receptive field toàn cục (global). Để tiến gần hơn, paper thử tăng kích thước kernel của depthwise conv. Điều kiện cần trước: **di chuyển depthwise conv lên trước** 1×1 conv (Hình 3b→3c, giống việc đặt khối MSA trước MLP trong Transformer: module phức tạp/kém hiệu quả (attention, kernel lớn) nên có ít channel, còn các lớp 1×1 dày đặc hiệu quả làm phần việc chính). Bước trung gian này giảm FLOPs xuống 4,1G nhưng **tạm thời giảm** accuracy xuống $79{,}9\%$. Sau đó tăng kernel size:

$3 \to 5 \to 7 \to 9 \to 11$, accuracy tăng từ $79{,}9\%$ (3×3) lên $80{,}6\%$ (7×7), FLOPs gần như không đổi; lợi ích **bão hoà tại** 7×7 (kernel 9×9, 11×11 không cải thiện thêm, hành vi này được xác nhận lại ở quy mô ResNet-200). Từ đây dùng depthwise conv 7×7 trong mọi block.

3.6 Micro design

- **ReLU $\to$ GELU**: thay activation, accuracy không đổi ($80{,}6\% \to 80{,}6\%$).
- **Giảm số activation**: một block Transformer chỉ có **một** activation (trong MLP), trong khi ConvNet thường gắn activation sau mỗi conv kể cả 1×1. Loại bỏ mọi GELU trừ một cái nằm giữa hai lớp 1×1 (mô phỏng đúng cấu trúc Transformer block): $+0{,}7$ điểm $\to 81{,}3\%$, **gần khớp Swin-T**.
- **Giảm số normalization**: Transformer block có ít lớp norm hơn. Bỏ 2 trong 3 lớp BatchNorm, chỉ giữ 1 lớp BN trước conv 1×1: $\to 81{,}4\%$, **đã vượt Swin-T (81,3%)**. (Thêm một lớp BN nữa ở đầu block được thử nhưng không cải thiện.)
- **BatchNorm $\to$ LayerNorm**: thay trực tiếp LN cho BN trong ResNet gốc (chưa hiện đại hoá) sẽ cho hiệu năng dưới tối ưu (paper dẫn [83]), nhưng sau khi đã qua các bước hiện đại hoá trên, mô hình huấn luyện ổn định với LN, hiệu năng tốt hơn chút: $\to 81{,}5\%$.
- **Lớp downsampling riêng**: ResNet gốc downsample ngay trong residual block (conv 3×3 stride 2); Swin dùng một lớp downsampling **riêng** giữa các stage. Thử conv 2×2 stride 2 riêng: huấn luyện **diverge bất ngờ**. Khảo sát thêm cho thấy cần **thêm các lớp LN** tại mọi nơi độ phân giải không gian thay đổi (trước mỗi lớp downsampling, sau stem, sau global average pooling cuối, giống Swin) để ổn định huấn luyện. Sau khi thêm: $\to 82{,}0\%$, **vượt rõ Swin-T (81,3%)**. Đây là mô hình cuối cùng, đặt tên **ConvNeXt**.

Ví dụ tính tay minh hoạ (tự tính, dựa trên số liệu kiến trúc ở Bảng 9 của paper): so số tham số của lớp stem đầu tiên giữa ResNet-50 và ConvNeXt-T, cả hai đều downsample ảnh input xuống đúng 56×56 (tức hệ số $4\times$, xác nhận ở cột "output size" của Bảng 9). ResNet: conv 7×7, $3 \to 64$ channel, số tham số (bỏ bias) $= 7 \times 7 \times 3 \times 64 = 49 \times 3 \times 64 = 9.408$. ConvNeXt-T: conv patchify 4×4, $3 \to 96$ channel, số tham số $= 4 \times 4 \times 3 \times 96 = 16 \times 3 \times 96 = 4.608$. Dù xuất ra **nhiều channel hơn** (96 so với 64, tăng 50%), lớp stem của ConvNeXt-T có **ít hơn một nửa** số tham số so với ResNet (4.608 so với 9.408), nhờ diện tích kernel nhỏ hơn nhiều ($4 \times 4 = 16$ so với $7 \times 7 = 49$), trong khi vẫn đạt đúng cùng hệ số downsample $4\times$; đây là một minh chứng số cụ thể cho lý do "patchify stem" rẻ hơn mà vẫn tương đương về hiệu năng (Mục 3.2).

Ví dụ tính tay thứ hai (tự tính): ước lượng FLOPs của riêng một lớp conv 1×1 "$384 \to 96$" (xuất hiện trong Hình 3 của paper, ở stage res4 có độ phân giải 14×14 theo Bảng 9). Số phép nhân-cộng (MAC) $\approx H \times W \times C_{in} \times C_{out} = 14 \times 14 \times 384 \times 96 = 196 \times 384 \times 96 \approx 7{,}23$ triệu MAC. So với FLOPs toàn mạng ConvNeXt-T ($\approx 4{,}5 \times 10^9$), một lớp 1×1 đơn lẻ này chỉ chiếm khoảng $0{,}16\%$ tổng FLOPs; cho thấy trực quan vì sao hàng chục lớp 1×1 nhỏ như vậy có thể cộng dồn thành vài GFLOPs trên một mạng sâu nhiều stage, nhưng từng lớp riêng lẻ rất rẻ, đúng với luận điểm của paper rằng "các lớp 1×1 dày đặc, hiệu quả làm phần việc chính" trong khi module đắt đỏ hơn (depthwise kernel lớn) chỉ cần ít channel.

3.7 Họ mô hình ConvNeXt

Chỉ khác số channel C và số block B mỗi stage (channel gấp đôi mỗi stage mới, theo đúng truyền thống ResNet/Swin):

Model	C (channel 4 stage)	B (block 4 stage)
ConvNeXt-T	$(96, 192, 384, 768)$	$(3, 3, 9, 3)$
ConvNeXt-S	$(96, 192, 384, 768)$	$(3, 3, 27, 3)$
ConvNeXt-B	$(128, 256, 512, 1024)$	$(3, 3, 27, 3)$
ConvNeXt-L	$(192, 384, 768, 1536)$	$(3, 3, 27, 3)$
ConvNeXt-XL	$(256, 512, 1024, 2048)$	$(3, 3, 27, 3)$

Huấn luyện ImageNet-1K: AdamW, lr 4×10^{-3}, 300 epoch, batch size 4096, weight decay 0,05, warmup 20 epoch (linear) + cosine decay, Mixup 0,8, Cutmix 1,0, RandAugment $(9, 0{,}5)$, Random Erasing 0,25, Label Smoothing 0,1, Stochastic Depth (tỉ lệ riêng theo model), Layer Scale khởi tạo 10^{-6}, EMA $0{,}9999$. Pretrain ImageNet-22K: 90 epoch, không dùng EMA. Fine-tune từ IN-22K về IN-1K: 30 epoch, lr 5×10^{-5}, layer-wise lr decay, weight decay 10^{-8}, batch 512. Một chi tiết thực nghiệm đáng chú ý (Phụ lục A.1): trong các thí nghiệm hiện đại hoá ở Mục 2 (nơi mô hình còn dùng BatchNorm ở giai đoạn đầu), **EMA bị tắt** vì paper nhận thấy dùng EMA "gây hại nghiêm trọng cho các mô hình có lớp BatchNorm".

4. Thí nghiệm và kết quả chính

4.1 ImageNet-1K, huấn luyện trực tiếp (Bảng 1, phần trên). So với Swin cùng quy mô: ConvNeXt-T 29M tham số/4,5G FLOPs/774,7 ảnh/s/**82,1%** top-1, vs Swin-T 28M/4,5G/757,9/**81,3%**; ConvNeXt-S 50M/8,7G/447,1/**83,1%** vs Swin-S 50M/8,7G/436,7/**83,0%**; ConvNeXt-B 89M/15,4G/292,1/**83,8%** vs Swin-B 88M/15,4G/286,6/**83,5%**. Ở độ phân giải 384^2: ConvNeXt-B 89M/45,0G/95,7 ảnh/s/**85,1%** vs Swin-B 88M/47,1G/85,1 ảnh/s/**84,5%**, tức ConvNeXt-B vượt Swin-B **+0,6 điểm accuracy** và **+12,5% throughput suy luận cao hơn** cùng lúc. ConvNeXt-L\protect\TU\textdollar@384^2$ đạt **85,5%**.

4.2 Pretrain ImageNet-22K rồi fine-tune ImageNet-1K (Bảng 1, phần dưới). ConvNeXt-XL\protect\TU\textdollar@384^2$ đạt **87,8%** top-1 (con số nêu ngay trong abstract), vượt EfficientNetV2-XL (87,3%, kiến trúc search tự động + module Squeeze-and-Excitation + progressive training) và vượt ViT-L/16 (86,8%). ConvNeXt-L\protect\TU\textdollar@384^2$ đạt 87,5% so với Swin-L\protect\TU\textdollar@384^2$ 87,3%. Paper nhấn mạnh: kết quả này phản bác quan điểm phổ biến rằng "Transformer có ít inductive bias hơn nên tận dụng pretraining quy mô lớn tốt hơn ConvNet": ConvNeXt pretrain trên IN-22K vẫn ngang hoặc tốt hơn Swin cùng kích cỡ.

4.3 Isotropic ConvNeXt vs ViT (Mục 3.3, Bảng 2). Kiểm tra liệu block ConvNeXt có tổng quát hoá được sang kiến trúc "isotropic" kiểu ViT (không downsample, giữ nguyên độ phân giải feature mọi tầng, ví dụ 14×14). Xây ConvNeXt-S/B/L isotropic cùng feature dimension với ViT-S/B/L (384/768/1024), độ sâu 18/18/36 để khớp tham số/FLOPs. Kết quả (224^2): ConvNeXt-S(iso) 79,7% vs ViT-S 79,8% (gần như hoà); ConvNeXt-B(iso) 82,0% vs ViT-B 81,8% (ConvNeXt nhích hơn, **và tốn ít memory huấn luyện hơn**: 7,7GB so với 9,1GB); ConvNeXt-L(iso) 82,6% vs ViT-L 82,6% (hoà về accuracy, nhưng ConvNeXt dùng 20,4GB memory huấn luyện so với 22,5GB của ViT-L).

4.4 COCO object detection/instance segmentation (Bảng 3, Mask R-CNN và Cascade Mask R-CNN, schedule 3x). ConvNeXt-T 262G FLOPs/25,6 FPS/AP^{box} 46,2/AP^{mask} 41,7 vs Swin-T 267G/23,1/46,0/41,6 (ConvNeXt thắng cả AP và FPS, ít FLOPs hơn). Ở quy mô lớn hơn với pretrain IN-22K (ký hiệu ‡): ConvNeXt-XL‡ đạt AP^{box} 55,2/AP^{mask} 47,7 (cao nhất trong bảng), so Swin-L‡ 53,9/46,7.

4.5 ADE20K semantic segmentation (Bảng 4, UperNet, multi-scale test). ConvNeXt-T 46,7 mIoU vs Swin-T 45,8; với pretrain IN-22K, ConvNeXt-XL‡ đạt 54,0 mIoU vs Swin-L‡ 53,5.

4.6 Robustness (Phụ lục B, Bảng 8). Test trực tiếp mô hình đã huấn luyện/fine-tune trên IN-1K lên các benchmark robustness (ImageNet-A/R/Sketch, ImageNet-C/$\bar{\text{C}}$), **không dùng module chuyên biệt hay fine-tune thêm nào**. ConvNeXt-XL (pretrain IN-22K, test 384^2) đạt 69,3%/68,2%/55,0% trên ImageNet-A/R/Sketch tương ứng, vượt mô hình Transformer "robust" chuyên dụng (RVT) trên một số benchmark.

4.7 Benchmark throughput trên A100 (Phụ lục E, Bảng 12). Dùng TF32 và layout "channel last" trên A100: ConvNeXt nhanh hơn Swin cùng FLOPs tới **~49%** (ví dụ ConvNeXt-T 1943,5 ảnh/s vs Swin-T 1325,6 ảnh/s, $+47\%$). Trên V100 (Bảng 1), lợi thế throughput nhỏ hơn nhưng vẫn luôn nghiêng về ConvNeXt.

4.8 Memory khi huấn luyện downstream task. Dù depthwise conv thường được biết là "chậm hơn và tốn memory hơn so với ConvNet chỉ dùng dense conv ở cùng FLOPs" (paper tự nêu), huấn luyện Cascade Mask-RCNN với backbone ConvNeXt-B chỉ tốn **17,4GB** peak memory (batch size 2/GPU) so với **18,5GB**

của Swin-B, tức ConvNeXt vẫn tiết kiệm memory hơn trên thực tế ở cấp độ toàn hệ thống.

5. Phân tích / ablation

Toàn bộ roadmap ở Mục 3 **chính là** một chuỗi ablation tuần tự (mỗi bước giữ nguyên mọi thứ khác, chỉ đổi một yếu tố). Bảng 10/11 của paper (Phụ lục C) cho số liệu chi tiết với **mean ± std qua 3 random seed** ở chế độ ResNet-50, giúp phân biệt cải thiện thật với nhiễu huấn luyện, ví dụ: "stage ratio" $79{,}36 \pm 0{,}07$, "ReLU→GELU" $80{,}62 \pm 0{,}14$, "fewer norms" $81{,}41 \pm 0{,}09$. Việc báo cáo độ lệch chuẩn minh bạch này giúp các bước cải thiện nhỏ (ví dụ ReLU→GELU, gần như không đổi) được phân biệt rõ với các bước cải thiện lớn, có ý nghĩa thống kê (ví dụ "fewer activations" $+0{,}7$ điểm).

So sánh hai quy mô (ResNet-50/Swin-T vs ResNet-200/Swin-B, Bảng 10 vs 11): cùng một chuỗi bước nhưng độ lớn hiệu ứng khác nhau theo quy mô mô hình:

- "Inverting dimensions": $+0{,}14$ điểm ở quy mô ResNet-50 ($80{,}50 \rightarrow 80{,}64$) nhưng $+0{,}79$ điểm ở quy mô ResNet-200 ($81{,}85 \rightarrow 82{,}64$), **lớn hơn đáng kể**.
- "Fewer norms": $+0{,}14$ điểm ở ResNet-50 nhưng $+0{,}46$ điểm ở ResNet-200, cũng lớn hơn ở mô hình lớn.
- Kernel size: ở ResNet-50, lợi ích bão hoà tại 7×7; ở ResNet-200, lợi ích bão hoà **sớm hơn, tại** 5×5 (không tăng thêm ở $7 \times 7, 9 \times 9, 11 \times 11$) - một khác biệt định tính thú vị giữa hai quy mô, cho thấy "kernel tối ưu" có thể phụ thuộc vào độ sâu/width mô hình, không phải một con số cố định tuyệt đối.

Tương tác giữa EMA và BatchNorm: phát hiện thực nghiệm rằng EMA (Exponential Moving Average weights) "gây hại nghiêm trọng" cho mô hình còn dùng BatchNorm, nên bị tắt trong toàn bộ pha hiện đại hoá (khi kiến trúc còn ở dạng BN) nhưng được dùng trở lại ở mô hình ConvNeXt cuối (dùng LayerNorm) khi huấn luyện ImageNet-1K từ đầu. Đây là một ví dụ cụ thể cho thấy các lựa chọn kỹ thuật huấn luyện không độc lập với kiến trúc, mà tương tác với nhau.

Phân tích hiệu quả mô hình (Mục 3.3, "Remarks on model efficiency"): paper tự đối chiếu với hiểu biết thông thường rằng depthwise conv kém hiệu quả hơn dense conv ở cùng FLOPs trên phần cứng thực, nhưng kết quả đo thực tế (throughput Bảng 1/12, memory Mục 4.4) cho thấy ConvNeXt **vẫn** nhanh hơn và tiết kiệm memory hơn Swin ở cấp độ toàn mô hình, nhờ "local computation" nói chung (cả ConvNeXt và Swin) có accuracy-FLOPs trade-off tốt hơn ViT thuần; paper nhấn mạnh **đây là hệ quả của ConvNet inductive bias, không liên quan trực tiếp đến cơ chế self-attention** của Transformer.

6. Giới hạn

Paper tự nêu (Phụ lục F, G):

- ConvNeXt có thể phù hợp hơn cho một số task, còn Transformer có thể linh hoạt hơn cho các task khác; ví dụ cụ thể paper đưa ra: multi-modal learning, nơi module cross-attention có thể tốt hơn để mô hình hoá tương tác đặc trưng giữa nhiều modality; Transformer cũng có thể linh hoạt hơn cho các task cần output rời rạc/thưa/có cấu trúc (discretized, sparse, hoặc structured outputs). Paper tự kết luận: "lựa chọn kiến trúc nên phù hợp với nhu cầu của từng task cụ thể, trong khi vẫn hướng tới sự đơn giản."
- Societal impact: các mô hình lớn (ViT, Swin, ConvNeXt đều đạt kết quả tốt nhất ở các biến thể rất lớn) kéo theo chi phí tính toán/carbon lớn; việc dùng dữ liệu công khai qui mô lớn (ImageNet-22K) cần một cách tiếp cận "cẩn trọng và trách nhiệm hơn" để tránh vấn đề data bias; paper tự nhận cần "điều tra thêm" (further investigation) về robustness/fairness của ConvNeXt so với Transformer, chưa có trong bài.

Người ghi chú nhận xét:

- Bản thân các thành phần riêng lẻ (depthwise conv, inverted bottleneck, patchify stem, LayerNorm trong CNN...) **không mới**; paper tự thừa nhận "các lựa chọn thiết kế này đều đã được nghiên cứu riêng lẻ

trong tài liệu ConvNet trong thập kỷ qua, nhưng chưa được kết hợp cùng nhau" (not novel even in the ConvNet literature, researched separately but not collectively). Đóng góp chính là quy trình thực nghiệm hệ thống + bộ kết quả tổng hợp, không phải một kỹ thuật hoàn toàn mới.

- Quá trình "hiện đại hoá" mang tính **trial-and-error có định hướng** (dẫn dắt bởi so sánh với Swin), không có một lý thuyết nền tảng giải thích trước **vì sao** mỗi thay đổi cải thiện hiệu năng (ví dụ paper không giải thích sâu cơ chế toán học vì sao thêm LN ở các vị trí downsampling lại chống được divergence, chỉ quan sát thực nghiệm). Người đọc cần cẩn trọng khi khái quát hoá các con số cụ thể (ví dụ "bão hoà tại 7×7") sang các bộ dữ liệu/kiến trúc rất khác ImageNet.
- Benchmark robustness (Bảng 8) chỉ so với một baseline Transformer "robust" cụ thể (RVT); không có so sánh với các kỹ thuật robustness khác (adversarial training kiểu PGD ở "ml basics 39", hay các phương pháp augmentation robustness khác), nên khó đánh giá ConvNeXt có thực sự robust "nội tại" hơn, hay chỉ tốt hơn trên đúng các test set được chọn.

7. Liên hệ với các bài khác

- **ml basics 07** (ResNet): kiến trúc gốc, điểm khởi đầu trực tiếp cho toàn bộ roadmap hiện đại hoá của bài này; mọi con số cải thiện trong Mục 3 đều so với baseline ResNet-50/ResNet-200.
- **ml basics 08** (VGG): paper trích dẫn trực tiếp VGGNet là nguồn "gold standard" của chiến lược xếp chồng kernel nhỏ 3×3 (Mục 2.5), điều mà ConvNeXt sau đó thay đổi bằng cách dùng kernel lớn hơn (7×7) cho depthwise conv.
- **llm content 29** (ViT): đối tượng so sánh và động lực trực tiếp xuyên suốt cả bài ("Roaring 20s" bắt đầu từ ViT); Mục 3.3 của ghi chú này (isotropic ConvNeXt vs ViT) đối chiếu trực tiếp hai kiến trúc trên cùng cấu hình.
- **pruning content 04** (Pruning Filters for Efficient ConvNets): cùng quan tâm tới hiệu quả tính toán của kiến trúc CNN, nhưng theo hướng khác hẳn (cắt tỉa một mạng đã có, thay vì thiết kế lại kiến trúc từ đầu); hai hướng có thể bổ sung nhau (cắt tỉa một ConvNeXt đã huấn luyện là một bài toán mở không được đề cập trong bài này).

8. Gợi ý áp dụng

ConvNeXt cho thấy nhiều lựa chọn thiết kế và quy trình huấn luyện có thể cải thiện ConvNet. Khi chọn backbone, nên đo chất lượng và độ trễ trên phần cứng đích thay vì xem FLOPs là đại diện hoàn hảo cho tốc độ.

9. Thí nghiệm gợi ý

1. **Tái tạo một phần roadmap trên CIFAR-10/100 với ResNet nhỏ:** đổi stage ratio và thay stem sang patchify (kernel nhỏ hơn, ví dụ 2×2 stride 2 do ảnh CIFAR chỉ 32×32), đo accuracy/FLOPs; kỳ vọng thấy xu hướng tương tự paper (cải thiện nhỏ) dù độ lớn hiệu ứng có thể khác do ảnh nhỏ hơn nhiều so với ImageNet.
2. **So sánh tốc độ/memory depthwise vs dense conv ở cùng FLOPs:** xây hai block nhỏ (dense conv 3×3 thường, và depthwise 3×3 + 1×1) khớp FLOPs, đo latency/memory thực tế; kiểm tra claim của paper rằng depthwise "chậm hơn, tốn memory hơn ở cùng FLOPs" có đúng ở quy mô nhỏ hay không.
3. **Sweep kernel size trên một CNN nhỏ kiểu ConvNeXt (CIFAR):** thử depthwise kernel $\{3, 5, 7, 9\}$ trên một mạng nhỏ vài block, đo accuracy; xem điểm bão hoà xuất hiện ở đâu (paper thấy 7×7 cho ResNet-50 regime, 5×5 cho ResNet-200 regime) với ảnh nhỏ hơn nhiều.
4. **Kiểm tra claim BN→LN gây suboptimal nếu đổi "thô":** trên một ResNet nhỏ CIFAR CHƯA hiện đại hoá, đổi trực tiếp BN sang LN, so với đổi BN sang LN **sau khi** đã áp các bước macro/micro design khác; kỳ vọng đổi "thô" cho kết quả kém hơn rõ, khớp với quan sát của paper.
5. **Kiểm tra độ nhạy với đổi độ phân giải:** huấn luyện một CNN nhỏ kiểu ConvNeXt (fully-convolutional) và một ViT nhỏ (có position embedding) cùng ở một độ phân giải (ví dụ 32×32),

rồi test trực tiếp ở độ phân giải khác (ví dụ 48×48) KHÔNG áp dụng kỹ thuật nội suy position embedding nào; đo mức giảm accuracy của từng loại, kiểm tra xem CNN fully-convolutional có "chịu đựng" tốt hơn ViT khi đổi độ phân giải hay không, một phép thử về ảnh có độ phân giải khác nhau.

10. Tóm tắt công thức / khái niệm

Bước	Thay đổi	Accuracy (ResNet-50/Swin-T regime)	FLOPs
Baseline	ResNet-50 gốc	76,1%	4,1G
+Training recipe	AdamW, 300 epoch, Mixup/Cutmix/RandAug...	78,8%	4,1G
Macro: stage ratio	$(3,4,6,3) \rightarrow (3,3,9,3)$	79,4%	4,53G
Macro: patchify stem	conv 7×7+maxpool $\rightarrow$ conv 4×4 stride 4	79,5%	4,42G
ResNeXt-ify	depthwise conv + width $64 \rightarrow 96$	80,5%	5,27G
Inverted bottleneck	MLP-style 4x expansion	80,6%	4,64G
Move up depthwise (tạm)	chuẩn bị cho kernel lớn	79,9%	4,07G
Kernel $\rightarrow 7 \times 7$	tăng từ 3×3, bão hoà tại 7×7	80,6%	4,15G
ReLU$\rightarrow$GELU		80,6%	4,15G
Ít activation hơn	chỉ 1 GELU/block	81,3%	4,15G
Ít norm hơn	chỉ 1 BN/block	81,4%	4,15G
BN$\rightarrow$LN		81,5%	4,46G
Downsampling riêng + LN ổn định	**ConvNeXt-T cuối cùng**	**82,0%**	4,49G
(so sánh) Swin-T		81,3%	4,50G

Kết quả nổi bật khác	Giá trị
ConvNeXt-XL (IN-22K pretrain, 384^2)	**87,8%** top-1 ImageNet-1K
ConvNeXt-B\protect\TU\textdollar@384^2$ vs Swin-B\protect\TU\textdollar@384^2\protect\TU\textdollar	$+0{,}6$ điểm acc, $+12{,}5\%$ throughput
COCO AP^{box}, ConvNeXt-XL‡	55,2 (cao nhất trong Bảng 3)
ADE20K mIoU, ConvNeXt-XL‡	54,0
Throughput A100 (TF32, channel-last)	tới $+49\%$ so với Swin cùng FLOPs
Robustness IN-A/R/Sketch, ConvNeXt-XL	69,3% / 68,2% / 55,0%

11. Kết luận ngắn

Bằng cách hiện đại hoá từng bước một ResNet chuẩn theo hướng thiết kế của Swin Transformer (macro design, ResNeXt-ify, inverted bottleneck, kernel lớn, micro design), không thêm bất kỳ cơ chế attention nào, paper tạo ra ConvNeXt, một họ ConvNet thuần cạnh tranh ngang ngửa hoặc vượt Swin Transformer trên classification (82,0% vs 81,3% ở quy mô T, và 87,8% khi scale lên với IN-22K pretraining), object detection/segmentation (COCO), semantic segmentation (ADE20K), và cả robustness, đồng thời nhanh hơn và tiết kiệm memory hơn khi huấn luyện/suy luận trên GPU hiện đại. Kết luận cốt lõi: phần lớn lợi thế được gán cho "self-attention" trong các so sánh ViT/Swin-vs-ConvNet trước đó, thực ra đến từ công thức huấn luyện hiện đại và các lựa chọn thiết kế kiến trúc không gắn riêng với attention, một thông điệp đáng cân nhắc khi đánh giá bất kỳ tuyên bố "kiến trúc X vượt trội vì cơ chế Y" trong các so sánh kiến trúc tương tự.

Nguồn gốc: https://arxiv.org/abs/2201.03545

Bài 39

Towards Deep Learning Models Resistant to Adversarial Attacks (2017)

Tác giả: Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, Adrian Vladu (MIT).
Venue / arXiv: arXiv:1706.06083 (v4, 04/09/2019; nộp lần đầu 06/2017); paper không nêu venue hội nghị cụ thể trong phần văn bản đã trích xuất (không thấy dòng ghi tên hội nghị trên trang đầu bản PDF đọc được).
Nguồn: https://arxiv.org/abs/1706.06083
Chủ đề: nhìn adversarial robustness qua lăng kính robust optimization (bài toán min-max/saddle point), từ đó đề xuất Projected Gradient Descent (PGD) như một "universal first-order adversary" và PGD adversarial training như một phương pháp huấn luyện mạng neural chống chịu được một phạm vi rộng các kiểu tấn công đối kháng (ℓ_∞-bounded).
Vị trí trong lộ trình: Đọc sau bài 23 về FGSM và bài 24 về adversarial examples.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận.

1. Bối cảnh

Deep neural network dễ bị **adversarial example**: input gần như không phân biệt được bằng mắt người so với input gốc, nhưng bị mô hình phân loại sai, thường với độ tin cậy cao. Hiện tượng này (phát hiện lần đầu bởi Szegedy et al., "ml basics 24") đặt ra vấn đề an toàn nghiêm trọng khi mô hình được dùng trong hệ thống quan trọng về bảo mật (xe tự hành, nhận diện khuôn mặt, phát hiện malware).

Trước bài báo này, nhiều cơ chế phòng thủ đã được đề xuất (defensive distillation, feature squeezing, các phương pháp phát hiện adversarial example), nhưng paper nhận xét: đây chỉ là "những bước đầu quan trọng" (important first steps), **không đi kèm hiểu biết rõ ràng về bảo đảm mà chúng cung cấp**. Không thể chắc một attack đã tìm ra "ví dụ đối kháng tệ nhất", hay một cơ chế phòng thủ có ngăn được một lớp tấn công được định nghĩa rõ ràng nào không. Điều này khiến việc đánh giá đúng mức độ an toàn của một mô hình rất khó khăn.

Câu hỏi nghiên cứu trung tâm: **làm sao huấn luyện deep neural network chống chịu được input đối kháng, với một bảo đảm an toàn cụ thể, rõ ràng (không chỉ chống lại một vài attack cụ thể đã biết)?**

2. Ý tưởng chính

Paper đóng khung adversarial robustness như một bài toán **robust optimization** dạng **saddle point (min-max)** (công thức 2.1):

$$\min_{\theta} \rho(\theta), \quad \text{với } \rho(\theta) = \mathbb{E}_{(x,y)\sim D}\left[\max_{\delta \in S} L(\theta, x+\delta, y)\right]$$

với $S \subseteq \mathbb{R}^d$ là tập nhiễu được phép (ví dụ ℓ_∞-ball bán kính ε quanh x, bài này tập trung vào threat model này). Công thức này tách bài toán thành hai nửa có ý nghĩa rõ ràng:

- **Bài toán max bên trong** = tìm ví dụ đối kháng (attack): với x, θ cố định, tìm $\delta \in S$ làm loss lớn nhất.
- **Bài toán min bên ngoài** = huấn luyện mô hình chống chịu (defense): tìm θ làm "adversarial loss" (giá trị của bài toán max bên trong) nhỏ nhất.

Khung nhìn này **thống nhất** cả attack và defense đã biết trước đó: FGSM (một bước, tuyến tính hoá bài toán max) và các biến thể là các cách xấp xỉ bài toán max bên trong; adversarial training (thêm ví dụ đối kháng vào dữ liệu huấn luyện) chính là tối ưu hoá trực tiếp bài toán saddle point này.

Phát hiện cốt lõi của bài báo: dù bài toán max bên trong **phi lõm** (non-concave, nhiều local maxima), thực nghiệm cho thấy nó **"dễ xử lý" một cách bất ngờ**: Projected Gradient Descent (PGD), khởi động lại ngẫu nhiên nhiều lần, luôn tìm được các local maxima có **giá trị loss tập trung rất gần nhau** (không có outlier cực đoan). Điều này gợi ý PGD là một **"universal first-order adversary"**: nhóm tác giả xem PGD là đối thủ đại diện cho các tấn công dùng thông tin đạo hàm bậc một; đây là nhận định dựa trên thực nghiệm, không phải bảo đảm trước mọi tấn công. Kết hợp với việc tăng **model capacity** (một yếu tố quan trọng khác), paper huấn luyện được mạng MNIST/CIFAR10 chống chịu một phạm vi rộng các attack, đạt mức robustness mà bài báo tự nhận là "cao nhất từng đạt được" tính đến thời điểm viết bài.

3. Phương pháp chi tiết

3.1 FGSM và PGD như hai cách giải bài toán max bên trong

FGSM (Fast Gradient Sign Method, Goodfellow et al., "ml basics 23"): một bước tuyến tính hoá,

$$x^{adv} = x + \varepsilon \operatorname{sgn}(\nabla_x L(\theta, x, y))$$

PGD: lặp nhiều bước, mỗi bước là gradient ascent trên loss (tương đương gradient descent trên "negative loss"), rồi **chiếu (project) về lại tập** $x + S$ **cho phép**:

$$x^{t+1} = \Pi_{x+S}(x^t + \alpha \operatorname{sgn}(\nabla_x L(\theta, x^t, y)))$$

PGD khởi động từ một điểm ngẫu nhiên trong ℓ_∞-ball quanh x (không khởi động từ x gốc), và được paper coi là attack **"hoàn chỉnh" (complete)** cho bài toán max bên trong theo nghĩa first-order: nó khai thác tối đa thông tin đạo hàm bậc một tại mỗi bước.

3.2 Khảo sát "cảnh quan" (landscape) của bài toán max bên trong (Mục 3.1)

Để kiểm tra PGD có đáng tin không, paper chạy PGD khởi động lại từ 10^5 **điểm ngẫu nhiên** trong ℓ_∞-ball quanh cùng một ví dụ MNIST/CIFAR10 (cả trên mạng huấn luyện chuẩn và mạng đã adversarial training), quan sát:

- Loss tăng theo cách nhất quán và **bão hoà nhanh** (vài chục iteration) ở mọi điểm khởi động (Hình 1).
- Phân phối giá trị loss tại các local maxima **tập trung rất chặt, không có outlier cực đoan** (Hình 2, xác nhận qua 10^5 restart).
- Khoảng cách ℓ_2 và góc giữa các cặp local maxima gần với giá trị kỳ vọng của hai điểm ngẫu nhiên trong ℓ_∞-ball (góc gần $90°$); loss trên đoạn thẳng nối hai local maxima là **lồi**, đạt max tại hai đầu, giảm theo một hệ số hằng ở giữa, nhưng toàn đoạn vẫn cao hơn một điểm ngẫu nhiên.

Những quan sát này cùng chỉ về một cấu trúc "cảnh quan" tương đối đều, hỗ trợ giả thuyết PGD là adversary phổ quát cho các phương pháp first-order.

3.3 Định lý Danskin và tính hợp lệ của adversarial training (Mục 3.3, Phụ lục A)

Huấn luyện bằng SGD cần một descent direction cho $\rho(\theta)$. Cách tự nhiên: tính gradient của loss **tại điểm tối ưu của bài toán max bên trong** (tức tại ví dụ đối kháng). **Định lý Danskin** (một định lý cổ điển trong optimization, phát biểu và chứng minh lại ở Phụ lục A) khẳng định điều này **đúng về lý thuyết**: nếu $\delta^*(\theta)$ là tập nghiệm tối ưu của $\max_{\delta \in S} g(\theta, \delta)$, và tập này chỉ gồm một điểm, thì $\nabla\phi(\theta) = \nabla_\theta g(\theta, \delta^*_\theta)$, với $\phi(\theta) =$

$\max_\delta g(\theta, \delta)$. Hệ quả (Corollary A.2): $-\nabla_\theta L(\theta, x+\delta, y)$ tại một nghiệm (xấp xỉ) tối ưu δ là một descent direction hợp lệ cho $\phi(\theta)$.

Paper **tự thừa nhận hai vấn đề kỹ thuật** khi áp định lý này vào thực tế: (1) hàm loss **không liên tục khả vi** do ReLU và max-pooling, nhưng tập điểm gián đoạn có đo (measure) bằng 0, nên paper cho rằng trong thực tế sẽ không gặp phải; (2) bài toán max bên trong **phi lõm**, nên PGD chỉ tìm được **local** maximum, không phải global maximum như định lý yêu cầu nghiêm ngặt. Paper lập luận (không chứng minh chặt) rằng nếu coi $S' \subseteq S$ là một vùng nhỏ quanh local maximum nơi nó là global maximum **trong vùng đó**, định lý vẫn áp dụng được cục bộ, nên gradient tại local maximum "có khả năng cao" vẫn là một descent direction hữu ích, và thực nghiệm (Hình 5, loss giảm ổn định suốt huấn luyện) xác nhận điều này hoạt động trong thực tế.

3.4 Thiết lập huấn luyện cụ thể

MNIST: mạng 2 lớp conv (32, 64 filter), mỗi lớp theo sau bởi max-pool 2×2, rồi FC 1024 unit. $\varepsilon = 0{,}3$. PGD huấn luyện: 40 iteration, step size $0{,}01$ (bước theo dấu gradient trong norm ℓ_∞, để đơn giản hoá việc chọn step size).

Ví dụ tính tay minh hoạ (tự tính), dựa trực tiếp trên hai số này: nếu mọi bước PGD (giả định xấu nhất) đều dịch chuyển cùng hướng dấu gradient, sau 40 bước độ dịch chuyển tối đa tích lũy $= 40 \times 0{,}01 = 0{,}4$, **vượt** $\varepsilon = 0{,}3$. Điều này cho thấy toán tử chiếu Π_{x+S} là **thực sự cần thiết** (không chỉ hình thức): với số bước và step size đã chọn, PGD hoàn toàn có khả năng "muốn" đi ra ngoài ℓ_∞-ball nếu không bị chiếu lại, và việc chọn $40 \times 0{,}01 > \varepsilon$ đảm bảo thuật toán có đủ "sải bước" để chạm tới biên của ball từ bất kỳ điểm khởi đầu nào bên trong, đúng như paper tự giải thích khi chọn step size cho đánh giá CIFAR10 ($2{,}5\varepsilon/100$ cho 100 bước, "để đảm bảo có thể chạm tới biên của ε-ball từ bất kỳ điểm khởi đầu nào bên trong nó, và vẫn cho phép di chuyển trên biên").

CIFAR10: kiến trúc ResNet ("ml basics 07"), augment bằng random crop/flip và chuẩn hoá ảnh; bản "wide" nhân rộng width $\times 10$ ($16, 160, 320, 640$ filter mỗi 5 residual unit), đạt $95{,}2\%$ clean accuracy khi huấn luyện tự nhiên. $\varepsilon = 8$ (trên thang pixel $[0, 255]$). PGD huấn luyện: 7 bước, step size 2; đánh giá với adversary mạnh hơn: 20 bước (các lựa chọn hyperparameter khác không giảm accuracy đáng kể thêm).

4. Thí nghiệm và kết quả chính

4.1 MNIST ($\varepsilon = 0{,}3$, Bảng 1). Mạng huấn luyện tự nhiên đạt $99{,}2\%$ accuracy sạch nhưng rơi xuống $6{,}4\%$ khi bị tấn công FGSM, minh chứng rõ mạng chuẩn **không hề robust**. Mạng đã PGD-adversarial-train:

Attack	Steps	Restart	Nguồn	Accuracy
Natural	-	-	-	98,8%
FGSM	-	-	A (white-box)	95,6%
PGD	40	1	A	93,2%
PGD	100	1	A	91,8%
PGD	40	20	A	90,4%
PGD	**100**	**20**	**A**	**89,3%** (mạnh nhất, white-box)
CW+ ($\kappa = 50$)	40	1	A	93,9%
FGSM/PGD/CW	-	-	A' (black-box, bản copy độc lập)	95,7%-97,0%
FGSM/PGD/CW+	-	-	B (kiến trúc khác, Tramer et al.)	95,4%-96,4%

Khớp đúng con số "**hơn 89%**" (white-box mạnh nhất) và "**hơn 95%**" (black-box) nêu trong abstract.

4.2 CIFAR10 ($\varepsilon = 8$, Bảng 2).

Attack	Steps	Nguồn	Accuracy
Natural	-	-	87,3%
FGSM	-	A	56,1%
PGD	7	A	50,0%
PGD	**20**	**A**	**45,8%** (mạnh nhất, white-box, abstract nêu tròn "46%")
CW	30	A	46,8%
FGSM/PGD	- /7	A' (black-box copy độc lập)	64,2%-67,0%
FGSM/PGD	- /7	A_{nat} (bản huấn luyện tự nhiên)	85,6%-86,0%

Khớp đúng "**hơn 64%**" cho black-box nêu trong abstract. Paper tự nhận CIFAR10 "còn xa mức thỏa đáng" (far from satisfactory) so với MNIST, nhưng là "cải thiện đáng kể" so với mạng không được huấn luyện đối kháng (gần như 0% dưới PGD, xem Mục 5).

4.3 Độ nhạy theo ε và attack ℓ_2 (Hình 6). Với ε nhỏ hơn mức dùng khi huấn luyện, accuracy bằng hoặc cao hơn, như kỳ vọng. MNIST có **sụt giảm mạnh** ngay khi ε vượt nhẹ mức huấn luyện (do các "threshold filter" học được bị tinh chỉnh khớp đúng giá trị ε huấn luyện, xem Mục 5 và Phụ lục C); CIFAR10 suy giảm **mượt hơn**.

Phát hiện quan trọng (và tự phê bình) về attack ℓ_2 trên MNIST: PGD **không tìm được** adversarial example cho mô hình MNIST dù ε_{ℓ_2} lớn (ví dụ $4{,}5$), ban đầu có vẻ là một kết quả robustness rất mạnh. Nhưng paper (ở chính bản v4 đang đọc) tự dẫn **công trình sau đó** ([20],[25]) cho thấy **PGD đã overestimate** độ robust ℓ_2 của mô hình này: các "threshold filter" học được **che (mask) gradient**, khiến PGD (dựa vào gradient) không tìm ra được điểm yếu thật; một attack không cần gradient (decision-based attack) cho thấy mô hình **dễ bị tấn công ℓ_2 hơn nhiều** so với con số PGD gợi ý. Đây là một ví dụ cụ thể, được paper tự công khai, về nguy cơ **gradient masking** làm sai lệch đánh giá robustness nếu chỉ dùng attack dựa-gradient.

5. Phân tích / ablation

5.1 Model capacity (Mục 4, Hình 4, Bảng CIFAR10 tái hiện). So sánh Simple vs Wide ResNet (CIFAR10) qua 3 chế độ huấn luyện:

Huấn luyện	Natural acc. (Simple/Wide)	FGSM-attacked acc.	PGD-attacked acc.	Training loss cuối (Simple/Wide)
Standard	92,7% / 95,2%	27,5% / 32,7%	**0,8% / 3,5%**	0,00357 / 0,00371
FGSM	87,4% / 90,3%	90,9% / 95,1% (cao hơn cả natural, do "label leaking")	**0,0% / 0,0%**	0,0115 / 0,00557
PGD	79,4% / 87,3%	51,7% / 56,1%	**43,7% / 45,8%**	**1,11 / 0,0218**

Quan sát:

- **Mạng huấn luyện chuẩn gần như hoàn toàn vô dụng dưới PGD** ($0{,}8\%$-$3{,}5\%$), xác nhận mức độ nghiêm trọng của vấn đề.
- **Huấn luyện bằng FGSM KHÔNG tăng robustness thật** với ε lớn: mô hình overfit vào đúng kiểu nhiễu FGSM ("label leaking" [18]), accuracy dưới FGSM thậm chí **cao hơn** accuracy tự nhiên (một dấu hiệu bất thường, không phải robustness thật), nhưng dưới PGD rơi về **đúng** $0{,}0\%$.

- **Chỉ PGD training mới cho robustness thật, đáng kể** (dù chưa hoàn hảo): $43{,}7\%$-$45{,}8\%$ dưới PGD, đồng thời vẫn giữ accuracy sạch hợp lý ($79{,}4\%$-$87{,}3\%$).
- **Capacity giúp cả khi chỉ huấn luyện tự nhiên** (robustness một-bước tăng theo capacity, rõ hơn ở ε nhỏ) và **giúp rõ rệt hơn khi huấn luyện đối kháng** (Wide luôn nhích hơn Simple ở mọi cột).
- **Mạng capacity nhỏ huấn luyện với PGD có thể "sụp" về một lớp cố định** (không học được gì có ý nghĩa), đánh đổi hoàn toàn accuracy sạch để "có vẻ" robust.

Ví dụ tính tay minh hoạ (tự tính): so sánh training loss cuối của PGD training giữa Simple ($1{,}11$) và Wide ($0{,}0218$) với loss của một bộ phân loại 10 lớp đoán ngẫu nhiên hoàn toàn, $\ln(10) \approx 2{,}303$. Simple đạt $1{,}11/2{,}303 \approx 0{,}482$ (khoảng **48% của loss ngẫu nhiên**, cho thấy mạng nhỏ vẫn "vật lộn" rõ rệt với bài toán max bên trong dù không sụp hoàn toàn về ngẫu nhiên); Wide đạt $0{,}0218/2{,}303 \approx 0{,}0095$ (chỉ **xấp xỉ 1% của loss ngẫu nhiên**, tức giải bài toán max bên trong gần như triệt để). Khoảng cách gần $50\times$ về tỉ lệ này cho thấy rõ "giá trị bài toán saddle point giảm mạnh khi tăng capacity" là một hiệu ứng có độ lớn thực sự đáng kể, không phải khác biệt nhỏ. (Kiểm tra chéo: giá trị Natural $87{,}3\%$ của cột Wide/PGD ở bảng trên khớp đúng với dòng "Natural: 87,3%" trong Bảng 2 ở Mục 4.2, xác nhận hai bảng của paper nhất quán với nhau, không có mâu thuẫn số liệu.)

5.2 Transferability (Phụ lục B, Bảng 3-5). Qua ma trận transfer accuracy đầy đủ (Simple/Wide $\times$ standard/FGSM/PGD training, 6×6): **adversary mạnh hơn (PGD > FGSM > standard) khi dùng để huấn luyện làm giảm transferability** của attack từ các mô hình khác tới nó; **capacity cao hơn cũng làm giảm transferability**. Cả hai xu hướng nhất quán với lập luận ở Mục 4, được xác nhận qua tương quan góc gradient giữa hai mạng giảm khi capacity/độ mạnh adversary tăng.

5.3 Soi kiến trúc mô hình MNIST đã robust (Phụ lục C). Mạng nhỏ đủ để quan sát trực tiếp tham số: lớp conv đầu tiên của mạng robust chỉ dùng **3 trong 32 filter**, mỗi filter chỉ có **đúng một trọng số khác không**, khiến phép conv suy biến thành một phép scale ảnh gốc; kết hợp bias và ReLU, kết quả là một **"thresholding filter"** $\mathrm{ReLU}(\alpha x - \beta)$, vốn **tự động bất khả xâm phạm** với nhiễu trên pixel có giá trị $< \beta - \varepsilon$. Lớp conv thứ hai: trọng số thưa, dải giá trị rộng hơn mạng chuẩn, chỉ các filter tương tác với 3 channel "sống" ở trên là có ý nghĩa. Lớp softmax cuối: trọng số tương tự giữa 3 mạng (chuẩn, và hai bản robust độc lập "public"/"secret" dùng cho thử thách công khai), nhưng **class bias khác xa 0 và giống nhau một cách có hệ thống** giữa hai mạng robust, gợi ý một số lớp vốn dễ bị tấn công hơn, mạng tự học cách "thận trọng" hơn khi dự đoán các lớp đó. Paper nhấn mạnh: **không hề hard-code** các cơ chế này; khi thử **tự tay** đưa các "mẹo" này vào mạng (không qua adversarial training), **không thành công**, một PGD adversary đơn giản vẫn phá được toàn bộ test set.

6. Giới hạn

Paper tự nêu:

- CIFAR10 "còn xa mức thỏa đáng" (far from satisfactory); paper tin rằng kết quả có thể cải thiện thêm bằng cách tiếp tục tăng capacity, nhưng **không chứng minh** điều này trong bài, chỉ nêu như hướng tương lai.
- Chỉ nghiên cứu threat model ℓ_∞-bounded; paper tự nhận "các khái niệm tương đồng tri giác toàn diện hơn là một hướng nghiên cứu quan trọng trong tương lai", chưa làm trong bài.
- Danskin's theorem **không thỏa mãn chính xác** giả định (tính liên tục khả vi bị vi phạm bởi ReLU/max-pool; chỉ tìm được local, không phải global, maximizer), chỉ được lập luận là "vẫn nên đúng trong thực tế", không có chứng minh hình thức đầy đủ cho trường hợp phi lõm.
- Đánh giá ℓ_2-robustness trên MNIST bằng PGD **tự bị phát hiện là sai lệch** (overestimate) do gradient masking, phải dựa vào công trình sau đó dùng attack không-gradient để sửa lại kết luận; đây là một lời tự phê bình công khai hiếm thấy, đáng chú ý khi đọc các con số robustness của paper.
- Khi so sánh với công trình liên quan (Shaham et al. 2018), paper chỉ ra một lỗi cụ thể: báo cáo 70% accuracy tại $\varepsilon = 0{,}7$ là không đáng tin, vì một adversary được phép làm nhiễu mỗi pixel hơn $0{,}5$ hoàn

toàn có thể tạo ra một ảnh xám đều để đánh lừa **bất kỳ** classifier; đây là minh chứng cho tầm quan trọng của việc chọn ε và attack hợp lý khi đánh giá.

Người ghi chú nhận xét:

- PGD training tốn chi phí tính toán **gấp nhiều lần** huấn luyện chuẩn (mỗi batch cần 7-40 forward+backward pass phụ để tạo ví dụ đối kháng, tuỳ dataset), nhưng paper không định lượng rõ chi phí wall-clock tương đối trong văn bản chính.
- Bộ attack dùng để đánh giá (FGSM, PGD, CW, black-box transfer) phản ánh **state-of-the-art tại thời điểm 2017**; các attack mạnh hơn phát triển sau đó (ví dụ AutoAttack và các kỹ thuật phá gradient-masking) không có trong bài này, nên các con số $89\%/46\%$ nên được hiểu là "chống chịu được bộ attack đã biết tại thời điểm viết bài", không phải một chặn an toàn vĩnh viễn, đúng với đúng tinh thần thận trọng mà chính paper cũng nhấn mạnh khi phê bình công trình khác.
- Thí nghiệm capacity scale trên MNIST (Hình 4, cột trái) chỉ trình bày dưới dạng đồ thị, không có bảng số cụ thể trong văn bản chính; vì vậy ghi chú này không trích số liệu MNIST-theo-capacity, chỉ mô tả định tính (xem Mục 5), tránh suy diễn số liệu không có trong text.

7. Liên hệ với các bài khác

- **ml basics 23** (FGSM, Goodfellow et al.): phương pháp attack một-bước mà PGD cải tiến trực tiếp (PGD = phiên bản lặp nhiều bước, có khởi động ngẫu nhiên, của chính FGSM); paper này trích dẫn trực tiếp ([11]) và chỉ ra rõ vì sao FGSM-training không đủ (Mục 4, "FGSM adversaries don't increase robustness for large ε").
- **ml basics 24** (Intriguing Properties of Neural Networks, Szegedy et al.): paper phát hiện hiện tượng adversarial example lần đầu, được trích dẫn ([28]) như một trong các công trình đặt nền cho toàn bộ hướng nghiên cứu này.
- **ml basics 07** (ResNet): kiến trúc cơ sở dùng cho thí nghiệm CIFAR10 (cả bản gốc và bản "wide" $\times 10$ width).
- **ml basics 36** (Deep Ensembles): dùng chính FGSM (được trích dẫn qua ref [18] trong bài Deep Ensembles) như một kỹ thuật "làm trơn" (smoothing) phân phối dự đoán trong huấn luyện ensemble, một ứng dụng khác của cùng công cụ FGSM mà bài PGD này phân tích sâu hơn về mặt lý thuyết robust optimization.

8. Gợi ý áp dụng

Huấn luyện đối kháng và đánh giá robustness phải cùng xác định threat model, ngân sách nhiễu và tấn công sử dụng. Cần đo cả độ chính xác trên dữ liệu sạch và độ bền trước tấn công, đồng thời kiểm tra nguy cơ che gradient.

9. Thí nghiệm gợi ý

1. **Tái tạo thí nghiệm capacity trên MNIST nhỏ:** huấn luyện mạng CNN nhỏ (như kiến trúc 2-conv-layer của paper) ở vài mức capacity, với 3 chế độ (natural/FGSM/PGD training, $\varepsilon = 0{,}3$), đo accuracy dưới FGSM và PGD; kỳ vọng tái hiện mẫu hình "PGD training mới cho robustness thật, FGSM training không đủ với ε lớn".
2. **Quan sát "label leaking" trực tiếp:** huấn luyện một mạng nhỏ bằng FGSM-adversarial-training, so sánh accuracy dưới FGSM attack (kỳ vọng cao bất thường, thậm chí cao hơn accuracy tự nhiên) với accuracy dưới PGD attack (kỳ vọng sụp mạnh), minh hoạ cụ thể hiện tượng được mô tả ở Mục 4/5.
3. **Soi filter của mạng robust (tái tạo Phụ lục C ở quy mô nhỏ):** sau khi huấn luyện mạng MNIST robust, trực quan hoá trọng số lớp conv đầu tiên, kiểm tra có bao nhiêu filter "suy biến" thành thresholding filter (ít trọng số khác 0); so sánh với mạng huấn luyện tự nhiên.

4. **Kiểm tra gradient masking bằng một attack không-gradient đơn giản:** với mạng robust ở thí nghiệm 1, thử một attack ngẫu nhiên/tìm kiếm cục bộ không dùng gradient (ví dụ random search trong ℓ_∞-ball), so sánh tỉ lệ thành công với PGD; nếu attack không-gradient tìm được nhiều lỗi hơn PGD trên cùng ngân sách truy vấn, đó là dấu hiệu gradient masking, tái hiện tinh thần phát hiện ở Mục 4.3.
5. **Đo chi phí PGD training thực tế:** trên một mạng nhỏ, đo wall-clock time huấn luyện 1 epoch với PGD training (7, 20, 40 bước) so với huấn luyện chuẩn; định lượng hệ số chậm đi mà paper không nêu rõ, hữu ích để ước lượng tính khả thi khi áp cho một mô hình lớn hơn.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Ghi chú
Saddle point formulation	$\min_\theta \mathbb{E}_{(x,y)}[\max_{\delta\in S} L(\theta, x+\delta, y)]$	công thức (2.1), trung tâm toàn bài
FGSM	$x + \varepsilon\,\mathrm{sgn}(\nabla_x L)$	một bước, tuyến tính hoá
PGD	$x^{t+1} = \Pi_{x+S}(x^t + \alpha\,\mathrm{sgn}(\nabla_x L))$	nhiều bước, "universal first-order adversary"
Định lý Danskin	$\nabla\phi(\theta) = \nabla_\theta g(\theta, \delta^*_\theta)$ khi $\delta^*(\theta)$ là singleton	cơ sở lý thuyết cho adversarial training
MNIST setup	$\varepsilon = 0{,}3$, PGD train 40 bước, step 0,01	mạng 2 conv (32,64) + FC 1024
CIFAR10 setup	$\varepsilon = 8$, PGD train 7 bước, step 2; eval 20 bước	ResNet, bản Wide $\times 10$ width
Kết quả nổi bật MNIST	$89{,}3\%$ dưới PGD mạnh nhất (white-box); $> 95\%$ black-box	Bảng 1
Kết quả nổi bật CIFAR10	$45{,}8\%$ dưới PGD 20 bước (white-box); $> 64\%$ black-box	Bảng 2
Hiện tượng capacity	mạng nhỏ PGD-train có thể sụp về dự đoán hằng số	Mục 4, Hình 4

11. Kết luận ngắn

Bằng cách đóng khung adversarial robustness như một bài toán saddle point (min-max) theo robust optimization, paper chỉ ra PGD, qua khảo sát thực nghiệm cấu trúc cảnh quan loss, hoạt động như một "universal first-order adversary", và rằng huấn luyện chống chịu PGD (kết hợp capacity mô hình đủ lớn) cải thiện robustness trong các threat model được khảo sát, đạt $89{,}3\%$ trên MNIST và $45{,}8\%$ trên CIFAR10 dưới attack PGD mạnh nhất được thử, cùng robustness cao hơn với black-box/transfer attack. Paper cũng tự phát hiện và công khai một hạn chế quan trọng: đánh giá ℓ_2-robustness bằng PGD có thể bị gradient masking đánh lừa, một bài học về phương pháp luận đánh giá robustness vẫn còn giá trị. PGD adversarial training từ bài này trở thành baseline/thước đo chuẩn cho rất nhiều nghiên cứu robustness sau đó.

Nguồn gốc: https://arxiv.org/abs/1706.06083

Bài 40

UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction (2018)

Tác giả: Leland McInnes, John Healy, James Melville (Tutte Institute for Mathematics and Computing).
Venue / arXiv: arXiv:1802.03426 (v3, 18/09/2020; nộp lần đầu 02/2018, bản PDF đọc được ghi ngày hoàn thiện "September 21, 2020" trên trang đầu); paper không nêu venue hội nghị hoặc tạp chí cụ thể trong phần văn bản đọc được.
Nguồn: https://arxiv.org/abs/1802.03426
Chủ đề: một thuật toán giảm chiều dữ liệu (dimension reduction) tổng quát, xây dựng trên nền lý thuyết hình học Riemann và tô pô đại số (fuzzy simplicial sets), cạnh tranh với t-SNE về chất lượng trực quan hoá nhưng nhanh hơn, scale tốt hơn, và không giới hạn số chiều embedding đầu ra.
Vị trí trong lộ trình: Bài cuối trình bày UMAP, một công cụ giảm chiều bổ sung cho các phương pháp phân tích biểu diễn.

> Ghi chú học tập và diễn giải bằng tiếng Việt, không phải bản dịch. Các phép tính được đánh dấu "tự tính" là phép tính bổ sung; các liên hệ ứng dụng là suy luận. Phần lý thuyết phạm trù và chứng minh phụ lục được diễn giải ở mức khái niệm, không tái hiện đầy đủ.

1. Bối cảnh

Giảm chiều dữ liệu (dimension reduction) là một kỹ thuật nền tảng trong khoa học dữ liệu, dùng cho hai mục đích chính: trực quan hoá (visualization, đưa dữ liệu nhiều chiều về 2D/3D để nhìn bằng mắt) và tiền xử lý (preprocessing) cho các pipeline machine learning khác. Paper mở đầu bằng cách chia các thuật toán giảm chiều hiện có thành hai nhóm (phân loại của chính paper, Section 1):

- Nhóm bảo toàn cấu trúc khoảng cách toàn cục (pairwise distance toàn bộ dữ liệu): PCA, MDS, Sammon mapping.
- Nhóm ưu tiên bảo toàn cấu trúc cục bộ (local distance) hơn cấu trúc toàn cục: t-SNE, Isomap, LargeVis, Laplacian eigenmaps, diffusion maps.

Tại thời điểm paper được viết, t-SNE là thuật toán "state-of-the-art" cho trực quan hoá giảm chiều, nhưng có hai hạn chế thực tế: (a) chạy chậm và khó scale lên dữ liệu lớn, (b) gần như chỉ dùng được cho 2-3 chiều đầu ra (vì cần cấu trúc space-tree, quad-tree/oct-tree, để tính nhanh phần chuẩn hoá toàn cục, mà các cấu trúc này scale rất kém khi số chiều tăng).

UMAP (Uniform Manifold Approximation and Projection) được xây dựng trên nền các công trình của Belkin và Niyogi về Laplacian eigenmaps, kết hợp thêm hình học Riemann và cách nhìn lý thuyết phạm trù (category theory) về "fuzzy simplicial set" của David Spivak (2012). Mục tiêu nêu rõ trong Abstract/Introduction: tạo ra một thuật toán (i) cạnh tranh với t-SNE về chất lượng trực quan hoá, (ii) được bài báo lập luận là bảo toàn cấu trúc toàn cục tốt hơn, (iii) có thời gian chạy vượt trội, và (iv) không có giới hạn tính toán nào về số chiều embedding đầu ra, nên dùng được như một kỹ thuật giảm chiều tổng quát cho machine learning (không chỉ để vẽ hình). Paper cũng ghi nhận: dựa trên các bản phát hành phần mềm sơ bộ trước khi công bố lý thuyết đầy đủ, UMAP đã được dùng trong sinh tin học (bioinformatics), khoa học vật liệu, và machine learning.

2. Ý tưởng chính

Paper tự đặt UMAP vào lớp thuật toán "k-neighbour graph based" (giống Laplacian Eigenmaps, Isomap, t-SNE), và mô tả mọi thuật toán trong lớp này gồm hai giai đoạn (Section 3):

- **Xây dựng đồ thị (Graph Construction):** (1) xây đồ thị k-nearest-neighbor có trọng số, (2) biến đổi trọng số cạnh để xấp xỉ khoảng cách "cục bộ" trên manifold, (3) xử lý tính bất đối xứng vốn có của đồ thị k-NN (có hướng).
- **Bố trí đồ thị (Graph Layout):** (1) định nghĩa một hàm mục tiêu bảo toàn đặc tính của đồ thị trên, (2) tìm biểu diễn chiều thấp tối ưu hàm mục tiêu đó.

Ba giả định/axiom UMAP dựa vào (nêu rõ trong Section 3): (a) có một manifold mà dữ liệu được phân bố đều (uniformly) trên đó; (b) manifold đó liên thông cục bộ (locally connected); (c) bảo toàn cấu trúc tô pô của manifold là mục tiêu chính.

Ý tưởng cốt lõi, giải thích cho người chưa biết gì về manifold: một "manifold" là một không gian mà ở quy mô nhỏ (cục bộ, local) trông giống không gian Euclid phẳng, dù ở quy mô lớn có thể cong/phức tạp (ví dụ: một tờ giấy cuộn tròn trong không gian 3D vẫn là một mặt "phẳng" 2D nếu nhìn ở một vùng nhỏ). Nhiều dữ liệu thực tế (ảnh của cùng một chữ số viết tay với các biến dạng nhỏ, câu văn gần nghĩa nhau...) được giả định nằm gần một manifold có số chiều thực nhỏ hơn rất nhiều so với số chiều "thô" (ví dụ 784 pixel của ảnh MNIST).

Vấn đề: dữ liệu thật hiếm khi phân bố đều trên manifold (có vùng dày, vùng thưa). UMAP giải quyết bằng cách gán cho **mỗi điểm dữ liệu một thước đo Riemann cục bộ riêng** (chuẩn hoá theo khoảng cách tới hàng xóm gần nhất và tới k hàng xóm gần nhất), khiến dữ liệu "trông" đều theo thước đo cục bộ đó (Lemma 1). Hệ quả: ta có N không gian metric cục bộ khác nhau (một cho mỗi điểm), không tương thích trực tiếp với nhau. Để ghép chúng lại mà không ép về một metric toàn cục cứng nhắc, paper chuyển mỗi không gian metric cục bộ thành một **fuzzy simplicial set** (mở rộng khái niệm simplicial set cổ điển sang "độ thuộc mờ", fuzzy membership, dùng ngôn ngữ lý thuyết phạm trù), rồi lấy **hợp mờ (fuzzy union)** của tất cả N fuzzy simplicial set đó (dùng phép "probabilistic t-conorm") để ra một biểu diễn tô pô mờ toàn cục duy nhất.

Ở không gian chiều thấp (kết quả embedding), manifold mục tiêu đã biết trước (thường chỉ là $\mathbb{R}^d$ thông thường), nên có thể tính trực tiếp fuzzy simplicial set tương ứng (chỉ cần thêm tham số `min-dist` để mô phỏng ràng buộc liên thông cục bộ). Cuối cùng, UMAP tối ưu vị trí các điểm ở chiều thấp để **cross-entropy mờ (fuzzy set cross entropy)** giữa hai fuzzy simplicial set (xem như hai đồ thị có trọng số, chỉ dùng 1-skeleton) là nhỏ nhất, bằng stochastic gradient descent. Khác với t-SNE (dùng KL-divergence trên hai *phân phối xác suất* đã chuẩn hoá toàn cục), cross-entropy của UMAP làm việc trực tiếp trên các "xác suất tồn tại cạnh" chưa chuẩn hoá, nên không cần bước chuẩn hoá toàn cục tốn kém (chính bước này buộc t-SNE phải dùng quad-tree/oct-tree, scale rất kém theo số chiều).

3. Phương pháp chi tiết

3.1 Nền lý thuyết (tóm tắt ở mức khái niệm)

- Một **simplicial set** được định nghĩa thuần phạm trù học: là một functor từ Δ^{op} (phạm trù các tập hữu hạn có thứ tự) vào **Sets**; đây là cách mã hoá tổ hợp (combinatorial) của một không gian tô pô bằng "đỉnh, cạnh, tam giác, tứ diện..." (n-simplex).
- Một **fuzzy set** được hình thức hoá (theo Michael Barr, 1986) là một presheaf trên khoảng đơn vị $I = (0, 1]$ sao cho mọi ánh xạ hạn chế là đơn ánh (Definition 3). Một **fuzzy simplicial set** là presheaf của Δ nhận giá trị trong phạm trù fuzzy set đó (Definition 5), tức một cấu trúc simplicial mà mỗi simplex mang kèm một "độ tin cậy" (membership strength) trong $[0, 1]$ thay vì chỉ "có/không".
- **EPMet** (extended-pseudo-metric space) là phạm trù không gian metric đã nới lỏng (cho phép khoảng cách 0 giữa hai điểm khác nhau, và khoảng cách vô hạn), với morphism là ánh xạ không giãn (non-expansive).
- Spivak (2012) xây dựng một cặp functor liên hợp (adjoint) `Real` và `Sing` giữa fuzzy simplicial set và EPMet, mở rộng cặp functor "hiện thực hoá / tập đơn hình" cổ điển của tô pô đại số. UMAP dùng bản hữu hạn `FinReal`/`FinSing`; Theorem 1 khẳng định đây cũng là một adjunction, chứng minh ở

Appendix B (dựa trên bổ đề Yoneda và mở rộng Kan trái; về bản chất, chứng minh nói rằng vì mọi simplicial set đều là colimit của các simplex chuẩn, và `FinReal` bảo toàn colimit, nên `FinReal` được xác định hoàn toàn qua ảnh của nó trên các simplex chuẩn, từ đó suy ra tính liên hợp).

- Với mỗi điểm x_i, không gian metric cục bộ (khoảng cách tới k hàng xóm, đã chuẩn hoá bằng ρ_i, σ_i ở dưới) được chuyển qua `FinSing` thành một fuzzy simplicial set; khoảng cách giữa hai điểm khác x_i (không phải x_i chính nó) trong không gian cục bộ này được đặt là vô hạn, vì ta không có thông tin đáng tin cậy về khoảng cách đó từ góc nhìn của x_i (Definition 9). Biểu diễn toàn cục là hợp mờ của N fuzzy simplicial set cục bộ này.
- Ràng buộc liên thông cục bộ: khoảng cách từ x_i tới hàng xóm gần nhất của nó được đặt bằng 0 (sau khi trừ ρ_i), bảo đảm mỗi điểm luôn có ít nhất một cạnh với độ thuộc bằng 1, tức biểu diễn tô pô kết quả luôn liên thông cục bộ ở mọi điểm. Paper cho rằng chính cơ chế này giúp UMAP xử lý dữ liệu chiều cao tốt hơn (t-SNE, dựa trên kernel Gauss cố định theo perplexity, dễ gặp vấn đề "curse of dimensionality" khi dữ liệu rất thưa ở chiều cao).

3.2 Thuật toán tính toán (phần thực dụng, Algorithm 1-5)

1. Với mỗi x_i, tìm n hàng xóm gần nhất xấp xỉ (approximate k-NN) bằng Nearest-Neighbor-Descent (Dong et al., 2011); paper trích dẫn độ phức tạp thực nghiệm $O(N^{1.14})$ của thuật toán này, hoạt động với bất kỳ hàm dissimilarity nào, hiệu quả cả ở chiều cao.
2. ρ_i = khoảng cách từ x_i tới hàng xóm gần nhất (offset liên thông cục bộ).
3. σ_i là nhân tố chuẩn hoá mượt, tìm bằng tìm kiếm nhị phân (binary search, Algorithm 3) sao cho:

$$\sum_{j=1}^{k} \exp\Big(- \frac{\max(0, d(x_i, x_{ij}) - \rho_i)}{\sigma_i} \Big) = \log_2(k)$$

   (chọn $\log_2(k)$ dựa trên thực nghiệm, không có lý giải lý thuyết chặt).
4. Trọng số cạnh có hướng: $w(x_i, x_{ij}) = \exp\big(- \max(0, d(x_i, x_{ij}) - \rho_i)/\sigma_i \big)$.
5. Đối xứng hoá đồ thị có hướng A thành đồ thị vô hướng B bằng t-conorm xác suất: $B = A + A^\top - A \circ A^\top$ (ký hiệu $\circ$ là tích Hadamard). Nếu coi A_{ij} là "xác suất cạnh có hướng $i \to j$ tồn tại", B_{ij} chính là xác suất ít nhất một trong hai hướng tồn tại.
6. Khởi tạo vị trí Y bằng spectral embedding: $L = D^{1/2}(D - A)D^{1/2}$ (Laplacian chuẩn hoá đối xứng của đồ thị UMAP), lấy các eigenvector đầu làm toạ độ khởi tạo. Đây thực chất chính là Laplacian Eigenmaps cổ điển, vì Laplacian chuẩn hoá là xấp xỉ rời rạc của toán tử Laplace-Beltrami trên manifold.
7. Hàm tương đồng ở chiều thấp được xấp xỉ mượt bằng (Definition 11):

$$\Phi(x, y) = \big(1 + a \, \|x - y\|_2^{2b}\big)^{-1}$$

   với a, b được tìm bằng bình phương tối thiểu phi tuyến (non-linear least squares) khớp với đường cong mục tiêu Ψ xác định bởi `min-dist`: $\Psi = 1$ nếu $\|x - y\|_2 \le$ min-dist, ngược lại $\Psi = \exp(-(\|x - y\|_2 - \text{min-dist}))$. Theo Appendix C, với các giá trị mặc định của phần mềm UMAP, $a \approx 1.929$ và $b \approx 0.7915$; đặt $a = 1, b = 1$ cho lại đúng kernel Student-t một chiều mà t-SNE dùng.
8. Tối ưu bằng SGD với lấy mẫu cạnh dương theo xác suất độ thuộc, cộng lấy mẫu âm (negative sampling, mượn từ LargeVis/word2vec) cho các cặp ngẫu nhiên khác (Algorithm 5). Lực hút giữa hai điểm i, j:

$$F_{\text{hút}} = \frac{-2ab \, \|y_i - y_j\|_2^{2(b-1)}}{1 + \|y_i - y_j\|_2^2} \, w(x_i, x_j) \, (y_i - y_j)$$

   Lực đẩy (chỉ tính qua lấy mẫu, vì tính đẩy cho mọi cặp là quá đắt):

$$F_{\text{đẩy}} = \frac{2b}{(\varepsilon + \|y_i - y_j\|_2^2)\big(1 + a\|y_i - y_j\|_2^{2b}\big)} \, (1 - w(x_i, x_j)) \, (y_i - y_j)$$

với $\varepsilon = 0.001$ để tránh chia 0. Tốc độ học α giảm dần tuyến tính: $\alpha \leftarrow 1 - e/\text{n-epochs}$ (giống kiểu luyện kim mô phỏng, simulated annealing).

3.3 Hàm mục tiêu cross-entropy (Definition 10, công thức 1)

Với hai fuzzy set cùng tập mang A, độ thuộc μ (cố định, từ dữ liệu chiều cao) và ν (đang được tối ưu, từ embedding chiều thấp):

$$C(\mu, \nu) = \sum_{a \in A} \left[\mu(a) \log \frac{\mu(a)}{\nu(a)} + (1 - \mu(a)) \log \frac{1 - \mu(a)}{1 - \nu(a)} \right]$$

Phần đầu (chỉ phụ thuộc μ) là hằng số trong quá trình tối ưu; phần thực sự cần tối thiểu hoá là $-\sum_a \mu(a) \log \nu(a) + (1 - \mu(a)) \log(1 - \nu(a))$. Khác biệt quan trọng so với KL-divergence của t-SNE: cross-entropy này làm việc trên từng cạnh (độ thuộc 1-simplex) riêng lẻ, chưa chuẩn hoá thành một phân phối xác suất trên toàn bộ N^2 cặp điểm, nên không cần bước chuẩn hoá toàn cục.

3.4 Siêu tham số (hyperparameter, 4 cái, Section 4.3)

Siêu tham số	Ý nghĩa	Ảnh hưởng
n (số hàng xóm)	quy mô cục bộ coi manifold là "phẳng"	nhỏ: giữ chi tiết cục bộ, dễ vỡ thành nhiều thành phần liên thông nhỏ; lớn: giữ cấu trúc quy mô lớn, mất chi tiết nhỏ
d (số chiều embedding)	số chiều đầu ra	"hiển nhiên", không giới hạn tính toán (khác t-SNE)
`min-dist`	khoảng cách mong muốn giữa các điểm gần nhau ở không gian nhúng	nhỏ: các điểm đóng gói sát, trung thực hơn với cấu trúc manifold; lớn: dàn trải hơn, hỗ trợ trực quan hoá (paper gọi đây là tham số "thẩm mỹ", essentially aesthetic)
`n-epochs`	lượng công việc tối ưu	"hiển nhiên", nhiều hơn = tối ưu kỹ hơn

3.5 Ví dụ tính tay (tự tính)

Ví dụ 1 (minh hoạ Algorithm 3): giả sử với một điểm x, 4 khoảng cách tới 4 hàng xóm gần nhất là $d = [0.10, 0.20, 0.30, 0.40]$, nên $\rho = 0.10$ (khoảng cách nhỏ nhất). Mục tiêu: tìm σ sao cho $\sum \exp(-(d_i - \rho)/\sigma) = \log_2(4) = 2$. Thử $\sigma = 0.2$ (tự tính): các hạng tử là $\exp(0) = 1$ (vì $d_1 - \rho = 0$), $\exp(-0.1/0.2) = \exp(-0.5) \approx 0.6065$, $\exp(-0.2/0.2) = \exp(-1) \approx 0.3679$, $\exp(-0.3/0.2) = \exp(-1.5) \approx 0.2231$. Tổng $\approx 1 + 0.6065 + 0.3679 + 0.2231 = 2.1975$, lớn hơn 2 một chút, nên σ thật sẽ nhỏ hơn 0.2 (vòng tìm kiếm nhị phân tiếp theo sẽ thử một giá trị nhỏ hơn). Ví dụ này minh hoạ cụ thể Algorithm 3 hoạt động thế nào trên vài số liệu tay, điều paper chỉ mô tả bằng công thức trừu tượng.

Ví dụ 2 (so sánh kernel Φ của UMAP với Student-t của t-SNE): tại khoảng cách $\|y_i - y_j\|_2 = 1$ (đơn vị bất kỳ trong không gian nhúng), với $a \approx 1.929, b \approx 0.7915$ (giá trị mặc định paper nêu ở Appendix C): $\Phi = 1/(1 + 1.929 \times 1^{2 \times 0.7915}) = 1/(1 + 1.929 \times 1) = 1/2.929 \approx 0.3414$ (tự tính). So với Student-t gốc của t-SNE ($a = 1, b = 1$): $\Phi_{t\text{-}SNE} = 1/(1 + 1^2) = 0.5$. Vậy ở khoảng cách 1 đơn vị, kernel mặc định của UMAP (≈ 0.34) giảm nhanh hơn kernel Student-t thuần (0.5), tức UMAP mặc định "đẩy" các điểm không liên quan ra xa nhau quyết liệt hơn một chút so với t-SNE gốc ở cùng thang đo (tự tính, chỉ là minh hoạ bằng số cụ thể cho công thức Definition 11 và giá trị a, b mà paper cho ở Appendix C; paper không tự đưa ra phép so sánh số này).

4. Thí nghiệm và kết quả chính

Paper không có bộ dữ liệu benchmark chuẩn hoá kiểu ImageNet; thực nghiệm dùng 9 bộ dữ liệu khác nhau (Section 5): Pen Digits (1797×64, ảnh số viết tay 8x8), COIL-20 (1440×16384, 20 vật thể x 72 góc quay),

COIL-100 (7200×49152, 100 vật thể x 72 góc quay, ảnh màu), Mouse scRNA-seq (20.921 tế bào × 26.774 chỉ số gen), Statlog Shuttle (58.000×9), MNIST (70.000×784), Fashion-MNIST (70.000×784), Flow Cytometry (1.000.000×17), và GoogleNews word vectors (3.000.000 từ × 300 chiều, từ word2vec của Mikolov et al. 2013). Khoảng cách Euclid được dùng cho mọi bộ trừ GoogleNews (dùng cosine/angular distance theo đúng cách bài word2vec gốc dùng).

So sánh định tính (Figure 4): UMAP, t-SNE (MulticoreTSNE), LargeVis, Laplacian Eigenmaps, PCA trên COIL20, MNIST, Fashion-MNIST, GoogleNews (Isomap không chạy xong trong thời gian hợp lý với bộ nào lớn hơn COIL20). Paper lập luận UMAP giữ được nhiều vòng lặp (loops) lồng nhau của COIL20 hơn; cấu trúc toàn cục giữa các chữ số MNIST rõ hơn (chữ số 1 và 0 ở hai góc xa nhau, nhóm 4/7/9 và nhóm 3/5/8 tách biệt); phân biệt rõ hơn giữa quần áo và giày dép trong Fashion-MNIST; cấu trúc cụm từ toàn cục rõ hơn trong GoogleNews.

So sánh định lượng bằng độ chính xác kNN classifier (huấn luyện trên không gian embedding, đo qua cross-validation), Table 1 và 2 của paper:

k	t-SNE	UMAP	LargeVis	Eigenmaps	PCA
COIL-20, $k = 10$	0.934	0.921	0.888	0.629	0.667
COIL-20, $k = 160$	0.609	0.803	0.616	0.446	0.375
PenDigits, $k = 10$	0.977	0.973	0.966	0.778	0.622
PenDigits, $k = 160$	0.949	0.951	0.921	0.747	0.629
Shuttle, $k = 100$	0.994	0.993	0.992	0.962	0.833
Shuttle, $k = 3200$	0.828	0.957	0.850	0.895	0.786
MNIST, $k = 100$	0.967	0.967	0.962	0.668	0.462
MNIST, $k = 3200$	0.946	0.964	0.920	0.639	0.459
F-MNIST, $k = 100$	0.818	0.790	0.808	0.631	0.564
F-MNIST, $k = 3200$	0.727	0.730	0.726	0.542	0.533

Nhận xét của paper (khớp với bảng trên): *ở* k nhỏ-trung bình, UMAP tương đương t-SNE/LargeVis; *ở* k lớn, UMAP vượt rõ (đặc biệt COIL-20, Shuttle); trên Fashion-MNIST *ở* k nhỏ, UMAP **thua** t-SNE và LargeVis (0.790 và 0.785 so với 0.818 và 0.810 của t-SNE *ở* $k = 100, 200$), paper tự ghi nhận thẳng điều này ("notably underperforms on the Fashion-MNIST dataset" *ở* k nhỏ), chỉ bắt kịp *ở* k lớn (1600, 3200).

Độ ổn định embedding (Section 5.3): đo bằng khoảng cách Procrustes chuẩn hoá giữa embedding của mẫu con (sub-sample) và embedding tương ứng từ dữ liệu đầy đủ, trên Flow Cytometry. Kết quả: ngay *ở* mẫu con 5%, sai số mỗi điểm của UMAP đã thấp hơn **bất kỳ** giá trị mà t-SNE đạt được (kể cả dùng toàn bộ 1 triệu điểm), cho thấy UMAP ổn định hơn nhiều qua các lần chạy/qua sub-sampling.

Hiệu năng tính toán (Table 3, Section 5.4): benchmark trên MacBook Pro (i7 3.1GHz, 8GB RAM) cho Table 3, và server Xeon E5-2697v4/512GB RAM cho các thí nghiệm scaling lớn hơn. Một số số liệu (thời gian chạy, giây):

Dataset	UMAP	FIt-SNE	t-SNE	LargeVis	Eigenmaps
Pen Digits (1797×64)	9	48	17	20	2
COIL100 (7200×49152)	85	2681	810	3197	3268
MNIST (70000×784)	87	292	1450	1298	40709
Flow (100000×17)	102	164	1135	1127	30654
GoogleNews (200000×300)	361	652	16906	5392	–

UMAP nhanh hơn mọi thuật toán khác trừ Pen Digits (dataset rất nhỏ, Eigenmaps/Isomap nhanh hơn vài giây). Lưu ý: LargeVis trong Table 3 được chạy với 8 luồng trên máy 4 lõi, còn UMAP chạy đơn luồng, nghĩa là so sánh tốc độ ở Table 3 không hoàn toàn cùng điều kiện phần cứng, dù UMAP vẫn thắng.

Scaling theo số chiều embedding d (Fig 9): UMAP và LargeVis gần như tuyến tính/phẳng theo d, còn t-SNE scale tệ hơn cả hàm mũ (do cần quad/oct-tree, vốn scale theo cấp mũ với số chiều): điều này quan trọng vì nó cho phép dùng UMAP cho các chiều embedding lớn hơn 2-3, không chỉ để vẽ hình.

Scaling theo chiều dữ liệu gốc (ambient dimension, Fig 10): dùng Mouse scRNA; UMAP chạy hiệu quả cả trên dữ liệu gốc 26.774 chiều (không cần PCA tiền xử lý), trong khi t-SNE/LargeVis thường cần giảm chiều bằng PCA trước. Paper ghi chú COIL100 là ví dụ mà PCA tiền xử lý phá vỡ cấu trúc manifold, nên khả năng bỏ qua PCA của UMAP có giá trị thực tế.

Scaling theo số lượng điểm (Fig 11, 12): embedding toàn bộ 3 triệu từ của GoogleNews chỉ mất khoảng 200 phút với UMAP, so với "vài ngày" với MulticoreTSNE dù dùng nhiều lõi.

Trình diễn quy mô cực lớn: 30.000.000 số nguyên mã hoá dưới dạng vector nhị phân thưa (biểu diễn tính chia hết cho các số nguyên tố), không gian khoảng 1,8 triệu chiều, mất khoảng 2 tuần chạy trên một máy SMP bộ nhớ lớn (paper không nêu thêm chi tiết phần cứng cụ thể); không có bảng so sánh thời gian chạy với thuật toán khác cho riêng thí nghiệm này, đây chỉ là minh hoạ khả năng scale, không phải benchmark có đối chứng.

5. Phân tích / ablation

Paper **không có ablation study thực nghiệm** kiểu "tắt từng thành phần rồi đo lại" (như kiểu ConvNeXt hay ResNet làm); đây là một khoảng trống mà chính các tác giả tự thừa nhận ở đầu Section 7 (Future Work): "A comprehensive empirical study which examines the impact of the various algorithmic components, choices, and hyper-parameters of the algorithm would be beneficial" - nói cách khác, nghiên cứu đó **chưa được làm** trong paper này.

Những gì paper có thay vào đó:

- **Khảo sát độ nhạy siêu tham số bằng hình ảnh** (Figure 1, 2, 3): quét n và `min-dist` trên dữ liệu đồ chơi (color-cube ngẫu nhiên đều 3D, không có cấu trúc manifold thật), trên PenDigits, và trên MNIST. Với color-cube (không có cấu trúc thật), n nhỏ khiến UMAP "nhìn ra" các cụm giả từ nhiễu lấy mẫu (paper gọi là "constellation effect", hiệu ứng chòm sao, giống cách con người thấy hình thù trong các vì sao xếp ngẫu nhiên), đây là một quan sát định tính tự nhận, không đo bằng một chỉ số định lượng cụ thể nào.
- **Thảo luận tự phản biện (Section 6, trích dẫn Kobak & Linderman 2019, bioRxiv):** một phần lớn lợi thế "giữ cấu trúc toàn cục tốt hơn t-SNE" mà UMAP thường được ca ngợi, theo Kobak & Linderman, đến từ việc **khởi tạo bằng Laplacian eigenmaps** (một lựa chọn kỹ thuật, không phải bản chất của hàm mục tiêu cross-entropy), vì khi cho t-SNE cùng cách khởi tạo đó, t-SNE đạt hiệu năng tương đương về chỉ số bảo toàn cấu trúc toàn cục. Paper của UMAP không phản bác điều này mà chỉ hạ giọng: hàm mục tiêu cross-entropy (khác KL-divergence) "cũng là một yếu tố đóng góp đáng kể", một khẳng định có phần dè dặt (hedged), không đưa thêm thực nghiệm riêng để tách bạch hai yếu tố (khởi tạo vs. hàm mục tiêu).

6. Giới hạn

Section 6 ("Weaknesses") của paper tự liệt kê các điểm yếu khá đầy đủ:

- **Thiếu khả năng giải thích (interpretability):** không giống PCA/NMF, các chiều của không gian embedding UMAP không có ý nghĩa cụ thể, và vì UMAP dựa trên khoảng cách giữa các quan sát (không dựa trên đặc trưng gốc), nó không có khái niệm tương đương "factor loading". Paper khuyên dùng PCA/NMF/pLSA nếu cần giải thích mạnh.

- **Dễ "thấy" cấu trúc manifold giả từ nhiễu** ("constellation effect"), đặc biệt với mẫu nhỏ hoặc dữ liệu chỉ có cấu trúc quy mô lớn; phát hiện khi nào một embedding là giả được paper gọi là "đề tài nghiên cứu tiếp theo" (chưa giải quyết).
- **Ưu tiên cấu trúc cục bộ hơn toàn cục** (giống t-SNE, LargeVis): nếu mục tiêu chính là giữ cấu trúc toàn cục/khoảng cách thật, paper khuyên dùng MDS hoặc PHATE.
- **Reverse-nearest-neighbors:** việc đối xứng hoá đồ thị k-NN (bước $B = A + A^\top - A \circ A^\top$) không giữ nguyên cấu trúc k-NN "thuần", nên UMAP kém hơn các phương pháp tối ưu trực tiếp cấu trúc metric (như MDS) trên các chỉ số đo bằng cấu trúc metric.
- **Giả định phân bố đều trên manifold:** nếu người dùng tin tưởng mạnh vào khoảng cách ambient thật của dữ liệu (ví dụ một vùng thưa, một vùng dày có ý nghĩa quan trọng), UMAP sẽ có xu hướng "san đều" các vùng đó, có thể không phải điều mong muốn.
- **Xấp xỉ tính toán ảnh hưởng dữ liệu nhỏ:** nearest-neighbor xấp xỉ và negative sampling có thể cho embedding kém tối ưu với dataset dưới 500 mẫu; một bản triển khai chính xác (exact) hơn nhưng chậm hơn được paper nêu là dự án tương lai (chưa có ở thời điểm paper).
- **Chưa hỗ trợ nhúng điểm mới (out-of-sample):** Section 7 chỉ nêu đây là khả năng "có thể phát triển trong tương lai" (framework có cho phép về lý thuyết), không phải một tính năng đã có trong thuật toán được mô tả ở paper này.
- **Chưa có thước đo chuẩn cho "bảo toàn cấu trúc toàn cục":** paper tự nhận lĩnh vực manifold learning còn thiếu định nghĩa/chỉ số rõ ràng cho khái niệm này, nên các khẳng định "giữ cấu trúc toàn cục tốt hơn" trong chính paper này chủ yếu dựa trên quan sát định tính (nhìn hình) và một vài chỉ số gián tiếp (Procrustes, kNN accuracy), không có một benchmark global-structure chuẩn hoá.

Hai điểm mơ hồ/chi tiết kỹ thuật đáng chú ý khi đọc văn bản gốc (không phải lỗi khái niệm, nhưng cần lưu ý):

1. Tại phần mô tả khởi tạo spectral (ngay sau Algorithm 4 trong văn bản trích xuất), có một đoạn trích dẫn bị lỗi hiển thị dạng `[?, see]]belkin2002laplacian, belkin2003laplacian`, rõ ràng là một macro `\cite` LaTeX bị lỗi khi chuyển PDF sang text, không phải lỗi nội dung khoa học, nhưng khiến đoạn văn đó khó đọc nguyên văn.
2. Câu mô tả khoảng cách dùng cho GoogleNews ("we use cosine distance (or angular distance in t-SNE which does support non-metric distances, in contrast to UMAP)") viết khá rối: có vẻ ngụ ý UMAP "không hỗ trợ khoảng cách phi metric" trong khi ngay trước đó paper nói chính UMAP cũng dùng cosine distance cho benchmark này. Ghi chú này không tự diễn giải lại ý paper muốn nói gì ở câu này, chỉ nêu rõ đây là một câu mơ hồ trong bản gốc.

7. Liên hệ với các bài khác

- **"math content/03_2014_pca_tutorial.md"** (PCA): chính paper UMAP khuyến nghị PCA khi cần interpretability (Mục 6 ở trên); PCA cũng là baseline tuyến tính trong mọi bảng so sánh của Section 5.
- **"math content/05_2007_spectral_clustering_tutorial.md"** (spectral clustering): bước khởi tạo spectral của UMAP (Algorithm 4, Laplacian chuẩn hoá $L = D^{1/2}(D - A)D^{1/2}$) chính là kỹ thuật Laplacian eigenmaps/spectral clustering cổ điển, nền tảng trực tiếp UMAP kế thừa.
- **"math content/19_2018_intrinsic_dimension.md"** (Measuring the Intrinsic Dimension of Objective Landscapes): cùng chủ đề "số chiều thật" của dữ liệu/không gian, liên quan trực tiếp tới câu hỏi chọn d khi dùng UMAP cho mục đích khác ngoài trực quan hoá.
- **"ml basics content/09_2013_word2vec_cbow_skipgram.md"** (word2vec): bộ dữ liệu GoogleNews word vectors mà UMAP dùng để benchmark chính là sản phẩm của word2vec (Mikolov et al. 2013, trích dẫn trực tiếp là ref [41] trong paper).
- **"ml basics content/26_2012_representation_learning_review.md"** (representation learning review): bối cảnh rộng hơn cho câu hỏi "tại sao cần nhìn/giảm chiều các representation học được", chủ đề UMAP

phục vụ trực tiếp.

8. Gợi ý áp dụng

UMAP hữu ích để khám phá cấu trúc biểu diễn và đặt giả thuyết, nhưng các cụm trên hình chiếu không tự chứng minh cấu trúc trong không gian gốc. Nên khảo sát siêu tham số, độ ổn định theo seed và đối chiếu với các phép đo định lượng.

9. Thí nghiệm gợi ý

1. **Tái hiện "constellation effect":** sinh $N = 1000$ điểm ngẫu nhiên đều trong khối lập phương 3D (giống Figure 1 của paper), chạy UMAP (CPU, thư viện `umap-learn`) với n nhỏ (ví dụ 5) và n lớn (ví dụ 200). Đo: embedding với n nhỏ có tạo ra các cụm "giả" rõ rệt dù dữ liệu gốc hoàn toàn đều và không có cấu trúc thật không? Kỳ vọng: có (theo tuyên bố của paper). Ý nghĩa: xác nhận cảnh báo của paper trước khi tin vào bất kỳ cụm nào nhìn thấy trên một UMAP plot với n nhỏ.
2. **So sánh UMAP vs PCA bằng kNN accuracy:** lấy một mạng CNN nhỏ (hoặc mô hình toy) đã huấn luyện trên MNIST, trích đặc trưng tầng áp cuối, giảm về 2D bằng UMAP và bằng PCA, huấn luyện kNN classifier trên mỗi loại embedding với $k \in \{10, 40, 160\}$, đo accuracy (giống phương pháp luận Table 1/2 của paper nhưng ở quy mô toy). Kỳ vọng: UMAP vượt PCA rõ rệt ở k lớn. Ý nghĩa: định lượng giá trị thêm của UMAP so với một phép chiếu tuyến tính đơn giản.
3. **Đo scaling theo số chiều embedding d:** trên dữ liệu tổng hợp $N = 5000$, $D = 500$ chiều, đo thời gian chạy UMAP với $d \in \{2, 8, 32, 64\}$ (chỉ CPU) và so với t-SNE (scikit-learn) ở cùng các d đó. Kỳ vọng: UMAP gần tuyến tính/phẳng theo d, t-SNE chạy rất chậm hoặc không hỗ trợ tốt khi $d > 3$. Ý nghĩa: xác nhận trực tiếp Fig 9 của paper, và quyết định công cụ nào nên dùng nếu cần nhúng xuống nhiều hơn 2-3 chiều (ví dụ để làm input cho một bước phân tích/clustering tiếp theo trên embedding, không chỉ để vẽ hình).
4. **Quét siêu tham số n, `min-dist` trên activation thật nhỏ:** chạy một mô hình vision nhỏ (ResNet-18/ViT-tiny) trên ~2000 ảnh (có thể dùng một tập dữ liệu ảnh tài liệu/OCR công khai nhỏ), trích embedding tầng áp cuối, quét lưới nhỏ $n \in \{5, 15, 50, 200\}$, `min-dist` $\in \{0.0, 0.1, 0.5\}$ (12 lần chạy UMAP, đều rẻ trên CPU), đánh giá bằng mắt hoặc bằng silhouette score (nếu có nhãn) xem tổ hợp nào tách nhóm tốt nhất. Ý nghĩa: xây trực giác thực hành chọn n/`min-dist`.
5. **Kiểm tra độ ổn định bằng Procrustes:** lấy ~3000-5000 vector embedding (ví dụ từ một CNN toy trên MNIST test set), chạy UMAP hai lần với seed khác nhau (cùng siêu tham số) và một lần nữa trên mẫu con 80%, tính khoảng cách Procrustes chuẩn hoá (theo đúng Mục 5.3 của paper, dùng `scipy.spatial.procrustes` hoặc tự viết bằng SVD) giữa các embedding. Kỳ vọng: khoảng cách nhỏ, nhất quán với tuyên bố UMAP ổn định hơn các phương pháp stochastic khác. Ý nghĩa: có một con số cụ thể để tin tưởng (hoặc không) một lần chạy UMAP duy nhất trước khi dùng nó làm "bằng chứng trực quan".

10. Tóm tắt công thức / khái niệm

Thuật ngữ cho người mới:

- *Manifold*: không gian "trông phẳng" ở quy mô cục bộ dù có thể cong ở quy mô lớn; giả định dữ liệu thật nằm gần một manifold chiều thấp bên trong không gian chiều cao (pixel, embedding...).
- *Geodesic distance*: khoảng cách đo theo đường đi trên manifold (không phải đường thẳng xuyên qua không gian ambient).
- *Fuzzy simplicial set*: cấu trúc tổ hợp (đỉnh, cạnh,...) mà mỗi phần tử có một "độ thuộc" trong $[0, 1]$ thay vì chỉ 0/1, dùng để biểu diễn tô pô "mờ" của một đồ thị hàng xóm.
- *k-NN graph*: đồ thị nối mỗi điểm với k hàng xóm gần nhất của nó.

- *Local connectivity*: ràng buộc để mỗi điểm luôn có ít nhất một kết nối "chắc chắn" (độ thuộc 1) với một hàng xóm.
- *Cross-entropy (ở đây) vs KL-divergence (t-SNE)*: cross-entropy mờ làm việc trên độ thuộc từng cạnh riêng lẻ (không chuẩn hoá toàn cục); KL-divergence của t-SNE đòi hỏi hai phân phối xác suất đã chuẩn hoá trên toàn bộ N^2 cặp điểm.
- *Procrustes distance*: khoảng cách giữa hai tập điểm sau khi đã căn chỉnh tối ưu (dịch, xoay, co giãn đều) để so khớp nhau, dùng đo độ ổn định embedding.

Công thức chính:

- σ_i: $\sum_{j=1}^{k} \exp(-\max(0, d(x_i, x_{ij}) - \rho_i)/\sigma_i) = \log_2(k)$ (binary search).
- Trọng số cạnh: $w(x_i, x_{ij}) = \exp(-\max(0, d(x_i, x_{ij}) - \rho_i)/\sigma_i)$.
- Đối xứng hoá: $B = A + A^\top - A \circ A^\top$.
- Khởi tạo spectral: $L = D^{1/2}(D - A)D^{1/2}$, lấy eigenvector đầu.
- Kernel chiều thấp: $\Phi(x, y) = (1 + a\|x - y\|_2^{2b})^{-1}$; mặc định $a \approx 1.929$, $b \approx 0.7915$; $a = 1, b = 1$ cho lại Student-t của t-SNE.
- Cross-entropy mờ: $C(\mu, \nu) = \sum_a \mu(a) \log \frac{\mu(a)}{\nu(a)} + (1 - \mu(a)) \log \frac{1-\mu(a)}{1-\nu(a)}$.
- Tốc độ học: $\alpha \leftarrow 1 - e/$n-epochs.
- Độ phức tạp: NN-descent $\approx O(N^{1.14})$ (thực nghiệm), SGD tối ưu $O(kN)$.
- 4 siêu tham số: n, d, `min-dist`, `n-epochs` (xem bảng ở Mục 3.4).
- Hai ví dụ tự tính ở Mục 3.5: (a) kiểm tra $\sigma = 0.2$ cho 4 khoảng cách $[0.10, 0.20, 0.30, 0.40]$ ra tổng ≈ 2.1975 (mục tiêu là 2, nên σ thật nhỏ hơn 0.2); (b) tại khoảng cách 1 đơn vị, $\Phi_{\text{UMAP, mặc định}} \approx 0.3414$ so với $\Phi_{t\text{-}SNE} = 0.5$.

11. Kết luận ngắn

UMAP là một thuật toán giảm chiều được xây dựng từ một khung lý thuyết khá nặng (hình học Riemann, fuzzy simplicial set, lý thuyết phạm trù), nhưng kết quả thực dụng (Section 3-4) lại gọn: xây một đồ thị k-NN có trọng số chuẩn hoá cục bộ, đối xứng hoá, rồi tối ưu layout chiều thấp bằng SGD với một hàm cross-entropy không cần chuẩn hoá toàn cục. Nhờ bỏ được bước chuẩn hoá toàn cục (và do đó bỏ được yêu cầu space-tree của t-SNE), UMAP nhanh hơn rõ rệt, scale tốt hơn theo cả số mẫu, chiều dữ liệu gốc, và chiều embedding đầu ra, đồng thời ổn định hơn qua các lần chạy (Procrustes, Section 5.3). Tuy vậy, cần đọc các tuyên bố "giữ cấu trúc toàn cục tốt hơn t-SNE" một cách cẩn trọng: chính paper trích dẫn công trình của Kobak & Linderman (2019) cho thấy phần lớn lợi thế đó có thể đến từ việc dùng chung cách khởi tạo Laplacian eigenmaps, chứ không hẳn từ bản chất hàm mục tiêu; và paper tự thừa nhận chưa có ablation study hay thước đo global-structure chuẩn hoá nào để phân định rõ. UMAP là công cụ khám phá biểu diễn; các hình chiếu cần được đối chiếu với phép đo định lượng.

Nguồn gốc: https://arxiv.org/abs/1802.03426